LOCALIZATION & NAVIGATION HANDBOOK

定位与导航技术文档

从里程计到 SLAM，从建图到自主导航

C·ONE 战队 · 算法组

2026.11

FAST-LIO2 / Point-LIO / Cartographer / Nav2 / Z-Localization

定位与导航技术文档

从里程计到 SLAM，从建图到自主导航

作者：C • ONE 战队算法组

组织：RoboMaster C • ONE 战队算法组

时间：2026 年 11 月

版本：1.0

适用对象：算法组新队员 / 进阶队员

配套解析：FAST-LIO2 · Point-LIO · Cartographer · Nav2 · Z-Localization



前言

为什么写这本书

算法组新队员入门定位与导航，最常见的困境不是“找不到资料”，而是“资料太多且互相矛盾”：有人推荐先看《视觉 SLAM 十四讲》，有人让你直接读 FAST-LIO 源码；有人说定位就是卡尔曼滤波，有人说是因子图优化；有人把 Cartographer 当标准答案，有人只用 LIO。结果往往是：视频看了几十个小时，代码 clone 了一堆，真到机器人上还是不知道从哪下手。

这本书试图解决这个问题。它不假设你学过机器人学，也不假设你会 ROS，而是从“机器人怎么知道自己在哪”这个最朴素的问题出发，一层一层把概念、公式、算法、工程串起来。我们尽量做到三件事：

1. **讲清“为什么”**：每个算法先说出它要解决什么问题、在前一个方法上改进了什么，再给公式；
2. **推导到能算数**：关键公式从定义开始推，不跳步，并且尽量给一个可以手算的数字例子；
3. **落到工程**：每个模块都说明输入输出、频率、参数、失效场景，而不是停在论文层面。

本书面向谁

主要读者是 RoboMaster 算法组的新队员，以及任何“会 C++、学过高数线代概率、但没系统学过定位导航”的同学。我们假设你：

- 能看懂基本的矩阵运算、求导、概率分布；
- 写过一些 C++/Python，能跑通 Linux 命令；
- 对机器人有热情，愿意动手。

如果你已经做过一两个赛季的定位，这本书的后半部分（SLAM 篇、导航篇、工程篇）仍然值得一读——它会把你在调参中积累的“直觉”补上数学解释。

怎么读

全书五篇三十章，建议按顺序读完第一篇（第 1–3 章），之后按需选择：

- **一周速成**：第 1、2、4、6、11、16、17 章。建立“里程计—前端—后端—LIO”的主干。
- **建图方向**：第 1–3、7、11–15 章。重点是 Cartographer、地图表示与回环。
- **导航方向**：第 1、10、11、18–22 章。重点是 Nav2 与 CMU 3D 导航栈。
- **落地实战**：第 23、25–29 章。Z-Localization 的架构、ROS 2 工程、标定与调参。

每章开头有“本章导读”，结尾有“本章小结”“思考与练习”“延伸阅读”。建议至少把小结和练习过一遍——练习里的题目大多是“把本章公式代入具体数字”，能算出来才算真懂。

关于“前端/后端”

这是本书反复出现的两个词，也是新手最容易混淆的地方。第 1 章先给直觉，第 11 章给出正式定义，并特别用一节讲清楚它们在三种语境下的不同含义：

- SLAM 语境：前端造约束，后端解约束；
- 定位系统语境：前端是里程计，后端是先验地图配准与因子图；
- 导航系统语境：前端是感知（定位 + 建图），后端是决策（规划 + 控制）。

读任何一章时，请先确认该章用的是哪种语境。第 23 章 Z-Localization 用的是第二种，第 18 章 Nav2 用的是第三种。

数学约定

为了让全书符号一致，我们约定：

- 向量、矩阵用粗体：$\boldsymbol{p}, \boldsymbol{R}$；标量用普通斜体：$x, \theta$；
- 估计值加帽子：$\hat{\boldsymbol{x}}$；IMU 原始测量加波浪号：$\tilde{\boldsymbol{\omega}}$；
- 坐标系：W 世界系、O 里程计系、B 机体系、L 雷达系、C 相机系；
- 位姿 T_{AB} 表示“从 B 系到 A 系”的变换，满足 ${}^{A}\boldsymbol{p} = T_{AB}\,{}^{B}\boldsymbol{p}$；
- 重力 $\boldsymbol{g} = [0, 0, -9.81]^{\top}$，世界系 z 轴向上；
- 旋转用右扰动更新：$\boldsymbol{R} \leftarrow \boldsymbol{R}\exp(\delta\boldsymbol{\phi}^{\wedge})$。

完整的符号表见第 2 章与附录 A。

关于开源代码

本书大量参考了以下开源项目，特此致谢：

- **FAST-LIO / FAST-LIO2 / Point-LIO / VoxelMap**（香港大学 MARS 实验室，Fu Zhang 组）：第 16–17 章的主要素材；
- **Cartographer**（Google）：第 15 章；
- **Nav2**（Open Navigation LLC 与社区）：第 18、22 章；
- **CMU Autonomous Exploration Development Environment**（CMU AirLab，Ji Zhang 组）：第 21 章；
- **Z-Localization**（东北大学 T-DT 战队，作者 mozihe）：第 23 章。该项目的设计说明写得非常坦诚，本章的很多工程判断直接来自作者的分享。

书中对代码的描述基于公开文档与论文，若与最新版本有出入，请以官方仓库为准。

致谢与反馈

感谢算法组历届队员积累的调试笔记与踩坑记录，它们是本书“工程篇”的素材来源。感谢所有把代码与设计思路开源出来的团队——没有这些开源项目，这本书无从写起。

本书难免有错误与过时之处（尤其涉及具体参数与版本的部分）。如果你发现公式推导有误、代码描述过时、或者某一段实在看不懂，欢迎指出。对一本教材来说，读者的困惑和作者的笔误同样重要。

目录

前言 i

第一部分　基础篇：先建立正确的地图 1

第一章 定位、建图与导航：全景导论 2

1.1 一个场景：机器人第一次进场地 . . . 2
1.2 三个问题的精确定义 . . . 2
1.2.1 定位 . . . 2
1.2.2 建图 . . . 2
1.2.3 导航 . . . 3
1.2.4 三者的关系 . . . 3
1.3 定位问题的分类 . . . 3
1.4 SLAM：先有鸡还是先有蛋 . . . 3
1.5 “前端/后端”到底指什么 . . . 4
1.6 全书地图与阅读路线 . . . 4
1.7 给新手的三个建议 . . . 4

第二章 数学基础：从概率到李群 6

2.1 坐标系与刚体变换 . . . 6
2.1.1 点、向量与坐标系 . . . 6
2.1.2 旋转矩阵 . . . 6
2.1.3 齐次坐标与 SE(3) . . . 6
2.1.4 欧拉角与万向锁 . . . 7
2.2 概率论最小工具箱 . . . 7
2.2.1 随机变量与概率 . . . 7
2.2.2 期望 . . . 7
2.2.3 方差与标准差 . . . 8
2.2.4 协方差与协方差矩阵 . . . 9
2.2.5 高斯分布 . . . 9
2.2.6 条件概率与贝叶斯公式 . . . 10
2.2.7 独立性 . . . 10
2.2.8 卡方分布 . . . 10
2.3 微积分与优化 . . . 11
2.3.1 泰勒展开 . . . 11
2.3.2 雅可比矩阵 . . . 11
2.3.3 海森矩阵与极值 . . . 11
2.3.4 最小二乘与正规方程 . . . 12
2.3.5 高斯–牛顿法 . . . 12
2.4 线性代数补充 . . . 12
2.4.1 特征值与特征向量 . . . 12
2.4.2 二次型与正定 . . . 13

2.4.3 奇异值分解 . . . 13
2.5 李群与李代数 . . . 13
2.5.1 SO(3) 与 $\mathfrak{so}(3)$ . . . 13
2.5.2 SE(3) 与 $\mathfrak{se}(3)$ . . . 14
2.5.3 扰动模型 . . . 14
2.5.4 为什么必须在流形上优化 . . . 14

第三章 传感器与观测模型 16
3.1 观测与测量模型 . . . 16
3.2 IMU：惯性测量单元 . . . 16
3.2.1 它测什么 . . . 16
3.2.2 测量模型 . . . 16
3.2.3 噪声参数与离散化 . . . 17
3.2.4 关键指标 . . . 17
3.3 轮式编码器 . . . 17
3.3.1 原理与分辨率 . . . 17
3.3.2 输出 . . . 18
3.4 激光雷达 . . . 18
3.4.1 分类 . . . 18
3.4.2 关键参数 . . . 18
3.4.3 点云与运动畸变 . . . 18
3.5 相机 . . . 19
3.6 其他传感器 . . . 19
3.7 时间同步与坐标系 . . . 19
3.7.1 时间同步 . . . 19
3.7.2 外参 . . . 20
3.8 传感器选型对比 . . . 20

第二部分 里程计篇：机器人如何推算自己的位置 22

第四章 里程计总论：相对定位的艺术 23
4.1 为什么需要里程计 . . . 23
4.2 位姿、绝对定位与相对定位 . . . 23
4.3 位姿递推与指数映射 . . . 24
4.4 里程计的四种来源 . . . 26
4.5 累积误差与漂移 . . . 27
4.6 里程计的评价 . . . 28
4.7 工程视角：里程计模块的输入输出 . . . 28

第五章 轮式里程计：从编码器到航迹推算 30
5.1 编码器：把一个脉冲变成一段位移 . . . 30
5.2 两轮差速运动学 . . . 30
5.2.1 从轮速到车体速度 . . . 30
5.2.2 非完整约束 . . . 31
5.3 离散航迹推算：圆弧模型与直线近似 . . . 31

5.3.1 圆弧模型 . . . 31
5.3.2 直线模型与极限 . . . 32
5.3.3 三种离散化误差对比 . . . 32
5.4 麦轮与全向底盘 . . . 33
5.4.1 麦轮逆运动学推导 . . . 33
5.4.2 正运动学：从轮速回到速度 . . . 33
5.4.3 为什么麦轮里程计更容易漂 . . . 34
5.5 误差来源与标定 . . . 34
5.5.1 误差从哪里来 . . . 34
5.5.2 标定流程：直线定半径、旋转定轮距 . . . 34
5.6 工程实现：ROS 2 中的轮式里程计 . . . 35

第六章 IMU 姿态解算与航迹推算 37
6.1 IMU 测量模型与噪声 . . . 37
6.1.1 从连续噪声到离散噪声 . . . 37
6.2 姿态表示与四元数更新 . . . 38
6.3 由加速度计估计重力方向与横滚俯仰 . . . 39
6.4 互补滤波与 Mahony 滤波 . . . 39
6.5 ESKF：误差状态卡尔曼滤波 . . . 40
6.5.1 先复习普通卡尔曼滤波 . . . 40
6.5.2 误差状态的定义 . . . 41
6.5.3 误差状态运动方程 . . . 41
6.5.4 离散化、状态转移矩阵与过程噪声 . . . 42
6.5.5 观测方程、更新与重置 . . . 42
6.6 航迹推算与零速修正 . . . 43
6.7 工程实现：ROS 2 中的 IMU . . . 44

第七章 激光里程计与扫描匹配 45
7.1 为什么用 LiDAR 做里程计 . . . 45
7.2 点云配准的问题定义 . . . 45
7.3 ICP：从闭式解到迭代流程 . . . 46
7.3.1 点到点 ICP 的闭式解 . . . 46
7.3.2 迭代流程、对应关系与收敛 . . . 47
7.3.3 点到面 ICP 与 Gauss-Newton 求解 . . . 47
7.4 NDT：用高斯分布代替最近邻 . . . 48
7.5 相关性搜索与分支定界 . . . 49
7.6 LOAM 框架：特征匹配的激光里程计 . . . 49
7.7 退化分析：为什么长廊里会失效 . . . 50

第八章 视觉里程计与 VIO 52
8.1 相机模型与畸变 . . . 52
8.1.1 针孔模型 . . . 52
8.1.2 径向与切向畸变 . . . 52
8.2 特征点法视觉里程计 . . . 53
8.2.1 对极几何：2D-2D 的约束 . . . 53
8.2.2 三角化 . . . 54

8.2.3 PnP 与重投影误差 . . . 54
8.2.4 光束法平差 . . . 55
8.3 直接法与光流 . . . 55
8.4 VIO：视觉与惯性的融合 . . . 56
8.5 视觉里程计的局限 . . . 56

第九章 多传感器融合定位 58
9.1 融合的层次与松紧耦合 . . . 58
9.1.1 三个融合层次 . . . 58
9.1.2 松耦合与紧耦合 . . . 58
9.2 卡尔曼融合的统一形式 . . . 59
9.2.1 状态增广与观测堆叠 . . . 59
9.2.2 序贯更新与批量更新的等价性 . . . 59
9.3 信息矩阵、马氏距离与卡方检验 . . . 59
9.3.1 信息矩阵 . . . 59
9.3.2 马氏距离及其卡方分布 . . . 60
9.4 因子图视角：融合就是最大后验 . . . 60
9.5 时间同步与延时补偿 . . . 61
9.6 典型融合架构 . . . 61

第十章 全局定位与重定位 63
10.1 局部定位与全局定位 . . . 63
10.2 基于地图的定位方法 . . . 63
10.3 贝叶斯滤波递归 . . . 64
10.4 粒子滤波 . . . 64
10.4.1 重要性采样 . . . 64
10.4.2 序贯重要性采样 . . . 64
10.4.3 重采样与样本贫化 . . . 65
10.5 AMCL：工程化的粒子滤波 . . . 66
10.5.1 运动模型：从里程计增量采样 . . . 66
10.5.2 观测模型：似然场 . . . 66
10.5.3 KLD 采样：自适应粒子数 . . . 66
10.6 重定位与绑架恢复 . . . 67

第三部分 SLAM 篇：前端、后端与建图 69

第十一章 SLAM 框架：前端与后端 70
11.1 先有鸡还是先有蛋：SLAM 到底难在哪 . . . 70
11.2 从最大后验到最小二乘 . . . 71
11.3 前端与后端：职责划分 . . . 72
11.4 三种语境下的“前端/后端”（重点） . . . 73
11.5 数据流：一帧数据在系统里走过的路 . . . 74
11.6 SLAM 系统的分类 . . . 74
11.7 为什么现在的主流是“前端 LIO + 后端因子图” . . . 75

第十二章 SLAM 前端详解 77

12.1 前端流水线 . . . 77
12.2 数据关联：前端的心脏 . . . 77
12.3 从 “帧到帧” 升级到 “帧到局部地图” . . . 78
12.4 关键帧策略 . . . 79
12.5 点云预处理 . . . 80
12.5.1 体素降采样 . . . 80
12.5.2 地面分割（RANSAC 平面拟合） . . . 80
12.5.3 去畸变与动态点剔除 . . . 81
12.6 描述子与回环检测 . . . 81
12.7 前端的失效模式 . . . 82

第十三章 SLAM 后端详解 84
13.1 从最大后验到最小二乘 . . . 84
13.2 Gauss-Newton 与 Levenberg-Marquardt . . . 84
13.2.1 线性化与正规方程 . . . 84
13.2.2 Levenberg-Marquardt：给步长加个 “刹车” . . . 85
13.2.3 海森矩阵为什么是稀疏的 . . . 85
13.3 位姿图 . . . 86
13.3.1 残差的定义 . . . 86
13.3.2 雅可比的逐步推导（二维情形） . . . 86
13.3.3 三维情形 . . . 88
13.4 从位姿图到 BA：舒尔补 . . . 88
13.4.1 BA 的残差与雅可比 . . . 88
13.4.2 分块与舒尔补 . . . 88
13.5 回环与鲁棒核 . . . 89
13.6 边缘化：把旧状态 “移出去” 但不丢信息 . . . 91
13.7 求解器工程 . . . 91

第十四章 地图与建图 93
14.1 地图的用途决定地图的形式 . . . 93
14.2 占据栅格地图与 log-odds 更新 . . . 93
14.2.1 从贝叶斯公式推导 log-odds . . . 94
14.2.2 逆观测模型与数值例子 . . . 94
14.3 八叉树地图 . . . 96
14.4 TSDF：截断符号距离场 . . . 96
14.4.1 单帧观测给出的 SDF 值 . . . 97
14.4.2 多帧融合：加权平均就是最小二乘 . . . 97
14.4.3 权重设计与表面提取 . . . 97
14.5 ESDF：欧氏符号距离场 . . . 98
14.6 点云地图与地图管理 . . . 98
14.7 地图精度评价 . . . 99

第十五章 2D 激光 SLAM：Cartographer 101
15.1 Cartographer 的定位与系统架构 . . . 101
15.2 子图：地图的基本单位 . . . 101
15.3 扫描匹配：把 scan 对齐到子图 . . . 102

15.3.1 得分函数 . . . 102
15.3.2 双线性插值及其导数 . . . 102
15.3.3 非线性最小二乘与雅可比 . . . 103
15.4 分支定界：在离散位姿空间里找全局最优 . . . 103
15.4.1 问题与离散化 . . . 103
15.4.2 上界的推导 . . . 104
15.4.3 搜索策略与伪代码 . . . 104
15.4.4 为什么能实时 . . . 105
15.5 全局 SLAM：把子图连成一张图 . . . 105
15.6 IMU 与里程计的作用 . . . 105
15.7 关键参数与调参 . . . 106
15.8 优缺点与适用场景 . . . 106

第十六章 3D 激光惯性里程计：从 LOAM 到 LIO **108**
16.1 为什么需要激光惯性里程计 . . . 108
16.1.1 2D 激光 SLAM 的三条隐含假设 . . . 108
16.1.2 纯激光里程计的三种失效模式 . . . 108
16.1.3 IMU 补上了什么 . . . 108
16.2 去畸变：把一帧点云对齐到同一时刻 . . . 109
16.2.1 问题有多严重 . . . 109
16.2.2 去畸变公式 . . . 109
16.2.3 匀速模型与 IMU 辅助模型 . . . 110
16.3 演化时间线：从 LOAM 到 Point-LIO . . . 110
16.3.1 LOAM（2014）：两阶段、特征法、无 IMU . . . 111
16.3.2 LeGO-LOAM（2018）：先分割再匹配 . . . 111
16.3.3 LIO-SAM（2020）：紧耦合 + 因子图 . . . 111
16.3.4 FAST-LIO（2021）与 FAST-LIO2（2022）：把后端换成迭代 EKF . . . 112
16.3.5 Point-LIO（2023）与 VoxelMap（2022）：两个方向的突破 . . . 112
16.3.6 视觉-激光-惯性：FAST-LIVO、R3LIVE、LVI-SAM . . . 112
16.4 两条技术路线：特征法与直接法、滤波与优化 . . . 113
16.5 紧耦合到底“紧”在哪 . . . 113
16.5.1 两种耦合的数学形式 . . . 113
16.5.2 紧耦合为什么更鲁棒 . . . 113
16.6 选型建议 . . . 114

第十七章 FAST-LIO2、Point-LIO 与先进 LIO 深度解析 **116**
17.1 FAST-LIO2 的总体架构 . . . 116
17.2 状态定义与流形更新 . . . 116
17.2.1 主状态 . . . 116
17.2.2 为什么把重力放进状态 . . . 117
17.2.3 误差状态与两个流形运算 . . . 117
17.3 IMU 运动学：从连续方程到离散传播 . . . 117
17.3.1 测量模型 . . . 117
17.3.2 连续运动方程 . . . 118
17.3.3 离散传播 . . . 118

17.4 误差状态传播：F 矩阵逐块推导 . . . 119
17.4.1 姿态误差 $\delta\boldsymbol{\theta}$ . . . 119
17.4.2 位置误差 $\delta\boldsymbol{p}$ . . . 120
17.4.3 速度误差 $\delta\boldsymbol{v}$ . . . 120
17.4.4 零偏与重力 . . . 120
17.4.5 组装 F 与 Q . . . 121
17.4.6 协方差传播 . . . 121
17.5 迭代扩展卡尔曼滤波（IEKF） . . . 122
17.5.1 标准 EKF 更新回顾 . . . 122
17.5.2 从最大后验估计推导迭代更新 . . . 122
17.5.3 收敛判据与实现细节 . . . 123
17.6 点到面观测模型与雅可比 . . . 123
17.6.1 数据关联：从最近邻到平面 . . . 123
17.6.2 残差与雅可比 . . . 124
17.6.3 堆叠与退化分析 . . . 125
17.6.4 零偏与重力如何被间接修正 . . . 125
17.7 iKD-Tree：增量式地图 . . . 125
17.7.1 为什么不能每帧重建 . . . 125
17.7.2 节点里存什么 . . . 125
17.7.3 增量插入 . . . 126
17.7.4 惰性删除与 box-wise 操作 . . . 126
17.7.5 复杂度与内存 . . . 126
17.8 Point-LIO：逐点更新 . . . 127
17.8.1 FAST-LIO2 留下的两个”帧”假设 . . . 127
17.8.2 核心思想：点一到就更新 . . . 127
17.8.3 逐点传播与标量更新 . . . 127
17.8.4 顺序更新与批量更新等价吗 . . . 128
17.8.5 IMU 饱和时：把 IMU 从”传播输入”降级为”大噪声” . . . 128
17.8.6 代价与收益 . . . 128
17.9 VoxelMap 与地图表示对比 . . . 129
17.9.1 自适应体素与平面模型 . . . 129
17.10 视觉-激光-惯性：FAST-LIVO2 一句话 . . . 129
17.11 论文公式与源码对照 . . . 130
17.11.1 对照表 . . . 130
17.11.2 一段关键代码的骨架 . . . 130
17.12 工程实践：参数、发散与开销 . . . 130

第四部分　导航篇：从地图到运动　132

第十八章 导航系统总览与 Nav2　133
18.1 导航到底在做什么 . . . 133
18.2 “前端/后端”到底指什么：三种语境 . . . 133
18.2.1 语境一：SLAM 的前端/后端（第 11–13 章） . . . 134
18.2.2 语境二：定位系统的前端/后端（第 9、10、23 章） . . . 134
18.2.3 语境三：导航系统的前端/后端（本章、第 19–22 章） . . . 134

18.3 Nav2：ROS 2 的导航事实标准 . . . 134
18.3.1 节点分工 . . . 135
18.3.2 action 接口：为什么不是 topic . . . 135
18.3.3 与 ROS 1 `move_base` 的差异 . . . 136
18.4 代价地图：导航的“地面真相” . . . 136
18.4.1 分层结构 . . . 136
18.4.2 内切半径、外切半径与膨胀 . . . 136
18.4.3 更新频率与发布频率 . . . 137
18.5 TF 树与坐标系 . . . 137
18.6 全局规划与局部规划的分工 . . . 138

第十九章 全局路径规划 140
19.1 问题定义与评价标准 . . . 140
19.2 Dijkstra 算法 . . . 140
19.3 A*：用启发函数剪枝 . . . 141
19.3.1 可采纳性：为什么 $h \le h^*$ 就保证最优 . . . 141
19.3.2 一致性：为什么可以不重复扩展 . . . 141
19.3.3 启发函数的设计 . . . 142
19.3.4 10×10 网格手算例子 . . . 142
19.4 JPS：跳点搜索 . . . 143
19.5 Hybrid A*：给车用的 A* . . . 143
19.6 采样规划：RRT 与 RRT* . . . 144
19.7 其他经典方法与工程实践 . . . 145

第二十章 局部规划与控制 147
20.1 局部规划要解决什么 . . . 147
20.2 DWA：把可行速度采样一遍 . . . 147
20.2.1 运动模型与动态窗口 . . . 147
20.2.2 评价函数 . . . 148
20.2.3 为什么能实时，局限在哪 . . . 148
20.3 TEB：把轨迹变成图优化 . . . 149
20.4 MPC：滚动时域的最优控制 . . . 149
20.5 梯度类方法：B 样条轨迹优化 . . . 150
20.6 人工势场：最直观也最容易卡住 . . . 151
20.7 底盘约束如何进入局部规划 . . . 151
20.8 Nav2 的 controller 插件怎么选 . . . 152

第二十一章 3D 导航与 CMU 自主探索栈 154
21.1 为什么 2D 导航不够 . . . 154
21.2 CMU 自主探索栈的总体架构 . . . 154
21.3 地形分析：把点云压成 2.5D 高程图 . . . 155
21.3.1 2.5D 栅格的组织 . . . 155
21.3.2 坡度：从高度差到角度 . . . 155
21.3.3 粗糙度与台阶高度 . . . 156
21.3.4 可通行性判定与伪代码 . . . 156
21.3.5 2.5D 与真 3D 的折中 . . . 157

21.4 局部规划与 FAR Planner . . . 157
21.4.1 local_planner：在地形图上做带净空的局部规划 . . . 157
21.4.2 FAR Planner：可视图 + attemptable . . . 157
21.4.3 TARE：从“导航”到“探索” . . . 158
21.5 部署、调参与和 LIO 的配合 . . . 159
21.5.1 与 LIO 的分工 . . . 159
21.5.2 关键参数与常见问题 . . . 159

第二十二章 2D 建图 + 导航完整链路实战 161
22.1 全链路总览 . . . 161
22.2 Cartographer 建图实操 . . . 161
22.2.1 配置文件 . . . 161
22.2.2 建图、结束与导出 . . . 162
22.2.3 建图质量自检 . . . 162
22.3 Nav2 配置 . . . 163
22.3.1 参数文件骨架 . . . 163
22.3.2 地图、定位与生命周期 . . . 163
22.3.3 仿真验证 . . . 164
22.4 常见问题排查表 . . . 164
22.5 性能、安全与恢复行为 . . . 165

第二十三章 Z-Localization 深度解析 166
23.1 背景与定位 . . . 166
23.2 总体架构：前后端分离 . . . 166
23.2.1 为什么拆 . . . 166
23.2.2 与 SLAM 前后端的区别 . . . 166
23.3 前端 ZLIO . . . 167
23.3.1 误差状态建模 . . . 167
23.3.2 不要求静止的初始化 . . . 167
23.3.3 去畸变 . . . 168
23.3.4 地图结构：用统计量代替点 . . . 168
23.3.5 面建模与残差 . . . 168
23.3.6 时空自适应 . . . 169
23.3.7 不用在线动态点滤除 . . . 169
23.4 后端定位 . . . 169
23.4.1 因子图 . . . 169
23.4.2 状态机 . . . 170
23.4.3 描述子 . . . 170
23.4.4 配准与验收 . . . 170
23.4.5 UWB 初始化 . . . 171
23.4.6 Reset . . . 171
23.5 使用流程：从 bag 建图到先验定位 . . . 171
23.6 与其他 LIO 方案的对比 . . . 172
23.7 可借鉴的设计思想 . . . 172

第二十四章 前沿方向与展望 174

24.1 从几何 SLAM 到语义与学习 . . . 174
24.1.1 语义 SLAM . . . 174
24.1.2 学习式特征与描述子 . . . 174
24.1.3 神经隐式表示 . . . 174
24.1.4 端到端里程计 . . . 174
24.2 多模态融合 . . . 175
24.3 长期建图与地图维护 . . . 175
24.4 学习式导航 . . . 175
24.5 对 RoboMaster 的启示 . . . 175

第五部分　工程篇：把算法跑起来　177

第二十五章 ROS 2 工程实践　178
25.1 ROS 2 是什么：从节点到计算图 . . . 178
25.2 最小工作空间：colcon 编译流程 . . . 179
25.3 话题、服务与动作：什么时候用哪个 . . . 179
25.4 QoS：ROS 2 最容易踩的坑 . . . 180
25.5 参数与 launch：把配置从代码里赶出去 . . . 181
25.6 TF2：坐标系的”宪法” . . . 182
25.7 调试工具与工程习惯 . . . 183

第二十六章 标定与时间同步　186
26.1 为什么标定：误差是怎么被放大的 . . . 186
26.1.1 旋转外参误差：$d\sin\theta$ . . . 186
26.1.2 时间误差：$v\Delta t$ . . . 186
26.2 外参的数学定义与标定链 . . . 187
26.3 相机内参：针孔模型与畸变 . . . 187
26.4 张正友标定法：从单应到内参 . . . 188
26.4.1 第一步：平面到图像的单应 . . . 188
26.4.2 第二步：用旋转矩阵的正交性构造约束 . . . 188
26.4.3 第三步：把约束写成线性方程组 . . . 188
26.4.4 第四步：由 $\boldsymbol{B}$ 闭式解出内参 . . . 189
26.4.5 第五步：每张图的外参 . . . 189
26.4.6 第六步：最大似然细化 . . . 189
26.5 IMU 标定与 Allan 方差 . . . 189
26.5.1 Allan 方差：定义 . . . 190
26.5.2 怎么从曲线上读参数 . . . 190
26.5.3 从 Allan 读数到滤波器参数 . . . 191
26.6 雷达-IMU 外参：手眼标定 $AX = XB$ . . . 191
26.7 时间同步与时间偏移估计 . . . 192
26.8 标定验证与常见错误 . . . 193

第二十七章 调参与排错手册　195
27.1 调参方法论 . . . 195
27.2 FAST-LIO / FAST-LIO2：关键参数 . . . 195

27.3 Point-LIO：差异参数 . . . 197
27.4 Cartographer：2D 建图参数 . . . 198
27.5 Nav2：代价地图与控制器参数 . . . 199
27.6 排错速查表 . . . 200
27.7 日志与可视化 . . . 201

第二十八章 数据集、评测与复现 202
28.1 公开数据集 . . . 202
28.2 评价指标：ATE、RPE 与漂移率 . . . 202
28.2.1 ATE：绝对轨迹误差 . . . 202
28.2.2 Umeyama 对齐的闭式解 . . . 203
28.2.3 RPE：相对位姿误差 . . . 204
28.2.4 漂移率与工程指标 . . . 204
28.3 evo：命令与输出解读 . . . 204
28.4 自采数据规范 . . . 205
28.5 可复现实验 . . . 205

第二十九章 RoboMaster 场景实践建议 207
29.1 赛场环境特点 . . . 207
29.2 定位方案选型 . . . 207
29.3 哨兵、步兵与雷达站的差异 . . . 207
29.4 坐标系与标定清单 . . . 208
29.5 备赛流程与验收标准 . . . 208
29.6 常见坑 . . . 209

第三十章 学习路线与资源清单 210
30.1 分阶段路线 . . . 210
30.1.1 第 0–1 月：把环境与概念立起来 . . . 210
30.1.2 第 1–3 月：里程计与 IMU . . . 210
30.1.3 第 3–6 月：SLAM 与导航 . . . 211
30.1.4 第 6–12 月：工程化与独立思考 . . . 211
30.2 代码阅读顺序 . . . 211
30.3 书、课程与文档 . . . 212
30.4 工具链清单 . . . 212
30.5 给新队员的 10 条建议 . . . 212
30.6 结语 . . . 212

参考文献 214

附录 A 公式与符号速查 217

附录 B 术语中英对照表 220

第一部分

基础篇：先建立正确的地图

第一章 定位、建图与导航：全景导论

本章导读

这是全书的“地图”。我们不急着推公式，而是先回答三个最朴素的问题：机器人怎么知道自己在哪里？它怎么知道周围长什么样？它怎么决定往哪走？答案分别对应定位、建图、导航。你会看到这三件事是如何互相纠缠、又如何在工程上被拆开的；也会第一次听到“前端/后端”这两个词，并知道它们在三种不同语境下分别指什么。读完本章，你应该能在脑子里画出整本书的结构，知道后面每一章大概在解决哪一块拼图。

1.1 一个场景：机器人第一次进场地

想象一台刚组装好的哨兵机器人，被放在 RoboMaster 场地的一角。它有一台激光雷达、一个 IMU、四个麦轮，还有一块算力有限的板子。比赛开始前，它需要回答三个问题：

1. **我在哪？**具体到“我在场地的 (x, y) 坐标、朝向多少度”。这叫**定位**（localization）。
2. **周围长什么样？**哪里有墙、哪里是通道、哪里可以走。这叫**建图**（mapping）。
3. **我该怎么过去？**从当前位置到目标点，走哪条路、怎么避开动态的对手。这叫**导航**（navigation）。

三个问题看起来可以按顺序解决，实际却是一个循环：没有地图，定位只能靠猜；没有定位，建出来的图是歪的；没有可靠的定位和地图，导航就是蒙眼狂奔。本章先把它们的定义和关系理清楚。

1.2 三个问题的精确定义

1.2.1 定位

定义 1.1 (定位)

定位是估计机器人在某个参考坐标系中的位姿（pose）的过程。位姿包含位置和姿态：

$$\boldsymbol{x} = (x,\ y,\ z,\ \text{roll},\ \text{pitch},\ \text{yaw}), \tag{1.1}$$

在平面移动机器人上通常简化为 (x, y, θ) 三个量。

“位姿”这个词在后面会反复出现，务必先建立直觉：位置是“你在哪”，姿态是“你朝哪”。两者缺一不可——只知道位置不知道朝向，机器人不知道往哪走；只知道朝向不知道位置，它不知道自己在场地的哪个角落。

按参考坐标系的不同，定位又分为两类：

- **相对定位**（relative localization）：相对于起点估计位姿，误差会累积。里程计就是典型。
- **绝对定位**（absolute localization）：直接在地图坐标系中估计位姿，误差有界。AMCL、先验地图配准属于这一类。

1.2.2 建图

定义 1.2 (建图)

建图是把传感器观测转换成环境表示的过程。环境表示可以是二维占据栅格、三维点云、八叉树、TSDF、语义地图等，取决于用途。

地图不是“越精细越好”。给导航用的地图要能快速查询“这一点能不能走”；给定位用的地图要能快速做点云配准；给人看的地图要清晰。同一个环境，三种用途往往需要三种不同的地图表示。第 14 章会详细讨论。

1.2.3 导航

定义 1.3 (导航)

导航是在已知（或未知）环境中，从当前位置安全、高效地到达目标位置的过程。一个完整的导航系统通常包含：定位、地图、全局规划、局部规划、运动控制。

♣

导航不是一个算法，而是一套系统。它把定位和建图的输出当作输入，把速度和转向指令当作输出。第 18 章会给出完整的系统架构。

1.2.4 三者的关系

三者的关系可以用图 1.1 概括：定位和建图互相依赖（定位需要地图，建图需要位姿），导航同时依赖两者。

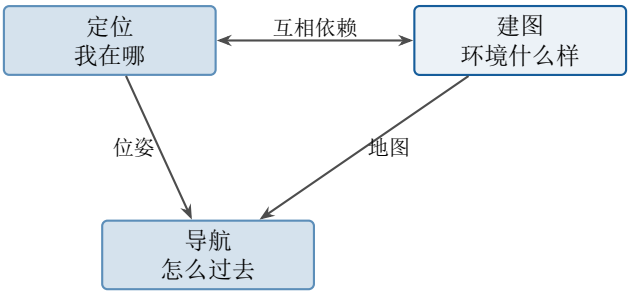


图 1.1: 定位、建图、导航三者的关系。定位与建图互为前提，导航同时依赖两者。

1.3 定位问题的分类

初学者最容易混淆的，是“定位”这个词在不同上下文里指的东西完全不同。表 1.1 给出四种常见的分类维度。

表 1.1: 定位问题的四种分类维度

维度	两类	典型方法
参考系	相对定位 / 绝对定位	里程计 / AMCL、先验地图配准
初始信息	局部定位 / 全局定位	已知初始位姿 / 绑架问题、重定位
时间特性	单帧定位 / 递推滤波	单帧配准 / EKF、粒子滤波
先验地图	有地图 / 无地图	地图匹配 / SLAM、UWB 辅助

这四组概念经常组合出现。比如“哨兵开机后不知道自己在场地哪里，靠着先验地图找回位置”，就是“绝对定位 + 全局定位 + 有地图”；而“比赛过程中用 FAST-LIO 连续估计位姿”，是“相对定位 + 局部定位 + 无地图”。

理解这张表的意义在于：**没有“最好的定位算法”，只有“匹配场景的定位方案”**。这也是为什么本书会花大量篇幅讲每种方法的适用条件和失效场景。

1.4 SLAM：先有鸡还是先有蛋

把定位和建图放在一起，就出现了著名的“鸡生蛋”问题：定位需要地图，建图需要定位，那到底先有谁？

这个问题的正式名字叫**同时定位与建图**（Simultaneous Localization and Mapping，SLAM）。它的解决思路不是“先有鸡还是先有蛋”，而是**同时估计两者**：把位姿序列和地图都当作未知量，用所有观测一起做最大后验估计。

用数学语言写出来，就是

$$(\boldsymbol{X}^*, \boldsymbol{M}^*) = \arg\max_{\boldsymbol{X},\boldsymbol{M}} \; p\left(\boldsymbol{X}, \boldsymbol{M} \mid \boldsymbol{Z}\right), \tag{1.2}$$

其中 $\boldsymbol{X}$ 是位姿序列，$\boldsymbol{M}$ 是地图，$\boldsymbol{Z}$ 是所有观测。对式 (1.2) 取负对数，就变成一个大规模最小二乘问题——这正是第 13 章“后端”要做的事。

直观上，SLAM 的逻辑是：

1. 先用短时间内的观测估计相邻两帧之间的相对运动（这一步不需要地图）；
2. 把相对运动串起来，得到一条“局部一致”的轨迹；
3. 随着轨迹变长，误差累积，轨迹开始漂；
4. 当机器人回到曾经去过的地方（回环），用这个信息把整条轨迹“拉回来”；
5. 轨迹修正后，地图也跟着修正。

第 1–2 步是前端，第 4–5 步是后端。这就是下一节要讲的划分。

1.5 “前端/后端”到底指什么

“前端”和“后端”是本书出现频率最高的两个词，也是最容易被误用的两个词。它们在不同语境下含义不同，先在这里建立直觉，细节分别在第 11 章（SLAM）、第 18 章（导航）展开。

一句话版本

前端负责“局部、快速、增量”的估计，后端负责“全局、较慢、一致”的优化。

三种常见语境：

1. **SLAM 语境**（最常见）：前端 = 数据关联 + 帧间位姿估计 + 局部地图；后端 = 回环检测 + 全局优化 + 地图修正。这是第 11–13 章的内容。
2. **定位系统语境**：前端 = 里程计（LIO/VIO），后端 = 先验地图配准 + 因子图 + 状态机。这是第 23 章 Z-Localization 的划分：LIO 自己维护 odom 系，后端负责 map → odom 的变换。
3. **导航系统语境**：前端 = 定位与建图（感知），后端 = 规划与控制（决策）。这种说法在工程口语里很常见，但并不严谨——它把“导航”当成了一个更大的系统。

第 11 章会用一张表把三种语境并列对比。你现在只需要记住：**看到“前端/后端”时，先问一句“相对于什么系统而言”。**

1.6 全书地图与阅读路线

本书共五篇三十章，结构如图 1.2 所示。

如果时间有限，有三种读法：

- **速成路线**（一周）：第 1、2、4、6、11、16、17 章。理解“里程计怎么算、前端后端各干什么、FAST-LIO 怎么工作”。
- **建图路线**（两周）：第 1–3、7、11–15 章。重点是 Cartographer 和地图表示。
- **导航路线**（两周）：第 1、10、11、18–22 章。重点是 Nav2 与 CMU 3D 导航栈。

1.7 给新手的三个建议

1. **先在脑子里跑通数据流，再抠公式。**每读一个算法，先问：输入是什么？输出是什么？中间经过几个模块？每个模块的频率是多少？把这张图画出来，比记公式有用得多。
2. **公式要能“算一个数”。**看到一个公式，尝试代入具体数字算一遍（比如 $\Delta t = 10$ ms、速度 1 m/s，走 1 秒误差多少）。能算出数量级，才算真懂。

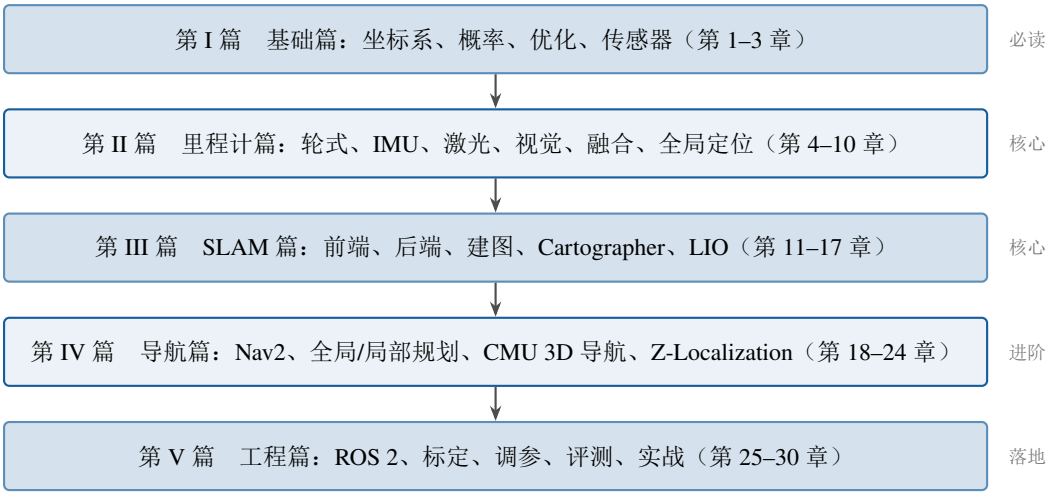


图 1.2: 全书结构。建议按顺序阅读第一篇，之后可按兴趣跳读。

3. **不要跳过标定和坐标变换**。新手 80% 的“算法不工作”，最后都发现是外参错了、时间戳错了、TF 树断了。第 26 章值得反复看。

本章小结

1. 定位估计位姿，建图表示环境，导航规划并执行运动；三者互相依赖，构成一个闭环。
2. 定位可按参考系、初始信息、时间特性、先验地图四个维度分类；没有万能的算法，只有匹配场景的方案。
3. SLAM 通过“同时估计位姿与地图”解决鸡生蛋问题，本质是一个大规模最大后验估计。
4. 前端负责局部、快速、增量的估计；后端负责全局、较慢、一致的优化。看到这两个词时先明确“相对于哪个系统”。
5. 全书五篇三十章：基础 → 里程计 → SLAM → 导航 → 工程。

第一章 练习

1. 用一句话分别定义定位、建图、导航，并各举一个 RoboMaster 场景中的例子。
2. 为什么“相对定位”的误差会累积，而“绝对定位”的误差有界？请从参考系的角度解释。
3. 如果机器人开机时不知道自己在场地的哪个角落，它属于哪一种定位问题？可以用哪些方法解决？
4. 用自己的话解释“前端/后端”，并说明在 SLAM 语境和导航语境下有什么不同。
5. 画出你心目中一台哨兵机器人的数据流图（传感器 → 算法模块 → 执行器），标出每一路数据的频率量级。

延伸阅读

- [38]：第 1 章给出了定位、建图、SLAM 的经典定义与分类，是本章最好的补充。
- [1]：第 1 章从状态估计角度统一了定位与 SLAM。
- [27]：了解 ROS 2 的工程背景，为第 25 章做准备。
- 本书第 11 章：前端/后端的正式定义与数据流。

第二章 数学基础：从概率到李群

本章导读

这一章不追求数学上的完备，只准备“够用”的工具箱。我们按定位与导航里真正会用到的东西来组织：坐标系怎么描述、概率怎么描述不确定性、方差和协方差到底在算什么、如何用最小二乘去拟合一个位姿、以及为什么旋转不能直接相加而要用李群。每一个概念都会从定义讲起，给出计算公式和一个可以手算的小例子。读完这一章，后面所有章节里的公式你至少能“看懂每一行在做什么”。

2.1 坐标系与刚体变换

2.1.1 点、向量与坐标系

一个三维空间中的点，只有在指定了坐标系之后才有坐标。本书约定：用 $\{W\}$ 表示世界系（map 系），$\{B\}$ 表示机体系（默认与 IMU 系重合），$\{L\}$、$\{C\}$ 分别表示雷达系和相机系。点 P 在 $\{B\}$ 系下的坐标记作 ${}^{B}\boldsymbol{p} \in \mathbb{R}^3$，写成列向量

$$ {}^{B}\boldsymbol{p} = \begin{bmatrix} p_x \\ p_y \\ p_z \end{bmatrix}. \tag{2.1} $$

同一个点在不同坐标系下的坐标不同，它们之间通过“旋转 + 平移”联系起来：

$$ {}^{W}\boldsymbol{p} = {}^{W}\boldsymbol{R}_{B}\,{}^{B}\boldsymbol{p} + {}^{W}\boldsymbol{p}_{B}, \tag{2.2} $$

其中 ${}^{W}\boldsymbol{R}_{B} \in \mathbb{R}^{3\times 3}$ 是旋转矩阵，${}^{W}\boldsymbol{p}_{B} \in \mathbb{R}^3$ 是 B 系原点在 W 系中的位置。式 (2.2) 就是刚体变换（rigid transformation）的全部内容：先转，再平移。

2.1.2 旋转矩阵

旋转矩阵的定义

一个 3×3 矩阵 $\boldsymbol{R}$ 是旋转矩阵，当且仅当它满足

$$ \boldsymbol{R}^{\top}\boldsymbol{R} = \boldsymbol{I}, \qquad \det(\boldsymbol{R}) = 1. \tag{2.3} $$

所有这样的矩阵构成特殊正交群 $\mathrm{SO}(3)$。

为什么是这两个条件？$\boldsymbol{R}^{\top}\boldsymbol{R} = \boldsymbol{I}$ 保证长度和夹角不变：对任意向量 $\boldsymbol{v}$，有 $\|\boldsymbol{R}\boldsymbol{v}\|^2 = \boldsymbol{v}^{\top}\boldsymbol{R}^{\top}\boldsymbol{R}\boldsymbol{v} = \boldsymbol{v}^{\top}\boldsymbol{v} = \|\boldsymbol{v}\|^2$。而 $\det(\boldsymbol{R}) = \pm 1$ 中取 $+1$ 是为了排除“镜像”，因为镜子里的世界不是旋转出来的。

旋转矩阵的复合就是矩阵乘法：先转 ${}^{B}\boldsymbol{R}_{C}$ 再转 ${}^{W}\boldsymbol{R}_{B}$，总旋转是 ${}^{W}\boldsymbol{R}_{B}\,{}^{B}\boldsymbol{R}_{C}$。注意下标“相邻相消”：

$$ {}^{W}\boldsymbol{p} = {}^{W}\boldsymbol{R}_{B}\left({}^{B}\boldsymbol{R}_{C}\,{}^{C}\boldsymbol{p}\right) = \left({}^{W}\boldsymbol{R}_{B}\,{}^{B}\boldsymbol{R}_{C}\right){}^{C}\boldsymbol{p}. \tag{2.4} $$

这个“下标相消”规则在后面的 TF 树、标定、外参里会反复出现，值得记牢。

2.1.3 齐次坐标与 SE(3)

式 (2.2) 是“先乘再加”，不便于连续复合。引入齐次坐标可以把旋转和平移合并成一次矩阵乘法。定义

$$ {}^{W}\bar{\boldsymbol{p}} = \begin{bmatrix} {}^{W}\boldsymbol{p} \\ 1 \end{bmatrix} \in \mathbb{R}^4, \qquad {}^{W}\boldsymbol{T}_{B} = \begin{bmatrix} {}^{W}\boldsymbol{R}_{B} & {}^{W}\boldsymbol{p}_{B} \\ \boldsymbol{0}^{\top} & 1 \end{bmatrix} \in \mathbb{R}^{4\times 4}, \tag{2.5} $$

于是

$$ {}^{W}\bar{\boldsymbol{p}} = {}^{W}\boldsymbol{T}_{B}\,{}^{B}\bar{\boldsymbol{p}}. \tag{2.6} $$

所有这样的 $\boldsymbol{T}$ 构成特殊欧氏群 SE(3)。复合规则同样是矩阵乘法，且

$$ \boldsymbol{T}^{-1} = \begin{bmatrix} \boldsymbol{R}^{\top} & -\boldsymbol{R}^{\top}\boldsymbol{t} \\ \boldsymbol{0}^{\top} & 1 \end{bmatrix}. \tag{2.7} $$

验证式 (2.7)：直接乘开，左上角 $\boldsymbol{R}^{\top}\boldsymbol{R} = \boldsymbol{I}$，右上角 $\boldsymbol{R}^{\top}\boldsymbol{t} - \boldsymbol{R}^{\top}\boldsymbol{t} = \boldsymbol{0}$。这个逆的闭式表达比数值求逆快得多，在代码里几乎总用这个形式。

2.1.4 欧拉角与万向锁

把旋转拆成绕 z、y、x 三个轴依次转的角度 (ψ, θ, ϕ)（yaw、pitch、roll），叫欧拉角。它直观、人类可读（IMU 输出的姿态常用它显示），但有两个致命问题：

1. **万向锁（gimbal lock）**：当 pitch 接近 ±90° 时，yaw 和 roll 退化成同一个自由度，姿态表示不唯一，运动学方程出现奇异。
2. **插值困难**：两组欧拉角之间直接线性插值，得到的中间姿态往往不是“最短路径”旋转。

因此，内部计算一律用旋转矩阵或四元数，欧拉角只在“给人看”的时候使用。

2.2 概率论最小工具箱

定位的本质是“在不确定中估计”。传感器有噪声，运动模型有误差，所以我们用概率分布描述“我认为机器人在哪”。这一节把后面会用到的概率概念逐个讲清楚，尤其是**方差**和**协方差**——它们是全书出现频率最高的两个词。

2.2.1 随机变量与概率

一个随机变量 X 是“取值不确定的量”。如果 X 只能取有限个离散值 $x_1, x_2, \ldots$，用概率质量函数 $p(X = x_i)$ 描述，满足

$$ p(X = x_i) \geq 0, \qquad \sum_{i} p(X = x_i) = 1. \tag{2.8} $$

如果 X 可以在实数轴上连续取值，用概率密度函数 $p(x)$ 描述，满足

$$ p(x) \geq 0, \qquad \int_{-\infty}^{+\infty} p(x)\mathrm{d}x = 1, \tag{2.9} $$

并且落在区间 $[a, b]$ 的概率是

$$ P(a \leq X \leq b) = \int_{a}^{b} p(x)\mathrm{d}x. \tag{2.10} $$

注意：$p(x)$ 本身不是概率，它可能大于 1；只有积分出来的面积才是概率。

2.2.2 期望

定义：期望

离散随机变量的期望是

$$ \mathbb{E}[X] = \sum_{i} x_i\, p(X = x_i), \tag{2.11} $$

连续随机变量的期望是

$$ \mathbb{E}[X] = \int_{-\infty}^{+\infty} x\, p(x)\mathrm{d}x. \tag{2.12} $$

期望就是“加权平均”，权重是概率。

例：掷一枚均匀硬币，正面记 1、反面记 0，则

$$\mathbb{E}[X] = 1 \times 0.5 + 0 \times 0.5 = 0.5. \tag{2.13}$$

期望最重要的性质是**线性性**，它对任意随机变量（无论是否独立）都成立：

$$\mathbb{E}[aX + b] = a\mathbb{E}[X] + b, \qquad \mathbb{E}[X + Y] = \mathbb{E}[X] + \mathbb{E}[Y]. \tag{2.14}$$

证明第一个式子（离散情形）：

$$\mathbb{E}[aX + b] = \sum_i (ax_i + b)p(x_i) = a\sum_i x_i p(x_i) + b\sum_i p(x_i) = a\mathbb{E}[X] + b. \tag{2.15}$$

最后一步用了 $\sum_i p(x_i) = 1$。这个性质是后面所有“误差传播”公式的基础。

2.2.3 方差与标准差

定义：方差

方差衡量随机变量偏离其期望的程度，定义为

$$\mathrm{Var}(X) = \mathbb{E}\left[(X - \mathbb{E}[X])^2\right]. \tag{2.16}$$

它的平方根 $\sigma_X = \sqrt{\mathrm{Var}(X)}$ 叫标准差。

把定义展开，可以得到更常用的计算公式：

$$\mathrm{Var}(X) = \mathbb{E}\left[X^2 - 2X\mathbb{E}[X] + (\mathbb{E}[X])^2\right] = \mathbb{E}[X^2] - 2\mathbb{E}[X]\mathbb{E}[X] + (\mathbb{E}[X])^2 = \mathbb{E}[X^2] - (\mathbb{E}[X])^2. \tag{2.17}$$

第二步用了期望的线性性，第三步用了 $\mathbb{E}[\mathbb{E}[X]] = \mathbb{E}[X]$（$\mathbb{E}[X]$ 是一个常数）。所以记住这个等价形式：

$$\boxed{\mathrm{Var}(X) = \mathbb{E}[X^2] - (\mathbb{E}[X])^2} \tag{2.18}$$

即“平方的期望减去期望的平方”。

例：掷一枚均匀骰子，$X \in \{1, 2, 3, 4, 5, 6\}$，每个概率 $1/6$。

$$\mathbb{E}[X] = \frac{1 + 2 + 3 + 4 + 5 + 6}{6} = 3.5, \tag{2.19}$$

$$\mathbb{E}[X^2] = \frac{1 + 4 + 9 + 16 + 25 + 36}{6} = \frac{91}{6} \approx 15.17, \tag{2.20}$$

$$\mathrm{Var}(X) = \frac{91}{6} - 3.5^2 = 15.17 - 12.25 \approx 2.92, \tag{2.21}$$

$$\sigma_X = \sqrt{2.92} \approx 1.71. \tag{2.22}$$

也就是说，掷骰子的结果平均偏离 3.5 约 1.71。方差是“偏离程度的平方的平均”，开根号后才和原数据同量纲。

方差有两个常用性质，后面推导漂移时会直接用到：

$$\mathrm{Var}(aX + b) = a^2\,\mathrm{Var}(X), \tag{2.23}$$

$$\text{若} X, Y \text{ 相互独立，则 } \mathrm{Var}(X + Y) = \mathrm{Var}(X) + \mathrm{Var}(Y). \tag{2.24}$$

先证式 (2.23)：

$$\mathrm{Var}(aX + b) = \mathbb{E}\left[(aX + b - \mathbb{E}[aX + b])^2\right] = \mathbb{E}\left[(aX - a\mathbb{E}[X])^2\right] = a^2\mathbb{E}\left[(X - \mathbb{E}[X])^2\right] = a^2\,\mathrm{Var}(X). \tag{2.25}$$

再证式 (2.24)。先算

$$\mathrm{Var}(X + Y) = \mathbb{E}\left[(X + Y)^2\right] - (\mathbb{E}[X + Y])^2 = \mathbb{E}[X^2] + 2\mathbb{E}[XY] + \mathbb{E}[Y^2] - (\mathbb{E}[X] + \mathbb{E}[Y])^2. \tag{2.26}$$

展开右边最后一项：

$$(\mathbb{E}[X] + \mathbb{E}[Y])^2 = (\mathbb{E}[X])^2 + 2\mathbb{E}[X]\mathbb{E}[Y] + (\mathbb{E}[Y])^2. \tag{2.27}$$

两式相减得

$$\mathrm{Var}(X+Y) = \big[\mathbb{E}[X^2] - (\mathbb{E}[X])^2\big] + \big[\mathbb{E}[Y^2] - (\mathbb{E}[Y])^2\big] + 2\,(\mathbb{E}[XY] - \mathbb{E}[X]\mathbb{E}[Y])\,. \tag{2.28}$$

前两个方括号分别是 $\mathrm{Var}(X)$、$\mathrm{Var}(Y)$；而独立意味着 $\mathbb{E}[XY] = \mathbb{E}[X]\mathbb{E}[Y]$，最后一项为零。于是 $\mathrm{Var}(X+Y) = \mathrm{Var}(X) + \mathrm{Var}(Y)$。

2.2.4 协方差与协方差矩阵

方差只描述一个变量。两个变量“一起变化”的程度用协方差描述：

$$\mathrm{Cov}(X,Y) = \mathbb{E}\left[(X - \mathbb{E}[X])(Y - \mathbb{E}[Y])\right] = \mathbb{E}[XY] - \mathbb{E}[X]\mathbb{E}[Y]. \tag{2.29}$$

展开方式与方差完全一样。$\mathrm{Cov}(X,X) = \mathrm{Var}(X)$。若 X, Y 独立，则 $\mathrm{Cov}(X,Y) = 0$；但反过来不成立——协方差为零只说明“没有线性相关”，可能存在非线性关系。

对多维随机向量 $\boldsymbol{x} = [x_1, \ldots, x_n]^\top$，把所有两两协方差排成矩阵：

$$\boldsymbol{\Sigma} = \mathbb{E}\left[(\boldsymbol{x} - \mathbb{E}[\boldsymbol{x}])(\boldsymbol{x} - \mathbb{E}[\boldsymbol{x}])^\top\right] \in \mathbb{R}^{n\times n}, \tag{2.30}$$

叫协方差矩阵。它的对角元是各方差 $\Sigma_{ii} = \mathrm{Var}(x_i)$，非对角元是协方差 $\Sigma_{ij} = \mathrm{Cov}(x_i, x_j)$。协方差矩阵一定是对称半正定的，因为对任意向量 $\boldsymbol{a}$，

$$\boldsymbol{a}^\top \boldsymbol{\Sigma} \boldsymbol{a} = \mathbb{E}\left[\left(\boldsymbol{a}^\top(\boldsymbol{x} - \mathbb{E}[\boldsymbol{x}])\right)^2\right] \geq 0. \tag{2.31}$$

协方差的几何意义：二维情况下，协方差矩阵的特征向量给出分布的“主轴方向”，特征值给出沿主轴的方差大小。特征值越大，说明该方向上不确定性越大，分布椭圆越长。第 9 章用“马氏距离”做外点剔除、第 13 章用“信息矩阵”加权残差，本质都是在利用这个椭圆。

2.2.5 高斯分布

一维高斯（正态）分布的概率密度为

$$p(x) = \frac{1}{\sqrt{2\pi}\,\sigma} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right), \tag{2.32}$$

记作 $x \sim \mathcal{N}(\mu, \sigma^2)$。它由两个参数完全确定：均值 μ 和方差 σ^2。

验证它确实是概率密度：首先换元 $u = (x-\mu)/(\sqrt{2}\sigma)$，则 $\mathrm{d}x = \sqrt{2}\sigma \mathrm{d}u$，

$$\int_{-\infty}^{+\infty} p(x)\mathrm{d}x = \frac{1}{\sqrt{2\pi}\sigma} \int_{-\infty}^{+\infty} e^{-u^2} \sqrt{2}\sigma \mathrm{d}u = \frac{1}{\sqrt{\pi}} \int_{-\infty}^{+\infty} e^{-u^2} \mathrm{d}u. \tag{2.33}$$

利用高斯积分 $\int_{-\infty}^{+\infty} e^{-u^2}\mathrm{d}u = \sqrt{\pi}$，得到积分等于 1。其次算期望：

$$\mathbb{E}[X] = \int x\, p(x) \mathrm{d}x = \mu, \tag{2.34}$$

因为密度关于 $x = \mu$ 对称。再算方差，令 $u = (x-\mu)/\sigma$：

$$\mathrm{Var}(X) = \frac{1}{\sqrt{2\pi}\sigma} \int_{-\infty}^{+\infty} (x-\mu)^2 e^{-\frac{(x-\mu)^2}{2\sigma^2}} \mathrm{d}x = \frac{\sigma^2}{\sqrt{2\pi}} \int_{-\infty}^{+\infty} u^2 e^{-u^2/2} \mathrm{d}u = \sigma^2, \tag{2.35}$$

其中用到分部积分得到的 $\int u^2 e^{-u^2/2}\mathrm{d}u = \sqrt{2\pi}$。

多维高斯分布写成

$$p(\boldsymbol{x}) = \frac{1}{(2\pi)^{n/2}\,|\boldsymbol{\Sigma}|^{1/2}} \exp\left(-\frac{1}{2}(\boldsymbol{x}-\boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1} (\boldsymbol{x}-\boldsymbol{\mu})\right), \tag{2.36}$$

记作 $\boldsymbol{x} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$。指数里的二次型

$$d^2 = (\boldsymbol{x}-\boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1} (\boldsymbol{x}-\boldsymbol{\mu}) \tag{2.37}$$

叫马氏距离的平方。它和欧氏距离的区别是“按不确定性的方向加权”：方差小的方向权重更大。第 9 章会证明，d^2 服从卡方分布，这正是外点剔除检验的依据。

2.2.6 条件概率与贝叶斯公式

条件概率的定义是

$$p(A \mid B) = \frac{p(A,B)}{p(B)}, \qquad p(B) > 0, \tag{2.38}$$

即“在 B 发生的条件下 A 发生的概率”等于联合概率除以 B 的概率。由对称性 $p(A,B) = p(B \mid A)p(A)$，代入式 (2.38) 并交换 A、B 的位置，得到贝叶斯公式：

$$p(A \mid B) = \frac{p(B \mid A)\, p(A)}{p(B)}. \tag{2.39}$$

把它放到定位里：A 是“机器人的位姿”，B 是“传感器观测”。于是

$$\underbrace{p(\text{位姿} \mid \text{观测})}_{\text{后验}} = \frac{\overbrace{p(\text{观测} \mid \text{位姿})}^{\text{似然}}\ \overbrace{p(\text{位姿})}^{\text{先验}}}{\underbrace{p(\text{观测})}_{\text{证据}}}. \tag{2.40}$$

后验就是“看到这帧点云后，我认为自己在哪”。分母与位姿无关，在优化里通常被忽略（当作常数）。整本书的定位与 SLAM 算法，本质上都在算这个后验：滤波器用递推近似它，优化器用最小二乘找它的最大值。

2.2.7 独立性

两个随机变量独立，当且仅当

$$p(X,Y) = p(X)p(Y) \quad \Longleftrightarrow \quad p(X \mid Y) = p(X). \tag{2.41}$$

独立是后面大量公式成立的前提：多个观测独立时，似然可以连乘，取对数后变成求和，最小二乘的各项才能分开写。工程上要特别小心——时间上相邻的传感器读数、同一平面上相邻的点，往往并不独立，第 23 章 Z-Localization 讨论的“平面内点残差相关”就是一例。

2.2.8 卡方分布

定义 2.1 (卡方分布)

设 $X_1, X_2, \ldots, X_k$ 是相互独立的标准正态随机变量，$X_i \sim \mathcal{N}(0,1)$，则

$$Q = X_1^2 + X_2^2 + \cdots + X_k^2 \tag{2.42}$$

服从自由度为 k 的卡方分布，记作 $Q \sim \chi^2(k)$。

♣

自由度 $k = 1$ 时密度函数为

$$p(q) = \frac{1}{\sqrt{2\pi q}} e^{-q/2}, \qquad q \geq 0, \tag{2.43}$$

$k = 2$ 时是 $p(q) = \frac{1}{2}e^{-q/2}$，$k$ 一般时

$$p(q) = \frac{1}{2^{k/2}\Gamma(k/2)} q^{k/2-1} e^{-q/2}, \tag{2.44}$$

其中 Γ 是伽马函数（$\Gamma(n) = (n-1)!$）。我们不要求记住这个密度，只需要知道**给定置信水平 α 时，分位数 $\chi^2_{k,\alpha}$ 可以从表里查到**。

为什么工程里关心卡方分布？因为如果残差 $\boldsymbol{r}$ 满足零均值高斯分布、协方差为 $\boldsymbol{S}$，那么二次型

$$d^2 = \boldsymbol{r}^\top \boldsymbol{S}^{-1} \boldsymbol{r} \tag{2.45}$$

就是若干个标准正态变量的平方和，服从 $\chi^2(k)$（k 是残差维数）。于是可以设定一个阈值 $\chi^2_{k,0.95}$：如果 d^2 超过它，说明这个观测“太离谱”，应当作为外点剔除。这就是第 9 章马氏距离检验、第 23 章 NIS 检验的数学基础。

2.3 微积分与优化

2.3.1 泰勒展开

一元函数在 x_0 处的泰勒展开是

$$f(x_0+\delta)=f(x_0)+f'(x_0)\delta+\frac{1}{2}f''(x_0)\delta^2+\frac{1}{6}f'''(x_0)\delta^3+\cdots \tag{2.46}$$

它的含义是：用多项式在局部逼近函数。当我们只关心很小的 δ 时，可以只保留前两项（一阶近似）或前三项（二阶近似），把高阶项写成 $O(\delta^2)$、$O(\delta^3)$。

多元函数的一阶近似为

$$f(\boldsymbol{x}_0+\delta\boldsymbol{x})\approx f(\boldsymbol{x}_0)+\nabla f(\boldsymbol{x}_0)^{\top}\delta\boldsymbol{x}, \tag{2.47}$$

其中梯度 $\nabla f=[\partial f/\partial x_1,\ldots,\partial f/\partial x_n]^{\top}$。二阶近似为

$$f(\boldsymbol{x}_0+\delta\boldsymbol{x})\approx f(\boldsymbol{x}_0)+\nabla f^{\top}\delta\boldsymbol{x}+\frac{1}{2}\delta\boldsymbol{x}^{\top}\boldsymbol{H}\delta\boldsymbol{x}, \tag{2.48}$$

其中 $\boldsymbol{H}$ 是海森矩阵（见下节）。**整本书的非线性优化，本质上就是反复做这个一阶/二阶近似，然后解一个线性问题。**

2.3.2 雅可比矩阵

定义 2.2 (雅可比矩阵)

设 $\boldsymbol{f}:\mathbb{R}^n\to\mathbb{R}^m$ 是一个向量函数，$\boldsymbol{f}(\boldsymbol{x})=[f_1(\boldsymbol{x}),\ldots,f_m(\boldsymbol{x})]^{\top}$。它的雅可比矩阵 $\boldsymbol{J}\in\mathbb{R}^{m\times n}$ 定义为

$$\boldsymbol{J}=\frac{\partial \boldsymbol{f}}{\partial \boldsymbol{x}}=\begin{bmatrix}\dfrac{\partial f_1}{\partial x_1} & \cdots & \dfrac{\partial f_1}{\partial x_n}\\ \vdots & \ddots & \vdots\\ \dfrac{\partial f_m}{\partial x_1} & \cdots & \dfrac{\partial f_m}{\partial x_n}\end{bmatrix}. \tag{2.49}$$

♣

雅可比就是“多元函数的导数”，它告诉你：输入变化一点点，输出会怎么变。$\boldsymbol{f}(\boldsymbol{x}+\delta\boldsymbol{x})\approx\boldsymbol{f}(\boldsymbol{x})+\boldsymbol{J}\delta\boldsymbol{x}$。

例：$\boldsymbol{f}(x,y)=\begin{bmatrix}x^2y\\ \sin x\end{bmatrix}$，则

$$\boldsymbol{J}=\begin{bmatrix}2xy & x^2\\ \cos x & 0\end{bmatrix}. \tag{2.50}$$

2.3.3 海森矩阵与极值

定义 2.3 (海森矩阵)

标量函数 $f:\mathbb{R}^n\to\mathbb{R}$ 的二阶偏导矩阵

$$\boldsymbol{H}=\frac{\partial^2 f}{\partial \boldsymbol{x}^2},\qquad H_{ij}=\frac{\partial^2 f}{\partial x_i\partial x_j} \tag{2.51}$$

叫海森矩阵。

♣

若 $\nabla f(\boldsymbol{x}^*)=\boldsymbol{0}$ 且 $\boldsymbol{H}(\boldsymbol{x}^*)$ 正定，则 $\boldsymbol{x}^*$ 是局部极小点；若负定则是极大点；若不定则是鞍点。这个判别法与一元函数“二阶导数为正”完全对应。

2.3.4 最小二乘与正规方程

定义 2.4 (最小二乘)

给定超定方程组 $\boldsymbol{A}\boldsymbol{x} \approx \boldsymbol{b}$（$\boldsymbol{A} \in \mathbb{R}^{m\times n}$，$m > n$），最小二乘解是使残差平方和最小的 $\boldsymbol{x}$：

$$\boldsymbol{x}^* = \arg\min_{\boldsymbol{x}} \|\boldsymbol{A}\boldsymbol{x} - \boldsymbol{b}\|^2 . \tag{2.52}$$

♣

推导：令 $F(\boldsymbol{x}) = \|\boldsymbol{A}\boldsymbol{x} - \boldsymbol{b}\|^2 = (\boldsymbol{A}\boldsymbol{x} - \boldsymbol{b})^\top(\boldsymbol{A}\boldsymbol{x} - \boldsymbol{b})$。展开：

$$F(\boldsymbol{x}) = \boldsymbol{x}^\top\boldsymbol{A}^\top\boldsymbol{A}\boldsymbol{x} - 2\boldsymbol{b}^\top\boldsymbol{A}\boldsymbol{x} + \boldsymbol{b}^\top\boldsymbol{b}. \tag{2.53}$$

对 $\boldsymbol{x}$ 求梯度并令其为零：

$$\nabla F = 2\boldsymbol{A}^\top\boldsymbol{A}\boldsymbol{x} - 2\boldsymbol{A}^\top\boldsymbol{b} = \boldsymbol{0}, \tag{2.54}$$

得到**正规方程（normal equation）**

$$\boxed{\boldsymbol{A}^\top\boldsymbol{A}\,\boldsymbol{x} = \boldsymbol{A}^\top\boldsymbol{b}} \tag{2.55}$$

若 $\boldsymbol{A}^\top\boldsymbol{A}$ 可逆，则 $\boldsymbol{x} = (\boldsymbol{A}^\top\boldsymbol{A})^{-1}\boldsymbol{A}^\top\boldsymbol{b}$。

几何解释：$\boldsymbol{A}\boldsymbol{x}^*$ 是 $\boldsymbol{b}$ 在 $\boldsymbol{A}$ 的列空间上的正交投影，残差 $\boldsymbol{A}\boldsymbol{x}^* - \boldsymbol{b}$ 垂直于列空间。这就是“最小二乘”名字的来源。

2.3.5 高斯–牛顿法

实际问题里残差通常是非线性的：$\boldsymbol{r}(\boldsymbol{x})$，目标是最小化 $\frac{1}{2}\|\boldsymbol{r}(\boldsymbol{x})\|^2$。对残差做一阶泰勒展开：

$$\boldsymbol{r}(\boldsymbol{x} + \delta\boldsymbol{x}) \approx \boldsymbol{r}(\boldsymbol{x}) + \boldsymbol{J}\delta\boldsymbol{x}, \tag{2.56}$$

代入目标函数：

$$\frac{1}{2}\|\boldsymbol{r} + \boldsymbol{J}\delta\boldsymbol{x}\|^2 = \frac{1}{2}\left(\boldsymbol{r}^\top\boldsymbol{r} + 2\boldsymbol{r}^\top\boldsymbol{J}\delta\boldsymbol{x} + \delta\boldsymbol{x}^\top\boldsymbol{J}^\top\boldsymbol{J}\delta\boldsymbol{x}\right). \tag{2.57}$$

对 $\delta\boldsymbol{x}$ 求导并令其为零：

$$\boldsymbol{J}^\top\boldsymbol{J}\,\delta\boldsymbol{x} = -\boldsymbol{J}^\top\boldsymbol{r}, \tag{2.58}$$

这就是**高斯–牛顿方程**。解出 $\delta\boldsymbol{x}$ 后更新 $\boldsymbol{x} \leftarrow \boldsymbol{x} + \delta\boldsymbol{x}$，重复直到收敛。它是式 (2.55) 的非线性版本，也是第 7 章 ICP、第 13 章因子图、第 15 章 Cartographer 扫描匹配共同的核心。

高斯–牛顿在残差较大或 $\boldsymbol{J}^\top\boldsymbol{J}$ 接近奇异时会不稳定。Levenberg–Marquardt（LM）给它加了一个阻尼项：

$$\left(\boldsymbol{J}^\top\boldsymbol{J} + \lambda\boldsymbol{I}\right)\delta\boldsymbol{x} = -\boldsymbol{J}^\top\boldsymbol{r}, \tag{2.59}$$

λ 大时退化为梯度下降（稳定但慢），λ 小时退化为高斯–牛顿（快但可能发散）。实际实现中 λ 会根据每步误差是变大还是变小自适应调整。

2.4 线性代数补充

2.4.1 特征值与特征向量

定义 2.5 (特征值与特征向量)

对方阵 $\boldsymbol{A} \in \mathbb{R}^{n\times n}$，若存在非零向量 $\boldsymbol{v}$ 和标量 λ 满足

$$\boldsymbol{A}\boldsymbol{v} = \lambda\boldsymbol{v}, \tag{2.60}$$

则称 λ 为特征值，$\boldsymbol{v}$ 为对应的特征向量。

♣

它的几何意义是：矩阵 $\boldsymbol{A}$ 作用在 $\boldsymbol{v}$ 上不改变方向，只把长度缩放 λ 倍。

对称矩阵（如协方差矩阵、$\boldsymbol{J}^\top\boldsymbol{J}$）有非常好的性质：

1. 所有特征值都是实数；
2. 不同特征值对应的特征向量相互正交；
3. 可以正交对角化：$\boldsymbol{A}=\boldsymbol{Q}\boldsymbol{\Lambda}\boldsymbol{Q}^\top$，其中 $\boldsymbol{Q}$ 的列是单位正交特征向量，$\boldsymbol{\Lambda}=\mathrm{diag}(\lambda_1,\dots,\lambda_n)$。

在定位里，特征值常被用来判断“可观测性”。例如扫描匹配的信息矩阵 $\boldsymbol{\Lambda}=\boldsymbol{J}^\top\boldsymbol{J}$，若某个特征值接近零，说明对应方向上“怎么动残差都不变”，即该方向退化（长走廊、大平面就是这样）。第 7 章和第 23 章都会用到这个判据。

2.4.2 二次型与正定

形如 $q(\boldsymbol{x})=\boldsymbol{x}^\top\boldsymbol{A}\boldsymbol{x}$ 的表达式叫二次型。若对任意 $\boldsymbol{x}\neq\boldsymbol{0}$ 都有 $q(\boldsymbol{x})>0$，称 $\boldsymbol{A}$ 正定（$\boldsymbol{A}\succ 0$），等价于所有特征值大于零。正定二次型的等高线是椭球，椭球的半轴长度是 $1/\sqrt{\lambda_i}$——这就是协方差矩阵“椭圆”的严格解释。

2.4.3 奇异值分解

任意矩阵 $\boldsymbol{A}\in\mathbb{R}^{m\times n}$ 可以分解为

$$\boldsymbol{A}=\boldsymbol{U}\boldsymbol{\Sigma}\boldsymbol{V}^\top, \tag{2.61}$$

其中 $\boldsymbol{U}\in\mathbb{R}^{m\times m}$、$\boldsymbol{V}\in\mathbb{R}^{n\times n}$ 是正交矩阵，$\boldsymbol{\Sigma}$ 是对角矩阵（对角元为非负奇异值）。SVD 在点云配准里有关键作用：第 7 章 ICP 的闭式旋转解就是用 $\boldsymbol{W}=\boldsymbol{U}\boldsymbol{\Sigma}\boldsymbol{V}^\top$ 的 SVD 得到的。

2.5 李群与李代数

旋转矩阵有 9 个元素，但只有 3 个自由度（正交约束 6 个）。这种“参数多于自由度”的表示没法直接做“加法更新”——$\boldsymbol{R}+\delta\boldsymbol{R}$ 一般不再是旋转矩阵。李群/李代数就是解决这个问题的工具。

2.5.1 SO(3) 与 $\mathfrak{so}(3)$

旋转矩阵构成群 SO(3)，它既是群又是光滑流形，所以叫李群。它在单位元处的切空间叫李代数 $\mathfrak{so}(3)$，由所有 3×3 反对称矩阵组成：

$$\mathfrak{so}(3)=\left\{\boldsymbol{\phi}^\wedge \;\middle|\; \boldsymbol{\phi}\in\mathbb{R}^3\right\},\qquad \boldsymbol{\phi}^\wedge=\begin{bmatrix}0&-\phi_3&\phi_2\\ \phi_3&0&-\phi_1\\ -\phi_2&\phi_1&0\end{bmatrix}. \tag{2.62}$$

符号 $(\cdot)^\wedge$ 表示“向量变反对称矩阵”，逆运算 $(\cdot)^\vee$ 表示“反对称矩阵变回向量”。

指数映射把李代数元素变成旋转矩阵：$\boldsymbol{R}=\exp(\boldsymbol{\phi}^\wedge)$。推导它的闭式表达（Rodrigues 公式）。把指数按泰勒级数展开：

$$\exp(\boldsymbol{\phi}^\wedge)=\boldsymbol{I}+\boldsymbol{\phi}^\wedge+\frac{1}{2!}(\boldsymbol{\phi}^\wedge)^2+\frac{1}{3!}(\boldsymbol{\phi}^\wedge)^3+\cdots \tag{2.63}$$

注意到 $\boldsymbol{\phi}^\wedge$ 有一个重要性质：若记 $\theta=\|\boldsymbol{\phi}\|$、$\boldsymbol{a}=\boldsymbol{\phi}/\theta$，则

$$(\boldsymbol{a}^\wedge)^2=\boldsymbol{a}\boldsymbol{a}^\top-\boldsymbol{I},\qquad (\boldsymbol{a}^\wedge)^3=-\boldsymbol{a}^\wedge. \tag{2.64}$$

第一个式子可以直接验证（对任意单位向量 $\boldsymbol{a}$）；第二个由第一个两边乘 $\boldsymbol{a}^\wedge$ 得到。于是奇数项和偶数项可以分别合并：

$$\begin{aligned}\exp(\theta\boldsymbol{a}^\wedge)&=\boldsymbol{I}+\left(\theta-\frac{\theta^3}{3!}+\frac{\theta^5}{5!}-\cdots\right)\boldsymbol{a}^\wedge+\left(\frac{\theta^2}{2!}-\frac{\theta^4}{4!}+\cdots\right)(\boldsymbol{a}^\wedge)^2\\&=\boldsymbol{I}+\sin\theta\,\boldsymbol{a}^\wedge+(1-\cos\theta)(\boldsymbol{a}^\wedge)^2.\end{aligned} \tag{2.65}$$

这就是 **Rodrigues 公式**。它说明：绕单位轴 $\boldsymbol{a}$ 旋转 θ 角的旋转矩阵只由 $\boldsymbol{\phi} = \theta\boldsymbol{a}$ 决定。反过来，给定 $\boldsymbol{R}$，也能求出 $\boldsymbol{\phi}$（对数映射）：

$$\theta = \arccos\left(\frac{\mathrm{tr}(\boldsymbol{R})-1}{2}\right), \qquad \boldsymbol{\phi} = \frac{\theta}{2\sin\theta}\left(\boldsymbol{R}-\boldsymbol{R}^{\top}\right)^{\vee}, \tag{2.66}$$

当 $\theta \to 0$ 或 $\theta \to \pi$ 时需要特殊处理（代码里通常都有分支）。

2.5.2 SE(3) 与 $\mathfrak{se}(3)$

把平移也纳入进来，SE(3) 的李代数 $\mathfrak{se}(3)$ 是 6 维的：

$$\mathfrak{se}(3) = \left\{ \boldsymbol{\xi}^{\wedge} \;\middle|\; \boldsymbol{\xi} = \begin{bmatrix}\boldsymbol{\rho}\\ \boldsymbol{\phi}\end{bmatrix} \in \mathbb{R}^6 \right\}, \qquad \boldsymbol{\xi}^{\wedge} = \begin{bmatrix}\boldsymbol{\phi}^{\wedge} & \boldsymbol{\rho}\\ \boldsymbol{0}^{\top} & 0\end{bmatrix}. \tag{2.67}$$

指数映射为

$$\exp(\boldsymbol{\xi}^{\wedge}) = \begin{bmatrix}\exp(\boldsymbol{\phi}^{\wedge}) & \boldsymbol{J}\boldsymbol{\rho}\\ \boldsymbol{0}^{\top} & 1\end{bmatrix}, \qquad \boldsymbol{J} = \boldsymbol{I} + \frac{1-\cos\theta}{\theta^2}\boldsymbol{\phi}^{\wedge} + \frac{\theta-\sin\theta}{\theta^3}(\boldsymbol{\phi}^{\wedge})^2, \tag{2.68}$$

其中 $\boldsymbol{J}$ 叫左雅可比。这里不要求会推导 $\boldsymbol{J}$，只要求知道“SE(3) 上的增量也是 6 维向量”。

2.5.3 扰动模型

在优化中，我们关心的是“给位姿加一个很小的增量”。有两种常见定义：

$$\text{左扰动：} \quad \boldsymbol{R} \leftarrow \exp(\delta\boldsymbol{\phi}^{\wedge})\boldsymbol{R}, \tag{2.69}$$

$$\text{右扰动：} \quad \boldsymbol{R} \leftarrow \boldsymbol{R}\exp(\delta\boldsymbol{\phi}^{\wedge}). \tag{2.70}$$

两种都对，区别在于雅可比的形式不同。本书约定：位姿用右扰动，即

$$\boldsymbol{R} \leftarrow \boldsymbol{R}\exp(\delta\boldsymbol{\phi}^{\wedge}), \qquad \boldsymbol{t} \leftarrow \boldsymbol{t} + \boldsymbol{R}\,\delta\boldsymbol{\rho}, \tag{2.71}$$

这样雅可比只和当前位姿有关，推导更整齐。注意 $\delta\boldsymbol{\rho}$ 是在机体系下定义的平移增量。

两个旋转复合的对数并不等于两个对数相加，而是有 BCH（Baker–Campbell–Hausdorff）公式：

$$\log\left(\exp(\boldsymbol{\phi}_1^{\wedge})\exp(\boldsymbol{\phi}_2^{\wedge})\right) \approx \begin{cases} \boldsymbol{J}_l(\boldsymbol{\phi}_2)^{-1}\boldsymbol{\phi}_1 + \boldsymbol{\phi}_2, & \text{左乘,}\\ \boldsymbol{J}_r(\boldsymbol{\phi}_1)^{-1}\boldsymbol{\phi}_2 + \boldsymbol{\phi}_1, & \text{右乘,}\end{cases} \tag{2.72}$$

当 $\boldsymbol{\phi}_2$ 很小时取一阶近似即可。这个公式出现在第 13 章位姿图残差的雅可比推导里。

2.5.4 为什么必须在流形上优化

如果直接对旋转矩阵的 9 个元素做梯度下降，更新后的矩阵不再满足 $\boldsymbol{R}^{\top}\boldsymbol{R} = \boldsymbol{I}$，需要每步重新正交化，既慢又不自然。正确做法是：**在切空间（李代数）里求增量，再通过指数映射回到流形：**

$$\boldsymbol{R}_{k+1} = \boldsymbol{R}_k \exp(\delta\boldsymbol{\phi}^{\wedge}), \tag{2.73}$$

其中 $\delta\boldsymbol{\phi} \in \mathbb{R}^3$ 是普通的、可以随便加减的向量。这样优化变量永远是 3 维的，约束自动满足。所有现代 SLAM 后端（g2o、GTSAM、Ceres 的 `LocalParameterization`）都是这么做的。

本章小结

1. 刚体变换是“旋转 + 平移”，写成 SE(3) 齐次矩阵后复合就是乘法，逆有闭式表达。
2. 期望是概率加权平均；方差是偏离期望的平方的期望，有 $\mathrm{Var}(X) = \mathbb{E}[X^2] - (\mathbb{E}[X])^2$、$\mathrm{Var}(aX+b) = a^2\,\mathrm{Var}(X)$、独立时方差可加三个常用性质。
3. 协方差矩阵的对角元是方差，非对角元是协方差；它的特征向量给出不确定性的主轴。
4. 高斯分布由均值与协方差完全确定；马氏距离的平方服从卡方分布，可用于外点剔除。

5. 贝叶斯公式把“先验 + 似然”变成“后验”，这是所有定位算法的出发点。
6. 非线性最小二乘 = 泰勒展开 + 正规方程；高斯–牛顿迭代求解，LM 加阻尼保证稳定。
7. 旋转不能直接相加，要在李代数里求增量再用指数映射更新；Rodrigues 公式给出指数映射的闭式解。

第二章 练习

1. 掷一枚均匀骰子，求 X 的期望、方差和标准差。若把每个结果乘以 2 再加 1，新的期望和方差是多少？
2. 证明 $\mathrm{Cov}(X,Y) = \mathbb{E}[XY] - \mathbb{E}[X]\mathbb{E}[Y]$，并说明为什么“协方差为零”不能推出“独立”。
3. 一维高斯分布中，若 $\mu = 2$、$\sigma = 3$，求 $P(2 \le X \le 5)$（提示：用标准正态分布表，$P(-1 \le Z \le 1) \approx 0.6827$）。
4. 对 $\boldsymbol{f}(\boldsymbol{x}) = \boldsymbol{R}\boldsymbol{x} + \boldsymbol{t}$，求雅可比 $\partial \boldsymbol{f} / \partial \boldsymbol{x}$；若 $\boldsymbol{R}$ 用增量 $\exp(\delta\boldsymbol{\phi}^\wedge)$ 扰动，求 $\partial \boldsymbol{f} / \partial \delta\boldsymbol{\phi}$。
5. 用 Rodrigues 公式计算绕 z 轴转 90° 的旋转矩阵，并与直觉核对。
6. 设 $\boldsymbol{A} = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$，求特征值、特征向量，并说明它对应的二次型椭球的半轴方向。

延伸阅读

- [1]：第 2–4 章系统讲解概率、估计与李群，适合作为本章的补充教材。
- [37]：用 50 多页把四元数、SO(3)、ESKF 讲透，是 IMU 状态估计必读。
- [38]：第 2 章讲贝叶斯滤波，第 6 章讲栅格地图，经典且清晰。
- 3Blue1Brown 的线性代数系列视频：直观理解特征值、SVD 和矩阵变换。

第三章 传感器与观测模型

本章导读

所有定位算法都建立在传感器之上。这一章把机器人常用的五类传感器——IMU、轮式编码器、激光雷达、相机、UWB/GNSS——逐个拆开：它们测什么、输出什么、噪声长什么样、在什么情况下会骗你。我们会为每类传感器建立“测量模型”，也就是“测量值 = 真值 + 零偏 + 噪声”这样的数学表达。后面所有估计器（卡尔曼滤波、因子图、ICP）都是在这个模型上做概率推断。

3.1 观测与测量模型

定义 3.1 (测量模型)

测量模型（measurement model）描述传感器读数与真实物理量之间的统计关系，通常写成

$$\boldsymbol{z} = \boldsymbol{h}(\boldsymbol{x}) + \boldsymbol{n}, \tag{3.1}$$

其中 $\boldsymbol{z}$ 是测量值，$\boldsymbol{x}$ 是待估计的真实状态，$\boldsymbol{h}(\cdot)$ 是已知的观测函数，$\boldsymbol{n}$ 是观测噪声。

观测噪声通常假设为零均值高斯：$\boldsymbol{n} \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{R})$，$\boldsymbol{R}$ 叫噪声协方差（在卡尔曼滤波里也记作测量噪声矩阵）。这个假设不是“真理”，而是工程上的折中：高斯分布只需要均值和方差两个数，且高斯经过线性变换后仍是高斯，这让滤波器的推导变得极其简洁。第 2 章已经证明，若 $\boldsymbol{n} \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{R})$，则负对数似然正比于马氏范数 $\|\boldsymbol{z} - \boldsymbol{h}(\boldsymbol{x})\|^2_{\boldsymbol{R}^{-1}}$——这正是最小二乘的来源。

为什么噪声协方差这么重要

残差同样大，协方差小的观测会被“更相信”，协方差大的会被“更怀疑”。调参时如果发现某个传感器“压不住”或“带偏”了融合结果，第一反应应该是检查它的协方差是否设得合理。

3.2 IMU：惯性测量单元

3.2.1 它测什么

IMU 通常由两部分组成：

- **陀螺仪**（gyroscope）：测量角速度 $\boldsymbol{\omega} = [\omega_x, \omega_y, \omega_z]^\top$，单位 rad/s 或 °/s。
- **加速度计**（accelerometer）：测量比力（specific force）$\boldsymbol{f}$，单位 m/s^2。注意它测的不是“加速度”，而是“加速度减去重力”，静止时读数是 ≈ 9.81 m/s^2 指向天空。

现代 IMU 多为 MEMS 工艺，体积小、成本低，但噪声大、零偏随温度漂移。工业级/战术级 IMU 精度高一个到几个数量级，价格也高一个到几个数量级。

3.2.2 测量模型

IMU 的测量模型是全书第一个正式的传感器模型，务必理解每一项：

$$\tilde{\boldsymbol{\omega}} = \boldsymbol{\omega} + \boldsymbol{b}_g + \boldsymbol{n}_g, \tag{3.2}$$

$$\tilde{\boldsymbol{a}} = \boldsymbol{R}_{WB}^\top (\boldsymbol{a}_W - \boldsymbol{g}) + \boldsymbol{b}_a + \boldsymbol{n}_a, \tag{3.3}$$

其中：

- $\tilde{\boldsymbol{\omega}}, \tilde{\boldsymbol{a}}$ 是**测量值**（带弯弯的波浪号在全书统一表示原始测量）；

- $\boldsymbol{\omega}, \boldsymbol{a}_W$ 是真实的角速度和世界系加速度；
- $\boldsymbol{b}_g, \boldsymbol{b}_a$ 是**零偏**（bias），即传感器在零输入时的非零输出，缓慢变化；
- $\boldsymbol{n}_g, \boldsymbol{n}_a$ 是零均值高斯白噪声；
- $\boldsymbol{g}$ 是重力向量，本书约定 $\boldsymbol{g} = [0, 0, -9.81]^\top$（世界系 z 轴向上）；
- $\boldsymbol{R}_{WB}^\top$ 把世界系向量转到机体系。

加速度计的“坑”

式 (3.3) 最容易被新手写错的地方是重力符号和参考系。静止时 $\boldsymbol{a}_W = \boldsymbol{0}$，测量值应为 $\tilde{\boldsymbol{a}} = -\boldsymbol{R}_{WB}^\top \boldsymbol{g}$。如果模拟数据里 IMU 读数模长不是 9.81 左右，或者静止时方向不对，先别怀疑算法，先检查模型实现。

零偏不是常数，而是缓慢漂移的随机过程。常用模型是**随机游走**：

$$\dot{\boldsymbol{b}}_g = \boldsymbol{n}_{bg}, \qquad \dot{\boldsymbol{b}}_a = \boldsymbol{n}_{ba}, \tag{3.4}$$

其中 $\boldsymbol{n}_{bg}, \boldsymbol{n}_{ba}$ 是零均值高斯白噪声。随机游走意味着零偏的方差随时间的增长而线性增长——这也是 IMU 不能长时间单独定姿的根本原因。

3.2.3 噪声参数与离散化

厂商给出的噪声密度通常是连续时间参数（如 $\sigma_g = 0.001$ rad/s/$\sqrt{\text{Hz}}$）。把它离散到采样周期 Δt 上，噪声标准差变为

$$\sigma_{g,d} = \frac{\sigma_g}{\sqrt{\Delta t}}, \qquad \sigma_{bg,d} = \sigma_{bg}\sqrt{\Delta t}. \tag{3.5}$$

为什么采样越快、单步噪声越大？因为连续白噪声的方差是无穷的，我们实际观测的是它在 Δt 上的积分；采样间隔越短，积分区间越小，但单位和归一化方式使得离散标准差与 $\sqrt{\Delta t}$ 成反比。第 6 章会从功率谱密度出发给出这个关系的完整推导。

3.2.4 关键指标

选购或评估 IMU 时关注这些量：

- **零偏稳定性**（bias instability）：零偏在长时间内的波动，单位 °/h 或 μg。越小越好。
- **角度随机游走**（ARW）：陀螺噪声，单位 °/$\sqrt{\text{h}}$。
- **速度随机游走**（VRW）：加速度计噪声。
- **量程**（range）：超过量程会**饱和**，读数被截断——这是 FAST-LIO 类算法在剧烈运动下的典型失效原因。
- **带宽**与采样率：常见 100–1000 Hz。

图 3.1 展示了 IMU 测量模型中各分量如何叠加成读数。

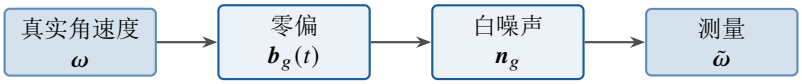


图 3.1: IMU 角速度的测量模型：真值叠加零偏与白噪声。

3.3 轮式编码器

3.3.1 原理与分辨率

增量式编码器装在电机或轮轴上，轮子每转一圈输出固定数量的脉冲。设编码器线数为 N（每圈 N 个脉冲）、减速比为 i、轮半径为 r，则轮子每转一圈，电机端产生 N 个脉冲，轮端产生 Ni 个脉冲。若采用四倍频计数（A/B

相正交，上升沿和下降沿都计数），单圈计数为 $4Ni$，于是每个计数对应的行进距离为

$$\Delta s = \frac{2\pi r}{4Ni}. \tag{3.6}$$

例：$N = 500$、$i = 20$、$r = 0.075$ m，则

$$\Delta s = \frac{2\pi \times 0.075}{4 \times 500 \times 20} \approx 5.9 \times 10^{-6}\ \text{m} \approx 5.9\ \mu\text{m}. \tag{3.7}$$

分辨率远高于实际精度——真正的误差来自轮子打滑、地面不平、轮胎磨损，而不是编码器本身。

3.3.2 输出

编码器本身只给出“转了多少”，需要结合时间得到轮速 $v = \Delta s/\Delta t$，再结合底盘运动学（第 5 章）得到车体线速度和角速度。ROS 2 中通常由底盘驱动节点发布 `nav_msgs/Odometry`，并广播 odom 到 base_link 的变换。

3.4 激光雷达

3.4.1 分类

按扫描方式：

- **机械旋转式**：电机带动激光模块旋转，360° 扫描，线数 16/32/64/128。代表：Velodyne、Ouster、Hesai。点云规则、每线一个 `ring` 编号。
- **半固态/固态**：MEMS 微振镜、转镜、OPA 等。视场角有限（如 120° × 90°），点云非重复。代表：Livox Mid-360、Avia。
- **面阵/Flash**：一次曝光成图，无扫描线。

按测距原理：dToF（直接飞行时间）、iToF（间接飞行时间）、FMCW（调频连续波，可测速、抗干扰）。这个细节对算法影响不大，知道即可。

3.4.2 关键参数

表 3.1: 激光雷达关键参数

参数	含义	对算法的影响
线数 / 视场角	垂直方向的分辨能力	影响点云密度与建图完整性
水平角分辨率	相邻两束激光的角度间隔	影响远处点云稀疏程度
测距范围与盲区	最近/最远有效距离	盲区内的障碍物“看不见”
点频	每秒输出点数	影响单帧点数与匹配耗时
精度	单点测距误差（cm 级）	影响点面残差噪声模型
回波模式	单回波/双回波	影响玻璃、雨雾场景

3.4.3 点云与运动畸变

定义 3.2 (点云)

点云是三维点的集合 $\mathcal{P} = \{\boldsymbol{p}_i\}_{i=1}^{N}$，每个点通常带有坐标、反射强度、时间戳，机械式雷达还带 `ring` 编号。♣

激光雷达不是“瞬间拍一张照片”，而是逐点扫描。一帧点云里，第一个点和最后一个点的采样时间可能相差一个扫描周期（10 Hz 雷达约 100 ms）。如果机器人在这一帧内运动了，点云就会“拖影”——同一个平面被扫成斜面。这叫**运动畸变**，必须用 IMU 或匀速模型做**去畸变**（第 16 章详述）。

Livox 的非重复扫描

Livox 系列采用非重复花瓣式扫描，单帧点云不形成规则的行列结构，不能直接套用“按 ring 处理”的算法。它的优势是同样点频下覆盖更均匀、近距离更密，适合 FAST-LIO 这类直接配准方法。代价是点云无序，很多传统特征提取方法需要改造。

3.5 相机

相机输出二维图像，信息丰富但缺少深度。按获取深度的方式分：

- **单目**：结构最简单，但尺度不可观（需要 IMU 或已知物体提供尺度）。
- **双目**：通过视差恢复深度，$d = \dfrac{f\,b}{Z}$（f 焦距、b 基线、Z 深度），远处精度迅速下降。
- **RGB-D**：结构光或 ToF 直接给深度，室内近距离好用，阳光下易失效。

内参与外参

内参（intrinsic）描述相机自身的成像几何，如焦距 f_x, f_y、主点 c_x, c_y、畸变系数。**外参**（extrinsic）描述相机与另一个坐标系（如 IMU、车体）之间的旋转和平移。两者都要标定，混用会直接导致投影错位。第 26 章给出完整标定流程。

相机在定位里的角色通常是“提供视觉约束”：特征点重投影误差（第 8 章）、光度误差、回环场景识别。纯视觉定位的优点是信息丰富、成本低；缺点是对光照、纹理、动态物体敏感，且单目有尺度问题。

3.6 其他传感器

- **UWB**（超宽带）：通过无线信号飞行时间测距，多个基站可解算三维位置。精度 10–30 cm，室内可用，但多径、遮挡、基站几何分布会显著影响精度。第 23 章会看到它如何作为先验地图缺失时的位置辅助。
- **GNSS**（卫星导航）：室外绝对定位，RTK 可达厘米级；室内、隧道、城市峡谷不可用。
- **二维码/ArUco**：视觉标签，精度高但需要布置，适合定点校准或小范围定位。
- **磁力计**：提供航向参考，易受电机、金属干扰，在机器人上通常只做辅助。
- **轮式触觉/IMU 之外的里程计**：如视觉里程计、雷达里程计，本质上都是“用其他传感器推算运动”，见第 4 章。

3.7 时间同步与坐标系

3.7.1 时间同步

定义 3.3 (时间同步)

时间同步是让不同传感器的数据带有同一时间基准的过程，包括：硬件触发（PPS、PTP）、软件时间戳对齐、以及估计并补偿传感器之间的固定延迟。

为什么重要？假设机器人以 2 m/s 运动，两个传感器时间戳相差 10 ms，位置差就是 $2 \times 0.01 = 2$ cm。对于厘米级定位，这个误差已经不可忽略；对点云配准，时间错位会让点云“错层”，残差永远降不下去。

工程上的检查清单：

1. 所有传感器的时间戳是否来自同一个时钟源（`/clock` 或系统时钟）？
2. 回放 bag 时是否加了`--clock`、所有节点是否设置了 `use_sim_time:=true`？
3. 雷达时间戳是“帧起始”还是“帧结束”？IMU 时间戳是否已做延迟补偿？
4. 外参标定文件中，平移和旋转的方向是否与公式一致？

3.7.2 外参

定义 3.4 (外参)

外参是传感器坐标系相对于机器人本体坐标系（或另一个传感器坐标系）的位姿，记作 $\boldsymbol{T}_{BL} = (\boldsymbol{R}_{BL}, {}^{B}\boldsymbol{p}_{L})$，表示“雷达系到机体系的变换”。

♣

外参错误的典型表现：点云与图像对不齐、地图出现双层墙、LIO 建图“越走越歪”、静止时点云漂移。所有外参都必须标定并在配置文件中正确填写（第 26 章）。

3.8 传感器选型对比

表 3.2: 常用传感器对比（数量级参考）

传感器	频率	精度	成本	主要失效场景
轮式编码器	50–100 Hz	1–10% 行程	极低	打滑、悬空、地面沉降
MEMS IMU	100–1000 Hz	姿态短时好	低	零偏漂移、饱和、振动
2D 激光	10–40 Hz	cm 级	中	无高度信息、玻璃、长走廊
3D 激光	10–20 Hz	cm 级	中高	雨雾、强光、稀疏远处
单目相机	20–60 Hz	尺度不可观	低	光照、纹理、动态物体
RGB-D	10–30 Hz	近距离 cm 级	中	阳光、远距离、反光
UWB	10–100 Hz	10–30 cm	中	多径、遮挡、基站几何差
GNSS-RTK	1–20 Hz	cm–m 级	高	室内、遮挡、多路径

这张表的用法不是“选最贵的”，而是“看清每种传感器的失效场景，用另一种去补它”。这也是第 9 章多传感器融合的出发点。

本章小结

1. 测量模型是“读数 = 真值 + 零偏 + 噪声”，高斯噪声假设让最小二乘与卡尔曼滤波得以成立。
2. IMU 测角速度与比力，零偏缓慢漂移，量程饱和是剧烈运动下的典型问题；离散噪声与 $\sqrt{\Delta t}$ 相关。
3. 编码器分辨率很高，但轮式里程计的真正误差来自打滑与地面。
4. 激光雷达逐点扫描产生运动畸变，机械式与固态的点云结构差异很大，算法选型要匹配。
5. 相机内参、外参、时间同步三者任何一个出错，都会表现为“算法不收敛”。
6. 没有完美的传感器，只有互补的组合。

第三章 练习

1. 静止时 IMU 加速度计读数应该是多少？如果模长是 9.5 或 10.2，可能是什么原因？
2. 一台 16 线雷达、水平角分辨率 0.2°，在 10 m 处的相邻两点间距约是多少？
3. 机器人以 1.5 m/s 运动，雷达与 IMU 时间戳相差 20 ms，会产生多大的位置误差？如果这个误差一直存在，点云配准会表现为什么现象？
4. 为什么单目相机有“尺度不可观”问题？双目为什么可以恢复尺度？请写出视差与深度的关系式并解释。
5. 列出你所用机器人上所有传感器，写出它们的时间戳来源和坐标系名称，画出 TF 树草图。

延伸阅读

- [38]：第 2–5 章详细讨论传感器模型与噪声。
- [37]：IMU 噪声模型与离散化的标准参考。

- [1]：第 6–7 章从估计理论角度讲传感器模型。
- 各传感器厂商数据手册：噪声密度、零偏稳定性、量程等参数的实际含义。

第二部分

里程计篇：机器人如何推算自己的位置

第四章 里程计总论：相对定位的艺术

本章导读

从本章开始，我们正式进入「机器人如何知道自己在哪」这个核心问题。本章先不碰任何具体传感器，只建立一张地图：什么叫位姿，什么叫位姿递推，为什么里程计天然是「相对」的，它的误差为什么注定越积越大，以及我们该用什么指标评价它。学完本章，你应当能回答三个问题：里程计在整个定位系统里处于什么位置；为什么所有里程计都会漂移、漂移的数学形状是什么；为什么评价里程计要看相对误差而不是绝对误差。后六章分别是本章四种里程计来源的展开。

4.1 为什么需要里程计

设想你在一间没有窗户的房间里闭着眼走路。你记得自己先向前迈了三步、再向右转九十度、又走了两步。凭这些「自己相对于自己的运动」，你能大致画出走过的轨迹，但你并不知道房间的绝对位置，也不知道自己的朝向与地理北方的夹角。更麻烦的是，每迈一步你都会有几厘米的估计误差，走得越久，累积的位置偏差越大。

机器人的处境完全相同，只是它可以靠传感器把「迈了几步」测得更准。定位问题通常被拆成两个不同的问题：

- **我在世界的哪里？**——需要地图或外部基准（GPS、已知信标、先验地图）。这类问题叫绝对定位，详见第 10 章。
- **我相对出发时动了多少？**——只需要传感器对自身运动的测量，连续、高频，不需要任何外部设施。这类问题叫相对定位，也就是本章的主角：里程计。

为什么不能只靠绝对定位？三个现实原因。其一是**频率不够**：GNSS 在室外通常只有 1–10 Hz，而差速底盘的控制回路常常要 50–100 Hz 的位姿反馈，用 1 Hz 的位姿闭环，控制器在两个更新之间只能靠「猜」。其二是**室内不可用**：RoboMaster 赛场是典型室内环境，GNSS 信号被建筑强烈衰减甚至完全遮挡。其三是**绝对定位会失败**：基于地图的定位在长廊、对称场地、动态障碍物遮挡时会短时失效，而控制与规划不能因为定位模块「卡了一下」就停车。

所以真实系统几乎总是「里程计 + 绝对定位修正」：里程计提供高频、平滑、局部一致的位姿，绝对定位（或 SLAM 后端、回环检测）以低频把它拉回全局正确的位置——里程计负责「连续」，绝对定位负责「不漂」。

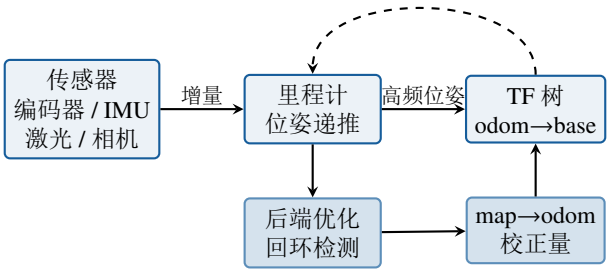


图 4.1: 里程计在定位系统中的位置。里程计把传感器增量递推成高频位姿，直接发布 `odom→base` 变换；后端优化低频地给出 `map→odom` 的修正量，两者叠加才得到全局位姿。

4.2 位姿、绝对定位与相对定位

先把最基础的对象说清楚。全书统一使用如下坐标系约定（见第 2 章）：世界系 W 的 z 轴向上（ENU 式），机体系 B 与 IMU 系重合，重力 $\boldsymbol{g} = [0, 0, -9.81]^{\top}$。

定义 4.1 (位姿)

机器人机体系 B 在世界系 W 中的位姿（pose）是一个二元组

$$T_{WB} = \left(\boldsymbol{R}_{WB},\ {}^{W}\boldsymbol{p}_{B}\right), \tag{4.1}$$

其中 $\boldsymbol{R}_{WB} \in SO(3)$ 是旋转矩阵，${}^{W}\boldsymbol{p}_{B} \in \mathbb{R}^3$ 是机体原点在世界系中的位置。它作用在任意一点上：

$${}^{W}\boldsymbol{q} = \boldsymbol{R}_{WB}\,{}^{B}\boldsymbol{q} + {}^{W}\boldsymbol{p}_{B}. \tag{4.2}$$

所有满足式 (4.2) 的 $(\boldsymbol{R}, \boldsymbol{p})$ 构成特殊欧氏群 $SE(3)$，即 $T_{WB} \in SE(3)$。

定义 4.2 (绝对定位与相对定位)

绝对定位（absolute localization）指估计机体在世界系 W 中的位姿 T_{WB}，其结果是相对某个固定原点而言的；相对定位（relative localization）指只估计两个时刻之间的位姿差 $T_k^{-1}T_{k+1}$，其结果是相对自身历史而言的。

定义 4.3 (里程计与位姿递推)

里程计（odometry）是利用传感器测得的相邻时刻运动增量，从一个已知初始位姿出发，递推地累加出连续位姿序列的算法。把增量复合回历史位姿的过程称为位姿递推（pose propagation）：

$$T_{k+1} = T_k\,\Delta T_k, \tag{4.3}$$

其中 $\Delta T_k \in SE(3)$ 是第 k 步的运动增量。

递推式 (4.3) 的合理性来自位姿复合。设 T_{AB} 表示「从 B 系到 A 系」的变换，则连续复合满足

$$T_{AC} = T_{AB}\,T_{BC}. \tag{4.4}$$

把式 (4.2) 代入验证：对任意点 ${}^{C}\boldsymbol{q}$，有

$$\begin{aligned} {}^{A}\boldsymbol{q} &= \boldsymbol{R}_{AB}\left(\boldsymbol{R}_{BC}\,{}^{C}\boldsymbol{q} + {}^{B}\boldsymbol{p}_{C}\right) + {}^{A}\boldsymbol{p}_{B} \\ &= (\boldsymbol{R}_{AB}\boldsymbol{R}_{BC})\,{}^{C}\boldsymbol{q} + \left(\boldsymbol{R}_{AB}\,{}^{B}\boldsymbol{p}_{C} + {}^{A}\boldsymbol{p}_{B}\right). \end{aligned} \tag{4.5}$$

因此复合后的旋转与平移分别是

$$\boldsymbol{R}_{AC} = \boldsymbol{R}_{AB}\boldsymbol{R}_{BC}, \qquad {}^{A}\boldsymbol{p}_{C} = \boldsymbol{R}_{AB}\,{}^{B}\boldsymbol{p}_{C} + {}^{A}\boldsymbol{p}_{B}. \tag{4.6}$$

逆变换同样可以逐项写出：在式 (4.2) 两边左乘 $\boldsymbol{R}_{WB}^{\top}$，得

$${}^{B}\boldsymbol{q} = \boldsymbol{R}_{WB}^{\top}\,{}^{W}\boldsymbol{q} - \boldsymbol{R}_{WB}^{\top}\,{}^{W}\boldsymbol{p}_{B}, \tag{4.7}$$

即

$$T_{BW} = \left(\boldsymbol{R}_{WB}^{\top},\ -\boldsymbol{R}_{WB}^{\top}\,{}^{W}\boldsymbol{p}_{B}\right). \tag{4.8}$$

注意式 (4.8) 中平移不是简单取负，而是「转回本体系再取负」——新手最常见的错误之一。

4.3 位姿递推与指数映射

里程计输出的原始量通常是速度或加速度。例如轮式底盘给出线速度 v 与角速度 ω，IMU 给出角速度与比力。要把「速度乘时间」变成一个位姿增量 $\Delta T_k \in SE(3)$，就需要指数映射。

设第 k 步内机体在本体系下的速度与角速度为常值，记为 $\boldsymbol{v}_k, \boldsymbol{\omega}_k \in \mathbb{R}^3$，时间间隔 Δt。定义六维运动量

$$\boldsymbol{\xi}_k = \begin{bmatrix} \boldsymbol{\rho}_k \\ \boldsymbol{\phi}_k \end{bmatrix} = \begin{bmatrix} \boldsymbol{v}_k \Delta t \\ \boldsymbol{\omega}_k \Delta t \end{bmatrix} \in \mathbb{R}^6. \tag{4.9}$$

则位姿增量为指数映射

$$\Delta T_k = \exp\left(\boldsymbol{\xi}_k^{\wedge}\right), \qquad \boldsymbol{\xi}^{\wedge} = \begin{bmatrix} \boldsymbol{\phi}^{\wedge} & \boldsymbol{\rho} \\ \mathbf{0}^{\top} & 0 \end{bmatrix}. \tag{4.10}$$

下面把 exp 的每一块完整推出来。先定义反对称矩阵算子：对 $\boldsymbol{\omega} = [\omega_1, \omega_2, \omega_3]^{\top}$，

$$\boldsymbol{\omega}^{\wedge} = \begin{bmatrix} 0 & -\omega_3 & \omega_2 \\ \omega_3 & 0 & -\omega_1 \\ -\omega_2 & \omega_1 & 0 \end{bmatrix}, \qquad \boldsymbol{\omega}^{\wedge}\boldsymbol{x} = \boldsymbol{\omega} \times \boldsymbol{x}. \tag{4.11}$$

它的关键性质是 $\boldsymbol{\omega}^{\wedge\top} = -\boldsymbol{\omega}^{\wedge}$（反对称）。

第一步：旋转矩阵的微分方程。设机体以常值角速度 $\boldsymbol{\omega}$ 旋转，任意一个固连在机体上的向量 ${}^{B}\boldsymbol{q}$ 在世界系中的像为 ${}^{W}\boldsymbol{q}(t) = \boldsymbol{R}(t){}^{B}\boldsymbol{q}$。由于该向量在机体系中不变，有

$$\dot{}{}^{W}\boldsymbol{q}(t) = \boldsymbol{R}(t)\left(\boldsymbol{\omega} \times {}^{B}\boldsymbol{q}\right) = \boldsymbol{R}(t)\boldsymbol{\omega}^{\wedge}{}^{B}\boldsymbol{q}. \tag{4.12}$$

另一方面直接对 ${}^{W}\boldsymbol{q} = \boldsymbol{R}\,{}^{B}\boldsymbol{q}$ 求导得 $\dot{}{}^{W}\boldsymbol{q} = \dot{\boldsymbol{R}}{}^{B}\boldsymbol{q}$。比较两式，并注意到 ${}^{B}\boldsymbol{q}$ 任意，得到

$$\dot{\boldsymbol{R}} = \boldsymbol{R}\,\boldsymbol{\omega}^{\wedge}. \tag{4.13}$$

第二步：把解写成级数。常系数线性微分方程 (4.13) 在初值 $\boldsymbol{R}(0) = \boldsymbol{I}$ 下的解可以用幂级数表示。把 $\boldsymbol{R}(t) = \sum_{n\geq 0} \frac{t^n}{n!}\boldsymbol{A}^n$ 代入式 (4.13) 逐项求导（左端 $\sum \frac{t^{n-1}}{(n-1)!}\boldsymbol{A}^n$，右端 $\sum \frac{t^n}{n!}\boldsymbol{A}^n\boldsymbol{\omega}^{\wedge}$），比较 t^n 的系数即知取 $\boldsymbol{A} = \boldsymbol{\omega}^{\wedge}$ 时逐项相等，因此

$$\boldsymbol{R}(t) = \exp\left(t\boldsymbol{\omega}^{\wedge}\right) = \sum_{n=0}^{\infty} \frac{t^n}{n!}\left(\boldsymbol{\omega}^{\wedge}\right)^n. \tag{4.14}$$

第三步：化简级数。令 $\boldsymbol{\phi} = \boldsymbol{\omega}t$，$\theta = \|\boldsymbol{\phi}\|$，$\boldsymbol{u} = \boldsymbol{\phi}/\theta$（当 $\theta \neq 0$）。先求单位轴幂次的闭式。取 $\boldsymbol{u} = \boldsymbol{e}_3 = [0,0,1]^{\top}$，由式 (4.11) 直接算出

$$\boldsymbol{e}_3^{\wedge} = \begin{bmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}, \qquad \left(\boldsymbol{e}_3^{\wedge}\right)^2 = \begin{bmatrix} -1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 0 \end{bmatrix} = \boldsymbol{e}_3\boldsymbol{e}_3^{\top} - \boldsymbol{I}. \tag{4.15}$$

再用它算三次幂：

$$\left(\boldsymbol{e}_3^{\wedge}\right)^3 = \left(\boldsymbol{e}_3^{\wedge}\right)^2\boldsymbol{e}_3^{\wedge} = \begin{bmatrix} -1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 0 \end{bmatrix}\begin{bmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 \\ -1 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} = -\boldsymbol{e}_3^{\wedge}. \tag{4.16}$$

对任意单位轴 $\boldsymbol{u}$，存在旋转矩阵 $\boldsymbol{Q}$ 使 $\boldsymbol{u} = \boldsymbol{Q}\boldsymbol{e}_3$，而 $\boldsymbol{u}^{\wedge} = \boldsymbol{Q}\boldsymbol{e}_3^{\wedge}\boldsymbol{Q}^{\top}$，故上述两条关系在一般轴下同样成立：

$$\left(\boldsymbol{u}^{\wedge}\right)^2 = \boldsymbol{u}\boldsymbol{u}^{\top} - \boldsymbol{I}, \qquad \left(\boldsymbol{u}^{\wedge}\right)^3 = -\boldsymbol{u}^{\wedge}. \tag{4.17}$$

把式 (4.17) 反复使用，任意整数次幂都可降到 $\boldsymbol{I}$、$\boldsymbol{u}^{\wedge}$、$(\boldsymbol{u}^{\wedge})^2$ 三档。代入式 (4.14) 并按这三档重新分组（设 $\boldsymbol{\phi} = \theta\boldsymbol{u}$）：

$$\begin{aligned} \exp\left(\boldsymbol{\phi}^{\wedge}\right) &= \boldsymbol{I} + \left(\sum_{j=0}^{\infty} \frac{(-1)^j\theta^{2j+1}}{(2j+1)!}\right)\boldsymbol{u}^{\wedge} + \left(\sum_{j=1}^{\infty} \frac{(-1)^{j+1}\theta^{2j}}{(2j)!}\right)\left(\boldsymbol{u}^{\wedge}\right)^2 \\ &= \boldsymbol{I} + \sin\theta\,\boldsymbol{u}^{\wedge} + (1-\cos\theta)\left(\boldsymbol{u}^{\wedge}\right)^2. \end{aligned} \tag{4.18}$$

其中第一个括号正是 $\sin\theta$ 的泰勒级数，第二个括号是 $1-\cos\theta$ 的泰勒级数。式 (4.18) 就是 **Rodrigues 公式**（李群指数映射的系统推导见 [1, 37]）。把 $\boldsymbol{u}^{\wedge}$ 与 $(\boldsymbol{u}^{\wedge})^2$ 按分量展开，令 $c = \cos\theta$、$s = \sin\theta$、$\boldsymbol{u} = [u_x, u_y, u_z]^{\top}$：

$$\boldsymbol{R} = \begin{bmatrix} c + u_x^2(1-c) & u_xu_y(1-c) - u_zs & u_xu_z(1-c) + u_ys \\ u_yu_x(1-c) + u_zs & c + u_y^2(1-c) & u_yu_z(1-c) - u_xs \\ u_zu_x(1-c) - u_ys & u_zu_y(1-c) + u_xs & c + u_z^2(1-c) \end{bmatrix}. \tag{4.19}$$

第四步：小角度近似。当 $\theta \to 0$ 时，$\sin\theta = \theta - \theta^3/6 + O(\theta^5)$，$1-\cos\theta = \theta^2/2 + O(\theta^4)$，因此

$$\exp\left(\boldsymbol{\phi}^{\wedge}\right) = \boldsymbol{I} + \boldsymbol{\phi}^{\wedge} + \frac{1}{2}\left(\boldsymbol{\phi}^{\wedge}\right)^2 + O(\theta^3), \qquad \boldsymbol{R} \approx \boldsymbol{I} + \boldsymbol{\phi}^{\wedge} \quad (\theta \ll 1). \tag{4.20}$$

这就是「一帧内转角很小」时的线性化，它是后面所有滤波与优化推导的核心近似。

第五步：平移部分。对 $SE(3)$ 的指数映射，把式 (4.10) 写成块形式并求解微分方程。设 $T(t)=\exp(t\boldsymbol{\xi}^\wedge)$，其块形式为

$$T(t)=\begin{bmatrix}\boldsymbol{R}(t) & \boldsymbol{p}(t)\\ \boldsymbol{0}^\top & 1\end{bmatrix}, \tag{4.21}$$

两边对 t 求导并利用 $T(t)$ 与 $\boldsymbol{\xi}^\wedge$ 交换，得到 $\dot{\boldsymbol{R}}=\boldsymbol{R}\boldsymbol{\phi}^\wedge$ 与 $\dot{\boldsymbol{p}}=\boldsymbol{R}\boldsymbol{\rho}$。第一式已由式 (4.18) 解出。第二式两边积分：

$$\boldsymbol{p}(t)=\int_0^t \boldsymbol{R}(s)\boldsymbol{\rho}\,\mathrm{d}s=\int_0^t\left[\boldsymbol{I}+\frac{\sin(s\theta)}{\theta}\boldsymbol{\phi}^\wedge+\frac{1-\cos(s\theta)}{\theta^2}\left(\boldsymbol{\phi}^\wedge\right)^2\right]\boldsymbol{\rho}\,\mathrm{d}s. \tag{4.22}$$

逐项积分，注意 $\int_0^t \sin(s\theta)\,\mathrm{d}s=(1-\cos(t\theta))/\theta$ 与 $\int_0^t(1-\cos(s\theta))\,\mathrm{d}s=t-\sin(t\theta)/\theta$，并记 $t=1$（$\boldsymbol{\xi}$ 已包含时间），得到

$$\boldsymbol{p}=\boldsymbol{J}\boldsymbol{\rho},\qquad \boldsymbol{J}=\boldsymbol{I}+\frac{1-\cos\theta}{\theta^2}\boldsymbol{\phi}^\wedge+\frac{\theta-\sin\theta}{\theta^3}\left(\boldsymbol{\phi}^\wedge\right)^2. \tag{4.23}$$

于是完整的 $SE(3)$ 指数映射为

$$\exp\left(\boldsymbol{\xi}^\wedge\right)=\begin{bmatrix}\exp\left(\boldsymbol{\phi}^\wedge\right) & \boldsymbol{J}\boldsymbol{\rho}\\ \boldsymbol{0}^\top & 1\end{bmatrix}. \tag{4.24}$$

当 $\|\boldsymbol{\phi}\|$ 很小时，$\boldsymbol{J}\approx\boldsymbol{I}$，式 (4.24) 退化为

$$\Delta T\approx\begin{bmatrix}\boldsymbol{I}+\boldsymbol{\phi}^\wedge & \boldsymbol{\rho}\\ \boldsymbol{0}^\top & 1\end{bmatrix}=\boldsymbol{I}+\boldsymbol{\xi}^\wedge, \tag{4.25}$$

这正好对上式 (4.3) 的直觉：$\Delta\boldsymbol{R}\approx\boldsymbol{I}+\boldsymbol{\omega}^\wedge\Delta t$，$\Delta\boldsymbol{p}\approx\boldsymbol{v}\Delta t$。

例题 4.1 一次增量到底转了多少 设 $\boldsymbol{\omega}=[0,0,1]^\top$ rad/s，$\Delta t=0.1$ s，则 $\boldsymbol{\phi}=[0,0,0.1]^\top$，$\theta=0.1$ rad，转轴 $\boldsymbol{u}=\boldsymbol{e}_3$。由式 (4.18)，$\sin 0.1=0.099833$，$\cos 0.1=0.995004$，故

$$\Delta\boldsymbol{R}=\begin{bmatrix}0.995004 & -0.099833 & 0\\ 0.099833 & 0.995004 & 0\\ 0 & 0 & 1\end{bmatrix}\approx\begin{bmatrix}0.995 & -0.0998 & 0\\ 0.0998 & 0.995 & 0\\ 0 & 0 & 1\end{bmatrix}, \tag{4.26}$$

与一阶近似 $\boldsymbol{I}+0.1\boldsymbol{e}_3^\wedge$ 相比，对角元差约 5×10^{-6}，在这个尺度上完全可以忽略。若同时有 $\boldsymbol{v}=[1,0,0]^\top$ m/s，则位移增量是 $\boldsymbol{J}\boldsymbol{\rho}\approx[0.1,0,0]^\top$ m，误差来自 $\boldsymbol{J}$ 的高阶项，量级约 10^{-5} m。

指数映射的三条实用结论

1. $\exp(\boldsymbol{\xi}^\wedge)$ 把「本体系下的速度 × 时间」统一变成 $SE(3)$ 增量，旋转和平移可以一起处理；
2. 小角度时 $\exp(\boldsymbol{\phi}^\wedge)\approx\boldsymbol{I}+\boldsymbol{\phi}^\wedge$，误差量级 $O(\theta^2)$，一般 $\theta<0.1$ rad（约 5.7°）时工程上可接受；
3. 指数映射与矩阵乘法不可交换：$\exp(\boldsymbol{\xi}_1^\wedge)\exp(\boldsymbol{\xi}_2^\wedge)\neq\exp((\boldsymbol{\xi}_1+\boldsymbol{\xi}_2)^\wedge)$。因此位姿递推必须做矩阵（或四元数）乘法，不能直接把增量相加。

4.4 里程计的四种来源

同一件事——估计本体的运动——可以用完全不同的物理量来测量。表 4.1 给出四种主流方案的数量级对比。记住这张表比记住任何公式都有用：它决定了你在什么场景该信谁。

表 4.1: 四类里程计的数量级对比（典型值，随器件与场景变化）

来源	典型频率	直接观测量	相对精度（短期）	主要失效场景
轮式编码器	50–100 Hz	轮转角	行程的 1–10 %	打滑、悬空、地面沉降
IMU	100–1000 Hz	角速度、比力	姿态 0.1–1°，位置漂移极快	零偏、振动、温漂
激光雷达	5–20 Hz	点云	行程的 0.1–1 %	长廊、空旷、无结构
视觉相机	20–60 Hz	图像	行程的 0.5–5 %	弱纹理、强光、动态物体

轮式（第 5 章）最便宜、频率最高、短时最稳，但它是「假设轮子不滑」的里程计：轮子一打滑，它测的就

是轮子的转数而非车体的运动，误差立刻变成系统性偏差。**IMU**（第 6 章）不依赖外部环境，短时角速度精度极高，是姿态估计的主力；但加速度要积分两次才得到位置，零偏误差随时间平方增长。**激光雷达**（第 7 章）深度精度在厘米级，点云配准能给出很准的帧间位姿，是 FAST-LIO 系列的基础，代价是数据量大、对几何结构敏感。**视觉**（第 8 章）信息量最丰富，但受光照、纹理、动态物体影响，单目还有尺度不可观的问题。四种来源强弱互补，这正是第 9 章要讲的融合动机。

4.5 累积误差与漂移

里程计最本质的缺陷是：它把每一步的误差都攒了下来。本节把这个「攒」字算清楚。

第一步：复习期望与方差。（定义见第 2 章。）设 X 是连续型随机变量，密度为 $p(x)$，则

$$\mathbb{E}[X] = \int x\, p(x)\, \mathrm{d}x, \qquad \mathrm{Var}(X) = \mathbb{E}\left[(X - \mathbb{E}[X])^2\right] = \mathbb{E}[X^2] - (\mathbb{E}[X])^2 . \tag{4.27}$$

方差衡量 X 围绕均值的散布程度；它的平方根 $\sigma = \sqrt{\mathrm{Var}(X)}$ 叫标准差，与 X 同量纲。

第二步：两个独立变量之和的方差。由式 (4.27)，

$$\begin{aligned}
\mathrm{Var}(X+Y) &= \mathbb{E}\left[(X+Y)^2\right] - (\mathbb{E}[X+Y])^2 \\
&= \mathbb{E}[X^2] + 2\mathbb{E}[XY] + \mathbb{E}[Y^2] - \left(\mathbb{E}[X]^2 + 2\mathbb{E}[X]\mathbb{E}[Y] + \mathbb{E}[Y]^2\right) \\
&= \left(\mathbb{E}[X^2] - \mathbb{E}[X]^2\right) + \left(\mathbb{E}[Y^2] - \mathbb{E}[Y]^2\right) + 2\left(\mathbb{E}[XY] - \mathbb{E}[X]\mathbb{E}[Y]\right) \\
&= \mathrm{Var}(X) + \mathrm{Var}(Y) + 2\,\mathrm{Cov}(X,Y).
\end{aligned} \tag{4.28}$$

若 X 与 Y 独立，则 $\mathbb{E}[XY] = \mathbb{E}[X]\mathbb{E}[Y]$，协方差项为零，于是

$$\mathrm{Var}(X+Y) = \mathrm{Var}(X) + \mathrm{Var}(Y). \tag{4.29}$$

第三步：n 步的累积误差。设第 i 步的位置误差为 ϵ_i，各步相互独立、同分布，均值为零、方差为 σ^2。n 步后的总误差为 $E_n = \sum_{i=1}^{n} \epsilon_i$。对式 (4.29) 反复使用 $n-1$ 次（每加一项只用一次两变量结论），得

$$\mathrm{Var}(E_n) = \sum_{i=1}^{n} \mathrm{Var}(\epsilon_i) = n\sigma^2, \qquad \sigma_n = \sqrt{n}\,\sigma. \tag{4.30}$$

这就是全书反复出现的**随机游走**（random walk）结论：误差的方差随步数线性增长，标准差随步数按 $\sqrt{n}$ 增长。

第四步：换成时间与距离。设里程计频率为 $f = 1/\Delta t$，运行时间 t，则 $n = ft$，于是 $\sigma(t) = \sigma\sqrt{ft}$：它不是随时间线性增长，而是随时间开方增长，这一点常常被误解。但如果每步误差有非零均值 b（称为**系统误差**或**零偏**），则

$$\mathbb{E}[E_n] = \sum_{i=1}^{n} \mathbb{E}[\epsilon_i] = nb, \tag{4.31}$$

偏差随时间线性增长，是比随机游走更危险的一类误差。真实里程计是两者叠加：随时间一长，系统误差必然主导。

例题 4.2 一步一厘米，一百秒后差多少 设 $\Delta t = 0.1\,\mathrm{s}$，每步位置误差标准差 $\sigma = 0.01\,\mathrm{m}$，各步独立。运行 $t = 100\,\mathrm{s}$ 共 $n = 1000$ 步，则 $\sigma_n = \sqrt{1000} \times 0.01 \approx 0.316\,\mathrm{m}$。若同样的误差里还藏着 $b = 0.5\,\mathrm{mm}$ 的均值偏差，则 1000 步后平均偏差是 $1000 \times 0.0005 = 0.5\,\mathrm{m}$，反而比随机部分更大。

第五步：漂移率。为了横向比较不同里程计，定义**漂移率**（drift rate）

$$\rho_{\mathrm{drift}} = \frac{\|\hat{\boldsymbol{p}}(t) - \boldsymbol{p}(t)\|_{\mathrm{RMS}}}{L(t)}, \tag{4.32}$$

即位置误差的均方根除以累计行程 $L(t)$，常用百分数表示。轮式里程计典型值 1–10 %，激光/视觉 0.1–1 %。注意漂移率是「误差/距离」而不是「误差/时间」，因为大多数里程计误差主要来自走过的距离与转弯次数，而不是纯粹的时间。

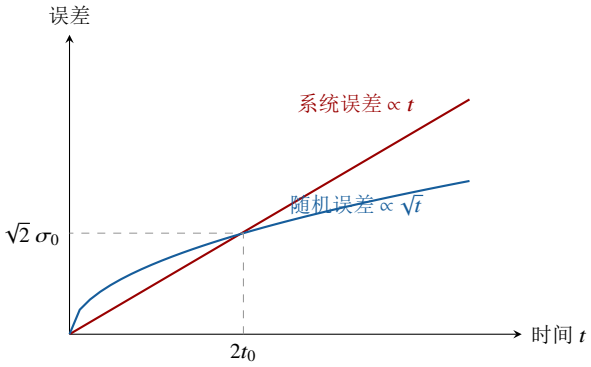


图 4.2: 两类误差的增长形状。随机误差按 $\sqrt{t}$ 增长（蓝线），系统误差按 t 线性增长（红线）。长时间运行后，系统的零偏一定是主要矛盾，这也是为什么所有里程计都需要外部修正。

4.6 里程计的评价

评价里程计不能用「我觉得挺准」这种描述，工程上使用两个标准指标（工具链见第 28 章，这里只给定义）。

绝对轨迹误差 ATE。设估计位姿序列为 $\{\hat{T}_i\}_{i=1}^N$，真值序列为 $\{T_i\}$。先求一个全局对齐变换 $\boldsymbol{S} \in SE(3)$（通常是第一帧对齐或 Umeyama 对齐，消除初始位姿的任意性），定义第 i 个误差

$$\boldsymbol{E}_i = T_i^{-1}\, \boldsymbol{S}\, \hat{T}_i \in SE(3). \tag{4.33}$$

绝对轨迹误差（absolute trajectory error, ATE）定义为平移部分的均方根：

$$\mathrm{ATE}_{\mathrm{RMSE}} = \sqrt{\frac{1}{N}\sum_{i=1}^{N} \left\| \mathrm{trans}\left(\boldsymbol{E}_i\right) \right\|^2}, \tag{4.34}$$

其中 $\mathrm{trans}(\cdot)$ 表示取 $SE(3)$ 的平移分量。

相对位姿误差 RPE。ATE 会随着轨迹变长而系统性变大，不适合单独评价「里程计本身」。相对位姿误差（relative pose error, RPE）只比较固定间隔 Δ 内的相对运动：

$$\boldsymbol{E}_i = \left(T_i^{-1} T_{i+\Delta}\right)^{-1} \left(\hat{T}_i^{-1} \hat{T}_{i+\Delta}\right), \tag{4.35}$$

再对平移部分取 RMSE 得到 $\mathrm{RPE}_{\mathrm{RMSE}}$；对旋转部分取 SO(3) 上的角度误差 RMSE 得到 $\mathrm{RPE}_{\mathrm{rot}}$。

为什么里程计要特别看 RPE？因为里程计的输出本来就是一系列相对增量，它没有全局原点；完美的里程计应当「任意两帧之间的相对变换都准」，式 (4.35) 恰好把全局漂移消掉，只留下局部一致性。

ATE 与 RPE 的使用场景

- **ATE**：评价「整个系统」——把里程计 + 回环 + 后端作为一个黑箱，看最终轨迹与真值的偏离。SLAM 论文一般报 ATE。
- **RPE**：评价「里程计本身」——只看局部相对运动，不受累计漂移干扰。调里程计参数（如 ICP 阈值、噪声协方差）时应多看 RPE。
- 两者必须一起看：ATE 小可能是回环在「救场」，掩盖了里程计本身很差；RPE 小但 ATE 大，说明里程计准但存在未修正的长期漂移。

4.7 工程视角：里程计模块的输入输出

不管底层用哪种传感器，一个里程计节点在工程上的接口几乎是固定的：输入带时间戳的传感器测量与外参 T_{SB}，输出每个时刻的位姿 T_{OB}（O 为里程计系，连续、无跳变）及协方差，并约定 $T_{OB}(t_0) = \boldsymbol{I}$。全局修正由 `map→odom` 这个额外的变换承担，绝不直接改里程计输出——这样短期控制器拿到的位姿不会因为一次回环而突变。

算法：通用里程计递推循环

1. 初始化：$T_{OB} \leftarrow \boldsymbol{I}$，$t \leftarrow t_0$
2. 当收到时间戳 t_k 的测量 z_k 时：
3. 　计算时间间隔 $\Delta t \leftarrow t_k - t_{k-1}$
4. 　由 z_k 得到本体系速度估计 $(\hat{v}_k, \hat{\omega}_k)$
5. 　构造增量 $\boldsymbol{\xi}_k \leftarrow [\hat{v}_k \Delta t;\ \hat{\omega}_k \Delta t]$
6. 　$T_{OB} \leftarrow T_{OB} \exp(\boldsymbol{\xi}_k^{\wedge})$
7. 　按式 (4.30) 同款逻辑传播协方差
8. 　发布 T_{OB}、协方差与 `odom→base` 变换

本章小结

1. 定位问题分为绝对定位与相对定位；里程计解决相对定位，特点是高频、连续、局部一致，但会累积漂移。
2. 位姿 $T_{WB} = (\boldsymbol{R}_{WB}, {}^{W}\boldsymbol{p}_B)$ 属于 $SE(3)$；复合 $T_{AC} = T_{AB}T_{BC}$ 对应旋转相乘、平移按「先转再加」；逆变换的平移是 $-\boldsymbol{R}^{\top}\boldsymbol{p}$ 而不是 $-\boldsymbol{p}$。
3. 位姿递推是 $T_{k+1} = T_k \Delta T_k$，增量由指数映射给出：$\Delta T_k = \exp(\boldsymbol{\xi}_k^{\wedge})$；SO(3) 的闭式是 Rodrigues 公式 $\boldsymbol{R} = \boldsymbol{I} + \sin\theta\, \boldsymbol{u}^{\wedge} + (1-\cos\theta)(\boldsymbol{u}^{\wedge})^2$，小角度时 $\boldsymbol{R} \approx \boldsymbol{I} + \boldsymbol{\phi}^{\wedge}$。
4. $SE(3)$ 指数映射为 $\exp(\boldsymbol{\xi}^{\wedge}) = [\exp(\boldsymbol{\phi}^{\wedge}), \boldsymbol{J}\boldsymbol{\rho}; \boldsymbol{0}^{\top}, 1]$，其中 $\boldsymbol{J}$ 由式 (4.23) 给出，小角度时 $\boldsymbol{J} \approx \boldsymbol{I}$。
5. 四种里程计来源各有短长：轮式便宜但怕打滑，IMU 高频但位置漂移快，激光/视觉几何精度高但怕退化与光照。
6. 独立同分布误差的方差相加：$\mathrm{Var}(\sum \epsilon_i) = n\sigma^2$，标准差按 $\sqrt{n}$ 增长；有零偏时均值按 n 线性增长，长时间后前者必被后者超过。
7. 评价里程计用 ATE 与 RPE，前者看整体、后者看局部一致性；工程上里程计输出接在 `odom→base`，全局修正接在 `map→odom`。

第四章 练习

1. 设初始位姿 $T_{WB} = (\boldsymbol{R}_{WB}, {}^{W}\boldsymbol{p}_B)$，验证式 (4.4) 的逆变换为 $T_{BW} = (\boldsymbol{R}_{WB}^{\top}, -\boldsymbol{R}_{WB}^{\top}{}^{W}\boldsymbol{p}_B)$，并用一个具体的二维例子（$\boldsymbol{R}$ 为 90° 旋转，${}^{W}\boldsymbol{p}_B = [1,0,0]^{\top}$）算一遍。
2. 用 Rodrigues 公式计算 $\boldsymbol{\omega} = [0, 0, \pi/2]^{\top}$ rad/s、$\Delta t = 1$ s 时的 $\Delta\boldsymbol{R}$，并与 $\boldsymbol{I} + \boldsymbol{\phi}^{\wedge}$ 比较误差的量级。
3. 证明：若每步误差独立且标准差为 σ，走 n 步后的位置标准差为 $\sigma\sqrt{n}$；若每步误差都带同样大小的均值 b，则期望误差为 nb。当 n 从 10^2 增到 10^4 时，两者比例如何变化？
4. 某轮式里程计在 10 m 直线行程中位置误差 0.3 m，在 50 m 行程中位置误差 1.2 m。分别计算漂移率，并判断误差更像随机游走还是系统偏差。
5. 一个团队报告「ATE 为 0.05 m，RPE 为 0.4 m」，请你解释这组数字可能意味着什么，以及应当先排查哪个模块。

延伸阅读

- Sola 的 ESKF 教程（arXiv:1711.02508）：$SO(3)/SE(3)$ 求导与误差状态。
- Barfoot《State Estimation for Robotics》（Cambridge, 2017）：李群上的估计理论。

第五章 轮式里程计：从编码器到航迹推算

本章导读

轮式里程计是所有移动机器人最基础、最廉价、也最常被低估的定位手段：两个编码器加初中数学就能给出 100 Hz 的位姿。但「便宜」不等于「简单」——轮子会打滑，轮距会标不准，地面不会永远平。本章从编码器的一个脉冲讲起，一路推到差速底盘的圆弧航迹推算、麦轮底盘的逆运动学，以及一套可落地的标定流程。学完本章，你应当能独立写出一个差速里程计节点，并知道它的误差从哪来、该怎么标、协方差该怎么填。

5.1 编码器：把一个脉冲变成一段位移

车轮的运动最终由编码器变成数字。**增量式编码器**（incremental encoder）输出 A、B 两路正交方波，转子每转过一个固定角度产生一个脉冲，数脉冲得到「转了多少」，比较 A、B 相位得到「往哪转」；**绝对式编码器**（absolute encoder）每个角度对应唯一编码，上电即知绝对角度。本书主要用增量式，因为便宜、用定时器正交解码即可读取。

设编码器**线数**（每转脉冲数，pulses per revolution, PPR）为 N，电机与轮之间的**减速比**为 i，车轮半径为 r。工程上普遍采用**四倍频**（quadrature）计数：A、B 两相的上升沿与下降沿都计数，因此电机每转一圈得到 $4N$ 个计数。电机转 i 圈车轮转一圈，于是车轮每转一圈对应

$$C_{\text{rev}} = 4Ni \tag{5.1}$$

个计数。车轮转一圈走过弧长 $2\pi r$，所以单个计数对应的行进距离是

$$\Delta s = \frac{2\pi r}{4Ni}. \tag{5.2}$$

式 (5.2) 里有三个容易搞错的量：N 是**电机轴**上的线数而不是轮轴上的；i 是减速比，$i > 1$ 表示减速；四倍频的因子 4 只在 A/B 正交解码时成立，若只数单相上升沿则没有这个 4。

例题 5.1 一个计数有多长 取 $N = 13$、四倍频后电机每转 52 个计数、减速比 $i = 30$、轮半径 $r = 0.065\,\text{m}$，则

$$\Delta s = \frac{2\pi \times 0.065}{4 \times 13 \times 30} = \frac{0.4084}{1560} \approx 2.62 \times 10^{-4}\,\text{m}, \tag{5.3}$$

即每个计数约 0.26 mm；若车轮以 1 m/s 前进，则每秒产生约 3817 个计数。

单个计数太粗怎么办？用时间换精度：在固定周期 Δt 内数计数 n，得到平均速度 $\hat{v} = n\Delta s/\Delta t$。但计数本身有 ± 1 的量化误差，因此速度估计的量化噪声为 $\Delta s/\Delta t$；仍用上例，$\Delta t = 10\,\text{ms}$ 时噪声约 2.6 cm/s，对 1 m/s 的车是 2.6 %。这解释了为什么低速时编码器测速很差。

编码器分辨率的三个坑

1. **线数不等于计数**：N 线四倍频后是 $4N$ 计数；某些厂商标的是「每转计数」。
2. **减速比方向**：$i = 30$ 表示电机转 30 圈车轮转 1 圈；若把 i 当成增速比，结果会差 i^2 倍。
3. **分辨率不等于精度**：0.26 mm 只是量化步长，真实精度还取决于轮半径误差、打滑与安装偏心。

5.2 两轮差速运动学

5.2.1 从轮速到车体速度

考虑两轮差速底盘：左右两个驱动轮位于车体两侧，轮心连线过车体中心，轮距为 L。记左、右轮的线速度（轮缘相对地面的速度）为 v_L、v_R，车体中心的速度为 v、角速度为 ω。刚体上任意一点的速度等于「中心速度 + 角速度叉乘该点相对中心的位置」。右轮相对中心的位置是 $(0, -L/2)$（机体系 x 向前、y 向左），而 $\boldsymbol{\omega} = [0, 0, \omega]^{\top}$，

于是右轮接地点的速度为

$$\boldsymbol{v}_R = \boldsymbol{v} + \boldsymbol{\omega} \times \boldsymbol{r}_R = \begin{bmatrix} v \\ 0 \\ 0 \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ \omega \end{bmatrix} \times \begin{bmatrix} 0 \\ -L/2 \\ 0 \end{bmatrix} = \begin{bmatrix} v + \dfrac{\omega L}{2} \\ 0 \\ 0 \end{bmatrix}. \tag{5.4}$$

轮子不打滑时，轮缘速度沿前进方向的分量就是 v_R，横向分量被地面约束为零。左轮同理，于是

$$v_R = v + \frac{\omega L}{2}, \qquad v_L = v - \frac{\omega L}{2}. \tag{5.5}$$

把两式相加、相减，解出车体速度：

$$v = \frac{v_R + v_L}{2}, \qquad \omega = \frac{v_R - v_L}{L}. \tag{5.6}$$

式 (5.6) 的物理含义很直白：v 是两轮速度的平均（决定直线快慢），ω 是两轮速度之差除以轮距（决定转向快慢）。当 $v_R = v_L$ 时 $\omega = 0$，车走直线；当 $v_R = -v_L$ 时 $v = 0$，车原地转。

5.2.2 非完整约束

车体在世界系中的位姿是三维的 (x, y, θ)，但上式只有两个独立输入 (v, ω)。把车体速度投影到世界系：

$$\dot{x} = v\cos\theta, \qquad \dot{y} = v\sin\theta, \qquad \dot{\theta} = \omega. \tag{5.7}$$

消去 v 得到

$$\dot{x}\sin\theta - \dot{y}\cos\theta = 0. \tag{5.8}$$

这是一个不能积分成「位置约束」的微分约束，叫**非完整约束**（nonholonomic constraint）：普通轮式车不能侧向平移，必须先转再走。它决定了轮式车不能像全向底盘那样直接横移，也决定了路径规划必须考虑曲率（见第 19 章）。

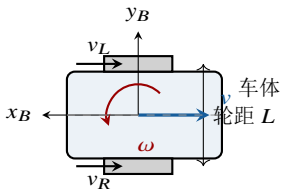


图 5.1: 两轮差速底盘的俯视图（机体系 x_B 向前、y_B 向左）。车体中心速度 v 是两轮速度的平均，角速度 ω 由两轮速度差除以轮距决定；轮距 L 的标定误差会直接变成角速度的比例误差。

5.3 离散航迹推算：圆弧模型与直线近似

里程计在每个采样周期执行一次「已知上一时刻位姿与本周期的 v, ω，求新位姿」。这就是**航迹推算**（dead reckoning，航位推算），它的概率版本（把轮速误差建模成噪声）见 [38]。关键在于：一个周期内 v、ω 怎么变化？最常用的是**常值假设**：周期内 v、ω 不变。

5.3.1 圆弧模型

在常值假设且 $\omega \neq 0$ 时，车体走一段圆弧。由式 (5.7)，第 k 个周期初始位姿为 (x_k, y_k, θ_k)，则时刻 $t \in [0, \Delta t]$ 的朝向为 $\theta(t) = \theta_k + \omega t$，而 $\dot{x} = v\cos\theta$ 从 0 到 Δt 的积分为 $x_{k+1} - x_k = \int_0^{\Delta t} v\cos(\theta_k + \omega t)\mathrm{d}t$。做换元 $u = \theta_k + \omega t$，则 $\mathrm{d}u = \omega\,\mathrm{d}t$，$\mathrm{d}t = \mathrm{d}u/\omega$，积分限从 $t = 0, \Delta t$ 变为 $u = \theta_k,\ \theta_k + \omega\Delta t$：

$$x_{k+1} - x_k = \frac{v}{\omega}\int_{\theta_k}^{\theta_k + \omega\Delta t} \cos u\,\mathrm{d}u = \frac{v}{\omega}\left[\sin(\theta_k + \omega\Delta t) - \sin\theta_k\right]. \tag{5.9}$$

同理对 $\dot{y}=v\sin\theta$ 做同样的换元（$\int \sin u\,\mathrm{d}u=-\cos u$），得 $y_{k+1}-y_k=-\frac{v}{\omega}[\cos(\theta_k+\omega\Delta t)-\cos\theta_k]$。再加上朝向更新 $\theta_{k+1}=\theta_k+\omega\Delta t$，就得到完整的**圆弧模型**：

$$\begin{aligned} x_{k+1} &= x_k+\frac{v}{\omega}\left[\sin\left(\theta_k+\omega\Delta t\right)-\sin\theta_k\right],\\ y_{k+1} &= y_k-\frac{v}{\omega}\left[\cos\left(\theta_k+\omega\Delta t\right)-\cos\theta_k\right],\\ \theta_{k+1} &= \theta_k+\omega\Delta t. \end{aligned} \tag{5.10}$$

圆弧模型的几何解释。记转角 $\Delta\theta=\omega\Delta t$、转弯半径 $R=v/\omega$。用和差化积公式 $\sin a-\sin b=2\cos\frac{a+b}{2}\sin\frac{a-b}{2}$，取 $a=\theta_k+\Delta\theta$、$b=\theta_k$，式 (5.9) 变为

$$x_{k+1}-x_k=\frac{v}{\omega}\cdot 2\cos\left(\theta_k+\frac{\Delta\theta}{2}\right)\sin\frac{\Delta\theta}{2}=\underbrace{2R\sin\frac{\Delta\theta}{2}}_{\text{弦长}d}\cos\left(\theta_k+\frac{\Delta\theta}{2}\right). \tag{5.11}$$

对 y 分量做同样的换元积分，再使用和差化积公式 $\cos a-\cos b=-2\sin\frac{a+b}{2}\sin\frac{a-b}{2}$（取 $a=\theta_k+\Delta\theta$、$b=\theta_k$），得到

$$y_{k+1}-y_k=-\frac{v}{\omega}\left[\cos\left(\theta_k+\Delta\theta\right)-\cos\theta_k\right]=d\sin\left(\theta_k+\frac{\Delta\theta}{2}\right), \tag{5.12}$$

其中 $d=2R\sin(\Delta\theta/2)$ 就是同一步的弦长。因此圆弧模型可以统一地理解为：**一步的位移等于弦长 d，方向是中间朝向** $\theta_k+\Delta\theta/2$。这比硬背公式好记，也直接给出下面的直线极限。

5.3.2 直线模型与极限

当 $\omega\to 0$ 时 $v/\omega\to\infty$、$\Delta\theta\to 0$，式 (5.10) 是 0/0 型极限。对 x 分量做一阶展开：

$$\sin\left(\theta_k+\Delta\theta\right)-\sin\theta_k=\sin\theta_k\cos\Delta\theta+\cos\theta_k\sin\Delta\theta-\sin\theta_k. \tag{5.13}$$

当 $\Delta\theta\to 0$ 时 $\cos\Delta\theta\to 1$、$\sin\Delta\theta\to\Delta\theta+O(\Delta\theta^3)$，故上式 $=\Delta\theta\cos\theta_k+O(\Delta\theta^2)$。于是

$$x_{k+1}-x_k=\frac{v}{\omega}\cdot\Delta\theta\cos\theta_k+O(\Delta\theta^2)=v\Delta t\cos\theta_k+O(\Delta t^2), \tag{5.14}$$

其中用了 $v\Delta\theta/\omega=v\Delta t$。对 y 分量同理可得 $y_{k+1}-y_k\to v\Delta t\sin\theta_k$。于是 $\omega=0$ 时圆弧模型退化为**直线模型**：

$$x_{k+1}=x_k+v\Delta t\cos\theta_k,\quad y_{k+1}=y_k+v\Delta t\sin\theta_k,\quad \theta_{k+1}=\theta_k. \tag{5.15}$$

5.3.3 三种离散化误差对比

同一个物理运动，用不同离散公式近似，误差量级不同。以式 (5.7) 的 x 分量为例如下对比。

欧拉法用周期初朝向：$x_{k+1}=x_k+v\Delta t\cos\theta_k$。真实位移为 $\int_0^{\Delta t}v\cos(\theta_k+\omega t)\mathrm{d}t$，把余弦在 $t=0$ 展开得 $\cos(\theta_k+\omega t)=\cos\theta_k-\omega t\sin\theta_k+O(t^2)$，逐项积分：

$$\Delta x_{\text{true}}=v\Delta t\cos\theta_k-\frac{1}{2}v\omega\Delta t^2\sin\theta_k+O(\Delta t^3), \tag{5.16}$$

故欧拉法局部误差是 $O(v\omega\Delta t^2)$，是**一阶**方法。**中点法**用中间朝向 $x_{k+1}=x_k+v\Delta t\cos(\theta_k+\omega\Delta t/2)$，把余弦在区间中点展开后奇次项对称抵消，局部误差为 $O(\Delta t^3)$，是**二阶**方法。**圆弧公式**在常值假设下没有离散化误差。所以急转弯时不要图省事用欧拉法；经验上 $|\omega|\,\Delta t<0.05$ rad 时欧拉法误差可忽略，否则应当升级。

圆弧模型的三条实践结论

1. 一步走的是圆弧：位移是弦长 $d=2R\sin(\Delta\theta/2)$，方向是中间朝向；$\omega\to 0$ 时 $d\to v\Delta t$，退化为直线。
2. 圆弧公式与直线公式可以统一写成「d 沿中间朝向投影」，代码里用一个 `if` 判断 $|\omega|$ 是否小于阈值即可。
3. 当 v、ω 在一个周期内变化剧烈时，任何离散模型都有误差；解决办法是提高采样频率。

算法：差速轮式航迹推算（单周期）

1. 读取左、右轮累计计数 c_L, c_R，与上一周期作差得 $\Delta c_L, \Delta c_R$
2. 换算成轮子行进距离：$\Delta s_{L,R} \leftarrow \Delta c_{L,R} \cdot \Delta s$
3. 计算轮速：$v_{L,R} \leftarrow \Delta s_{L,R} / \Delta t$
4. 由式 (5.6) 得 $v \leftarrow (v_R + v_L)/2$，$\omega \leftarrow (v_R - v_L)/L$
5. 若 $|\omega| < \epsilon$（ϵ 取 10^{-3} rad/s）：
6. $\quad x \leftarrow x + v\Delta t \cos\theta$，$y \leftarrow y + v\Delta t \sin\theta$
7. 否则：
8. $\quad x \leftarrow x + \frac{v}{\omega}[\sin(\theta + \omega\Delta t) - \sin\theta]$
9. $\quad y \leftarrow y - \frac{v}{\omega}[\cos(\theta + \omega\Delta t) - \cos\theta]$
10. 更新 $\theta \leftarrow \theta + \omega\Delta t$，发布位姿与协方差

5.4 麦轮与全向底盘

差速底盘只能前进和转向，横向移动要靠多次转向完成。麦轮（Mecanum wheel）解决了这个问题：每个轮子的轮缘上装有一圈与轮轴成 45° 的辊子，轮子既能沿自身滚动方向驱动，又能沿辊子轴向被动滑动；四个轮子配合，底盘就能同时实现前后、左右、自转三个自由度的运动。

5.4.1 麦轮逆运动学推导

采用机体系：x 向前、y 向左、ω 逆时针为正。四个轮心位置为 $\boldsymbol{r}_1 = (a, b)$、$\boldsymbol{r}_2 = (a, -b)$、$\boldsymbol{r}_3 = (-a, b)$、$\boldsymbol{r}_4 = (-a, -b)$，其中 $2a$ 为前后轮距、$2b$ 为左右轮距（$a, b > 0$）。第 i 个轮子的接地点速度（把该点当作刚体上一点）为

$$\boldsymbol{u}_i = \boldsymbol{v} + \omega\hat{\boldsymbol{z}} \times \boldsymbol{r}_i = \begin{bmatrix} v_x - \omega r_{iy} \\ v_y + \omega r_{ix} \end{bmatrix}, \tag{5.17}$$

其中 $\boldsymbol{v} = [v_x, v_y]^\top$ 是底盘中心速度。麦轮的约束是：接地点速度沿「轮子约束方向」的分量必须等于轮子线速度 s_i，而垂直方向的分量由辊子自由滑动吸收。记第 i 个轮子的约束方向单位向量为 $\boldsymbol{e}_i$，则

$$s_i = \boldsymbol{u}_i \cdot \boldsymbol{e}_i. \tag{5.18}$$

由于辊子与轮轴成45°，四个轮子的约束方向与坐标轴也成45°。取 $\boldsymbol{e}_1 = \frac{1}{\sqrt{2}}[1, -1]^\top$、$\boldsymbol{e}_2 = \frac{1}{\sqrt{2}}[1, 1]^\top$、$\boldsymbol{e}_3 = \frac{1}{\sqrt{2}}[1, 1]^\top$、$\boldsymbol{e}_4 = \frac{1}{\sqrt{2}}[1, -1]^\top$，逐轮代入式 (5.17) 与式 (5.18)。以轮 1 为例，$\boldsymbol{r}_1 = (a, b)$，由式 (5.17) 得 $\boldsymbol{u}_1 = [v_x - \omega b,\ v_y + \omega a]^\top$，于是

$$s_1 = \frac{1}{\sqrt{2}}\left[(v_x - \omega b) - (v_y + \omega a)\right] = \frac{1}{\sqrt{2}}\left[v_x - v_y - (a + b)\omega\right]. \tag{5.19}$$

其余三轮同样计算，得到麦轮的**逆运动学**（由底盘速度求四个轮速）：

$$\begin{aligned} s_1 &= \tfrac{1}{\sqrt{2}}\left[v_x - v_y - (a + b)\omega\right], \\ s_2 &= \tfrac{1}{\sqrt{2}}\left[v_x + v_y + (a + b)\omega\right], \\ s_3 &= \tfrac{1}{\sqrt{2}}\left[v_x + v_y - (a + b)\omega\right], \\ s_4 &= \tfrac{1}{\sqrt{2}}\left[v_x - v_y + (a + b)\omega\right]. \end{aligned} \tag{5.20}$$

5.4.2 正运动学：从轮速回到速度

里程计需要的是反方向：已知四个轮速，求底盘速度。把式 (5.20) 的四式做三种线性组合：四式相加时含 v_y 与 ω 的项成对抵消，只剩 $4v_x$；取 $s_2 + s_3 - s_1 - s_4$ 时含 v_x 与 ω 的项抵消，只剩 $4v_y$；取 $s_2 + s_4 - s_1 - s_3$ 时只剩

$4(a+b)\omega$。整理得

$$\begin{aligned} v_x &= \frac{\sqrt{2}}{4}\,(s_1+s_2+s_3+s_4)\,, \\ v_y &= \frac{\sqrt{2}}{4}\,(s_2+s_3-s_1-s_4)\,, \\ \omega &= \frac{\sqrt{2}}{4(a+b)}\,(s_2+s_4-s_1-s_3)\,, \end{aligned} \quad\Longleftrightarrow\quad \begin{bmatrix} v_x \\ v_y \\ \omega \end{bmatrix} = \frac{\sqrt{2}}{4}\begin{bmatrix} 1 & 1 & 1 & 1 \\ -1 & 1 & 1 & -1 \\ -\frac{1}{a+b} & \frac{1}{a+b} & -\frac{1}{a+b} & \frac{1}{a+b} \end{bmatrix}\begin{bmatrix} s_1 \\ s_2 \\ s_3 \\ s_4 \end{bmatrix}. \tag{5.21}$$

积分 (v_x, v_y, ω) 即可得到位姿；工程上把轮子角速度乘以轮半径得到 s_i 后直接代入。这张矩阵就是麦轮里程计的全部「物理模型」，剩下的事情都是噪声与打滑。

5.4.3 为什么麦轮里程计更容易漂

式 (5.21) 是在「辊子与地面纯滚动、无侧向滑移」假设下推出来的，而现实中这个假设很差：辊子只在与地面接触的瞬间近似纯滚动，其余时刻存在横向滑动，滑移量随负载和速度变化；四个轮子的半径、减速比、编码器分辨率、轮胎磨损很难一致，线性叠加会把差异放大成系统性偏差；麦轮对地面起伏极敏感，一个 1 mm 的凸起就足以改变接触点速度方向；横移时四个轮子都在侧向滑移，编码器却只测到滚动分量，v_y 的估计必然偏小。因此在 RoboMaster 这类高速对抗场景中，麦轮里程计通常只用来做短时速度反馈，位置估计要靠 IMU、激光或视觉兜底（第 6 章与第 9 章）。

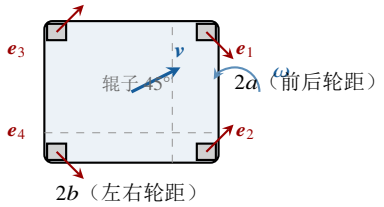


图 5.2: 麦轮底盘俯视图。四轮约束方向 $\boldsymbol{e}_i$ 与坐标轴成 45°；每个轮子只能约束接地点速度在 $\boldsymbol{e}_i$ 方向的分量，另一个方向靠辊子滑动吸收——这既是全向移动的来源，也是打滑误差的来源。

5.5 误差来源与标定

5.5.1 误差从哪里来

由式 (5.6) 与式 (5.2)，轮式里程计的参数误差可以分成两类。**轮半径误差**：r 有相对误差 e_r 时，Δs 以及 v、ω 都同比放大，整条轨迹被等比例放大而形状不变，这叫**尺度误差**。**轮距误差**：L 有相对误差 e_L 时，ω 同比缩小而 v 不变，转弯半径 $R = v/\omega$ 被放大，轨迹形状被扭曲，且转角越大误差越大。此外还有打滑（轮子转但车没走）、地面沉降（有效半径变小）、轮胎磨损与安装偏心等。前两类是**系统性**误差，可以通过标定压到很小；打滑是**随机且时变**的，只能靠外部传感器修正。

5.5.2 标定流程：直线定半径、旋转定轮距

第一步：直线标定轮半径。让机器人沿直线走过一段已知距离 D（卷尺测量），记录编码器给出的左右轮累计距离 $\hat{s}_R$、$\hat{s}_L$，并记编码器算出的行驶距离为 $\hat{D} = \frac{1}{2}(\hat{s}_R + \hat{s}_L)$。设标称半径为 r_{nom}，真实半径为 r。由 $\Delta s \propto r$ 可知真实距离与编码器距离之比等于半径之比：

$$\frac{D}{\hat{D}} = \frac{r}{r_{\text{nom}}} \;\Longrightarrow\; r = r_{\text{nom}} \cdot \frac{D}{\hat{D}}. \tag{5.22}$$

这里 r 是式 (5.2) 中唯一的线性因子，其余量（N、i、4）在标定期间固定，所以这个比例关系成立。

第二步：旋转标定轮距。让机器人原地旋转 n 整圈（n 取 10 圈以减小单圈误差），实际转角 $\theta_{\text{true}} = 2\pi n$。编码器给出的转角为 $\hat{\theta} = (\hat{s}_R - \hat{s}_L)/L_{\text{nom}}$，其中 $\hat{s}_{R,L}$ 是左右轮累计走过的弧长。真实轮距满足同一关系，由

$\hat{\theta}/\theta_{\text{true}} = L_{\text{true}}/L_{\text{nom}}$ 得

$$L = L_{\text{nom}} \cdot \frac{\theta_{\text{true}}}{\hat{\theta}}. \tag{5.23}$$

注意：原地旋转时左右轮弧长只依赖半径与转角，而两者之差又正比于轮距，所以旋转实验**只标定** L；半径必须由直线实验单独标定，否则两者会混在一起。

第三步：多组实验的最小二乘拟合。式 (5.23) 只用了一次实验。更稳的做法是做 M 组不同圈数的旋转实验，最小化

$$J(L) = \sum_{j=1}^{M} \left(\theta_j - \frac{d_j}{L} \right)^2, \qquad d_j = \hat{s}_{R,j} - \hat{s}_{L,j}, \tag{5.24}$$

其中 θ_j 是第 j 组实验的真实转角，d_j 是左右轮累计距离之差。对 L 求导并令其为零，由链式法则

$$\frac{\mathrm{d}J}{\mathrm{d}L} = \sum_{j=1}^{M} 2\left(\theta_j - \frac{d_j}{L} \right) \cdot \frac{d_j}{L^2} = 0. \tag{5.25}$$

两边乘以 $L^2/2$（$L \neq 0$），再移项得正规方程 $\sum_j \theta_j d_j = \sum_j d_j^2 / L$，解出

$$\hat{L} = \frac{\sum_{j=1}^{M} d_j^2}{\sum_{j=1}^{M} \theta_j d_j}. \tag{5.26}$$

目标函数对 $1/L$ 是二次的，所以有闭式解。工程上可以把 r 与 L 一起做二维最小二乘，但要注意可辨识性：直线实验只约束 r，旋转实验只约束 L，两组实验缺一不可。

标定的正确顺序

1. 先检查硬件：轮胎磨损、轮子偏心、编码器线数与减速比是否和配置一致；
2. 直线标定 r：走 10 m 以上，往返各三次取平均，消除打滑不对称；
3. 旋转标定 L：原地转 10 圈，正反各一次，用式 (5.26) 拟合；
4. 回到实际场地验证漂移率：走一个 20 m 的矩形回路，看位置闭合误差；若仍大，说明打滑是主因，标定救不了。

5.6 工程实现：ROS 2 中的轮式里程计

在 ROS 2 中，轮式里程计的标准接口是三件事：话题 `/odom`（类型 `nav_msgs/msg/Odometry`，包含位姿、速度及它们的协方差）、TF 变换 `odom→base_link`（由里程计节点维护，不跳变），以及约定 `odom` 的初始位姿为单位阵、之后只由里程计递推。协方差是 6×6 矩阵按行优先展开的 36 个数，通常只填 x, y, θ 对应的对角元（索引 0、7、35），数值随累计行程增大。

协方差的量级可以这样估：设单步距离误差标准差为 σ_s，走 n 步后按独立误差假设有 $\sigma_x^2 \approx n\sigma_s^2$（这里用了第 4 章方差相加的结论）。很多团队的 `odom` 协方差是随手填的常量，这会让下游融合算法（第 9 章）错误地信任里程计，是排错时的常见根源。

差速里程计的核心计算只有十几行，但几个符号一错就全错，下面给出参考实现（注释与字符串为英文）：

Listing 5.1: 差速里程计核心计算（节选）

```
// inputs: wheel distance increments (m) and time step (s)
double v = 0.5 * (ds_l + ds_r) / dt;
double w = (ds_r - ds_l) / dt / wheel_separation;
if (std::fabs(w) < 1e-3) {
    x += v * dt * std::cos(theta);
    y += v * dt * std::sin(theta);
} else {
    x += v / w * (std::sin(theta + w * dt) - std::sin(theta));
```

```
    y -= v / w * (std::cos(theta + w * dt) - std::cos(theta));
    theta += w * dt;
}
```

参数含义：`wheel_separation` 就是式 (5.6) 中的 L，`wheel_radius` 决定 `dist_per_count`，`publish_rate` 决定 Δt，`enable_odom_tf` 为真时节点自己发布 `odom→base_link`，与外部的 TF 广播器二选一。

本章小结

1. 增量式编码器四倍频后电机每转 $4N$ 计数，车轮每转 $4Ni$ 计数，单计数位移 $\Delta s = 2\pi r/(4Ni)$。
2. 差速底盘由刚体速度关系得 $v_R = v + \omega L/2$、$v_L = v - \omega L/2$，反解得 $v = (v_R + v_L)/2$、$\omega = (v_R - v_L)/L$；轮式底盘满足非完整约束 $\dot{x}\sin\theta - \dot{y}\cos\theta = 0$。
3. 常值 v, ω 假设下航迹推算有闭式圆弧解：位移等于弦长 $d = 2R\sin(\Delta\theta/2)$，方向为中间朝向；$\omega \to 0$ 时退化为直线模型。
4. 欧拉法局部误差 $O(\Delta t^2)$，中点法与圆弧模型更高阶；急转弯时应使用圆弧模型或提高采样频率。
5. 麦轮逆运动学由接地点速度在约束方向上的投影推出，正运动学是它的线性反解；麦轮漂移主要来自辊子滑移与四轮不一致。
6. 轮半径误差导致轨迹尺度误差，轮距误差导致曲率误差；直线实验标定 r，旋转实验用最小二乘标定 L。
7. 工程上里程计发布 `/odom` 与 `odom→base_link`，协方差应按累计行程传播。

第五章 练习

1. 编码器 $N = 1024$ 线、四倍频、直驱、$r = 0.1\,\text{m}$，求单计数位移与 10 ms 周期下的速度量化噪声。
2. 差速车 $L = 0.5\,\text{m}$，左右轮 0.6、0.9 m/s，求 v、ω、R 及 1 s 后的位姿（初始朝向 0）。
3. 用式 (5.10) 计算 $v = 1\,\text{m/s}$、$\omega = 2\,\text{rad/s}$、$\Delta t = 0.1\,\text{s}$、$\theta_k = 0$ 的一步位移，并与欧拉法、中点法比较误差量级。
4. 麦轮底盘 $a = 0.25\,\text{m}$、$b = 0.22\,\text{m}$，四个轮速均为 1 m/s，求底盘速度；若 s_4 因打滑偏小 10 %，估算 ω 的偏差。
5. 用式 (5.26) 处理 $(d_j, \theta_j) = (1.0, 6.28)$、$(2.0, 12.57)$、$(3.0, 18.85)$，求 $\hat{L}$ 并说明它为何只适用于「原地旋转」。

延伸阅读

- ROS 2 `diff_drive_controller` 文档（control.ros.org）：参数与协方差语义。
- Thrun 等《Probabilistic Robotics》第 5 章：里程计运动模型的概率表述。
- 麦轮底盘厂家手册：核对轮距、辊子角度与运动学矩阵的符号约定。

第六章 IMU 姿态解算与航迹推算

本章导读

IMU 是本篇最特殊的传感器：它测的不是位移，而是角速度与比力；数据最「脏」，噪声、零偏、温漂全在里面；但它又是唯一在轮子打滑、激光退化、相机过曝时仍然工作的传感器。本章把 IMU 从「一个报姿态的模块」讲清楚：测量模型怎么来、噪声怎么离散化、姿态怎么用四元数更新、横滚俯仰为什么能从重力读出来而偏航不能、互补滤波与 Mahony 为什么有效、ESKF 的 15 维公式每一块怎么推、以及为什么纯 IMU 位置积分几十秒就废。公式较多，请动笔跟着推。

6.1 IMU 测量模型与噪声

IMU 通常包含三轴陀螺仪与三轴加速度计。陀螺仪测的是机体系相对惯性系的角速度；加速度计测的是**比力**（specific force），即单位质量所受的非引力合力，在地表附近近似等于「加速度减重力」。沿用全书符号：$\boldsymbol{g} = [0, 0, -9.81]^{\top}$ 为世界系下的重力向量，$\boldsymbol{\omega}$ 为机体系下的真实角速度，$\boldsymbol{a}_W$ 为机体系原点在世界系下的真实加速度。记 IMU 测量值为 $\tilde{\boldsymbol{\omega}}$、$\tilde{\boldsymbol{a}}$，则

$$\tilde{\boldsymbol{\omega}} = \boldsymbol{\omega} + \boldsymbol{b}_g + \boldsymbol{n}_g, \qquad \tilde{\boldsymbol{a}} = \boldsymbol{R}_{WB}^{\top}\left(\boldsymbol{a}_W - \boldsymbol{g}\right) + \boldsymbol{b}_a + \boldsymbol{n}_a. \tag{6.1}$$

逐项解释：$\boldsymbol{b}_g, \boldsymbol{b}_a$ 是陀螺仪、加速度计的**零偏**（bias），是一条缓慢变化的非零常值；$\boldsymbol{n}_g, \boldsymbol{n}_a$ 是零均值高斯白噪声；$\boldsymbol{R}_{WB}^{\top}(\boldsymbol{a}_W - \boldsymbol{g})$ 说明加速度计测的是本体系下的比力——静止时 $\boldsymbol{a}_W = 0$，读数不是零，而是 $-\boldsymbol{R}^{\top}\boldsymbol{g}$，这就是为什么静置的 IMU 加速度读数约为 $9.81\,\mathrm{m/s^2}$ 且指向「上」。零偏本身也在缓慢游走，建模为

$$\dot{\boldsymbol{b}}_g = \boldsymbol{n}_{bg}, \qquad \dot{\boldsymbol{b}}_a = \boldsymbol{n}_{ba}, \tag{6.2}$$

其中 $\boldsymbol{n}_{bg}, \boldsymbol{n}_{ba}$ 也是零均值白噪声。这种「白噪声的积分」叫**随机游走**（random walk）。

6.1.1 从连续噪声到离散噪声

仿真与滤波都需要离散时间的噪声参数，而传感器手册给的是连续时间的噪声密度。下面把两套参数的关系推出来。连续时间白噪声 $\boldsymbol{n}(t) \in \mathbb{R}^3$ 定义为

$$\mathbb{E}\left[\boldsymbol{n}(t)\right] = \boldsymbol{0}, \qquad \mathbb{E}\left[\boldsymbol{n}(t)\boldsymbol{n}(t')^{\top}\right] = \sigma_c^2\,\delta(t - t')\,\boldsymbol{I}, \tag{6.3}$$

其中 $\delta(\cdot)$ 是狄拉克函数，σ_c 是**连续噪声密度**，单位是「物理量单位 / $\sqrt{\mathrm{Hz}}$」（陀螺仪常见的 $0.01^\circ/\mathrm{s}/\sqrt{\mathrm{Hz}}$ 即属此类）。滤波器在周期 Δt 内实际用到的是该区间上的平均噪声

$$\boldsymbol{n}_d = \frac{1}{\Delta t}\int_{t_k}^{t_k+\Delta t} \boldsymbol{n}(t)\,\mathrm{d}t. \tag{6.4}$$

它的方差由式 (6.3) 的二重积分得到：

$$\begin{aligned}
\mathbb{E}\left[\boldsymbol{n}_d\boldsymbol{n}_d^{\top}\right] &= \frac{1}{\Delta t^2}\int_{t_k}^{t_k+\Delta t}\int_{t_k}^{t_k+\Delta t} \mathbb{E}\left[\boldsymbol{n}(t)\boldsymbol{n}(t')^{\top}\right]\mathrm{d}t\,\mathrm{d}t' \\
&= \frac{\sigma_c^2}{\Delta t^2}\int_{t_k}^{t_k+\Delta t}\int_{t_k}^{t_k+\Delta t} \delta(t-t')\,\boldsymbol{I}\,\mathrm{d}t\,\mathrm{d}t' \\
&= \frac{\sigma_c^2}{\Delta t^2}\int_{t_k}^{t_k+\Delta t} \boldsymbol{I}\,\mathrm{d}t = \frac{\sigma_c^2}{\Delta t}\,\boldsymbol{I}.
\end{aligned} \tag{6.5}$$

第二个等号用了「δ 函数只在 $t = t'$ 处非零」的性质：先对 t' 积分得到 $\boldsymbol{I}$，再对 t 积分得到 $\Delta t\,\boldsymbol{I}$。于是离散噪声的标准差为

$$\sigma_d = \frac{\sigma_c}{\sqrt{\Delta t}}. \tag{6.6}$$

注意 Δt 越小，σ_d 越大：单次采样的噪声更「尖」，但积分一段时间后的总方差 $\sigma_d^2 \Delta t = \sigma_c^2 \Delta t$ 随时间线性增长，与物理直觉一致。

对零偏随机游走式 (6.2) 在一个周期上积分，得 $\boldsymbol{b}_{k+1} - \boldsymbol{b}_k = \int_{t_k}^{t_k+\Delta t} \boldsymbol{n}_b(t)\mathrm{d}t$，其方差为

$$\mathbb{E}\left[(\boldsymbol{b}_{k+1} - \boldsymbol{b}_k)\,(\boldsymbol{b}_{k+1} - \boldsymbol{b}_k)^{\top}\right] = \sigma_{bc}^2\,\Delta t\,\boldsymbol{I}, \tag{6.7}$$

即零偏离散化后的标准差为

$$\sigma_{bd} = \sigma_{bc}\sqrt{\Delta t}. \tag{6.8}$$

与白噪声相反，随机游走的离散标准差随 Δt 的平方根增大。

例题 6.1 把手册参数变成滤波器参数 某 IMU 陀螺噪声密度 $0.01^\circ/\mathrm{s}/\sqrt{\mathrm{Hz}}$，零偏随机游走 $0.001^\circ/\mathrm{s}/\sqrt{\mathrm{s}}$，滤波周期 $\Delta t = 5\,\mathrm{ms}$。由式 (6.6)，单周期角速度噪声标准差为 $0.01/\sqrt{0.005} \approx 0.141^\circ/\mathrm{s}$；由式 (6.8)，单周期零偏随机游走标准差为 $0.001 \times \sqrt{0.005} \approx 7.1 \times 10^{-5}\,^\circ/\mathrm{s}$。两者相差三个数量级，这就是为什么陀螺零偏可以近似当常值、而噪声必须每个周期都处理。

6.2 姿态表示与四元数更新

姿态解算（attitude estimation）指利用陀螺仪积分、加速度计重力观测（以及磁力计）估计机体相对世界系的旋转 $\boldsymbol{R}_{WB}$。工程上几乎都用单位四元数存储：$\boldsymbol{q} = [q_w, q_x, q_y, q_z]^{\top}$，$\|\boldsymbol{q}\| = 1$。四元数乘法定义为

$$\boldsymbol{q}_1 \otimes \boldsymbol{q}_2 = \begin{bmatrix} w_1 w_2 - \boldsymbol{v}_1^{\top}\boldsymbol{v}_2 \\ w_1\boldsymbol{v}_2 + w_2\boldsymbol{v}_1 + \boldsymbol{v}_1 \times \boldsymbol{v}_2 \end{bmatrix}, \tag{6.9}$$

其中 $\boldsymbol{v} = [q_x, q_y, q_z]^{\top}$ 是四元数的向量部分。单位四元数到旋转矩阵的完整换算是

$$\boldsymbol{R}(\boldsymbol{q}) = \begin{bmatrix} 1-2(q_y^2+q_z^2) & 2(q_xq_y - q_wq_z) & 2(q_xq_z + q_wq_y) \\ 2(q_xq_y + q_wq_z) & 1-2(q_x^2+q_z^2) & 2(q_yq_z - q_wq_x) \\ 2(q_xq_z - q_wq_y) & 2(q_yq_z + q_wq_x) & 1-2(q_x^2+q_y^2) \end{bmatrix}. \tag{6.10}$$

运动学方程。设机体角速度在机体系下为 $\boldsymbol{\omega}$，考察极短时间 $\mathrm{d}t$ 内的姿态变化。由于 $\boldsymbol{\omega}$ 是机体系下的量，增量旋转应右乘：

$$\boldsymbol{q}(t + \mathrm{d}t) = \boldsymbol{q}(t) \otimes \exp\left(\tfrac{1}{2}\boldsymbol{\omega}\mathrm{d}t\right). \tag{6.11}$$

当 $\mathrm{d}t$ 很小时，四元数指数映射一阶近似为 $\exp(\frac{1}{2}\boldsymbol{\omega}\mathrm{d}t) \approx [1,\ \frac{1}{2}\boldsymbol{\omega}\mathrm{d}t]$（完整形式为 $[\cos(\theta/2),\ \boldsymbol{u}\sin(\theta/2)]$，$\theta = \|\boldsymbol{\omega}\|\,\mathrm{d}t$）。代入并取极限：

$$\begin{aligned} \frac{\boldsymbol{q}(t+\mathrm{d}t) - \boldsymbol{q}(t)}{\mathrm{d}t} &= \frac{1}{\mathrm{d}t}\left(\boldsymbol{q}(t) \otimes \begin{bmatrix} 1 \\ \frac{1}{2}\boldsymbol{\omega}\mathrm{d}t \end{bmatrix} - \boldsymbol{q}(t) \otimes \begin{bmatrix} 1 \\ \boldsymbol{0} \end{bmatrix}\right) \\ &= \boldsymbol{q}(t) \otimes \begin{bmatrix} 0 \\ \frac{1}{2}\boldsymbol{\omega} \end{bmatrix} = \frac{1}{2}\,\boldsymbol{q}(t) \otimes \begin{bmatrix} 0 \\ \boldsymbol{\omega} \end{bmatrix}. \end{aligned} \tag{6.12}$$

于是得到四元数运动学方程

$$\dot{\boldsymbol{q}} = \frac{1}{2}\,\boldsymbol{q} \otimes \begin{bmatrix} 0 \\ \boldsymbol{\omega} \end{bmatrix}. \tag{6.13}$$

离散实现中直接用估计角速度做增量更新：

$$\boldsymbol{q}_{k+1} = \boldsymbol{q}_k \otimes \exp\left(\tfrac{1}{2}\left(\tilde{\boldsymbol{\omega}}_k - \hat{\boldsymbol{b}}_{g,k}\right)\Delta t\right), \tag{6.14}$$

指数项按 $\theta = \|\boldsymbol{\omega}\|\,\Delta t$、$\boldsymbol{u} = \boldsymbol{\omega}/\theta$ 算成 $[\cos(\theta/2),\ \boldsymbol{u}\sin(\theta/2)]$，更新后归一化。

四元数更新的两个易错点

1. 左乘还是右乘：$\boldsymbol{q}_{WB} \otimes \delta\boldsymbol{q}$ 是「绕机体系轴转」，$\delta\boldsymbol{q} \otimes \boldsymbol{q}_{WB}$ 是「绕世界系轴转」。用机体系角速度必须右乘。

2. 归一化：数值积分会让范数缓慢偏离 1，每次更新后应归一化，否则式 (6.10) 得到的矩阵不再正交。

6.3 由加速度计估计重力方向与横滚俯仰

静止或匀速运动时 $\boldsymbol{a}_W \approx \boldsymbol{0}$，式 (6.1) 的加速度计读数（忽略噪声与零偏）为

$$\tilde{\boldsymbol{a}} = -\boldsymbol{R}_{WB}^{\top}\boldsymbol{g}. \tag{6.15}$$

记重力大小 $g_0 = 9.81$，则 $-\boldsymbol{g} = [0, 0, g_0]^{\top}$。设 $\boldsymbol{R}_{WB} = \boldsymbol{R}_z(\psi)\boldsymbol{R}_y(\theta)\boldsymbol{R}_x(\phi)$（先横滚 ϕ、再俯仰 θ、最后偏航 ψ），下面展开式 (6.15)。先算两个基本旋转矩阵的乘积：

$$\boldsymbol{R}_y(\theta)\boldsymbol{R}_x(\phi) = \begin{bmatrix} \cos\theta & 0 & \sin\theta \\ 0 & 1 & 0 \\ -\sin\theta & 0 & \cos\theta \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos\phi & -\sin\phi \\ 0 & \sin\phi & \cos\phi \end{bmatrix} = \begin{bmatrix} \cos\theta & \sin\theta\sin\phi & \sin\theta\cos\phi \\ 0 & \cos\phi & -\sin\phi \\ -\sin\theta & \cos\theta\sin\phi & \cos\theta\cos\phi \end{bmatrix}. \tag{6.16}$$

记这个矩阵为 $\boldsymbol{M}$。由于 $\boldsymbol{R}_{WB} = \boldsymbol{R}_z(\psi)\boldsymbol{M}$，而 $\boldsymbol{R}_z$ 只作用于 x, y 分量、不改变 z 分量，所以 $\boldsymbol{R}_{WB}^{\top}\boldsymbol{e}_z$（即重力方向）的 z 分量等于 $\boldsymbol{M}^{\top}\boldsymbol{e}_z$ 的 z 分量。$\boldsymbol{M}^{\top}$ 的第三行就是 $\boldsymbol{M}$ 的第三列 $[\sin\theta\cos\phi,\ -\sin\phi,\ \cos\theta\cos\phi]$，于是

$$\tilde{\boldsymbol{a}} = g_0 \begin{bmatrix} \sin\theta\cos\phi \\ -\sin\phi \\ \cos\theta\cos\phi \end{bmatrix}. \tag{6.17}$$

由第二、三分量解出横滚，由第一、三分量解出俯仰：

$$\phi = \arctan 2\left(-\tilde{a}_y,\ \tilde{a}_z\right), \qquad \theta = \arctan 2\left(\tilde{a}_x,\ \sqrt{\tilde{a}_y^2 + \tilde{a}_z^2}\right). \tag{6.18}$$

有些资料写成 $\phi = \arctan 2(a_y, a_z)$、$\theta = \arctan 2(-a_x, \sqrt{a_y^2 + a_z^2})$，那是把重力方向取反的约定；两种写法只差整体符号，使用时必须与自己的加速度计符号约定一致。

为什么偏航不可观测？式 (6.17) 中没有 ψ：绕世界系 z 轴的旋转矩阵 $\boldsymbol{R}_z(\psi)$ 被 $\boldsymbol{R}_z^{\top}$ 作用在 $\boldsymbol{e}_z$ 上时满足 $\boldsymbol{R}_z^{\top}\boldsymbol{e}_z = \boldsymbol{e}_z$，因此 $\boldsymbol{R}_{WB}^{\top}\boldsymbol{g}$ 与 ψ 无关。物理上，重力方向只提供「哪边是下」，无法提供「哪边是北」——绕重力轴的旋转不改变任何加速度计读数。这就是所谓**重力对齐**（gravity alignment）：加速度计只能把姿态对齐到重力方向（两个自由度），第三个自由度（偏航）必须靠磁力计、轮速、激光或视觉提供。

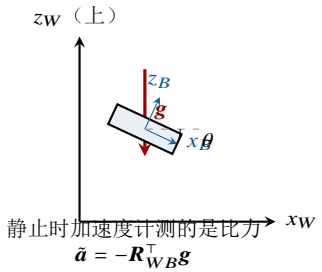


图 6.1: 加速度计测量重力方向示意。静止时读数只与「z_B 轴相对重力的倾角」有关，与绕重力轴的偏航角 ψ 无关，因此重力只能观测横滚与俯仰两个自由度。

6.4 互补滤波与 Mahony 滤波

陀螺积分短期准、长期漂；加速度计长期无漂、短期噪声大。两者恰好互补，于是有了**互补滤波**（complementary filter）。设真实角度为 θ，陀螺积分得到 θ_g（含低频漂移），加速度计得到 θ_a（含高频噪声）。取一阶低通、高通滤波器

$$H_{\mathrm{LP}}(s) = \frac{\omega_c}{s + \omega_c}, \qquad H_{\mathrm{HP}}(s) = \frac{s}{s + \omega_c}, \tag{6.19}$$

其中 ω_c 是截止角频率。两者之和恒为 1：$H_{\mathrm{LP}} + H_{\mathrm{HP}} = (s+\omega_c)/(s+\omega_c) = 1$，于是

$$\hat{\theta}(s) = H_{\mathrm{LP}}(s)\,\theta_a(s) + H_{\mathrm{HP}}(s)\,\theta_g(s). \tag{6.20}$$

这就是互补滤波：两个通道的传递函数互补成 1，低频信加速度计（不受漂移影响），高频信陀螺（不受噪声影响）。用后向差分 $s \approx (1-z^{-1})/\Delta t$ 离散化，得

$$\hat{\theta}_k = \alpha\left(\hat{\theta}_{k-1} + \omega_k \Delta t\right) + (1-\alpha)\,\theta_{a,k}, \qquad \alpha = \frac{\omega_c \Delta t}{1+\omega_c \Delta t}. \tag{6.21}$$

工程上常写成 $\alpha = \tau/(\tau+\Delta t)$，$\tau$ 是时间常数。α 越接近 1 越信任陀螺，越接近 0 越信任加速度计；典型值 0.95–0.99。

Mahony 滤波。互补滤波逐轴处理横滚俯仰，难以直接推广到三维。Mahony 的做法是构造一个三维「姿态误差」向量，再用 PI 控制器把它反馈到角速度上。加速度计测得的方向归一化为 $\hat{\boldsymbol{a}} = \tilde{\boldsymbol{a}}/\|\tilde{\boldsymbol{a}}\|$，当前姿态估计预测的「上」方向为 $\hat{\boldsymbol{g}}_b = \hat{\boldsymbol{R}}^{\top}\boldsymbol{e}_z$。定义误差

$$\boldsymbol{e} = \hat{\boldsymbol{g}}_b \times \hat{\boldsymbol{a}}. \tag{6.22}$$

为什么这样定义？设真实姿态与估计姿态差一个小角度 $\delta\boldsymbol{\theta}$，即 $\boldsymbol{R} = \hat{\boldsymbol{R}}\exp(\delta\boldsymbol{\theta}^{\wedge})$。真实「上」方向在机体系下为 $\boldsymbol{u} = \boldsymbol{R}^{\top}\boldsymbol{e}_z$，代入 $\boldsymbol{R}^{\top} = \exp(-\delta\boldsymbol{\theta}^{\wedge})\hat{\boldsymbol{R}}^{\top} \approx (\boldsymbol{I} - \delta\boldsymbol{\theta}^{\wedge})\hat{\boldsymbol{R}}^{\top}$：

$$\boldsymbol{u} \approx \left(\boldsymbol{I} - \delta\boldsymbol{\theta}^{\wedge}\right)\hat{\boldsymbol{g}}_b = \hat{\boldsymbol{g}}_b - \delta\boldsymbol{\theta} \times \hat{\boldsymbol{g}}_b = \hat{\boldsymbol{g}}_b + \hat{\boldsymbol{g}}_b \times \delta\boldsymbol{\theta}, \tag{6.23}$$

其中最后一步用了 $-\delta\boldsymbol{\theta} \times \hat{\boldsymbol{g}}_b = \hat{\boldsymbol{g}}_b \times \delta\boldsymbol{\theta}$。又因为静止时 $\hat{\boldsymbol{a}}$ 就是 $\boldsymbol{u}$，代入式 (6.22)：

$$\boldsymbol{e} = \hat{\boldsymbol{g}}_b \times \left(\hat{\boldsymbol{g}}_b + \hat{\boldsymbol{g}}_b \times \delta\boldsymbol{\theta}\right) = \hat{\boldsymbol{g}}_b \times \left(\hat{\boldsymbol{g}}_b \times \delta\boldsymbol{\theta}\right) = \hat{\boldsymbol{g}}_b\left(\hat{\boldsymbol{g}}_b^{\top}\delta\boldsymbol{\theta}\right) - \delta\boldsymbol{\theta} = -\left(\boldsymbol{I} - \boldsymbol{P}\right)\delta\boldsymbol{\theta}, \tag{6.24}$$

最后一步用了向量三重积公式 $\boldsymbol{a}\times(\boldsymbol{b}\times\boldsymbol{c}) = \boldsymbol{b}(\boldsymbol{a}\cdot\boldsymbol{c}) - \boldsymbol{c}(\boldsymbol{a}\cdot\boldsymbol{b})$ 与 $\boldsymbol{P} = \hat{\boldsymbol{g}}_b\hat{\boldsymbol{g}}_b^{\top}$。式 (6.24) 说明两件事：$\boldsymbol{e}$ 等于「姿态误差中垂直于重力方向的那一部分」取负；平行于重力方向（即偏航）的误差被投影矩阵 $\boldsymbol{I}-\boldsymbol{P}$ 滤掉，所以 Mahony 只能修正横滚俯仰。

既然 $\boldsymbol{e}$ 指向「把估计转回真值」的方向，把它按 PI 律加进角速度即可收敛：

$$\boldsymbol{\omega}_{\mathrm{cmd}} = \tilde{\boldsymbol{\omega}} - \hat{\boldsymbol{b}}_g + K_p\,\boldsymbol{e} + K_i \int \boldsymbol{e}\,\mathrm{d}t, \qquad \dot{\hat{\boldsymbol{b}}}_g = -K_i\,\boldsymbol{e}. \tag{6.25}$$

比例项 $K_p\boldsymbol{e}$ 负责快速纠正倾斜；积分项同时估计陀螺零偏——当姿态稳定、$\boldsymbol{e}\to 0$ 时 $\hat{\boldsymbol{b}}_g$ 停止变化，残差被积分吸收。注意 $\boldsymbol{e}$ 恒为零只说明估计方向与加速度计一致，偏航仍自由漂移，所以 Mahony 只适用于「横滚俯仰必须准、偏航交给别的传感器」的场景（第 9 章）。

6.5 ESKF：误差状态卡尔曼滤波

6.5.1 先复习普通卡尔曼滤波

线性高斯系统 $\boldsymbol{x}_{k+1} = \boldsymbol{F}_k\boldsymbol{x}_k + \boldsymbol{w}_k$、$\boldsymbol{z}_k = \boldsymbol{H}_k\boldsymbol{x}_k + \boldsymbol{v}_k$，其中 $\boldsymbol{w}_k \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{Q}_k)$、$\boldsymbol{v}_k \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{R}_k)$ 且相互独立。卡尔曼滤波 [21] 的预测与更新为

$$\hat{\boldsymbol{x}}_k^{-} = \boldsymbol{F}_{k-1}\hat{\boldsymbol{x}}_{k-1}, \qquad \boldsymbol{P}_k^{-} = \boldsymbol{F}_{k-1}\boldsymbol{P}_{k-1}\boldsymbol{F}_{k-1}^{\top} + \boldsymbol{Q}_{k-1}, \tag{6.26}$$

$$\boldsymbol{K}_k = \boldsymbol{P}_k^{-}\boldsymbol{H}_k^{\top}\left(\boldsymbol{H}_k\boldsymbol{P}_k^{-}\boldsymbol{H}_k^{\top} + \boldsymbol{R}_k\right)^{-1}, \qquad \hat{\boldsymbol{x}}_k = \hat{\boldsymbol{x}}_k^{-} + \boldsymbol{K}_k\left(\boldsymbol{z}_k - \boldsymbol{H}_k\hat{\boldsymbol{x}}_k^{-}\right), \tag{6.27}$$

$$\boldsymbol{P}_k = \left(\boldsymbol{I} - \boldsymbol{K}_k\boldsymbol{H}_k\right)\boldsymbol{P}_k^{-}. \tag{6.28}$$

预测把不确定性按动力学传播，更新用测量收缩不确定性；$\boldsymbol{K}$ 是「测量比先验可信多少」的加权。这些公式成立的前提是系统线性、噪声高斯——IMU 的姿态部分是矩阵乘法，不是向量加法，直接套 EKF 会在线性化处引入误差，于是有了 ESKF。

6.5.2 误差状态的定义

ESKF 误差状态（error-state Kalman filter）的核心思想：把真实状态写成「名义状态 ⊕ 小误差」。对 IMU 惯导取 15 维状态

$$\boldsymbol{x} = (\boldsymbol{p},\ \boldsymbol{v},\ \boldsymbol{R},\ \boldsymbol{b}_g,\ \boldsymbol{b}_a), \qquad \delta\boldsymbol{x} = (\delta\boldsymbol{p},\ \delta\boldsymbol{v},\ \delta\boldsymbol{\theta},\ \delta\boldsymbol{b}_g,\ \delta\boldsymbol{b}_a) \in \mathbb{R}^{15}, \tag{6.29}$$

其中 $\boldsymbol{p},\boldsymbol{v} \in \mathbb{R}^3$ 是世界系下的位置与速度，$\boldsymbol{R} \in SO(3)$，$\boldsymbol{b}_g, \boldsymbol{b}_a \in \mathbb{R}^3$。真实状态与名义状态的关系为

$$\boldsymbol{p} = \hat{\boldsymbol{p}} + \delta\boldsymbol{p}, \quad \boldsymbol{v} = \hat{\boldsymbol{v}} + \delta\boldsymbol{v}, \quad \boldsymbol{R} = \hat{\boldsymbol{R}}\exp(\delta\boldsymbol{\theta}^\wedge), \quad \boldsymbol{b}_g = \hat{\boldsymbol{b}}_g + \delta\boldsymbol{b}_g, \quad \boldsymbol{b}_a = \hat{\boldsymbol{b}}_a + \delta\boldsymbol{b}_a. \tag{6.30}$$

注意姿态不能写成「$\hat{\boldsymbol{R}} + \delta\boldsymbol{\theta}$」——旋转矩阵相加后不再正交，必须写成名义旋转右乘一个小旋转。

6.5.3 误差状态运动方程

把式 (6.30) 代入真实状态的动力学，减去名义状态的动力学，保留一阶项。

位置误差。由 $\dot{\boldsymbol{p}} = \boldsymbol{v}$ 与 $\dot{\hat{\boldsymbol{p}}} = \hat{\boldsymbol{v}}$ 直接相减得

$$\delta\dot{\boldsymbol{p}} = \delta\boldsymbol{v}. \tag{6.31}$$

速度误差。真实加速度与名义加速度分别为

$$\boldsymbol{a}_W = \boldsymbol{R}(\tilde{\boldsymbol{a}} - \boldsymbol{b}_a - \boldsymbol{n}_a) + \boldsymbol{g}, \qquad \hat{\boldsymbol{a}}_W = \hat{\boldsymbol{R}}\left(\tilde{\boldsymbol{a}} - \hat{\boldsymbol{b}}_a\right) + \boldsymbol{g}. \tag{6.32}$$

两式相减得 $\delta\dot{\boldsymbol{v}} = \boldsymbol{a}_W - \hat{\boldsymbol{a}}_W$。代入 $\boldsymbol{R} = \hat{\boldsymbol{R}}(\boldsymbol{I} + \delta\boldsymbol{\theta}^\wedge)$ 与 $\boldsymbol{b}_a = \hat{\boldsymbol{b}}_a + \delta\boldsymbol{b}_a$：

$$\begin{aligned}\boldsymbol{a}_W - \hat{\boldsymbol{a}}_W &= \hat{\boldsymbol{R}}(\boldsymbol{I} + \delta\boldsymbol{\theta}^\wedge)\left(\tilde{\boldsymbol{a}} - \hat{\boldsymbol{b}}_a - \delta\boldsymbol{b}_a - \boldsymbol{n}_a\right) - \hat{\boldsymbol{R}}\left(\tilde{\boldsymbol{a}} - \hat{\boldsymbol{b}}_a\right)\\ &= \hat{\boldsymbol{R}}\Big[(\boldsymbol{I} + \delta\boldsymbol{\theta}^\wedge)\left(\tilde{\boldsymbol{a}} - \hat{\boldsymbol{b}}_a\right) - (\boldsymbol{I} + \delta\boldsymbol{\theta}^\wedge)(\delta\boldsymbol{b}_a + \boldsymbol{n}_a) - \left(\tilde{\boldsymbol{a}} - \hat{\boldsymbol{b}}_a\right)\Big].\end{aligned} \tag{6.33}$$

第二项里 $\delta\boldsymbol{\theta}^\wedge(\delta\boldsymbol{b}_a + \boldsymbol{n}_a)$ 是两个小量相乘，属于二阶项，按一阶近似舍去；第一项与第三项相消后剩下 $\delta\boldsymbol{\theta}^\wedge(\tilde{\boldsymbol{a}} - \hat{\boldsymbol{b}}_a)$。于是

$$\delta\dot{\boldsymbol{v}} = \hat{\boldsymbol{R}}\left[\delta\boldsymbol{\theta}^\wedge\left(\tilde{\boldsymbol{a}} - \hat{\boldsymbol{b}}_a\right) - \delta\boldsymbol{b}_a - \boldsymbol{n}_a\right] = -\hat{\boldsymbol{R}}\left(\tilde{\boldsymbol{a}} - \hat{\boldsymbol{b}}_a\right)^\wedge\delta\boldsymbol{\theta} - \hat{\boldsymbol{R}}\,\delta\boldsymbol{b}_a - \hat{\boldsymbol{R}}\,\boldsymbol{n}_a, \tag{6.34}$$

最后一步用了 $\delta\boldsymbol{\theta}^\wedge\boldsymbol{x} = -\boldsymbol{x}^\wedge\delta\boldsymbol{\theta}$。式 (6.34) 说明三件事：姿态误差会「漏」到速度里（耦合项 $-\hat{\boldsymbol{R}}\boldsymbol{a}_b^\wedge$，其中 $\boldsymbol{a}_b = \tilde{\boldsymbol{a}} - \hat{\boldsymbol{b}}_a$）；加速度计零偏误差直接进速度；噪声经当前姿态旋转到世界系。

姿态误差。对 $\boldsymbol{R} = \hat{\boldsymbol{R}}\exp(\delta\boldsymbol{\theta}^\wedge)$ 求导，代入真实/名义动力学 $\dot{\boldsymbol{R}} = \boldsymbol{R}\boldsymbol{\omega}^\wedge$、$\dot{\hat{\boldsymbol{R}}} = \hat{\boldsymbol{R}}\hat{\boldsymbol{\omega}}^\wedge$（$\hat{\boldsymbol{\omega}} = \tilde{\boldsymbol{\omega}} - \hat{\boldsymbol{b}}_g$）：

$$\dot{\boldsymbol{R}} = \dot{\hat{\boldsymbol{R}}}\exp(\delta\boldsymbol{\theta}^\wedge) + \hat{\boldsymbol{R}}\frac{\mathrm{d}}{\mathrm{d}t}\exp(\delta\boldsymbol{\theta}^\wedge) \approx \hat{\boldsymbol{R}}\hat{\boldsymbol{\omega}}^\wedge(\boldsymbol{I} + \delta\boldsymbol{\theta}^\wedge) + \hat{\boldsymbol{R}}\delta\dot{\boldsymbol{\theta}}^\wedge, \tag{6.35}$$

另一方面 $\boldsymbol{\omega} = \tilde{\boldsymbol{\omega}} - \boldsymbol{b}_g - \boldsymbol{n}_g = \hat{\boldsymbol{\omega}} - \delta\boldsymbol{b}_g - \boldsymbol{n}_g$，故

$$\dot{\boldsymbol{R}} = \boldsymbol{R}\boldsymbol{\omega}^\wedge \approx \hat{\boldsymbol{R}}(\boldsymbol{I} + \delta\boldsymbol{\theta}^\wedge)\left(\hat{\boldsymbol{\omega}}^\wedge - \delta\boldsymbol{b}_g^\wedge - \boldsymbol{n}_g^\wedge\right). \tag{6.36}$$

令两式右边相等，两边左乘 $\hat{\boldsymbol{R}}^\mathsf{T}$，只保留一阶项：

$$\hat{\boldsymbol{\omega}}^\wedge + \hat{\boldsymbol{\omega}}^\wedge\delta\boldsymbol{\theta}^\wedge + \delta\dot{\boldsymbol{\theta}}^\wedge = \hat{\boldsymbol{\omega}}^\wedge - \delta\boldsymbol{b}_g^\wedge - \boldsymbol{n}_g^\wedge + \delta\boldsymbol{\theta}^\wedge\hat{\boldsymbol{\omega}}^\wedge. \tag{6.37}$$

消去两边的 $\hat{\boldsymbol{\omega}}^\wedge$，移项得

$$\delta\dot{\boldsymbol{\theta}}^\wedge = \delta\boldsymbol{\theta}^\wedge\hat{\boldsymbol{\omega}}^\wedge - \hat{\boldsymbol{\omega}}^\wedge\delta\boldsymbol{\theta}^\wedge - \delta\boldsymbol{b}_g^\wedge - \boldsymbol{n}_g^\wedge. \tag{6.38}$$

用反对称矩阵恒等式 $\boldsymbol{a}^\wedge\boldsymbol{b}^\wedge - \boldsymbol{b}^\wedge\boldsymbol{a}^\wedge = (\boldsymbol{a}\times\boldsymbol{b})^\wedge$，第一项为 $(\delta\boldsymbol{\theta}\times\hat{\boldsymbol{\omega}})^\wedge$；而 $\delta\boldsymbol{\theta}\times\hat{\boldsymbol{\omega}} = -\hat{\boldsymbol{\omega}}\times\delta\boldsymbol{\theta} = -\hat{\boldsymbol{\omega}}^\wedge\delta\boldsymbol{\theta}$。脱掉 $\wedge$ 得

$$\delta\dot{\boldsymbol{\theta}} = -\hat{\boldsymbol{\omega}}^\wedge\,\delta\boldsymbol{\theta} - \delta\boldsymbol{b}_g - \boldsymbol{n}_g. \tag{6.39}$$

零偏误差。零偏的随机游走对真实与名义状态相同，相减后噪声保留：

$$\delta\dot{\boldsymbol{b}}_g = \boldsymbol{n}_{bg}, \qquad \delta\dot{\boldsymbol{b}}_a = \boldsymbol{n}_{ba}. \tag{6.40}$$

6.5.4 离散化、状态转移矩阵与过程噪声

把式 (6.31)、(6.34)、(6.39)、(6.40) 写成 $\delta\dot{\boldsymbol{x}} = \boldsymbol{A}\,\delta\boldsymbol{x} + \boldsymbol{B}\boldsymbol{n}$，$\boldsymbol{A}$ 是 15×15 分块矩阵：

$$\boldsymbol{A} = \begin{bmatrix} \boldsymbol{0} & \boldsymbol{I} & \boldsymbol{0} & \boldsymbol{0} & \boldsymbol{0} \\ \boldsymbol{0} & \boldsymbol{0} & -\hat{\boldsymbol{R}}\boldsymbol{a}_b^{\wedge} & \boldsymbol{0} & -\hat{\boldsymbol{R}} \\ \boldsymbol{0} & \boldsymbol{0} & -\hat{\boldsymbol{\omega}}^{\wedge} & -\boldsymbol{I} & \boldsymbol{0} \\ \boldsymbol{0} & \boldsymbol{0} & \boldsymbol{0} & \boldsymbol{0} & \boldsymbol{0} \\ \boldsymbol{0} & \boldsymbol{0} & \boldsymbol{0} & \boldsymbol{0} & \boldsymbol{0} \end{bmatrix}, \qquad \boldsymbol{a}_b = \tilde{\boldsymbol{a}} - \hat{\boldsymbol{b}}_a. \tag{6.41}$$

一阶离散化 $\boldsymbol{F} = \boldsymbol{I} + \boldsymbol{A}\Delta t$，逐块写出：

$$\boldsymbol{F} = \begin{bmatrix} \boldsymbol{I} & \boldsymbol{I}\Delta t & \boldsymbol{0} & \boldsymbol{0} & \boldsymbol{0} \\ \boldsymbol{0} & \boldsymbol{I} & -\hat{\boldsymbol{R}}\boldsymbol{a}_b^{\wedge}\Delta t & \boldsymbol{0} & -\hat{\boldsymbol{R}}\Delta t \\ \boldsymbol{0} & \boldsymbol{0} & \boldsymbol{I} - \hat{\boldsymbol{\omega}}^{\wedge}\Delta t & -\boldsymbol{I}\Delta t & \boldsymbol{0} \\ \boldsymbol{0} & \boldsymbol{0} & \boldsymbol{0} & \boldsymbol{I} & \boldsymbol{0} \\ \boldsymbol{0} & \boldsymbol{0} & \boldsymbol{0} & \boldsymbol{0} & \boldsymbol{I} \end{bmatrix}. \tag{6.42}$$

逐块解释：第一行表示位置误差由速度误差积分而来；第二行表示速度误差由「姿态误差耦合的加速度」与「加速度计零偏误差」驱动，这两块分别是 $\partial\dot{\boldsymbol{v}}/\partial\delta\boldsymbol{\theta}$ 与 $\partial\dot{\boldsymbol{v}}/\partial\delta\boldsymbol{b}_a$ 乘以 Δt；第三行表示姿态误差由「自身随角速度旋转」与「陀螺零偏误差」驱动；第四、五行表示零偏误差本身不传播，只有噪声驱动。过程噪声协方差（一阶离散）为

$$\boldsymbol{Q}_d = \operatorname{diag}\left(\boldsymbol{0},\ \sigma_a^2\Delta t\,\boldsymbol{I},\ \sigma_g^2\Delta t\,\boldsymbol{I},\ \sigma_{bg}^2\Delta t\,\boldsymbol{I},\ \sigma_{ba}^2\Delta t\,\boldsymbol{I}\right), \tag{6.43}$$

其中 σ_a, σ_g 是加速度计、陀螺仪的连续噪声密度，σ_{bg}, σ_{ba} 为零偏随机游走密度；系数 Δt 正是式 (6.5)、(6.8) 的结论。位置块为零，因为位置误差的随机性完全来自速度误差的积分。

6.5.5 观测方程、更新与重置

位置观测。若外部（激光、视觉、轮速积分）给出世界系位置 $\boldsymbol{z}_p = \boldsymbol{p} + \boldsymbol{n}_p$，残差 $\boldsymbol{r} = \boldsymbol{z}_p - \hat{\boldsymbol{p}}$，则

$$\boldsymbol{H}_p = [\boldsymbol{I}\ \boldsymbol{0}\ \boldsymbol{0}\ \boldsymbol{0}\ \boldsymbol{0}] \in \mathbb{R}^{3\times 15}. \tag{6.44}$$

本体系速度观测。轮式里程计给出机体系速度 $\boldsymbol{z}_v = \boldsymbol{R}_{WB}^{\top}\boldsymbol{v}_W + \boldsymbol{n}_v$。对姿态误差求雅可比：$\boldsymbol{R}^{\top} \approx (\boldsymbol{I} - \delta\boldsymbol{\theta}^{\wedge})\hat{\boldsymbol{R}}^{\top}$，故

$$\boldsymbol{z}_v \approx \hat{\boldsymbol{R}}^{\top}\hat{\boldsymbol{v}} + \hat{\boldsymbol{R}}^{\top}\delta\boldsymbol{v} - \delta\boldsymbol{\theta}^{\wedge}\hat{\boldsymbol{R}}^{\top}\hat{\boldsymbol{v}} = \hat{\boldsymbol{R}}^{\top}\hat{\boldsymbol{v}} + \hat{\boldsymbol{R}}^{\top}\delta\boldsymbol{v} + \left(\hat{\boldsymbol{R}}^{\top}\hat{\boldsymbol{v}}\right)^{\wedge}\delta\boldsymbol{\theta}, \tag{6.45}$$

其中用了 $-\delta\boldsymbol{\theta}^{\wedge}\boldsymbol{w} = \boldsymbol{w}^{\wedge}\delta\boldsymbol{\theta}$。于是

$$\boldsymbol{H}_v = \begin{bmatrix} \boldsymbol{0} & \hat{\boldsymbol{R}}^{\top} & \left(\hat{\boldsymbol{R}}^{\top}\hat{\boldsymbol{v}}\right)^{\wedge} & \boldsymbol{0} & \boldsymbol{0} \end{bmatrix} \in \mathbb{R}^{3\times 15}. \tag{6.46}$$

这两项很有代表性：位置观测只「看」位置块；速度观测同时通过 $\hat{\boldsymbol{R}}^{\top}$ 与姿态误差耦合，所以它还能顺带修正姿态。残差 $\boldsymbol{r} = \boldsymbol{z} - \boldsymbol{h}(\hat{\boldsymbol{x}})$，按式 (6.27) 算 $\boldsymbol{K}$ 与 $\delta\hat{\boldsymbol{x}}$，再注入名义状态：

$$\hat{\boldsymbol{p}} \leftarrow \hat{\boldsymbol{p}} + \delta\hat{\boldsymbol{p}}, \quad \hat{\boldsymbol{v}} \leftarrow \hat{\boldsymbol{v}} + \delta\hat{\boldsymbol{v}}, \quad \hat{\boldsymbol{R}} \leftarrow \hat{\boldsymbol{R}}\exp\left(\delta\hat{\boldsymbol{\theta}}^{\wedge}\right), \quad \hat{\boldsymbol{b}}_g \leftarrow \hat{\boldsymbol{b}}_g + \delta\hat{\boldsymbol{b}}_g, \quad \hat{\boldsymbol{b}}_a \leftarrow \hat{\boldsymbol{b}}_a + \delta\hat{\boldsymbol{b}}_a. \tag{6.47}$$

注入后误差状态被「用掉」，应重置为零。由于旋转的注入不是加法，协方差需要乘重置雅可比：

$$\boldsymbol{P} \leftarrow \boldsymbol{G}\boldsymbol{P}\boldsymbol{G}^{\top}, \qquad \boldsymbol{G} = \operatorname{diag}\left(\boldsymbol{I},\ \boldsymbol{I},\ \boldsymbol{I} - \tfrac{1}{2}\left[\delta\hat{\boldsymbol{\theta}}\right]^{\wedge},\ \boldsymbol{I},\ \boldsymbol{I}\right). \tag{6.48}$$

$\boldsymbol{G}$ 的姿态块来自「注入后新误差状态与旧误差状态之差」的一阶展开；$\delta\hat{\boldsymbol{\theta}}$ 很小时 $\boldsymbol{G} \approx \boldsymbol{I}$，许多实现直接省略这一步。完整推导见 [37]，李群视角见 [1]。

算法：ESKF 单次迭代（IMU 预测 + 观测更新）

1. 读取 $\tilde{\boldsymbol{\omega}}, \tilde{\boldsymbol{a}}$ 与时间间隔 Δt

2. 名义状态预测：$\hat{\boldsymbol{a}}_W \leftarrow \hat{\boldsymbol{R}}(\tilde{\boldsymbol{a}} - \hat{\boldsymbol{b}}_a) + \boldsymbol{g}$

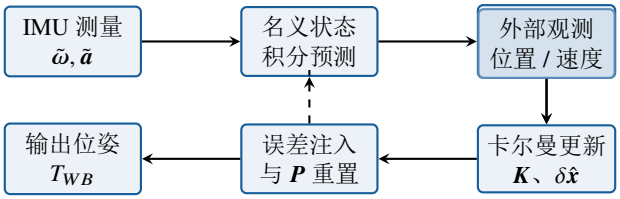


图 6.2: ESKF 的单次迭代。IMU 驱动名义状态积分与误差状态协方差传播；有观测时做卡尔曼更新，把 $\delta\hat{\boldsymbol{x}}$ 注入名义状态并重置协方差；注入后的名义状态继续作为下一次积分的起点。

3. $\hat{\boldsymbol{p}} \leftarrow \hat{\boldsymbol{p}} + \hat{\boldsymbol{v}}\Delta t + \frac{1}{2}\hat{\boldsymbol{a}}_W\Delta t^2$，$\hat{\boldsymbol{v}} \leftarrow \hat{\boldsymbol{v}} + \hat{\boldsymbol{a}}_W\Delta t$
4. $\hat{\boldsymbol{R}} \leftarrow \hat{\boldsymbol{R}}\exp((\tilde{\boldsymbol{\omega}} - \hat{\boldsymbol{b}}_g)^\wedge\Delta t)$
5. 协方差预测：由式 (6.42)、(6.43) 算 $\boldsymbol{P} \leftarrow \boldsymbol{F}\boldsymbol{P}\boldsymbol{F}^\top + \boldsymbol{Q}_d$
6. 若本周期有观测 $\boldsymbol{z}$：$\boldsymbol{r} \leftarrow \boldsymbol{z} - \boldsymbol{h}(\hat{\boldsymbol{x}})$，$\boldsymbol{S} \leftarrow \boldsymbol{H}\boldsymbol{P}\boldsymbol{H}^\top + \boldsymbol{R}$
7. $\boldsymbol{K} \leftarrow \boldsymbol{P}\boldsymbol{H}^\top\boldsymbol{S}^{-1}$，$\delta\hat{\boldsymbol{x}} \leftarrow \boldsymbol{K}\boldsymbol{r}$
8. 按式 (6.47) 注入，按式 (6.48) 重置 $\boldsymbol{P}$

ESKF 与 EKF 的对比。普通 EKF 直接在 15 维状态上线性化，姿态块要处理「旋转矩阵加向量」的非法运算，且线性化点离真值越远误差越大。ESKF 的线性化点始终在名义状态附近，误差状态每次更新后被重置、始终很小，因此线性化误差小；姿态更新用指数映射保持正交性；协方差描述小误差，数值条件更好。代价是多一套注入与重置的簿记。

6.6 航迹推算与零速修正

航迹推算（strapdown，捷联惯导）就是按测量模型把加速度积分两次。由式 (6.1) 去掉噪声并用估计零偏：

$$\hat{\boldsymbol{a}}_W = \hat{\boldsymbol{R}}\left(\tilde{\boldsymbol{a}} - \hat{\boldsymbol{b}}_a\right) + \boldsymbol{g}, \tag{6.49}$$

然后欧拉积分

$$\hat{\boldsymbol{v}}_{k+1} = \hat{\boldsymbol{v}}_k + \hat{\boldsymbol{a}}_{W,k}\Delta t, \qquad \hat{\boldsymbol{p}}_{k+1} = \hat{\boldsymbol{p}}_k + \hat{\boldsymbol{v}}_k\Delta t + \frac{1}{2}\hat{\boldsymbol{a}}_{W,k}\Delta t^2. \tag{6.50}$$

为什么纯 IMU 位置会很快不可用？设加速度计零偏残差为 b，则速度误差按 bt 增长、位置误差按 $\frac{1}{2}bt^2$ 增长。取 $b = 0.01\,\mathrm{m/s^2}$、$t = 60\,\mathrm{s}$，得 $\Delta p \approx \frac{1}{2}\times 0.01\times 60^2 = 18\,\mathrm{m}$。更隐蔽的是姿态误差：一个 0.01° 的倾角误差 $\epsilon = 1.75\times10^{-4}\,\mathrm{rad}$ 会把重力「漏」进水平加速度，产生 $g_0\epsilon \approx 1.7\times10^{-3}\,\mathrm{m/s^2}$ 的虚假加速度，60 s 后位置误差约 $\frac{1}{2}g_0\epsilon t^2 \approx 3.1\,\mathrm{m}$。这就是为什么纯 IMU 必须配合外部观测使用，也是第 9 章融合动机的定量依据。

零速修正（zero velocity update, ZUPT）是最廉价的修正手段：机器人静止时真实速度为零，把「速度为零」当作一次观测即可约束速度与零偏。检测条件通常是「角速度模长小于阈值、加速度模长接近 g_0、且持续若干周期」。触发后构造伪观测 $\boldsymbol{z}_v = \boldsymbol{0}$：

$$\boldsymbol{r} = \boldsymbol{0} - \hat{\boldsymbol{v}}, \qquad \boldsymbol{H} = [\boldsymbol{0}\ \boldsymbol{I}\ \boldsymbol{0}\ \boldsymbol{0}\ \boldsymbol{0}], \tag{6.51}$$

按式 (6.27) 更新。零速时速度误差被观测直接压小，同时通过 $\boldsymbol{F}$ 的耦合把零偏误差也约束住，这就是「ZUPT 能估零偏」的机理。工程上 $\boldsymbol{R}$ 不能过大（没效果）也不能过小（误检测时会把状态错误拉回）。

ZUPT 的三条工程经验

1. 检测比修正更重要：先保证「确实静止」再修正，宁可漏检不要误检。
2. 静止判据用滑动窗口（如连续 10 个周期都满足），单周期判据容易被撞击或振动骗过。
3. ZUPT 只约束速度与零偏，不约束位置；位置漂移需要外部绝对观测来消除。

6.7 工程实现：ROS 2 中的 IMU

ROS 2 的 `sensor_msgs/msg/Imu` 包含 `orientation`（四元数）、`angular_velocity`（rad/s）、`linear_acceleration`（$\mathrm{m/s^2}$，是比力而非世界系加速度）以及三组协方差。最常见的坑：

- **单位与坐标系**：角速度必须转成 rad/s，IMU 轴系必须通过外参与全书约定（x 前、y 左、z 上）统一，否则重力方向符号全错。
- **协方差**：填的是测量噪声而非零偏随机游走，混填会让滤波器过度自信。
- **振动**：高速运动时振动使加速度模长远超重力，静止检测会失效。

IMU 预积分（preintegration）是另一种使用方式：在两个关键帧 i, j 之间，把 IMU 测量积分成与初始状态无关的相对运动量

$$\Delta \boldsymbol{R}_{ij} = \prod_{k=i}^{j-1} \exp\left(\left(\tilde{\boldsymbol{\omega}}_k - \hat{\boldsymbol{b}}_g\right)^{\wedge} \Delta t\right), \quad \Delta \boldsymbol{v}_{ij} = \sum_{k=i}^{j-1} \Delta \boldsymbol{R}_{ik}\left(\tilde{\boldsymbol{a}}_k - \hat{\boldsymbol{b}}_a\right) \Delta t, \tag{6.52}$$

$\Delta \boldsymbol{p}_{ij}$ 则由 $\Delta \boldsymbol{v}_{ik}$ 与加速度再做一次积分得到。从而在优化时用 $\boldsymbol{R}_j = \boldsymbol{R}_i \Delta \boldsymbol{R}_{ij}$、$\boldsymbol{v}_j = \boldsymbol{v}_i + \boldsymbol{g}\Delta t_{ij} + \boldsymbol{R}_i \Delta \boldsymbol{v}_{ij}$、$\boldsymbol{p}_j = \boldsymbol{p}_i + \boldsymbol{v}_i \Delta t_{ij} + \frac{1}{2}\boldsymbol{g}\Delta t_{ij}^2 + \boldsymbol{R}_i \Delta \boldsymbol{p}_{ij}$ 恢复状态，不必重复积分原始 IMU 数据。零偏更新后只需对预积分量做一阶修正。其噪声传播与图优化中的用法见第 13 章与 [11]。

本章小结

1. IMU 测量模型：$\tilde{\boldsymbol{\omega}} = \boldsymbol{\omega} + \boldsymbol{b}_g + \boldsymbol{n}_g$，$\tilde{\boldsymbol{a}} = \boldsymbol{R}^{\top}(\boldsymbol{a}_W - \boldsymbol{g}) + \boldsymbol{b}_a + \boldsymbol{n}_a$；零偏按随机游走建模。
2. 白噪声离散化 $\sigma_d = \sigma_c/\sqrt{\Delta t}$，零偏随机游走 $\sigma_{bd} = \sigma_{bc}\sqrt{\Delta t}$，两者都由积分方差推出。
3. 四元数运动学 $\dot{\boldsymbol{q}} = \frac{1}{2}\boldsymbol{q} \otimes [0, \boldsymbol{\omega}]$，离散更新 $\boldsymbol{q}_{k+1} = \boldsymbol{q}_k \otimes \exp(\frac{1}{2}\boldsymbol{\omega}\Delta t)$，必须右乘并归一化。
4. 静止时加速度计读数为 $-\boldsymbol{R}^{\top}\boldsymbol{g}$，可解出横滚与俯仰；偏航不可观测，因为绕重力轴旋转不改变读数。
5. 互补滤波把低频的加速度计与高频的陀螺按 $H_{\mathrm{LP}} + H_{\mathrm{HP}} = 1$ 合成；Mahony 用叉乘误差 $\boldsymbol{e} = \hat{\boldsymbol{g}}_b \times \hat{\boldsymbol{a}}$ 加 PI 反馈，误差只含垂直重力方向的分量。
6. ESKF 以 $\boldsymbol{R} = \hat{\boldsymbol{R}}\exp(\delta\boldsymbol{\theta}^{\wedge})$ 定义误差状态，15 维 $\boldsymbol{F}$ 由式 (6.42) 给出，每块都对应一条一阶误差传播；更新后要注入并重置。
7. 纯 IMU 位置误差随 t^2 增长：$b_a = 0.01\,\mathrm{m/s^2}$ 时 60 s 约 18 m，0.01° 倾角误差时约 3 m。
8. ZUPT 用「速度为零」的伪观测约束速度与零偏，检测的可靠性比修正本身更重要。

第六章 练习

1. 由式 (6.3) 完整重推 $\sigma_d = \sigma_c/\sqrt{\Delta t}$ 与 $\sigma_{bd} = \sigma_{bc}\sqrt{\Delta t}$，并说明两者随 Δt 的变化方向为何相反。
2. 取 $\boldsymbol{q} = [\cos(\pi/8), 0, 0, \sin(\pi/8)]$，用式 (6.10) 求 $\boldsymbol{R}$，并验证它是绕 z 轴转 $\pi/4$。
3. 用式 (6.18) 计算 $\tilde{\boldsymbol{a}} = [0.5, 0.3, 9.7]^{\top}$ 对应的横滚与俯仰，并解释为什么偏航无法从这组数得到。
4. 写出 Mahony 误差 $\boldsymbol{e}$ 的一阶展开，证明 $\boldsymbol{e} = -(\boldsymbol{I} - \hat{\boldsymbol{g}}_b\hat{\boldsymbol{g}}_b^{\top})\delta\boldsymbol{\theta}$，并说明为什么偏航分量不进入误差。
5. 推导式 (6.42) 中「速度误差对姿态误差」那一块的来源，并说明为什么系数是 $\hat{\boldsymbol{R}}\boldsymbol{a}_b^{\wedge}$ 而不是 $\boldsymbol{a}_b^{\wedge}\hat{\boldsymbol{R}}$。
6. 设加速度计零偏残差 $0.02\,\mathrm{m/s^2}$、陀螺零偏残差 $0.05°/\mathrm{s}$，估算 30 s 后的位置误差量级，并判断哪个误差源主导。

延伸阅读

- Sola 的 ESKF 教程（arXiv:1711.02508）：本章第 6 节几乎逐式对应。
- Barfoot《State Estimation for Robotics》（Cambridge, 2017）：李群视角下的误差状态与协方差传播。
- Mahony 2008 与 Madgwick 2011 的开源实现（x-io.co.uk）：工程细节与参数整定。

第七章 激光里程计与扫描匹配

本章导读

激光雷达给的是「一圈点的三维坐标」，它不告诉你自己动了多少，却把周围几何形状以毫米级精度记录下来。于是激光里程计的核心问题变成：给定两帧点云，如何求出它们之间的相对位姿？这就是**扫描匹配**（scan matching）。本章从「最小化对应点距离」这一句话出发，把 ICP 的闭式解一步步推完，再讲点到面残差的 Gauss-Newton 求解、NDT 的高斯化建模、LOAM 的特征匹配框架，最后用信息矩阵的特征值解释「为什么激光里程计在长廊里会失效」。学完本章，你应当能看懂 FAST-LIO、LOAM、Cartographer 前端里那些矩阵是怎么来的。

7.1 为什么用 LiDAR 做里程计

激光雷达的优势是几何精度高、不受光照影响、测距范围远：一个 16 线雷达在 10 m 处的测距误差通常只有几厘米，而两帧点云之间的相对位姿可以由成百上千个点共同约束，理论上精度可以做到行程的 0.1 %–1 %。它不依赖纹理，所以在白墙、暗处、逆光等视觉失效的场景仍然可用。

缺点同样明显：点云稀疏且分布不均（近处密、远处疏），对几何结构敏感（长廊、大平面、空旷场地里约束不足），并且存在**运动畸变**——雷达转一圈需要 100 ms，这期间机器人已经动了，一帧点云的各个点其实是在不同位姿下测到的（去畸变的方法见第 16 章）。此外还有数据量大、算力消耗高的问题。

激光里程计和 SLAM 的关系是：它只负责「相邻帧之间怎么动」，不建全局地图、不做回环。SLAM 篇（第 11 章）会把它当作前端的一部分。

7.2 点云配准的问题定义

定义 7.1 (扫描匹配与点云配准)

给定源点云 $\mathcal{P}=\{\boldsymbol{p}_i\}_{i=1}^{N}$ 与目标点云 $\mathcal{Q}=\{\boldsymbol{q}_j\}_{j=1}^{M}$（都在同一坐标系下表达，例如雷达系），点云配准（point cloud registration）指求一个刚体变换 $T=(\boldsymbol{R},\boldsymbol{t})\in SE(3)$，使变换后的源点云与目标点云尽量重合；扫描匹配（scan matching）特指以激光扫描为输入的配准，它是激光里程计的核心步骤。

最自然的度量是「对应点距离平方和」：假设已经知道每个 $\boldsymbol{p}_i$ 与哪个 $\boldsymbol{q}_i$ 对应，则

$$\min_{\boldsymbol{R}\in SO(3),\,\boldsymbol{t}\in\mathbb{R}^3}\sum_{i=1}^{N}\left\|\boldsymbol{R}\boldsymbol{p}_i+\boldsymbol{t}-\boldsymbol{q}_i\right\|^2. \tag{7.1}$$

但「哪个点对应哪个点」本身也不知道，于是问题变成鸡生蛋：知道对应关系就能求变换，知道变换就能找对应关系。ICP（iterative closest point）的解法是交替迭代：用当前变换找最近邻作为对应，再用对应求变换。

定义 7.2 (最近邻与 KD 树)

对点 $\boldsymbol{p}$，它在目标点云中的最近邻（nearest neighbor）是 $\arg\min_{\boldsymbol{q}\in\mathcal{Q}}\|\boldsymbol{p}-\boldsymbol{q}\|$。若对每个点都暴力扫描全部 M 个点，单次查找是 $O(M)$；**KD 树**（k-d tree）把点云按坐标轴递归二分建树，把单次最近邻查询降到平均 $O(\log M)$，是 ICP 能实时运行的关键数据结构。

7.3 ICP：从闭式解到迭代流程

7.3.1 点到点 ICP 的闭式解

先解决「对应关系已知」时的子问题。目标函数

$$J(\boldsymbol{R},\boldsymbol{t}) = \sum_{i=1}^{N} \left\| \boldsymbol{R}\boldsymbol{p}_i + \boldsymbol{t} - \boldsymbol{q}_i \right\|^2 \tag{7.2}$$

对 $\boldsymbol{t}$ 是二次的，可以直接求极值。对 $\boldsymbol{t}$ 求偏导（对向量求导，梯度为零向量）：

$$\frac{\partial J}{\partial \boldsymbol{t}} = \sum_{i=1}^{N} 2\left(\boldsymbol{R}\boldsymbol{p}_i + \boldsymbol{t} - \boldsymbol{q}_i\right) = \boldsymbol{0} \implies N\boldsymbol{t} = \sum_{i=1}^{N}\boldsymbol{q}_i - \boldsymbol{R}\sum_{i=1}^{N}\boldsymbol{p}_i. \tag{7.3}$$

记质心 $\bar{\boldsymbol{p}} = \frac{1}{N}\sum_i \boldsymbol{p}_i$、$\bar{\boldsymbol{q}} = \frac{1}{N}\sum_i \boldsymbol{q}_i$，则

$$\boldsymbol{t} = \bar{\boldsymbol{q}} - \boldsymbol{R}\bar{\boldsymbol{p}}. \tag{7.4}$$

把式 (7.4) 代回 J，并记去质心坐标 $\tilde{\boldsymbol{p}}_i = \boldsymbol{p}_i - \bar{\boldsymbol{p}}$、$\tilde{\boldsymbol{q}}_i = \boldsymbol{q}_i - \bar{\boldsymbol{q}}$：

$$\boldsymbol{R}\boldsymbol{p}_i + \boldsymbol{t} - \boldsymbol{q}_i = \boldsymbol{R}\left(\tilde{\boldsymbol{p}}_i + \bar{\boldsymbol{p}}\right) + \bar{\boldsymbol{q}} - \boldsymbol{R}\bar{\boldsymbol{p}} - \left(\tilde{\boldsymbol{q}}_i + \bar{\boldsymbol{q}}\right) = \boldsymbol{R}\tilde{\boldsymbol{p}}_i - \tilde{\boldsymbol{q}}_i, \tag{7.5}$$

于是

$$J = \sum_{i=1}^{N} \left\| \boldsymbol{R}\tilde{\boldsymbol{p}}_i - \tilde{\boldsymbol{q}}_i \right\|^2 = \sum_{i=1}^{N} \left(\tilde{\boldsymbol{p}}_i^{\top}\tilde{\boldsymbol{p}}_i + \tilde{\boldsymbol{q}}_i^{\top}\tilde{\boldsymbol{q}}_i - 2\tilde{\boldsymbol{q}}_i^{\top}\boldsymbol{R}\tilde{\boldsymbol{p}}_i \right). \tag{7.6}$$

前两项与 $\boldsymbol{R}$ 无关，因此最小化 J 等价于最大化

$$\sum_{i=1}^{N} \tilde{\boldsymbol{q}}_i^{\top}\boldsymbol{R}\tilde{\boldsymbol{p}}_i = \operatorname{tr}\left(\boldsymbol{R}\sum_{i=1}^{N}\tilde{\boldsymbol{p}}_i\tilde{\boldsymbol{q}}_i^{\top}\right) = \operatorname{tr}\left(\boldsymbol{R}\boldsymbol{W}\right), \tag{7.7}$$

其中第一步用了 $\boldsymbol{a}^{\top}\boldsymbol{b} = \operatorname{tr}(\boldsymbol{b}\boldsymbol{a}^{\top})$，并定义了 3×3 矩阵

$$\boldsymbol{W} = \sum_{i=1}^{N} \tilde{\boldsymbol{p}}_i\tilde{\boldsymbol{q}}_i^{\top}. \tag{7.8}$$

现在问题变成「在 $SO(3)$ 上最大化 $\operatorname{tr}(\boldsymbol{R}\boldsymbol{W})$」。对 $\boldsymbol{W}$ 做奇异值分解（SVD）：

$$\boldsymbol{W} = \boldsymbol{U}\boldsymbol{\Sigma}\boldsymbol{V}^{\top}, \qquad \boldsymbol{\Sigma} = \operatorname{diag}(\sigma_1,\sigma_2,\sigma_3), \quad \sigma_1 \ge \sigma_2 \ge \sigma_3 \ge 0. \tag{7.9}$$

代入目标函数并利用迹的循环置换性质 $\operatorname{tr}(ABC) = \operatorname{tr}(BCA)$：

$$\operatorname{tr}(\boldsymbol{R}\boldsymbol{W}) = \operatorname{tr}\left(\boldsymbol{R}\boldsymbol{U}\boldsymbol{\Sigma}\boldsymbol{V}^{\top}\right) = \operatorname{tr}\left(\boldsymbol{\Sigma}\,\boldsymbol{V}^{\top}\boldsymbol{R}\,\boldsymbol{U}\right). \tag{7.10}$$

记 $\boldsymbol{A} = \boldsymbol{V}^{\top}\boldsymbol{R}\boldsymbol{U}$。由于 $\boldsymbol{V},\boldsymbol{R},\boldsymbol{U}$ 都是正交阵，$\boldsymbol{A}$ 也是正交阵，其列向量是单位向量，故对角元满足 $A_{ii} \le 1$。于是

$$\operatorname{tr}(\boldsymbol{R}\boldsymbol{W}) = \sum_{k=1}^{3}\sigma_k A_{kk} \le \sum_{k=1}^{3}\sigma_k \left\|A_{kk}\right\| \le \sum_{k=1}^{3}\sigma_k. \tag{7.11}$$

上界 $\sum_k \sigma_k$ 当且仅当 $A_{kk} = 1$（$k = 1,2,3$）时取到，此时 $\boldsymbol{A} = \boldsymbol{I}$，即

$$\boldsymbol{V}^{\top}\boldsymbol{R}\boldsymbol{U} = \boldsymbol{I} \implies \boldsymbol{R} = \boldsymbol{V}\boldsymbol{U}^{\top}. \tag{7.12}$$

反射情形。式 (7.12) 只有在 $\det(\boldsymbol{V}\boldsymbol{U}^{\top}) = +1$ 时才是合法的旋转；若 $\det(\boldsymbol{V}\boldsymbol{U}^{\top}) = -1$，说明最优解是「旋转 + 镜像」，必须改成旋转。此时取

$$\boldsymbol{R} = \boldsymbol{V}\,\operatorname{diag}(1,1,-1)\,\boldsymbol{U}^{\top}, \tag{7.13}$$

它等价于把 $\boldsymbol{V}$ 的最后一列取反，在保持正交的前提下使 $\det = +1$，并且是满足该约束时使 $\sum_k \sigma_k A_{kk}$ 最大的选择（因为取反只影响最小的 σ_3 对应的那一项）。求出 $\boldsymbol{R}$ 后，由式 (7.4) 得 $\boldsymbol{t} = \bar{\boldsymbol{q}} - \boldsymbol{R}\bar{\boldsymbol{p}}$。

ICP 闭式解的三步记忆法

1. 去质心：$\tilde{\boldsymbol{p}}_i = \boldsymbol{p}_i - \bar{\boldsymbol{p}}$，$\tilde{\boldsymbol{q}}_i = \boldsymbol{q}_i - \bar{\boldsymbol{q}}$，把平移分离出去；
2. 攒矩阵 $\boldsymbol{W} = \sum \tilde{\boldsymbol{p}}_i \tilde{\boldsymbol{q}}_i^{\top}$，做 SVD $\boldsymbol{W} = \boldsymbol{U}\boldsymbol{\Sigma}\boldsymbol{V}^{\top}$；
3. 取 $\boldsymbol{R} = \boldsymbol{V}\boldsymbol{U}^{\top}$（$\det < 0$ 时用 $\boldsymbol{V}\mathrm{diag}(1,1,-1)\boldsymbol{U}^{\top}$），$\boldsymbol{t} = \bar{\boldsymbol{q}} - \boldsymbol{R}\bar{\boldsymbol{p}}$。

7.3.2 迭代流程、对应关系与收敛

闭式解要求对应关系已知，而实际只知点云。ICP 用「最近邻 + 闭式解」交替逼近，单次迭代流程如下面的伪代码所示。

算法：点到点 ICP 的单次迭代

1. 输入：源点云 $\mathcal{P}$、目标点云 $\mathcal{Q}$、当前变换 $T = (\boldsymbol{R}, \boldsymbol{t})$
2. 把源点云变换到目标系：$\boldsymbol{p}'_i \leftarrow \boldsymbol{R}\boldsymbol{p}_i + \boldsymbol{t}$
3. 对每个 $\boldsymbol{p}'_i$，在 KD 树中查最近邻 $\boldsymbol{q}_{c(i)}$
4. 剔除坏对应：距离 $\left\|\boldsymbol{p}'_i - \boldsymbol{q}_{c(i)}\right\| > \tau_d$ 或法向夹角大的点对丢弃
5. 用剩下的点对，按式 (7.8)、(7.12)、(7.4) 求新变换 ΔT
6. 更新 $T \leftarrow \Delta T\, T$（或右乘，取决于变换定义约定）
7. 若残差相对变化小于阈值或达到最大迭代次数则停止，否则回到第 2 步

几点工程经验：**初值**很重要——ICP 只有局部收敛性，两帧之间转得太快或位移太大时最近邻会连错，导致收敛到错误解，所以实际系统会用 IMU 或匀速模型预测初值；**外点剔除**必须有——动态物体、遮挡边缘、雷达噪点都会产生距离很大的错误对应，通常按距离阈值（如 3σ）或法向一致性剔除；**收敛判据**一般用相邻两次迭代的残差下降量，而不是只看迭代次数。

ICP 的精度与局限。点到点 ICP 的残差直接是欧氏距离，它隐含假设「场景中任意方向都有足够约束」。在走廊里，沿走廊方向平移几乎不改变点到点距离，这个方向就不可观（第 7.7 节详述）。点到面 ICP 通过引入法向信息缓解了这个问题，也是现代 LIO 系统的默认选择。

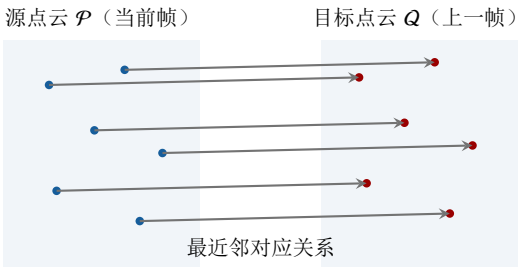


图 7.1: 点到点 ICP 的一步。对当前帧每个点找到上一帧的最近点，用这些对应点对求闭式刚体变换；迭代若干次后对应关系与变换同时收敛。

7.3.3 点到面 ICP 与 Gauss-Newton 求解

点到点残差对「沿平面方向的滑动」不敏感，点到面残差则直接把点压到平面上。设目标点云中与 $\boldsymbol{p}_i$ 对应的平面参数为 $(\boldsymbol{n}_i, \boldsymbol{q}_i)$，其中 $\boldsymbol{n}_i$ 是单位法向、$\boldsymbol{q}_i$ 是平面上一点。**点到面残差**（point-to-plane residual）定义为点 $\boldsymbol{p}_i$ 到该平面的有符号距离：

$$r_i = \boldsymbol{n}_i^{\top}\left(\boldsymbol{R}\boldsymbol{p}_i + \boldsymbol{t} - \boldsymbol{q}_i\right). \tag{7.14}$$

它是非线性的（$\boldsymbol{R}$ 在旋转）。设当前估计附近的旋转增量为 $\delta\boldsymbol{\theta}$，用一阶近似 $\boldsymbol{R} \approx (\boldsymbol{I} + \delta\boldsymbol{\theta}^\wedge)\boldsymbol{R}_0$；为简洁起见把 $\boldsymbol{R}_0\boldsymbol{p}_i$ 记作 $\boldsymbol{p}_i$（即先变换到当前估计的坐标系），并把 $\boldsymbol{t}$ 写成增量形式：

$$\begin{aligned} r_i &\approx \boldsymbol{n}_i^\top \left[\left(\boldsymbol{I} + \delta\boldsymbol{\theta}^\wedge\right) \boldsymbol{p}_i + \boldsymbol{t} - \boldsymbol{q}_i \right] \\ &= \boldsymbol{n}_i^\top \left(\boldsymbol{p}_i - \boldsymbol{q}_i\right) + \boldsymbol{n}_i^\top \boldsymbol{t} + \boldsymbol{n}_i^\top \delta\boldsymbol{\theta}^\wedge \boldsymbol{p}_i . \end{aligned} \tag{7.15}$$

最后一项用 $\boldsymbol{n}^\top \delta\boldsymbol{\theta}^\wedge \boldsymbol{p} = \delta\boldsymbol{\theta}^\top (\boldsymbol{p} \times \boldsymbol{n})$（由混合积 $\boldsymbol{n} \cdot (\delta\boldsymbol{\theta} \times \boldsymbol{p}) = \delta\boldsymbol{\theta} \cdot (\boldsymbol{p} \times \boldsymbol{n})$ 得到），于是

$$r_i \approx \underbrace{\boldsymbol{n}_i^\top \left(\boldsymbol{p}_i - \boldsymbol{q}_i\right)}_{r_i^0} + \left(\boldsymbol{p}_i \times \boldsymbol{n}_i\right)^\top \delta\boldsymbol{\theta} + \boldsymbol{n}_i^\top \boldsymbol{t}. \tag{7.16}$$

把待求量写成六维 $\delta\boldsymbol{x} = [\delta\boldsymbol{\theta};\, \boldsymbol{t}] \in \mathbb{R}^6$，则

$$r_i \approx r_i^0 + \boldsymbol{A}_i \, \delta\boldsymbol{x}, \qquad \boldsymbol{A}_i = \begin{bmatrix} \left(\boldsymbol{p}_i \times \boldsymbol{n}_i\right)^\top & \boldsymbol{n}_i^\top \end{bmatrix} \in \mathbb{R}^{1\times 6}. \tag{7.17}$$

最小化 $\sum_i w_i r_i^2$（w_i 为权重，可选鲁棒核），令梯度为零：

$$\frac{\partial}{\partial \delta\boldsymbol{x}} \sum_i w_i \left(r_i^0 + \boldsymbol{A}_i \delta\boldsymbol{x}\right)^2 = 2 \sum_i w_i \boldsymbol{A}_i^\top \left(r_i^0 + \boldsymbol{A}_i \delta\boldsymbol{x}\right) = \boldsymbol{0}. \tag{7.18}$$

整理成**正规方程**（normal equation）：

$$\left(\sum_i w_i \boldsymbol{A}_i^\top \boldsymbol{A}_i \right) \delta\boldsymbol{x} = - \sum_i w_i \boldsymbol{A}_i^\top r_i^0 . \tag{7.19}$$

左边矩阵 $\boldsymbol{H} = \sum_i w_i \boldsymbol{A}_i^\top \boldsymbol{A}_i \in \mathbb{R}^{6\times 6}$ 就是**信息矩阵**（information matrix）的近似，右边是梯度。解出 $\delta\boldsymbol{x}$ 后更新 $\boldsymbol{R} \leftarrow \exp(\delta\boldsymbol{\theta}^\wedge)\boldsymbol{R}$、$\boldsymbol{t} \leftarrow \boldsymbol{t} + \delta\boldsymbol{t}$，再重新找对应、重新线性化——这就是 Gauss-Newton 版的 ICP。实践中常加 Levenberg-Marquardt 阻尼 $\boldsymbol{H} + \lambda\boldsymbol{I}$ 防止病态。

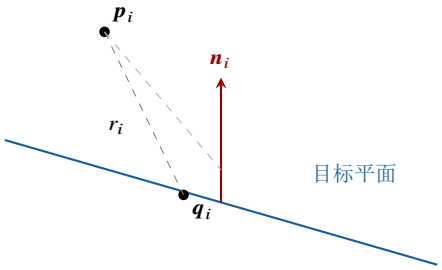


图 7.2: 点到面残差。点 $\boldsymbol{p}_i$ 到目标平面的有符号距离 $r_i = \boldsymbol{n}_i^\top(\boldsymbol{p}_i - \boldsymbol{q}_i)$；与点到点距离相比，它只惩罚「离开平面」的方向，对沿平面滑动不敏感，因此对走廊、墙面等结构收敛更好。

7.4 NDT：用高斯分布代替最近邻

ICP 的每一步都要找最近邻，代价高且对初值敏感。NDT（normal distributions transform）换了个思路：先把目标点云切成体素，每个体素内的点拟合成一个高斯分布；配准时不再找「最近点」，而是看当前帧的点落在目标体素高斯分布里的「概率」有多大。

定义 7.3 (NDT 与体素高斯)

把目标点云按固定边长划分成体素，第 j 个体素内若有 N_j 个点，定义其均值与协方差

$$\boldsymbol{\mu}_j = \frac{1}{N_j} \sum_{i=1}^{N_j} \boldsymbol{x}_i, \qquad \boldsymbol{\Sigma}_j = \frac{1}{N_j} \sum_{i=1}^{N_j} \left(\boldsymbol{x}_i - \boldsymbol{\mu}_j\right)\left(\boldsymbol{x}_i - \boldsymbol{\mu}_j\right)^\top . \tag{7.20}$$

当 N_j 太小时该体素不参与配准（或需要正则化 $\boldsymbol{\Sigma}_j$）。

对当前帧的每个点 $\boldsymbol{p}_i$，用变换 T 把它变到目标系得 $\boldsymbol{p}_i' = \boldsymbol{R}\boldsymbol{p}_i + \boldsymbol{t}$，找到它所在体素的高斯参数，定义该点

的「匹配得分」

$$s_i(T) = \exp\left(-\frac{1}{2}\left(\boldsymbol{p}_i' - \boldsymbol{\mu}_j\right)^{\top} \boldsymbol{\Sigma}_j^{-1}\left(\boldsymbol{p}_i' - \boldsymbol{\mu}_j\right)\right). \tag{7.21}$$

整体目标是最小化负得分（等价于让点尽量落在高斯中心）：

$$\min_{T} J(T) = -\sum_{i=1}^{N} \exp\left(-\frac{1}{2}\boldsymbol{d}_i^{\top}\boldsymbol{\Sigma}_j^{-1}\boldsymbol{d}_i\right), \qquad \boldsymbol{d}_i = \boldsymbol{p}_i' - \boldsymbol{\mu}_j. \tag{7.22}$$

Newton 法求解。记 $\boldsymbol{g}_i = \boldsymbol{\Sigma}_j^{-1}\boldsymbol{d}_i$，把 s_i 对变换增量 $\delta\boldsymbol{x} = [\delta\boldsymbol{\theta};\, \delta\boldsymbol{t}]$ 求导。先算 $\boldsymbol{d}_i$ 的雅可比：$\boldsymbol{p}_i' = \boldsymbol{R}\boldsymbol{p}_i + \boldsymbol{t} \approx \boldsymbol{p}_i + \delta\boldsymbol{\theta}^{\wedge}\boldsymbol{p}_i + \boldsymbol{t}$，故

$$\frac{\partial \boldsymbol{d}_i}{\partial \delta\boldsymbol{x}} = \left[-\boldsymbol{p}_i^{\wedge} \;\; \boldsymbol{I}\right] \in \mathbb{R}^{3\times 6}, \tag{7.23}$$

其中 $-\boldsymbol{p}_i^{\wedge}$ 来自 $\delta\boldsymbol{\theta}^{\wedge}\boldsymbol{p}_i = -\boldsymbol{p}_i^{\wedge}\delta\boldsymbol{\theta}$。对 $s_i = \exp(-\frac{1}{2}\boldsymbol{d}_i^{\top}\boldsymbol{g}_i)$ 求导：

$$\frac{\partial s_i}{\partial \delta\boldsymbol{x}} = -\frac{1}{2}s_i \frac{\partial\left(\boldsymbol{d}_i^{\top}\boldsymbol{\Sigma}_j^{-1}\boldsymbol{d}_i\right)}{\partial \delta\boldsymbol{x}} = -s_i\left(\frac{\partial \boldsymbol{d}_i}{\partial \delta\boldsymbol{x}}\right)^{\top}\boldsymbol{\Sigma}_j^{-1}\boldsymbol{d}_i, \tag{7.24}$$

最后一步用了 $\partial(\boldsymbol{d}^{\top}\boldsymbol{\Sigma}^{-1}\boldsymbol{d})/\partial\boldsymbol{x} = 2(\partial\boldsymbol{d}/\partial\boldsymbol{x})^{\top}\boldsymbol{\Sigma}^{-1}\boldsymbol{d}$，因子 2 与前面的 1/2 抵消。于是梯度

$$\boldsymbol{J}_{\mathrm{NDT}} = \frac{\partial J}{\partial \delta\boldsymbol{x}} = \sum_{i=1}^{N} s_i\left(\frac{\partial \boldsymbol{d}_i}{\partial \delta\boldsymbol{x}}\right)^{\top}\boldsymbol{\Sigma}_j^{-1}\boldsymbol{d}_i. \tag{7.25}$$

Hessian 用 Gauss-Newton 近似 $\boldsymbol{H}_{\mathrm{NDT}} \approx \sum_i s_i(\partial\boldsymbol{d}_i/\partial\delta\boldsymbol{x})^{\top}\boldsymbol{\Sigma}_j^{-1}(\partial\boldsymbol{d}_i/\partial\delta\boldsymbol{x})$，然后迭代

$$\delta\boldsymbol{x} = -\boldsymbol{H}_{\mathrm{NDT}}^{-1}\boldsymbol{J}_{\mathrm{NDT}}. \tag{7.26}$$

NDT 与 ICP 的对比：NDT 不需要显式的最近邻一一对应，因此收敛域更大、对初值更宽容；代价是体素化与高斯拟合并不能表达精细结构（体素内的平面/边缘会被「糊」成高斯），精度通常略低于点到面 ICP。两者在现代系统中经常混用，例如 Cartographer 的 2D 前端用相关性搜索，而 FAST-LIO 系列用点到面 ICP。原始论文见 [2]。

7.5 相关性搜索与分支定界

2D 激光场景下还有一种简单粗暴但极其鲁棒的方法：**相关扫描匹配**（correlative scan matching, CSM）。它把位姿空间离散成栅格 (x_k, y_k, θ_k)，对每个候选位姿计算「扫描点落在障碍栅格上的数量」作为得分，取最高分。由于是穷举搜索，它不依赖初值、不会陷入局部极小，但计算量是 $O(N_x N_y N_\theta)$。

工程上用**分支定界**（branch and bound）加速：把位姿空间组织成树，每个节点保存一个上界（该子树内所有位姿得分的上界），优先搜索上界高的节点，一旦某节点的上界低于当前最优解就整枝剪掉。这样绝大多数分支都不用展开。Cartographer 的 2D 前端正是这个思路（详细推导见第 15 章与 [19]）。本章只需要记住：在 2D 场景、初值不可靠时，相关性搜索比 ICP 更稳。

7.6 LOAM 框架：特征匹配的激光里程计

LOAM（lidar odometry and mapping）是 3D 激光里程计的里程碑式工作 [46]，后续的 LeGO-LOAM [34]、LIO-SAM [35]、KISS-ICP [39] 都受它影响。它的核心是「不再逐点找最近邻，而是先提取线、面特征，再让特征之间匹配」。

第一步：特征提取。对每个点 $\boldsymbol{p}_i$，取同一扫描线上前后各若干点组成集合 $\mathcal{S}$（如前后各 5 个点），定义曲率

$$\kappa_i = \frac{1}{|\mathcal{S}|}\sum_{j\in\mathcal{S}} \left\|\boldsymbol{p}_j - \boldsymbol{p}_i\right\|^2. \tag{7.27}$$

κ_i 大说明该点邻域起伏剧烈，是**边缘点**（线特征）；κ_i 小说明邻域平坦，是**平面点**（面特征）。按曲率排序后取最大的若干作为边缘点、最小的若干作为平面点，同时剔除可能属于同一物体边缘的邻近点，保证特征在空间中分散。

第二步：帧间匹配。对当前帧的每个边缘点 $\boldsymbol{p}_i$，在上一帧的边缘点中找最近的两个点 $\boldsymbol{p}_a, \boldsymbol{p}_b$（构成一条线），定义**点到线残差**为点到直线的距离：

$$r_{\text{edge}} = \frac{\left\| (\boldsymbol{p}_i - \boldsymbol{p}_a) \times (\boldsymbol{p}_i - \boldsymbol{p}_b) \right\|}{\left\| \boldsymbol{p}_a - \boldsymbol{p}_b \right\|}. \tag{7.28}$$

这个公式来自「三角形面积 = 底乘高 / 2 = 叉乘模长 / 2」：分子是平行四边形的面积，除以底边长度就是高，即点到直线的距离。对平面点 $\boldsymbol{p}_i$，在上一帧平面点中找最近的三个点 $\boldsymbol{p}_a, \boldsymbol{p}_b, \boldsymbol{p}_c$，用法向

$$\boldsymbol{n} = \frac{(\boldsymbol{p}_a - \boldsymbol{p}_b) \times (\boldsymbol{p}_c - \boldsymbol{p}_b)}{\left\| (\boldsymbol{p}_a - \boldsymbol{p}_b) \times (\boldsymbol{p}_c - \boldsymbol{p}_b) \right\|} \tag{7.29}$$

定义**点到面残差** $r_{\text{plane}} = \boldsymbol{n}^{\top}(\boldsymbol{p}_i - \boldsymbol{p}_b)$。把两类残差按式 (7.19) 组装正规方程，用 LM 迭代求解帧间位姿。

第三步：帧间 → 帧到地图。LOAM 把里程计与建图分开：里程计以 10 Hz 估计相邻帧位姿 $T_{k-1,k}$，再以更低频率把当前帧配准到已建立的地图（局部地图点云）上，得到全局一致的位姿 T_{WB}。位姿合成关系为

$$T_{WB}(k) = T_{WB}(k-1)\, T_{k-1,k}, \tag{7.30}$$

而帧到地图的配准会对 $T_{WB}(k)$ 做一次修正。两阶段的好处是：高频部分只做局部配准（快），低频部分做全局配准（准）。下面的伪代码给出一次迭代的骨架。

算法：LOAM 单帧处理流程

1. 输入：当前帧点云 $\mathcal{P}_k$、上一帧位姿 T_{k-1}、局部地图 $\mathcal{M}$
2. 提取曲率 κ_i，选出边缘点集与平面点集
3. 用匀速模型给初值 $T_k \leftarrow T_{k-1} T_{k-2}^{-1} T_{k-1}$
4. 重复若干次：
5. 　把 $\mathcal{P}_k$ 变换到上一帧系，为边缘点找最近线段、为平面点找最近平面
6. 　组装点到线、点到面残差，解正规方程得增量 $\delta\boldsymbol{x}$
7. 　更新 $T_k \leftarrow \exp(\delta\boldsymbol{x}^{\wedge}) T_k$，剔除大残差点
8. 把 $\mathcal{P}_k$ 按 T_k 加入局部地图 $\mathcal{M}$，输出位姿 T_k

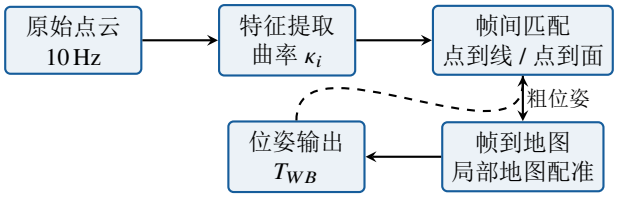


图 7.3: LOAM 的数据流。高频的帧间匹配给出粗位姿，低频的帧到地图配准把它拉回全局一致；两阶段结构后来被 LeGO-LOAM、LIO-SAM 等继承。

7.7 退化分析：为什么长廊里会失效

激光里程计的位姿解由正规方程 (7.19) 决定，其中 $\boldsymbol{H} = \sum_i w_i \boldsymbol{A}_i^{\top} \boldsymbol{A}_i$ 是 6×6 信息矩阵。$\boldsymbol{H}$ 的特征值刻画了「各个方向上有多少约束」：特征值大说明该方向被点云牢牢约束，特征值小说明该方向几乎无约束。

定义 7.4 (退化)

若信息矩阵 $\boldsymbol{H}$ 的某个特征值接近零（或远小于最大特征值），则对应的特征向量方向称为退化方向（degenerate direction）：在这个方向上，位姿解对残差几乎不敏感，估计结果由噪声决定而不是由数据决定。 ♣

长廊的例子。一条无限长的走廊，两侧是平行墙面。沿走廊方向平移时，每个点到墙面的距离完全不变，点到面残差不变，因此该方向对应的 $\boldsymbol{A}_i$ 列全为零，$\boldsymbol{H}$ 的最小特征值趋于零。类似地，空旷场地里围绕竖直轴的旋

转、大平面场景里垂直于平面的平移，都可能退化。

检测方法。对 $\boldsymbol{H}$ 做特征分解 $\boldsymbol{H} = \sum_k \lambda_k \boldsymbol{v}_k \boldsymbol{v}_k^\top$（特征值与特征向量定义见第 2 章），两种常用判据：

- 最小特征值阈值：$\lambda_{\min} < \lambda_{\text{th}}$ 判为退化；
- 条件数：$\lambda_{\max}/\lambda_{\min} > \kappa_{\text{th}}$ 判为病态。

退化方向就是 $\lambda_{\min}$ 对应的特征向量 $\boldsymbol{v}_{\min}$；工程上可以把解在这个方向上「冻结」或「用先验填充」：把 $\boldsymbol{H}$ 在该方向上的元素替换成一个很大的值（等价于加入极强的先验），使解不再依赖无约束的数据方向。

退化处理的三种手段

1. **换传感器**：加入 IMU 提供姿态与加速度先验，这是 LIO 紧耦合（第 16 章）的核心动机。
2. **换残差**：点到面比点到点约束更强，加入法向、平面、地面约束可以补充缺失方向。
3. **换判据**：实时监控 $\lambda_{\min}$ 与条件数，一旦判为退化就降低该方向权重、拒绝输出不可信位姿，而不是硬算。

本章小结

1. 激光里程计 = 扫描匹配：给定两帧点云求 $\boldsymbol{T}$ 使对应点距离最小。
2. 点到点 ICP 的闭式解：去质心后最大化 $\operatorname{tr}(\boldsymbol{R}\boldsymbol{W})$，$\boldsymbol{W} = \sum \tilde{\boldsymbol{p}}_i \tilde{\boldsymbol{q}}_i^\top$，SVD 得 $\boldsymbol{R} = \boldsymbol{V}\boldsymbol{U}^\top$（$\det < 0$ 时修正），$\boldsymbol{t} = \bar{\boldsymbol{q}} - \boldsymbol{R}\bar{\boldsymbol{p}}$。
3. ICP 迭代 = 找最近邻 + 闭式解，初值、外点剔除、收敛判据是三个工程要点；KD 树把最近邻查询降到 $O(\log M)$。
4. 点到面残差线性化后得到 $\boldsymbol{A}_i = [(\boldsymbol{p}_i \times \boldsymbol{n}_i)^\top, \boldsymbol{n}_i^\top]$，正规方程 $\boldsymbol{H}\delta\boldsymbol{x} = -\boldsymbol{b}$ 就是 Gauss-Newton 的一步。
5. NDT 用体素高斯代替最近邻，得分是负指数函数，Newton 法迭代；收敛域更大但精度略低。
6. 相关性搜索穷举位姿空间，鲁棒但慢；分支定界把它降到实用，是 Cartographer 的核心。
7. LOAM 用曲率提取边缘点与平面点，点到线、点到面残差两阶段（帧间 + 帧到地图）求解，位姿按 $\boldsymbol{T}_{WB}(k) = \boldsymbol{T}_{WB}(k-1)\boldsymbol{T}_{k-1,k}$ 合成。
8. 退化由信息矩阵 $\boldsymbol{H}$ 的小特征值检测，长廊、空旷、大平面是典型场景；对策是换传感器、换残差、换判据。

第七章 练习

1. 给两组对应点：$\mathcal{P} = \{(0,0),(1,0),(0,1)\}$、$\mathcal{Q} = \{(0,1),(1,1),(0,2)\}$，手工算出 $\bar{\boldsymbol{p}}$、$\bar{\boldsymbol{q}}$，验证这是纯平移 $[0,1]^\top$，并用式 (7.4) 反推 $\boldsymbol{t}$。
2. 推导式 (7.4)：从 $\partial J/\partial \boldsymbol{t} = 0$ 出发，写出每一步，并解释为什么平移可以「先分离出去」。
3. 设 $\boldsymbol{W} = \boldsymbol{U}\operatorname{diag}(2,1,0.1)\boldsymbol{V}^\top$ 且 $\det(\boldsymbol{V}\boldsymbol{U}^\top) = -1$，写出正确的 $\boldsymbol{R}$ 并说明为什么不能直接用 $\boldsymbol{V}\boldsymbol{U}^\top$。
4. 用式 (7.17) 写出一个点的 $\boldsymbol{A}_i$，并说明为什么沿平面方向的位移在残差中不出现。
5. 解释 NDT 得分函数式 (7.22) 中协方差逆矩阵的作用：如果某个体素内点云近似一条直线，$\boldsymbol{\Sigma}_j^{-1}$ 会把哪个方向的残差放大？
6. 走廊场景中，若只用点到面残差，$\boldsymbol{H}$ 的哪个特征值会接近零？请给出特征向量的物理含义，并设计一个检测方案。

延伸阅读

- Besl & McKay 1992 的 ICP 原始论文（IEEE TPAMI）：闭式解与收敛性分析。
- Biber & Strasser 2003 的 NDT 论文：体素高斯与 2D 激光配准。
- Zhang & Singh 2014 的 LOAM 论文（RSS）：特征提取与两阶段框架，建议对照本章的伪代码与图 7.3 阅读。

第八章 视觉里程计与 VIO

本章导读

相机每秒送来几十张图像，每张图里有成千上万个「特征」，本质上是把三维世界压扁成二维。视觉里程计（VO）要做的，就是从这一串二维投影里反推出相机的三维运动。本章按「相机怎么成像 → 特征怎么匹配 → 运动怎么求解」的顺序展开：先把针孔模型与畸变讲透，再推对极几何与三角化，接着用重投影误差把位姿优化写出来，然后介绍直接法与 VIO 的三条主流路线（MSCKF、VINS-Mono、ORB-SLAM3），最后说清单目尺度不可观这一根本局限。学完本章，你应当能看懂 ORB-SLAM 与 VINS 前端在算什么。

8.1 相机模型与畸变

8.1.1 针孔模型

最简单的成像模型是**针孔模型**（pinhole model）：光线从物体出发，穿过光心，落在成像平面上。设相机系 C 的原点在光心，z 轴朝前（光轴方向），x 轴向右、y 轴向下（与图像坐标一致）。空间点 ${}^{C}\boldsymbol{X} = [X, Y, Z]^{\top}$ 投影到成像平面上的坐标为 $(X/Z, Y/Z)$（用相似三角形：成像平面距光心 f，则像高与物高的比等于像距与物距的比）。

像素坐标还要经过缩放与平移。设焦距以像素为单位为 f_x, f_y（$f_x = f/s_x$，s_x 是单个像素的物理宽度），主点（光轴与成像平面的交点）像素坐标为 (c_x, c_y)，则

$$u = f_x \frac{X}{Z} + c_x, \qquad v = f_y \frac{Y}{Z} + c_y. \tag{8.1}$$

写成齐次坐标：

$$Z \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \underbrace{\begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}}_{\boldsymbol{K}} \begin{bmatrix} X \\ Y \\ Z \end{bmatrix}. \tag{8.2}$$

矩阵 $\boldsymbol{K}$ 叫**内参矩阵**（intrinsic matrix），它只与相机本身有关；f_x, f_y, c_x, c_y 叫**内参**。记号 $\pi(\cdot)$ 表示把三维点投影到像素平面，π^{-1} 表示「给定深度」时的反投影。

例题 8.1 一组典型的 USB 相机内参 分辨率 640×480，$f_x = f_y = 500$，$c_x = 320$、$c_y = 240$。空间点 $[1, 0.5, 5]^{\top}$ m 的投影像素为

$$u = 500 \times \frac{1}{5} + 320 = 420, \qquad v = 500 \times \frac{0.5}{5} + 240 = 290, \tag{8.3}$$

即位于图像右半边、中心偏下。若该点深度从 5 m 变到 10 m，像素 u 会从 420 移到 $500/10 + 320 = 370$：深度翻倍，像素向主点靠近一半。

8.1.2 径向与切向畸变

真实镜头不是理想针孔，直线在图像里会变成曲线。工程上采用如下畸变模型：先把像素坐标转成归一化坐标 $x = (u - c_x)/f_x$、$y = (v - c_y)/f_y$，令 $r^2 = x^2 + y^2$，

$$x_{\text{dist}} = x\left(1 + k_1 r^2 + k_2 r^4 + k_3 r^6\right) + 2p_1 xy + p_2\left(r^2 + 2x^2\right), \tag{8.4}$$

$$y_{\text{dist}} = y\left(1 + k_1 r^2 + k_2 r^4 + k_3 r^6\right) + p_1\left(r^2 + 2y^2\right) + 2p_2 xy, \tag{8.5}$$

其中 k_1, k_2, k_3 是**径向畸变**系数（$k_1 < 0$ 表现为桶形畸变），p_1, p_2 是**切向畸变**系数（由镜头与成像面不平行引起）。最后再乘回内参得到畸变后的像素坐标。

去畸变。标定得到的是「畸变后像素 → 归一化坐标」的映射，而后续几何计算需要「无畸变像素」。由于式 (8.4) 不可解析求逆，工程上用迭代法：以畸变点 x_d 为初值，反复执行

$$x \leftarrow \frac{x_d - 2p_1xy - p_2\left(r^2 + 2x^2\right)}{1 + k_1r^2 + k_2r^4 + k_3r^6}, \tag{8.6}$$

（y 同理）直到 x 的变化小于阈值。多数库（OpenCV `undistort`）的做法是预先算好整幅图像的映射表，运行时查表，速度更快。

相机模型的三个易错点

1. 图像 v 轴向下：c_y 与 y 方向不要搞反，否则三角化出的三维点会上下颠倒。
2. 内参来自标定：f_x 与 f_y 可以不相等（非方形像素），不要强行假设相等。
3. 畸变系数作用于**归一化坐标**：先除内参再去畸变，顺序不能反。

8.2 特征点法视觉里程计

VO 的两大流派是特征点法与直接法。本节先讲特征点法：先在图像里找「好认的点」，再在两张图之间匹配它们，最后用匹配点求运动。

定义 8.1 (特征点与描述子匹配)

特征点（feature point / keypoint）是图像中可重复检测、对光照和小视角变化稳定的位置，如角点。描述子（descriptor）是描述该点邻域外观的向量，用于判断两个特征点是否是同一物理点。描述子匹配（descriptor matching）指按描述子距离为两帧特征点建立对应关系，常用欧氏距离（SIFT）或汉明距离（ORB 的二进制描述子）。

检测与描述。FAST 特征检测角点：以像素 p 为中心、半径 3 的圆上取 16 个像素，若其中连续一段（如 9 个）都明显亮于或暗于 p，则 p 是角点。ORB 在 FAST 基础上加了方向（用灰度质心法算主方向）与旋转不变的 BRIEF 二进制描述子（256 bit），匹配时用汉明距离（两串二进制不同位的个数），可以用位运算快速计算。匹配后常做**比值检验**（ratio test）：对每个特征点，若最近与次近描述子距离之比小于 0.7 才接受匹配，这能剔除大量误匹配。

运动求解的两条路。匹配点若都是二维像素，用对极几何求相对位姿（2D-2D）；若其中一帧有深度（RGB-D 或三角化过），用 PnP 求位姿（3D-2D）；若两帧都有三维点，用 ICP（第 7 章）。下面依次推导。

8.2.1 对极几何：2D-2D 的约束

设同一空间点在两帧相机系下的坐标为 $\boldsymbol{X}_1, \boldsymbol{X}_2$，两帧间的位姿变换为 $\boldsymbol{X}_2 = \boldsymbol{R}\boldsymbol{X}_1 + \boldsymbol{t}$，归一化像点为 $\boldsymbol{x}_1 = \boldsymbol{X}_1/Z_1$、$\boldsymbol{x}_2 = \boldsymbol{X}_2/Z_2$（即 $Z[\boldsymbol{x}^\top, 1]^\top = \boldsymbol{X}$）。关键观察：$\boldsymbol{x}_2$、$\boldsymbol{R}\boldsymbol{x}_1$、$\boldsymbol{t}$ 三个向量共面，因为 $\boldsymbol{X}_2 = Z_1\boldsymbol{R}\boldsymbol{x}_1 + \boldsymbol{t}$，而 $\boldsymbol{x}_2$ 与 $\boldsymbol{X}_2$ 平行。三个共面向量的混合积为零：

$$\boldsymbol{x}_2^\top\left(\boldsymbol{t} \times \boldsymbol{R}\boldsymbol{x}_1\right) = 0 \quad \Longleftrightarrow \quad \boldsymbol{x}_2^\top \boldsymbol{t}^\wedge \boldsymbol{R}\,\boldsymbol{x}_1 = 0, \tag{8.7}$$

其中 $\boldsymbol{t}^\wedge$ 是反对称矩阵，满足 $\boldsymbol{t}^\wedge\boldsymbol{x} = \boldsymbol{t} \times \boldsymbol{x}$。式 (8.7) 就是**对极约束**（epipolar constraint），它说明「一对匹配点」就提供了关于 $\boldsymbol{R}, \boldsymbol{t}$ 的一个标量方程。定义

$$\boldsymbol{E} = \boldsymbol{t}^\wedge\boldsymbol{R}, \qquad \boldsymbol{F} = \boldsymbol{K}_2^{-\top}\boldsymbol{E}\,\boldsymbol{K}_1^{-1}, \tag{8.8}$$

其中 $\boldsymbol{E}$ 叫**本质矩阵**（essential matrix），$\boldsymbol{F}$ 叫**基础矩阵**（fundamental matrix，作用在像素坐标上）。$\boldsymbol{E}$ 有 5 个自由度（$\boldsymbol{R}$ 三个、$\boldsymbol{t}$ 方向两个，尺度不可观），$\boldsymbol{F}$ 有 7 个自由度；作为 3×3 矩阵它们有 9 个元素，去掉整体尺度剩 8 个，再减去秩 2 约束正好是 7 个。

八点法。把 $\boldsymbol{F} = [f_1, \ldots, f_9]^\top$ 拉直，每对匹配点给一个线性方程

$$[u_2u_1,\ u_2v_1,\ u_2,\ v_2u_1,\ v_2v_1,\ v_2,\ u_1,\ v_1,\ 1]\,\boldsymbol{f} = 0. \tag{8.9}$$

$N \geq 8$ 对点堆成 $\boldsymbol{A}\boldsymbol{f} = \boldsymbol{0}$，取 $\boldsymbol{A}$ 的最小奇异值对应的右奇异向量作为 $\boldsymbol{f}$。由于噪声，这样得到的 $\boldsymbol{F}$ 通常是满秩的，需要**强制秩 2**：对 $\boldsymbol{F}$ 做 SVD $\boldsymbol{F} = \boldsymbol{U}\,\mathrm{diag}(\sigma_1, \sigma_2, \sigma_3)\boldsymbol{V}^\top$，把最小的 σ_3 置零，得 $\hat{\boldsymbol{F}} = \boldsymbol{U}\,\mathrm{diag}(\sigma_1, \sigma_2, 0)\boldsymbol{V}^\top$。再从 $\hat{\boldsymbol{E}} = \boldsymbol{K}_2^\top \hat{\boldsymbol{F}} \boldsymbol{K}_1$ 分解出 $\boldsymbol{R}, \boldsymbol{t}$（SVD 分解有 4 组解，用「点必须在相机前方」筛选）。工程上通常先用 RANSAC 剔除误匹配，再解 $\boldsymbol{E}$。

8.2.2 三角化

已知 $\boldsymbol{R}, \boldsymbol{t}$ 与匹配点 $\boldsymbol{x}_1, \boldsymbol{x}_2$，要恢复该点的三维坐标。由 $\boldsymbol{X}_2 = \boldsymbol{R}\boldsymbol{X}_1 + \boldsymbol{t}$ 与 $\boldsymbol{X}_i = Z_i[\boldsymbol{x}_i^\top, 1]^\top$：

$$Z_2 \begin{bmatrix} \boldsymbol{x}_2 \\ 1 \end{bmatrix} = Z_1 \boldsymbol{R} \begin{bmatrix} \boldsymbol{x}_1 \\ 1 \end{bmatrix} + \boldsymbol{t}. \tag{8.10}$$

两边左乘 $\boldsymbol{x}_2^\wedge$ 消掉左边的未知量 Z_2：

$$\boldsymbol{0} = Z_1 \boldsymbol{x}_2^\wedge \boldsymbol{R} \begin{bmatrix} \boldsymbol{x}_1 \\ 1 \end{bmatrix} + \boldsymbol{x}_2^\wedge \boldsymbol{t} \implies Z_1 \boldsymbol{x}_2^\wedge \boldsymbol{R} \bar{\boldsymbol{x}}_1 = -\boldsymbol{x}_2^\wedge \boldsymbol{t}, \tag{8.11}$$

其中 $\bar{\boldsymbol{x}}_i = [\boldsymbol{x}_i^\top, 1]^\top$。这是关于 Z_1 的超定线性方程（三个方程，秩 2）。用最小二乘解：两边左乘 $(\boldsymbol{x}_2^\wedge \boldsymbol{R}\bar{\boldsymbol{x}}_1)^\top$，得

$$Z_1 = -\frac{\left(\boldsymbol{x}_2^\wedge \boldsymbol{R}\bar{\boldsymbol{x}}_1\right)^\top \left(\boldsymbol{x}_2^\wedge \boldsymbol{t}\right)}{\left\|\boldsymbol{x}_2^\wedge \boldsymbol{R}\bar{\boldsymbol{x}}_1\right\|^2}. \tag{8.12}$$

代回得 $\boldsymbol{X}_1 = Z_1 \bar{\boldsymbol{x}}_1$，再用 $\boldsymbol{X}_2 = \boldsymbol{R}\boldsymbol{X}_1 + \boldsymbol{t}$ 得到另一帧坐标。注意三角化的精度与视差有关：两帧间基线太短（视差小）时，深度对像素噪声极其敏感，这是单目 VO 在慢速直线运动时深度估计很差的根本原因。

8.2.3 PnP 与重投影误差

当一帧有深度信息时，问题变成 3D-2D：已知三维点 $\boldsymbol{X}_j$ 与像素观测 $\boldsymbol{u}_{ij}$，求相机位姿 $\boldsymbol{T}_i$。最直接的做法是最小化**重投影误差**（reprojection error）：

$$\boldsymbol{e}_{ij} = \boldsymbol{u}_{ij} - \pi\left(\boldsymbol{R}_i \boldsymbol{X}_j + \boldsymbol{t}_i\right), \qquad \min_{\boldsymbol{T}_i} \sum_j \left\|\boldsymbol{e}_{ij}\right\|^2. \tag{8.13}$$

它是 $\boldsymbol{R}$ 的非线性函数，用 Gauss-Newton 迭代。先给出投影对三维点的雅可比。由式 (8.1)，

$$\frac{\partial \pi}{\partial \boldsymbol{p}} = \begin{bmatrix} \dfrac{f_x}{Z} & 0 & -\dfrac{f_x X}{Z^2} \\ 0 & \dfrac{f_y}{Z} & -\dfrac{f_y Y}{Z^2} \end{bmatrix} \in \mathbb{R}^{2\times 3}, \qquad \boldsymbol{p} = \begin{bmatrix} X \\ Y \\ Z \end{bmatrix}. \tag{8.14}$$

再给出三维点对位姿增量 $\delta\boldsymbol{\xi} = [\delta\boldsymbol{\theta}; \delta\boldsymbol{t}]$ 的雅可比。$\boldsymbol{p} = \boldsymbol{R}\boldsymbol{X} + \boldsymbol{t}$，对 $\delta\boldsymbol{\theta}$ 用 $\boldsymbol{R} \leftarrow \exp(\delta\boldsymbol{\theta}^\wedge)\boldsymbol{R} \approx (\boldsymbol{I} + \delta\boldsymbol{\theta}^\wedge)\boldsymbol{R}$：

$$\frac{\partial \boldsymbol{p}}{\partial \delta\boldsymbol{\theta}} = -\boldsymbol{p}^\wedge \in \mathbb{R}^{3\times 3}, \qquad \frac{\partial \boldsymbol{p}}{\partial \delta\boldsymbol{t}} = \boldsymbol{I} \in \mathbb{R}^{3\times 3}. \tag{8.15}$$

链式法则得 2×6 的雅可比

$$\boldsymbol{J}_{ij} = \frac{\partial \boldsymbol{e}_{ij}}{\partial \delta\boldsymbol{\xi}} = -\frac{\partial \pi}{\partial \boldsymbol{p}}\bigg|_{\boldsymbol{p}=\boldsymbol{R}_i\boldsymbol{X}_j+\boldsymbol{t}_i} \begin{bmatrix} -\boldsymbol{p}^\wedge & \boldsymbol{I} \end{bmatrix} \in \mathbb{R}^{2\times 6}. \tag{8.16}$$

正规方程为 $\left(\sum_j \boldsymbol{J}_{ij}^\top \boldsymbol{J}_{ij}\right)\delta\boldsymbol{\xi} = -\sum_j \boldsymbol{J}_{ij}^\top \boldsymbol{e}_{ij}$，与第 7 章的点到面完全同构。若相机内参也未知，可以同时对 $\boldsymbol{K}$ 求导做自标定，工程上一般先离线标定好。

PnP 的初值与外点。重投影误差是局部方法，初值差会发散。常用的初始化方法有 DLT（直接线性变换）与 EPnP：DLT 把投影方程写成

$$u\,\boldsymbol{p}_3^\top \boldsymbol{X} = \boldsymbol{p}_1^\top \boldsymbol{X}, \qquad v\,\boldsymbol{p}_3^\top \boldsymbol{X} = \boldsymbol{p}_2^\top \boldsymbol{X}, \tag{8.17}$$

其中 $\boldsymbol{P} = \boldsymbol{K}[\boldsymbol{R}\ \boldsymbol{t}] \in \mathbb{R}^{3\times 4}$ 有 12 个未知数，$N \geq 6$ 个点即可线性求解（SVD 取零空间）；EPnP 则把三维点表示成 4 个虚拟控制点的加权和，把未知量降到 12 维以内，更快更稳。外点用 RANSAC：随机取最小点集求 PnP，统计内点数，迭代取内点最多的模型，最后用全部内点做一次重投影误差优化。

8.2.4 光束法平差

把多帧、多个点的重投影误差放在一起联合优化，就叫**光束法平差**（bundle adjustment, BA）：

$$\min_{\{T_i\},\{\boldsymbol{X}_j\}} \sum_{(i,j)\in O} \left\| \boldsymbol{u}_{ij} - \pi\left(\boldsymbol{R}_i \boldsymbol{X}_j + \boldsymbol{t}_i\right) \right\|^2, \tag{8.18}$$

其中 O 是所有观测。BA 的变量包含位姿与三维点，规模很大，但雅可比矩阵高度稀疏（每个观测只涉及一个位姿与一个点），可以用舒尔补（Schur complement）把点变量消掉，只解位姿（推导见第 13 章）。视觉 SLAM 的后端本质上就是「带鲁棒核与边缘化的增量 BA」。

算法：特征点法 VO 的单帧流程

1. 输入：当前帧图像 I_k、上一帧状态
2. 提取 FAST 角点，计算 ORB 描述子
3. 与上一帧描述子做匹配，比值检验剔除误匹配
4. RANSAC 估计本质矩阵 $\boldsymbol{E}$（或基础矩阵 $\boldsymbol{F}$），得到初始 $\boldsymbol{R}, \boldsymbol{t}$
5. 三角化匹配点得到三维坐标；若上一帧已有三维点则改做 PnP
6. 以重投影误差做 Gauss-Newton 局部优化，剔除大残差点
7. 输出位姿 T_k，把新三角化的点加入地图

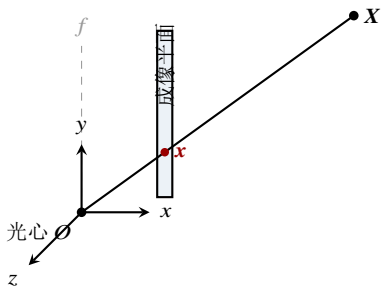


图 8.1: 针孔成像模型。三维点 $\boldsymbol{X}$ 与光心连线交成像平面于像点 $\boldsymbol{x}$，像素坐标由 $u = f_x X/Z + c_x$ 给出；焦距 f、深度 Z 与像高构成相似三角形。

8.3 直接法与光流

特征点法要先检测、匹配，再丢弃大量信息（一幅图只用几百个点）。直接法（direct method）反过来：直接假设「同一个空间点在两帧中的像素灰度不变」，最小化**光度误差**（photometric error）：

$$e_i = I_1\left(\boldsymbol{u}_i\right) - I_2\left(\pi\left(\boldsymbol{R}\,\pi^{-1}\left(\boldsymbol{u}_i, d_i\right) + \boldsymbol{t}\right)\right), \tag{8.19}$$

其中 d_i 是像素 $\boldsymbol{u}_i$ 对应的深度。目标是最小化 $\sum_i e_i^2$，用 Gauss-Newton 对位姿（以及可能的深度）迭代，雅可比通过图像梯度链式得到：

$$\frac{\partial e_i}{\partial \delta\boldsymbol{\xi}} = -\nabla I_2^{\top} \frac{\partial \pi}{\partial \boldsymbol{p}} \left[-\boldsymbol{p}^{\wedge} \;\; \boldsymbol{I}\right]. \tag{8.20}$$

直接法不需要特征，弱纹理区域只要有小梯度就能用，还能利用全部像素；代价是光度误差对光照变化、相机自动曝光、卷帘快门非常敏感，且需要较好的深度初值。DSO [9] 是直接法的代表作，它联合优化光度参数与几何参数，在无纹理、高动态范围场景表现突出。

光流可以看作直接法的「前身」：为每个像素在下一帧里找到灰度最相似的位置（LK 光流），再把这些像素位移当作特征匹配的替代品。光流匹配的位移通常很小，所以适合视频这种帧间运动小的场景。

8.4 VIO：视觉与惯性的融合

定义 8.2 (VIO)

视觉惯性里程计（visual-inertial odometry, VIO）指把相机与 IMU 的数据联合起来估计位姿与速度：相机提供长期无漂移的几何约束（但频率低、有尺度模糊），IMU 提供高频运动与尺度信息（但漂移快）。二者互补，是现代无人机、机器人的主流定位方案。

♣

尺度问题：单目为什么不能定尺度。由式 (8.2)，投影只依赖 X/Z 的比值。若把相机系下所有三维点与平移同时放大 s 倍：$\boldsymbol{X} \to s\boldsymbol{X}$、$\boldsymbol{t} \to s\boldsymbol{t}$，则

$$\frac{s\boldsymbol{X}}{sZ} = \frac{\boldsymbol{X}}{Z}, \tag{8.21}$$

像素投影完全不变。因此单目视觉只能恢复「相差一个未知尺度」的轨迹，速度、位置、地图尺度都不可观。IMU 的加速度计提供绝对加速度（真实物理量），能在运动激励足够时把尺度观测出来，这是 VIO 相对纯单目 VO 的核心优势。

MSCKF。MSCKF（multi-state constraint Kalman filter）[28] 用 EKF 维护一个滑动窗口内的多个相机位姿，特征点不进入状态。当某个特征点离开视野时，它的多次观测给窗口内的位姿提供了一组约束；为了不把特征点坐标加进状态，用**零空间投影**把这些约束投影到「与特征点无关」的子空间，得到一个只含位姿的残差，再做 EKF 更新。好处是状态维度小、计算量可控；代价是信息有损（投影后约束变弱）。

VINS-Mono。VINS-Mono [31] 走优化路线：IMU 预积分（第 6 章）连接相邻关键帧，视觉观测提供重投影残差，二者放进同一个滑动窗口做 BA；窗口滑出时用边缘化（marginalization）把旧状态的信息转成先验，保留约束、丢弃变量。它是「视觉惯性紧耦合 + 滑动窗口优化」的经典实现。

ORB-SLAM3。ORB-SLAM3 [3] 把特征点、多地图（ATLAS）、视觉惯性紧耦合整合在一起，支持单目/双目/RGB-D 与 IMU 的组合，是工程完整度最高的视觉 SLAM 系统之一。它的前端用 ORB 特征与对极几何/PnP，后端用局部 BA + 回环 + 位姿图。

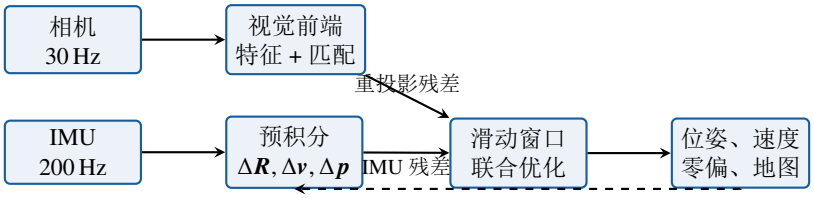


图 8.2: VIO 的信息流。视觉前端给出重投影残差，IMU 预积分给出帧间运动约束，二者在滑动窗口里联合优化；优化结果反过来用于下一轮预积分与边缘化。

8.5 视觉里程计的局限

- **尺度**：单目尺度不可观（上面已证明）；双目/IMU 可以补上，但需要基线或运动激励。
- **光照**：强光、逆光、暗处、曝光变化会让特征/光度误差失效；直接法比特征点法更敏感。
- **纹理**：白墙、地板、大面积单色物体上没有可用的角点或梯度，VO 会退化（与激光的长廊问题本质相同）。
- **动态物体**：行人、其他机器人会引入错误的特征匹配，需要 RANSAC、语义分割或运动一致性检验。
- **滚动快门**：高速旋转时 CMOS 逐行曝光会引入几何畸变，需要专门建模。
- **尺度漂移**：即使有 IMU，长时间慢速运动时尺度仍可能缓慢漂移，需要绝对观测（GPS、地图）来闭合。

这也是为什么真实的定位系统几乎都是「视觉 + IMU + 轮速 + 激光」多传感器融合：没有任何单一传感器能在所有场景下可靠工作（第 9 章）。

选择 VO 方案的三条原则

1. 有纹理、要精度：特征点法 + 回环（ORB-SLAM3 一类）；
2. 弱纹理、要速度：直接法（DSO 一类）或光流；
3. 要尺度、要鲁棒：一定要上 IMU（VIO），纯单目 VO 不适合作为长期定位的唯一来源。

本章小结

1. 针孔模型：$Z[u,v,1]^\top = \boldsymbol{K}[X,Y,Z]^\top$，内参含 f_x, f_y, c_x, c_y；畸变由 k_1, k_2, k_3, p_1, p_2 描述，作用在归一化坐标上。
2. 特征点法：FAST 检测、ORB 描述子、汉明距离匹配与比值检验，是 VO 前端的基础。
3. 对极约束 $\boldsymbol{x}_2^\top \boldsymbol{t}^\wedge \boldsymbol{R}\boldsymbol{x}_1 = 0$ 来自三个向量共面；八点法 + 秩 2 约束求 $\boldsymbol{F}$，再分解出 $\boldsymbol{R}, \boldsymbol{t}$。
4. 三角化把匹配点变成三维点，深度精度强烈依赖视差；视差小则深度不可靠。
5. PnP 最小化重投影误差，雅可比由 $\partial\pi/\partial\boldsymbol{p}$ 与 $[-\boldsymbol{p}^\wedge, \boldsymbol{I}]$ 链式相乘得到；初值可用 DLT/EPnP，外点用 RANSAC。
6. BA 是联合优化多帧位姿与三维点，雅可比稀疏，可用舒尔补消元。
7. 直接法最小化光度误差，适应弱纹理但怕光照变化；DSO 是代表。
8. 单目尺度不可观（投影对整体缩放不变）；VIO 用 IMU 补尺度与高频运动，MSCKF、VINS-Mono、ORB-SLAM3 是三条代表性路线。

第八章 练习

1. 用 $f_x = f_y = 500$、$c_x = c_y = 320$ 的内参，求空间点 $[2,-1,4]^\top$ 的像素坐标；再求 $[4,-2,8]^\top$ 的像素坐标，验证两者相同（说明尺度不可观）。
2. 从 $\boldsymbol{X}_2 = \boldsymbol{R}\boldsymbol{X}_1 + \boldsymbol{t}$ 与 $\boldsymbol{x}_i = \boldsymbol{X}_i / Z_i$ 出发，完整推导对极约束 (8.7)，并说明每一步用了什么性质。
3. 写出八点法的线性方程组，说明为什么最少需要 8 对点，以及为什么解出的 $\boldsymbol{F}$ 需要强制秩 2。
4. 用式 (8.14) 与 $[-\boldsymbol{p}^\wedge, \boldsymbol{I}]$ 推导 $\boldsymbol{J}_{ij}$，并写出 $\boldsymbol{J}_{ij}$ 的形状与每一块的含义。
5. 若两帧间基线为 0.05 m、目标深度 5 m，像素噪声 0.5 像素、$f = 500$，估算三角化深度的相对误差量级。
6. 比较特征点法与直接法的优缺点，并说明为什么 VIO 能解决单目的尺度问题。

延伸阅读

- DSO 论文（IEEE TPAMI 2018）：直接法的光度误差与模型参数联合优化。
- VINS-Mono 论文与开源代码（GitHub）：预积分 + 滑动窗口 + 边缘化的完整实现。
- ORB-SLAM3 论文与代码（GitHub）：多地图与视觉惯性紧耦合的工程范本。

第九章 多传感器融合定位

本章导读

前面五章分别讲了轮式、IMU、激光、视觉四种里程计，每一种都有自己的死穴：轮式怕打滑、IMU 漂移快、激光怕退化、视觉怕光照。真实系统从来不会只押注一个传感器，而是把它们的信息按各自的可信度加权组合——这就是**多传感器融合**。本章先讲清融合的层次与松紧耦合的区别，再把卡尔曼融合、序贯更新与批量更新的等价性推完，接着定义信息矩阵与马氏距离并用卡方检验做外点剔除，最后讲时间同步这个「看起来是工程细节、实际上决定成败」的问题。学完本章，你应当能设计一个多传感器融合的架构，并解释为什么 FAST-LIO 和 VINS 要那么写。

9.1 融合的层次与松紧耦合

9.1.1 三个融合层次

按信息被处理的深度，融合分三个层次：

- **数据级融合**（data-level fusion）：直接融合原始测量。例如把激光点云与相机像素做深度对齐，或把 IMU 与轮速放进同一个滤波器。信息损失最小，但对时间同步与标定要求最高。
- **特征级融合**（feature-level fusion）：各传感器先提取特征（点云特征、图像特征、轮速积分位姿），再融合特征。多数 LIO/VIO 系统属于这一层。
- **决策级融合**（decision-level fusion）：各传感器独立算出位姿，再对结果做加权平均或投票。实现最简单，但丢失了传感器间的相关性信息。

9.1.2 松耦合与紧耦合

定义 9.1 (松耦合与紧耦合)

松耦合（loosely coupled）：每个传感器子系统先独立估计出一个位姿/速度，再把估计结果作为「观测」送进融合滤波器。紧耦合（tightly coupled）：不使用子系统的中间位姿结果，而是把原始测量（原始点云残差、原始像素残差、IMU 预积分）直接放进同一个滤波器或优化器，联合估计状态。

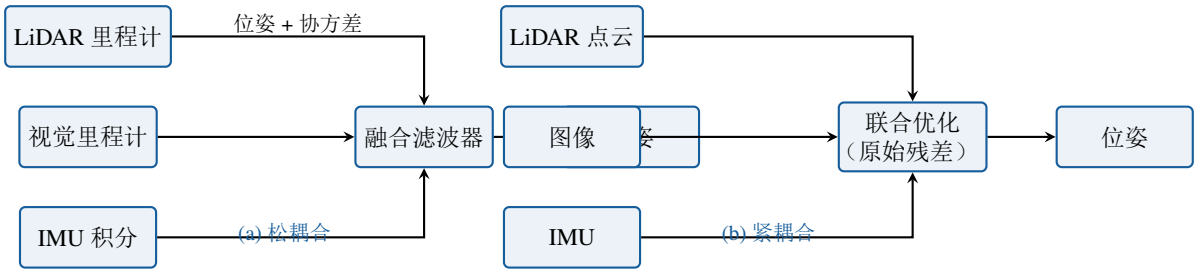


图 9.1: 松耦合与紧耦合。松耦合先把各传感器压成「位姿 + 协方差」再融合，实现简单但会丢失相关性；紧耦合把原始测量直接送入联合估计，信息保留最完整，代价是系统复杂度与标定要求更高。

为什么紧耦合通常更好？松耦合把子系统的估计当作独立观测，但子系统的误差其实与原数据相关（例如激光里程计位姿误差与 IMU 预积分误差共享同一段运动），当成独立会低估不确定性、导致融合权重失真；更重要的是，松耦合在子系统失效时（如激光退化）无法利用其他传感器的原始信息补救。紧耦合则可以在同一个优化里用 IMU 先验补上激光缺失的方向，这正是 FAST-LIO 在长廊里仍能工作的原因（第 16 章）。

9.2 卡尔曼融合的统一形式

9.2.1 状态增广与观测堆叠

多传感器融合最直接的实现是**状态增广**：把所有需要估计的量放进同一个状态向量 $\boldsymbol{x}$，每个传感器的测量都写成 $\boldsymbol{z}_i = \boldsymbol{H}_i\boldsymbol{x} + \boldsymbol{v}_i$。这样问题就退化成「一个状态、多组观测」的卡尔曼滤波。假设先验 $\boldsymbol{x} \sim \mathcal{N}(\hat{\boldsymbol{x}}_0, \boldsymbol{P}_0)$，两组独立观测 $\boldsymbol{z}_1 = \boldsymbol{H}_1\boldsymbol{x} + \boldsymbol{v}_1$、$\boldsymbol{z}_2 = \boldsymbol{H}_2\boldsymbol{x} + \boldsymbol{v}_2$，噪声 $\boldsymbol{v}_i \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{R}_i)$。批量融合的结果是

$$\hat{\boldsymbol{x}} = \left(\boldsymbol{P}_0^{-1} + \boldsymbol{H}_1^{\top}\boldsymbol{R}_1^{-1}\boldsymbol{H}_1 + \boldsymbol{H}_2^{\top}\boldsymbol{R}_2^{-1}\boldsymbol{H}_2\right)^{-1}\left(\boldsymbol{P}_0^{-1}\hat{\boldsymbol{x}}_0 + \boldsymbol{H}_1^{\top}\boldsymbol{R}_1^{-1}\boldsymbol{z}_1 + \boldsymbol{H}_2^{\top}\boldsymbol{R}_2^{-1}\boldsymbol{z}_2\right). \tag{9.1}$$

9.2.2 序贯更新与批量更新的等价性

实现上更常见的是序贯更新：先更新 $\boldsymbol{z}_1$，把结果作为先验再更新 $\boldsymbol{z}_2$。下面证明两者等价。

第一步：更新 $\boldsymbol{z}_1$。令 $\boldsymbol{\Lambda}_0 = \boldsymbol{P}_0^{-1}$、$\boldsymbol{b}_0 = \boldsymbol{P}_0^{-1}\hat{\boldsymbol{x}}_0$。卡尔曼更新的信息形式为

$$\boldsymbol{\Lambda}_1 = \boldsymbol{\Lambda}_0 + \boldsymbol{H}_1^{\top}\boldsymbol{R}_1^{-1}\boldsymbol{H}_1, \qquad \boldsymbol{b}_1 = \boldsymbol{b}_0 + \boldsymbol{H}_1^{\top}\boldsymbol{R}_1^{-1}\boldsymbol{z}_1, \qquad \hat{\boldsymbol{x}}_1 = \boldsymbol{\Lambda}_1^{-1}\boldsymbol{b}_1. \tag{9.2}$$

第二步：以 $\hat{\boldsymbol{x}}_1$ 为先验更新 $\boldsymbol{z}_2$。

$$\boldsymbol{\Lambda}_2 = \boldsymbol{\Lambda}_1 + \boldsymbol{H}_2^{\top}\boldsymbol{R}_2^{-1}\boldsymbol{H}_2, \qquad \boldsymbol{b}_2 = \boldsymbol{b}_1 + \boldsymbol{H}_2^{\top}\boldsymbol{R}_2^{-1}\boldsymbol{z}_2, \qquad \hat{\boldsymbol{x}}_2 = \boldsymbol{\Lambda}_2^{-1}\boldsymbol{b}_2. \tag{9.3}$$

把式 (9.2) 代入：

$$\boldsymbol{\Lambda}_2 = \boldsymbol{P}_0^{-1} + \boldsymbol{H}_1^{\top}\boldsymbol{R}_1^{-1}\boldsymbol{H}_1 + \boldsymbol{H}_2^{\top}\boldsymbol{R}_2^{-1}\boldsymbol{H}_2, \tag{9.4}$$

$$\boldsymbol{b}_2 = \boldsymbol{P}_0^{-1}\hat{\boldsymbol{x}}_0 + \boldsymbol{H}_1^{\top}\boldsymbol{R}_1^{-1}\boldsymbol{z}_1 + \boldsymbol{H}_2^{\top}\boldsymbol{R}_2^{-1}\boldsymbol{z}_2. \tag{9.5}$$

这与式 (9.1) 完全相同。结论：**信息形式下，多观测融合就是信息矩阵与信息向量的相加：**

$$\boldsymbol{\Lambda} = \boldsymbol{P}_0^{-1} + \sum_i \boldsymbol{H}_i^{\top}\boldsymbol{R}_i^{-1}\boldsymbol{H}_i, \qquad \boldsymbol{b} = \boldsymbol{P}_0^{-1}\hat{\boldsymbol{x}}_0 + \sum_i \boldsymbol{H}_i^{\top}\boldsymbol{R}_i^{-1}\boldsymbol{z}_i. \tag{9.6}$$

这个结论带来两个实用推论：第一，观测的**顺序不影响结果**（序贯更新可以任意排列）；第二，每个观测对信息矩阵的贡献是 $\boldsymbol{H}_i^{\top}\boldsymbol{R}_i^{-1}\boldsymbol{H}_i$，与状态维度无关——这就是为什么紧耦合里可以把上千个点云残差直接累加，而不必把它们变成「一个位姿观测」。

松耦合/紧耦合的取舍

1. 松耦合适合「传感器模块已经很成熟、接口稳定」的场合，调试简单、故障隔离好；
2. 紧耦合适合「单传感器会失效、需要互相补位」的场合，精度更高但对标定、时间同步、状态维数要求更高；
3. 工程折中：关键子系统紧耦合（如 IMU + 激光），外围模块松耦合（如全局定位）。

9.3 信息矩阵、马氏距离与卡方检验

9.3.1 信息矩阵

定义 9.2 (信息矩阵)

协方差矩阵 $\boldsymbol{P}$ 的逆 $\boldsymbol{\Lambda} = \boldsymbol{P}^{-1}$ 叫信息矩阵（information matrix）。$\boldsymbol{P}$ 描述不确定性的大小，$\boldsymbol{\Lambda}$ 描述「信息量」的大小：方差越小、信息越多、Λ 越大。在线性高斯系统中，融合后的信息矩阵等于各观测信息矩阵之和（式 (9.6)）。

9.3.2 马氏距离及其卡方分布

考虑一个观测残差 $\boldsymbol{r} = \boldsymbol{z} - \boldsymbol{H}\hat{\boldsymbol{x}}$。不同残差分量量纲不同，不能直接平方相加，需要按各自的不确定性归一化。设噪声与状态误差独立，真实状态为 $\boldsymbol{x}$，先验误差 $\boldsymbol{e} = \hat{\boldsymbol{x}} - \boldsymbol{x} \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{P})$，则

$$\boldsymbol{r} = \boldsymbol{z} - \boldsymbol{H}\hat{\boldsymbol{x}} = (\boldsymbol{H}\boldsymbol{x} + \boldsymbol{v}) - \boldsymbol{H}(\boldsymbol{x} + \boldsymbol{e}) = \boldsymbol{v} - \boldsymbol{H}\boldsymbol{e}. \tag{9.7}$$

它的协方差为（用了 $\boldsymbol{v}$ 与 $\boldsymbol{e}$ 独立、均为零均值）

$$\mathbb{E}\left[\boldsymbol{r}\boldsymbol{r}^{\top}\right] = \mathbb{E}\left[\boldsymbol{v}\boldsymbol{v}^{\top}\right] + \boldsymbol{H}\mathbb{E}\left[\boldsymbol{e}\boldsymbol{e}^{\top}\right]\boldsymbol{H}^{\top} = \boldsymbol{R} + \boldsymbol{H}\boldsymbol{P}\boldsymbol{H}^{\top} = \boldsymbol{S}. \tag{9.8}$$

马氏距离（Mahalanobis distance）定义为用 $\boldsymbol{S}$ 归一化后的平方：

$$d^2 = \boldsymbol{r}^{\top}\boldsymbol{S}^{-1}\boldsymbol{r} = \boldsymbol{r}^{\top}\left(\boldsymbol{H}\boldsymbol{P}\boldsymbol{H}^{\top} + \boldsymbol{R}\right)^{-1}\boldsymbol{r}. \tag{9.9}$$

为什么它服从卡方分布？对 $\boldsymbol{S}$ 做特征分解 $\boldsymbol{S} = \boldsymbol{U}\boldsymbol{\Lambda}_S\boldsymbol{U}^{\top}$（$\boldsymbol{U}$ 正交、$\boldsymbol{\Lambda}_S$ 对角正定），定义

$$\boldsymbol{y} = \boldsymbol{\Lambda}_S^{-1/2}\boldsymbol{U}^{\top}\boldsymbol{r}. \tag{9.10}$$

由于 $\boldsymbol{r}$ 是零均值高斯，$\boldsymbol{y}$ 也是零均值高斯；其协方差为

$$\mathbb{E}\left[\boldsymbol{y}\boldsymbol{y}^{\top}\right] = \boldsymbol{\Lambda}_S^{-1/2}\boldsymbol{U}^{\top}\boldsymbol{S}\,\boldsymbol{U}\boldsymbol{\Lambda}_S^{-1/2} = \boldsymbol{\Lambda}_S^{-1/2}\boldsymbol{\Lambda}_S\boldsymbol{\Lambda}_S^{-1/2} = \boldsymbol{I}. \tag{9.11}$$

即 $\boldsymbol{y}$ 的各分量是独立标准正态变量。于是

$$d^2 = \boldsymbol{r}^{\top}\boldsymbol{S}^{-1}\boldsymbol{r} = \boldsymbol{r}^{\top}\boldsymbol{U}\boldsymbol{\Lambda}_S^{-1}\boldsymbol{U}^{\top}\boldsymbol{r} = \boldsymbol{y}^{\top}\boldsymbol{y} = \sum_{i=1}^{m} y_i^2 \;\sim\; \chi^2(m), \tag{9.12}$$

其中 m 是残差维数。卡方分布的定义见第 2 章。直观上：d^2 是「m 个独立标准正态变量的平方和」，它超过某个阈值的概率可以查表。

用于外点剔除。若观测是内点，d^2 应落在卡方分布的高概率区间；若超出 $1-\alpha$ 分位数，就认为该观测异常并拒绝。对 $m = 3$、$\alpha = 0.05$，阈值 $\chi^2_{3,0.95} = 7.815$；$\alpha = 0.01$ 时阈值 11.345。工程上要注意：外点剔除必须基于「先验 + 观测」的 $\boldsymbol{S}$，而不是只基于观测噪声 $\boldsymbol{R}$，否则会把「先验本来就不确定」的情况误判为外点。

算法：序贯融合与卡方检验

1. 初始化先验 $\hat{\boldsymbol{x}}_0$, $\boldsymbol{P}_0$
2. 对每个传感器观测 $\boldsymbol{z}_i = \boldsymbol{H}_i\boldsymbol{x} + \boldsymbol{v}_i$：
3. 　计算残差 $\boldsymbol{r}_i \leftarrow \boldsymbol{z}_i - \boldsymbol{H}_i\hat{\boldsymbol{x}}$
4. 　计算新息协方差 $\boldsymbol{S}_i \leftarrow \boldsymbol{H}_i\boldsymbol{P}\boldsymbol{H}_i^{\top} + \boldsymbol{R}_i$
5. 　计算 $d_i^2 \leftarrow \boldsymbol{r}_i^{\top}\boldsymbol{S}_i^{-1}\boldsymbol{r}_i$，若 d_i^2 超过卡方阈值则拒绝该观测
6. 　$\boldsymbol{K}_i \leftarrow \boldsymbol{P}\boldsymbol{H}_i^{\top}\boldsymbol{S}_i^{-1}$
7. 　$\hat{\boldsymbol{x}} \leftarrow \hat{\boldsymbol{x}} + \boldsymbol{K}_i\boldsymbol{r}_i$，$\boldsymbol{P} \leftarrow (\boldsymbol{I} - \boldsymbol{K}_i\boldsymbol{H}_i)\boldsymbol{P}$
8. 输出融合位姿与协方差

9.4 因子图视角：融合就是最大后验

卡尔曼滤波是「逐时刻递推」的融合，而因子图（factor graph）是「全局一起解」的融合。把所有测量写成残差 $\boldsymbol{r}_i(\boldsymbol{x})$，假设残差零均值、协方差 $\boldsymbol{\Sigma}_i$，则最大后验估计（MAP）为

$$\hat{\boldsymbol{x}} = \arg\min_{\boldsymbol{x}} \sum_i \|\boldsymbol{r}_i(\boldsymbol{x})\|^2_{\boldsymbol{\Sigma}_i} = \arg\min_{\boldsymbol{x}} \sum_i \boldsymbol{r}_i(\boldsymbol{x})^{\top}\boldsymbol{\Sigma}_i^{-1}\boldsymbol{r}_i(\boldsymbol{x}), \tag{9.13}$$

其中 $\|\boldsymbol{r}\|^2_{\boldsymbol{\Sigma}} = \boldsymbol{r}^{\top}\boldsymbol{\Sigma}^{-1}\boldsymbol{r}$ 是马氏距离平方。对线性高斯情形，每个残差的雅可比为 $\boldsymbol{H}_i$，令梯度为零：

$$\sum_i \boldsymbol{H}_i^{\top}\boldsymbol{\Sigma}_i^{-1}\left(\boldsymbol{r}_i^0 + \boldsymbol{H}_i\delta\boldsymbol{x}\right) = \boldsymbol{0} \;\Longrightarrow\; \left(\sum_i \boldsymbol{H}_i^{\top}\boldsymbol{\Sigma}_i^{-1}\boldsymbol{H}_i\right)\delta\boldsymbol{x} = -\sum_i \boldsymbol{H}_i^{\top}\boldsymbol{\Sigma}_i^{-1}\boldsymbol{r}_i^0. \tag{9.14}$$

左边正是式 (9.6) 的信息矩阵累加。也就是说：**滤波与优化在「信息形式」下是同一个东西**——滤波是优化在马尔可夫假设下的一种增量、递推解法。因子图的详细推导与求解见第 13 章，可参考 [7]。

9.5 时间同步与延时补偿

前面所有公式都隐含一个假设：参与融合的测量是「同一时刻」的。现实中这是最容易出错的地方。

为什么 10 ms 会致命？设机器人以 2 m/s 运动，时间戳错 10 ms，则位姿的对应位置偏差约

$$\delta p \approx v\,\delta t = 2 \times 0.01 = 0.02\,\text{m} = 2\,\text{cm}. \tag{9.15}$$

对激光里程计来说，2 cm 的偏差在配准时会变成系统性残差，滤波器会把它当作「真实运动」去修正，于是状态被持续拖偏；如果时间错的是旋转量（角速度 1 rad/s 错 10 ms 即 0.57°），姿态也会被带偏。更隐蔽的是，时间戳错误往往表现为「缓慢漂移 + 偶发跳变」，很难归因。

补偿方法。设测量时刻为 t，需要把它对齐到统一时刻 t_0。若已有两个时刻的位姿 $T_a(T_a \text{ at } t_a)$、$T_b(T_b \text{ at } t_b)$，用 SE(3) 插值：

$$T(t) = T_a \exp\left(\alpha \log\left(T_a^{-1} T_b\right)\right), \qquad \alpha = \frac{t - t_a}{t_b - t_a}, \tag{9.16}$$

其中 $\log(\cdot)$ 是 $\exp(\cdot)$ 的逆映射（把 $SE(3)$ 元素变回六维 twist）。$\alpha = 0$ 时得到 T_a，$\alpha = 1$ 时得到 T_b，中间按指数映射平滑过渡。若只有速度，也可以做一阶补偿：$T(t) \approx T_a \exp((t - t_a)\boldsymbol{\xi}^\wedge)$。

工程上还有两件事必须做：一是**统一时钟**——相机、IMU、激光的驱动都把时间戳打到同一时钟源（ROS 2 用 `node->now()` 或硬件触发）；二是**检查频率与延迟**——用 `ros2 topic hz` 与 `ros2 topic delay` 确认没有丢帧与过大延迟。ROS 2 的 `message_filters` 提供 `ApproximateTime` 策略，可以在时间戳不完全一致时按最近邻配对，但它只是「配对」，并不能消除真正的延时误差。

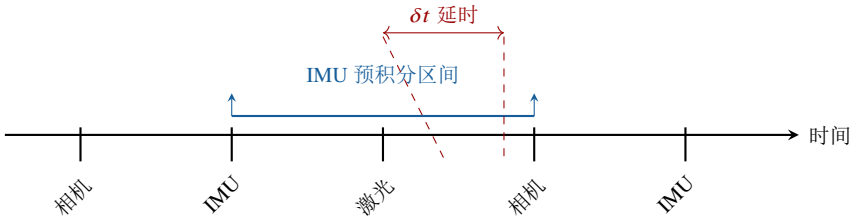


图 9.2: 时间同步示意。不同传感器的时间戳必须对齐到同一时钟；若激光帧的位姿被错误地关联到 δt 之前的状态，残差里就会混入 $v\,\delta t$ 的系统偏差。

9.6 典型融合架构

FAST-LIO（激光 + IMU 紧耦合）[41]：用迭代扩展卡尔曼滤波（IEKF）把激光点到面残差直接作为观测更新，状态含位置、速度、姿态、零偏与重力；IMU 负责预测，激光负责更新；点云通过增量式 KD 树直接配准到地图，不需要提取特征。它的核心优势是把「激光退化方向靠 IMU 先验补」写进了同一个滤波器。详见第 17 章。

VINS-Mono（视觉 + IMU 紧耦合）[31]：视觉重投影残差与 IMU 预积分残差放进滑动窗口联合优化，窗口滑出时边缘化；视觉前端用光流跟踪，后端用 Ceres 求解。详见第 16 章与第 13 章。

Z-Localization（前后端分离）[29]：前端做高频的里程计与局部跟踪，后端做低频的全局匹配与位姿图优化；前后端通过位姿与协方差解耦，兼顾实时性与全局一致性。详见第 23 章。

融合系统的排错顺序

1. 先查时间戳：每个话题的 header.stamp 是否单调、是否来自同一时钟；
2. 再查外参：传感器之间的 T_{SB} 是否标定、符号是否正确；
3. 再查协方差：$\boldsymbol{R}$、$\boldsymbol{Q}$ 的量级是否与真实噪声匹配（过大/过小都会让融合偏科）；

4. 最后才调参数：在数据一致的前提下，融合参数才是有意义的。

本章小结

1. 融合分数据级、特征级、决策级；按接口分松耦合与紧耦合，紧耦合保留原始测量、精度更高，代价是复杂度与标定要求。
2. 状态增广后多传感器融合就是「一个状态、多组观测」；信息形式下融合 = 信息矩阵与信息向量相加：$\boldsymbol{\Lambda} = \boldsymbol{P}_0^{-1} + \sum \boldsymbol{H}_i^{\top} \boldsymbol{R}_i^{-1} \boldsymbol{H}_i$。
3. 序贯更新与批量更新在信息形式下完全等价，观测顺序不影响结果。
4. 信息矩阵 $\boldsymbol{\Lambda} = \boldsymbol{P}^{-1}$；马氏距离 $d^2 = \boldsymbol{r}^{\top}(\boldsymbol{H}\boldsymbol{P}\boldsymbol{H}^{\top} + \boldsymbol{R})^{-1}\boldsymbol{r}$，在零均值高斯假设下服从 $\chi^2(m)$。
5. 卡方检验用于外点剔除：$d^2 > \chi^2_{m,1-\alpha}$ 时拒绝观测；$\boldsymbol{S}$ 必须包含先验不确定性。
6. 因子图把融合写成 MAP：$\min \sum \|\boldsymbol{r}_i\|^2_{\boldsymbol{\Sigma}_i}$，线性化后的正规方程与信息滤波一致。
7. 时间同步是融合成败的关键：2 m/s 下 10 ms 错位就是 2 cm；用 SE(3) 插值补偿，并用统一时钟源避免错误。
8. 典型架构：FAST-LIO（激光-IMU 紧耦合）、VINS-Mono（视觉-IMU 紧耦合）、Z-Localization（前后端分离）。

第九章 练习

1. 用一维情形验证式 (9.1)：两个观测 $z_1 = x + v_1$、$z_2 = x + v_2$，方差分别为 r_1, r_2，先验方差 p_0，分别用批量与序贯公式求后验方差，验证相同。
2. 证明式 (9.6)：从卡尔曼更新的信息形式出发，写出 $\boldsymbol{P}^{-1}$ 的更新式，并说明为什么观测顺序无关。
3. 推导式 (9.7) 与式 (9.8)，说明为什么 $\boldsymbol{r}$ 的协方差不是 $\boldsymbol{R}$。
4. 对 $m = 2$、$\alpha = 0.05$ 求卡方阈值（查表或记忆），并解释为什么外点剔除阈值不能直接用 $\boldsymbol{R}$ 归一化。
5. 某系统相机在 $t = 1.000$ s、IMU 在 $t = 1.008$ s 各有一组数据，机器人速度 1.5 m/s、角速度 0.8 rad/s，估算不对齐造成的位移与角度偏差。
6. 解释为什么「FAST-LIO 在长廊里仍能用」而「纯激光里程计会退化」，用本章的紧耦合概念与第 7 章的退化分析回答。

延伸阅读

- Thrun 等《Probabilistic Robotics》第 2–3 章：高斯滤波与信息形式的经典推导。
- Dellaert 的因子图教程（gtsam.org）：从 MAP 到因子图的完整视角。
- FAST-LIO2 论文与代码（GitHub）：紧耦合 IEKF 的工程实现。

第十章 全局定位与重定位

本章导读

里程计回答的是「我相对刚才动了多少」，但它永远回答不了「我现在在地图的哪里」。机器人可能被放到赛场的任意角落开机，可能在比赛中被撞得转了三圈，也可能在长廊里迷失——这些情况都需要**全局定位**：在一张先验地图中确定自己的绝对位姿。本章讲清楚三件事：为什么里程计无法解决全局定位、粒子滤波为什么是解决它的标准工具（从贝叶斯递归一路推到重采样）、以及 AMCL 这个 ROS 里最常见的实现是怎么做的。学完本章，你应当能解释「绑架问题」为什么难，以及 KLD 采样公式里每个符号的含义。

10.1 局部定位与全局定位

第 4 章已经区分了绝对定位与相对定位。从地图的使用方式看，还有一个更实用的分法：

- **局部定位**（local localization，也叫位姿跟踪）：已知初始位姿（或初始位姿误差很小），在运动过程中持续修正。它依赖局部地图匹配，初值附近只有一个「吸引域」，一旦偏差超过这个范围就会跟丢。
- **全局定位**（global localization）：初始位姿未知，需要在地图全局搜索所有可能的位姿，并从中挑出与当前观测一致的解。

里程计只能做局部定位：它提供的位姿是相对初始时刻的，初始时刻在地图里的位置它并不知道；而且里程计的漂移是累积的，时间越长，它离真实位姿越远，最终会超出任何匹配算法的收敛域。所以全局定位必须有「地图先验」和「全局搜索」两样东西。

绑架问题（kidnapped robot problem）是全局定位的极端版本：机器人在正常运行时被突然「瞬移」到地图的另一个位置（现实中可能是被搬起、被撞飞、或开机后被人挪动）。此时它自己的里程计还停留在旧位置，所有基于「上一时刻位姿」的跟踪都会失效，必须能够主动怀疑自己、重新搜索全局。

10.2 基于地图的定位方法

按匹配与搜索的方式，基于地图的定位可以分为四类：

- **栅格匹配**：把激光扫描与占据栅格地图做相关匹配，在位姿空间里搜索得分最高的位置。2D 场景下典型代表是相关性扫描匹配（第 7 章）。
- **特征匹配**：从地图与当前扫描中提取几何特征（直线、角点、平面），用特征对应关系解算位姿。结构化场景（走廊、房间）效果好，但特征提取本身可能失败。
- **描述子检索 + 配准**：先用地点的全局描述子在地图数据库中检索候选位置，再做局部配准。Scan Context [23] 是典型代表：把点云按「环—扇区」划分成二维矩阵，每个元素取该区域最大高度，得到一个对旋转敏感、对平移鲁棒的描述子；检索时按列移位匹配，找到最相似的历史地点。这类方法适合大场景重定位与回环检测。
- **粒子滤波**：用一组带权样本表示位姿的置信度分布，能同时维护多个假设，天然适合全局定位与绑架恢复。AMCL（adaptive Monte Carlo localization）就是它的工程实现 [12]。

本章重点讲粒子滤波，因为它是唯一能同时处理「多峰分布」与「全局搜索」的通用框架，也是理解 AMCL、Cartographer 定位模式、以及第 23 章重定位模块的基础。

10.3 贝叶斯滤波递归

全局定位的本质是「在给定历史观测与运动的情况下，推断当前位姿的后验分布」。记时刻 t 的位姿为 $\boldsymbol{x}_t$，控制/里程计输入为 $\boldsymbol{u}_t$，观测为 $\boldsymbol{z}_t$，历史为 $\boldsymbol{z}_{1:t}=\{\boldsymbol{z}_1,\dots,\boldsymbol{z}_t\}$、$\boldsymbol{u}_{1:t}$。定义**置信度**（belief）

$$\mathrm{bel}(\boldsymbol{x}_t)=p\left(\boldsymbol{x}_t \mid \boldsymbol{z}_{1:t},\boldsymbol{u}_{1:t}\right). \tag{10.1}$$

下面从条件概率与贝叶斯公式把它写成递推形式。由条件概率定义 $p(a\mid b)=p(a,b)/p(b)$，有

$$p\left(\boldsymbol{x}_t \mid \boldsymbol{z}_{1:t},\boldsymbol{u}_{1:t}\right)=\frac{p\left(\boldsymbol{z}_t \mid \boldsymbol{x}_t,\boldsymbol{z}_{1:t-1},\boldsymbol{u}_{1:t}\right)p\left(\boldsymbol{x}_t \mid \boldsymbol{z}_{1:t-1},\boldsymbol{u}_{1:t}\right)}{p\left(\boldsymbol{z}_t \mid \boldsymbol{z}_{1:t-1},\boldsymbol{u}_{1:t}\right)}. \tag{10.2}$$

分母只与观测有关、与 $\boldsymbol{x}_t$ 无关，可以写成一个归一化常数 η。对第一个因子用**马尔可夫假设**（观测只依赖当前状态）：$p(\boldsymbol{z}_t\mid \boldsymbol{x}_t,\boldsymbol{z}_{1:t-1},\boldsymbol{u}_{1:t})=p(\boldsymbol{z}_t\mid\boldsymbol{x}_t)$。对第二个因子，把 $\boldsymbol{x}_{t-1}$ 作为中间变量引入全概率公式：

$$p\left(\boldsymbol{x}_t \mid \boldsymbol{z}_{1:t-1},\boldsymbol{u}_{1:t}\right)=\int p\left(\boldsymbol{x}_t \mid \boldsymbol{x}_{t-1},\boldsymbol{z}_{1:t-1},\boldsymbol{u}_{1:t}\right)p\left(\boldsymbol{x}_{t-1} \mid \boldsymbol{z}_{1:t-1},\boldsymbol{u}_{1:t}\right)\mathrm{d}\boldsymbol{x}_{t-1}. \tag{10.3}$$

再用一次马尔可夫假设：$\boldsymbol{x}_t$ 只依赖 $\boldsymbol{x}_{t-1}$ 与当前输入 $\boldsymbol{u}_t$，即 $p(\boldsymbol{x}_t\mid\boldsymbol{x}_{t-1},\boldsymbol{z}_{1:t-1},\boldsymbol{u}_{1:t})=p(\boldsymbol{x}_t\mid\boldsymbol{x}_{t-1},\boldsymbol{u}_t)$；且 $\boldsymbol{x}_{t-1}$ 不依赖未来的输入 $\boldsymbol{u}_t$，故 $p(\boldsymbol{x}_{t-1}\mid\boldsymbol{z}_{1:t-1},\boldsymbol{u}_{1:t})=\mathrm{bel}(\boldsymbol{x}_{t-1})$。代回式 (10.2) 得

$$\mathrm{bel}(\boldsymbol{x}_t)=\eta\,p\left(\boldsymbol{z}_t \mid \boldsymbol{x}_t\right)\int p\left(\boldsymbol{x}_t \mid \boldsymbol{x}_{t-1},\boldsymbol{u}_t\right)\mathrm{bel}(\boldsymbol{x}_{t-1})\mathrm{d}\boldsymbol{x}_{t-1}. \tag{10.4}$$

式 (10.4) 就是**贝叶斯滤波递归**：$\int p(\boldsymbol{x}_t\mid\boldsymbol{x}_{t-1},\boldsymbol{u}_t)\mathrm{bel}(\boldsymbol{x}_{t-1})\mathrm{d}\boldsymbol{x}_{t-1}$ 是「预测」，用运动模型把上一时刻的置信度推演到当前时刻；乘以 $p(\boldsymbol{z}_t\mid\boldsymbol{x}_t)$ 是「更新」，用当前观测给预测加权；η 是归一化。卡尔曼滤波、粒子滤波、直方图滤波都是它的不同实现：区别只在「怎么表示 bel」。

10.4 粒子滤波

贝叶斯递归里的积分一般没有解析解，粒子滤波的解法是：用 N 个带权样本（粒子）近似置信度：

$$\mathrm{bel}(\boldsymbol{x}_t)\approx\sum_{i=1}^{N}w_t^i\,\delta\left(\boldsymbol{x}_t-\boldsymbol{x}_t^i\right), \tag{10.5}$$

其中 $\boldsymbol{x}_t^i$ 是第 i 个粒子（一个位姿假设），w_t^i 是它的归一化权重（$\sum_i w_t^i=1$），δ 是狄拉克函数。位姿的估计值可以取加权均值，也可以取权重最大的粒子。

10.4.1 重要性采样

直接按 bel 采样做不到（bel 本身就是要估计的量）。**重要性采样**（importance sampling）的思路是：从一个容易采样的**提议分布**（proposal distribution）$q(\boldsymbol{x})$ 采样，再用权重修正它与 bel 的差异。对任意函数 f，其期望为

$$\begin{aligned}\mathbb{E}_{\mathrm{bel}}\left[f(\boldsymbol{x})\right]&=\int f(\boldsymbol{x})\,\mathrm{bel}(\boldsymbol{x})\mathrm{d}\boldsymbol{x}\\&=\int f(\boldsymbol{x})\,\frac{\mathrm{bel}(\boldsymbol{x})}{q(\boldsymbol{x})}\,q(\boldsymbol{x})\mathrm{d}\boldsymbol{x}\\&=\mathbb{E}_q\left[f(\boldsymbol{x})\,w(\boldsymbol{x})\right],\qquad w(\boldsymbol{x})=\frac{\mathrm{bel}(\boldsymbol{x})}{q(\boldsymbol{x})}.\end{aligned} \tag{10.6}$$

第二个等号是在积分里「乘一个 q 再除一个 q」，前提是 $q(\boldsymbol{x})>0$ 在所有 $\mathrm{bel}(\boldsymbol{x})>0$ 处成立。实践中 bel 只知道到常数因子（归一化常数未知），因此改用未归一化权重 $\tilde{w}(\boldsymbol{x})=\mathrm{bel}(\boldsymbol{x})/q(\boldsymbol{x})$，再归一化：

$$w^i=\frac{\tilde{w}^i}{\sum_{j=1}^{N}\tilde{w}^j}. \tag{10.7}$$

10.4.2 序贯重要性采样

把重要性采样用到贝叶斯递归上，得到**序贯重要性采样**（sequential importance sampling, SIS）。关键选择：用「运动模型」作为提议分布，即从 $q(\boldsymbol{x}_t\mid\boldsymbol{x}_{t-1}^i,\boldsymbol{u}_t)=p(\boldsymbol{x}_t\mid\boldsymbol{x}_{t-1}^i,\boldsymbol{u}_t)$ 采样当前粒子。此时权重可以递推。由权重定

义

$$w_t^i = \frac{\mathrm{bel}(\boldsymbol{x}_t^i)}{q(\boldsymbol{x}_t^i \mid \boldsymbol{x}_{t-1}^i, \boldsymbol{u}_t)}. \tag{10.8}$$

把式 (10.4) 的 $\mathrm{bel}(\boldsymbol{x}_t^i) \propto p(\boldsymbol{z}_t \mid \boldsymbol{x}_t^i) \int p(\boldsymbol{x}_t^i \mid \boldsymbol{x}_{t-1}, \boldsymbol{u}_t)\mathrm{bel}(\boldsymbol{x}_{t-1})\mathrm{d}\boldsymbol{x}_{t-1}$ 代入，并用粒子近似把积分写成求和：

$$\mathrm{bel}(\boldsymbol{x}_t^i) \propto p\left(\boldsymbol{z}_t \mid \boldsymbol{x}_t^i\right) \sum_{j=1}^{N} w_{t-1}^j \, p\left(\boldsymbol{x}_t^i \mid \boldsymbol{x}_{t-1}^j, \boldsymbol{u}_t\right). \tag{10.9}$$

对我们从 $\boldsymbol{x}_{t-1}^i$ 采样得到的那个粒子，求和里只有 $j = i$ 的项被采样到，其余项在粒子近似下不贡献，于是

$$\mathrm{bel}(\boldsymbol{x}_t^i) \propto p\left(\boldsymbol{z}_t \mid \boldsymbol{x}_t^i\right) w_{t-1}^i \, p\left(\boldsymbol{x}_t^i \mid \boldsymbol{x}_{t-1}^i, \boldsymbol{u}_t\right). \tag{10.10}$$

把它代回权重定义，提议分布恰好把这个运动模型因子约掉：

$$w_t^i \propto \frac{p\left(\boldsymbol{z}_t \mid \boldsymbol{x}_t^i\right) w_{t-1}^i \, p\left(\boldsymbol{x}_t^i \mid \boldsymbol{x}_{t-1}^i, \boldsymbol{u}_t\right)}{p\left(\boldsymbol{x}_t^i \mid \boldsymbol{x}_{t-1}^i, \boldsymbol{u}_t\right)} = w_{t-1}^i \, p\left(\boldsymbol{z}_t \mid \boldsymbol{x}_t^i\right). \tag{10.11}$$

这就是粒子滤波最常用的权重更新式：**新权重 = 旧权重 × 观测似然**。它把「运动模型采样」与「观测加权」两件事分得干干净净。归一化后即得式 (10.7)。

粒子滤波的三步循环

1. **预测**：对每个粒子，按运动模型 $p(\boldsymbol{x}_t \mid \boldsymbol{x}_{t-1}^i, \boldsymbol{u}_t)$ 采样一个新位姿（把里程计增量加上噪声）；
2. **加权**：用观测模型算 $p(\boldsymbol{z}_t \mid \boldsymbol{x}_t^i)$，乘到旧权重上并归一化；
3. **重采样**：按权重重新抽取 N 个粒子，权重重置为 $1/N$。

循环往复，粒子会逐渐聚集到与观测一致的位置。

10.4.3 重采样与样本贫化

只加权不重采样，运行一段时间后会出现**权重退化**：绝大多数粒子的权重趋近于零，有效粒子数急剧减少，计算资源被浪费在无意义的粒子上。重采样的做法是「按权重的比例复制粒子」：权重大的粒子被多复制，权重小的被淘汰。重采样后所有粒子权重相等，再继续下一轮。

重采样也有代价：它会让粒子多样性下降（同一个高权重粒子被复制多份），长期运行后可能出现**样本贫化**（sample impoverishment）——粒子全都挤在一个假设附近，丧失探索其他可能位置的能力。缓解手段包括：只在必要时重采样、加入少量随机粒子、使用低方差重采样算法。

何时重采样？用**有效粒子数**（effective sample size）判断：

$$N_{\mathrm{eff}} = \frac{1}{\sum_{i=1}^{N} (w^i)^2}. \tag{10.12}$$

它的含义是：这组带权粒子「等效于」多少个等权粒子。当所有权重相等（$w^i = 1/N$）时，$N_{\mathrm{eff}} = N$；当一个粒子独占权重时，$N_{\mathrm{eff}} = 1$。典型做法是 $N_{\mathrm{eff}} < N/2$ 时才重采样。

系统重采样是最常用的低方差重采样算法：把权重累积成区间 $[0, 1)$，产生一个在 $[0, 1/N)$ 内均匀分布的随机数 u，然后依次取 $u,\ u + 1/N,\ u + 2/N, \ldots$，看每个点落在哪个粒子的累积区间里，就复制该粒子。它只用一次随机数，方差比「独立重复抽样」小。

算法：系统重采样

1. 输入：粒子 $\{\boldsymbol{x}^i\}_{i=1}^N$ 与权重 $\{w^i\}$（$\sum_i w^i = 1$）
2. 计算累积权重：$c_1 \leftarrow w^1$，$c_i \leftarrow c_{i-1} + w^i$
3. 采样起点 $u \sim \mathcal{U}\,[0,\ 1/N)$，令 $r \leftarrow u$
4. 对 $k = 1, \ldots, N$：
5. 　找到最小的 i 使 $c_i \geq r$，复制粒子 $\boldsymbol{x}^i$ 到新集合

6. $r \leftarrow r + 1/N$

7. 输出：新粒子集合，权重全部置为 $1/N$

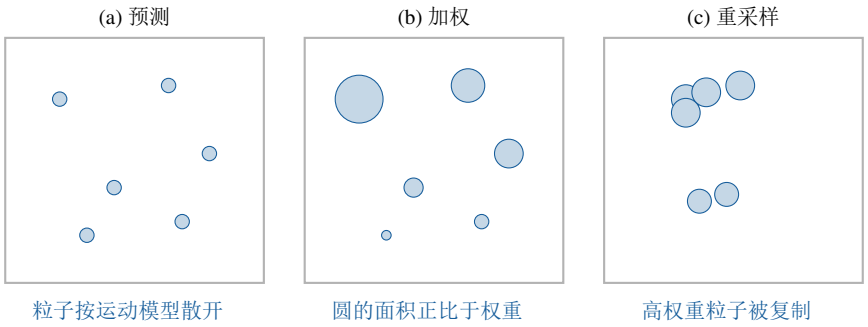


图 10.1: 粒子滤波的一轮迭代：预测让粒子按运动模型散开并加入噪声；加权让与观测一致的粒子变大；重采样把高权重粒子复制、低权重粒子淘汰。

10.5 AMCL：工程化的粒子滤波

AMCL [12] 是 ROS 中实现 2D 全局定位的标准模块。它的输入是占据栅格地图、激光扫描与里程计，输出是 `map`→`odom` 变换（以及位姿协方差）。它把粒子滤波拆成三个具体模型。

10.5.1 运动模型：从里程计增量采样

里程计给出相邻时刻的位姿增量，把它分解成三段：先原地转 δ_{rot1}，再直行 δ_{trans}，最后原地转 δ_{rot2}。真实运动与里程计读数有偏差，AMCL 按如下方式加噪声再执行（α_1–α_4 是噪声系数）：

$$\begin{aligned}
\hat{\delta}_{\text{rot1}} &= \delta_{\text{rot1}} + \text{noise}\left(\alpha_1 \delta_{\text{rot1}}^2 + \alpha_2 \delta_{\text{trans}}^2\right), \\
\hat{\delta}_{\text{trans}} &= \delta_{\text{trans}} + \text{noise}\left(\alpha_3 \delta_{\text{trans}}^2 + \alpha_4 \left(\delta_{\text{rot1}}^2 + \delta_{\text{rot2}}^2\right)\right), \\
\hat{\delta}_{\text{rot2}} &= \delta_{\text{rot2}} + \text{noise}\left(\alpha_1 \delta_{\text{rot2}}^2 + \alpha_2 \delta_{\text{trans}}^2\right),
\end{aligned} \tag{10.13}$$

其中 $\text{noise}(\sigma^2)$ 表示从零均值高斯分布中采样，方差为 σ^2。这个模型的物理含义是：转得越多、走得越远，误差越大；而且「走」与「转」的误差相互耦合（走的时候会带一点转，转的时候会带一点走）。

10.5.2 观测模型：似然场

给定一个粒子位姿，怎么算「在这个位姿下看到当前激光扫描的概率」？AMCL 用**似然场**（likelihood field）模型：对每条激光束，取它的末端点在全局地图中的位置，查「该点到最近障碍物的距离」d，然后

$$p\left(z_t \mid \boldsymbol{x}_t^i\right) = \prod_{k=1}^{K} \left[z_{\text{hit}} \cdot \exp\left(-\frac{d_k^2}{2\sigma_{\text{hit}}^2}\right) + \frac{z_{\text{rand}}}{z_{\text{max}}} \right], \tag{10.14}$$

其中 K 是激光束数，d_k 是第 k 条束末端到最近障碍的距离，z_{hit} 是「命中」权重、z_{rand} 是随机噪声权重，z_{max} 是最大量程。这个模型的好处是不需要做射线投射（ray casting），只需查一张预先算好的距离场（ESDF 的思想），速度快；缺点是它对「光束穿过墙」不敏感（只看到端点）。

10.5.3 KLD 采样：自适应粒子数

粒子数 N 怎么定？固定值要么太浪费（简单场景），要么不够用（复杂场景）。AMCL 采用 **KLD 采样**（Kullback-Leibler divergence sampling）：边采样边观察「落到了多少个不同的状态格（bin）」k，当 k 不再明显增长时停止。

所需粒子数由下式给出：

$$n = \frac{1}{2\varepsilon^2}\,\chi^2_{k-1,\,1-\delta}, \tag{10.15}$$

其中各符号含义如下：

- ε：允许的误差尺度。本书按提纲约定，把它对应的 KL 散度上界记作 ε^2（有的文献直接把该上界记作 ε，公式就写成 $n = \chi^2_{k-1,1-\delta}/(2\varepsilon)$，两者只差一个记号约定）；
- δ：允许的失败概率，即真实分布与样本分布偏离超过上界的概率；
- k：当前已经被采样到的不同 bin 的数量，bin 越多样本越分散，需要的粒子越多；
- $\chi^2_{k-1,1-\delta}$：自由度为 $k-1$ 的卡方分布的 $1-\delta$ 分位数（卡方分布见第 2 章）。

这个公式来自多项分布的统计推断：k 个 bin 上的样本计数服从多项分布，其 Pearson 统计量 $\sum_j (N_j - np_j)^2/(np_j)$ 近似服从 $\chi^2(k-1)$；要让经验分布与真实分布的 KL 散度以概率 $1-\delta$ 不超过上界，就需要 n 至少取上式。直观上：bin 越多、要求越准、要求越有把握，粒子就越多。

例题 10.1KLD 采样的粒子数 设当前采样到 $k = 30$ 个 bin，$\delta = 0.01$（要求 99 % 的把握），查表得 $\chi^2_{29,0.99} \approx 49.6$。若取 $\varepsilon = 0.2$，则

$$n = \frac{49.6}{2 \times 0.2^2} = \frac{49.6}{0.08} \approx 620. \tag{10.16}$$

也就是说这个场景下 600 多个粒子就够了。若 ε 缩小到 0.1（精度要求翻倍），粒子数会变成约 2480——这说明「精度」与「计算量」是平方关系，调参时要格外小心。

10.6 重定位与绑架恢复

有了全局定位的框架，重定位的具体策略还需要考虑工程可行性：

- **初始化**：机器人开机时，把粒子均匀撒在地图的可行区域（或按已知的初始区域撒），第一帧观测后权重会迅速集中在少数假设上。若初始位置已知（如比赛固定起点），可以直接在起点附近撒，收敛更快。
- **丢失恢复**：持续监控粒子权重与观测似然。如果长时间似然很低、N_{eff} 很小，说明当前假设与观测不符，可能是跟丢了；此时注入随机粒子（在全局或局部重新撒），让滤波器有机会「重新找到自己」。
- **多假设**：人在长廊里也会犹豫，粒子滤波天然维护多个假设。不要过早把粒子聚成一个——在对称场景（两条相似的走廊）里，保留多个峰比强行选一个更安全。
- **开关式重定位**：把「局部跟踪」与「全局重定位」做成两个模式，局部跟踪置信度低时切到全局模式，置信度恢复后切回。这是第 23 章 Z-Localization 前后端分离思想的一种体现。

全局定位的四个工程要点

1. 粒子数要自适应：用 KLD 采样，别硬编码一个巨大值；
2. 观测模型要快：似然场查表比射线投射快一个数量级；
3. 重采样要克制：N_{eff} 低于阈值再重采样，避免样本贫化；
4. 要知道自己可能错：永远保留随机粒子与多假设，才能从绑架中恢复。

本章小结

1. 局部定位是位姿跟踪，全局定位要在整张地图上搜索；里程计只能做局部，因为它不知道初始位姿且会累积漂移。
2. 基于地图的定位分栅格匹配、特征匹配、描述子检索 + 配准、粒子滤波四类；Scan Context 是描述子检索的代表。
3. 贝叶斯滤波递归：$\text{bel}(\boldsymbol{x}_t) = \eta\, p(\boldsymbol{z}_t \mid \boldsymbol{x}_t) \int p(\boldsymbol{x}_t \mid \boldsymbol{x}_{t-1}, \boldsymbol{u}_t)\text{bel}(\boldsymbol{x}_{t-1})\mathrm{d}\boldsymbol{x}_{t-1}$，由条件概率、贝叶斯公式与两处马尔可夫假设推出。
4. 粒子滤波用带权样本近似置信度；序贯重要性采样的权重递推为 $w_t^i \propto w_{t-1}^i\, p(\boldsymbol{z}_t \mid \boldsymbol{x}_t^i)$。

5. 重采样按权重复制粒子，代价是样本贫化；用 $N_{\text{eff}} = 1/\sum (w^i)^2$ 判断是否需要重采样，系统重采样方差最小。
6. AMCL 由运动模型采样、似然场观测模型、KLD 自适应粒子数三部分组成；KLD 公式 $n = \chi^2_{k-1,1-\delta}/(2\varepsilon^2)$ 中 ε 是误差尺度、δ 是失败概率、k 是 bin 数。
7. 绑架问题需要「怀疑自己」的机制：随机粒子注入、多假设、开关式重定位是常用手段。

第十章 练习

1. 从条件概率定义出发，重推贝叶斯滤波递归式 (10.4)，并在每一步旁标注用了哪条马尔可夫假设。
2. 解释为什么提议分布取运动模型时，权重递推式 (10.11) 里没有运动模型项；如果提议分布改成「带噪声的运动模型」，权重会变成什么形式？
3. 设 $N = 5$，权重为 $\{0.4, 0.3, 0.2, 0.05, 0.05\}$，计算 N_{eff}，并手动执行一次系统重采样（给定 $u = 0.03$），写出新粒子集合。
4. 用式 (10.15) 计算 $k = 50$、$\delta = 0.05$、$\varepsilon = 0.15$ 时的粒子数（$\chi^2_{49,0.95} \approx 66.3$）。
5. 说明似然场模型式 (10.14) 中 z_{hit}、z_{rand}、z_{max} 各自的物理作用，以及 d_k 是怎么算出来的。
6. 设计一个「绑架检测」方案：用哪些量、什么阈值、检出后如何恢复？请结合本章内容说明。

延伸阅读

- Thrun 等《Probabilistic Robotics》第 8 章：粒子滤波与 Monte Carlo 定位的系统推导。
- Fox 等 2003 年的 AMCL 论文（IEEE TPAMI）：似然场模型与 KLD 采样。
- Scan Context 论文与代码（GitHub）：大场景地点检索的工程实现。

第三部分

SLAM 篇：前端、后端与建图

第十一章 SLAM 框架：前端与后端

本章导读

从这一章开始进入 SLAM 篇。前面两部分我们先后回答了三个问题：位置、姿态与坐标变换怎么描述（第 2 章），传感器能测到什么（第 3 章），以及只靠里程计能不能定位（第 4 章至第 10 章）。里程计给出的答案有一个绕不开的宿命：误差随时间累积，而它自己没有任何机制把误差“拉回来”。SLAM 要回答的正是这个问题——在没有先验地图的未知环境里，机器人能否一边估计自己的轨迹、一边把地图建出来？本章不写具体算法，而是先把骨架立起来：把 SLAM 写成最大后验（MAP）估计，定义前端与后端，讲清它们各自负责什么、凭什么可以分开，并特别澄清一个新手最容易混淆的点——在 SLAM、定位系统、导航系统三种语境下，“前端/后端”指的并不是同一件事。读完本章，你应该能看懂任何一套 SLAM 系统的框图，并在读源码时迅速判断“这段代码属于哪一层”。

11.1 先有鸡还是先有蛋：SLAM 到底难在哪

先做一个思想实验。假设机器人手里已经有一张精确的地图 $\mathcal{M}$，那么定位就是回答“在当前观测 $\boldsymbol{z}$ 下，我在哪”，即求后验 $p(\boldsymbol{x} \mid \boldsymbol{z}, \mathcal{M})$。这正是第 10 章的粒子滤波定位在做的事：地图是已知的，状态只有一个位姿。

反过来，假设有人告诉机器人它每一时刻的精确位姿 $\boldsymbol{x}_0, \boldsymbol{x}_1, \ldots, \boldsymbol{x}_N$，那么建图就是把这些位姿下的观测“贴”到地图上做融合，这正是第 14 章要讲的占据栅格、TSDF 等内容：位姿是已知的，地图是待求量。

SLAM 的处境是两者都没有。机器人既不知道位姿，也没有地图，只能拿着一串观测 $\boldsymbol{Z} = \{\boldsymbol{z}_1, \ldots, \boldsymbol{z}_M\}$ 和一串输入 $\boldsymbol{U} = \{\boldsymbol{u}_1, \ldots, \boldsymbol{u}_N\}$（轮速、IMU 读数等），指望同时把它们都估出来。这就是著名的“鸡生蛋”困境：定位需要地图，建图需要位姿。

定义 11.1 (SLAM)

同时定位与建图（Simultaneous Localization and Mapping，SLAM）是指：机器人在未知环境中运动时，仅利用自身携带的传感器观测（可包含里程计等 proprioceptive 输入），在线地、同时地估计自身位姿轨迹并构建环境地图的问题。

把它写成联合后验的形式，就是本章所有内容的出发点：

$$p(X, \mathcal{M} \mid \boldsymbol{Z}, \boldsymbol{U}), \qquad X = \{\boldsymbol{x}_0, \boldsymbol{x}_1, \ldots, \boldsymbol{x}_N\}, \quad \mathcal{M} = \text{地图}. \tag{11.1}$$

式 (11.1) 中 X 是待估轨迹，$\mathcal{M}$ 是待估地图，条件是全部观测 $\boldsymbol{Z}$ 与输入 $\boldsymbol{U}$。这个式子看着简单，却藏着两个根本困难。

困难一：维度爆炸。一张 100 m × 100 m、分辨率 5 cm 的二维栅格地图有 $2000 \times 2000 = 4 \times 10^6$ 个格子，每个格子一个未知量；再乘上上千个位姿，联合状态的维数是百万量级。想直接套用卡尔曼滤波把这么大的协方差矩阵求逆，内存与计算都无法接受。

困难二：数据关联。机器人并不知道“这一帧点云里的第 37 个点”与“上一帧里的哪个点”观测的是同一个物理表面。连”哪一帧和哪一帧到过同一个地方”都不知道。这个”配对”过程一旦出错，就会把一个错误的约束喂给优化器，让整张地图扭曲。因此我们先把这件事定义清楚。

定义 11.2 (数据关联)

数据关联（data association）是指在两次或多次传感器观测之间，确定“哪些观测量对应同一个物理实体”的过程。激光 SLAM 中它表现为点与点、点与面、帧与子图之间的对应；视觉 SLAM 中它表现为特征点匹配；在整个系统层面，它表现为“当前帧是否回到了历史某处”，即回环检测。

数据关联可以分为三个层次，后面各章会依次展开：**帧间关联**（当前帧与上一帧，见第 12 章）、**帧到地图关联**（当前帧与局部地图，见第 12 章）、**全局关联**（当前帧与历史所有子图，即回环检测，见第 12 章与第 15 章）。

一句话记住 SLAM 的两张皮

数据关联把传感器的原始数据变成“谁和谁有关系”；**状态估计**把这些关系变成“一致的轨迹和地图”。SLAM 系统的一切复杂度，都来自这两件事互相依赖：关联错了，估计就歪；估计歪了，下一帧的关联更容易错。

11.2 从最大后验到最小二乘

上面说的是“难在哪”，这一节说”怎么写成可计算的式子”。SLAM 的标准做法是把式 (11.1) 当成一个最大后验估计问题。

定义 11.3 (最大后验估计)

最大后验估计（Maximum A Posteriori，MAP）是指：在已知观测 $\boldsymbol{z}$ 的条件下，取使后验概率密度最大的状态作为估计值，即 $\hat{\boldsymbol{x}} = \arg\max_{\boldsymbol{x}} p(\boldsymbol{x} \mid \boldsymbol{z})$。它与最大似然估计（ML，$\arg\max p(\boldsymbol{z} \mid \boldsymbol{x})$）的区别是：MAP 额外乘上了先验 $p(\boldsymbol{x})$，因此可以融合“观测之外的已有知识”。

由贝叶斯公式（定义见第 2 章）有

$$p(X, \mathcal{M} \mid \boldsymbol{Z}, \boldsymbol{U}) = \frac{p(\boldsymbol{Z} \mid X, \mathcal{M})\, p(X, \mathcal{M} \mid \boldsymbol{U})}{p(\boldsymbol{Z} \mid \boldsymbol{U})}. \tag{11.2}$$

分母 $p(\boldsymbol{Z} \mid \boldsymbol{U})$ 与待估量 $X, \mathcal{M}$ 无关，在对 $X, \mathcal{M}$ 取 arg max 时可以丢掉，于是

$$p(X, \mathcal{M} \mid \boldsymbol{Z}, \boldsymbol{U}) \propto \underbrace{p(\boldsymbol{Z} \mid X, \mathcal{M})}_{\text{似然}}\ \underbrace{p(X, \mathcal{M} \mid \boldsymbol{U})}_{\text{先验}}. \tag{11.3}$$

接下来对式 (11.3) 做两个标准假设，把联合概率拆成一个个小因子——这一步是把“一个巨大的概率”变成“一堆可分别计算的残差”的关键。

假设 A（观测条件独立）：给定位姿与地图后，各次观测互相独立，

$$p(\boldsymbol{Z} \mid X, \mathcal{M}) = \prod_{j=1}^{M} p\left(\boldsymbol{z}_j \mid \boldsymbol{x}_{i(j)}, \mathcal{M}\right), \tag{11.4}$$

其中 $\boldsymbol{x}_{i(j)}$ 表示第 j 个观测所涉及的那个位姿。直观理解：传感器每一帧的噪声不会“串”到下一帧。

假设 B（一阶马尔可夫性）：位姿轨迹的生成只依赖上一时刻，

$$p(X, \mathcal{M} \mid \boldsymbol{U}) = p(\boldsymbol{x}_0)\, p(\mathcal{M}) \prod_{i=1}^{N} p\left(\boldsymbol{x}_i \mid \boldsymbol{x}_{i-1}, \boldsymbol{u}_i\right). \tag{11.5}$$

这相当于说：知道了上一帧位姿和这一帧的里程计读数，这一帧位姿就与更早的历史无关。

把 A、B 代入式 (11.3)，得到

$$p(X, \mathcal{M} \mid \boldsymbol{Z}, \boldsymbol{U}) \propto p(\boldsymbol{x}_0)\, p(\mathcal{M}) \prod_{i=1}^{N} p(\boldsymbol{x}_i \mid \boldsymbol{x}_{i-1}, \boldsymbol{u}_i) \prod_{j=1}^{M} p(\boldsymbol{z}_j \mid \boldsymbol{x}_{i(j)}, \mathcal{M}). \tag{11.6}$$

式 (11.6) 是一个连乘。连乘不好优化，但取对数就变成连加，而 log 是单调递增函数，不会改变极值点的位置：

$$\begin{aligned} X^{\star}, \mathcal{M}^{\star} &= \arg\max_{X, \mathcal{M}} p(X, \mathcal{M} \mid \boldsymbol{Z}, \boldsymbol{U}) \\ &= \arg\max_{X, \mathcal{M}} \log p(X, \mathcal{M} \mid \boldsymbol{Z}, \boldsymbol{U}) \\ &= \arg\max_{X, \mathcal{M}} \left\{ \log p(\boldsymbol{x}_0) + \log p(\mathcal{M}) + \sum_{i=1}^{N} \log p(\boldsymbol{x}_i \mid \boldsymbol{x}_{i-1}, \boldsymbol{u}_i) + \sum_{j=1}^{M} \log p(\boldsymbol{z}_j \mid \boldsymbol{x}_{i(j)}, \mathcal{M}) \right\}. \end{aligned} \tag{11.7}$$

注意 $\arg\max f = \arg\min(-f)$，把上式整体加负号、再改写成最小化，就得到“负对数后验”。这一步是整个 SLAM 后端（以及所有状态估计）的公共起点。

现在代入高斯假设。若某次测量满足 $\boldsymbol{z} = \boldsymbol{h}(\boldsymbol{x}) + \boldsymbol{n}$，噪声 $\boldsymbol{n} \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{\Sigma})$（高斯的定义见第 2 章），则

$$p(\boldsymbol{z} \mid \boldsymbol{x}) = \frac{1}{\sqrt{(2\pi)^k \det \boldsymbol{\Sigma}}} \exp\Big(-\frac{1}{2} (\boldsymbol{z} - \boldsymbol{h}(\boldsymbol{x}))^\top \boldsymbol{\Sigma}^{-1} (\boldsymbol{z} - \boldsymbol{h}(\boldsymbol{x})) \Big). \tag{11.8}$$

两边取负对数，记残差 $\boldsymbol{r}(\boldsymbol{x}) = \boldsymbol{z} - \boldsymbol{h}(\boldsymbol{x})$：

$$-\log p(\boldsymbol{z} \mid \boldsymbol{x}) = \frac{1}{2} \boldsymbol{r}^\top \boldsymbol{\Sigma}^{-1} \boldsymbol{r} + \frac{1}{2} \log \det \boldsymbol{\Sigma} + \frac{k}{2} \log 2\pi. \tag{11.9}$$

后两项与 $\boldsymbol{x}$ 无关，优化时可以扔掉。于是定义**马氏范数**（Mahalanobis norm，其几何含义见第 9 章）

$$\|\boldsymbol{r}\|^2_{\boldsymbol{\Sigma}^{-1}} \triangleq \boldsymbol{r}^\top \boldsymbol{\Sigma}^{-1} \boldsymbol{r}, \tag{11.10}$$

它相当于先把残差除以“该方向上的标准差”再做平方和：$\boldsymbol{\Sigma}$ 大（这个方向测不准）的方向权重小，$\boldsymbol{\Sigma}$ 小（测不准的程度低）的方向权重大。

把式 (11.9) 代回式 (11.7)，并记每个因子对应一个残差 $\boldsymbol{r}_k$，最终得到 SLAM 的标准形式：

$$X^\star = \arg\min_X \sum_{k=1}^{K} \frac{1}{2} \|\boldsymbol{r}_k(X)\|^2_{\boldsymbol{\Sigma}_k^{-1}}. \tag{11.11}$$

式 (11.11) 中每一类残差的物理含义如下（$K = N + M + 1$ 个因子）：

- **先验项** $\|\boldsymbol{r}_0\|^2_{\boldsymbol{\Sigma}_0^{-1}}$：$-\log p(\boldsymbol{x}_0)$，来自起点先验或边缘化得到的信息；
- **运动项** $\|\boldsymbol{r}_i\|^2_{\boldsymbol{\Sigma}_i^{-1}}$：$-\log p(\boldsymbol{x}_i \mid \boldsymbol{x}_{i-1}, \boldsymbol{u}_i)$，即里程计/IMU 预积分约束，量化“相邻两帧应该差多少”；
- **观测项** $\|\boldsymbol{r}_j\|^2_{\boldsymbol{\Sigma}_j^{-1}}$：$-\log p(\boldsymbol{z}_j \mid \boldsymbol{x}_{i(j)}, \mathcal{M})$，即扫描匹配、重投影、点到面等几何约束，量化“当前位姿下，地图上应该看到什么”。

例题 11.1 一维两帧：最小二乘就是加权平均 设只有两个位姿 x_0, x_1（一维）。先验说 $x_0 = 0$，标准差 $\sigma_0 = 0.1$；里程计说 $x_1 - x_0 = 1$，$\sigma_u = 0.2$；观测说 $x_1 = 0.9$，$\sigma_z = 0.1$。信息矩阵就是 $1/\sigma^2$，于是

$$J(x_0, x_1) = \frac{(x_0 - 0)^2}{2 \times 0.01} + \frac{(x_1 - x_0 - 1)^2}{2 \times 0.04} + \frac{(x_1 - 0.9)^2}{2 \times 0.01}. \tag{11.12}$$

分别对 x_0, x_1 求偏导并令其为零：

$$\frac{\partial J}{\partial x_0} = \frac{x_0}{0.01} - \frac{x_1 - x_0 - 1}{0.04} = 0 \implies 5x_0 - x_1 + 1 = 0, \tag{11.13}$$

$$\frac{\partial J}{\partial x_1} = \frac{x_1 - x_0 - 1}{0.04} + \frac{x_1 - 0.9}{0.01} = 0 \implies 5x_1 - x_0 - 4.6 = 0. \tag{11.14}$$

联立解得 $x_0 = -1/60 \approx -0.017$，$x_1 = 11/12 \approx 0.917$。可以看到：观测的 σ 更小（更可信），所以 x_1 停在 0.9 附近而不是里程计说的 1.0；同时 x_0 被微微拉离 0，以缓和“里程计说走了 1 米”与“观测说在 0.9 米”的矛盾。这就是最小二乘全部的工作方式——**在互相矛盾的约束之间按可信度做折中**。

本部分的地基

式 (11.11) 说明 SLAM 在数学上就是**一个大规模稀疏非线性最小二乘问题**。“大规模”来自百万级地图与上千位姿，“稀疏”来自每个因子只涉及极少数变量。第 13 章会专门讲怎么高效地解它。但解它之前，得先把残差造出来——造残差靠数据关联，这就是前端。

11.3 前端与后端：职责划分

既然 SLAM = 数据关联 + 状态估计，一个自然的工程划分就出现了：把“造残差”和“解残差”拆成两个模块。这就是前端与后端。

定义 11.4 (前端（Front-end）)

前端是 SLAM 系统中直接处理传感器原始数据、完成数据关联与局部位姿估计的部分。它的输出不是最终地图或最终轨迹，而是一组相对位姿约束（以及关键帧、回环候选），供后端使用。

定义 11.5 (后端（Back-end）)

后端是 SLAM 系统中接收前端给出的因子/约束、对整条轨迹（以及地图状态）做全局优化与一致性维护的部分。它的输出是全局一致的位姿轨迹、状态协方差以及更新后的地图，供定位、规划与显示使用。♣

两者为什么必须分开？因为它们的**目标、节奏和失败模式完全不同**，见表 11.1。

表 11.1: SLAM 前端与后端的职责对比

维度	前端（Front-end）	后端（Back-end）
一句话	负责局部一致	负责全局一致
输入	传感器原始数据（scan、图像、IMU、轮速）	前端给出的因子/约束（相对位姿、回环边）
输出	相对位姿、关键帧、回环候选	全局位姿、协方差、地图更新量
运行频率	高（10 ~ 100 Hz，与传感器同量级）	低（0.1 ~ 5 Hz，或事件触发）
关键算法	预处理、扫描匹配、数据关联、外点剔除	因子图/位姿图优化、鲁棒核、边缘化、稀疏求解
失败表现	漂移、跟踪丢失、关联错乱	全局变形、优化不收敛、回环错闭
能否替代	可整体换掉（LOAM 换 Cartographer 前端）	相对稳定（g2o/GTSAM/Ceres 通用）

表 11.1 里最关键的一行是“一句话”。把它展开成一句可背诵的话：**前端把数据变成约束，后端把约束变成一致的轨迹与地图；前端可以错，但不能慢；后端可以慢，但不能错。**

所谓“全局一致”是什么意思？设想机器人绕一个矩形走廊走了一圈回到起点。由于里程计的累积误差，前端给出的轨迹是一条不闭合的螺旋线：最后一帧的位姿和第一帧差了两米。后端通过回环检测发现“这两帧其实是同一个地方”，于是把这个“差两米”的信息作为一条强约束加进去，重新优化全部轨迹，让首尾闭合。优化之后，整条轨迹上的每一帧位姿都会被微调——这就是后端的作用范围：它改的是整条轨迹，而前端只改当前帧。

11.4 三种语境下的“前端/后端”（重点）

“前端/后端”这个词在中文技术圈用得极多，但在三种语境下指的东西完全不同。新手最容易犯的错是：看完 SLAM 的前后端划分，转头去看定位系统或导航系统的框图，发现“前端”指的不是同一个东西，于是怀疑自己没学懂。其实不是你错，是术语重载了。本节把三种语境并排摆出来（表 11.2），并给出判断规则。

表 11.2: 三种语境下“前端/后端”的所指

语境	“前端”指什么	“后端”指什么	本书在哪讲
SLAM（建图）	里程计 + 数据关联：预处理、帧间/帧到地图匹配、关键帧选取	全局优化 + 回环：因子图/位姿图优化、地图更新	第 11–15 章
定位系统	实时状态估计：LIO/VIO、ESKF，输出高频位姿与协方差	先验地图配准 + 因子图更新，低频修正漂移并判断可信度	第 23 章
导航系统	感知：定位 + 建图 + 障碍检测，回答“我在哪、周围是什么”	决策与控制：行为树、规划器、控制器，回答“我该往哪走”	第 18 章

语境一：SLAM 意义下。这是本章与第 12–15 章默认使用的划分。前端 = 里程计 + 数据关联，后端 = 优化 + 回环。划分依据是**数学结构**：前端产出因子，后端求解式 (11.11)。

语境二：定位系统意义下。一个定位系统（比如 RoboMaster 赛场上跑的 Z-Localization）并不建图，它面对的是一张**已有的**先验地图。此时“前端”是保持高频、连续的实时状态估计（LIO 或 ESKF），“后端”是与先验地图做配准、并用因子图把“先验地图约束”和“里程计约束”融合起来的那一层。划分依据是**信息的来源**：前端只用自己的传感器，后端引入了先验地图这个外部信息源。

语境三：导航系统意义下。在 Nav2 这类导航框架里，“前端/后端”是按**信息流方向**划分的：定位与建图属于“感知前端”，回答“世界现在是什么样”；规划与控制属于“决策后端”，回答“接下来怎么做”。划分依据是**抽象层次**：前者输出世界状态，后者输出动作。

判断规则：先问信息链的终点

看到“前端/后端”，先别急着套定义，先问一句：**这条信息链的终点是什么？**终点是“地图与轨迹”，就是语境一；终点是“当前位姿”，就是语境二；终点是“速度指令”，就是语境三。三种语境并不冲突：同一个系统里可以同时存在三对前后端——Z-Localization 内部有语境二的一对，它又把位姿喂给 Nav2，成为语境三中“感知前端”的一部分。

笔记 本书的默认约定：第 11 章到第 15 章，以及第 16 章、第 17 章，“前端/后端”一律指语境一；第 23 章按语境二讲；第 18 章按语境三讲。写作与阅读时请以各章开头的一句话声明为准。

11.5 数据流：一帧数据在系统里走过的路

把上面的职责划分画成图，就是图 11.1。

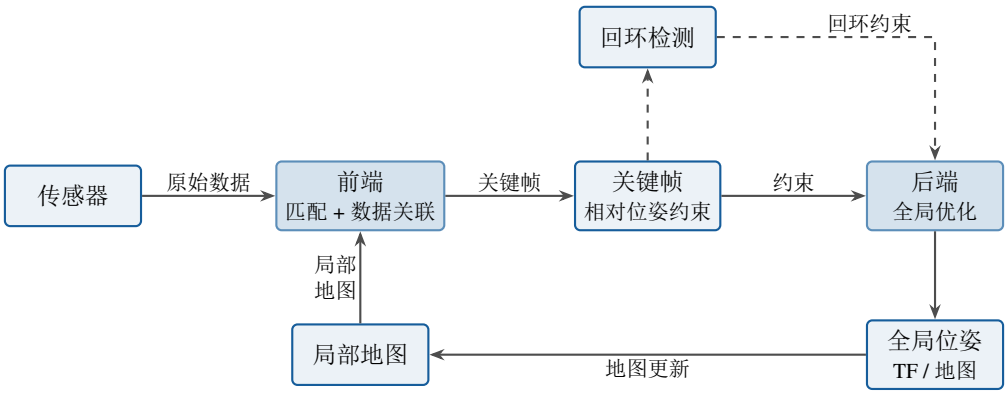


图 11.1: SLAM 系统的总体数据流。实线为在线主通路：传感器 → 前端 → 关键帧与约束 → 后端 → 全局位姿与地图；下方是“地图回喂前端”的闭环。虚线为回环检测这条全局数据关联支路：它由前端发起、向后端提供回环约束。正是这条支路把“局部一致”升级为“全局一致”。

沿着图 11.1 把各个箭头上流动的数据说清楚，读源码时就能对上号：

1. **传感器 → 前端**：原始 scan/图像/IMU，带时间戳，通常先做时间同步（第 3 章）。
2. **前端 → 关键帧**：每帧的相对位姿估计及其协方差，以及“这一帧够不够格当关键帧”的判断（第 12 章）。
3. **关键帧 → 后端**：一条条因子。最典型的是位姿图里的二元边（第 13 章）。
4. **后端 → 全局位姿**：优化后的轨迹，发布成 TF 变换与地图坐标系（第 25 章会讲 TF 树）。
5. **全局位姿 / 地图 → 局部地图 → 前端**：前端做帧到地图匹配时需要一块“附近的地图”，这块地图由后端维护的全局地图裁剪而来。
6. **回环检测**：注意它在图里是“从前端出发”的。回环检测本质是全局范围的数据关联，属于前端职责；它只负责“找出候选 + 校验”，把约束交给后端去优化。这是第 12 章要专门解释的一个常见误解。

11.6 SLAM 系统的分类

不同文献里的 SLAM 系统五花八门，其实可以从五个维度切开。每一项的取舍都对应着具体的工程权衡。

1. **滤波 vs 优化**。滤波类（EKF-SLAM、FastSLAM）把状态和协方差一路递推，优点是天然增量、能做在线预测，缺点是协方差矩阵随状态维数平方增长，且线性化点一旦固定就无法回头修正。优化类（因子图、位姿图）把历史状态全部保留，用迭代最小二乘求解，优点是精度高、能全局重线性化，缺点是计算量随规模增长（靠稀疏性与增量求解缓解）。**现在主流是优化类**，原因见 §11.7。
2. **直接法 vs 特征法**。直接法直接用像素灰度/点云强度构造残差（如 DSO[9]），省去特征提取，在弱纹理场景更稳，但对光度变化与初值敏感；特征法先提取点/线/面特征再匹配（如 ORB-SLAM3[3]），对光照鲁棒、容易做回环，但弱纹理处会丢特征。激光 SLAM 介于两者之间：“点到面”用的是几何结构而非人工特征。

3. **2D vs 3D**。2D 假设机器人在地面运动、地图是一层栅格，计算量小、结构简单，适合单层场地（第 15 章）；3D 要恢复完整六自由度与三维地图，计算与内存大一个量级，但能处理坡道、多层、空中（第 16 章与第 17 章）。
4. **单机 vs 多机**。多机 SLAM 要让多个机器人共享地图并保持坐标系一致，核心难点从“估计”变成“通信与一致性”。
5. **离线 vs 在线**。离线（如用 rosbag 建图）可以用全局优化反复迭代，追求最好结果；在线必须满足实时性，只能“够用就好、逐步逼近”。

11.7 为什么现在的主流是“前端 LIO + 后端因子图”

把本节前面的内容串起来，就能理解为什么近十年的主流架构几乎都是“前端高频里程计（LIO/VIO）+ 后端低频因子图”：LIO-SAM[35]、FAST-LIO2[41]、Cartographer[19] 都是这个骨架。原因有五条。

1. **符合信息流**。前端能拿到的是“相邻两帧之间的关系”，这天然是相对约束；后端要回答的是“全局怎么摆”，天然是全局优化。把两者硬塞进一个滤波器，等于用递推的方式做全局问题，事倍功半。
2. **计算量可扩展**。式 (11.11) 的海森矩阵是稀疏的：每个因子只连接少数变量。1000 个位姿的位姿图，海森矩阵的非零元只占百分之几，稀疏 Cholesky 分解可以做到毫秒级；而同样规模的 EKF 要维护 6000×6000 的稠密协方差并每次求逆 [24]，根本跑不动。
3. **精度更高**。优化在后端每轮都重新线性化，而 EKF 只在一步处线性化一次，误差会沿着轨迹累积。因子图的表述方式在数学上等价于“对所有历史观测做联合最大似然”，这正是式 (11.11) 想做的事 [7]。
4. **可恢复性**。前端总会漂移，甚至短时跟丢。只要后端还保存着历史因子，一次成功的回环或重定位就能把整条轨迹“拉正”；滤波器模式下历史状态已经被边缘化掉，想改也改不了。
5. **工程解耦**。前端可以整体替换（今天用 LOAM，明天换 Cartographer，甚至换成视觉），后端则可以一直用现成的图优化库（如 g2o、GTSAM、Ceres[24]），基本不用动。这种接口稳定性对多人协作的算法组极其重要。

代价也要说清楚：后端是滞后的，前端已经漂了几十米，后端才通过回环把误差摊平；如果前端漂移大到连回环都找不到（描述子已经对不上），后端也无能为力。所以“前端准”和“后端稳”是互补的两件事，缺一不可。

本章小结

1. SLAM 是“定位需要地图、建图需要位姿”的鸡生蛋问题，形式化为联合后验 $p(X, \mathcal{M} \mid \boldsymbol{Z}, \boldsymbol{U})$（式 (11.1)）。
2. 做两个假设（观测条件独立、一阶马尔可夫）后，联合后验拆成先验、运动、观测三类因子的连乘；取负对数并把噪声建模为高斯，就得到标准最小二乘形式 $\min_X \sum_k \frac{1}{2} \|\boldsymbol{r}_k\|^2_{\boldsymbol{\Sigma}_k^{-1}}$（式 (11.11)）。
3. 马氏范数 $\|\boldsymbol{r}\|^2_{\boldsymbol{\Sigma}^{-1}} = \boldsymbol{r}^\top \boldsymbol{\Sigma}^{-1} \boldsymbol{r}$ 的含义是“按各方向可信度加权的平方误差”，信息矩阵 $\boldsymbol{\Sigma}^{-1}$ 越大该约束越硬。
4. 前端 = 数据关联 + 局部位姿估计，追求实时与鲁棒，负责局部一致；后端 = 全局优化 + 回环，追求精度与可恢复，负责全局一致（表 11.1）。
5. “前端/后端”有三种语境：SLAM 下按数学结构分（造因子/解因子），定位系统下按信息来源分（自身传感器/先验地图），导航系统下按抽象层次分（感知/决策）。判断规则是“先问信息链的终点”（表 11.2）。
6. 回环检测属于前端（全局数据关联），它产出约束，由后端优化；把回环检测归到后端是常见误解。
7. 主流架构“前端 LIO + 后端因子图”的五条理由：符合信息流、计算可扩展、精度更高、可恢复、工程解耦；代价是后端滞后。

第十一章 练习

1. 在式 (11.6) 的两个假设中，如果去掉“观测条件独立”，式子会变成什么样？举一个现实中不独立的例子（提示：同一帧内相邻点的噪声、动态物体）。

2. 用本节的加权平均方法重算：若观测标准差改为 $\sigma_z = 0.3$（比里程计更不可信），x_0, x_1 会偏向哪边？请给出解析解并与前面的 0.917 对比。
3. 为什么说“回环检测属于前端”？如果把它硬放到后端，会在工程上带来什么麻烦？请从运行频率和数据依赖两个角度回答。
4. Z-Localization 里“前端”指 LIO、“后端”指先验地图配准 + 因子图。请说明为什么这里的“后端”不需要维护地图，而 SLAM 的后端需要。
5. 假设你只有一只 2D 激光雷达和轮式里程计，在一个 30 m × 20 m 的单层场地里建图，你会选滤波式还是优化式？说明你的取舍依据。

延伸阅读

1. Thrun, Burgard, Fox，《Probabilistic Robotics》第 10–11 章 [38]：SLAM 的概率表述与 EKF-SLAM 的原点，用来对照“滤波 vs 优化”。
2. Dellaert & Kaess, “Factor Graphs for Robot Perception”[7]：因子图视角下的 SLAM，读完能理解为什么式 (11.11) 可以画成一张图。
3. Kümmerle 等，g2o 论文 [24]：把“稀疏最小二乘”落到代码，适合读完后去翻 g2o 的 `OptimizableGraph` 与稀疏求解器部分。
4. LIO-SAM 论文 [35]：“前端里程计 + 后端因子图”的教科书级实现，读完再回看本章图 11.1 会非常直观。

第十二章 SLAM 前端详解

本章导读

第 11 章把 SLAM 拆成了“造约束的前端”和“解约束的后端”。本章专门讲前端：它从传感器原始数据出发，经过预处理、数据关联、位姿估计、外点剔除、关键帧选取、局部地图维护，最终给后端交出一串可信的相对位姿约束。前端是整套系统里离传感器最近、也最容易出错的一环——后端的数学再漂亮，前端喂进去一条错误约束，整张地图照样会歪。本章会逐个环节讲清楚“它解决什么问题、怎么做、什么时候会失效”，重点讲透三件事：数据关联为什么是前端的心脏、为什么“帧到局部地图”比“帧到帧”稳、以及回环检测为什么属于前端而不属于后端。

12.1 前端流水线

一个典型的激光 SLAM 前端每收到一帧点云，都会走一遍图 12.1 的流程。视觉前端结构类似，只是把“点云”换成“图像 + 特征”。

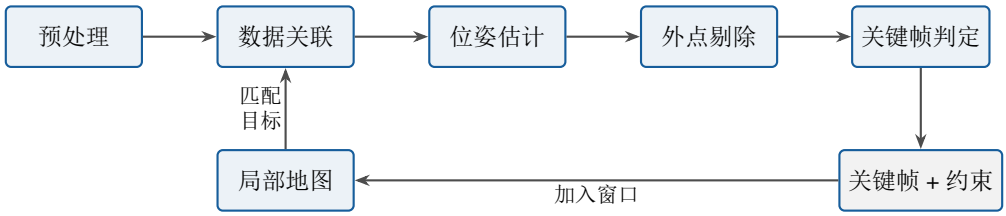


图 12.1: SLAM 前端流水线。主链是“预处理 → 数据关联 → 位姿估计 → 外点剔除 → 关键帧判定”；下方闭环表示关键帧被加入局部地图窗口，成为下一帧匹配的目标。

每一步的输入输出见表 12.1。读源码时，只要能把一个函数映射到表里的某一行，就基本知道它在干什么。

表 12.1: 前端各环节的输入与输出

环节	输入	输出
预处理	原始 scan/图像、IMU、时间戳	去畸变点云、降采样点云、地面/动态点标记
数据关联	当前帧 + 局部地图	点—点、点—面、点—体素对应关系
位姿估计	对应关系 + 初值	当前帧相对位姿 $\hat{T}$ 及协方差
外点剔除	对应关系 + 残差	内点集合、内点比例、是否跟踪失败
关键帧判定	位姿、视角变化、观测新区域比例	是否为关键帧的布尔量

为什么要拆成这么多步？因为每一步的**失败模式**不同，拆开才能分别监控、分别兜底。比如“内点比例骤降”是关联出了问题的信号，而“海森矩阵最小特征值过小”是几何退化（§12.7）的信号。把所有步骤揉成一个黑盒，出了问题就无从下手。

12.2 数据关联：前端的心脏

定义 12.1 (数据关联（工程视角）)

在 SLAM 前端中，数据关联指：给当前帧中的每个观测（点、特征、整帧），在局部地图或历史帧中找到它所对应的那个物理实体，并给出对应关系集合 $C = \{(i, j)\}$，其中 i 是当前帧观测的编号，j 是地图元素的编号。

第 11 章已经说明关联错了后果严重，这里把它落到具体做法上。最常用的三种手段如下。

（1）最近邻（Nearest Neighbor，NN）。对当前帧的每个点 $\boldsymbol{p}_i$，在局部地图里找欧氏距离最近的元素：

$$j^\star(i) = \arg\min_j \left\| \boldsymbol{p}_i - \boldsymbol{q}_j \right\|_2, \qquad \text{若 } \left\| \boldsymbol{p}_i - \boldsymbol{q}_{j^\star} \right\|_2 > \tau_d \text{ 则丢弃.} \tag{12.1}$$

式中 τ_d 是距离阈值，通常取“期待的最大位移加上若干倍点间距”。直接用 NN 的问题是对称性：点 i 认为 j 最近，但 j 可能认为别的点更近。

（2）相互最近邻（Mutual NN，MNN）。要求双向成立，即 $j^\star(i) = j$ 且 $i^\star(j) = i$ 才保留。这在两帧点密度接近时能剔除大量错误对应，代价是内点变少。

（3）RANSAC 外点剔除。先随机抽最少的点对（2D 刚体变换最少 2 对，3D 最少 3 对）解出一个候选变换，统计有多少点对在该变换下残差小于阈值（内点），重复若干次取内点最多的一次。它能容忍高比例的错关联，代价是需要多次抽样。

图 12.2 展示了错误关联的破坏力：当机器人从侧面靠近时，由于初值不准，A 墙上的点被错误地关联到 B 墙，优化器为了“满足”这条约束，会把两堵墙拉近，于是在地图上出现一堵“重影墙”。

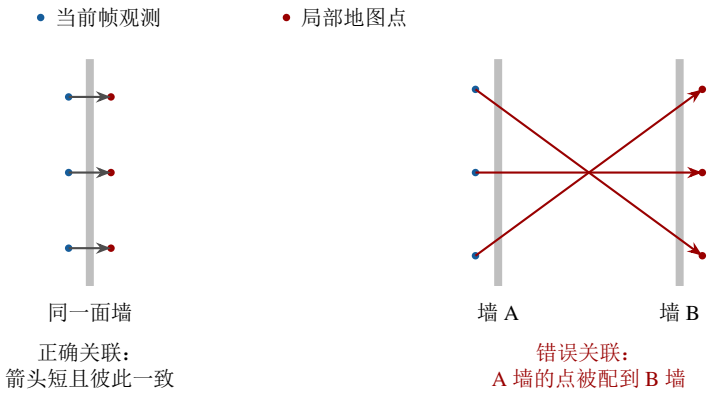


图 12.2: 左：正确关联——两次观测落在同一面墙上，对应箭头短且彼此一致；右：错误关联——初值偏差使 A 墙的观测点被配到 B 墙，对应箭头跨越两墙、互相交叉。后者会在优化中产生巨大残差，迫使轨迹或地图扭曲，形成“重影墙”。

前端的第一原则

前端的第一原则不是“算得准”，而是**“别把错误关联喂给后端”**。一条错误约束的破坏力，远大于十条正确约束的收益：正确约束只是让结果更精确一点，错误约束足以让整段轨迹塌陷。所以外点剔除、鲁棒核（第 13 章）、回环几何校验这些“保险”，在工程上比提升匹配精度更重要。

12.3 从“帧到帧”升级到“帧到局部地图”

第 7 章讲的扫描匹配是帧到帧（frame-to-frame）：把当前帧和上一帧配准，得到两帧之间的相对位姿。这在里程计里够用，但在 SLAM 前端里不够，原因是误差会沿着链式匹配累积。

假设每一帧匹配都有独立、零均值、标准差为 σ 的误差。那么第 k 帧相对第 0 帧的误差是 k 个误差之和。由方差的可加性（独立随机变量之和的方差等于方差之和，见第 2 章）：

$$\mathrm{Var}\left(\sum_{i=1}^{k} \epsilon_i\right) = \sum_{i=1}^{k} \mathrm{Var}(\epsilon_i) = k\sigma^2 \quad \Longrightarrow \quad \sigma_k = \sqrt{k}\,\sigma. \tag{12.2}$$

$k = 100$ 时标准差已经放大 10 倍。这就是漂移的数学本质：误差按“随机游走”的方式增长。

帧到局部地图（frame-to-map）改动了一个关键设定：**匹配目标不再是单帧，而是多帧融合成的局部地图**。于是问题变成“当前帧在该地图坐标系下处于什么位姿”，误差不沿着链传递。

定义 12.2 (局部地图)

局部地图（local map）是以当前位姿为中心、由最近若干个关键帧的观测融合而成的、用于当前帧匹配的地图子集。它只覆盖机器人附近的一小块区域，因而可以实时查询与更新。

局部地图的工程组织方式有三种，各有取舍：

- **滑动窗口**：保留最近 N 个关键帧的原始点云，匹配时全部拼起来。实现最简单，内存随 N 线性增长。
- **体素地图**：把点云按体素栅格（§12.5）降采样后长期保存，查询时取当前位姿邻域的体素。内存可控，适合大场景。
- **子图（submap）**：把连续若干帧融合成一张概率栅格（第 15 章），匹配时直接对栅格做插值查询。Cartographer 用的就是它。

为什么帧到地图更稳？有三个原因。第一，**目标更“干净”**：多帧融合相当于对同一个表面做了平均，若 n 次独立观测的噪声标准差为 σ，平均后降到 $\sigma/\sqrt{n}$。第二，**约束更多**：一帧只和上一帧比，只能约束两帧之间的相对关系；而帧到地图同时受地图中所有可见表面的约束。第三，**误差不进入目标**：帧到帧时，上一帧本身带着误差，等于用“错的东西”当基准；帧到地图时，基准是后端优化过的全局一致地图（或至少是融合后的局部地图），误差被摊平了。

笔记 帧到帧并非一无是处。它不需要维护地图、计算量小，非常适合做“里程计”（第 7 章），也常被用作帧到地图匹配的初值。实际系统里两者往往同时存在：先用帧到帧（或 IMU 预测）给出初值，再用帧到地图精化。

12.4 关键帧策略

定义 12.3 (关键帧)

关键帧（keyframe）是从连续帧序列中挑选出的、代表局部观测的子集帧。只有关键帧参与地图维护与后端约束构建；非关键帧只用于帧间追踪，不进入优化。

为什么要有关键帧？因为相邻帧之间信息高度冗余：机器人以 0.5 m/s 运动、雷达 10 Hz 时，相邻两帧只差 5 cm，把它们都塞进后端，节点数会爆炸而信息量几乎不增加。

关键帧的触发条件通常有四类，工程上取它们的“或”：

1. **平移足够大**：$\|\boldsymbol{t}_k - \boldsymbol{t}_{\text{last}}\|_2 > d_{\max}$，典型 $d_{\max} = 0.5 \sim 1.0$ m；
2. **旋转足够大**：$|\Delta\theta_k| > \theta_{\max}$，典型 $\theta_{\max} = 10^\circ \sim 20^\circ$（3D 时要分别看 roll/pitch/yaw 或转角轴）；
3. **时间足够长**：距上一关键帧超过 $t_{\max}$，典型 0.5 ~ 2 s，防止静止时长时间没有关键帧、导致地图更新停滞；
4. **信息量足够大**：当前帧观测到的“新区域”（从未见过的栅格/体素）占比超过阈值，或匹配协方差的迹（trace，见第 2 章）超过阈值。这条最接近本质，但实现最复杂。

合起来写成一个判定式：

$$\text{新关键帧} \iff \Big(\|\boldsymbol{t}_k - \boldsymbol{t}_{\text{last}}\|_2 > d_{\max}\Big) \vee \Big(|\Delta\theta_k| > \theta_{\max}\Big) \vee \Big(t_k - t_{\text{last}} > t_{\max}\Big) \vee \Big(\text{新区域占比} > \eta\Big). \tag{12.3}$$

式 (12.3) 中四个条件互相独立，工程上取“或”是为了在任意一种情况下都能及时更新地图：走得快时靠平移条件，原地转身时靠旋转条件，静止时靠时间条件，进入新房间时靠信息量条件。

关键帧太多或太少都有问题，见表 12.2。取值的“甜点”取决于场景：室内小场地 0.5 m 一帧比较合适，大场地可以放宽到 1 ~ 2 m。

表 12.2: 关键帧数量对系统的影响

情形	好处	代价
关键帧过多	地图细节丰富、约束密集、回环易检测	后端节点多、优化变慢、内存与存储膨胀
关键帧过少	后端轻量、优化快	约束稀疏、回环描述子区分度低、地图轮廓破碎

12.5 点云预处理

预处理决定了“喂给匹配的数据质量”。这里讲三个最常用、也最影响效果的步骤。

12.5.1 体素降采样

原始雷达一帧可能有十万个点，直接匹配太慢。体素降采样（voxel grid downsampling）把空间切成边长为 r 的小立方体（体素），每个体素只保留一个代表点。体素索引由坐标直接算出：

$$(i,j,k)=\left(\left\lfloor\frac{x-x_{\min}}{r}\right\rfloor,\left\lfloor\frac{y-y_{\min}}{r}\right\rfloor,\left\lfloor\frac{z-z_{\min}}{r}\right\rfloor\right), \tag{12.4}$$

其中 $\lfloor\cdot\rfloor$ 是向下取整，$x_{\min},y_{\min},z_{\min}$ 是点云包围盒的最小角。同一体素内的点被划分为一组。

代表点取体素质心，即

$$\bar{\boldsymbol{p}}_{ijk}=\frac{1}{n_{ijk}}\sum_{m=1}^{n_{ijk}}\boldsymbol{p}_m, \tag{12.5}$$

其中 n_{ijk} 是该体素内的点数。为什么取质心而不是随便取一个点？因为质心是这个体素内所有点的最小二乘代表：我们要找一个点 $\boldsymbol{q}$，使得到体内所有点的距离平方和最小，

$$J(\boldsymbol{q})=\sum_{m=1}^{n}\|\boldsymbol{q}-\boldsymbol{p}_m\|_2^2. \tag{12.6}$$

对 $\boldsymbol{q}$ 求导并令其为零：

$$\frac{\partial J}{\partial \boldsymbol{q}}=2\sum_{m=1}^{n}(\boldsymbol{q}-\boldsymbol{p}_m)=\boldsymbol{0}\;\Longrightarrow\; n\boldsymbol{q}=\sum_{m=1}^{n}\boldsymbol{p}_m\;\Longrightarrow\;\boldsymbol{q}=\frac{1}{n}\sum_{m=1}^{n}\boldsymbol{p}_m. \tag{12.7}$$

可见式 (12.5) 不是“约定俗成”，而是“误差平方和最小”的直接结果。分辨率 r 的取法：太小则降采样无效，太大则丢失结构，经验值是略大于点间距（如 0.1 ~ 0.2 m）。

12.5.2 地面分割（RANSAC 平面拟合）

地面点占一帧点云的很大比例，且对匹配帮助不大（平坦地面在任何水平平移下都能配上，反而会“吸住”错误的位姿），所以先把它分出来。最常用的方法是 RANSAC 拟合平面。

平面用单位法向 $\boldsymbol{n}$ 和截距 d 表示：$\boldsymbol{n}^{\top}\boldsymbol{p}=d$，$\|\boldsymbol{n}\|=1$。取三个不共线的点 $\boldsymbol{p}_1,\boldsymbol{p}_2,\boldsymbol{p}_3$，法向由叉积给出：

$$\boldsymbol{n}=\frac{(\boldsymbol{p}_2-\boldsymbol{p}_1)\times(\boldsymbol{p}_3-\boldsymbol{p}_1)}{\big\|(\boldsymbol{p}_2-\boldsymbol{p}_1)\times(\boldsymbol{p}_3-\boldsymbol{p}_1)\big\|},\qquad d=\boldsymbol{n}^{\top}\boldsymbol{p}_1. \tag{12.8}$$

点到该平面的距离是 $|\boldsymbol{n}^{\top}\boldsymbol{p}-d|$（因为 $\boldsymbol{n}$ 是单位向量，投影长度就是距离）。距离小于阈值 τ_p 的点称为内点。重复抽样 k 次，取内点最多的平面作为地面。

抽多少次才够？设内点占全体的比例为 w，一次抽样三个点都落在内点的概率是 w^3（近似认为独立），则 k 次中都失败的概率为 $(1-w^3)^k$，至少成功一次的概率为

$$P_{\text{succ}}=1-(1-w^3)^k. \tag{12.9}$$

要求 $P_{\text{succ}}\ge P_0$，解出

$$k\;\ge\;\frac{\log(1-P_0)}{\log(1-w^3)}. \tag{12.10}$$

代入数字：若地面点占六成（$w=0.6$），要求 $P_0=0.99$，则 $1-w^3=1-0.216=0.784$，

$$k\ge\frac{\log 0.01}{\log 0.784}=\frac{-4.605}{-0.243}=18.9\;\Rightarrow\;k=19. \tag{12.11}$$

所以 20 次抽样就能以 99% 的概率找到地面，这就是 RANSAC 快的原因。

12.5.3 去畸变与动态点剔除

激光雷达一帧的采集需要时间（0.1 s 量级），机器人在这段时间里一直在动，于是“同一帧”里的点其实是在不同位姿下测到的——这就是运动畸变。去畸变要用 IMU 或匀速模型把每个点补偿到同一时刻，做法与第 16 章相同。动态点（行人、另一个机器人、传送带）则会违反“环境静止”的假设，简单做法是检查该点在后几帧是否稳定存在，或直接用鲁棒核把大残差降权（第 13 章）。

12.6 描述子与回环检测

回环检测要回答一个全局问题：**当前帧是不是回到了很久以前来过的地方？**这属于“全局范围的数据关联”。它不能靠暴力匹配所有历史帧（太慢），所以要先算一个紧凑的“场景描述子”做粗筛，再做几何校验。

激光 SLAM 里最常用的描述子之一是 Scan Context[23]。它的构造非常直观：把点云按极坐标切成 N_r 个同心环（ring）、N_s 个扇区（sector），得到一个 $N_r \times N_s$ 的矩阵，第 (r, s) 个元素取落入该 bin 的所有点的最大高度：

$$I_{rs} = \max\left\{ z_k \;\middle|\; \frac{r-1}{N_r} R < \rho_k \le \frac{r}{N_r} R,\ \frac{2\pi(s-1)}{N_s} \le \varphi_k < \frac{2\pi s}{N_s} \right\}, \tag{12.12}$$

其中 (ρ_k, φ_k, z_k) 是第 k 个点的柱坐标，R 是最大量程。若某 bin 没有点，取 0。图 12.3 是它的示意。

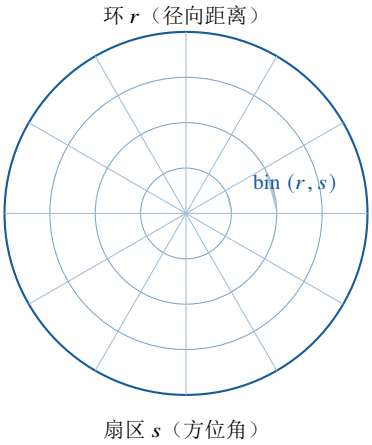


图 12.3: Scan Context 的环—扇区划分。极坐标平面被切为 N_r 个同心环与 N_s 个扇区，每个 bin 记录该区域内点的最大高度，于是环境被压缩成一个 $N_r \times N_s$ 的矩阵描述子。

为什么要这么切？因为**平移会让点云的笛卡尔坐标剧变，但旋转只会让极坐标矩阵的列发生循环移位**。所以比较两帧时，只需在列方向上做循环移位搜索：

$$D(I^q, I^c) = \min_{\delta \in \{0,1,\ldots,N_s-1\}} \frac{1}{N_s} \sum_{s=1}^{N_s} \left(1 - \frac{\boldsymbol{c}_s^{q\top} \boldsymbol{c}_{s+\delta}^{c}}{\|\boldsymbol{c}_s^q\| \, \|\boldsymbol{c}_{s+\delta}^c\|} \right), \tag{12.13}$$

其中 $\boldsymbol{c}_s \in \mathbb{R}^{N_r}$ 是矩阵的第 s 列，δ 是候选的列移位量，相当于“假设机器人转过 $\delta \cdot 360°/N_s$”；下标 $s+\delta$ 在超过 N_s 时按模 N_s 回绕（循环移位）。两列方向一致时余弦相似度为 1、距离为 0；正交时为 1。取所有移位中的最小值 D，若 D 小于阈值，就认为“这两帧很可能在同一个地方”。

回环检测为什么属于前端

回环检测的本质是**数据关联**——判断“这两帧观测的是不是同一个地方”，其输出是一条相对位姿约束（回环边），交给后端去优化。它不负责改轨迹，只负责“提出并校验假设”。把它放到后端会有两个问题：一是后端要接触原始点云，破坏了“前端造因子、后端解因子”的接口；二是回环检测必须按帧执行（频率高），而后端是低频的。所以正确划分是：**回环检测归前端，回环优化归后端。**

Scan Context 只是粗筛：D 小不代表真是回环（相似的走廊、对称的房间都会骗过它），还必须做几何校验。典型做法是用粗筛给出的位姿作初值，做一次 ICP/NDT（第 7 章）或分支定界搜索（第 15 章），用匹配得分与内点比例确认；校验不过就丢掉这个候选。

12.7 前端的失效模式

前端失效是 SLAM 最常见的故障来源。表 12.3 把四类典型失效整理成“场景—信号—对策”，工程上可以照着做监控。

表 12.3: 前端典型失效模式与对策

失效类型	典型场景	可观测信号	对策
几何退化	长走廊、隧道、空旷场地：某一方向没有约束	海森矩阵最小特征值接近 0、条件数骤增、匹配得分对该方向不敏感	引入 IMU 强约束、对该方向降权或锁定、切换匹配特征
动态物体	行人、对方机器人、传送带	内点比例骤降、残差分布重尾、匹配得分忽高忽低	动态点剔除、鲁棒核降权、多帧一致性检验
传感器失效	遮挡、强光、雨雾、断流	点数骤减、强度分布异常、时间戳不连续	健康检查、故障切换、用运动模型预测维持
时间不同步	多传感器融合、急加速	残差随速度增大、标定外参漂移	硬件触发同步/PTP、在线估计时延、重新标定

几何退化值得单独强调，因为它最隐蔽：机器人走在一条笔直的长走廊里，两侧墙壁平行，前后方向几乎没有可区分的结构，于是这个方向的位姿是“不可观”的。此时匹配照样会给出一个数，但这个数完全不可信。检测方法是对海森矩阵做特征值分解，若最小特征值 $\lambda_{\min}$ 太小（相对于最大特征值 $\lambda_{\max}$），就说明该方向退化。对策是“承认自己不知道”：在这个方向上相信 IMU 或保持上一时刻的估计，而不是相信匹配结果。

算法：前端单帧处理主循环

1. 读取一帧点云、IMU 与时间戳，做时间同步
2. 去畸变，体素降采样，地面分割，标记动态点
3. 用 IMU 或匀速模型预测当前位姿作为初值
4. 在局部地图中按最近邻建立对应关系，用距离阈值粗筛
5. 以初值为起点迭代求解位姿（扫描匹配），期间用鲁棒核降权
6. 统计内点比例：低于阈值则判定跟踪失败，进入重定位流程
7. 检查海森矩阵条件数：过大则标记该方向退化，切换融合策略
8. 按式 (12.3) 判定是否生成关键帧
9. 若是关键帧：加入局部地图窗口，计算描述子并检索回环候选
10. 把相对位姿约束与回环候选发送给后端

本章小结

1. 前端流水线是“预处理 → 数据关联 → 位姿估计 → 外点剔除 → 关键帧判定”（图 12.1），分步的目的是让每类失败都可监控、可兜底。
2. 数据关联是前端的心脏，常用 NN（式 (12.1)）、MNN、RANSAC 三件套；一条错误关联的破坏力大于十条正确关联的收益。
3. 帧到帧匹配的误差按 $\sqrt{k}$ 放大（式 (12.2)）；帧到局部地图匹配把目标换成多帧融合地图，噪声按 $\sigma/\sqrt{n}$ 降低，且误差不进入匹配目标，所以更稳。
4. 关键帧由平移、旋转、时间、信息量四类条件触发（式 (12.3)），数量过多与过少都有代价（表 12.2）。
5. 体素降采样用式 (12.4) 划分、式 (12.5) 代表，质心是“误差平方和最小”的解；RANSAC 平面拟合的抽样次数由式 (12.10) 决定。
6. Scan Context 把点云压成“环—扇区”矩阵（式 (12.12)），用列移位最小距离（式 (12.13)）做旋转不变的粗筛，再用几何校验确认。
7. 回环检测属于前端（全局数据关联），回环优化属于后端；前端典型失效有几何退化、动态物体、传感器失效、时间不同步四类（表 12.3）。

第十二章 练习

1. 设每帧匹配误差标准差 $\sigma = 1$ cm，$k = 400$ 帧后按式 (12.2) 的标准差是多少？若改成帧到地图匹配、局部地图由 $n = 9$ 帧融合，目标噪声标准差降为多少？两者相差几个数量级？
2. 在式 (12.1) 中，如果 τ_d 取得过大和过小分别会发生什么？请结合“重影墙”现象说明。
3. 为什么 RANSAC 的抽样次数只与内点比例 w 有关，而与点的总数无关？这个结论在什么条件下不成立？
4. Scan Context 的列移位为什么能处理旋转、却处理不了明显的平移？如果两帧之间存在较大平移，D 会怎样变化？
5. 机器人走在长走廊里，匹配器仍然输出了位姿。你会用哪些信号判断它其实已经退化？请给出至少两个可计算的指标。
6. 把表 12.3 中“动态物体”一行扩写成一段伪代码：你会怎样判断一个点是不是动态点？

延伸阅读

1. Kim & Kim, “Scan Context”[23]：本章回环描述子的原始论文，重点看第三节的列移位距离与第四节的候选校验。
2. 第 7 章：扫描匹配与 ICP/NDT 的完整推导，本章默认你已经掌握，只做了“升级”讨论。
3. Cartographer 论文 [19] 的前端部分：帧到子图匹配的工业级实现，可与本章 §12.3 对照阅读。
4. FAST-LIO2[41]：直接法前端的代表，看看“不做特征提取”时数据关联如何组织。

第十三章 SLAM 后端详解

本章导读

第 12 章的前端把原始数据变成一条条相对位姿约束，本章的后端要回答：拿到这一大堆互相矛盾的约束之后，怎么算出一条“全局一致”的轨迹？这是全书数学密度最高的章节之一。我们会从第 11 章的 MAP 表述出发，完整推导三件事：高斯噪声下“最大后验”如何变成“最小二乘”并如何迭代求解；位姿图的残差与雅可比如何一步一步求出来；束调整（BA）里让实时成为可能的“舒尔补”到底在做什么。最后两节讲工程上躲不开的两个问题：错误回环如何毁掉整张图（鲁棒核），以及滑窗优化里旧状态怎么“移出去而不丢信息”（边缘化）。

13.1 从最大后验到最小二乘

第 11 章已经做过完整推导：把联合后验拆成先验、运动、观测三类因子，在高斯噪声、观测独立、系统一阶马尔可夫的假设下取负对数，得到

$$X^\star = \arg\min_X \sum_{k=1}^{K} \frac{1}{2} \|\boldsymbol{r}_k(X)\|^2_{\boldsymbol{\Sigma}_k^{-1}}, \qquad \|\boldsymbol{r}\|^2_{\boldsymbol{\Sigma}^{-1}} = \boldsymbol{r}^\top \boldsymbol{\Sigma}^{-1} \boldsymbol{r}. \tag{13.1}$$

其中 $\boldsymbol{\Sigma}_k^{-1}$ 是第 k 个因子的信息矩阵（定义见第 9 章），$\boldsymbol{r}_k$ 是该因子对应的残差，X 是全部待估状态（位姿，可能还有地图点）。式 (13.1) 称为**非线性最小二乘**问题：“非线性”指 $\boldsymbol{r}_k(X)$ 一般不是 X 的线性函数（旋转矩阵、投影、对数映射都是非线性的）。它成立的三个前提决定了本章方法的边界：噪声已知且为高斯（否则二范数会被外点带偏，见 §13.5）、数据关联已确定（关联错了该式本身发现不了）、线性化只是局部的（初值太差会收敛到错误的局部极小）。

13.2 Gauss-Newton 与 Levenberg-Marquardt

式 (13.1) 没有解析解，只能迭代。高斯-牛顿（Gauss-Newton，GN）的思路是：在当前估计 X 附近把残差线性化，把非线性问题变成一系列线性子问题。

13.2.1 线性化与正规方程

设当前估计为 X，给状态一个小的增量 δ（在流形上用“$\oplus$”表示，定义见第 2 章，例如 $\boldsymbol{R} \oplus \delta\boldsymbol{\theta} = \boldsymbol{R}\exp(\delta\boldsymbol{\theta}^\wedge)$）。对残差做一阶泰勒展开：

$$\boldsymbol{r}_k(X \oplus \delta) \approx \boldsymbol{r}_k(X) + \boldsymbol{J}_k \delta, \qquad \boldsymbol{J}_k = \frac{\partial \boldsymbol{r}_k}{\partial \delta} \in \mathbb{R}^{m_k \times n}, \tag{13.2}$$

其中 m_k 是残差维数，n 是全部状态维数。代入式 (13.1)，第 k 项变成

$$\begin{aligned} \frac{1}{2}\|\boldsymbol{r}_k + \boldsymbol{J}_k\delta\|^2_{\boldsymbol{\Sigma}_k^{-1}} &= \frac{1}{2}(\boldsymbol{r}_k + \boldsymbol{J}_k\delta)^\top \boldsymbol{\Sigma}_k^{-1}(\boldsymbol{r}_k + \boldsymbol{J}_k\delta) \\ &= \frac{1}{2}\boldsymbol{r}_k^\top \boldsymbol{\Sigma}_k^{-1}\boldsymbol{r}_k + \delta^\top \boldsymbol{J}_k^\top \boldsymbol{\Sigma}_k^{-1}\boldsymbol{r}_k + \frac{1}{2}\delta^\top \boldsymbol{J}_k^\top \boldsymbol{\Sigma}_k^{-1}\boldsymbol{J}_k\delta. \end{aligned} \tag{13.3}$$

第一项与 δ 无关，可以丢掉。于是整体目标函数变成 δ 的二次型：

$$F(\delta) = \delta^\top \boldsymbol{b} + \frac{1}{2}\delta^\top \boldsymbol{H}\delta, \qquad \boldsymbol{H} = \sum_k \boldsymbol{J}_k^\top \boldsymbol{\Sigma}_k^{-1}\boldsymbol{J}_k, \quad \boldsymbol{b} = \sum_k \boldsymbol{J}_k^\top \boldsymbol{\Sigma}_k^{-1}\boldsymbol{r}_k. \tag{13.4}$$

对 δ 求导并令其为零：

$$\frac{\partial F}{\partial \delta} = \boldsymbol{b} + \boldsymbol{H}\delta = \boldsymbol{0} \implies \boldsymbol{H}\delta = -\boldsymbol{b}. \tag{13.5}$$

式 (13.5) 就是**正规方程**（定义见第 2 章），$\boldsymbol{H}$ 称为**海森矩阵**。严格的海森矩阵还应包含 $\sum_k \sum_i r_{k,i} \nabla^2 r_{k,i}$ 这样的二阶项，GN 把它忽略了：当残差接近零或残差的二阶导数很小时，这一项可以忽略；代价是初值很差时 GN 可能不收敛甚至发散。求解得到增量后按 $X \leftarrow X \oplus \delta$ 更新状态，再重新计算残差与雅可比，进入下一轮，这就是 GN 迭代。

13.2.2 Levenberg-Marquardt：给步长加个“刹车”

GN 的问题在于它假设线性化在整个步长内都有效。初值差时，算出来的 δ 可能很大，一步跨到线性化失效的区域，目标函数反而上升。Levenberg-Marquardt（LM）的解决办法很朴素：**在目标函数里惩罚过大的步长**，即求解

$$\min_{\delta} \ \frac{1}{2} \left\| \boldsymbol{r} + \boldsymbol{J}\delta \right\|^2_{\boldsymbol{\Sigma}^{-1}} + \frac{1}{2} \lambda \left\| \delta \right\|^2_2 . \tag{13.6}$$

对 δ 求导：

$$\frac{\partial}{\partial \delta} \left[\frac{1}{2} (\boldsymbol{r} + \boldsymbol{J}\delta)^{\top} \boldsymbol{\Sigma}^{-1} (\boldsymbol{r} + \boldsymbol{J}\delta) + \frac{\lambda}{2} \delta^{\top} \delta \right] = \boldsymbol{J}^{\top} \boldsymbol{\Sigma}^{-1} \boldsymbol{r} + \boldsymbol{J}^{\top} \boldsymbol{\Sigma}^{-1} \boldsymbol{J} \delta + \lambda \delta. \tag{13.7}$$

令其为零，得到 LM 的线性方程：

$$(\boldsymbol{H} + \lambda \boldsymbol{I}) \delta = -\boldsymbol{b}, \tag{13.8}$$

其中 $\boldsymbol{I}$ 是单位矩阵。式 (13.8) 的物理意义很清楚：$\lambda \to 0$ 时它退化为 GN 方程 (13.5)，适合初值好、残差小的情形；$\lambda \to \infty$ 时 $\boldsymbol{H}$ 相对可忽略，$\delta \approx -\boldsymbol{b}/\lambda$，退化为沿负梯度方向的一小步（梯度下降），适合初值差的情形。工程上常取 λ 与 $\boldsymbol{H}$ 对角线同量级，即 $(\boldsymbol{H} + \lambda \operatorname{diag}(\boldsymbol{H}))\delta = -\boldsymbol{b}$，以抵消不同状态量纲不同的影响。

LM 的迭代策略是“试探—接受/拒绝”：算出一个 δ 后先试探，若目标函数真的下降，就接受并**减小** λ（$\lambda \leftarrow \lambda/2$，更信任 GN）；若上升，就拒绝并**增大** λ（$\lambda \leftarrow 2\lambda$，更保守）。这套策略使 LM 在初值差时也能稳定收敛，是 Ceres、g2o、GTSAM 的默认选择。

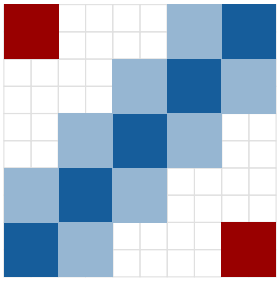
算法：Gauss-Newton / Levenberg-Marquardt 迭代

1. 输入：初始状态 X_0、最大迭代次数、收敛阈值
2. 初始化 $\lambda = \lambda_0$（如 10^{-4}），计算当前目标函数值 F_0
3. **repeat**
4. 　对每个因子计算残差 $\boldsymbol{r}_k$ 与雅可比 $\boldsymbol{J}_k$
5. 　累加 $\boldsymbol{H} = \sum_k \boldsymbol{J}_k^{\top} \boldsymbol{\Sigma}_k^{-1} \boldsymbol{J}_k$，$\boldsymbol{b} = \sum_k \boldsymbol{J}_k^{\top} \boldsymbol{\Sigma}_k^{-1} \boldsymbol{r}_k$
6. 　求解 $(\boldsymbol{H} + \lambda \boldsymbol{I})\delta = -\boldsymbol{b}$（用稀疏 Cholesky 分解）
7. 　若 $\|\delta\|$ 小于阈值：收敛，跳出
8. 　试探 $X' = X \oplus \delta$，计算 F'
9. 　若 $F' < F$：接受 $X \leftarrow X'$，$\lambda \leftarrow \lambda/2$
10. 　否则：拒绝，保留 X，$\lambda \leftarrow 2\lambda$
11. **until** 达到最大迭代次数

13.2.3 海森矩阵为什么是稀疏的

式 (13.4) 里 $\boldsymbol{H}$ 是全部因子的 $\boldsymbol{J}_k^{\top} \boldsymbol{\Sigma}_k^{-1} \boldsymbol{J}_k$ 之和。关键事实是：**每个因子只涉及少数几个状态变量**。一个里程计因子只连第 i 帧和第 $i+1$ 帧；一个点到面因子只连一帧位姿；回环因子只连两帧位姿。于是 $\boldsymbol{J}_k$ 在绝大多数列上都是零，$\boldsymbol{H}$ 的绝大部分元素也是零。

图 13.1 画出了 5 个位姿的位姿图对应的 $\boldsymbol{H}$ 结构（每个位姿 2 维，共 10 维）。对角块来自“自己与自己”的项，相邻块来自里程计边，左下与右上那个非零块来自回环边。一个 N 帧的位姿图，$\boldsymbol{H}$ 是 $2N \times 2N$（3D 时 $6N \times 6N$）的矩阵，但非零元只有 $O(N)$ 个。正是这种稀疏性，让“$N = 1000$ 也能毫秒级求解”成为可能。



5 个位姿、每个 2 维，共 10×10

图 13.1: 位姿图海森矩阵的稀疏结构。深色为对角块（单个位姿自身项），中色为相邻位姿之间的里程计块，红色为回环边产生的远距离非零块。

13.3 位姿图

定义 13.1 (因子图)

因子图（factor graph）是一种二分图，用来表示“一堆变量通过一堆约束联系在一起”的概率模型：图中有两类节点——变量节点（待估状态，如位姿、地图点）与因子节点（每个观测/先验构成的约束）。因子节点与它涉及的所有变量节点相连。式 (13.1) 的每一项就对应一个因子。 ♣

定义 13.2 (位姿图)

位姿图（pose graph）是因子图的一个特例：变量节点只有位姿 $T_1, \dots, T_N \in SE(3)$，因子节点只有两类——相邻帧之间的“里程计因子”和回环产生的“回环因子”，每个因子都是一条二元边 (i, j)，约束 T_i 与 T_j 的相对位姿。位姿图是 SLAM 后端最常用的模型，因为它把地图点全部消掉，只优化位姿，规模小得多。 ♣

13.3.1 残差的定义

对每条边 (i, j)，前端给出的测量是一个相对位姿 Z_{ij}（带信息矩阵 $\boldsymbol{\Omega}_{ij}$）。若两个位姿完全正确，那么用 T_i, T_j 算出来的相对位姿应等于测量值：

$$Z_{ij} \overset{?}{=} T_i^{-1} T_j. \tag{13.9}$$

二者之差用“误差变换” E_{ij} 表示：

$$E_{ij} = Z_{ij}^{-1}\, T_i^{-1} T_j \in SE(3). \tag{13.10}$$

E_{ij} 是单位变换就说明没有误差。用李代数把它映射成 $\mathbb{R}^6$ 中的向量（对数映射，定义见第 2 章）：

$$\boldsymbol{r}_{ij} = \log\left(Z_{ij}^{-1} T_i^{-1} T_j\right)^{\vee} \in \mathbb{R}^6. \tag{13.11}$$

式 (13.11) 中 $(\cdot)^{\vee}$ 把反对称矩阵变回向量，log 是李群到李代数的对数映射。为什么要取 log 而不是直接减？因为 $SE(3)$ 不是向量空间，两个变换“相减”没有定义；log 把“差多少”映射到切空间上，才能用欧氏范数量化。

13.3.2 雅可比的逐步推导（二维情形）

3D 的完整推导要用到 BCH 公式，篇幅很长。为了把每一步都写清楚，我们在二维 $SE(2)$ 上推导，最后给出 3D 的结论。二维位姿写成 $T_i = (\boldsymbol{R}_i, \boldsymbol{t}_i)$，其中 $\boldsymbol{R}_i$ 是 2×2 旋转矩阵，$\boldsymbol{t}_i \in \mathbb{R}^2$。

先把式 (13.11) 展开成具体分量。预测的相对位姿是

$$\hat{\boldsymbol{R}} = \boldsymbol{R}_i^{\top} \boldsymbol{R}_j, \qquad \hat{\boldsymbol{t}} = \boldsymbol{R}_i^{\top} (\boldsymbol{t}_j - \boldsymbol{t}_i). \tag{13.12}$$

误差变换 $E = Z^{-1}\hat{T}$ 的旋转与平移分别是

$$\boldsymbol{R}_E = \boldsymbol{R}_z^\top \hat{\boldsymbol{R}} = \boldsymbol{R}_z^\top \boldsymbol{R}_i^\top \boldsymbol{R}_j, \qquad \boldsymbol{t}_E = \boldsymbol{R}_z^\top (\hat{\boldsymbol{t}} - \boldsymbol{t}_z). \tag{13.13}$$

残差取 $\boldsymbol{r} = (\boldsymbol{t}_E, \theta_E)$，其中 θ_E 是 $\boldsymbol{R}_E$ 对应的转角：

$$\theta_E = \operatorname{atan2}\big(R_E(2,1),\, R_E(1,1)\big). \tag{13.14}$$

残差维数 3，正好等于单个二维位姿的自由度（2 平移 + 1 旋转）。

现在施加扰动。采用**右扰动**（扰动加在自身坐标系，见第 2 章）：$T_i \leftarrow T_i \exp(\delta_i)$，$T_j \leftarrow T_j \exp(\delta_j)$，其中 $\delta = (\delta\boldsymbol{\rho}, \delta\theta) \in \mathbb{R}^3$。对 $SE(2)$ 的指数映射做一阶展开（$\delta\theta$ 很小时 $\sin\delta\theta \approx \delta\theta$、$1-\cos\delta\theta \approx 0$）：

$$\exp(\delta) \approx \left(\boldsymbol{I} + \begin{bmatrix} 0 & -\delta\theta \\ \delta\theta & 0 \end{bmatrix},\, \delta\boldsymbol{\rho}\right) = \big(\boldsymbol{I} + [\delta\theta]_\times,\, \delta\boldsymbol{\rho}\big), \tag{13.15}$$

其中 $[\delta\theta]_\times$ 是二维反对称矩阵。于是

$$\boldsymbol{R}_i \to \boldsymbol{R}_i(\boldsymbol{I} + [\delta\theta_i]_\times), \qquad \boldsymbol{t}_i \to \boldsymbol{t}_i + \boldsymbol{R}_i \delta\boldsymbol{\rho}_i. \tag{13.16}$$

第一步：算 $\boldsymbol{t}_E$ 对 δ_i 的导数。把式 (13.16) 代入 $\hat{\boldsymbol{t}}$：

$$\begin{aligned} \hat{\boldsymbol{t}} \to \big(\boldsymbol{R}_i(\boldsymbol{I} + [\delta\theta_i]_\times)\big)^\top \big((\boldsymbol{t}_j - \boldsymbol{t}_i) - \boldsymbol{R}_i\delta\boldsymbol{\rho}_i\big) &= \big(\boldsymbol{I} - [\delta\theta_i]_\times\big)\boldsymbol{R}_i^\top(\boldsymbol{t}_j - \boldsymbol{t}_i) - \big(\boldsymbol{I} - [\delta\theta_i]_\times\big)\delta\boldsymbol{\rho}_i \\ &= \hat{\boldsymbol{t}} - [\delta\theta_i]_\times \hat{\boldsymbol{t}} - \delta\boldsymbol{\rho}_i + O(\delta^2). \end{aligned} \tag{13.17}$$

这里用了 $(\boldsymbol{R}_i(\boldsymbol{I} + [\delta\theta]_\times))^\top = (\boldsymbol{I} + [\delta\theta]_\times)^\top \boldsymbol{R}_i^\top = (\boldsymbol{I} - [\delta\theta]_\times)\boldsymbol{R}_i^\top$，以及 $\boldsymbol{R}_i^\top \boldsymbol{R}_i = \boldsymbol{I}$。所以

$$\delta\hat{\boldsymbol{t}} = -\delta\boldsymbol{\rho}_i - [\delta\theta_i]_\times \hat{\boldsymbol{t}}. \tag{13.18}$$

再由 $\boldsymbol{t}_E = \boldsymbol{R}_z^\top(\hat{\boldsymbol{t}} - \boldsymbol{t}_z)$ 得

$$\delta\boldsymbol{t}_E = \boldsymbol{R}_z^\top \delta\hat{\boldsymbol{t}} = -\boldsymbol{R}_z^\top \delta\boldsymbol{\rho}_i - \delta\theta_i\, \boldsymbol{R}_z^\top [\boldsymbol{e}_z]_\times \hat{\boldsymbol{t}} = -\boldsymbol{R}_z^\top \delta\boldsymbol{\rho}_i - \delta\theta_i\, [\boldsymbol{e}_z]_\times \boldsymbol{R}_z^\top \hat{\boldsymbol{t}}, \tag{13.19}$$

最后一步用了二维情形的交换关系 $\boldsymbol{R}_z^\top [\boldsymbol{e}_z]_\times = [\boldsymbol{e}_z]_\times \boldsymbol{R}_z^\top$（旋转与“逆时针转 90 度”可交换）。式 (13.19) 说明：$\delta\boldsymbol{\rho}_i$ 经 $\boldsymbol{R}_z^\top$ 旋转后直接进入 $\boldsymbol{t}_E$，而 $\delta\theta_i$ 的效果是让 $\hat{\boldsymbol{t}}$ 绕原点转了一下，幅度与 $\hat{\boldsymbol{t}}$ 的长度成正比。

第二步：算 $\boldsymbol{t}_E$ 对 δ_j 的导数。同理

$$\hat{\boldsymbol{t}} \to \boldsymbol{R}_i^\top \big(\boldsymbol{t}_j + \boldsymbol{R}_j\delta\boldsymbol{\rho}_j - \boldsymbol{t}_i\big) = \hat{\boldsymbol{t}} + \boldsymbol{R}_i^\top \boldsymbol{R}_j \delta\boldsymbol{\rho}_j = \hat{\boldsymbol{t}} + \hat{\boldsymbol{R}}\, \delta\boldsymbol{\rho}_j, \tag{13.20}$$

于是

$$\delta\boldsymbol{t}_E = \boldsymbol{R}_z^\top \hat{\boldsymbol{R}}\, \delta\boldsymbol{\rho}_j = \boldsymbol{R}_E\, \delta\boldsymbol{\rho}_j, \tag{13.21}$$

即 $\partial\boldsymbol{t}_E / \partial\delta\boldsymbol{\rho}_j = \boldsymbol{R}_E$，而 $\partial\boldsymbol{t}_E / \partial\delta\theta_j = \boldsymbol{0}$（$\hat{\boldsymbol{t}}$ 与 $\boldsymbol{R}_j$ 无关）。

第三步：算 θ_E 的导数。由式 (13.13)，

$$\boldsymbol{R}_E \to \boldsymbol{R}_z^\top \big(\boldsymbol{R}_i(\boldsymbol{I} + [\delta\theta_i]_\times)\big)^\top \boldsymbol{R}_j = \boldsymbol{R}_z^\top (\boldsymbol{I} - [\delta\theta_i]_\times)\boldsymbol{R}_i^\top \boldsymbol{R}_j = \boldsymbol{R}_E - \boldsymbol{R}_E [\delta\theta_i]_\times. \tag{13.22}$$

最后一步把 $[\delta\theta_i]_\times$ 移到最右边（同样利用交换性）。而 $\boldsymbol{R}_E - \boldsymbol{R}_E[\delta\theta]_\times = \boldsymbol{R}_E(\boldsymbol{I} - [\delta\theta]_\times) = \boldsymbol{R}_E\boldsymbol{R}(-\delta\theta)$，相当于转角减少 $\delta\theta_i$，所以

$$\frac{\partial\theta_E}{\partial\delta\theta_i} = -1. \tag{13.23}$$

对 δ_j 同理有 $\boldsymbol{R}_E \to \boldsymbol{R}_E(\boldsymbol{I} + [\delta\theta_j]_\times) = \boldsymbol{R}_E\boldsymbol{R}(\delta\theta_j)$，转角增加 $\delta\theta_j$，故 $\partial\theta_E/\partial\delta\theta_j = 1$。平移扰动对转角没有影响。

第四步：拼成矩阵。把上面三步的结果按 $\delta = (\delta\boldsymbol{\rho}, \delta\theta)$ 的列顺序拼起来，得到 3×6 的分块雅可比：

$$\frac{\partial \boldsymbol{r}}{\partial(\delta_i, \delta_j)} = \left[\begin{array}{cc|cc} -\boldsymbol{R}_z^\top & -[\boldsymbol{e}_z]_\times \boldsymbol{R}_z^\top \hat{\boldsymbol{t}} & \boldsymbol{R}_E & \boldsymbol{0} \\ \boldsymbol{0}^\top & -1 & \boldsymbol{0}^\top & 1 \end{array}\right] \in \mathbb{R}^{3\times 6}. \tag{13.24}$$

（为紧凑起见，矩阵中每个 2×2 或 2×1 的块各写成一个元素；按行展开即为标准的 3×6 矩阵。）式 (13.24) 中左半 3×3 是残差对 T_i 的雅可比，右半 3×3 是对 T_j 的雅可比，所有量都在当前估计处求值，每轮迭代重算一次。读它时不必死记：平移残差对自己平移扰动的响应是 $-\boldsymbol{R}_z^\top$（把误差转到测量坐标系），旋转残差对自己是 -1、对对方是 $+1$（因为相对位姿是 $T_i^{-1}T_j$，一正一负），而两处旋转扰动都会“串”到平移残差里。

13.3.3 三维情形

三维位姿图残差仍是式 (13.11)，只是 $\boldsymbol{r}_{ij} \in \mathbb{R}^6$（3 平移 + 3 旋转）。按同样的 “右扰动 + 一阶 BCH” 推导（BCH 公式见第 2 章），可得

$$\frac{\partial \boldsymbol{r}_{ij}}{\partial \delta \boldsymbol{\xi}_i} = -\boldsymbol{J}_l^{-1}(\boldsymbol{r}_{ij}) \operatorname{Ad}\left(T_j^{-1} T_i\right), \qquad \frac{\partial \boldsymbol{r}_{ij}}{\partial \delta \boldsymbol{\xi}_j} = \boldsymbol{J}_r^{-1}(\boldsymbol{r}_{ij}), \tag{13.25}$$

其中 $\operatorname{Ad}(\cdot)$ 是 $SE(3)$ 的伴随矩阵，

$$\operatorname{Ad}(T) = \begin{bmatrix} \boldsymbol{R} & [\boldsymbol{t}]_\times \boldsymbol{R} \\ \boldsymbol{0} & \boldsymbol{R} \end{bmatrix} \in \mathbb{R}^{6\times 6}, \tag{13.26}$$

$\boldsymbol{J}_l^{-1}, \boldsymbol{J}_r^{-1}$ 是 $SE(3)$ 左/右雅可比的逆。当残差较小时 $\boldsymbol{J}_l^{-1} \approx \boldsymbol{J}_r^{-1} \approx \boldsymbol{I}$，很多实现直接采用简化形式

$$\frac{\partial \boldsymbol{r}_{ij}}{\partial \delta \boldsymbol{\xi}_i} \approx -\operatorname{Ad}\left(T_j^{-1} T_i\right), \qquad \frac{\partial \boldsymbol{r}_{ij}}{\partial \delta \boldsymbol{\xi}_j} \approx \boldsymbol{I}, \tag{13.27}$$

这也是 g2o[24] 中位姿图边的经典形式。式 (13.25) 的推导与二维完全同构：把式 (13.16) 里的 $\boldsymbol{R}_i(\boldsymbol{I} + [\delta\theta]_\times)$ 换成 $\boldsymbol{R}_i \exp([\delta\boldsymbol{\theta}]_\times)$，把标量转角换成 $\mathbb{R}^3$ 旋转向量，把二维叉乘换成 3×3 反对称矩阵，再用 BCH 的一阶展开把 “exp 的乘积” 并成 “对数的和”。

13.4 从位姿图到 BA：舒尔补

位姿图把地图点消掉了。但如果我们要同时优化 “相机位姿 + 地图点”，模型里就多了一大类变量。这就是光束法平差（Bundle Adjustment，BA，定义见第 8 章）：以重投影误差为残差，同时优化相机位姿与三维点。

13.4.1 BA 的残差与雅可比

设第 i 个相机位姿为 $T_i \in SE(3)$（世界到相机的变换），第 j 个地图点在世界系下为 $\boldsymbol{X}_j \in \mathbb{R}^3$。投影与重投影残差为

$$\boldsymbol{u}_{ij} = \pi(\boldsymbol{R}_i \boldsymbol{X}_j + \boldsymbol{t}_i), \qquad \boldsymbol{r}_{ij} = \boldsymbol{z}_{ij} - \pi(\boldsymbol{R}_i \boldsymbol{X}_j + \boldsymbol{t}_i) \in \mathbb{R}^2, \tag{13.28}$$

其中 $\pi(\boldsymbol{p}) = \left(f_x p_x / p_z + c_x,\ f_y p_y / p_z + c_y\right)^\top$ 是针孔投影（f_x, f_y 为焦距，c_x, c_y 为主点），$\boldsymbol{z}_{ij}$ 是观测到的像素坐标。残差由两部分雅可比组成。先算投影对相机坐标的导数：

$$\frac{\partial \pi}{\partial \boldsymbol{p}} = \begin{bmatrix} \dfrac{f_x}{p_z} & 0 & -\dfrac{f_x p_x}{p_z^2} \\ 0 & \dfrac{f_y}{p_z} & -\dfrac{f_y p_y}{p_z^2} \end{bmatrix} \in \mathbb{R}^{2\times 3}. \tag{13.29}$$

再算相机坐标对位姿扰动 $\delta\boldsymbol{\xi} = (\delta\boldsymbol{\rho}, \delta\boldsymbol{\phi})$ 的导数。位姿扰动 $T \leftarrow T\exp(\delta\boldsymbol{\xi})$ 作用在点上，一阶展开为 $\boldsymbol{p} \to \boldsymbol{p} + [\delta\boldsymbol{\phi}]_\times \boldsymbol{p} + \delta\boldsymbol{\rho}$，写成矩阵即

$$\frac{\partial \boldsymbol{p}}{\partial \delta \boldsymbol{\xi}} = \begin{bmatrix} \boldsymbol{I} & -[\boldsymbol{p}]_\times \end{bmatrix} \in \mathbb{R}^{3\times 6} \tag{13.30}$$

（因为 $[\delta\boldsymbol{\phi}]_\times \boldsymbol{p} = -[\boldsymbol{p}]_\times \delta\boldsymbol{\phi}$）。对点 $\boldsymbol{X}_j$ 的导数是 $\partial \boldsymbol{p}/\partial \boldsymbol{X}_j = \boldsymbol{R}_i$。链式相乘，最终

$$\boldsymbol{J}_{ij}^{\text{pose}} = -\frac{\partial \pi}{\partial \boldsymbol{p}} \begin{bmatrix} \boldsymbol{I} & -[\boldsymbol{p}_{ij}]_\times \end{bmatrix} \in \mathbb{R}^{2\times 6}, \qquad \boldsymbol{J}_{ij}^{\text{point}} = -\frac{\partial \pi}{\partial \boldsymbol{p}} \boldsymbol{R}_i \in \mathbb{R}^{2\times 3}. \tag{13.31}$$

看式 (13.31) 的维度就知道：**一条观测只涉及一个位姿（6 维）和一个点（3 维）**，所以 $\boldsymbol{H}$ 一定是极端稀疏的。

13.4.2 分块与舒尔补

把所有变量排成 “相机在前、点在后” 的顺序，海森矩阵自然分成四块：

$$\begin{bmatrix} \boldsymbol{H}_{cc} & \boldsymbol{H}_{cp} \\ \boldsymbol{H}_{pc} & \boldsymbol{H}_{pp} \end{bmatrix} \begin{bmatrix} \delta\boldsymbol{\xi}_c \\ \delta\boldsymbol{X}_p \end{bmatrix} = \begin{bmatrix} -\boldsymbol{b}_c \\ -\boldsymbol{b}_p \end{bmatrix}. \tag{13.32}$$

其中 $\boldsymbol{H}_{cc}$ 是“相机—相机”块（尺寸 $6N_c \times 6N_c$），$\boldsymbol{H}_{pp}$ 是“点—点”块（$3N_p \times 3N_p$），$\boldsymbol{H}_{cp} = \boldsymbol{H}_{pc}^{\top}$ 是交叉块。

关键观察：**每个三维点只在“观测到它的那些相机”里出现，点与点之间没有任何因子**。所以 $\boldsymbol{H}_{pp}$ 是**块对角**的：

$$\boldsymbol{H}_{pp} = \operatorname{diag}(\boldsymbol{H}_1, \boldsymbol{H}_2, \ldots, \boldsymbol{H}_{N_p}), \qquad \boldsymbol{H}_j \in \mathbb{R}^{3\times 3}. \tag{13.33}$$

块对角矩阵的逆就是逐块求逆，每个只有 3×3，极其便宜。这正是舒尔补能用的前提。

定义 13.3 (舒尔补)

对分块矩阵 $\begin{bmatrix} \boldsymbol{A} & \boldsymbol{B} \\ \boldsymbol{C} & \boldsymbol{D} \end{bmatrix}$（$\boldsymbol{D}$ 可逆），称 $\boldsymbol{A} - \boldsymbol{B}\boldsymbol{D}^{-1}\boldsymbol{C}$ 为其（关于 $\boldsymbol{D}$ 的）舒尔补（Schur complement）。它在消去分块线性方程组中的变量 δ_p 时自然出现。

♣

下面一步一步消去 $\delta\boldsymbol{X}_p$。把式 (13.32) 写成两个方程：

$$\boldsymbol{H}_{cc}\delta\boldsymbol{\xi}_c + \boldsymbol{H}_{cp}\delta\boldsymbol{X}_p = -\boldsymbol{b}_c, \tag{13.34}$$

$$\boldsymbol{H}_{pc}\delta\boldsymbol{\xi}_c + \boldsymbol{H}_{pp}\delta\boldsymbol{X}_p = -\boldsymbol{b}_p. \tag{13.35}$$

由式 (13.35) 解出 $\delta\boldsymbol{X}_p$（$\boldsymbol{H}_{pp}$ 块对角可逆）：

$$\delta\boldsymbol{X}_p = \boldsymbol{H}_{pp}^{-1}\big(-\boldsymbol{b}_p - \boldsymbol{H}_{pc}\delta\boldsymbol{\xi}_c\big). \tag{13.36}$$

代入式 (13.34)：

$$\begin{aligned} \boldsymbol{H}_{cc}\delta\boldsymbol{\xi}_c + \boldsymbol{H}_{cp}\boldsymbol{H}_{pp}^{-1}\big(-\boldsymbol{b}_p - \boldsymbol{H}_{pc}\delta\boldsymbol{\xi}_c\big) &= -\boldsymbol{b}_c \\ \boldsymbol{H}_{cc}\delta\boldsymbol{\xi}_c - \boldsymbol{H}_{cp}\boldsymbol{H}_{pp}^{-1}\boldsymbol{H}_{pc}\delta\boldsymbol{\xi}_c &= -\boldsymbol{b}_c + \boldsymbol{H}_{cp}\boldsymbol{H}_{pp}^{-1}\boldsymbol{b}_p \\ \Big(\boldsymbol{H}_{cc} - \boldsymbol{H}_{cp}\boldsymbol{H}_{pp}^{-1}\boldsymbol{H}_{pc}\Big)\delta\boldsymbol{\xi}_c &= -\boldsymbol{b}_c + \boldsymbol{H}_{cp}\boldsymbol{H}_{pp}^{-1}\boldsymbol{b}_p. \end{aligned} \tag{13.37}$$

式 (13.37) 就是**缩减相机系统**（reduced camera system），它的系数矩阵

$$\boldsymbol{S} = \boldsymbol{H}_{cc} - \boldsymbol{H}_{cp}\boldsymbol{H}_{pp}^{-1}\boldsymbol{H}_{pc} \tag{13.38}$$

正是舒尔补。解出 $\delta\boldsymbol{\xi}_c$ 后，回代式 (13.36) 就得到所有点的增量。

为什么这能“实时”？设 $N_c = 200$ 个相机、$N_p = 20000$ 个点：直接解 $\boldsymbol{H}\delta = -\boldsymbol{b}$ 是 $6\times 200 + 3\times 20000 = 61200$ 维，点变量极多、填充（fill-in）严重；舒尔补后只解 $6\times 200 = 1200$ 维的 $\boldsymbol{S}$，点变量则通过 3×3 块逐块求逆、逐块回代，计算量与点数成线性。两者规模比约为 $(61200/1200)^3 \approx 1.3\times 10^5$，这就是“BA 能实时”的秘密。位姿图可以看作“把所有点都边缘化掉之后的 BA”，这也是它规模小、速度快的原因。

算法：基于舒尔补的 BA 一轮求解

1. 遍历所有观测，累加 $\boldsymbol{H}_{cc}, \boldsymbol{H}_{cp}, \boldsymbol{H}_{pp}, \boldsymbol{b}_c, \boldsymbol{b}_p$
2. 对每个点 j：求 $\boldsymbol{H}_j^{-1}$（3×3），计算 $\boldsymbol{H}_j^{-1}\boldsymbol{H}_{pc}$ 的对应行块与 $\boldsymbol{H}_j^{-1}\boldsymbol{b}_p$
3. 累加 $\boldsymbol{S} = \boldsymbol{H}_{cc} - \sum_j \boldsymbol{H}_{cp,j}\boldsymbol{H}_j^{-1}\boldsymbol{H}_{pc,j}$ 与右端
4. 用稀疏 Cholesky 解 $\boldsymbol{S}\,\delta\boldsymbol{\xi}_c =$ 右端
5. 对每个点 j：回代 $\delta\boldsymbol{X}_j = \boldsymbol{H}_j^{-1}(-\boldsymbol{b}_{p,j} - \boldsymbol{H}_{pc,j}\delta\boldsymbol{\xi}_c)$
6. 用 $\oplus$ 更新全部位姿与点，进入下一轮迭代

13.5 回环与鲁棒核

回环闭合是后端最有价值、也最危险的功能。图 13.2 上半部分是正常情况下的一条链式因子图：位姿 $x_0 \to x_1 \to \cdots \to x_5$，相邻帧有里程计因子，$x_5$ 与 x_0 之间有一条回环因子。正是这条回环边把“漂移了几十厘米”的轨迹拉回闭合。下半部分是错误回环：把“看起来很像、其实不是同一个地方”的两帧连起来，优化器为了满足这条假约束，会把整段轨迹拧成一团。

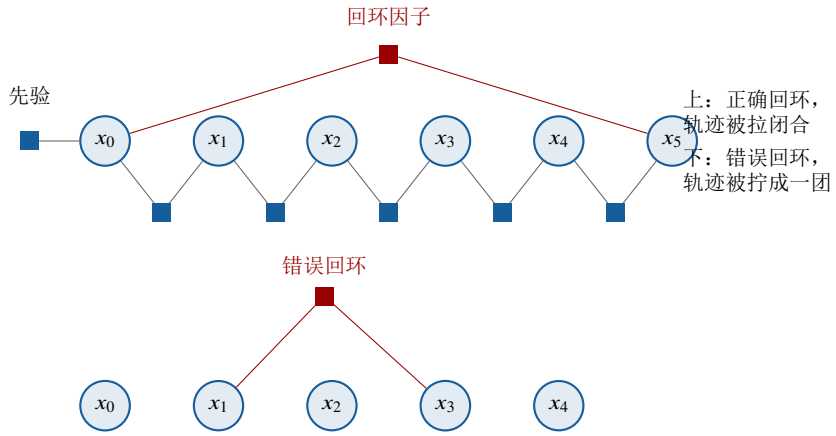


图 13.2: 上：正确的回环。圆圈是位姿变量，实心方块是因子节点；相邻位姿由里程计因子相连，x_5 与 x_0 之间的回环因子把累积漂移拉回闭合。下：错误回环。x_1 与 x_3 被错误地连起来，优化器为了满足这条假约束会把轨迹拧成一团，破坏地图一致性。

问题来了：最小二乘对残差是**平方**惩罚，一条错边的残差很大时，平方会把它的“话语权”放大到压倒所有正确约束。鲁棒核函数（robust kernel / M-estimator）的思路是：**让大残差的惩罚增长得比平方慢**，从而自动给可疑约束降权。

定义 13.4 (鲁棒核函数)

鲁棒核函数是一个非负、单调递增、在 $s=0$ 处为 0 的函数 $\rho(s)$，用于把“残差的平方” $\|\boldsymbol{r}\|_{\boldsymbol{\Omega}}^2$ 替换成 $\rho\big(\|\boldsymbol{r}\|_{\boldsymbol{\Omega}}^2\big)$，使优化目标变为

$$\min_{X} \sum_{k} \rho(s_k), \qquad s_k = \|\boldsymbol{r}_k\|_{\boldsymbol{\Omega}_k}^2 . \tag{13.39}$$

当 $\rho(s)=s$ 时退化为普通最小二乘；ρ 在 s 很大时增长得比线性慢（次线性），就得到鲁棒性。

♣

最常用的是 Huber 核：

$$\rho(s) = \begin{cases} s, & s \le \delta^2, \\ 2\delta\sqrt{s} - \delta^2, & s > \delta^2, \end{cases} \tag{13.40}$$

其中 δ 是“软阈值”，s 是残差平方范数（注意 $\sqrt{s} = \|\boldsymbol{r}\|_{\boldsymbol{\Omega}}$）。在 $s=\delta^2$ 处两侧值相等（δ^2），导数也相等，所以 ρ 是连续可导的。

推导它对迭代求解的影响。目标函数对状态的梯度为

$$\frac{\partial}{\partial X}\sum_{k}\rho(s_k) = \sum_{k}\rho'(s_k)\frac{\partial s_k}{\partial X} = \sum_{k}\rho'(s_k)\cdot 2\,\boldsymbol{J}_k^{\top}\boldsymbol{\Omega}_k\boldsymbol{r}_k, \tag{13.41}$$

其中用了 $\partial s_k/\partial X = 2\boldsymbol{J}_k^{\top}\boldsymbol{\Omega}_k\boldsymbol{r}_k$。令梯度为零、约掉常数 2，得

$$\sum_{k} w_k\,\boldsymbol{J}_k^{\top}\boldsymbol{\Omega}_k\boldsymbol{r}_k = \boldsymbol{0}, \qquad w_k = \rho'(s_k). \tag{13.42}$$

再用 GN 的线性化 $\boldsymbol{r}_k + \boldsymbol{J}_k\delta$ 代替 $\boldsymbol{r}_k$，就得到**加权正规方程**：

$$\Big(\sum_{k} w_k \boldsymbol{J}_k^{\top}\boldsymbol{\Omega}_k\boldsymbol{J}_k\Big)\delta = -\sum_{k} w_k \boldsymbol{J}_k^{\top}\boldsymbol{\Omega}_k\boldsymbol{r}_k. \tag{13.43}$$

式 (13.43) 与普通 GN 的唯一区别是每个因子的信息矩阵被乘上了权重 w_k。对 Huber 核，$\rho'(s)=1$（$s\le\delta^2$）或 $\rho'(s)=\delta/\sqrt{s}$（$s>\delta^2$），于是

$$w_k = \min\Big(1,\ \frac{\delta}{\|\boldsymbol{r}_k\|_{\boldsymbol{\Omega}_k}}\Big). \tag{13.44}$$

这就是 Huber 核的全部魔力：**残差小于阈值时权重为 1（完全相信），残差大于阈值时权重与残差成反比。**残差再大，它对法方程的贡献也不会超过阈值对应的量级。这种“用当前残差算权重、再解加权最小二乘”的迭代方

式，称为迭代重加权最小二乘（Iteratively Reweighted Least Squares，IRLS）。

鲁棒核不能“修正”错误关联

鲁棒核只是**降权**，不是**修正**。一条错误的回环边即便权重降到很低，它仍然在“往错误方向拉”，只是拉得轻了；当错误边很多、或者错误边与其他边形成“一致但错误”的组合时，降权救不了场。所以工程上要三层保险：前端回环几何校验（第 12 章）、后端鲁棒核降权、以及**开关变量**（给每条回环边一个 $s_k \in [0,1]$ 的连续变量，与位姿一起优化，收敛到 0 表示“这条边被判定为假”）。开源实现里 GTSAM 的 `SwitchableConstraint` 就是这个思想 [7]。

13.6 边缘化：把旧状态“移出去”但不丢信息

滑窗优化的困境是：窗口里只能保留最近 M 个位姿，更旧的状态必须移出窗口，否则计算量无限增长。但直接把旧状态“删掉”会丢信息——旧状态和保留状态之间是相关的，删掉就等于假装它们独立，会引入不一致。

正确做法是**边缘化**：把旧状态从概率分布里积掉，留下它对保留状态的影响。在线性化框架下，这件事正好又是舒尔补。

定义 13.5 (边缘化)

边缘化（marginalization）是指从联合概率分布 $p(X_m, X_r)$ 中，通过对被移除变量 X_m 积分得到保留变量 X_r 的边缘分布：

$$p(X_r) = \int p(X_m, X_r)\, \mathrm{d}X_m. \tag{13.45}$$

在高斯情形下，积分的结果仍是一个高斯分布，其信息矩阵恰好是舒尔补。

推导与 §13.4 的块消元完全相同：把联合目标函数在旧线性化点附近写成二次型，按 (δ_m, δ_r) 分块，从 δ_m 那一块解出

$$\boldsymbol{H}_{mm}\delta_m + \boldsymbol{H}_{mr}\delta_r + \boldsymbol{b}_m = \boldsymbol{0} \;\Longrightarrow\; \delta_m = \boldsymbol{H}_{mm}^{-1}\big(-\boldsymbol{b}_m - \boldsymbol{H}_{mr}\delta_r\big), \tag{13.46}$$

再代回另一块，就得到只含 δ_r 的二次型：

$$F(\delta_r) = \frac{1}{2}\delta_r^{\top}\underbrace{\left(\boldsymbol{H}_{rr} - \boldsymbol{H}_{rm}\boldsymbol{H}_{mm}^{-1}\boldsymbol{H}_{mr}\right)}_{\boldsymbol{\Lambda}}\delta_r + \underbrace{\left(\boldsymbol{b}_r - \boldsymbol{H}_{rm}\boldsymbol{H}_{mm}^{-1}\boldsymbol{b}_m\right)^{\top}}_{\boldsymbol{\eta}}\delta_r + \text{常数}. \tag{13.47}$$

式 (13.47) 中的 $\boldsymbol{\Lambda}$ 与 $\boldsymbol{\eta}$ 定义了一个关于 δ_r 的先验因子：$\boldsymbol{\Lambda}$ 是它的信息矩阵，$\boldsymbol{\eta}$ 是信息向量。下一轮优化时，把这个先验因子加到目标函数里，就等价于“没有丢掉旧状态的信息”。

边缘化有一个绕不开的副作用：**线性化误差**。式 (13.47) 是在旧线性化点 $\bar{X}$ 附近做的二次近似，它只在 $\bar{X}$ 附近有效。随着后续迭代把 X_r 更新到离 $\bar{X}$ 越来越远，这个先验会“拉”住新的估计，产生偏差。更麻烦的是，如果每一轮都用新的线性化点重新计算 $\boldsymbol{\Lambda}, \boldsymbol{\eta}$，边缘化的“零点”会漂移，导致同一信息被重复计数或丢失（不一致）。工程上的标准解法是**固定线性化点**（First Estimate Jacobian，FEJ）：边缘化产生的先验，其雅可比始终在第一次线性化点处计算，不再更新。这是 VINS-Mono[31]、OKVIS 等滑窗 VIO 的关键实现细节。

13.7 求解器工程

最后把“用什么库”这件事说清楚。表 13.1 对比了三个最常用的后端库。

三者底层是同一件事：构造 $\boldsymbol{H}\delta = -\boldsymbol{b}$，用稀疏 Cholesky（如 SuiteSparse 的 CHOLMOD）求解；区别只在**如何组织因子**与**如何利用增量信息**。g2o 用“顶点—边”的图结构，最贴近 $\boldsymbol{H}$ 的块稀疏本质，但不是增量式的，每轮都要重新分解；GTSAM 的 iSAM2 用贝叶斯树（Bayes tree）只更新被新因子影响的部分，适合在线长期运行；Ceres 提供自动微分与齐全的鲁棒核，写残差最省事，但通常整图重解。量级参考（桌面 CPU）：1000 个节点的

表 13.1: 三个常用后端求解器的定位

库	模型组织方式	优势	典型使用场景
g2o	图（顶点/边），C++	轻量、可定制、贴合 ROS 生态	教学与研究，Cartographer 早期版本
GTSAM	因子图，C++	增量式求解（iSAM2）、理论完备	LIO-SAM、需要在线增量的系统
Ceres	通用非线性最小二乘	自动微分、鲁棒核齐全、性能强	Cartographer、VINS-Mono

位姿图一轮优化 10 ~ 50 ms；200 个相机 + 2 万个点的 BA 一轮 50 ~ 200 ms。所以后端通常**降频运行**（每 1 ~ 5 s 一次，或检测到回环时触发），而不是每帧都优化。

本章小结

1. 后端要解的是第 11 章得到的非线性最小二乘式 (13.1)（前提：噪声高斯、关联正确、初值不太差）。Gauss-Newton 用一阶泰勒展开把残差线性化，得到正规方程 $\boldsymbol{H}\delta = -\boldsymbol{b}$，其中 $\boldsymbol{H} = \sum \boldsymbol{J}^{\top}\boldsymbol{\Sigma}^{-1}\boldsymbol{J}$、$\boldsymbol{b} = \sum \boldsymbol{J}^{\top}\boldsymbol{\Sigma}^{-1}\boldsymbol{r}$（式 (13.4)）；海森矩阵稀疏的根本原因是每个因子只连少数变量（图 13.1）。
2. LM 在目标函数上加 $\frac{\lambda}{2}\|\delta\|^2$ 的惩罚，得到 $(\boldsymbol{H}+\lambda\boldsymbol{I})\delta = -\boldsymbol{b}$（式 (13.8)）；$\lambda$ 大时接近梯度下降、小时接近 GN，按 “下降就减、上升就增” 调节，因此初值差时也稳。
3. 位姿图是只有位姿变量的因子图，残差取误差变换的李代数 $\boldsymbol{r}_{ij} = \log(Z_{ij}^{-1}T_i^{-1}T_j)^{\vee}$（式 (13.11)）；二维情形可逐步求出 3×6 的分块雅可比（式 (13.24)），3D 情形用伴随矩阵写成式 (13.25)。
4. BA 的 $\boldsymbol{H}$ 按相机与点分块后，$\boldsymbol{H}_{pp}$ 块对角，消去点变量得到缩减相机系统，其系数正是舒尔补 $\boldsymbol{S} = \boldsymbol{H}_{cc} - \boldsymbol{H}_{cp}\boldsymbol{H}_{pp}^{-1}\boldsymbol{H}_{pc}$（式 (13.38)），求解规模从 “点数” 降到 “相机数”。
5. 鲁棒核把残差平方换成次线性增长函数；Huber 核给出的 IRLS 权重是 $w_k = \min(1, \delta/\|\boldsymbol{r}_k\|)$（式 (13.44)），本质是给可疑约束降权，但**不能修正**错误关联。
6. 边缘化用舒尔补把旧状态积掉，得到保留状态的先验因子（式 (13.47)）；代价是线性化误差，工程上用固定线性化点（FEJ）缓解。g2o / GTSAM / Ceres 底层都是稀疏最小二乘，区别在因子组织方式与是否增量求解，后端通常降频运行。

第十三章 练习

1. 在式 (13.1) 中，若某个因子的信息矩阵 $\boldsymbol{\Sigma}^{-1}$ 取为单位阵，最小二乘就退化为 “残差平方和”。此时 Huber 核与直接对残差做截断（把大残差直接丢弃）有什么区别？
2. 用 $SU(2)$ 或 $SO(2)$ 的直觉解释：为什么位姿图残差必须取 log，而不能直接把两个变换矩阵相减再取范数？举一个数值反例。
3. 在式 (13.24) 中令 $\boldsymbol{R}_z = \boldsymbol{I}$、$\hat{\boldsymbol{t}} = \boldsymbol{0}$，请写出简化后的雅可比矩阵，并解释它的物理含义。
4. 假设 BA 有 $N_c = 100$ 个相机、$N_p = 10^4$ 个点。直接解与舒尔补解各要处理多少维的线性系统？若 Cholesky 复杂度为 $O(n^3)$，两者相差多少倍？
5. 边缘化产生的先验因子为什么不能每轮用新的线性化点重算？如果不固定线性化点，可能出现什么现象（提示：信息被重复计数或丢失）？

延伸阅读

1. Dellaert & Kaess, “Factor Graphs for Robot Perception”[7]：因子图与增量求解的系统教程，第 3–4 章可配合本章 §13.3 阅读。
2. Kümmerle 等，g2o 论文 [24]：本章位姿图部分的工程对照，重点看 “Types of Edges” 与稀疏求解器；Barfoot《State Estimation for Robotics》[1] 第 7 章可补 $SE(3)$ 雅可比与伴随矩阵的细节。

第十四章 地图与建图

本章导读

前两章都在回答“我在哪”，本章回答“世界是什么样”。地图是 SLAM 的输出，也是导航的输入：没有地图，全局规划无从谈起；地图错了，定位与避障都会跟着错。本章沿着“地图给谁用”这条线索，把最常用的五种地图讲清楚：占据栅格（概率化的二维地图）、八叉树（省内存的三维占据地图）、TSDF（能重建光滑表面的截断符号距离场）、ESDF（能给规划提供梯度的距离场）、以及点云地图。重点是两条完整推导：占据栅格的 log-odds 更新如何从贝叶斯公式一步步推出来，以及 TSDF 的加权平均为什么是“最小二乘”的自然结果。

14.1 地图的用途决定地图的形式

新手常问“哪种地图最好”，这个问题没有答案，因为地图是为用途服务的。同一片场地，用来做全局定位的“地图”可能是一张点云，用来做路径规划的“地图”可能是一张二维栅格，用来做局部避障的“地图”可能是占据栅格，用来做轨迹优化的“地图”则是 ESDF。表 14.1 给出常见的对应关系。

表 14.1: 地图类型与用途的对应关系（✓ 表示适合，∘ 表示勉强可用）

地图类型	定位	全局规划	局部避障	表面重建	内存
点云地图	✓	∘	∘	✓	大
占据栅格（2D）	✓	✓	✓	∘	中
八叉树（3D 占据）	✓	✓	✓	∘	中
TSDF	∘	∘	∘	✓	大
ESDF	∘	✓	✓	∘	大

表 14.1 的读法是：**要定位就选能算似然的**（点云、占据栅格），**要规划就选能算代价的**（栅格、ESDF），**要重建就选能算距离的**（TSDF）。实际系统里往往同时维护好几张地图，各司其职。

地图的三条工程准则

1. **分辨率决定一切**：5 cm 的栅格与 20 cm 的栅格是两种应用，前者能看清桌腿，后者只能看墙。分辨率减半，内存变四倍（2D）或八倍（3D）。
2. **只维护“用得到”的区域**：机器人周围 20 m 内的地图需要高频更新，远处的地图只需保存；这是八叉树、体素地图存在的理由。
3. **地图必须能“变旧”**：环境会变（人走了、箱子搬了），地图若永远只增不减，就会把历史当成现实。所有工程地图都要么带时间戳，要么允许权重衰减。

14.2 占据栅格地图与 log-odds 更新

定义 14.1 (占据栅格)

占据栅格（occupancy grid）把空间切成规则的格子（2D 是正方形，3D 是立方体），每个格子 m_i 只取两个状态：被占据（$m_i = 1$，有障碍）或空闲（$m_i = 0$，可通行）。由于传感器有噪声，我们不存“是/否”，而存“被占据的概率” $p(m_i = 1 \mid z_{1:t})$。

一张 100 m × 100 m、分辨率 5 cm 的二维栅格有 $2000 \times 2000 = 4 \times 10^6$ 个格子；若用单精度浮点存概率，需要 16 MB。这还只是一层。所以存储格式的优化从第一天就要考虑——这就是为什么工程上不存概率，而存“概率的对数”（log-odds）。

14.2.1 从贝叶斯公式推导 log-odds

目标：给定从第 1 帧到第 t 帧的全部观测 $z_{1:t}$，求每个格子被占据的后验概率 $p(m_i \mid z_{1:t})$。为简洁，记 $m = m_i$。由贝叶斯公式（定义见第 2 章）：

$$p(m \mid z_{1:t}) = \frac{p(z_t \mid m, z_{1:t-1})\, p(m \mid z_{1:t-1})}{p(z_t \mid z_{1:t-1})}. \tag{14.1}$$

假设（观测只依赖当前地图状态）：$p(z_t \mid m, z_{1:t-1}) = p(z_t \mid m)$。也就是说，这一帧的观测只和“这个格子现在是什么”有关，与历史无关。于是

$$p(m \mid z_{1:t}) = \frac{p(z_t \mid m)\, p(m \mid z_{1:t-1})}{p(z_t \mid z_{1:t-1})}. \tag{14.2}$$

对“未占据”事件 $\neg m$（即 $m = 0$）写同样的式子：

$$p(\neg m \mid z_{1:t}) = \frac{p(z_t \mid \neg m)\, p(\neg m \mid z_{1:t-1})}{p(z_t \mid z_{1:t-1})}. \tag{14.3}$$

注意式 (14.2) 与式 (14.3) 的分母相同。两式相除，分母消掉：

$$\frac{p(m \mid z_{1:t})}{p(\neg m \mid z_{1:t})} = \frac{p(z_t \mid m)}{p(z_t \mid \neg m)} \cdot \frac{p(m \mid z_{1:t-1})}{p(\neg m \mid z_{1:t-1})}. \tag{14.4}$$

左边是当前时刻的“占据比”，右边第二项是上一时刻的“占据比”，第一项是这一帧观测带来的“修正因子”。

定义占据比（odds）与对数占据比（log-odds）：

$$\operatorname{odds}(p) = \frac{p}{1-p}, \qquad l = \log \frac{p}{1-p}. \tag{14.5}$$

odds 把概率 $(0,1)$ 映射到 $(0,\infty)$，log 再把它映射到 $(-\infty,\infty)$——这正是我们想要的：概率接近 0 或 1 时不会“撞墙”，而且乘性更新变成加性更新。对式 (14.4) 两边取对数，得到本章的核心递推式：

$$l_t = l_{t-1} + \log \frac{p(z_t \mid m)}{p(z_t \mid \neg m)}. \tag{14.6}$$

再处理一下修正因子。对式 (14.1)（把 t 换成单帧）用一次贝叶斯，可得

$$\frac{p(m \mid z_t)}{p(\neg m \mid z_t)} = \frac{p(z_t \mid m)}{p(z_t \mid \neg m)} \cdot \frac{p(m)}{p(\neg m)}, \tag{14.7}$$

取对数即

$$\log \frac{p(z_t \mid m)}{p(z_t \mid \neg m)} = \underbrace{\log \frac{p(m \mid z_t)}{1 - p(m \mid z_t)}}_{l(m \mid z_t)} - \underbrace{\log \frac{p(m)}{1 - p(m)}}_{l(m)}. \tag{14.8}$$

把它代入式 (14.6)，就得到教材里最常见的形式：

$$\boxed{l_t(m) = l_{t-1}(m) + l(m \mid z_t) - l(m)} \tag{14.9}$$

其中 $l(m \mid z_t)$ 是“只用这一帧的观测算出的占据 log-odds”（称为**逆观测模型**的值），$l(m)$ 是先验 log-odds。

式 (14.9) 的物理含义：**每来一帧观测，就把“这一帧认为它有多大概率被占据”与“本来认为它有多大概率”之差，加到历史累积值上**。如果先验取 $p(m) = 0.5$（即 $l(m) = 0$，完全不知道），更新就简化为 $l_t = l_{t-1} + l(m \mid z_t)$，即“观测直接累加”。

14.2.2 逆观测模型与数值例子

激光雷达的逆观测模型非常直观：一条射线如果打到了某个格子，该格子就是“命中”（占据）；射线穿过但没打到的格子就是“空闲”；没有被任何射线穿过的格子“未知”，不更新。用两个常数近似：

$$p(m \mid z_t) = \begin{cases} p_{\text{occ}}, & \text{射线终点命中该格,} \\ p_{\text{free}}, & \text{射线穿过该格,} \\ 0.5\ (\text{不更新}), & \text{未被观测.} \end{cases} \tag{14.10}$$

常见取值 $p_{\text{occ}} = 0.7$、$p_{\text{free}} = 0.3$。

例题 14.1 三帧观测把概率推到多少 取先验 $p(m) = 0.5$，则 $l(m) = 0$，命中增量 $l_{\text{occ}} = \log(0.7/0.3) = 0.847$，空

闲增量 $l_{\text{free}} = \log(0.3/0.7) = -0.847$。假设同一个格子连续被命中 3 次：

$$l_3 = 0 + 3 \times 0.847 = 2.541 \implies p(m \mid z_{1:3}) = \frac{1}{1 + e^{-2.541}} = 0.927. \tag{14.11}$$

若连续命中 10 次，$l_{10} = 8.47$，$p = 0.99979$。可以看到概率会越来越“自信”，最终逼近 1 但永远不到 1——这在数值上是好事（避免除零），但也带来一个工程问题：如果某天这个位置被清空了（搬箱子），要把它“拉回”空闲需要很多帧观测，因为它的 log-odds 已经很大了。所以工程上会把 l 截断在 $[-l_{\max}, l_{\max}]$ 之间（常取 $l_{\max} = 2 \sim 4$），让地图保持“可修改”。

算法：占据栅格的 log-odds 更新（单帧）

1. 输入：栅格 l、当前帧位姿、雷达点云
2. 对每条射线：从传感器原点出发，沿射线方向步进
3. 沿途经过的格子：$l \leftarrow l + l_{\text{free}}$
4. 射线命中的格子：$l \leftarrow l + l_{\text{occ}}$
5. 对更新过的每个格子：$l \leftarrow \max(-l_{\max}, \min(l_{\max}, l))$ 截断
6. 需要查询概率时：$p = 1/(1 + e^{-l})$

把式 (14.9) 落到代码上只有几行——注意工程上还要做截断与概率还原：

```
// occupancy grid: log-odds update along a ray
float l_hit  = std::log(p_occ  / (1.0f - p_occ));   // hit increment
float l_miss = std::log(p_free / (1.0f - p_free));  // miss increment
for (auto& cell : cells_along_ray) {
    cell.log_odds += cell.is_endpoint ? l_hit : l_miss;
    // clamp so the map stays updatable
    cell.log_odds = std::clamp(cell.log_odds, -l_max, l_max);
}
// convert back to probability for display / planning
float p = 1.0f / (1.0f + std::exp(-cell.log_odds));
```

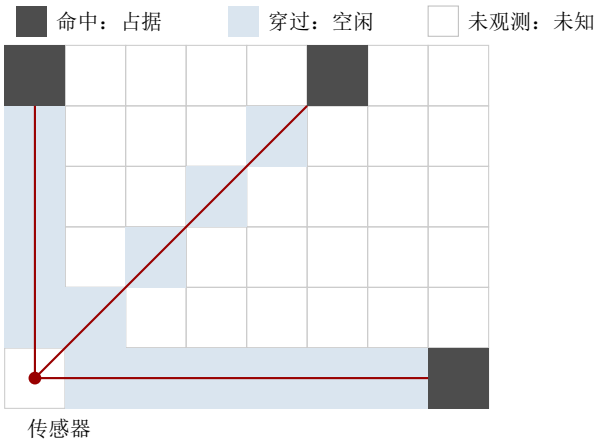


图 14.1: 一帧二维激光扫描后的占据栅格。三条射线从传感器出发，穿过的格子被更新为“空闲”，命中的格子被更新为“占据”，其余格子保持“未知”。log-odds 让多次观测可以像记账一样累加。

14.3 八叉树地图

占据栅格的内存问题在 3D 下被放大到不可接受：100 m × 100 m × 10 m 的场地、10 cm 分辨率，有 $1000 \times 1000 \times 100 = 10^8$ 个体素；每个体素一个概率就是几百 MB，而且绝大多数体素是空的、根本不需要存。

八叉树（octree）的基本思想是“**只存有差异的地方，相同的兄弟节点合并成一个父节点**”。把整个空间看成一个立方体（根节点），递归地切成 8 个小立方体（8 个孩子），直到达到目标分辨率。每个节点存一个占据概率：若 8 个孩子都一样（都空、都占据或都未知），就**剪枝**——只保留父节点，删掉孩子。

定义 14.2 (八叉树)

八叉树（octree）是一种每个内部节点恰好有 8 个子节点的树结构，用于对三维空间做递归八分。叶节点对应分辨率最高的体素，内部节点代表其覆盖区域的整体状态。若一个节点的所有子节点状态相同，则用该节点替代其子树（称为剪枝），从而以远小于稠密体素的内存表示大范围空间。

八叉树的深度与分辨率的关系是：若根节点边长 L、叶节点边长 r，则深度

$$d = \left\lceil \log_2 \frac{L}{r} \right\rceil. \tag{14.12}$$

定位一个空间点的过程就是“每层比较 x, y, z 与中点的大小，走 8 个分支之一”，$O(d)$ 次比较即可。更新一个体素就是沿这条路径走到叶节点、更新其概率，再沿路返回做剪枝判断。OctoMap[20] 就是用 log-odds 作为节点概率、并带剪枝的经典实现。

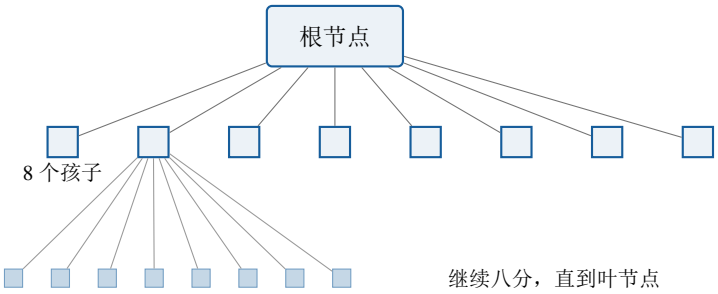


图 14.2: 八叉树的节点结构示意（示意图，实际每个节点对应一个空间立方体）。根节点代表整个空间，每层切成 8 份；当 8 个孩子状态相同时合并回父节点，这就是八叉树省内存的关键。

内存对比给出直观量级。同样是 100 m × 100 m × 10 m、10 cm 分辨率：

- 稠密体素：10^8 个体素，若每个 1 字节也要 100 MB，若存 float 则 400 MB，实际还会因为频繁查询而更慢；
- 八叉树：真正需要表示的只有“表面附近”的体素。室内场景的表面积约 2000 m^2，每个 0.1 m 体素覆盖 0.01 m^2，约 2×10^5 个叶节点；加上内部节点，总节点数在 $10^5 \sim 10^6$ 量级，内存只需几 MB 到几十 MB[20]。

两个数量级的差距，就是八叉树成为 3D 占据地图标准做法的原因。

14.4 TSDF：截断符号距离场

占据栅格只回答“这个体素有没有东西”，不回答“表面在哪、有多远”。要做高质量表面重建（比如 Marching Cubes 出网格、给规划做精细避障），需要一种连续的、带距离信息的地图——截断符号距离场（Truncated Signed Distance Field，TSDF）。

定义 14.3 (TSDF)

TSDF 是把空间中每个体素 x 映射到一个带符号的距离值 $D(x)$ 的场：$D(x) > 0$ 表示 x 在表面前方（自由空间），$D(x) < 0$ 表示在表面后方（被物体占据），$|D(x)|$ 是到最近表面的距离。为了控制内存与计算量，只保留 $|D(x)| \le \mu$ 的部分（μ 称为截断距离），超出部分截断为 $\pm\mu$。

为什么必须截断？因为远离表面的距离对建图没有用（我们只关心表面附近那一层薄壳），而“到处都要算到最近表面的精确距离”的计算量极大。截断 μ 通常取几倍体素分辨率（如 $\mu = 4r$）。

14.4.1 单帧观测给出的 SDF 值

设传感器在 $\boldsymbol{o}$，某条射线上的深度观测为 z（沿射线到表面的距离）。对射线上的一个体素 x，它到传感器的距离是 $\|\boldsymbol{x} - \boldsymbol{o}\|$。若这个体素位于表面之前，则它到表面的距离（沿射线方向）约为

$$d(\boldsymbol{x}) = z - \|\boldsymbol{x} - \boldsymbol{o}\|, \tag{14.13}$$

即“表面比体素远多少”。$d > 0$ 表示体素在表面前方，$d < 0$ 表示在后方，正是带符号距离。截断后

$$\hat{d}(\boldsymbol{x}) = \operatorname{clamp}\big(d(\boldsymbol{x}),\, -\mu,\, +\mu\big) = \begin{cases} +\mu, & d > \mu, \\ d, & -\mu \le d \le \mu, \\ -\mu, & d < -\mu. \end{cases} \tag{14.14}$$

14.4.2 多帧融合：加权平均就是最小二乘

一帧观测不够准（噪声、视角）。多帧观测同一个体素时，最自然的做法是加权平均。设第 t 帧给出的截断距离是 $\hat{d}_t(\boldsymbol{x})$，权重 $w_t(\boldsymbol{x}) \ge 0$，则

$$D_t(\boldsymbol{x}) = \frac{W_{t-1}(\boldsymbol{x})\, D_{t-1}(\boldsymbol{x}) + w_t(\boldsymbol{x})\, \hat{d}_t(\boldsymbol{x})}{W_{t-1}(\boldsymbol{x}) + w_t(\boldsymbol{x})}, \qquad W_t(\boldsymbol{x}) = W_{t-1}(\boldsymbol{x}) + w_t(\boldsymbol{x}). \tag{14.15}$$

这个式子是“加权平均”的递归写法。它为什么是最优的？因为它是下面最小二乘问题的解：

$$D_t(\boldsymbol{x}) = \arg\min_{D} \sum_{k=1}^{t} w_k(\boldsymbol{x}) \big(D - \hat{d}_k(\boldsymbol{x})\big)^2. \tag{14.16}$$

对 D 求导并令其为零：

$$\frac{\partial}{\partial D} \sum_{k=1}^{t} w_k (D - \hat{d}_k)^2 = 2 \sum_{k=1}^{t} w_k (D - \hat{d}_k) = 0 \implies D \sum_{k=1}^{t} w_k = \sum_{k=1}^{t} w_k \hat{d}_k. \tag{14.17}$$

所以

$$D_t = \frac{\sum_{k=1}^{t} w_k \hat{d}_k}{\sum_{k=1}^{t} w_k}. \tag{14.18}$$

把式 (14.18) 的分子分母同时拆成“前 $t-1$ 项 + 第 t 项”，并记 $W_{t-1} = \sum_{k=1}^{t-1} w_k$、$W_{t-1} D_{t-1} = \sum_{k=1}^{t-1} w_k \hat{d}_k$，立刻得到式 (14.15)。也就是说，**TSDF 的递归更新不是“工程上的近似”，而是最小二乘解的等价递推**——每来一帧只需 $O(1)$ 的更新，不需要保存全部历史。

14.4.3 权重设计与表面提取

权重 w_t 怎么选？物理上有两条直觉：**距离越远越不准**（深度误差随距离平方增长），**入射角越斜越不准**（斜射时同样的深度误差对应更大的表面位置误差）。所以常用

$$w_t(\boldsymbol{x}) = \frac{\cos\theta}{\|\boldsymbol{x} - \boldsymbol{o}\|^2}, \tag{14.19}$$

其中 θ 是射线方向与表面法向的夹角。工程上还会给累计权重设上限 $W \le W_{\max}$，使得“很久以前的观测”权重不再增长，地图才能慢慢忘掉旧东西、适应环境变化。

有了 $D(x)$ 的体素网格，用 Marching Cubes 在每个小立方体的 8 个角上找 D 的零交叉点，插值出表面三角形，就得到网格模型。这就是“重建”。

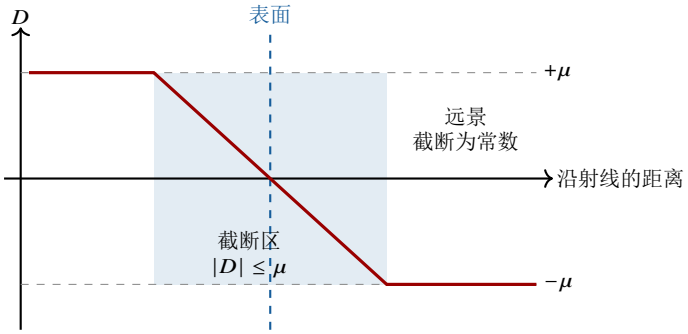


图 14.3: TSDF 沿一条射线的剖面。表面处 $D=0$，表面前方 $D>0$、后方 $D<0$；只有 $|D|\le\mu$ 的薄壳被保留（有色区域），远处一律截断为 $\pm\mu$。多帧观测在薄壳内做加权平均。

14.5 ESDF：欧氏符号距离场

TSDF 的距离是“沿射线的近似距离”，够用来重建表面，但不够用来做规划——轨迹优化需要的是“空间中每一点到最近障碍的**欧氏**距离”。这就是 ESDF（Euclidean Signed Distance Field）。

定义 14.4 (ESDF)

记障碍物点集为 $\mathcal{O}$（通常取 TSDF 的零交叉面或占据体素），ESDF 在每个点 $\boldsymbol{x}$ 处的值定义为

$$d(\boldsymbol{x})=\min_{\boldsymbol{y}\in\mathcal{O}}\|\boldsymbol{x}-\boldsymbol{y}\|_2, \tag{14.20}$$

即 $\boldsymbol{x}$ 到最近障碍点的欧氏距离；符号约定为障碍外部为正、内部为负（只取外部时称为距离场）。

ESDF 最关键的性质是它的**梯度**。设 $\boldsymbol{x}$ 的最近障碍点是 $\boldsymbol{y}^\star$，则 $d(\boldsymbol{x})=\|\boldsymbol{x}-\boldsymbol{y}^\star\|_2$。直接对 $\boldsymbol{x}$ 求导（$\boldsymbol{y}^\star$ 是使距离最小的点，由包络定理可以当作常数处理）：

$$\nabla d(\boldsymbol{x})=\frac{\boldsymbol{x}-\boldsymbol{y}^\star}{\|\boldsymbol{x}-\boldsymbol{y}^\star\|_2}, \tag{14.21}$$

这是一个**单位向量**，方向是“从最近障碍指向 $\boldsymbol{x}$”。它有两个直接推论：第一，$\|\nabla d\|=1$（处处成立，除了障碍边界上的奇点）；第二，沿着 $-\nabla d$ 走就是“最快远离障碍”的方向。

这正是规划器想要的：在轨迹优化（第 20 章）里，把“离障碍太近”写成惩罚项 $c(\boldsymbol{x})=\max(0,\,d_{\text{safe}}-d(\boldsymbol{x}))^2$，它的梯度可以直接由 ESDF 给出，从而把轨迹“推离”障碍。没有 ESDF，就需要对每个轨迹点做最近邻查询，代价高且不连续。

怎么高效算 ESDF？最直接的做法是对每个体素暴力搜索最近障碍点，复杂度 $O(N\cdot|\mathcal{O}|)$，不可接受。工程上有两类做法：一是**波前传播**（从障碍体素出发做 BFS，把距离值一层层推出去，即距离变换）；二是 Voxblox[30] 的增量做法——先建 TSDF，再只在“新观测改变了的区域”重新计算 ESDF，未变化区域沿用旧值。后者让 ESDF 可以在机器人运动时实时维护，是目前 3D 导航系统的常用方案。

14.6 点云地图与地图管理

点云地图是最“原始”的地图：把每帧点云按位姿变换到世界系，拼起来即可。它信息最全（保留了全部几何细节），但也最重（一帧 10 万点、1000 帧就是 1 亿点，几十 GB）。所以实际系统必须做管理：

- **体素降采样**：用第 12 章的体素栅格把点云稀疏化，通常按 0.1 ~ 0.2 m 的体素保留质心，点云规模能降一到两个数量级。
- **动态点清理**：环境里的行人、车辆会留下“鬼影”。常用做法是记录每个点的“被再次观测次数”，只保留被多帧一致观测到的点；或者用可见性检查（从其他位姿看过去应该看得见，却看不见的点判为动态）。
- **分块存储**：把地图切成块（如 10 m × 10 m），每块独立保存与加载。这样定位时只加载机器人附近的块，内存可控。

- **地图格式**：PCD 保存点云（含强度、法向），PLY 保存网格。工程上还要保存位姿图、关键帧索引、地图坐标系原点等元信息——这就是“地图包”（map bundle）的概念，第 23 章会讲 Z-Localization 如何用先验地图包做定位。

14.7 地图精度评价

建完图，怎么判断“好不好”？单看“好看不好看”不行，要量化。常用的四个指标如下。

（1）覆盖率（coverage）。重建出来的表面占真实表面的比例。做实验时用激光测距仪或全站仪测量若干条已知边长，与地图上量出的长度比较。

（2）点到面误差（point-to-plane RMSE）。把重建点云与真值点云（或 CAD 模型）配准后，计算每个重建点到真值表面的距离，取均方根：

$$\mathrm{RMSE} = \sqrt{\frac{1}{N}\sum_{i=1}^{N}\left(\boldsymbol{n}_i^{\top}(\boldsymbol{p}_i - \boldsymbol{q}_i)\right)^2}, \tag{14.22}$$

其中 $\boldsymbol{p}_i$ 是重建点，$\boldsymbol{q}_i$ 是真值表面上离它最近的点，$\boldsymbol{n}_i$ 是该处法向。它衡量“表面准不准”，通常要求与传感器噪声同量级（厘米级）。

（3）地图一致性。机器人绕一圈回到起点时，同一面墙会被观测两次。若两次重建的墙位置重合，地图是一致的；若错开，就会出现“双墙”（double wall）。一个实用的量化方法是测量同一面墙在重复观测处的厚度：理想情况下应等于单帧点云噪声，若明显更厚，说明回环后仍有残差。

（4）地图可用性。最终标准还是“下游任务能不能用”：全局规划能否找到路、局部避障会不会撞、定位重定位能否成功。工程上要拿真实场景跑一遍导航全流程，而不是只看点云好看不好看。

本章小结

1. 地图的形式由用途决定（表 14.1）：定位要似然（点云、栅格），规划要代价（栅格、ESDF），重建要距离（TSDF）。
2. 占据栅格存“被占据的概率”；由贝叶斯公式与观测独立性假设，可推出 log-odds 的加性更新 $l_t = l_{t-1} + l(m \mid \boldsymbol{z}_t) - l(m)$（式 (14.9)），把连乘变成连加，并可用截断保持地图可修改。
3. 命中/空闲增量由逆观测模型给出（常用 $p_{\mathrm{occ}} = 0.7$、$p_{\mathrm{free}} = 0.3$，增量 ± 0.847）；3 次命中后概率约 0.93，10 次后约 0.9998。
4. 八叉树用“递归八分 + 同质剪枝”把 3D 占据地图的内存从百 MB 量级降到几 MB 量级，深度 $d = \lceil \log_2(L/r) \rceil$。
5. TSDF 存带符号、截断的距离；多帧融合的递归式 (14.15) 正是加权最小二乘解 (14.18) 的等价递推，权重常取 $\cos\theta / \|\boldsymbol{x} - \boldsymbol{o}\|^2$。
6. ESDF 是到最近障碍的欧氏距离，梯度 ∇d 是单位向量、指向远离障碍的方向（式 (14.21)），因此可直接为规划提供“推离障碍”的方向；Voxblox 用增量更新维护它。
7. 点云地图要配合降采样、动态点清理、分块存储与地图包管理；地图质量要用覆盖率、点到面 RMSE、一致性与下游可用性来评价。

第十四章 练习

1. 若把 p_{occ} 从 0.7 改成 0.9，命中一次的 log-odds 增量是多少？同样的“3 次命中”后概率变成多少？这说明逆观测模型的取值如何影响地图“反应速度”。
2. 式 (14.9) 中假设了观测独立性。现实中同一面墙在相近位置被连续观测，这个假设成立吗？如果不成立，地图会出现什么偏差？
3. 为什么八叉树的剪枝要以“8 个子节点状态相同”为条件？如果只比较其中 7 个会怎样？

4. 在式 (14.15) 中，若某天开始权重不再增加（W 达到上限），新观测的影响会怎样体现？这与“地图要能变旧”的准则有什么关系？
5. 证明 $\nabla d(\boldsymbol{x})$ 是单位向量。提示：对 $d^2(\boldsymbol{x}) = \|\boldsymbol{x} - \boldsymbol{y}^\star\|^2$ 求导，并注意 $\boldsymbol{y}^\star$ 是最小值点。
6. 你的机器人在一个 200 m^2 的场地里建 2D 栅格图，分辨率 5 cm。估算纯占据概率存储需要多少内存；若改用 log-odds 并限制最大范围为 120 m $\times$ 120 m，又是多少？

延伸阅读

1. Thrun, Burgard, Fox，《Probabilistic Robotics》第 9 章 [38]：占据栅格与 log-odds 更新的标准参考，本章的推导与它一致。
2. Hornung 等，OctoMap[20]：八叉树的工程实现，重点看节点概率模型与剪枝策略。
3. Curless & Levoy[6]：TSDF 的原始论文，加权融合与表面提取的经典做法。
4. Oleynikova 等，Voxblox[30]：从 TSDF 增量构建 ESDF，是第 20 章轨迹优化常用的地图。

第十五章 2D 激光 SLAM：Cartographer

本章导读

Cartographer 是 Google 在 2016 年开源的激光 SLAM 系统 [19]，也是 RoboMaster 赛场上最常用的 2D 建图方案之一。它把前面几章的知识组装成了一个完整可运行的工程：用 “子图” 组织地图，用 “帧到子图匹配” 做前端，用 “分支定界” 在离散位姿空间里快速找到最优位姿，用 “位姿图优化” 做全局闭环。本章会按这个顺序逐层拆开：先看系统架构与子图概念，再完整推导扫描匹配的得分函数、双线性插值及其雅可比，然后重点推导分支定界的 “上界” 为什么成立、为什么能剪枝，最后讲全局 SLAM、IMU/里程计的作用与关键参数。读完本章，你应该能对着 Cartographer 的配置文件说清每个参数在调什么。

15.1 Cartographer 的定位与系统架构

在 Cartographer 之前，2D 激光 SLAM 的主流有两种：Gmapping[15] 用粒子滤波（第 10 章的粒子滤波在这里被用来 “同时定位与建图”），Hector 用无回环的扫描匹配。三者的对比如表 15.1。

表 15.1: 三种 2D 激光 SLAM 方案的对比

方案	核心方法	回环	特点与局限
Gmapping	粒子滤波 + 扫描匹配	有（粒子重采样）	小场景效果好；粒子数随地图增大而增多，大场景计算与内存吃紧
Hector	高斯-牛顿扫描匹配	无	不需要里程计；但无回环，长距离必漂移
Cartographer	子图 + 分支定界 + 位姿图	有（子图间约束）	大场景实时、回环可靠；配置参数多，需调参

Cartographer 的整体数据流如图 15.1，可以按第 11 章的语境一分成两半：

- **Local SLAM（前端）**：把每一帧 scan 与当前子图做扫描匹配，求出相对位姿，然后把 scan 插入子图。它连续运行、频率高。
- **Global SLAM（后端）**：每当一个子图 “完成”，就把它与其他历史子图做约束搜索（回环检测），并把所有约束丢进位姿图优化。它低频运行，负责全局一致。

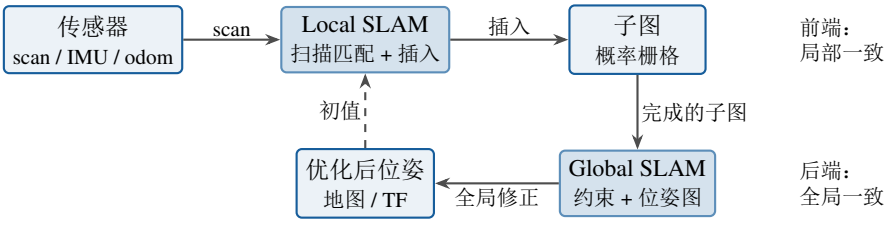


图 15.1: Cartographer 的系统架构。传感器数据进入 Local SLAM 做帧到子图匹配并插入子图；子图完成后进入 Global SLAM 做约束构建与位姿图优化，优化结果反过来给前端提供位姿初值，并发布地图与 TF。

15.2 子图：地图的基本单位

定义 15.1 (子图)

子图（submap）是 Cartographer 中地图的基本单位：它是一张以概率栅格形式存储的局部地图，由连续插入的若干帧 scan 融合而成。每个子图有自己的局部坐标系，子图内的栅格存的是 “被占据的 log-odds 值”（与第 14 章完全一致）。当插入的 scan 达到设定数量（参数 `num_range_data`，默认 90）后，该子图 “完成”，不再更新。

♣

用子图而不是一张全局大地图，有三个好处：第一，插入 scan 时只需查局部栅格，内存可控；第二，每个子图有独立的“原点”，避免全局地图越来越大导致的数值精度问题；第三，全局优化时只需要调整“子图位姿”（每个子图一个位姿），变量数大大减少——一张由 20 个子图拼成的地图，位姿图只有 20 多个节点，而不是几千帧。

子图的概率更新沿用第 14 章的 log-odds 公式：射线穿过的格子加“空闲”增量，命中格子加“占据”增量。查询时用

$$p(m) = \frac{1}{1 + e^{-l(m)}}, \qquad \mathrm{odds}(m) = \frac{p(m)}{1 - p(m)} = e^{l(m)}. \tag{15.1}$$

15.3 扫描匹配：把 scan 对齐到子图

15.3.1 得分函数

设当前 scan 有 K 个点，在 scan 系下的坐标为 $\boldsymbol{h}_k = (h_{k,x}, h_{k,y})$。要估计的是 scan 系到子图系的位姿 $\xi = (t_x, t_y, \theta)$，写成二维刚体变换

$$T_\xi \boldsymbol{h}_k = \boldsymbol{R}(\theta)\boldsymbol{h}_k + \boldsymbol{t} = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix} \boldsymbol{h}_k + \begin{bmatrix} t_x \\ t_y \end{bmatrix}. \tag{15.2}$$

把变换后的点拿到子图上查概率，得分定义为所有点概率之和：

$$S(\xi) = \sum_{k=1}^{K} M_{\mathrm{smooth}}\big(T_\xi \boldsymbol{h}_k\big), \tag{15.3}$$

其中 M_{smooth} 是子图栅格的双线性插值版本。S 越大，说明“按这个位姿把 scan 放上去，点落在已占据格子里的越多”，位姿越可能正确。Cartographer 要找的就是

$$\xi^\star = \arg\max_{\xi} S(\xi). \tag{15.4}$$

15.3.2 双线性插值及其导数

栅格是离散的，点 $\boldsymbol{p} = (x, y)$ 一般落在四个格子中间。最近邻插值会让得分函数变成阶梯状、处处不可导，梯度法就没法用；双线性插值给出连续可导的得分面，所以是标配。

设栅格分辨率为 r，把 $\boldsymbol{p}$ 换算成栅格坐标：

$$\frac{x}{r} = x_0 + u, \qquad \frac{y}{r} = y_0 + v, \qquad x_0 = \left\lfloor \frac{x}{r} \right\rfloor,\ y_0 = \left\lfloor \frac{y}{r} \right\rfloor,\ u, v \in [0, 1). \tag{15.5}$$

记四个相邻格子的概率为 $M_{00} = M(x_0, y_0)$、$M_{10} = M(x_0 + 1, y_0)$、$M_{01} = M(x_0, y_0 + 1)$、$M_{11} = M(x_0 + 1, y_0 + 1)$。先沿 x 方向插值两次，再沿 y 方向插值一次：

$$\begin{aligned} M_{\mathrm{smooth}}(\boldsymbol{p}) &= (1 - v)\Big[(1 - u)M_{00} + uM_{10}\Big] + v\Big[(1 - u)M_{01} + uM_{11}\Big] \\ &= (1 - u)(1 - v)M_{00} + u(1 - v)M_{10} + (1 - u)vM_{01} + uvM_{11}. \end{aligned} \tag{15.6}$$

式 (15.6) 就是双线性插值：四个角的权重是“到对角的距离”之积，离哪个角越近权重越大。

对 $\boldsymbol{p}$ 求偏导时要注意链式法则：$\partial u/\partial x = 1/r$、$\partial v/\partial y = 1/r$，而 x_0, y_0 在格子内部是常数。于是

$$\frac{\partial M_{\mathrm{smooth}}}{\partial x} = \frac{1}{r}\Big[(1 - v)\big(M_{10} - M_{00}\big) + v\big(M_{11} - M_{01}\big)\Big], \tag{15.7}$$

$$\frac{\partial M_{\mathrm{smooth}}}{\partial y} = \frac{1}{r}\Big[(1 - u)\big(M_{01} - M_{00}\big) + u\big(M_{11} - M_{10}\big)\Big]. \tag{15.8}$$

这两个式子是后面雅可比的“上游”：它们告诉你“点移动一点，得分变多少”。

15.3.3 非线性最小二乘与雅可比

把式 (15.4) 的求最大值改写为求最小值：定义残差

$$r_k(\xi) = 1 - M_{\text{smooth}}\big(T_\xi \boldsymbol{h}_k\big) \in \mathbb{R}, \tag{15.9}$$

则 $\min_\xi \sum_k r_k^2$ 与 $\max_\xi S$ 等价（常数 K 不影响极值点）。对残差做高斯-牛顿迭代，需要残差对位姿的雅可比。链式法则分成两段：**点对位姿、得分对点**。

第一段：变换后的点对位姿求导。记 $\boldsymbol{R}'\boldsymbol{h}_k$ 为旋转矩阵对 θ 求导再作用在点上的结果：

$$\frac{\partial (T_\xi \boldsymbol{h}_k)}{\partial t_x} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \qquad \frac{\partial (T_\xi \boldsymbol{h}_k)}{\partial t_y} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \qquad \frac{\partial (T_\xi \boldsymbol{h}_k)}{\partial \theta} = \begin{bmatrix} -(h_x \sin\theta + h_y \cos\theta) \\ h_x \cos\theta - h_y \sin\theta \end{bmatrix}. \tag{15.10}$$

注意最后一项可以写成 $\boldsymbol{J}_2 \boldsymbol{R}(\theta)\boldsymbol{h}_k$，其中

$$\boldsymbol{J}_2 = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix} \tag{15.11}$$

是“逆时针转 90°”的矩阵。合并成 2×3 的矩阵：

$$\frac{\partial (T_\xi \boldsymbol{h}_k)}{\partial \xi} = \begin{bmatrix} \boldsymbol{I}_2 & \boldsymbol{J}_2 \boldsymbol{R}(\theta)\boldsymbol{h}_k \end{bmatrix} \in \mathbb{R}^{2\times 3}. \tag{15.12}$$

第二段：把得分梯度式 (15.7)、式 (15.8) 与式 (15.12) 相乘，得到残差对位姿的 1×3 雅可比：

$$\boldsymbol{J}_k = \frac{\partial r_k}{\partial \xi} = -\begin{bmatrix} \dfrac{\partial M}{\partial x}, & \dfrac{\partial M}{\partial y} \end{bmatrix} \begin{bmatrix} \boldsymbol{I}_2 & \boldsymbol{J}_2 \boldsymbol{R}\boldsymbol{h}_k \end{bmatrix} \in \mathbb{R}^{1\times 3}. \tag{15.13}$$

它的每一列分别对应“平移 x、平移 y、旋转 θ 一点点，得分变多少”。有了 $\boldsymbol{J}_k$，高斯-牛顿方程就是（残差是一维，信息矩阵取 1）：

$$\Big(\sum_k \boldsymbol{J}_k^\top \boldsymbol{J}_k\Big)\delta\boldsymbol{\xi} = -\sum_k \boldsymbol{J}_k^\top r_k. \tag{15.14}$$

解出 $\delta\boldsymbol{\xi}$ 后按 $\xi \leftarrow \xi \oplus \delta\boldsymbol{\xi}$ 更新（二维时 $\oplus$ 就是直接相加），重复几轮即可收敛。

扫描匹配的三个“为什么”

为什么用双线性插值？为了让得分函数连续可导；最近邻会让 GN 拿不到梯度。

为什么只迭代几轮？匹配只是在给定位姿附近做局部精化，初值由里程计/IMU/上一次位姿给出；跑太多轮既慢又可能被局部极值带偏。

为什么还要分支定界？GN 只能做局部精化，初值差得多时（比如刚开机或回环搜索）会掉进错误的局部极小；此时需要全局搜索，而全局搜索靠分支定界加速。

15.4 分支定界：在离散位姿空间里找全局最优

15.4.1 问题与离散化

回环检测或重定位时，我们不知道大致位姿，只有“可能在这一带”的先验。此时要解的是全局的式 (15.4)。把位姿空间离散化：平移按子图分辨率 r 取整数格，旋转按固定角步长 δ_θ 取整。于是在一个窗口内，候选位姿数量是

$$N = N_x \cdot N_y \cdot N_\theta, \tag{15.15}$$

典型的回环搜索窗口 $N_x = N_y = 100$、$N_\theta = 180$（约 2° 一档），总数约 1.8×10^6。每个候选都要对 $K \sim 10^3$ 个点查一次子图，直接算就是 10^9 次查表，实时不可能。

分支定界（Branch and Bound）是解决“在离散空间里找最大值”的经典方法。它把搜索空间组织成一棵树：根节点代表整个搜索窗口，每个节点代表一个“位姿子集”，节点一分为四（平移二分 × 旋转二分）。对每个节点，我们不计算精确得分，而是计算一个**上界**——该子集里任何位姿的得分都不会超过这个上界。于是：若某节点的上界低于“目前已经找到的最好得分”，整棵子树直接剪掉，一个叶子都不用算；否则继续往下分。

定义 15.2 (分支定界)

分支定界（Branch and Bound）是一类求解离散最优化问题的方法：把候选解集合递归地划分成子集（分支），并给每个子集计算一个“其上所有解的目标值都不会超过”的上界（定界）。搜索时优先处理上界高的子集，一旦某子集的上界低于当前最优解，就剪掉该子集。若上界是紧的（接近真实最大值），可以在远少于穷举的节点数内找到最优解。

♣

15.4.2 上界的推导

关键问题：怎么给一个位姿子集算上界，而且算得便宜？答案是用**预计算的多分辨率栅格**。

设一个节点代表的位姿子集是：平移在以 ξ_c 为中心的半宽 c_h 个格子内，旋转在半宽 δ_h 内。对子集里的任意位姿 ξ 与任意 scan 点 $\boldsymbol{h}_k$，有

$$\begin{aligned}\left\|T_{\xi}\boldsymbol{h}_k - T_{\xi_c}\boldsymbol{h}_k\right\| &= \left\|\boldsymbol{R}(\theta)\boldsymbol{h}_k + \boldsymbol{t} - \boldsymbol{R}(\theta_c)\boldsymbol{h}_k - \boldsymbol{t}_c\right\| \\ &\le \underbrace{\left\|\boldsymbol{R}(\theta) - \boldsymbol{R}(\theta_c)\right\| \cdot \|\boldsymbol{h}_k\|}_{\text{旋转误差}} + \underbrace{\|\boldsymbol{t} - \boldsymbol{t}_c\|}_{\text{平移误差}}.\end{aligned} \tag{15.16}$$

二维旋转有个好性质：两旋转矩阵之差的作用可以界住角度差，

$$\left\|\boldsymbol{R}(\theta) - \boldsymbol{R}(\theta_c)\right\|_2 = 2\left|\sin\frac{\theta - \theta_c}{2}\right| \le |\theta - \theta_c|. \tag{15.17}$$

（用 $\sin\alpha \le \alpha$ 即得。）由节点定义，$|\theta - \theta_c| \le \delta_h$，$\|\boldsymbol{t} - \boldsymbol{t}_c\| \le \sqrt{2}\,c_h r$（二维平移偏差最大值是对角线），代回式 (15.16)：

$$\left\|T_{\xi}\boldsymbol{h}_k - T_{\xi_c}\boldsymbol{h}_k\right\| \le \delta_h\|\boldsymbol{h}_k\| + \sqrt{2}\,c_h r \;\triangleq\; \rho_h(\boldsymbol{h}_k). \tag{15.18}$$

也就是说：**子集内任意位姿下，第 k 个点落到的位置，一定在以 $T_{\xi_c}\boldsymbol{h}_k$ 为圆心、$\rho_h(\boldsymbol{h}_k)$ 为半径的圆内**。

于是对每个深度 h，预先算一张“膨胀栅格”：把子图里每个格子的值替换成它周围半径 ρ_h 范围内的最大值，

$$M^h_{\text{precomp}}(\boldsymbol{p}) = \max_{\|\boldsymbol{q}-\boldsymbol{p}\|\le\rho_h} M_{\text{smooth}}(\boldsymbol{q}). \tag{15.19}$$

由式 (15.18)，子集内任意位姿下 $M_{\text{smooth}}(T_{\xi}\boldsymbol{h}_k) \le M^h_{\text{precomp}}(T_{\xi_c}\boldsymbol{h}_k)$。对所有点求和：

$$S(\xi) = \sum_k M_{\text{smooth}}(T_{\xi}\boldsymbol{h}_k) \le \sum_k M^h_{\text{precomp}}\left(T_{\xi_c}\boldsymbol{h}_k\right) \;\triangleq\; \overline{S}_h(\xi_c). \tag{15.20}$$

右边只依赖节点中心位姿 ξ_c 与预计算栅格，查表即可算出——这就是**上界**。

两个必须讲清的性质：

1. **上界是“真上界”**：式 (15.20) 是严格推导出来的不等式，不是启发式，所以剪枝不会漏掉最优解。
2. **上界随深度单调收紧**：深度越大，c_h 与 δ_h 越小，由式 (15.18) 得到的 ρ_h 越小；ρ_h 越小，式 (15.19) 里取 max 的范围越小，M^h_{precomp} 逐点越小。所以**父节点的上界一定不小于子节点的上界**，这保证搜索树自顶向下“越来越紧”，剪枝才有意义。

15.4.3 搜索策略与伪代码

搜索用**优先队列**（每次取出上界最大的节点）最直观：由于父节点的上界不小于子孙，第一个被取出并展开到叶子的节点，其真实得分一定不低于队列里其他所有节点的上界，因此**第一个到达的叶子就是全局最优解**。这就是“最优优先”的分支定界。

算法：分支定界搜索最优位姿

1. 输入：先验位姿 ξ_0、搜索窗口、预计算的多层膨胀栅格

2. 根节点 $\leftarrow$ 整个窗口，用式 (15.20) 算上界

3. 优先队列 $Q \leftarrow$ {根节点}；最优得分 best $\leftarrow -\infty$

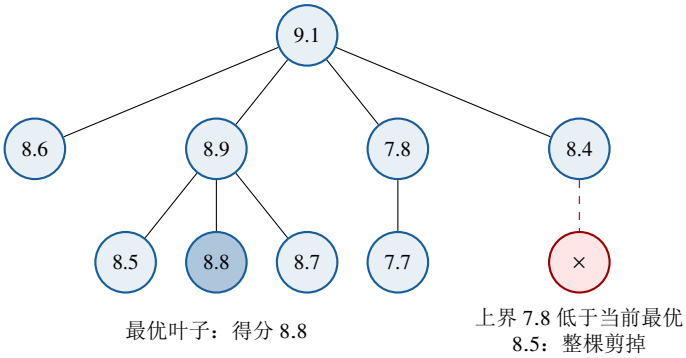


图 15.2: 分支定界的示意。圆圈内数字是该节点的上界。优先队列先展开上界 9.1 的根，再展开 8.9 的分支；当 8.8 的叶子被找到后，所有上界小于 8.8 的节点（如 7.8、8.4、8.5）整棵剪掉。上界必须“真”，否则会剪掉最优解。

4. **while** Q 非空 **do**
5. 从 Q 取出上界最大的节点 n
6. **if** n.上界 < best **then** 剪枝，**continue**
7. **if** n 是叶子 **then** best $\leftarrow S(n)$，$\xi^\star \leftarrow n$.位姿，**continue**
8. 把 n 划分成子节点；对每个子节点算上界并放入 Q
9. **end while**
10. **return** $\xi^\star$

15.4.4 为什么能实时

遍历所有候选是 $O(N)$，其中 $N = N_x N_y N_\theta$ 可达 10^6；每个候选算得分又要 $O(K)$，总代价 $O(NK)$。分支定界不会遍历所有叶子：由于上界随深度收紧得很快，大部分分支在高层就被剪掉。实际运行中，一次回环搜索通常只访问 $10^3 \sim 10^4$ 个节点，相比 10^6 个叶子少了两三个数量级——这就是 Cartographer 能“在后台实时做回环”的原因。当然，最坏情况下（得分面非常平坦、上界很松）它仍会退化为穷举，所以工程上还要限制搜索窗口大小与层数。

15.5 全局 SLAM：把子图连成一张图

Local SLAM 只保证“子图内部一致”，子图与子图之间会漂移。Global SLAM 要把它们缝起来：

1. **约束构建（constraint building）**：当有新的 scan 或新子图完成时，在“历史子图”里找可能重叠的那些，用分支定界在粗分辨率上搜索候选相对位姿。若该位姿下的匹配得分超过阈值（参数 `min_score`），就把它作为一条“子图—scan 约束”加入位姿图。
2. **回环检测**：本质上就是“在子图集合上做约束搜索”。Cartographer 不是拿两帧点云直接比，而是拿“当前 scan 对历史子图”匹配，这比 Scan Context 那样的描述子更准（直接优化得分），代价是搜索更重，所以要靠分支定界加速。
3. **位姿图优化**：节点是各子图的位姿，边是约束（含协方差），用 Ceres[19] 做稀疏最小二乘求解（第 13 章）。优化后，所有子图位姿被修正，地图重新拼合。

Cartographer 的位姿图规模很小：一张地图可能只有几十到几百个子图，每条边只连两个节点，海森矩阵极其稀疏，一次优化通常几十毫秒。这与前端每帧的扫描匹配（几毫秒）配合起来，整体可以实时运行。

15.6 IMU 与里程计的作用

2D 情形下，IMU 的价值不像 3D LIO 那么大，但仍然有用：

- **提供旋转初值**：陀螺仪积分给出 yaw 的短时变化，可以作为扫描匹配的旋转初值，把三维搜索（x, y, θ）里 θ 的范围缩小。
- **提供重力方向**：如果雷达不是水平安装，重力方向可以帮助把 scan 投影到水平面；纯 2D 系统里用得少，3D 版本则必须用。
- **给位姿图加约束**：IMU 的航向角可以作为一条 “绝对朝向” 约束加进优化，抑制长走廊里的旋转漂移。

里程计（轮速）的作用更直接：它给出相邻帧之间的平移与旋转估计，作为扫描匹配的初值。没有里程计时，Cartographer 用 “上一帧位姿” 或匀速模型预测作初值；有里程计时初值更准，匹配更稳。三者都必须做**时间同步**（第 3 章）：scan、IMU、odom 的时间戳若差几十毫秒，在机器人快速运动时会直接变成位姿误差。

15.7 关键参数与调参

Cartographer 的配置文件（`.lua`）参数很多，真正影响效果的是表 15.2 这几个。

表 15.2: Cartographer 关键参数及其影响

参数	默认值	调大 / 调小的后果
`submaps.resolution`	0.05 m	调小：更精细但内存与计算增大；调大：省资源但细节丢失
`submaps.num_range_data`	90	调大：子图更大、单图更稳、回环机会少；调小：子图碎、约束多、优化慢
`constraint_builder.min_score`	0.55	调大：只接受高置信回环，漏检多；调小：回环多但误检风险高
`optimization_problem.huber_scale`	1.0	调小：对错边更鲁棒；调大：更信任所有约束
`TRAJECTORY_BUILDER_2D.voxel_filter_size`	0.025 m	调大：降采样更狠、更快；调小：保留细节但更慢
`POSE_GRAPH.optimize_every_n_nodes`	90	调小：优化频繁、更跟手但更耗 CPU；调大：省 CPU 但闭环延迟

调参的顺序建议是：先保证**数据质量**（时间同步、外参、雷达去畸变），再调**分辨率与降采样**（决定计算量），最后调**回环相关参数**（`min_score`、优化频率）。顺序反了会陷入 “调一个坏一个” 的循环。

15.8 优缺点与适用场景

优点：回环可靠（子图 + 分支定界 + 位姿图，长距离建图能闭环）；实时性好（分支定界把全局搜索压到可接受的时间）；工程成熟（ROS/ROS 2 集成好，配置项清晰）；只依赖 2D 激光也能跑（可选 IMU/odom），成本低。

局限：第一，2D 假设机器人在地面单层运动，多层场地、坡道、空中场景都不适用，这时要换 3D SLAM（第 16 章与第 17 章）；第二，长走廊、空旷场地存在几何退化（第 12 章），单靠激光会漂移，必须靠 IMU/odom 兜底；第三，参数较多，场景变化大时需要重新调参；第四，子图是 “局部” 的，超大场景（几万平方米）需要分块建图再拼接。

对 RoboMaster 场景：赛场是单层、结构相对规整、有围挡与固定道具，2D Cartographer 是非常合适的选择；若场地有坡道或需要 3D 避障，则应采用第 16 章起的 LIO 方案。

本章小结

1. Cartographer 由 Local SLAM（前端：帧到子图匹配 + 插入）与 Global SLAM（后端：约束构建 + 位姿图优化）组成（图 15.1），是第 11 章语境一的典型实现。
2. 子图是地图的基本单位，本质是一张概率栅格（log-odds），插入 `num_range_data` 帧后完成；用子图位姿做全局优化，变量数从 “帧数” 降到 “子图数”。
3. 扫描匹配的得分是 $S(\xi) = \sum_k M_{\text{smooth}}(T_\xi \boldsymbol{h}_k)$（式 (15.3)），双线性插值式 (15.6) 让得分连续可导；其梯度由式 (15.7)、式 (15.8) 给出，残差雅可比为式 (15.13)，用高斯-牛顿式 (15.14) 迭代求解。

4. 分支定界把位姿空间组织成树，用预计算膨胀栅格式 (15.19) 给出上界式 (15.20)；上界严格成立、且随深度单调收紧，因此剪枝不会漏掉最优解，实际访问节点数比穷举少两三个数量级。
5. 全局 SLAM 用约束搜索（分支定界）找子图间回环，用 Ceres 做位姿图优化；IMU/odom 提供初值与旋转约束，时间同步是前提。
6. 参数调优顺序：数据质量 → 分辨率/降采样 → 回环与优化频率；2D 单层场景适用，多层与坡道需换 LIO。

第十五章 练习

1. 式 (15.6) 中，若点恰好落在格子中心（$u = v = 0.5$），四个格子的权重各是多少？这个结果符合直觉吗？
2. 为什么扫描匹配的残差定义为 $1 - M$ 而不是 M？两者对高斯-牛顿的迭代有什么影响？
3. 若把式 (15.18) 中的平移偏差从 $\sqrt{2}c_h r$ 改成 $2c_h r$，上界会变松还是变紧？变松之后剪枝效率如何变化？
4. 分支定界的上界如果取成“节点中心位姿的真实得分”，会有什么后果？为什么说“上界必须真”？
5. Cartographer 的位姿图节点是子图而不是每帧 scan，这样做省下了什么？又牺牲了什么？
6. 在长走廊里建图，Cartographer 的哪些环节最容易出问题？你会调整表 15.2 中的哪些参数、或者引入什么传感器？

延伸阅读

1. Hess 等，Cartographer 论文 [19]：本章的原始文献，重点读第 III 节的“Local SLAM”、第 IV 节的“Branch-and-Bound”与第 V 节的“Global SLAM”。
2. Grisetti 等，Gmapping[15]：粒子滤波 2D SLAM 的代表，用来对照表 15.1。
3. 第 14 章：子图的概率栅格更新与 log-odds，本章直接沿用。
4. Cartographer 官方文档与 `cartographer_ros` 配置示例：调参时对照表 15.2 阅读。

第十六章 3D 激光惯性里程计：从 LOAM 到 LIO

本章导读

第 15 章的 Cartographer 只处理机器人在平面内运动的情形（状态只有 x, y, θ 三个自由度）。可一旦机器人加上云台、上下坡、被撞飞，或者换成无人机、机械臂，运动就是 6 自由度的。本章把问题升到三维，回答三个问题：为什么 3D 激光必须要和 IMU 一起用？一帧点云内部的运动畸变到底有多严重、怎么消掉？从 2014 年的 LOAM 到 2023 年的 Point-LIO，这十年里每个算法究竟解决了前一个算法的什么毛病？读完本章，你应该能看懂任意一个 LIO 开源包的数据流，并能判断" 我这个场景该选哪一个"。本章只讲演化脉络与共性原理，FAST-LIO2 和 Point-LIO 的逐行数学推导放在第 17 章。

16.1 为什么需要激光惯性里程计

16.1.1 2D 激光 SLAM 的三条隐含假设

Cartographer [19] 之所以能把计算量压到嵌入式平台可接受的水平，靠的是三条假设：机器人只在水平面内运动、激光只有一个扫描平面、地图用一个二维栅格就能表达。这三条在 RoboMaster 的场地上其实已经摇摇欲坠：

1. 哨兵和步兵会过坡道、飞坡、被撞得整个车身翻滚，云台的 pitch 角也可能连续变化，此时"z 坐标恒定、roll/pitch 恒为零" 直接失效；
2. 单线激光只切出场景的一个水平薄片，两块高度不同的挡板在 2D 地图上完全一样，机器人无法区分" 前面是 20 cm 高的坎" 还是" 前面是三米高的墙"；
3. 换成 3D 激光后，一帧点云动辄 $10^4 \sim 10^5$ 个点，比 2D 单线激光多两个数量级，Cartographer 那套" 子图 + 分支定界" 的暴力搜索代价立刻变得不可接受。

所以 3D 场景需要 6 自由度的位姿 $(\boldsymbol{R}, \boldsymbol{p}) \in \mathrm{SE}(3)$（李群的定义见第 2 章），计算量又要求我们不能对全部点做穷举搜索。这两条约束一起，把 3D 激光里程计推向了" 用 IMU 提供运动先验、用点云提供几何约束" 的路子。

16.1.2 纯激光里程计的三种失效模式

回忆第 7 章的扫描匹配：给定两帧点云，通过最小化点到点或点到面残差求相对位姿。它的信息矩阵是 $\boldsymbol{H}^{\top}\boldsymbol{H}$（$\boldsymbol{H}$ 是残差对位姿的雅可比堆叠），当这个矩阵在某个方向上接近奇异时，那个方向上的位姿就解不准，这叫**几何退化**（degeneracy）。具体有三种典型情形：

- **长走廊、隧道、平行墙**：沿走廊轴线方向平移，点云外观几乎不变，$\boldsymbol{H}^{\top}\boldsymbol{H}$ 沿该方向的特征值接近零，机器人会以为" 自己没动"，位置估计沿轴线无界漂移；
- **开阔场地或单一平面**：所有点几乎落在一个平面上，只能约束垂直于平面的平移和两个转角，沿平面内的两个平移和绕平面法向的转角都约束不足；
- **剧烈运动与稀疏场景**：相邻两帧重叠区域太小，或者机器人转得太快导致两帧几乎没有共视点，匹配的初值差，ICP 类算法会收敛到错误的局部极小。

第二种失效其实不是几何问题，而是时间问题：3D 激光的" 一帧" 并不是一个瞬间，而是几十到一百毫秒的连续采集，帧内机器人在动，点云自己就被" 拉花" 了。这就是下一节要讲的去畸变。

16.1.3 IMU 补上了什么

IMU 的特点是频率高（100 ~ 1000 Hz）、短时间内的角速度和比力测量准，但积分会漂移。这两个特点和激光恰好互补，见表 16.1。

表 16.1: 激光与 IMU 的互补性

特性	3D 激光	IMU
输出频率	10 Hz（一帧 100 ms）	100 ~ 1000 Hz
擅长的时间尺度	> 0.1 s 的绝对几何	< 0.1 s 的高动态运动
提供的量	场景几何（绝对约束）	角速度、比力（增量约束）
随时间的误差	缓慢漂移、可回环消除	二重积分，秒级发散
典型失效	几何退化、帧内畸变、快速旋转	饱和、冲击、温度漂移

用两个数字感受一下 IMU 靠不住的程度：一块常见 MEMS 陀螺的零偏若为 0.01 rad/s（约 0.57°/s），纯积分 10 s 就会产生 $0.01 \times 10 \times 180/\pi \approx 5.7°$ 的姿态误差；加速度计零偏若为 0.02 m/s^2，同样的 10 s 内位置误差为 $\frac{1}{2} \times 0.02 \times 10^2 = 1$ m。反过来，激光在 100 ms 内几乎不可能漂移，所以把两者放进同一个估计器，正好让 IMU 负责” 帧内”、激光负责” 帧间”。

定义 16.1 (激光惯性里程计 LIO)

以 3D 激光点云和 IMU 测量为输入，在一个估计器中联合估计载体位姿 T_{WB}（以及速度、IMU 零偏等辅助状态）并增量维护地图的算法，称为激光惯性里程计（LiDAR-Inertial Odometry，LIO）。若激光与 IMU 的原始测量被送进同一个估计器的同一次状态更新中，称为紧耦合；若先由激光独立的扫描匹配算出位姿，再把这个位姿当作观测去修正 IMU 的递推结果，则称为松耦合。两者的数学差别在 §16.5 展开。

一句话记住 LIO 的价值

纯激光里程计在几何退化处” 不知道自己错了”，纯 IMU 在几秒后” 必然错”。LIO 把 IMU 的高频运动先验和激光的绝对几何约束放进同一个概率模型，使二者互相纠错：退化方向上靠 IMU 撑住，IMU 漂移靠激光拉回。

16.2 去畸变：把一帧点云对齐到同一时刻

16.2.1 问题有多严重

以常见的机械式 16 线激光为例：转速 10 Hz，即一帧 100 ms；一帧内每线约 1800 个点，相邻点间隔约 55 μs。也就是说，一帧点云里第一个点和最后一个点的采集时刻相差 100 ms。若机器人此时角速度为 90°/s，这 100 ms 内姿态变化 9°；若点距离 $r = 5$ m，则帧首帧尾的点在空间上会被错开约 $r \sin 9° \approx 0.78$ m。如果机器人同时以 2 m/s 前进，平移误差再叠加 0.2 m。而地图体素分辨率通常只有 0.1 ~ 0.2 m，这样的畸变会让整个配准直接崩溃。

定义 16.2 (运动畸变与去畸变)

激光雷达一帧点云中的每个点都带有自己的采集时刻 t_i（相对扫描起始时刻的偏移）。由于载体在帧内运动，若把整帧点云当作同一时刻 t_{ref} 的刚体扫描来处理，就会引入系统性形变，称为运动畸变。按每个点自己的时刻把它的坐标变换到同一参考时刻 t_{ref} 的过程，称为去畸变（deskewing / motion compensation）。

16.2.2 去畸变公式

设 t_i 时刻点 i 在雷达系中的坐标为 $\boldsymbol{p}_i^L$，该时刻雷达到世界系的位姿为 $T_{WL}(t_i) = (R(t_i), \boldsymbol{p}(t_i))$。那么该点在世界上真正的位置是

$$\boldsymbol{p}_i^W = R(t_i)\, \boldsymbol{p}_i^L + \boldsymbol{p}(t_i). \tag{16.1}$$

去畸变就是把同一批世界点重新表达在参考时刻的雷达系中。把式 (16.1) 反解，得到

$$\boldsymbol{p}_i^{L_{\text{ref}}} = T_{WL}(t_{\text{ref}})^{-1}\, \boldsymbol{p}_i^W = R(t_{\text{ref}})^{\top} \Big(R(t_i)\, \boldsymbol{p}_i^L + \boldsymbol{p}(t_i) - \boldsymbol{p}(t_{\text{ref}}) \Big). \tag{16.2}$$

式 (16.2) 的几何含义很直白：先把点搬回世界系，再用参考时刻的位姿搬进参考帧。若帧内没有运动（$R(t_i) = R(t_{\text{ref}})$，$\boldsymbol{p}(t_i) = \boldsymbol{p}(t_{\text{ref}})$），它就退化成恒等变换，说明去畸变只在”帧内动”的情况下才需要。

16.2.3 匀速模型与 IMU 辅助模型

式 (16.2) 需要帧内任意时刻的位姿 $T_{WL}(t)$，而我们只在一个帧边界上知道它。最简单的近似是**匀速模型**：假设帧内线速度和角速度恒定，用前后两帧的位姿插值：

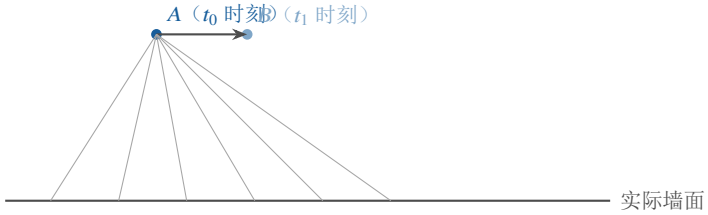
$$R(t) = R(t_0)\exp\Big(s\,\big[\mathrm{Log}(R(t_0)^{\top}R(t_1))\big]_{\times}\Big), \qquad \boldsymbol{p}(t) = (1-s)\,\boldsymbol{p}(t_0) + s\,\boldsymbol{p}(t_1), \quad s = \frac{t-t_0}{t_1-t_0}, \tag{16.3}$$

其中 $\mathrm{Log}(\cdot)$ 是 SO(3) 到旋转向量的对数映射（见第 2 章），s 是归一化时间。式 (16.3) 就是 SE(3) 上的线性插值：平移线性、旋转走测地线（SLERP）。

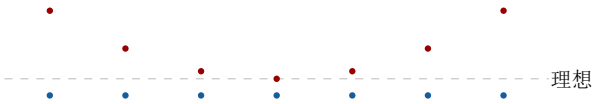
匀速模型在加速度不大时够用。它的误差量级可以估出来：帧内加速度 $a = 5\,\mathrm{m/s^2}$、时间 0.1 s 时，位置上的二阶误差约 $\frac{1}{2}a\Delta t^2 = 2.5\,\mathrm{cm}$，可以接受；但角加速度大或者有撞击时就不行了。

更可靠的做法是用 IMU 把 $T_{WL}(t)$ 真正积分出来。对 IMU 相邻两个采样时刻 t_α, t_β 之间的测量做积分（积分公式在第 6 章与第 17 章给出），得到一串离散位姿 $\{T_{WL}(t_\alpha)\}$；对落在 $[t_\alpha, t_\beta]$ 内的点，再按式 (16.3) 在相邻两个 IMU 位姿之间插值即可。至于更精细的连续时间表示（例如把整条轨迹参数化为 B 样条再做三次插值），DLIO 等工作专门讨论了这一点 [5]。实际工程中按”分段线性”处理已经能覆盖绝大多数场景。

笔记 去畸变只需要帧内相对运动，不需要绝对位姿。IMU 零偏在 100 ms 内造成的姿态漂移通常远小于 0.1°，所以去畸变对零偏不敏感；这也是为什么即使零偏还没收敛，FAST-LIO2 一类算法也能先把点云去畸变。



(a) 雷达在帧内从 A 平移到 B，同一堵墙被不同时刻“看到”



(b) 红点：整帧被当作同一时刻，沿墙“拉弯”；蓝点：去畸变后回到一条直线

图 16.1: 帧内运动造成的点云畸变。雷达在采集一帧的过程中从 A 走到 B，若不做补偿，同一堵平面墙会在点云中被弯成一条弧线；按每个点的时间戳去畸变后，点重新落到同一平面上。

16.3 演化时间线：从 LOAM 到 Point-LIO

十年间的路线可以概括成一句话：**特征越来越少，点用得越来越多，更新越来越频繁**。LOAM 靠人工设计的线/面特征做两阶段匹配；LeGO-LOAM 用地面分割把特征做得更可靠、更省算力；LIO-SAM 把估计框架换成因子图并首次把 IMU 与激光紧耦合地写进因子图；FAST-LIO 把后端换成迭代 EKF，用紧耦合把点面残差直接作用于状态；FAST-LIO2 干脆去掉特征提取，直接对原始点做配准，并配增量式 iKD-Tree 让”每帧都对新点做最近邻”变得便宜；Point-LIO 再进一步，把一个点当成一次更新，彻底摆脱”帧”的束缚。



图 16.2: 3D 激光惯性里程计的演化主线。VoxelMap（2022）与 Point-LIO（2023）几乎同时探索” 地图表示” 与” 更新粒度” 两个方向；2022 年之后视觉信息也被引入，形成 FAST-LIVO、R3LIVE 等 LVI 系统。

表 16.2: 代表性 LIO 系统一览

方法	年份	核心贡献	后端	典型场景
LOAM [46]	2014	线/面特征 + 帧间帧到图两阶段	优化	无 IMU 的室外测绘
LeGO-LOAM [34]	2018	地面分割 + 聚类 + 两步优化	优化	嵌入式、起伏地面
LIO-SAM [35]	2020	IMU 紧耦合 + 因子图 + 回环/GPS	因子图	有回环需求的室外
FAST-LIO [40]	2021	紧耦合 IEKF，点面残差	IEKF	高动态、室内外通用
FAST-LIO2 [41]	2022	去特征 + 原始点配准 + iKD-Tree	IEKF	通用主力方案
VoxelMap [43]	2022	自适应体素 + 概率平面地图	IEKF	内存敏感、结构化场景
Point-LIO [18]	2023	逐点更新、抗 IMU 饱和	逐点 KF	超高速旋转、IMU 量程小
FAST-LIVO2 [45]	2024	激光-惯性-视觉直接法紧耦合	IEKF	退化场景 + 纹理丰富

16.3.1 LOAM（2014）：两阶段、特征法、无 IMU

LOAM [46] 是这一支的鼻祖。它先按曲率把每帧点云分成” 线特征”（曲率大，如墙角、杆子）和” 面特征”（曲率小，如墙面、地面），然后用点到线、点到面两种残差做配准。它的关键设计是**两阶段分工**：高频（约 10 Hz）的里程计只拿当前帧和上一帧匹配，保证实时；低频（约 1 Hz）的建图模块把当前帧和已经建好的地图匹配，消除累积漂移。两阶段之间用位姿变换把新帧” 投” 进地图坐标系，避免了地图不断被重写。

LOAM 的局限也很清楚：没有 IMU，帧内运动畸变只能用匀速模型补偿，机器人急转时点云会被拉花；特征提取依赖人工阈值，场景纹理弱或点数稀疏时特征不稳定；两阶段之间的耦合只是” 传位姿”，本质仍是松耦合。

16.3.2 LeGO-LOAM（2018）：先分割再匹配

LeGO-LOAM [34] 的出发点是” 嵌入式平台算力不够”。它做了三件事：（1）**地面分割**：用相邻点的俯仰角把地面点分离出来；（2）**点云聚类**：对非地面点做聚类，去掉小于 30 点的杂簇，抑制噪声；（3）**两步优化**：先用地面点估计 $(z, \text{roll}, \text{pitch})$，再用非地面点估计 (x, y, yaw)，把 6 自由度问题拆成两个 3 自由度问题，计算量大幅下降。代价是：强依赖地面假设，机器人在空中或者地面点不足时失效；两步拆分也丢掉了部分耦合信息。

16.3.3 LIO-SAM（2020）：紧耦合 + 因子图

LIO-SAM [35] 是” 滤波 vs 优化” 这条分水岭上的代表作。它把 IMU 与激光首次真正紧耦合地放进一个因子图，用 GTSAM 求解。图中共有四类因子（见图 16.3）：IMU 预积分因子、激光里程计因子（点到面配准得到的相对位姿）、GPS 因子、回环因子。整个问题写成最大后验估计：

$$\mathcal{X}^\star = \arg\min_{\mathcal{X}} \frac{1}{2} \sum_{k} \| \boldsymbol{r}_k(\mathcal{X}) \|^2_{\boldsymbol{\Sigma}_k}, \tag{16.4}$$

其中 $\mathcal{X} = \{T_1, \ldots, T_N\}$ 是所有关键帧位姿，$\boldsymbol{r}_k$ 是第 k 个因子的残差（例如预积分因子预测的相对位姿与状态中相对位姿之差），$\|\boldsymbol{r}\|^2_{\boldsymbol{\Sigma}} = \boldsymbol{r}^\top \boldsymbol{\Sigma}^{-1} \boldsymbol{r}$ 是马氏距离的平方（定义见第 2 章）。式 (16.4) 用高斯-牛顿迭代求解，每次迭代解一个稀疏线性方程组（细节见第 13 章）。

因子图相对滤波器的根本优势是：**线性化点可以反复修正**。滤波器一旦把某一帧线性化掉、边缘化掉，就再也改不了；而因子图保留历史状态，回环时可以把整条轨迹一起拉直。代价是计算量随关键帧数量增长，必须靠滑窗边缘化控制规模。

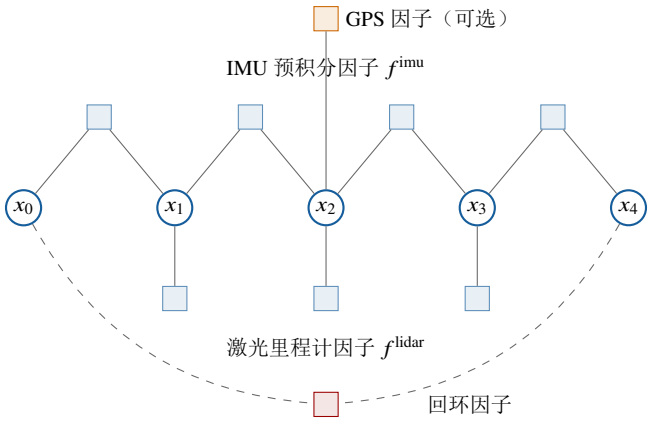


图 16.3: LIO-SAM 的因子图结构。圆形是待估状态（关键帧位姿），方块是因子（约束）。IMU 预积分因子约束相邻帧之间的相对运动，激光里程计因子给出相邻帧间的相对位姿观测，回环因子把相距很远的帧连起来，GPS 因子提供绝对位置。

16.3.4 FAST-LIO（2021）与 FAST-LIO2（2022）：把后端换成迭代 EKF

FAST-LIO [40] 的贡献是把” 紧耦合” 做到极致：状态里不仅有姿态、位置、速度、零偏，还包含重力方向；观测不再是” 位姿”，而是每个点与地图平面的点到面残差；求解器是**迭代扩展卡尔曼滤波**（IEKF），在一次更新里反复线性化、反复重新建立数据关联。为了保证实时，它引入了流形上的迭代公式，让每次迭代的更新量保持在切空间里，避免了四元数归一化之类的数值问题。

FAST-LIO2 [41] 做了一个看起来很小、实际上影响深远的决定：**去掉特征提取**。既然点到面残差已经足够，为什么还要花时间算曲率、挑特征？它直接拿降采样后的原始点云去和地图配准，配合**增量式 KD 树**（iKD-Tree）——地图点可以动态插入、惰性删除、按包围盒批量操作，不必每帧重建整棵树。这一改动同时带来了三个好处：一是省掉了特征提取的时间，二是弱纹理、非结构化场景也能用，三是原始点携带的信息比几百个特征点丰富得多。

FAST-LIO 系列的设计哲学

” 不要为了省计算量而丢信息。” 在 CPU 上，一个点到面残差的计算只要几十次浮点运算，而丢掉的几何信息可能让配准在退化场景下失败。FAST-LIO2 用更便宜的数据结构（iKD-Tree）换来了” 用全部点” 的奢侈。

16.3.5 Point-LIO（2023）与 VoxelMap（2022）：两个方向的突破

Point-LIO [18] 质疑了” 帧” 这个概念本身：为什么一定要等一帧点云攒齐再更新？它让**每个点一到就做一次更新**，点的时间戳直接用于状态传播，因此状态更新的频率等于点的频率（可达 10^5 Hz 量级），并且天然免疫 IMU 饱和——当 IMU 读数不可信时，状态靠逐点的激光观测一步一步拉回来。VoxelMap [43] 走的是地图表示路线：不用点云地图，而用**自适应体素**——每个体素里的点如果近似共面，就只存一个平面参数和它的协方差；不够平就继续八叉树细分裂开。这样地图又小又快，还能给出平面的不确定性。两者都会在第 17 章展开。

16.3.6 视觉-激光-惯性：FAST-LIVO、R3LIVE、LVI-SAM

激光在几何退化（长走廊、纯平面墙）时无解，而相机恰好能提供纹理约束；相机在弱光、无纹理、快速运动时失效，激光恰好不受影响。于是 2021 年之后出现了一批 LVI 系统：LVI-SAM [36] 用两个因子图（视觉-惯性和激光-惯性）互相提供初值；R3LIVE [25] 同时建几何地图和视觉纹理地图；FAST-LIVO [44] 与 FAST-LIVO2 [45] 把点到面残差与光度误差放进同一个 IEKF 中做直接法紧耦合。它们的共性是：状态估计的骨架仍然是 LIO，视觉只是” 多一类残差”。本书只在第 17 章用一小节说明其定位，不展开推导。

16.4 两条技术路线：特征法与直接法、滤波与优化

把上面这些方法按两个维度切开，就得到表 16.3。这两个维度经常被混为一谈，其实它们回答的是完全不同的两个问题：**用什么残差**（特征法 vs 直接法）和**用什么估计框架**（滤波 vs 优化）。

表 16.3: LIO 方法的二维分类

	滤波（递归）	优化（因子图/批量）
特征法	FAST-LIO（线/面点 + IEKF）	LOAM、LeGO-LOAM、LIO-SAM
直接法	FAST-LIO2、Point-LIO、VoxelMap	较少见（需把配准残差写成因子）

特征法 vs 直接法。特征法先挑出少量”好”的点（线点、面点），只对这些点建残差；直接法对全部（降采样后的）点建残差。前者的好处是残差数量少、每次迭代便宜，坏处是特征提取的阈值对场景敏感，且在结构化程度低的场景里挑不出特征；后者好处是信息完整、对场景没有假设，坏处是点数多，必须靠高效的数据结构（iKD-Tree、体素哈希）和降采样来控制计算量。值得注意的是，”直接法”在 LIO 语境里指的是直接使用原始点，而不是视觉里的”直接法”（不提取特征、直接最小化光度误差），两者思想相通但形式不同。

滤波 vs 优化。滤波（EKF/IEKF）只维护当前时刻的状态和协方差，来一帧就”吃掉”一帧，计算量与历史长度无关；优化（因子图）保留全部关键帧，一次性求解所有状态，计算量随规模增长，但可以反复线性化、天然支持回环和全局一致性。第 9 章和第 13 章分别讲过两种框架的概率基础，这里只强调一点：**在里程计这个局部问题上，二者的精度差距远小于在 SLAM 全局问题上的差距**。里程计只需要局部一致，滤波的”不能回头改”并不致命；而一旦要做回环，滤波器就必须额外加一个位姿图模块，否则误差永远补不回来。

16.5 紧耦合到底“紧”在哪

16.5.1 两种耦合的数学形式

16.5.1.0.1 松耦合（级联） 的做法是两级估计。第一步，独立的扫描匹配给出位姿观测 $\boldsymbol{z} = T_k^L$ 及其协方差 $\boldsymbol{\Sigma}_L$；第二步，把 IMU 递推结果当作先验，把 $\boldsymbol{z}$ 当作观测做一次 EKF：

$$\begin{aligned} \bar{\boldsymbol{x}}_k &= \boldsymbol{f}(\boldsymbol{x}_{k-1}, \boldsymbol{u}_k), & \bar{\boldsymbol{P}}_k &= \boldsymbol{F}\boldsymbol{P}_{k-1}\boldsymbol{F}^\top + \boldsymbol{Q}, \\ \boldsymbol{K} &= \bar{\boldsymbol{P}}_k\boldsymbol{H}^\top\left(\boldsymbol{H}\bar{\boldsymbol{P}}_k\boldsymbol{H}^\top + \boldsymbol{\Sigma}_L\right)^{-1}, & \boldsymbol{H} &= \begin{bmatrix}\boldsymbol{I}_6 & \boldsymbol{0}_{6\times 12}\end{bmatrix}. \end{aligned} \tag{16.5}$$

式 (16.5) 中 $\boldsymbol{H}$ 只把状态里的位姿部分”取出来”和观测比较，观测维度是 6（位姿）。这确实是紧耦合公式的简化版，但信息流是单向的：扫描匹配不知道 IMU 的存在，它算出来的 $\boldsymbol{\Sigma}_L$ 也没有考虑 IMU 先验；而 IMU 只知道”激光说我在这个位姿”，不知道这个位姿是靠哪几个点、在哪个方向上算出来的。

16.5.1.0.2 紧耦合（联合） 的做法是只保留一个估计器，观测不是 6 维位姿，而是 m 个点的点面残差：

$$\boldsymbol{K} = \bar{\boldsymbol{P}}_k\boldsymbol{H}^\top\left(\boldsymbol{H}\bar{\boldsymbol{P}}_k\boldsymbol{H}^\top + \boldsymbol{R}_m\right)^{-1}, \qquad \boldsymbol{H} = \frac{\partial \boldsymbol{r}}{\partial \delta\boldsymbol{x}} \in \mathbb{R}^{m\times 18}, \tag{16.6}$$

其中 $\boldsymbol{r} = [r_1, \ldots, r_m]^\top$，$r_i = \boldsymbol{n}_i^\top(\boldsymbol{R}\boldsymbol{p}_i + \boldsymbol{p} - \boldsymbol{q}_i)$ 是第 i 个点的点到面残差（$\boldsymbol{n}_i$ 是从地图拟合出的平面法向，$\boldsymbol{q}_i$ 是平面上一点），$\boldsymbol{R}_m$ 是各点观测噪声拼成的对角阵。式 (16.6) 和式 (16.5) 的代数形式完全一样，差别全在 $\boldsymbol{H}$ 和观测的含义上：前者是 6 行的”位姿投影”，后者是 m 行的”几何残差”，m 通常是几百到几千。

16.5.2 紧耦合为什么更鲁棒

把两者的差别想成一个很具体的场景：机器人正沿一条长走廊前进，走廊方向几何退化，扫描匹配在轴线方向上算不出位移。松耦合下，扫描匹配只好输出一个”轴线方向几乎没动”的位姿，并且给出一个看起来不大、但实际上错误的 $\boldsymbol{\Sigma}_L$；滤波器看到这个观测后会相信它，把 IMU 预积分算出来的正确位移拽回去。紧耦合下，$\boldsymbol{H}^\top\boldsymbol{H}$ 在轴线方向上同样接近奇异，但这个奇异方向上的不确定度由先验协方差 $\bar{\boldsymbol{P}}_k$ 提供（IMU 已经算出了位移），贝

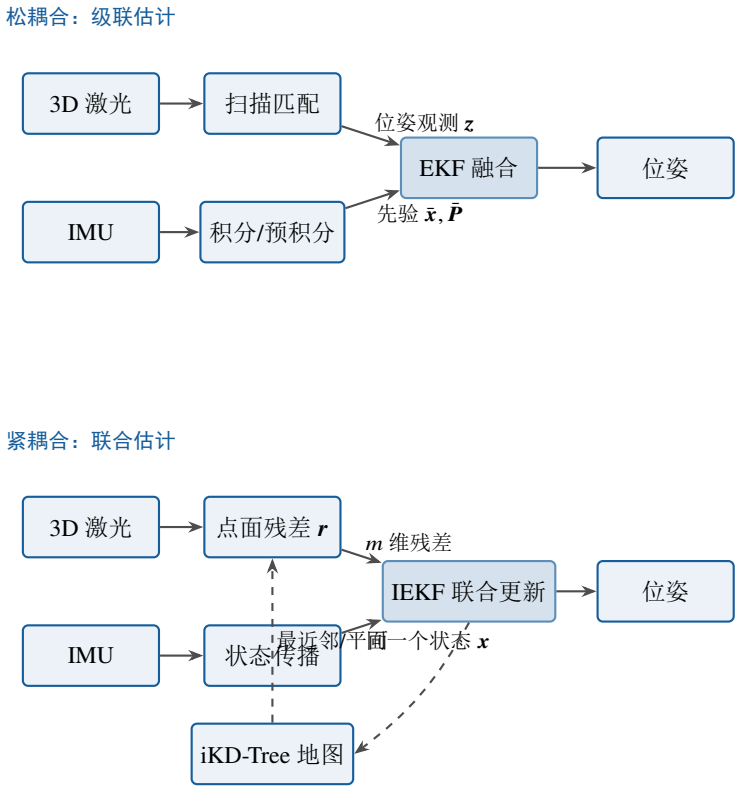


图 16.4: 松耦合与紧耦合的信息流对比。松耦合中扫描匹配是独立模块，只把 6 自由度位姿交给滤波器；紧耦合中点云直接产生残差，与 IMU 传播结果在同一个状态向量、同一个协方差里融合，地图也参与其中。

叶斯融合会自动在” 激光没约束” 的方向上保留 IMU 的推断，只在激光有约束的方向上修正。换句话说，**松耦合把退化藏进了 $\boldsymbol{\Sigma}_L$，紧耦合让退化在 $\boldsymbol{H}$ 里显式暴露出来**，滤波器就能按方向分别处理。

注 [一个容易误解的点] 在 FAST-LIO2 中，点到面残差对零偏的雅可比恰好为零（点的坐标变换不依赖零偏），那零偏是怎么被激光修正的？答案是：通过传播形成的先验互协方差 $\boldsymbol{P}_{\theta b_g}$。姿态误差与陀螺零偏在预测步就被耦合在一起，激光残差修正姿态时，$\boldsymbol{K}$ 中对应的行会把零偏一起拉动。这仍然是紧耦合——两者共享同一个状态和同一个协方差矩阵；只是耦合路径是” 先验相关性” 而不是” 残差显式依赖”。

16.6 选型建议

表 16.4: LIO 方案选型决策表

需求/场景	推荐方案	理由
纯 2D 平面、算力极低	Cartographer（第 15 章）	3 自由度，代价最小
地面机器人、需要回环与 GPS	LIO-SAM [35]	因子图可回环、可融合绝对观测
通用 3D、稳定优先	FAST-LIO2 [41]	无特征、直接法、社区成熟
剧烈旋转、IMU 量程不足	Point-LIO [18]	逐点更新，帧内不依赖 IMU 可信
长走廊/隧道等退化场景	视觉 + 激光紧耦合（LVI）	纹理补几何
内存/算力受限、结构化场景	VoxelMap [43]	平面地图，存储与查询都省
无 IMU 可用	LOAM 类纯激光	不依赖惯性观测

选型时还要考虑三个工程约束：（1）**时间同步**：激光与 IMU 的时间戳必须同步到毫秒级，否则去畸变和传播都会错，这是 LIO 现场翻车的第一大原因；（2）**外参标定**：雷达到 IMU 的旋转和平移要足够准，1° 的外参误差在 10 m 处就是 17 cm 的偏差；（3）**IMU 量程**：RoboMaster 的撞击瞬间角速度很容易超过 2000°/s，选型时要看陀螺量程和算法对饱和的鲁棒性。

本章小结

1. 3D 场景需要 6 自由度位姿，纯激光里程计在几何退化、帧内畸变、快速运动三种情况下失效；IMU 频率高、短时准，恰好补上帧内运动，激光则把 IMU 的漂移拉回来。
2. LIO 的定义是” 激光与 IMU 在同一个估计器中联合估计位姿及辅助状态”。紧耦合 = 原始测量进同一次更新；松耦合 = 一个传感器的估计结果当另一个的观测。
3. 运动畸变来自” 一帧点云是一个 100 ms 的时间窗”，去畸变的本质是把每个点按自己的时间戳变换到参考时刻：$\boldsymbol{p}_i^{L_{\text{ref}}} = R(t_{\text{ref}})^{\top}(R(t_i)\boldsymbol{p}_i^{L} + \boldsymbol{p}(t_i) - \boldsymbol{p}(t_{\text{ref}}))$。
4. 去畸变只需要帧内相对运动；匀速模型误差约 $\frac{1}{2}a\Delta t^2$，IMU 辅助模型用积分轨迹插值，连续时间表示（B 样条）精度更高但更贵。
5. 演化主线：LOAM（特征法两阶段）→ LeGO-LOAM（地面分割、轻量化）→ LIO-SAM（紧耦合因子图、可回环）→ FAST-LIO（IEKF + 点面残差）→ FAST-LIO2（去特征 + iKD-Tree）→ Point-LIO（逐点更新、抗饱和）。
6. 两个正交的维度：特征法/直接法回答” 用什么残差”，滤波/优化回答” 用什么估计框架”，不要混为一谈。
7. 紧耦合更鲁棒的根本原因：退化方向上的不确定度由先验协方差承担，滤波器可以” 按方向” 融合，而不是把一个退化的 6 维位姿当成可靠观测。

第十六章 练习

1. 设激光雷达转速 10 Hz，机器人以 1 m/s 平移、同时以 60°/s 绕竖直轴旋转，点距离 4 m。估算帧首帧尾的点在空间上错开多少（分别给出平移项与旋转项）。
2. 用式 (16.3) 推导匀速模型下帧中点（$s = 1/2$）的位姿，并说明若真实轨迹在帧内有恒定角加速度 α，插值误差的一阶项是多少。
3. 证明式 (16.5) 与式 (16.6) 在 $\boldsymbol{H}$ 取 $\begin{bmatrix}\boldsymbol{I}_6 & \boldsymbol{0}\end{bmatrix}$、且扫描匹配的 $\boldsymbol{\Sigma}_L$ 恰好等于” 把全部点残差边缘化掉” 的结果时等价。这个等价条件在实践中为什么几乎不成立？
4. 长走廊场景下，$\boldsymbol{H}^{\top}\boldsymbol{H}$ 为什么会奇异？请写出沿走廊方向的零空间向量，并解释为什么先验协方差可以补上这个方向。
5. LIO-SAM 用因子图、FAST-LIO2 用 IEKF，两者都能跑出厘米级轨迹。请各举一个” 其中一个明显更优” 的场景，并说明原因。

延伸阅读

- FAST-LIO2 论文 [41]：读它之前先读第 17 章，重点看状态定义、IEKF 更新式与 iKD-Tree 的算法描述。
- LIO-SAM 论文 [35] 与 GTSAM 文档：理解因子图如何把 IMU 预积分、激光位姿、回环写进同一个优化问题。
- Point-LIO 论文 [18]：重点看它如何用逐点更新绕开” 帧内 IMU 不可信” 的问题，以及饱和检测的处理方式。
- VoxelMap 论文 [43]：理解自适应体素与概率平面地图，对比 iKD-Tree 的存储与查询代价。

第十七章 FAST-LIO2、Point-LIO 与先进 LIO 深度解析

本章导读

这是全书数学密度最高的一章。前 16 章里我们反复听到两个名字：FAST-LIO2 和 Point-LIO。它们为什么能在普通 CPU 上跑到 10 kHz 的状态输出、能在剧烈撞击下不发散？本章把它们的推导从头拆到底：先在 $\mathrm{SO}(3)\times\mathbb{R}^{15}$ 上定义状态和误差状态，再把 IMU 连续运动方程积分成离散传播，然后一块一块推出 18×18 的误差状态转移矩阵 $\boldsymbol{F}$，接着用最大后验估计重新导出迭代 EKF（IEKF）的更新式与收敛判据，逐项推导点到面残差对状态的雅可比，最后讲清 iKD-Tree 的增量插入、box-wise 删除与 Point-LIO 的逐点更新。读完本章，你应该能对着论文公式直接找到源码里对应的函数，也能自己改一版滤波器。

17.1 FAST-LIO2 的总体架构

FAST-LIO2 [41] 的程序结构可以概括成” 一条主线、两个频率”：

- **主线**：状态 $\boldsymbol{x}$ 与协方差 $\boldsymbol{P}$ 是唯一被维护的东西。IMU 来了就**预测**（把 $\boldsymbol{x},\boldsymbol{P}$ 沿运动方程往前推），激光来了就**更新**（用点面残差把 $\boldsymbol{x},\boldsymbol{P}$ 修正）。
- **两个频率**：IMU 一般 100 ~ 200 Hz（$\Delta t\approx 5\sim 10$ ms），激光 10 Hz（一帧 100 ms）。预测在 IMU 频率上进行，状态输出（odom）也在 IMU 频率上；更新只在每帧点云到达时进行。
- **地图**：所有去畸变后的点按世界坐标插入**增量式 KD 树**（iKD-Tree），既作为下一帧最近邻查询的地图，也作为建图的输出。

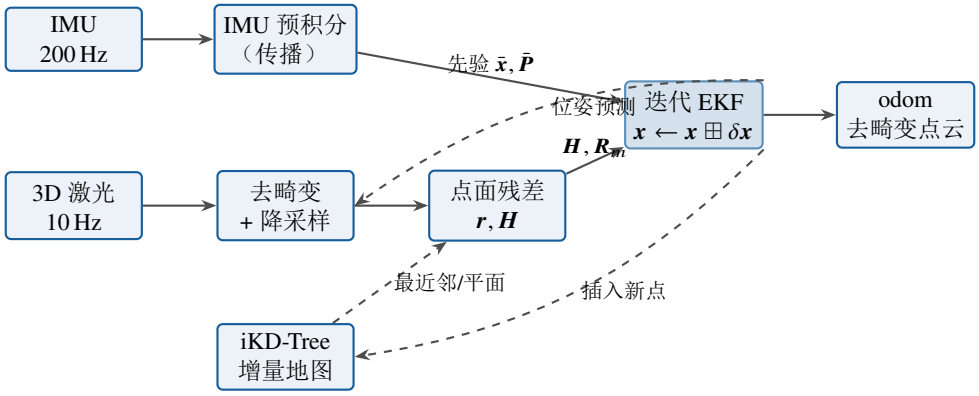


图 17.1: FAST-LIO2 的数据流。IMU 以 200 Hz 驱动状态传播并持续输出里程计；激光帧到达后，先用传播位姿把点云去畸变、降采样，再为每个点在 iKD-Tree 中找平面、算点面残差，最后用迭代 EKF 一次性融合全部残差，并把新点插回地图。

一句话总结 FAST-LIO2 的三个关键设计：状态里放重力并跟随估计；误差状态定义在流形切空间上，更新量小、线性化准；**一次更新用上全部点**，不做特征提取，靠 iKD-Tree 把” 对每个点找最近邻” 的成本压到可接受。

17.2 状态定义与流形更新

17.2.1 主状态

FAST-LIO2 的主状态取为

$$\boldsymbol{x}=\left[\boldsymbol{R},\ \boldsymbol{p},\ \boldsymbol{v},\ \boldsymbol{b}_g,\ \boldsymbol{b}_a,\ \boldsymbol{g}\right], \tag{17.1}$$

其中 $\boldsymbol{R} \in \mathrm{SO}(3)$ 是 IMU 系到世界系的旋转（T_{WB} 的旋转部分），$\boldsymbol{p} \in \mathbb{R}^3$ 是 IMU 系原点在世界系中的位置，$\boldsymbol{v} \in \mathbb{R}^3$ 是世界系速度，$\boldsymbol{b}_g, \boldsymbol{b}_a \in \mathbb{R}^3$ 是陀螺与加速度计零偏（定义见第 3 章），$\boldsymbol{g} \in \mathbb{R}^3$ 是重力向量，默认 $\boldsymbol{g} = [0, 0, -9.81]^\top$。注意 $\boldsymbol{R}$ 不是 $\mathbb{R}^3$ 里的普通向量，而是李群 $\mathrm{SO}(3)$ 上的元素，所以状态空间是流形

$$\mathcal{M} = \mathrm{SO}(3) \times \mathbb{R}^{15}, \tag{17.2}$$

$\mathbb{R}^{15}$ 来自 $\boldsymbol{p}, \boldsymbol{v}, \boldsymbol{b}_g, \boldsymbol{b}_a$（各 3 维）与 $\boldsymbol{g}$（按 3 维写）。严格说重力是单位向量，只有 2 个自由度，实际实现把它参数化在球面 S^2 上，于是误差状态是 17 维；本节为了式子整齐按 3 维 $\delta\boldsymbol{g}$ 推导，并对两者的关系给出说明。

17.2.2 为什么把重力放进状态

初学者常问：重力不是常数吗，为什么要估计？原因有三。其一，**初始姿态未知**：开机时并不知道机器人的 roll/pitch，而静止的加速度计测到的正是 $-\boldsymbol{R}^\top \boldsymbol{g}$，所以" 重力方向" 与" 初始姿态" 是同一件事的两面，把 $\boldsymbol{g}$ 放进状态等于让滤波器自己去对齐重力。其二，**标定残差**：加速度计的零偏与标度因数误差在长时间上表现为一个" 等效的重力方向偏差"，在线估 $\boldsymbol{g}$ 能吸收一部分。其三，**一致性**：若把 $\boldsymbol{g}$ 写死而初始姿态差 2°，这个误差会被永久固化在重力方向上，速度持续被一个错误的常值加速度污染，位置随时间二次发散；让 $\boldsymbol{g}$ 可估，滤波器就能把" 一开始估错了" 改过来。

17.2.3 误差状态与两个流形运算

我们不直接在 $\boldsymbol{x}$ 上做加法（$\boldsymbol{R} + \delta\boldsymbol{R}$ 会破坏正交性），而是定义**误差状态**

$$\delta\boldsymbol{x} = \left[\delta\boldsymbol{\theta},\ \delta\boldsymbol{p},\ \delta\boldsymbol{v},\ \delta\boldsymbol{b}_g,\ \delta\boldsymbol{b}_a,\ \delta\boldsymbol{g}\right] \in \mathbb{R}^{18}, \tag{17.3}$$

并用两个流形运算把" 状态" 和" 误差" 联系起来：

$$\boldsymbol{x} \boxplus \delta\boldsymbol{x} \triangleq \begin{cases} \boldsymbol{R} \leftarrow \boldsymbol{R}\exp\left([\delta\boldsymbol{\theta}]_\times\right), \\ \boldsymbol{p} \leftarrow \boldsymbol{p} + \delta\boldsymbol{p}, \quad \boldsymbol{v} \leftarrow \boldsymbol{v} + \delta\boldsymbol{v}, \\ \boldsymbol{b}_g \leftarrow \boldsymbol{b}_g + \delta\boldsymbol{b}_g, \quad \boldsymbol{b}_a \leftarrow \boldsymbol{b}_a + \delta\boldsymbol{b}_a, \quad \boldsymbol{g} \leftarrow \boldsymbol{g} + \delta\boldsymbol{g}, \end{cases} \tag{17.4}$$

$$\boldsymbol{x} \boxminus \bar{\boldsymbol{x}} \triangleq \left[\mathrm{Log}(\bar{\boldsymbol{R}}^\top \boldsymbol{R}),\ \boldsymbol{p} - \bar{\boldsymbol{p}},\ \boldsymbol{v} - \bar{\boldsymbol{v}},\ \boldsymbol{b}_g - \bar{\boldsymbol{b}}_g,\ \boldsymbol{b}_a - \bar{\boldsymbol{b}}_a,\ \boldsymbol{g} - \bar{\boldsymbol{g}}\right]. \tag{17.5}$$

式 (17.4) 中的旋转部分叫**右扰动**：新姿态等于旧姿态右乘一个由 $\delta\boldsymbol{\theta}$ 生成的小旋转。$\boxminus$ 是它的逆运算，$\bar{\boldsymbol{R}}^\top \boldsymbol{R}$ 表示" 从参考姿态到当前姿态还需要转多少"，再取对数变回旋转向量。这两个运算合起来构成切空间上的" 加法"，是 ESKF 与流形滤波的标准做法 [37]。

为什么必须用误差状态？因为 $\boldsymbol{R}$ 只有 3 个自由度却用 9 个数表示，直接对 9 个数加噪声会破坏 $\boldsymbol{R}^\top\boldsymbol{R} = \boldsymbol{I}$；而 $\delta\boldsymbol{\theta}$ 是 3 维的小量，可以放心地做线性化、做高斯假设。另一个好处是：误差状态在机器人正常运动时始终接近零，一阶线性化的误差极小，这正是一阶 EKF 能这么准的原因。

这里补充两点代码层面的约定。第一，官方实现把各状态块命名为 `pos`、`rot`、`vel`、`bg`、`ba`、`grav`，并默认把 LiDAR 到 IMU 的外参 $\boldsymbol{R}_L, \boldsymbol{t}_L$（各 3 维）也放进状态，所以误差状态是 23 维而不是 17 维，多出来的 6 维是外参；外参块的推导与位姿块完全同构，本书为突出主线只推导 18 维。第二，重力按 S^2 的 2 维切空间处理，最后一块是 3×2 的窄矩阵，读者在代码里看到 `dim` 为 23 时不要惊讶。

17.3 IMU 运动学：从连续方程到离散传播

17.3.1 测量模型

IMU 输出的是角速度 $\tilde{\boldsymbol{\omega}}$ 与比力 $\tilde{\boldsymbol{a}}$（加速度计测的是" 除重力以外的比力"，不是加速度本身）。设真实角速度 $\boldsymbol{\omega}$ 与真实加速度 $\boldsymbol{a}$，则测量模型为

$$\tilde{\boldsymbol{\omega}} = \boldsymbol{\omega} + \boldsymbol{b}_g + \boldsymbol{n}_g, \qquad \tilde{\boldsymbol{a}} = \boldsymbol{R}^\top\left(\boldsymbol{a} - \boldsymbol{g}\right) + \boldsymbol{b}_a + \boldsymbol{n}_a, \tag{17.6}$$

其中 $\boldsymbol{n}_g, \boldsymbol{n}_a$ 是零均值高斯白噪声，$\boldsymbol{b}_g, \boldsymbol{b}_a$ 是缓慢变化的零偏（建模为随机游走）。式 (17.6) 的第二个式子说明：加速度计在机体系里测到的是"世界系加速度 $\boldsymbol{a}$ 减去重力 $\boldsymbol{g}$"，再转到机体系并加上零偏噪声。静止时 $\boldsymbol{a}=0$，读数就是 $\boldsymbol{R}^{\top}(-\boldsymbol{g})$，这就是为什么静止时可以用加速度计测 roll/pitch。

17.3.2 连续运动方程

对式 (17.6) 反解出真实量，代入运动学，得到

$$\begin{cases} \dot{\boldsymbol{R}} = \boldsymbol{R}\,[\boldsymbol{\omega}]_\times, \\ \dot{\boldsymbol{p}} = \boldsymbol{v}, \\ \dot{\boldsymbol{v}} = \boldsymbol{R}\left(\tilde{\boldsymbol{a}} - \boldsymbol{b}_a - \boldsymbol{n}_a\right) + \boldsymbol{g}, \\ \dot{\boldsymbol{b}}_g = \boldsymbol{n}_{bg}, \qquad \dot{\boldsymbol{b}}_a = \boldsymbol{n}_{ba}, \qquad \dot{\boldsymbol{g}} = \boldsymbol{0}, \end{cases} \tag{17.7}$$

其中 $[\boldsymbol{\omega}]_\times$ 是由 $\boldsymbol{\omega} = [\omega_x, \omega_y, \omega_z]^{\top}$ 生成的反对称矩阵：

$$[\boldsymbol{\omega}]_\times = \begin{bmatrix} 0 & -\omega_z & \omega_y \\ \omega_z & 0 & -\omega_x \\ -\omega_y & \omega_x & 0 \end{bmatrix}, \qquad [\boldsymbol{\omega}]_\times \boldsymbol{u} = \boldsymbol{\omega} \times \boldsymbol{u}. \tag{17.8}$$

第一条 $\dot{\boldsymbol{R}} = \boldsymbol{R}[\boldsymbol{\omega}]_\times$ 值得单独推一遍。设 t 到 $t+\Delta t$ 内机体系下的角速度恒为 $\boldsymbol{\omega}$，这段时间机体系相对世界系转过一个小角度 $\boldsymbol{\omega}\Delta t$，于是

$$\boldsymbol{R}(t+\Delta t) = \boldsymbol{R}(t)\exp\big([\boldsymbol{\omega}\Delta t]_\times\big) \;\approx\; \boldsymbol{R}(t)\big(\boldsymbol{I} + [\boldsymbol{\omega}]_\times \Delta t\big), \tag{17.9}$$

移项、两边除以 Δt 并令 $\Delta t \to 0$，即得 $\dot{\boldsymbol{R}} = \boldsymbol{R}[\boldsymbol{\omega}]_\times$。注意 $\boldsymbol{\omega}$ 是**机体系**下的角速度，所以要右乘；若用世界系角速度 $\boldsymbol{\omega}^W$，则应是 $\dot{\boldsymbol{R}} = [\boldsymbol{\omega}^W]_\times \boldsymbol{R}$。这两种写法等价，但混用会导致符号错误，本书统一用机体系角速度、右乘。

式 (17.7) 前三条的物理含义：姿态由角速度积分，位置由速度积分，速度由"比力转到世界系再加上重力"积分。后三条说明零偏和重力在**两次 IMU 采样之间**被视为常量，它们的随机游走只在长时间尺度上起作用。

17.3.3 离散传播

IMU 采样间隔 Δt 通常为 5 ~ 10 ms，远小于激光帧长 100 ms。设第 k 个 IMU 采样时刻为 t_k，把式 (17.7) 在 $[t_k, t_k+\Delta t]$ 上积分。假设区间内 $\boldsymbol{\omega}$、$\boldsymbol{a}$ 恒定（零阶保持），且 t_k 时刻的状态已知，则对任意 $\tau \in [0, \Delta t]$：

$$\boldsymbol{R}(t_k+\tau) = \boldsymbol{R}_k \exp\big([\boldsymbol{\omega}_k \tau]_\times\big), \qquad \boldsymbol{\omega}_k \triangleq \tilde{\boldsymbol{\omega}}_k - \boldsymbol{b}_{g,k}. \tag{17.10}$$

速度通过对加速度积分得到：

$$\boldsymbol{v}(t_k+\tau) = \boldsymbol{v}_k + \int_0^{\tau} \Big[\boldsymbol{R}_k \exp\big([\boldsymbol{\omega}_k s]_\times\big)\boldsymbol{a}_k + \boldsymbol{g}\Big]\mathrm{d}s, \qquad \boldsymbol{a}_k \triangleq \tilde{\boldsymbol{a}}_k - \boldsymbol{b}_{a,k}. \tag{17.11}$$

把 $\exp([\boldsymbol{\omega}_k s]_\times) = \boldsymbol{I} + [\boldsymbol{\omega}_k]_\times s + O(s^2)$ 代入，只保留到一阶：

$$\boldsymbol{v}(t_k+\tau) = \boldsymbol{v}_k + \boldsymbol{R}_k \boldsymbol{a}_k\,\tau + \boldsymbol{g}\,\tau + O(\tau^2). \tag{17.12}$$

位置再由 $\boldsymbol{p}(t_k+\tau) = \boldsymbol{p}_k + \int_0^{\tau} \boldsymbol{v}(t_k+s)\mathrm{d}s$ 积分一次，保留到 τ^2：

$$\boldsymbol{p}(t_k+\tau) = \boldsymbol{p}_k + \boldsymbol{v}_k\tau + \frac{1}{2}\big(\boldsymbol{R}_k\boldsymbol{a}_k + \boldsymbol{g}\big)\tau^2 + O(\tau^3). \tag{17.13}$$

取 $\tau = \Delta t$，再补上零偏的零阶保持，得到工程上最常用的离散传播公式：

$$\boxed{\begin{aligned} \boldsymbol{R}_{k+1} &= \boldsymbol{R}_k \exp\big([\boldsymbol{\omega}_k \Delta t]_\times\big), \\ \boldsymbol{v}_{k+1} &= \boldsymbol{v}_k + \big(\boldsymbol{R}_k\boldsymbol{a}_k + \boldsymbol{g}_k\big)\Delta t, \\ \boldsymbol{p}_{k+1} &= \boldsymbol{p}_k + \boldsymbol{v}_k\Delta t + \frac{1}{2}\big(\boldsymbol{R}_k\boldsymbol{a}_k + \boldsymbol{g}_k\big)\Delta t^2, \\ \boldsymbol{b}_{g,k+1} &= \boldsymbol{b}_{g,k}, \qquad \boldsymbol{b}_{a,k+1} = \boldsymbol{b}_{a,k}, \qquad \boldsymbol{g}_{k+1} = \boldsymbol{g}_k. \end{aligned}} \tag{17.14}$$

式 (17.14) 是” 用第 k 个采样点的测量把状态推到第 $k+1$ 个采样点”，100 ms 的一帧里要做约 20 次这样的传播，这就是预测步。注意离散化丢掉了 $O(\Delta t^2)$ 的项（旋转对加速度方向的影响、角速度变化的影响等）；由于 Δt 只有几毫秒，这些项通常可以忽略。实际实现中，为了进一步减小误差，常见做法是用**中点积分**（用 t_k 与 t_{k+1} 的测量平均值）或者把 $\boldsymbol{R}$ 的旋转放在半步位置上计算，代价是代码复杂一些。

指数映射 $\exp([\boldsymbol{\phi}]_\times)$ 用 Rodrigues 公式计算：

$$\exp\big([\boldsymbol{\phi}]_\times\big) = \boldsymbol{I} + \frac{\sin\theta}{\theta}[\boldsymbol{\phi}]_\times + \frac{1-\cos\theta}{\theta^2}[\boldsymbol{\phi}]_\times^2, \qquad \theta = \|\boldsymbol{\phi}\|. \tag{17.15}$$

当 θ 很小时用一阶近似 $\exp([\boldsymbol{\phi}]_\times) \approx \boldsymbol{I} + [\boldsymbol{\phi}]_\times$。例如以 1 rad/s 转、$\Delta t = 5$ ms 时 $\boldsymbol{\phi} = (0, 0, 0.005)$、$\theta = 0.005$，Rodrigues 公式给出的各项系数为 0.999996 与 0.4999997，与一阶近似的差别在 10^{-6} 量级——这就是为什么小角度下两者可以互换。

17.4 误差状态传播：F 矩阵逐块推导

这一节是全章的核心。我们要回答的问题是：如果 t_k 时刻的状态有一个小误差 $\delta\boldsymbol{x}_k$，那么经过一次 IMU 传播后，t_{k+1} 时刻的误差 $\delta\boldsymbol{x}_{k+1}$ 是多少？答案写成一阶线性形式

$$\delta\boldsymbol{x}_{k+1} = \boldsymbol{F}_k\,\delta\boldsymbol{x}_k + \boldsymbol{G}_k\,\boldsymbol{n}_k, \tag{17.16}$$

其中 $\boldsymbol{F}_k \in \mathbb{R}^{18\times 18}$ 是误差状态转移矩阵，$\boldsymbol{G}_k \in \mathbb{R}^{18\times 12}$ 把过程噪声 $\boldsymbol{n} = [\boldsymbol{n}_g^\top, \boldsymbol{n}_a^\top, \boldsymbol{n}_{bg}^\top, \boldsymbol{n}_{ba}^\top]^\top$ 映射进误差状态。连续时间下，$\dot{\delta\boldsymbol{x}} = \boldsymbol{F}\delta\boldsymbol{x} + \boldsymbol{G}\boldsymbol{n}$，离散化取 $\boldsymbol{F}_k \approx \boldsymbol{I} + \boldsymbol{F}\Delta t$。

下面逐块推导。**约定**：真实状态 = 名义状态 ⊞ 误差，即

$$\boldsymbol{R} = \hat{\boldsymbol{R}}\exp([\delta\boldsymbol{\theta}]_\times), \quad \boldsymbol{p} = \hat{\boldsymbol{p}} + \delta\boldsymbol{p}, \quad \boldsymbol{v} = \hat{\boldsymbol{v}} + \delta\boldsymbol{v}, \quad \boldsymbol{b}_g = \hat{\boldsymbol{b}}_g + \delta\boldsymbol{b}_g, \quad \boldsymbol{b}_a = \hat{\boldsymbol{b}}_a + \delta\boldsymbol{b}_a, \quad \boldsymbol{g} = \hat{\boldsymbol{g}} + \delta\boldsymbol{g}, \tag{17.17}$$

带帽的量是滤波器当前的名义估计，且所有推导只保留 δ 的一阶项，忽略 $O(\delta^2)$。

17.4.1 姿态误差 $\delta\boldsymbol{\theta}$

对 $\boldsymbol{R} = \hat{\boldsymbol{R}}\exp([\delta\boldsymbol{\theta}]_\times)$ 求导。先算左边：

$$\frac{\mathrm{d}}{\mathrm{d}t}\Big(\hat{\boldsymbol{R}}\exp([\delta\boldsymbol{\theta}]_\times)\Big) = \dot{\hat{\boldsymbol{R}}}\exp([\delta\boldsymbol{\theta}]_\times) + \hat{\boldsymbol{R}}\frac{\mathrm{d}}{\mathrm{d}t}\exp([\delta\boldsymbol{\theta}]_\times). \tag{17.18}$$

其中 $\dot{\hat{\boldsymbol{R}}} = \hat{\boldsymbol{R}}[\hat{\boldsymbol{\omega}}]_\times$，且由 $\exp([\delta\boldsymbol{\theta}(t+\mathrm{d}t)]_\times) \approx \exp([\delta\boldsymbol{\theta}]_\times)\exp([\dot{\delta\boldsymbol{\theta}}\mathrm{d}t]_\times)$ 可得 $\frac{\mathrm{d}}{\mathrm{d}t}\exp([\delta\boldsymbol{\theta}]_\times) = \exp([\delta\boldsymbol{\theta}]_\times)[\dot{\delta\boldsymbol{\theta}}]_\times$。于是左边为

$$\hat{\boldsymbol{R}}[\hat{\boldsymbol{\omega}}]_\times\exp([\delta\boldsymbol{\theta}]_\times) + \hat{\boldsymbol{R}}\exp([\delta\boldsymbol{\theta}]_\times)[\dot{\delta\boldsymbol{\theta}}]_\times. \tag{17.19}$$

右边由真实运动方程 $\dot{\boldsymbol{R}} = \boldsymbol{R}[\boldsymbol{\omega}]_\times$ 给出，其中 $\boldsymbol{\omega} = \tilde{\boldsymbol{\omega}} - \boldsymbol{b}_g - \boldsymbol{n}_g = \hat{\boldsymbol{\omega}} - \delta\boldsymbol{b}_g - \boldsymbol{n}_g$，所以

$$\dot{\boldsymbol{R}} = \hat{\boldsymbol{R}}\exp([\delta\boldsymbol{\theta}]_\times)[\hat{\boldsymbol{\omega}} - \delta\boldsymbol{b}_g - \boldsymbol{n}_g]_\times. \tag{17.20}$$

令左右相等，并在两边左乘 $\hat{\boldsymbol{R}}^\top$：

$$[\hat{\boldsymbol{\omega}}]_\times\exp([\delta\boldsymbol{\theta}]_\times) + \exp([\delta\boldsymbol{\theta}]_\times)[\dot{\delta\boldsymbol{\theta}}]_\times = \exp([\delta\boldsymbol{\theta}]_\times)[\hat{\boldsymbol{\omega}} - \delta\boldsymbol{b}_g - \boldsymbol{n}_g]_\times. \tag{17.21}$$

用一阶近似 $\exp([\delta\boldsymbol{\theta}]_\times) \approx \boldsymbol{I} + [\delta\boldsymbol{\theta}]_\times$ 展开两边，保留一阶项：

$$\text{左} = [\hat{\boldsymbol{\omega}}]_\times + [\hat{\boldsymbol{\omega}}]_\times[\delta\boldsymbol{\theta}]_\times + [\dot{\delta\boldsymbol{\theta}}]_\times, \tag{17.22}$$

$$\text{右} = [\hat{\boldsymbol{\omega}}]_\times - [\delta\boldsymbol{b}_g]_\times - [\boldsymbol{n}_g]_\times + [\delta\boldsymbol{\theta}]_\times[\hat{\boldsymbol{\omega}}]_\times. \tag{17.23}$$

移项，利用反对称矩阵的交换子恒等式 $[\boldsymbol{a}]_\times[\boldsymbol{b}]_\times - [\boldsymbol{b}]_\times[\boldsymbol{a}]_\times = [\boldsymbol{a}\times\boldsymbol{b}]_\times$：

$$[\dot{\delta\boldsymbol{\theta}}]_\times = [\delta\boldsymbol{\theta}]_\times[\hat{\boldsymbol{\omega}}]_\times - [\hat{\boldsymbol{\omega}}]_\times[\delta\boldsymbol{\theta}]_\times - [\delta\boldsymbol{b}_g]_\times - [\boldsymbol{n}_g]_\times = [\delta\boldsymbol{\theta}\times\hat{\boldsymbol{\omega}} - \delta\boldsymbol{b}_g - \boldsymbol{n}_g]_\times. \tag{17.24}$$

去掉两边的 $[\cdot]_\times$，并注意 $\delta\boldsymbol{\theta}\times\hat{\boldsymbol{\omega}} = -\hat{\boldsymbol{\omega}}\times\delta\boldsymbol{\theta} = -[\hat{\boldsymbol{\omega}}]_\times\delta\boldsymbol{\theta}$，得到

$$\dot{\delta\boldsymbol{\theta}} = -[\hat{\boldsymbol{\omega}}]_\times\,\delta\boldsymbol{\theta} - \delta\boldsymbol{b}_g - \boldsymbol{n}_g. \tag{17.25}$$

式 (17.25) 三项各有物理含义：$-\delta\boldsymbol{b}_g$ 是" 零偏估错多少，姿态就以多快速度偏多少"；$-\boldsymbol{n}_g$ 是陀螺噪声直接积分成角度误差；$-[\hat{\boldsymbol{\omega}}]_\times\delta\boldsymbol{\theta}$ 是因为误差 $\delta\boldsymbol{\theta}$ 表达在机体系里，而机体系自己在转，所以这个误差向量要" 跟着坐标系转"，其变化率为 $-\boldsymbol{\omega}\times\delta\boldsymbol{\theta}$。第三项是初学者最容易漏掉的耦合项。

17.4.2 位置误差 δp

由 $\dot{\boldsymbol{p}}=\boldsymbol{v}$ 直接得

$$\dot{\delta\boldsymbol{p}}=\delta\boldsymbol{v}. \tag{17.26}$$

物理含义：位置误差的变化率就是速度误差，这是纯运动学关系，没有任何近似。

17.4.3 速度误差 δv

真实加速度为 $\boldsymbol{a}=\boldsymbol{R}(\tilde{\boldsymbol{a}}-\boldsymbol{b}_a-\boldsymbol{n}_a)+\boldsymbol{g}$，名义加速度为 $\hat{\boldsymbol{a}}=\hat{\boldsymbol{R}}(\tilde{\boldsymbol{a}}-\hat{\boldsymbol{b}}_a)+\hat{\boldsymbol{g}}$。相减：

$$\begin{aligned}\dot{\delta\boldsymbol{v}}&=\boldsymbol{R}(\tilde{\boldsymbol{a}}-\boldsymbol{b}_a-\boldsymbol{n}_a)+\boldsymbol{g}-\left[\hat{\boldsymbol{R}}(\tilde{\boldsymbol{a}}-\hat{\boldsymbol{b}}_a)+\hat{\boldsymbol{g}}\right]\\&\approx\hat{\boldsymbol{R}}\big(\boldsymbol{I}+[\delta\boldsymbol{\theta}]_\times\big)\big(\tilde{\boldsymbol{a}}-\hat{\boldsymbol{b}}_a-\delta\boldsymbol{b}_a-\boldsymbol{n}_a\big)-\hat{\boldsymbol{R}}\big(\tilde{\boldsymbol{a}}-\hat{\boldsymbol{b}}_a\big)+\delta\boldsymbol{g}.\end{aligned} \tag{17.27}$$

记 $\boldsymbol{a}\triangleq\tilde{\boldsymbol{a}}-\hat{\boldsymbol{b}}_a$，展开并只保留一阶项：

$$\begin{aligned}\dot{\delta\boldsymbol{v}}&=\hat{\boldsymbol{R}}\boldsymbol{a}+\hat{\boldsymbol{R}}[\delta\boldsymbol{\theta}]_\times\boldsymbol{a}-\hat{\boldsymbol{R}}\delta\boldsymbol{b}_a-\hat{\boldsymbol{R}}\boldsymbol{n}_a-\hat{\boldsymbol{R}}\boldsymbol{a}+\delta\boldsymbol{g}+O(\delta^2)\\&=\hat{\boldsymbol{R}}[\delta\boldsymbol{\theta}]_\times\boldsymbol{a}-\hat{\boldsymbol{R}}\delta\boldsymbol{b}_a-\hat{\boldsymbol{R}}\boldsymbol{n}_a+\delta\boldsymbol{g}.\end{aligned} \tag{17.28}$$

再用 $[\delta\boldsymbol{\theta}]_\times\boldsymbol{a}=\delta\boldsymbol{\theta}\times\boldsymbol{a}=-[\boldsymbol{a}]_\times\delta\boldsymbol{\theta}$，得

$$\dot{\delta\boldsymbol{v}}=-\hat{\boldsymbol{R}}[\tilde{\boldsymbol{a}}-\hat{\boldsymbol{b}}_a]_\times\,\delta\boldsymbol{\theta}-\hat{\boldsymbol{R}}\,\delta\boldsymbol{b}_a-\hat{\boldsymbol{R}}\,\boldsymbol{n}_a+\delta\boldsymbol{g}. \tag{17.29}$$

式 (17.29) 的四项：姿态误差让比力的方向偏了，于是产生加速度误差 $-\hat{\boldsymbol{R}}[\boldsymbol{a}]_\times\delta\boldsymbol{\theta}$；加速度计零偏误差直接产生 $\hat{\boldsymbol{R}}\delta\boldsymbol{b}_a$ 的加速度偏差；噪声 $\boldsymbol{n}_a$ 同理；重力估计误差 $\delta\boldsymbol{g}$ 原封不动地进入加速度误差。做个数量级估计：姿态差 $1^\circ=0.017\,\mathrm{rad}$，比力 $10\,\mathrm{m/s^2}$，则加速度误差约 $0.17\,\mathrm{m/s^2}$，1 s 就积累出 $0.17\,\mathrm{m/s}$ 的速度误差、$8.5\,\mathrm{cm}$ 的位置误差——所以姿态和速度是强耦合的，不能分开估。

17.4.4 零偏与重力

零偏建模为随机游走，其连续时间导数是白噪声：

$$\dot{\delta\boldsymbol{b}}_g=\boldsymbol{n}_{bg},\qquad\dot{\delta\boldsymbol{b}}_a=\boldsymbol{n}_{ba}. \tag{17.30}$$

重力在世界系中是常量（忽略地球曲率与自转），名义估计的导数为零，所以

$$\dot{\delta\boldsymbol{g}}=\boldsymbol{0}. \tag{17.31}$$

式 (17.31) 看起来" 重力误差不会变"，但它会通过式 (17.29) 影响速度；而且在流形参数化下，$\boldsymbol{g}$ 的误差被约束在切空间（球面的切平面）里，模长保持不变，这才是" 重力是单位向量" 的正确表达。

17.4.5 组装 F 与 Q

把式 (17.25)、(17.26)、(17.29)、(17.30)、(17.31) 按 $\delta\boldsymbol{x} = [\delta\boldsymbol{\theta}, \delta\boldsymbol{p}, \delta\boldsymbol{v}, \delta\boldsymbol{b}_g, \delta\boldsymbol{b}_a, \delta\boldsymbol{g}]$ 的顺序排成矩阵 $\dot{\delta\boldsymbol{x}} = \boldsymbol{F}\delta\boldsymbol{x} + \boldsymbol{G}\boldsymbol{n}$，得到

$$\boldsymbol{F} = \begin{bmatrix} -[\hat{\boldsymbol{\omega}}]_\times & \boldsymbol{0} & \boldsymbol{0} & -\boldsymbol{I} & \boldsymbol{0} & \boldsymbol{0} \\ \boldsymbol{0} & \boldsymbol{0} & \boldsymbol{I} & \boldsymbol{0} & \boldsymbol{0} & \boldsymbol{0} \\ -\hat{\boldsymbol{R}}[\tilde{\boldsymbol{a}} - \hat{\boldsymbol{b}}_a]_\times & \boldsymbol{0} & \boldsymbol{0} & \boldsymbol{0} & -\hat{\boldsymbol{R}} & \boldsymbol{I} \\ \boldsymbol{0} & \boldsymbol{0} & \boldsymbol{0} & \boldsymbol{0} & \boldsymbol{0} & \boldsymbol{0} \\ \boldsymbol{0} & \boldsymbol{0} & \boldsymbol{0} & \boldsymbol{0} & \boldsymbol{0} & \boldsymbol{0} \\ \boldsymbol{0} & \boldsymbol{0} & \boldsymbol{0} & \boldsymbol{0} & \boldsymbol{0} & \boldsymbol{0} \end{bmatrix}, \qquad \boldsymbol{G} = \begin{bmatrix} -\boldsymbol{I} & \boldsymbol{0} & \boldsymbol{0} & \boldsymbol{0} \\ \boldsymbol{0} & \boldsymbol{0} & \boldsymbol{0} & \boldsymbol{0} \\ \boldsymbol{0} & -\hat{\boldsymbol{R}} & \boldsymbol{0} & \boldsymbol{0} \\ \boldsymbol{0} & \boldsymbol{0} & \boldsymbol{I} & \boldsymbol{0} \\ \boldsymbol{0} & \boldsymbol{0} & \boldsymbol{0} & \boldsymbol{I} \\ \boldsymbol{0} & \boldsymbol{0} & \boldsymbol{0} & \boldsymbol{0} \end{bmatrix}. \tag{17.32}$$

$\boldsymbol{G}$ 的列按 $\boldsymbol{n} = [\boldsymbol{n}_g, \boldsymbol{n}_a, \boldsymbol{n}_{bg}, \boldsymbol{n}_{ba}]$ 排列。$\boldsymbol{Q}$ 是过程噪声的功率谱矩阵：

$$\boldsymbol{Q} = \operatorname{diag}\big(\sigma_g^2\boldsymbol{I}_3,\ \sigma_a^2\boldsymbol{I}_3,\ \sigma_{bg}^2\boldsymbol{I}_3,\ \sigma_{ba}^2\boldsymbol{I}_3\big), \tag{17.33}$$

其中 σ_g 是陀螺的角度随机游走密度（单位 rad/s/$\sqrt{\text{Hz}}$），σ_a 是加速度计的速度随机游走密度（m/s^2/$\sqrt{\text{Hz}}$），σ_{bg}, σ_{ba} 是零偏随机游走密度。这些参数一般从 IMU 数据手册的 Allan 方差曲线读出（见第 3 章）。

表 17.1: F 矩阵分块逐块解释（行/列顺序：$\delta\boldsymbol{\theta}, \delta\boldsymbol{p}, \delta\boldsymbol{v}, \delta\boldsymbol{b}_g, \delta\boldsymbol{b}_a, \delta\boldsymbol{g}$）

块	表达式	物理来源	是否为零
$(1,1)$ $\partial\dot{\delta\boldsymbol{\theta}}/\partial\delta\boldsymbol{\theta}$	$-[\hat{\boldsymbol{\omega}}]_\times$	误差向量随机体系旋转	非线性项
$(1,4)$ $\partial\dot{\delta\boldsymbol{\theta}}/\partial\delta\boldsymbol{b}_g$	$-\boldsymbol{I}$	陀螺零偏误差直接积分	常数
$(2,3)$ $\partial\dot{\delta\boldsymbol{p}}/\partial\delta\boldsymbol{v}$	$\boldsymbol{I}$	$\dot{\boldsymbol{p}} = \boldsymbol{v}$	常数
$(3,1)$ $\partial\dot{\delta\boldsymbol{v}}/\partial\delta\boldsymbol{\theta}$	$-\hat{\boldsymbol{R}}[\tilde{\boldsymbol{a}} - \hat{\boldsymbol{b}}_a]_\times$	姿态误差使比力方向偏转	关键耦合
$(3,5)$ $\partial\dot{\delta\boldsymbol{v}}/\partial\delta\boldsymbol{b}_a$	$-\hat{\boldsymbol{R}}$	加速度计零偏误差	常数
$(3,6)$ $\partial\dot{\delta\boldsymbol{v}}/\partial\delta\boldsymbol{g}$	$\boldsymbol{I}$	重力误差直接进入加速度	常数
$(4,4),(5,5),(6,6)$	$\boldsymbol{0}$	零偏/重力自身不传播（仅由噪声驱动）	—

17.4.6 协方差传播

连续时间下协方差满足 Lyapunov 微分方程 $\dot{\boldsymbol{P}} = \boldsymbol{F}\boldsymbol{P} + \boldsymbol{P}\boldsymbol{F}^\top + \boldsymbol{G}\boldsymbol{Q}\boldsymbol{G}^\top$。工程上做一阶离散化，令 $\boldsymbol{F}_k = \boldsymbol{I} + \boldsymbol{F}\Delta t$，得到

$$\bar{\boldsymbol{P}}_{k+1} = \boldsymbol{F}_k\bar{\boldsymbol{P}}_k\boldsymbol{F}_k^\top + \boldsymbol{Q}_k, \qquad \boldsymbol{Q}_k \approx \boldsymbol{G}\boldsymbol{Q}\boldsymbol{G}^\top\Delta t. \tag{17.34}$$

把 $\boldsymbol{G}\boldsymbol{Q}\boldsymbol{G}^\top\Delta t$ 展开，得到每个 IMU 步注入的噪声协方差：

$$\boldsymbol{Q}_k = \operatorname{diag}\big(\sigma_g^2\Delta t\,\boldsymbol{I}_3,\ \boldsymbol{0},\ \sigma_a^2\Delta t\,\boldsymbol{I}_3,\ \sigma_{bg}^2\Delta t\,\boldsymbol{I}_3,\ \sigma_{ba}^2\Delta t\,\boldsymbol{I}_3\big). \tag{17.35}$$

注意 $\delta\boldsymbol{p}$ 那一块是零：位置没有直接的噪声注入，它的不确定性完全来自速度的积分。这也解释了为什么位置误差随时间按 $\sqrt{t^3}$ 增长（速度误差按 $\sqrt{t}$，积分一次变 $\sqrt{t^3}$），是惯性导航最经典的误差增长规律。

推导过程本身只有矩阵乘除，但含义很重要：$\boldsymbol{F}$ 描述”误差如何生长和互相传染”，$\boldsymbol{Q}$ 描述”每一步注入多少新误差”。$\boldsymbol{P}$ 就是这两者的累积结果。$\boldsymbol{P}$ 中非对角块（如 $\boldsymbol{P}_{\theta b_g}$）记录了”姿态误差和零偏误差有多相关”，第 17.6.4 节会看到它正是零偏被激光间接修正的通道。

一阶形式与 FAST-LIO2 论文附录中”对旋转块做精确指数处理”的版本在 $\Delta t \to 0$ 时一致；$\Delta t = 5\,\text{ms}$、转速 1000°/s 时两者相差约 10^{-4} 量级。本书按一阶形式推导是为了让”每一项从哪来”看得清楚，读代码时若看到形如 $\boldsymbol{A}(\boldsymbol{\omega}\Delta t)$ 的矩阵，那就是论文里精确处理的版本。

17.5 迭代扩展卡尔曼滤波（IEKF）

17.5.1 标准 EKF 更新回顾

高斯假设下，预测步给出先验 $\boldsymbol{x} \sim \mathcal{N}(\bar{\boldsymbol{x}}, \bar{\boldsymbol{P}})$，观测为 $\boldsymbol{z} = \boldsymbol{h}(\boldsymbol{x}) + \boldsymbol{v}$，$\boldsymbol{v} \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{R})$。EKF 在 $\bar{\boldsymbol{x}}$ 处线性化 $\boldsymbol{h}$，得到

$$\boldsymbol{K} = \bar{\boldsymbol{P}}\boldsymbol{H}^{\top}\left(\boldsymbol{H}\bar{\boldsymbol{P}}\boldsymbol{H}^{\top} + \boldsymbol{R}\right)^{-1}, \qquad \boldsymbol{x} = \bar{\boldsymbol{x}} \boxplus \boldsymbol{K}\big(\boldsymbol{z} - \boldsymbol{h}(\bar{\boldsymbol{x}})\big), \qquad \boldsymbol{P} = (\boldsymbol{I} - \boldsymbol{K}\boldsymbol{H})\bar{\boldsymbol{P}}. \tag{17.36}$$

EKF 只线性化一次。问题在于：在 LIO 里 $\boldsymbol{h}$ 是通过” 找最近平面” 定义的，而” 最近” 这件事本身依赖于当前位姿估计——位姿差一点，找到的平面就可能不同。只线性化一次，等于把” 找平面” 和” 估位姿” 割裂开，初值差时容易收敛到错误的对应关系。

17.5.2 从最大后验估计推导迭代更新

IEKF 的想法是：把更新步看成一个最大后验（MAP）问题，用高斯-牛顿反复求解，每次迭代都在新的估计点上重新线性化、重新找平面。要求解的是

$$\boldsymbol{x}^{\star} = \arg\min_{\boldsymbol{x}} \ \frac{1}{2}\left\|\boldsymbol{x} \boxminus \bar{\boldsymbol{x}}\right\|^2_{\bar{\boldsymbol{P}}} + \frac{1}{2}\left\|\boldsymbol{z} - \boldsymbol{h}(\boldsymbol{x})\right\|^2_{\boldsymbol{R}}, \tag{17.37}$$

其中第一项是” 别离先验太远”，第二项是” 要拟合观测”，$\|\boldsymbol{a}\|^2_{\boldsymbol{\Sigma}} = \boldsymbol{a}^{\top}\boldsymbol{\Sigma}^{-1}\boldsymbol{a}$。式 (17.37) 就是把贝叶斯公式 $p(\boldsymbol{x}|\boldsymbol{z}) \propto p(\boldsymbol{z}|\boldsymbol{x})p(\boldsymbol{x})$ 取负对数后忽略常数项的结果——这也说明了滤波器与优化本质上是同一个问题，只是滤波器每来一帧就消掉历史。

设当前迭代点为 $\boldsymbol{x}_j$，令 $\boldsymbol{x} = \boldsymbol{x}_j \boxplus \delta\boldsymbol{x}$。在流形上做一阶展开：

$$\boldsymbol{x} \boxminus \bar{\boldsymbol{x}} \approx \left(\boldsymbol{x}_j \boxminus \bar{\boldsymbol{x}}\right) + \delta\boldsymbol{x}, \qquad \boldsymbol{h}(\boldsymbol{x}) \approx \boldsymbol{h}(\boldsymbol{x}_j) + \boldsymbol{H}_j\delta\boldsymbol{x}, \qquad \boldsymbol{H}_j = \left.\frac{\partial \boldsymbol{h}}{\partial \delta\boldsymbol{x}}\right|_{\boldsymbol{x}_j}. \tag{17.38}$$

第一个近似能用，是因为 $\boxplus$ 在切空间里就是加法：从 $\bar{\boldsymbol{x}}$ 出发先走 $\boldsymbol{x}_j \boxminus \bar{\boldsymbol{x}}$ 再走 $\delta\boldsymbol{x}$，一阶上等于直接走两者之和。代入式 (17.37)，代价函数变成 $\delta\boldsymbol{x}$ 的二次型：

$$J(\delta\boldsymbol{x}) = \frac{1}{2}\left\|\left(\boldsymbol{x}_j \boxminus \bar{\boldsymbol{x}}\right) + \delta\boldsymbol{x}\right\|^2_{\bar{\boldsymbol{P}}} + \frac{1}{2}\left\|\boldsymbol{z} - \boldsymbol{h}(\boldsymbol{x}_j) - \boldsymbol{H}_j\delta\boldsymbol{x}\right\|^2_{\boldsymbol{R}}. \tag{17.39}$$

对 $\delta\boldsymbol{x}$ 求导并令其为零（这正是” 正规方程” 的流形版本，见第 2 章）：

$$\bar{\boldsymbol{P}}^{-1}\left[\left(\boldsymbol{x}_j \boxminus \bar{\boldsymbol{x}}\right) + \delta\boldsymbol{x}\right] - \boldsymbol{H}_j^{\top}\boldsymbol{R}^{-1}\left[\boldsymbol{z} - \boldsymbol{h}(\boldsymbol{x}_j) - \boldsymbol{H}_j\delta\boldsymbol{x}\right] = \boldsymbol{0}, \tag{17.40}$$

整理成

$$\left(\bar{\boldsymbol{P}}^{-1} + \boldsymbol{H}_j^{\top}\boldsymbol{R}^{-1}\boldsymbol{H}_j\right)\delta\boldsymbol{x} = -\bar{\boldsymbol{P}}^{-1}\left(\boldsymbol{x}_j \boxminus \bar{\boldsymbol{x}}\right) + \boldsymbol{H}_j^{\top}\boldsymbol{R}^{-1}\left(\boldsymbol{z} - \boldsymbol{h}(\boldsymbol{x}_j)\right). \tag{17.41}$$

现在要解出 $\delta\boldsymbol{x}$，需要对左边那个 18×18 矩阵求逆。用两次矩阵求逆引理（Woodbury 恒等式）把它化成和 EKF 一样的形式。记 $\boldsymbol{M} = \bar{\boldsymbol{P}}^{-1} + \boldsymbol{H}_j^{\top}\boldsymbol{R}^{-1}\boldsymbol{H}_j$，先用 $(\boldsymbol{A} + \boldsymbol{B}\boldsymbol{C}\boldsymbol{D})^{-1} = \boldsymbol{A}^{-1} - \boldsymbol{A}^{-1}\boldsymbol{B}(\boldsymbol{C}^{-1} + \boldsymbol{D}\boldsymbol{A}^{-1}\boldsymbol{B})^{-1}\boldsymbol{D}\boldsymbol{A}^{-1}$（取 $\boldsymbol{A} = \bar{\boldsymbol{P}}^{-1}$、$\boldsymbol{B} = \boldsymbol{H}_j^{\top}$、$\boldsymbol{C} = \boldsymbol{R}^{-1}$、$\boldsymbol{D} = \boldsymbol{H}_j$）：

$$\boldsymbol{M}^{-1} = \bar{\boldsymbol{P}} - \bar{\boldsymbol{P}}\boldsymbol{H}_j^{\top}\left(\boldsymbol{R} + \boldsymbol{H}_j\bar{\boldsymbol{P}}\boldsymbol{H}_j^{\top}\right)^{-1}\boldsymbol{H}_j\bar{\boldsymbol{P}}. \tag{17.42}$$

右乘 $\boldsymbol{H}_j^{\top}\boldsymbol{R}^{-1}$：

$$\begin{aligned}
\boldsymbol{M}^{-1}\boldsymbol{H}_j^{\top}\boldsymbol{R}^{-1} &= \bar{\boldsymbol{P}}\boldsymbol{H}_j^{\top}\boldsymbol{R}^{-1} - \bar{\boldsymbol{P}}\boldsymbol{H}_j^{\top}\left(\boldsymbol{R} + \boldsymbol{H}_j\bar{\boldsymbol{P}}\boldsymbol{H}_j^{\top}\right)^{-1}\boldsymbol{H}_j\bar{\boldsymbol{P}}\boldsymbol{H}_j^{\top}\boldsymbol{R}^{-1} \\
&= \bar{\boldsymbol{P}}\boldsymbol{H}_j^{\top}\left[\boldsymbol{I} - \left(\boldsymbol{R} + \boldsymbol{H}_j\bar{\boldsymbol{P}}\boldsymbol{H}_j^{\top}\right)^{-1}\boldsymbol{H}_j\bar{\boldsymbol{P}}\boldsymbol{H}_j^{\top}\right]\boldsymbol{R}^{-1} \\
&= \bar{\boldsymbol{P}}\boldsymbol{H}_j^{\top}\left(\boldsymbol{R} + \boldsymbol{H}_j\bar{\boldsymbol{P}}\boldsymbol{H}_j^{\top}\right)^{-1}\boldsymbol{R}\,\boldsymbol{R}^{-1} = \boldsymbol{K}_j,
\end{aligned} \tag{17.43}$$

倒数第二步用了 $\boldsymbol{I} - (\boldsymbol{R} + \boldsymbol{S})^{-1}\boldsymbol{S} = (\boldsymbol{R} + \boldsymbol{S})^{-1}\boldsymbol{R}$（记 $\boldsymbol{S} = \boldsymbol{H}_j\bar{\boldsymbol{P}}\boldsymbol{H}_j^{\top}$）。同理右乘 $\bar{\boldsymbol{P}}^{-1}$：

$$\boldsymbol{M}^{-1}\bar{\boldsymbol{P}}^{-1} = \boldsymbol{I} - \bar{\boldsymbol{P}}\boldsymbol{H}_j^{\top}\left(\boldsymbol{R} + \boldsymbol{H}_j\bar{\boldsymbol{P}}\boldsymbol{H}_j^{\top}\right)^{-1}\boldsymbol{H}_j = \boldsymbol{I} - \boldsymbol{K}_j\boldsymbol{H}_j. \tag{17.44}$$

两式代回式 (17.41)，得到迭代更新公式：

$$\boxed{\delta\boldsymbol{x}_j = \boldsymbol{K}_j\left(\boldsymbol{z} - \boldsymbol{h}(\boldsymbol{x}_j)\right) - \left(\boldsymbol{I} - \boldsymbol{K}_j\boldsymbol{H}_j\right)\left(\boldsymbol{x}_j \boxminus \bar{\boldsymbol{x}}\right), \qquad \boldsymbol{x}_{j+1} = \boldsymbol{x}_j \boxplus \delta\boldsymbol{x}_j,} \tag{17.45}$$

其中 $\boldsymbol{K}_j = \bar{\boldsymbol{P}}\boldsymbol{H}_j^\top(\boldsymbol{H}_j\bar{\boldsymbol{P}}\boldsymbol{H}_j^\top + \boldsymbol{R})^{-1}$ 每次都重新计算（因为 $\boldsymbol{H}_j$ 变了），而 $\bar{\boldsymbol{P}}$ 在整个迭代过程中保持不变。

式 (17.45) 有两项，物理含义清楚：第一项 $\boldsymbol{K}_j(\boldsymbol{z} - \boldsymbol{h}(\boldsymbol{x}_j))$ 是" 按当前残差做一次标准的 EKF 修正"；第二项 $(\boldsymbol{I} - \boldsymbol{K}_j\boldsymbol{H}_j)(\boldsymbol{x}_j \boxminus \bar{\boldsymbol{x}})$ 是" 把当前迭代点相对先验的偏移扣掉"——因为 $\boldsymbol{K}_j$ 的设计是让更新" 从 $\bar{\boldsymbol{x}}$ 出发"，而我们现在站在 $\boldsymbol{x}_j$ 上，必须先把这段位移还回去，否则会重复计算先验信息。初值取 $\boldsymbol{x}_0 = \bar{\boldsymbol{x}}$ 时第二项为零，第一次迭代就退化成标准 EKF 更新；随着迭代进行，$\boldsymbol{x}_j \boxminus \bar{\boldsymbol{x}}$ 不再为零，第二项开始起作用，保证每次迭代都在解同一个 MAP 问题而不是越走越远。

17.5.3 收敛判据与实现细节

迭代终止条件通常取下面三条之一（或组合）：

1. 更新量足够小：$\|\delta\boldsymbol{x}_j\| < \varepsilon_x$（典型 $\varepsilon_x = 10^{-3}$，各分量单位不同，实现中常按块分别设阈值）；
2. 达到最大迭代次数 $j_{\max}$（典型取 4，也有取 3 ~ 5 的实现）；
3. 残差平方和不再明显下降。

FAST-LIO2 的工程实现主要用第 2 条：固定迭代若干次，因为残差与协方差都在实时变化，动态判据的收益不大。理论上，当 $\boldsymbol{H}_j$ 变化很小时，迭代式 (17.45) 会以二次速率收敛到 MAP 解——这是高斯-牛顿法的标准结论；实际中限制收敛速度的是" 最近邻对应关系" 的离散跳变：某次迭代中某个点换了平面，$\boldsymbol{h}$ 就会跳一下。

更新完成后协方差按 $\boldsymbol{P} = (\boldsymbol{I} - \boldsymbol{K}\boldsymbol{H})\bar{\boldsymbol{P}}$ 计算（$\boldsymbol{K}, \boldsymbol{H}$ 用最后一次迭代的值），或者用数值更稳的 Joseph 形式

$$\boldsymbol{P} = (\boldsymbol{I} - \boldsymbol{K}\boldsymbol{H})\bar{\boldsymbol{P}}(\boldsymbol{I} - \boldsymbol{K}\boldsymbol{H})^\top + \boldsymbol{K}\boldsymbol{R}\boldsymbol{K}^\top, \tag{17.46}$$

它在有限精度下能保证 $\boldsymbol{P}$ 对称正定。Point-LIO 因为每次只做一维观测更新，特别依赖 Joseph 形式来避免协方差失去正定性。

算法：FAST-LIO2 主循环（简化）

1. 初始化：读第一帧点云建立初始地图；用静止段估计初始重力方向与零偏
2. 收到 IMU 测量 $(\tilde{\boldsymbol{\omega}}_k, \tilde{\boldsymbol{a}}_k)$：按式 (17.14) 传播状态，按式 (17.34) 传播协方差；发布 odom
3. 收到一帧点云：用传播轨迹把每个点去畸变到扫描末时刻，再按距离/体素降采样
4. 对每个点：在 iKD-Tree 中找最近 5 点，PCA 拟合平面得到 $(\boldsymbol{n}_i, \boldsymbol{q}_i)$，算残差 r_i 与雅可比 $\boldsymbol{H}_i$
5. 令 $\bar{\boldsymbol{x}} = \boldsymbol{x}$（传播结果），$\boldsymbol{x}_0 = \bar{\boldsymbol{x}}$，迭代 $j = 0, 1, \ldots, j_{\max}$：用式 (17.45) 计算 $\delta\boldsymbol{x}_j$ 并更新 $\boldsymbol{x}_{j+1}$
6. 用最后一次的 $\boldsymbol{H}$ 按式 (17.46) 更新协方差 $\boldsymbol{P}$
7. 把去畸变后的点用最终位姿变换到世界系，插入 iKD-Tree；用 box-wise 删除清掉远离当前位置的老点

17.6 点到面观测模型与雅可比

17.6.1 数据关联：从最近邻到平面

对去畸变后的第 i 个点，令它在雷达系中的坐标为 $\boldsymbol{p}_i^L$。先把它转到 IMU 系：

$$\boldsymbol{p}_i^I = \boldsymbol{R}_L\,\boldsymbol{p}_i^L + \boldsymbol{t}_L, \tag{17.47}$$

其中 $(\boldsymbol{R}_L, \boldsymbol{t}_L)$ 是 LiDAR 到 IMU 的外参。再转到世界系：

$$\boldsymbol{p}_i^W = \boldsymbol{R}\,\boldsymbol{p}_i^I + \boldsymbol{p}. \tag{17.48}$$

在 iKD-Tree 中取 $\boldsymbol{p}_i^W$ 的最近 5 个地图点 $\{\boldsymbol{q}_{i,1}, \ldots, \boldsymbol{q}_{i,5}\}$，计算它们的均值与协方差：

$$\bar{\boldsymbol{q}}_i = \frac{1}{5}\sum_{s=1}^{5}\boldsymbol{q}_{i,s}, \qquad \boldsymbol{\Sigma}_i = \frac{1}{5}\sum_{s=1}^{5}(\boldsymbol{q}_{i,s} - \bar{\boldsymbol{q}}_i)(\boldsymbol{q}_{i,s} - \bar{\boldsymbol{q}}_i)^\top. \tag{17.49}$$

对 $\boldsymbol{\Sigma}_i$ 做特征分解，最小特征值对应的特征向量取作平面法向 $\boldsymbol{n}_i$（$\|\boldsymbol{n}_i\| = 1$），平面过点 $\bar{\boldsymbol{q}}_i$。如果最小特征值不够小（点不共面，如树叶、边缘处），该点应被剔除——这就是最朴素的平面性检验。取 5 个点而不是 1 个，是

因为 5 个点刚好够定出一个平面（3 个自由度）并附带一个可判断" 平不平" 的协方差。

17.6.2 残差与雅可比

点到面残差定义为世界系中点到平面的有符号距离：

$$r_i = \boldsymbol{n}_i^\top \left(\boldsymbol{R}\, \boldsymbol{p}_i^I + \boldsymbol{p} - \bar{\boldsymbol{q}}_i \right) = \boldsymbol{n}_i^\top \left(\boldsymbol{p}_i^W - \bar{\boldsymbol{q}}_i \right). \tag{17.50}$$

式 (17.50) 的几何含义：$\boldsymbol{n}_i^\top(\cdot)$ 把三维误差投影到法向上，只保留" 垂直于平面" 的那一个分量；平面内的偏差被投影掉了，因为平面内往哪儿挪都还在同一个平面上，这个残差无法也不应该约束它。这也正是点到面残差只提供 1 维约束、m 个点最多提供 m 维约束的原因。

下面求偏导。用右扰动约定 $\boldsymbol{R} = \hat{\boldsymbol{R}} \exp([\delta\boldsymbol{\theta}]_\times)$（与 §17.4 一致），$\boldsymbol{p} = \hat{\boldsymbol{p}} + \delta\boldsymbol{p}$。把扰动形式代入式 (17.50)：

$$\begin{aligned} r_i(\delta\boldsymbol{x}) &= \boldsymbol{n}_i^\top \left(\hat{\boldsymbol{R}} \exp([\delta\boldsymbol{\theta}]_\times) \boldsymbol{p}_i^I + \hat{\boldsymbol{p}} + \delta\boldsymbol{p} - \bar{\boldsymbol{q}}_i \right) \\ &\approx \boldsymbol{n}_i^\top \left(\hat{\boldsymbol{R}} (\boldsymbol{I} + [\delta\boldsymbol{\theta}]_\times) \boldsymbol{p}_i^I + \hat{\boldsymbol{p}} + \delta\boldsymbol{p} - \bar{\boldsymbol{q}}_i \right) \\ &= r_i(\boldsymbol{0}) + \boldsymbol{n}_i^\top \hat{\boldsymbol{R}} [\delta\boldsymbol{\theta}]_\times \boldsymbol{p}_i^I + \boldsymbol{n}_i^\top \delta\boldsymbol{p}. \end{aligned} \tag{17.51}$$

再用 $[\delta\boldsymbol{\theta}]_\times \boldsymbol{p}_i^I = \delta\boldsymbol{\theta} \times \boldsymbol{p}_i^I = -[\boldsymbol{p}_i^I]_\times \delta\boldsymbol{\theta}$（这是" 叉乘的反对称矩阵表示"，见第 2 章）：

$$r_i(\delta\boldsymbol{x}) \approx r_i(\boldsymbol{0}) \underbrace{-\boldsymbol{n}_i^\top \hat{\boldsymbol{R}} [\boldsymbol{p}_i^I]_\times}_{\partial r_i / \partial \delta\boldsymbol{\theta}} \delta\boldsymbol{\theta} + \underbrace{\boldsymbol{n}_i^\top}_{\partial r_i / \partial \delta\boldsymbol{p}} \delta\boldsymbol{p}. \tag{17.52}$$

于是第 i 个点的雅可比是一个 1×18 的行向量：

$$\boxed{\boldsymbol{H}_i = \frac{\partial r_i}{\partial \delta\boldsymbol{x}} = \left[-\boldsymbol{n}_i^\top \hat{\boldsymbol{R}} [\boldsymbol{p}_i^I]_\times, \quad \boldsymbol{n}_i^\top, \quad \boldsymbol{0}_3^\top, \quad \boldsymbol{0}_3^\top, \quad \boldsymbol{0}_3^\top, \quad \boldsymbol{0}_3^\top \right]} , \tag{17.53}$$

按 $[\delta\boldsymbol{\theta}, \delta\boldsymbol{p}, \delta\boldsymbol{v}, \delta\boldsymbol{b}_g, \delta\boldsymbol{b}_a, \delta\boldsymbol{g}]$ 分块，非零的只有前两块。

逐块解释：$\partial r_i / \partial \delta\boldsymbol{\theta} = -\boldsymbol{n}_i^\top \hat{\boldsymbol{R}} [\boldsymbol{p}_i^I]_\times$ 表" 姿态转一点，点在世界上移动 $\boldsymbol{R}\delta\boldsymbol{\theta} \times \boldsymbol{p}_i^I$，再投影到法向"；$\partial r_i / \partial \delta\boldsymbol{p} = \boldsymbol{n}_i^\top$ 表" 位置挪一点，点在世界上跟着挪，投影到法向"；$\delta\boldsymbol{v}, \delta\boldsymbol{b}_g, \delta\boldsymbol{b}_a, \delta\boldsymbol{g}$ 的块为零，因为点的坐标变换直接看不出速度、零偏、重力——它们只能通过先验协方差间接被修正（见 §17.6.4）。

表 17.2: 点到面观测的雅可比分块与含义

分块	维度	表达式	含义
$\partial r_i / \partial \delta\boldsymbol{\theta}$	1×3	$-\boldsymbol{n}_i^\top \hat{\boldsymbol{R}} [\boldsymbol{p}_i^I]_\times$	姿态扰动使点位置绕机体系转动
$\partial r_i / \partial \delta\boldsymbol{p}$	1×3	$\boldsymbol{n}_i^\top$	平移沿法向的分量
$\partial r_i / \partial \delta\boldsymbol{v}$	1×3	$\boldsymbol{0}$	点坐标不依赖速度
$\partial r_i / \partial \delta\boldsymbol{b}_g$	1×3	$\boldsymbol{0}$	通过先验协方差间接作用
$\partial r_i / \partial \delta\boldsymbol{b}_a$	1×3	$\boldsymbol{0}$	通过先验协方差间接作用
$\partial r_i / \partial \delta\boldsymbol{g}$	1×3	$\boldsymbol{0}$	通过先验协方差间接作用
$\partial r_i / \partial \boldsymbol{R}_L$	1×3	$-\boldsymbol{n}_i^\top \hat{\boldsymbol{R}} \hat{\boldsymbol{R}}_L [\boldsymbol{p}_i^L]_\times$	仅在外参可估时存在
$\partial r_i / \partial \boldsymbol{t}_L$	1×3	$\boldsymbol{n}_i^\top \hat{\boldsymbol{R}}$	仅在外参可估时存在

注 [左扰动与右扰动] 如果改用左扰动 $\boldsymbol{R} = \exp([\delta\boldsymbol{\theta}]_\times) \hat{\boldsymbol{R}}$，同样的一阶展开给出 $r_i \approx r_i(\boldsymbol{0}) - \boldsymbol{n}_i^\top [\hat{\boldsymbol{R}} \boldsymbol{p}_i^I]_\times \delta\boldsymbol{\theta}$，即雅可比是 $-\boldsymbol{n}_i^\top [\boldsymbol{p}_i^W - \hat{\boldsymbol{p}}]_\times$。两种约定的差别只是把旋转误差表达在世界系还是机体系，数值结果完全等价（相差一个 $\boldsymbol{R}$ 的相似变换）。**关键是不能混用**：如果雅可比按右扰动推导，更新步就必须按右扰动施加（$\boldsymbol{R} \leftarrow \boldsymbol{R} \exp(\delta\boldsymbol{\theta})$），否则会引入一个随姿态变化的系统性错误。FAST-LIO2 的代码用右扰动，本书全程保持一致。

17.6.3 堆叠与退化分析

一帧里有 m 个有效点，把它们堆起来：

$$\boldsymbol{r}=\begin{bmatrix}r_1\\ \vdots\\ r_m\end{bmatrix}\in\mathbb{R}^m,\qquad \boldsymbol{H}=\begin{bmatrix}\boldsymbol{H}_1\\ \vdots\\ \boldsymbol{H}_m\end{bmatrix}\in\mathbb{R}^{m\times 18},\qquad \boldsymbol{R}_m=\mathrm{diag}(\sigma_1^2,\dots,\sigma_m^2). \tag{17.54}$$

观测方程是 $\mathbf{0}=\boldsymbol{r}(\boldsymbol{x})+\boldsymbol{v}$，即" 理想情况下残差应为零"，所以式 (17.45) 里的 $\boldsymbol{z}$ 取 $\mathbf{0}$、$\boldsymbol{h}(\boldsymbol{x}_j)$ 就是 $\boldsymbol{r}(\boldsymbol{x}_j)$——这是把" 配准" 写进滤波器的标准技巧：残差的期望是零，方差是点的测量噪声。

只看平移部分，$\boldsymbol{H}^\top\boldsymbol{H}$ 的平移块是 $\sum_i \boldsymbol{n}_i\boldsymbol{n}_i^\top$。如果所有点都在同一面墙上，所有 $\boldsymbol{n}_i$ 相同，这个矩阵的秩是 1：**沿墙面方向的两个平移没有被任何残差约束**，这就是几何退化。如果场景有两面互相垂直的墙，秩变成 2；三面互相垂直的墙才满秩。这个观察直接给出退化检测方法：对 $\sum_i \boldsymbol{n}_i\boldsymbol{n}_i^\top$ 做特征分解，若最小特征值小于阈值（或小于最大特征值的某个比例），说明机器人在该特征向量方向上" 看不见" 约束，此时应当依赖 IMU 先验，或者降低该方向的更新增益。

为什么用点到面而不是点到点？点到点残差是 $\boldsymbol{p}_i^W-\boldsymbol{q}_i$ 的三维向量，表面上看提供了 3 维约束，但地图点本身有采样噪声，点到点残差会把" 地图点采在哪" 的误差当成位姿误差，导致收敛慢、精度低（经典 ICP 的问题，见第 7 章）。点到面残差把沿平面方向的噪声投影掉，只保留真正有信息的法向分量，所以同样的点能给出更准的位姿。代价是：法向估计本身有误差，平面拟合不可靠的点必须剔除，否则会引入系统偏差。

17.6.4 零偏与重力如何被间接修正

表 17.2 里 $\delta\boldsymbol{v},\delta\boldsymbol{b}_g,\delta\boldsymbol{b}_a,\delta\boldsymbol{g}$ 的分块全为零，很多读者由此误以为" 激光修正不了零偏"。修正其实走的是先验协方差这条路：把 $\boldsymbol{H}$ 分成位姿块 $\boldsymbol{H}_p$ 与零块，则 $\boldsymbol{K}$ 的零偏部分为

$$\boldsymbol{K}_b=\bar{\boldsymbol{P}}_{bp}\boldsymbol{H}_p^\top\left(\boldsymbol{H}_p\bar{\boldsymbol{P}}_p\boldsymbol{H}_p^\top+\boldsymbol{R}\right)^{-1}, \tag{17.55}$$

其中 $\bar{\boldsymbol{P}}_{bp}$ 是零偏与位姿的互协方差。只要 $\bar{\boldsymbol{P}}_{bp}\neq\mathbf{0}$，零偏就会被激光残差拉动。用一维例子验证：设 $\theta_{k+1}=\theta_k-b_k\Delta t$、$b_{k+1}=b_k$，则 $\boldsymbol{F}_k=\begin{bmatrix}1&-\Delta t\\0&1\end{bmatrix}$，传播一步后 $\bar{P}_{\theta b}=P_{\theta b}-\Delta t P_{bb}$——只要 $P_{bb}>0$ 就非零。此时观测到角度（$\boldsymbol{h}=[1,0]$）给出 $K_b=\bar{P}_{\theta b}/(\bar{P}_{\theta\theta}+\sigma^2)\neq 0$，零偏被修正，尽管观测对零偏的雅可比是零。

这就是" 紧耦合" 在数学上的真正含义：**两个传感器共享同一个状态和同一个协方差矩阵，一个传感器的观测可以通过相关性修正另一个传感器的状态**。$\boldsymbol{F}$ 的非对角块在传播中建立起来的 $P_{\theta b_g},P_{vb_a},P_{vg}$ 就是这条通道；没有传播，就没有相关性，也就没有紧耦合。

17.7 iKD-Tree：增量式地图

17.7.1 为什么不能每帧重建

FAST-LIO2 每帧要做的事是：对每个去畸变后的点，在地图里找最近的 5 个点。地图点数 n 随运行时间增长，典型值 $10^5\sim10^6$。若每帧都把地图点重新建一棵 KD 树，建树复杂度 $O(n\log n)$；$n=10^6$、10 Hz 时每帧约需 10^8 次比较，不可能实时。而实际上每帧只新增几千到几万个点、只删掉远处的一小部分——为一小部分变化付出重建整棵树的代价显然浪费。iKD-Tree（incremental k-d tree）的思路就是：**把" 建树" 拆成" 插入、删除、局部重建" 三种便宜的操作**，让树一帧一帧地长出来。

17.7.2 节点里存什么

iKD-Tree 的每个节点存五类信息：**一个地图点**；**分割轴与分割值**（用哪一维 $x/y/z$ 把子树分成两半）；**轴对齐包围盒**（AABB，即子树所有点的最小/最大坐标）；**子树规模**（有效点数与总点数，供平衡与重建判断）；**删**

除标记。AABB 是最近邻搜索能剪枝的关键：若” 查询点到某子树 AABB 的最近距离” 已大于当前找到的第 k 近邻距离，整棵子树都不用看；没有 AABB，KD 树就退化成链表。

17.7.3 增量插入

插入一个点 $\boldsymbol{q}$ 的过程就是从根往下走：遇到包围盒为空的节点就新建节点存放 $\boldsymbol{q}$；否则按该节点的分割轴取出 $\boldsymbol{q}$ 的对应坐标，与分割值比较决定去左还是右子树，递归直到插入完成，再沿路径回溯更新每个祖先的 AABB 与计数。插入后做平衡检查：若某节点左右子树规模之比超过 α（典型 $\alpha = 0.7$），说明树开始” 歪”，把该子树的所有有效点收集起来重建为一棵平衡树。这个条件与” 替罪羊树”（scapegoat tree）的思想一致：大部分插入不破坏平衡，偶尔一次重建把子树摊平，均摊到每次插入仍是 $O(\log n)$。若被删除点占该子树的比例超过阈值（典型 30%），同样触发重建，把删除标记真正” 兑现” 掉——否则树里全是” 幽灵节点”，搜索时白白遍历。

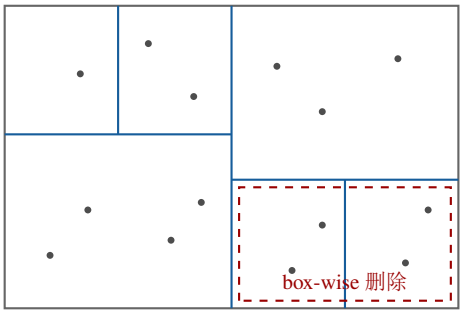
17.7.4 惰性删除与 box-wise 操作

删除一个点在 KD 树里是很麻烦的：不能直接摘掉节点（会破坏树的结构），重建整棵树又太贵。iKD-Tree 用**惰性删除**（lazy delete）：只把节点标记为” 已删除”，搜索时跳过它，树的形状不变。这样做的问题是被删节点还占着内存、还参与遍历；解决办法是在满足条件时重建（见上一段）。

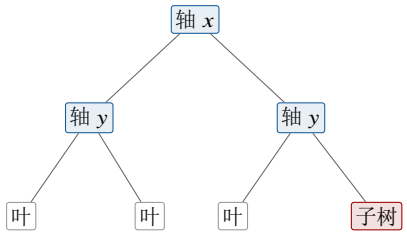
更妙的是 **box-wise 操作**。LIO 运行时经常需要” 删掉离当前位置超过 R 的所有点” 或” 把某个立方体里的点全部插入”。这类操作对应一个轴对齐盒子，iKD-Tree 可以直接利用 AABB 判断：

- **box-wise 删除**：从根开始，若某节点的 AABB 完全落在删除盒内，直接把这棵子树整体标记删除（$O(1)$），不再往下递归；若不相交，直接返回；只有部分相交时才递归到子节点。
- **box-wise 插入**：类似地，若插入盒完全包含某子树，可以把新点集整体挂成一个子树，而不是逐个插入。

一次 box-wise 删除的实际代价与” 落在盒边界上的节点数” 成正比，而不是与盒内点数成正比——这正是 LIO 里” 滑窗式局部地图” 能做到实时的原因。



(a) 平面划分与删除盒



(b) 对应的二叉树：红框子树整体标记删除

图 17.2: iKD-Tree 的空间划分与树结构。左图是一条条按轴切分的划分线，红框表示一次” 删除这个盒子里的点” 的请求；右图中红框对应的整棵子树只需在根上打一个删除标记，搜索时遇到它也只需检查一次标记，不用逐点遍历。

17.7.5 复杂度与内存

把每次操作摊开算：普通 KD 树每帧重建要 $O(n \log n)$，而 iKD-Tree 只在初始化时建一次树，之后每次插入均摊 $O(\log n)$、单点删除 $O(1)$（打标记）、box-wise 删除约 O(边界节点数)、k 近邻查询 $O(\log n)$（含剪枝）；最坏情况由 α 平衡保证不退化。这正是把” 每帧 $O(m \log n)$ 的查询 + $O(n \log n)$ 的重建” 变成” 只有查询和增量插入” 的关键。

” 地图只增不减” 会带来内存问题：10^5 点/帧、10 Hz，一分钟就是 6×10^7 个点。FAST-LIO2 的对策是**局部地图**：只维护以机器人当前位置为中心、边长 `cube_side_length`（典型 100 ~ 200 m）的立方体内的点，超出范围的用 box-wise 删除清掉；此外当前帧按 `filter_size_surf` 体素降采样（典型 0.5 m），插入地图前再按

`filter_size_map` 降采样（典型 0.5 m）。这样地图点数被限制在” 场景表面积 / 体素面积” 的量级，跑几小时也不会爆内存。并行方面，建树与查询常用 OpenMP 在左右子树上分任务，实际加速比受内存带宽限制。

17.8 Point-LIO：逐点更新

17.8.1 FAST-LIO2 留下的两个” 帧” 假设

FAST-LIO2 虽然把状态更新写得很漂亮，但它有两个隐含假设：其一，**帧内 IMU 可信**——100 ms 内的状态全靠 IMU 积分外推，直到帧末才被激光修正一次，若这 100 ms 里陀螺饱和（读数被截断在量程上限）或遭遇撞击（加速度计超量程），外推就会跑偏，帧末的数据关联（找最近平面）也会找错，滤波器可能被错误观测带得更偏；其二，**更新是帧同步的**——两次更新之间的 100 ms 内位姿本质上在” 盲推”，高速旋转时 100 ms 就是几十度。直观地说：机器人以 2000°/s 旋转（很多 MEMS 陀螺的量程上限）时，100 ms 就是 200°——半圈，这段时间里 IMU 一旦饱和，谁也不知道它转了多少。

17.8.2 核心思想：点一到就更新

Point-LIO [18] 的做法非常直接：**每收到一个激光点，就做一次状态更新**。激光点的到达间隔是微秒量级（10 Hz、每帧 10^4 点时约 10 μs），于是状态更新的频率从 10 Hz 变成 $10^4 \sim 10^5$ Hz。状态不再是” 每帧修正一次”，而是” 被每个点持续地拽着走”。

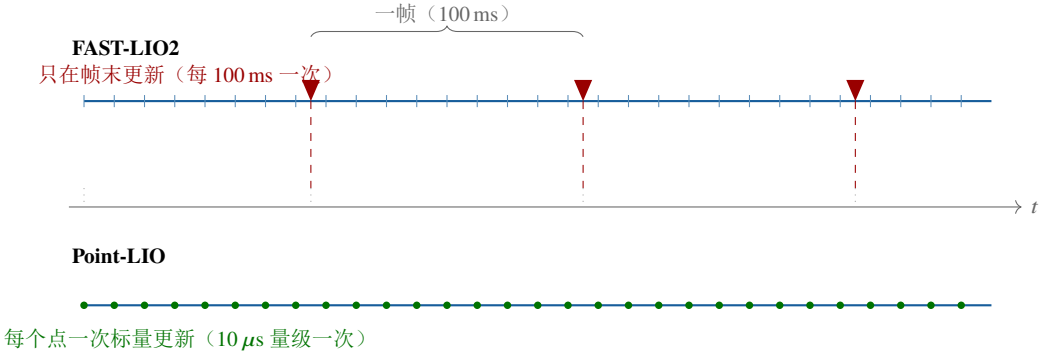


图 17.3: 两种更新粒度的时序对比。上：FAST-LIO2 在每帧末尾做一次 m 维（多残差）更新，两次更新之间状态只靠 IMU 外推；下：Point-LIO 对每个点做一次一维（单残差）更新，状态几乎连续地被观测修正。

17.8.3 逐点传播与标量更新

设在 t_{i-1} 时刻状态为 $\boldsymbol{x}_{i-1}$、协方差为 $\boldsymbol{P}_{i-1}$，第 i 个点的时间戳为 t_i。一次逐点处理的完整流程是：

（1）传播。用 $[t_{i-1}, t_i]$ 之间的 IMU 数据把状态推到 t_i：

$$\bar{\boldsymbol{x}}_i = \boldsymbol{f}\big(\boldsymbol{x}_{i-1}, \mathcal{U}_{[t_{i-1},t_i]}\big), \qquad \bar{\boldsymbol{P}}_i = \boldsymbol{F}_i \boldsymbol{P}_{i-1} \boldsymbol{F}_i^{\top} + \boldsymbol{Q}_i, \tag{17.56}$$

当 $t_i - t_{i-1}$ 只有几十微秒时，通常这段区间内没有 IMU 采样，直接用最近一次的 IMU 测量外推即可（$\boldsymbol{F}_i \approx \boldsymbol{I} + \boldsymbol{F}\Delta t_i$）。

（2）关联。用 $\bar{\boldsymbol{x}}_i$ 把点变换到世界系，在 iKD-Tree 中取最近 5 点、拟合平面，得到 $(\boldsymbol{n}_i, \bar{\boldsymbol{q}}_i)$ 与残差

$$r_i = \boldsymbol{n}_i^{\top}\big(\boldsymbol{R}_i \boldsymbol{p}_i^{I} + \boldsymbol{p}_i - \bar{\boldsymbol{q}}_i\big), \tag{17.57}$$

以及雅可比 $\boldsymbol{h}_i = \partial r_i / \partial \delta\boldsymbol{x} \in \mathbb{R}^{1\times 18}$（形式与式 (17.53) 完全相同）。

（3）更新。这是一次**一维观测**的卡尔曼更新：

$$\boldsymbol{K}_i = \frac{\bar{\boldsymbol{P}}_i \boldsymbol{h}_i^{\top}}{\boldsymbol{h}_i \bar{\boldsymbol{P}}_i \boldsymbol{h}_i^{\top} + \sigma_i^2}, \qquad \delta\boldsymbol{x}_i = \boldsymbol{K}_i\, r_i, \qquad \boldsymbol{x}_i = \bar{\boldsymbol{x}}_i \boxplus \delta\boldsymbol{x}_i, \tag{17.58}$$

$$\boldsymbol{P}_i = (\boldsymbol{I} - \boldsymbol{K}_i \boldsymbol{h}_i)\bar{\boldsymbol{P}}_i (\boldsymbol{I} - \boldsymbol{K}_i \boldsymbol{h}_i)^{\top} + \boldsymbol{K}_i \sigma_i^2 \boldsymbol{K}_i^{\top}. \tag{17.59}$$

式 (17.58) 的推导是标准高斯条件化：$\boldsymbol{K}_i$ 就是" 状态与观测的互协方差除以观测的总方差"，分母是个**标量**，所以这里没有矩阵求逆，只有一次除法。式 (17.59) 用 Joseph 形式，因为每次更新的步长很小、$\boldsymbol{P}$ 的更新次数极多（每秒上万次），必须保证数值上始终对称正定。每更新一次就立刻发布当前状态，于是里程计输出频率可以做到与点频同量级（论文报告可达 1 kHz 以上）。

17.8.4 顺序更新与批量更新等价吗

一个自然的问题：把一帧的 m 个点拆成 m 次一维更新，和 FAST-LIO2 一次 m 维更新，结果一样吗？在**线性高斯**情形下答案是一样：批量更新的后验信息矩阵 $\boldsymbol{\Lambda} = \bar{\boldsymbol{P}}^{-1} + \sum_i \boldsymbol{h}_i^{\top}\boldsymbol{h}_i/\sigma_i^2$ 只是把各点贡献相加，与顺序无关；顺序更新每做一次相当于用 Sherman–Morrison 公式在信息矩阵上加同一项，做完 m 次得到同一个和。所以在**线性**情形下二者的后验均值与协方差完全相同，区别只在计算路径。

在**非线性**情形下两者不再严格相等：Point-LIO 在每个点处重新线性化、重新关联，相当于把 IEKF 的迭代分散到时间上，对剧烈运动更友好；但单点的噪声、外点会直接污染状态，必须配合门限。计算上批量更新每帧 $O(m\cdot 18 + 18^3)$，顺序更新是 m 次 $O(18^2)$，后者其实更贵——逐点更新换来的不是算力节省，而是**更细的时间分辨率和更强的鲁棒性**。

17.8.5 IMU 饱和时：把 IMU 从" 传播输入" 降级为" 大噪声"

这是 Point-LIO 最关键的设计，也是它和 FAST-LIO2 最大的区别。

正常工作时，IMU 读数是" 传播的输入"：滤波器相信它，用它积分出两个点之间的状态变化。但陀螺或加速度计一旦饱和，读数被截断，继续信它就会把错误积分进去。Point-LIO 的处理思路是：**检测到饱和/不可信时，放弃" 用 IMU 积分" 这件事，改成随机过程模型**——也就是把状态传播写成

$$\bar{\boldsymbol{x}}_i = \boldsymbol{x}_{i-1} + \boldsymbol{w}_i, \qquad \boldsymbol{w}_i \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{Q}_i^{\mathrm{sat}}), \tag{17.60}$$

即" 两个点之间的状态怎么变，我不知道，用一个很大的过程噪声 $\boldsymbol{Q}_i^{\mathrm{sat}}$ 来表示这种无知"。物理上这相当于假设姿态、速度在两个点之间做随机游走，而不是按某个确定的动力学演化。$\boldsymbol{Q}_i^{\mathrm{sat}}$ 的量级由饱和程度决定：读数越接近量程、越不可信，$\boldsymbol{Q}$ 就放得越大（论文中的做法是按饱和检测结果自适应放大；不同版本实现细节略有差异，但思想都是" 越不可信，越不依赖它"）。

为什么这样做能行？因为逐点更新的频率极高：即使" 完全不知道" 两个点之间怎么动的，每过 $10\,\mu$s 就有一个激光点来告诉滤波器" 你现在应该在这个平面上"。用一维卡尔曼滤波的稳态解可以估出误差上界。标量情形下，每步过程噪声方差 q、观测噪声方差 σ^2，协方差递推为 $\bar{P} = P + q$、$P' = (P+q)\sigma^2/(P+q+\sigma^2)$；令 $P' = P$ 得二次方程 $P^2 + Pq - q\sigma^2 = 0$，正根为

$$P_\infty = \frac{-q + \sqrt{q^2 + 4q\sigma^2}}{2}. \tag{17.61}$$

代入 $q = 10^{-4}\,\mathrm{m}^2$（每步" 完全不知道" 的极端情况）、$\sigma^2 = 10^{-4}\,\mathrm{m}^2$（单点平面观测标准差 1 cm），得 $P_\infty \approx 6.2\times10^{-5}\,\mathrm{m}^2$，即 $\sqrt{P_\infty} \approx 7.9\,\mathrm{mm}$。也就是说：**即使动力学模型完全失效、每步注入 1 cm 量级的不确定性，只要每个点都提供一次 1 cm 精度的观测，稳态误差仍被压在 8 mm 左右**——这就是" 高频更新" 换来的鲁棒性。代价是：如果点云也退化（长走廊等），逐点更新同样失效，只能靠其他传感器或运动约束。

17.8.6 代价与收益

表 17.3 再补充两点工程经验。第一，Point-LIO 的" 逐点" 依赖激光点自带精确时间戳；机械式多线激光的每个点时间戳由方位角推算，扫描一圈内的时间精度是微秒级的，可以用；但某些雷达只给整帧时间戳，此时要先用帧内运动模型给每个点分配时间，效果会打折。第二，逐点更新对外点极其敏感：一个点如果落在了错误的平面（比如打到移动的人身上），它会把状态直接拽偏。实践中必须做残差门限（例如把 $|r_i|/\sigma$ 超过若干倍的点丢弃）或者用鲁棒核函数降权。

表 17.3: FAST-LIO2 与 Point-LIO 的对比

维度	FAST-LIO2	Point-LIO
更新粒度	每帧一次（m 维）	每个点一次（1 维）
更新频率	10 Hz	$10^4 \sim 10^5$ Hz（点频）
odom 输出	IMU 频率（100 ~ 200 Hz）	点频（可达 1 kHz 以上）
单次更新代价	$O(m \cdot 18 + 18^3)$	$O(18^2)$
单帧总代价	较低	较高（每点一次矩阵更新）
帧内 IMU 依赖	强（100 ms 全靠它外推）	弱（每个点都在纠偏）
IMU 饱和	可能发散	降级为随机过程模型，通常仍稳定
外点敏感性	一帧内平均，较鲁棒	单点直接进状态，需门限剔除
实现复杂度	中	高（时间戳粒度、饱和检测、门限）
适用场景	通用、高动态但 IMU 够用	超高速旋转、IMU 量程不足、需要极高频输出

17.9 VoxelMap 与地图表示对比

iKD-Tree 把地图存成" 点"，每次查询做 k 近邻搜索再拟合平面。VoxelMap [43] 换了一个思路：**把地图直接存成平面**。

17.9.1 自适应体素与平面模型

VoxelMap 把空间划分成体素（初始边长如 1 m），对每个体素内的点做 PCA：算出均值 $\bar{\boldsymbol{q}}$ 与协方差矩阵 $\boldsymbol{\Sigma}$，其特征值为 $\lambda_1 \geq \lambda_2 \geq \lambda_3$、对应特征向量 $\boldsymbol{u}_1, \boldsymbol{u}_2, \boldsymbol{u}_3$。若最小特征值 λ_3 足够小（相对 λ_1，或小于一个绝对阈值），说明这些点近似共面，就把该体素建模为一个平面 $\pi = (\boldsymbol{n}, d)$，其中 $\boldsymbol{n} = \boldsymbol{u}_3$（最小特征值对应的法向）、$d = \boldsymbol{n}^\top \bar{\boldsymbol{q}}$；否则把体素八等分，递归处理八个子体素，直到每个体素" 足够平" 或者小到不再细分。这就是**自适应体素**：平面区域用大块，复杂区域用小块。

每个平面不只存参数，还存它的不确定性：平面拟合的协方差可由点分布协方差近似（$\boldsymbol{n}$ 的不确定性大致与 λ_3/λ_2 相关），也可用递归最小二乘在信息形式下增量更新。这样平面参数随观测增多越来越准，残差的噪声方差可直接取平面参数的方差——比 iKD-Tree 里" 所有点用同一个常数 σ" 更精细。

表 17.4: iKD-Tree 与 VoxelMap 的地图表示对比

维度	iKD-Tree（点地图）	VoxelMap（平面地图）
存储单元	点	体素/平面（含协方差）
查询	k 近邻 + 现场 PCA，$O(\log n)$	哈希到体素，直接取平面，近似 $O(1)$
平面拟合	每次查询现算	增量维护，可复用
精度	高（点云原始信息全保留）	平面区域高，复杂区域退化
内存	与点数成正比，需滑窗删除	与体素数成正比，可预测
退化场景	依赖法向估计	平面本身即法向，更稳
非结构化场景	好	大量细分，优势下降
实现复杂度	中（树操作）	中高（八叉树 + 概率更新）

选型上可以这样记：**场景以平面为主、内存紧张、想要更快**，选 VoxelMap 类；**场景复杂、点云信息宝贵、想要更通用**，选 iKD-Tree 类。二者的残差与雅可比形式完全一样（都是点到面），差别只在地图这一层。

17.10 视觉-激光-惯性：FAST-LIVO2 一句话

FAST-LIVO2 [45] 沿着同一个 IEKF 框架把相机接了进来：状态不变，观测从" 点到面残差" 扩充为" 点到面残差 + 光度误差"。光度误差为 $r_j^{\text{vis}} = \boldsymbol{I}_j(\boldsymbol{\pi}(\boldsymbol{R}, \boldsymbol{p}, \boldsymbol{\rho}_j)) - \bar{I}_j$，即把地图点按当前位姿投影到图像上取灰度、与参考帧同名点相减，$\boldsymbol{\pi}(\cdot)$ 是针孔投影。两类残差在同一个 $\boldsymbol{H}$ 里堆叠、用同一个 $\boldsymbol{P}$ 融合，视觉便在激光退化的方向上补充约束。适用条件是环境有纹理、光照稳定；纯黑、纯白、强反光的场地仍只能靠激光和 IMU。

17.11 论文公式与源码对照

17.11.1 对照表

表 17.5 把本章的公式与 FAST-LIO2 官方仓库（以及同源的 Point-LIO）中常见的文件/函数对应起来。不同版本的仓库可能有重命名，但" 状态结构体 → 传播函数 → 更新函数 → 残差函数" 这条主线是不变的。

表 17.5: 论文公式与源码对照（以官方仓库某一版本为例，函数名可能随版本微调）

概念/公式	本章公式	文件	典型函数/变量
状态定义	式 (17.1)	`use-ikfom.hpp`	`struct state_ikfom {pos, rot, vel, bg, ba, grav}`
流形加减	式 (17.4)	`use-ikfom.hpp`	`operator+`、`operator-`
IMU 传播	式 (17.14)	`IMU_Processing.hpp`	`ProcessModel()`
去畸变	见第 16 章	`IMU_Processing.hpp`	`UndistortPcl()`
协方差传播	式 (17.34)	`esekfom.hpp`（IKFoM）	`predict()`、`predict_dyn()`
IEKF 迭代更新	式 (17.45)	同上	`update_iterated_dyn_share_modified()`
点面残差与雅可比	式 (17.50),(17.53)	`laserMapping.cpp`	`h_share_model()`
主循环与回调	伪代码骨架	`laserMapping.cpp`	`main()`、`imu_cbk()`
地图查询/插入/删除	§17.7	`ikd_Tree.cpp`	`Nearest_Search()` 等三接口
逐点更新	式 (17.58)	Point-LIO 仓库	逐点更新函数（与 FAST-LIO 同源）

17.11.2 一段关键代码的骨架

下面这段代码是 `h_share_model()` 的骨架（为可读性做了简化，变量名沿用官方风格；真正的实现还要处理外参、盲区、点的时间戳和并行化）：

Listing 17.1: 点到面残差与雅可比（示意图，非逐字源码）

```
// schematic skeleton, names follow the official FAST-LIO2 layout
for (int i = 0; i < feats_down_size; ++i) {
    Vector3d p_imu = offset_R_L_I * p_lidar + offset_T_L_I;   // to IMU frame
    Vector3d p_world = rot * p_imu + pos;                     // to world frame
    ikdtree.Nearest_Search(p_world, NUM_MATCH_POINTS,
                           nearest_points, sq_dist);          // 5-NN
    if (nearest_points.size() < NUM_MATCH_POINTS) continue;
    plane = fit_plane(nearest_points);                        // PCA normal
    if (!plane.is_planar) continue;                           // reject
    residual[i] = plane.normal.dot(p_world - plane.centroid);
    H.block<1, 3>(i, 0) = -plane.normal.transpose() * rot * SKEW(p_imu);
    H.block<1, 3>(i, 3) =  plane.normal.transpose();
}
// H is stacked as (m x 23); only pose (and extrinsics) blocks are non-zero
```

这段代码与式 (17.53) 一一对应：`p_imu` 是式 (17.47) 的 $\boldsymbol{p}_i^I$，`p_world` 是式 (17.48) 的 $\boldsymbol{p}_i^W$，`residual[i]` 是式 (17.50) 的 r_i，`H.block(0)` 与 `H.block(3)` 分别是表 17.2 的前两块。注意 `SKEW(p_imu)` 就是 $[\boldsymbol{p}_i^I]_\times$，负号来自 $[\delta\boldsymbol{\theta}]_\times \boldsymbol{p} = -\boldsymbol{p} \times \delta\boldsymbol{\theta}$。读代码时按" 状态 → 传播 → 更新 → 残差 → 地图" 的顺序走，正好对应表 17.5 的行序。

17.12 工程实践：参数、发散与开销

最常调的几个参数：`filter_size_surf` 与 `filter_size_map`（当前帧/入图前的降采样体素，典型 0.5 m，调大则点少更快但精度降）、`cube_side_length`（局部地图边长，典型 100 ~ 200 m，调大保留远处点但内存涨）、`point_filter_num`（每 N 点取 1，典型 3）、`maximum_iter`（IEKF 迭代次数，典型 4，调大更准更慢）、`extrinsic_T/R`（标定值，差 1° 在 10 m 处偏 17 cm）。

常见发散原因（按现场出现频率排序）：**时间同步**——激光与 IMU 时间戳差 10 ms，2 m/s 下就是 2 cm 错位，这是” 代码没问题但一跑就飘” 的头号原因；**外参错误**——标定不准或把 LiDAR 系/IMU 系搞反，表现为” 静止正常、一转就飘”；**IMU 饱和**——撞击瞬间角速度超量程，积分立刻跑偏；**初始化时机器人在动**——首帧需要近似静止来对齐重力、估零偏；**场景退化**——长走廊、纯平面、玻璃幕墙，激光约束不足；**点太少或太远**；**迭代次数不够**——被撞后初值差，4 次迭代可能没收敛，表现为一两帧内位姿跳变。

开销量级与调试（x86 单核、10 Hz、每帧 10^5 原始点降采样到 $5\times10^3 \sim 2\times10^4$）：$k$ 近邻查询约占一半时间、残差与矩阵运算占另一半；整机 CPU 占用通常在单核 20% ~ 40%，内存随局部地图规模在百 MB 量级。调参时先看 odom 是否连续（有无跳变），再看残差均值是否稳定在厘米级、协方差 trace 是否收敛；把 iKD-Tree 点数打出来看是否持续增长（增长说明滑窗删除没生效）；最后用 rosbag 回放对照真值。第 27 章给出更系统的排错流程。

本章小结

1. FAST-LIO2 的骨架是”IMU 高频预测 + 激光帧更新”：预测用式 (17.14)，协方差用式 (17.34)，更新用式 (17.45)；状态 $\boldsymbol{x} = [\boldsymbol{R}, \boldsymbol{p}, \boldsymbol{v}, \boldsymbol{b}_g, \boldsymbol{b}_a, \boldsymbol{g}]$ 中姿态在 SO(3)、重力在 S^2 上，误差状态用 $\boxplus/\boxminus$ 在切空间运算（官方实现含外参，共 23 维）。
2. $\boldsymbol{F}$ 的非零块只有六个：$-[\hat{\boldsymbol{\omega}}]_\times$、$-\boldsymbol{I}$（零偏进姿态）、$\boldsymbol{I}$（速度进位置）、$-\hat{\boldsymbol{R}}[\tilde{\boldsymbol{a}} - \hat{\boldsymbol{b}}_a]_\times$、$-\hat{\boldsymbol{R}}$（零偏进速度）、$\boldsymbol{I}$（重力进速度）；位置块不注入噪声，位置误差靠速度误差积分增长。
3. IEKF 是” 对 MAP 问题做高斯-牛顿”：$\delta\boldsymbol{x}_j = \boldsymbol{K}_j(\boldsymbol{z} - \boldsymbol{h}(\boldsymbol{x}_j)) - (\boldsymbol{I} - \boldsymbol{K}_j\boldsymbol{H}_j)(\boldsymbol{x}_j \boxminus \bar{\boldsymbol{x}})$，每次迭代重新线性化、重新关联，而 $\bar{\boldsymbol{P}}$ 保持不变。
4. 点到面残差 $r_i = \boldsymbol{n}_i^\top(\boldsymbol{R}\boldsymbol{p}_i^I + \boldsymbol{p} - \bar{\boldsymbol{q}}_i)$ 对右扰动的雅可比只有两块：$-\boldsymbol{n}_i^\top\boldsymbol{R}[\boldsymbol{p}_i^I]_\times$ 与 $\boldsymbol{n}_i^\top$；零偏、重力、速度的块为零，它们靠先验互协方差被间接修正。
5. iKD-Tree 用” 增量插入 + 惰性删除 + box-wise 操作 + 局部重建” 把地图维护降到每帧 $O(m \log n)$；Point-LIO 把更新粒度从” 帧” 降到” 点”，每点一次标量卡尔曼更新，IMU 饱和时改用随机过程模型、放大 $\boldsymbol{Q}$，稳态误差由式 (17.61) 给出。

第十七章 练习

1. 从 $\dot{\boldsymbol{R}} = \boldsymbol{R}[\boldsymbol{\omega}]_\times$ 出发，推导式 (17.14) 中 $\boldsymbol{v}_{k+1}$ 的表达式，并写出被忽略的二阶项（提示：把 $\exp([\boldsymbol{\omega}s]_\times)$ 展开到 s^2 后积分）。
2. 若把重力从状态里去掉（当作常量 $\boldsymbol{g} = [0, 0, -9.81]^\top$），误差状态维数变成多少？初始姿态差 2° 会导致什么后果？请用式 (17.29) 解释。
3. 用左扰动 $\boldsymbol{R} = \exp([\delta\boldsymbol{\theta}]_\times)\hat{\boldsymbol{R}}$ 重新推导式 (17.53)，验证雅可比为 $-\boldsymbol{n}_i^\top[\boldsymbol{R}\boldsymbol{p}_i^I]_\times$，并说明它与右扰动版本的关系。
4. 证明线性高斯下顺序标量更新与批量更新的后验相同（提示：信息矩阵可加），再推导式 (17.61) 并计算 $\sigma = 0.02\,\text{m}$、$q = 10^{-3}\,\text{m}^2$ 时的稳态标准差。
5. 一帧点云只有一面墙：写出 $\boldsymbol{H}^\top\boldsymbol{H}$ 的平移块、秩与零空间方向，说明此时为何要降低更新增益；再解释 box-wise 删除为何是 O(边界节点数)。

延伸阅读

- FAST-LIO2 论文 [41] 与官方仓库：按表 17.5 的顺序读代码，核对论文附录 $\boldsymbol{F}$ 与式 (17.32)，前置读 ESKF 教程 [37]。
- Point-LIO [18]、VoxelMap [43]、FAST-LIVO2 [45]：逐点更新、体素地图、视觉残差三支。

第四部分

导航篇：从地图到运动

第十八章 导航系统总览与 Nav2

本章导读

从这一章开始，我们进入全书第四部分：导航。里程计篇回答“机器人现在在哪”，SLAM 篇回答“地图长什么样”，而导航篇要回答一个更接近任务的问题——“知道自己在哪、也知道地图之后，怎么把机器人安全、平稳地送到目标点”。本章先建立导航系统的全局骨架：导航由定位、建图、规划、控制四块组成；然后正面解释一个几乎每个新队员都会问、但网上说法互相矛盾的概念——“导航的前端/后端”；最后以 ROS 2 的 Nav2 为实例，讲清行为树、代价地图、TF 树、全局/局部规划的分工。学完本章，你应该能画出 Nav2 的节点数据流图，并说出每个节点负责哪一件事、以什么频率运行、失败时由谁兜底。

18.1 导航到底在做什么

用一句话概括：**导航是在已知或部分已知的地图上，依据自身的位姿估计，生成一条满足机器人运动约束、并且不撞障碍的可行轨迹，并把轨迹变成轮子/电机的指令。**把它写成优化问题时，输入是地图 M、当前位姿 $\boldsymbol{x}_0$、目标位姿 $\boldsymbol{x}_g$、机器人模型（尺寸、最大速度、最大加速度、最小转弯半径），输出是一段控制序列 $\boldsymbol{u}_{0:T}$ 或一段轨迹 $\boldsymbol{x}_{0:T}$，约束是*不碰撞、运动学可行、动力学可行*，目标通常是最短、最平滑或最省时间。

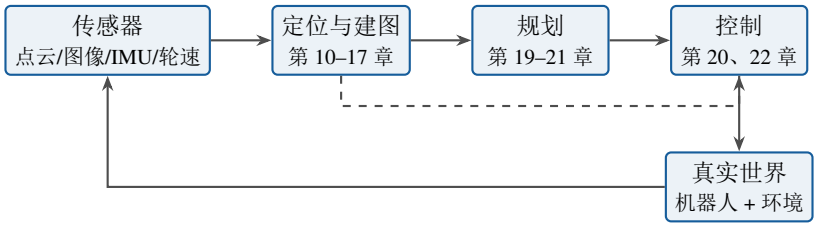


图 18.1: 导航系统的闭环。上半行是“信息流”：传感器数据经定位与建图变成位姿和地图，再经规划变成轨迹，经控制变成指令；下半行是“物理流”：指令驱动真实世界，真实世界的变化又被传感器观测到。虚线表示规划与控制都要使用定位结果（位姿）。

图 18.1 里有两点容易忽略，却决定了工程上的成败。第一，**闭环里每一环都有延迟**，位姿估计可能比真实状态晚几十毫秒，控制指令下发又要经过通信与执行器，所以“用当前位姿算出的避障动作”实际上是在处理几十毫秒前的世界；速度越快、延迟越大，越容易撞。第二，**每一环的频率差一到两个数量级**，定位 10–50 Hz，全局规划 1 Hz 量级，局部规划 10–20 Hz，电机控制 100 Hz 以上。频率不匹配就必须靠“缓存最近的位姿与地图”和“降频重规划”来衔接，这正是 Nav2 用生命周期节点、代价地图和速度平滑器要处理的事情。

表 18.1: 导航系统的四个环节及其在本书中的位置

环节	回答的问题	典型频率	对应章节
定位与建图	我在哪？地图什么样？	10–50 Hz	第 10–17 章
全局规划	大致该往哪走？	1 Hz	第 19、21 章
局部规划	接下来 1–3 秒怎么走？	10–20 Hz	第 20、21 章
控制与执行	左右轮各转多快？	50–200 Hz	第 20、22 章

18.2 “前端/后端”到底指什么：三种语境

这是本章最需要说清楚的一节。新队员常看到三句互相打架的话：“SLAM 的后端是图优化”“导航的后端是规划与控制”“定位系统的后端负责全局修正”。三句话都不算错，但它们在说三个不同的东西。之所以混乱，

是因为“前端/后端”原先是软件工程里的分层说法（**前端**负责与外界交互、做即时处理，**后端**负责汇总、优化、做全局决策），后来被三个不同领域各自借用，含义就分叉了。

18.2.1 语境一：SLAM 的前端/后端（第 11–13 章）

在 SLAM 里，分界线非常明确：**前端**是帧间/扫描间的数据关联与相对位姿估计（特征匹配、ICP、直接法），它只关心相邻几帧，结果快但有累积漂移；**后端**是把这些相对约束与回环约束放在一起做全局优化（位姿图、因子图、BA），它关心整段轨迹的一致性，慢但能把漂移“摊平”。判断标准是：*是否对全局变量做联合优化*。前端 10–30 Hz，后端 1 Hz 或只在回环时触发。第 11 章给出的定义是本书的基准，后面两节都要与它对照。

18.2.2 语境二：定位系统的前端/后端（第 9、10、23 章）

在定位（尤其是多传感器融合定位）里，**前端**指“用高频、局部的观测做状态递推与更新”，例如 IMU 预积分 + 激光/视觉里程计的 ESKF 滤波；**后端**指“用低频、全局的观测做长期修正”，例如 AMCL 粒子滤波重定位、UWB/二维码/UWB 基站给的绝对位置、回环检测给出的位姿修正。这里的分界不再是“有没有联合优化”，而是*观测是局部的还是全局的、修正的是短期还是长期漂移*。Z-Localization 那类系统把“前端滤波 + 后端全局约束检验”拆得更细，第 23 章会展开。

18.2.3 语境三：导航系统的前端/后端（本章、第 19–22 章）

在导航系统里，最常见的说法是：**前端 = 定位与地图**（回答“我在哪、哪里能走”），**后端 = 规划与控制**（回答“往哪走、怎么走”）。这个说法流行，是因为它对应了数据流的方向：感知与状态估计在前，决策与控制在后，和 Web 开发里“前端展示、后端计算”的直觉很像，讲课时容易理解。

但必须指出它**不严谨**的三点：

1. 导航的“后端”里并没有 SLAM 后端那种*全局联合优化*。规划是搜索/采样/优化一个轨迹，控制是求解一个跟踪问题，两者都不优化“整段运行历史”。把规划叫后端，只是借了“做决策”的意思。
2. 导航自己内部还有一层“全局/局部”的层次（全局规划 1 Hz、局部规划 10–20 Hz），这层“全局/局部”与“前端/后端”*不是一回事*。很多混淆就来自把“全局规划”误说成“导航后端”。
3. 被划进“导航前端”的定位模块内部，*自己又有前端/后端*（语境二）。所以“导航的前端”内部还套着一个“定位的后端”，逐层嵌套时字面意思会打架。

表 18.2: 三种“前端/后端”语境对照（建议背下这张表）

语境	前端	后端	分界依据
SLAM	帧间配准、数据关联	位姿图/因子图全局优化	是否联合优化全局变量
定位系统	高频滤波、局部观测递推	全局观测重定位、长期漂移修正	观测是局部还是全局
导航系统	定位与地图	规划与控制（口语说法）	数据流先后（非严格定义）

一句话判断法

听到“后端”时，先看它后面接什么：接“图优化/回环”→ SLAM 后端；接“全局定位/绝对观测修正”→ 定位后端；接“规划/控制/行为树”→ 导航后端（口语）。本书此后一律使用“SLAM 前后端”“定位前后端”这种带限定词的说法，避免歧义。

18.3 Nav2：ROS 2 的导航事实标准

Nav2 是 ROS 2 的导航框架，前身是 ROS 1 的 `move_base`，其设计在 [26] 中系统给出，整体软件架构与 ROS 2 的设计哲学见 [27]。它不是一个“程序”，而是一组**可组合、可替换、由行为树调度**的节点。

18.3.1 节点分工

表 18.3 是必须记住的节点清单。核心思路是：`bt_navigator` 不自己算路径，它只负责“按行为树的剧本调用别人”；真正算路的是 `planner_server`（全局）与 `controller_server`（局部）；`behavior_server` 提供“原地旋转、后退、等待”等恢复动作；`lifecycle_manager` 负责按依赖顺序把大家从“未配置”一路带到“激活”。

表 18.3: Nav2 主要节点及其职责

节点	职责	关键插件/接口
`bt_navigator`	行为树调度，对外提供导航 action	BehaviorTree.CPP
`planner_server`	全局路径规划	NavFn、Smac、Smac Hybrid
`controller_server`	局部轨迹跟踪与避障	DWB、TEB、MPPI、RPP
`smoother_server`	路径平滑（减少拐点）	Simple、Savitzky-Golay
`behavior_server`	恢复行为：旋转、后退、等待	spin、backup、wait
`waypoint_follower`	多点巡航	NavigateThroughPoses
`velocity_smoother`	速度限幅与加速度平滑	一阶低通
`map_server / amcl`	加载地图 / 粒子滤波定位	第 10、14 章
`lifecycle_manager`	生命周期编排	按序 configure/activate

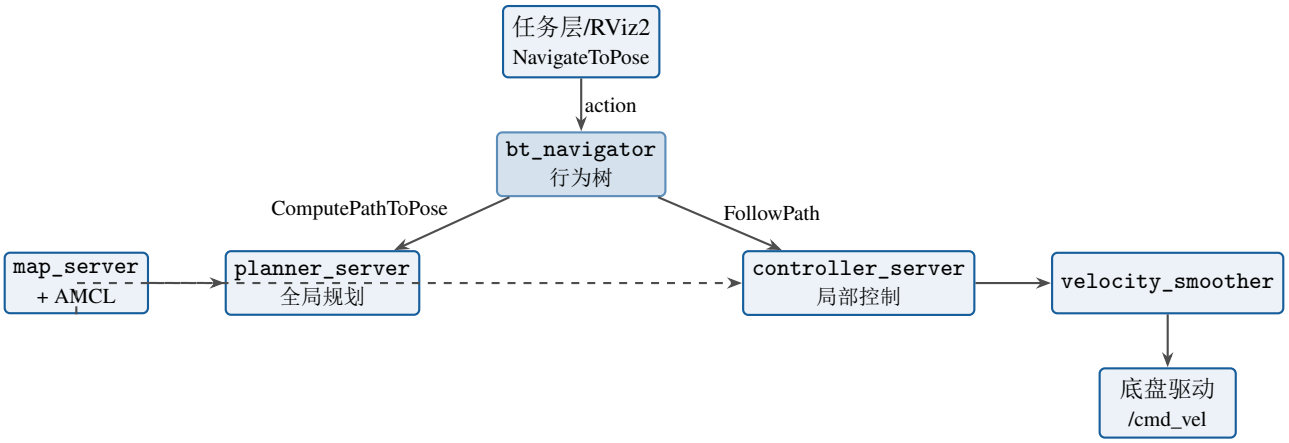


图 18.2: Nav2 的节点数据流（简化）。箭头方向是“谁调用谁/数据流向谁”。`bt_navigator` 通过 action 调用规划与控制服务；两者都以 `map_server` 的静态地图和 AMCL 的位姿为输入；`controller_server` 输出的速度指令经速度平滑器限幅后下发底盘。行为树的好处是：换一套 XML 脚本就能改变“先规划、失败则旋转、再失败则后退”的策略，而不用改 C++ 代码。

18.3.2 action 接口：为什么不是 topic

导航是“有开始、有结束、有反馈、可能失败、可取消”的长任务，正适合 ROS 2 的 **action**（动作）接口。ROS 2 的 action 由三部分组成：goal（目标）、feedback（周期反馈）、result（最终结果）；底层复用 topic 与 service，因此天然支持“发起后先返回、执行中持续反馈、随时取消”。Nav2 最常用的 action 是 `nav2_msgs/action/NavigateToPose`，可以这样从命令行发一个目标：

```
# Send a navigation goal from the command line (Nav2 must be running)
ros2 action send_goal /navigate_to_pose nav2_msgs/action/NavigateToPose \
  "{pose:␣{header:␣{frame_id:␣map},␣pose:␣{position:␣{x:␣1.0,␣y:␣2.0},
␣␣␣orientation:␣{w:␣1.0}}}}" --feedback
```

反馈里会周期性给出当前位姿、已走时间、剩余距离、已执行的恢复次数。工程上只要盯住“剩余距离长时间不下降 + 恢复次数持续增加”，基本就能判定机器人卡住了——这比看日志快得多。

18.3.3 与 ROS 1 move_base 的差异

表 18.4: move_base 与 Nav2 的对比

维度	ROS 1 move_base	Nav2
任务编排	固定流程 + 硬编码恢复行为	行为树 XML，可在线切换策略
生命周期	无统一管理，启动顺序靠脚本	生命周期节点 + lifecycle_manager
插件化	规划器/控制器插件，扩展有限	规划、控制、平滑、行为全部插件化
接口	MoveBaseAction	NavigateToPose 等一组 action
多机器人	需命名空间硬隔离	天然支持命名空间与多实例

18.4 代价地图：导航的“地面真相”

定义 18.1 (代价地图)

代价地图（costmap）是叠加在二维平面上的栅格，每个格子存一个 8 位整数 $c \in \{0, 1, \ldots, 255\}$，表示“机器人中心经过该格的风险”。约定 **0** 表示自由、**254** 表示致命障碍（LETHAL_OBSTACLE）、**253** 表示内切膨胀障碍（INSCRIBED_INFLATED_OBSTACLE，即“机器人中心到这里必然碰撞”）、**255** 表示未知（NO_INFORMATION）。中间值表示“离障碍越近代价越高”。

举个例子：10 m × 10 m 的场地、分辨率 0.05 m，则每边 10/0.05 = 200 格，共 4×10^4 格；每格 1 字节，单层约 39 KB，四层也不过 160 KB 左右，完全可以每帧全量更新。这就是为什么 2D 代价地图能在 5–20 Hz 下实时刷新。

18.4.1 分层结构

Nav2 的 2D 代价地图由若干 **layer**（层）叠加而成，自下而上依次是：

- **静态层**（static layer）：读 map_server 的 pgm 地图，给出已知障碍与未知区域，基本不变。
- **障碍层/体素层**（obstacle layer / voxel layer）：把实时传感器（激光、深度相机、3D 点云）观测到的障碍投影到 2D 栅格。体素层会把 3D 点云压成带高度过滤的 2D 障碍，是 3D 传感器进入 2D 导航的桥梁。
- **膨胀层**（inflation layer）：把障碍向外“扩散”出一圈代价梯度，让规划器主动远离墙壁。它不新增障碍，只改变代价值。

层的叠加规则是“取最大”：$c(x, y) = \max_k c_k(x, y)$。因此静态层里的致命障碍不会被障碍层覆盖成自由，而障碍层新看到的障碍会立即压过静态层的自由空间。

18.4.2 内切半径、外切半径与膨胀

设机器人俯视轮廓的最小外接圆半径为**外切半径** r_{out}，机器人能塞进的最窄通道半宽为**内切半径** r_{in}。对 0.6 m × 0.6 m 的方形底盘，$r_{\text{in}} = 0.3$ m，$r_{\text{out}} = \sqrt{0.3^2 + 0.3^2} = 0.424$ m。

膨胀层的代价值由下式给出：

$$c(d) = \begin{cases} 254, & d < r_{\text{in}}\text{（中心在障碍上）}, \\ 253, & r_{\text{in}} \le d \le r_{\text{in}} + \varepsilon, \\ \lfloor 253 \exp[-k\,(d - r_{\text{in}})] \rfloor, & r_{\text{in}} < d < r_{\text{infl}}, \\ 0, & d \ge r_{\text{infl}}, \end{cases} \tag{18.1}$$

其中 d 是该格到最近障碍的欧氏距离，r_{infl} 是 inflation_radius（膨胀半径），k 是 cost_scaling_factor（代价衰减系数）。d 用距离变换在整张图上一次算出，所以膨胀本身是 $O(N)$ 的。

取 $r_{\text{in}} = 0.3$、$k = 3.0$、$r_{\text{infl}} = 0.55$（单位 m），算三个数：

- $d = 0.35$：$c = 253e^{-3\times0.05} = 253 \times 0.861 = 217.8 \to 217$，很高，规划器会避开；
- $d = 0.45$：$c = 253e^{-3\times0.15} = 253 \times 0.638 = 161.4 \to 161$，仍然偏高；
- $d = 0.55$：$c = 253e^{-3\times0.25} = 253 \times 0.472 = 119.5 \to 119$，但按上界已截断为 0。

注意最后一行的截断：只要 $d \geq r_{\text{infl}}$ 就强制归零，否则指数衰减永远到不了 0，整张图都会“有代价”。

实例：机器人过窄门

门宽 0.9 m，方形底盘 0.6 m。机器人中心必须离两侧门框各 $\geq r_{\text{in}} = 0.3$ m，可行带宽度只剩 $0.9 - 2\times0.3 = 0.3$ m，按 0.05 m 分辨率约 6 格。若把 `inflation_radius` 设成 0.8、$k = 5$，门中心 $d = 0.45$ 处代价为 $253e^{-5\times0.15} = 253 \times 0.472 = 119$，两侧墙的膨胀区在门中央几乎重叠，规划器会认为“门口很贵”而绕路甚至报“无可行路径”。正确做法是：r_{in} 按真实车体设准（别虚大），`inflation_radius` 取略大于 r_{out}（如 0.45–0.5），k 取 2–3；或给规划器设置“允许穿越高代价区”的阈值。

18.4.3 更新频率与发布频率

`update_frequency` 是代价地图重新计算（融合新观测、重算膨胀）的频率，`publish_frequency` 是把它发布给其他节点/RViz 的频率。前者决定避障反应速度，后者只影响可视化与订阅端。常见配置是 update 5 Hz、publish 2 Hz；全局代价地图可以更低（1 Hz），因为它只服务 1 Hz 的全局规划。

18.5 TF 树与坐标系

定义 18.2 (TF 树)

TF 树是以坐标系为节点、以带时间戳的刚体变换为边的树。任意两个坐标系 A、C 之间的变换，通过沿树从 A 走到 C 的路径逐段复合得到：若路径是 $A \to B \to C$，则 $T_{AC} = T_{AB}\,T_{BC}$。TF 库（tf2）沿路径查询并插值到指定时刻，因此每个坐标系在任意时刻都只有一个父节点——这正是“树”而不是“图”的要求。♣

举个复合的例子。机体在世界系中位姿 T_{WB}：平移 $(1, 2)$ m、偏航 30°；激光雷达在机体系下 $(0.2, 0)$ m、无旋转。则

$$ {}^{W}\boldsymbol{p}_{L} = {}^{W}\boldsymbol{R}_{B}\,{}^{B}\boldsymbol{p}_{L} + {}^{W}\boldsymbol{p}_{B} = \begin{bmatrix} \cos 30^\circ & -\sin 30^\circ \\ \sin 30^\circ & \cos 30^\circ \end{bmatrix} \begin{bmatrix} 0.2 \\ 0 \end{bmatrix} + \begin{bmatrix} 1 \\ 2 \end{bmatrix} = \begin{bmatrix} 0.173 \\ 0.1 \end{bmatrix} + \begin{bmatrix} 1 \\ 2 \end{bmatrix} = \begin{bmatrix} 1.173 \\ 2.1 \end{bmatrix}. \tag{18.2} $$

也就是说，雷达看到的世界坐标要先减去这个偏移才能变回机体系——外参错了，障碍物在地图上的位置就会整体偏移，表现为“代价地图里的墙和真实墙错位”。

图 18.3 的树结构决定了三件事：

1. **谁发布**：`map→odom` 只能由定位模块（AMCL 或 SLAM）发布；`odom→base_footprint` 只能由里程计发布；传感器外参由 URDF 发布。重复发布会让 tf2 报“TF 树出现多个父节点”，整棵树失效。
2. **为什么不直接 `map→base_link`**：因为定位会跳变，而局部规划、控制需要平滑连续的位姿。把跳变隔离在 `map→odom` 这一段，局部控制始终用连续的 `odom` 系，就不会因为一次重定位而急打方向。
3. **时间戳**：tf2 默认只缓存 10 s，查询时刻超出缓存就抛 `ExtrapolationException`。常见原因是某个节点的时间戳用了系统时间而其他节点用 ROS 时间（仿真），或者传感器驱动停了。

```
# Check whether map->odom is actually published and how it evolves
ros2 run tf2_ros tf2_echo map odom
# Visualize the whole TF tree; look for disconnected branches
ros2 run tf2_tools view_frames
```

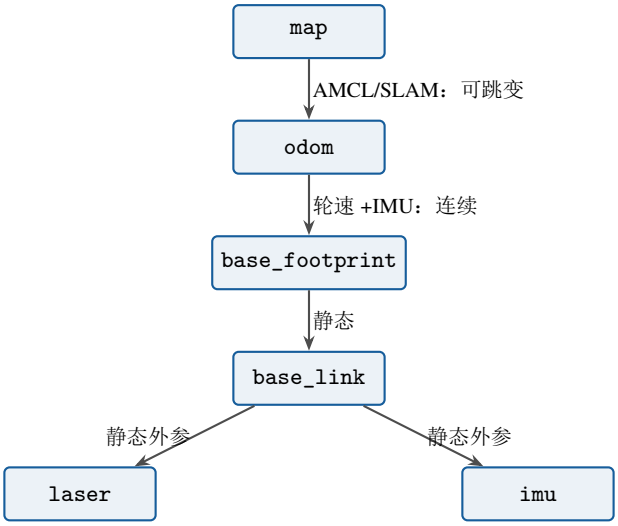


图 18.3: 2D 导航的标准 TF 树。map→odom 由定位模块发布，可以跳变（重定位时瞬间修正）；odom→base_footprint 由里程计发布，必须连续无跳变；base_link 以下的传感器外参是静态的，由 URDF 经 robot_state_publisher 广播。

18.6 全局规划与局部规划的分工

定义 18.3 (全局规划与局部规划)

全局规划（global planning）在静态或准静态的全局地图上，从起点到目标点求一条（近似）最优的几何路径，只考虑“能不能走”，不考虑“怎么拐”；局部规划（local planning，也叫局部轨迹规划/控制）在机器人附近几米的滚动窗口内，结合实时障碍与自身运动学、动力学约束，生成未来 1–3 秒的可执行轨迹并跟踪全局路径。

两者的输入输出完全不同：全局规划输入是起点、目标、全局代价地图，输出是一串路径点（path，只有几何没有时间）；局部规划输入是这条路径、局部代价地图、当前位姿与速度，输出是带时间信息的轨迹或直接的速度指令 (v, ω)。频率上差一个数量级，全局 1 Hz 足够（路径不会每秒都变），局部必须 10–20 Hz（障碍随时出现）。

评价一条全局路径看四点：**长度**（是否绕远）、**平滑度**（曲率是否连续，曲率突变会让底盘抖动）、**安全性**（离障碍的净空）、**可行性**（是否满足最小转弯半径）。评价局部规划则看：**跟踪误差**、**实时性**、**避障成功率**、**舒适性**（加速度、jerk）。第 19 章讲全局规划的算法，第 20 章讲局部规划与控制，第 21 章把两者搬到 3D 地形上，第 22 章把它们串成完整链路。

本章小结

1. 导航 = 定位/建图 + 全局规划 + 局部规划 + 控制，是一个带延迟、频率相差 1–2 个数量级的闭环；理解延迟与频率差是调参的前提。
2. “前端/后端”有三种语境：SLAM 看是否联合优化全局变量；定位系统看观测是局部还是全局；导航系统口语里指“定位与地图 / 规划与控制”，后者不严谨，本书一律加限定词。
3. Nav2 是行为树调度的插件化节点集合：bt_navigator 编排，planner_server 与 controller_server 干活，behavior_server 兜底，lifecycle_manager 负责启动顺序，对外用 action 接口。
4. 代价地图是 0–255 的分层栅格，取最大叠加；内切半径决定“必然碰撞区”，膨胀半径与衰减系数决定“多怕墙”。式 (18.1) 是调参的量化依据。
5. TF 树是一条链：map→odom 可跳变、odom→base 必须连续、传感器外参静态；重复发布与时间戳越界是最常见的两类故障。

6. 全局规划求几何路径（1 Hz，重长度/平滑/可行），局部规划求带时间的轨迹（10–20 Hz，重实时/避障）。

第十八章 练习

1. 用式 (18.1) 计算：$r_{\text{in}} = 0.25$、$k = 2.5$、$r_{\text{infl}} = 0.6$ 时，$d = 0.3, 0.4, 0.5, 0.6$ 四处的代价值，并解释为什么 $d = 0.6$ 必须归零。
2. 一辆 0.5 m × 0.4 m 的底盘要通过 0.75 m 宽的门。请给出内切半径 r_{in} 与外切半径 r_{out}，再给出一组可用的膨胀半径与代价衰减系数，并说明理由。
3. 如果 AMCL 和另一个建图节点同时发布 `map→odom`，会发生什么？如何用命令行确认是谁在发布？
4. 为什么全局规划不需要 10 Hz？如果强行提高到 10 Hz，会带来什么代价？
5. 用一句话分别回答“SLAM 的后端是什么”“定位系统的后端是什么”“导航的后端是什么”，并说明三者的分界依据。

延伸阅读

- [26]：Nav2 的原始设计论文，读它解决“为什么要有行为树、为什么拆成这么多 server”的问题。
- [27]：ROS 2 的设计与架构，理解 action、生命周期节点、组件化之后再读 Nav2 源码会轻松很多。
- [38]：第 1 章与第 5–6 章，概率机器人视角下“定位—规划—控制”的统一框架。
- Nav2 官方文档 docs.nav2.org：配置项、默认参数、行为树 XML 示例，工程落地时按图索骥。

第十九章 全局路径规划

本章导读

第 18 章把导航拆成“全局规划 + 局部规划”两层，本章专门讲全局规划：在一张已知的地图（栅格、路网、可通行区域）上，为机器人找一条从起点到目标点的路径。我们按“从严谨到实用”的顺序展开：先定义问题与评价标准，再讲 Dijkstra、A* 及其可采纳性证明，然后讲工业界最常用的三种加速/扩展——JPS（跳点搜索）、Hybrid A*（满足最小转弯半径）与 RRT*（高维采样），最后落到 Nav2 的规划器插件与路径平滑。学完本章，你应该能手推 A* 为什么最优、能根据底盘类型选规划器、并知道代价地图的代价值如何变成搜索的边权。

19.1 问题定义与评价标准

全局规划可以抽象成**图搜索**问题。把机器人可能处于的每个状态抽象成一个**节点**（node），把一次允许的转移抽象成一条**边**（edge），边上带非负代价 $c(n, n') > 0$；给定起点 s 与目标 t，要求一条路径 $\pi = (s = n_0, n_1, \dots, n_k = t)$，使其总代价

$$C(\pi) = \sum_{i=0}^{k-1} c(n_i, n_{i+1}) \tag{19.1}$$

最小。2D 栅格地图上，节点就是格子，边是 4 邻接或 8 邻接的相邻格，c 通常取 1（直走）与 $\sqrt{2}$（斜走），再加上该格的代价值惩罚。

先用一个具体例子把符号落地：分辨率 0.05 m 的栅格里，直走一格代价 1、实际长度 0.05 m；斜走一格代价 $\sqrt{2} \approx 1.414$、实际长度 0.0707 m。若某格代价值 $c_{\text{cell}} = 100$（比较靠近障碍），工程上把它折算成额外代价，例如边权取 $w = 1 + 3 \cdot (100/254) \approx 2.18$，于是规划器会“宁可绕一格也不贴着障碍走”。

定义 19.1 (完备性、最优性、启发函数)

完备性（completeness）：如果解存在，算法一定能在有限时间内找到一条解；最优性（optimality）：算法找到的解的代价等于最小代价 C^*；启发函数（heuristic）$h(n)$ 是对“从节点 n 到目标的剩余代价”的估计，真实剩余最优代价记作 $h^*(n)$。若对所有 n 都有 $0 \le h(n) \le h^*(n)$，称 h 可采纳（admissible）；若还满足 $h(n) \le c(n, n') + h(n')$（三角不等式），称 h 一致（consistent，或单调）。 ♣

用数字感受一下：在 10×10 栅格里从 $(1, 1)$ 到 $(8, 8)$，若只许四邻接直走，真实剩余代价 $h^* = 7 + 7 = 14$。曼哈顿距离 $h = |8 - 1| + |8 - 1| = 14$，恰好等于 h^*，既紧又可采纳；欧氏距离 $h = \sqrt{7^2 + 7^2} = 9.90$，也可采纳但偏小；若误用“直线距离 $\times 2$” $= 19.8 > h^*$，就**不可采纳**，A* 不再保证最优。

19.2 Dijkstra 算法

Dijkstra 是“无启发”的最短路：维护一个按 $g(n)$（从起点到 n 的当前最优代价）排序的优先队列，每次弹出 g 最小的节点并松弛它的邻居。

算法：Dijkstra（单源最短路）

1. 初始化：$g(s) = 0$，其余 $g(n) = +\infty$；优先队列 Q 只放 s；closed 集为空

2. 当 Q 非空：弹出 g 最小的节点 n，若 $n = t$ 则回溯得到路径并返回

3. 若 $n \in$ closed 则跳过；把 n 加入 closed

4. 对 n 的每个邻居 n'：若 $g(n)+c(n,n')<g(n')$，则更新 $g(n')$，记录父节点为 n，把 n' 压入 Q
5. 若 Q 空了仍未到 t，说明不存在可行路径

正确性要点：当 n 被弹出时，$g(n)$ 已经是最优的。因为所有边权非负，任何“更短、但尚未发现的路径”都必须经过 Q 中某个 g 不小于 $g(n)$ 的节点，而它之后再走非负代价只会更贵——这与 n 是当前最小矛盾。复杂度：用二叉堆实现时，每个节点入队/出队各 $O(\log V)$，每条边松弛一次，总计 $O((V+E)\log V)$；栅格图上 $E\approx 4V$ 或 $8V$，故约 $O(V\log V)$。

10×10 的图有 $V=100$ 个节点，计算量可忽略；换成 1000×1000 的室内地图，$V=10^6$，Dijkstra 就明显偏慢，这正是需要启发函数的原因。

19.3 A*：用启发函数剪枝

A* 在 Dijkstra 上只改一处：优先队列按

$$f(n)=g(n)+h(n) \tag{19.2}$$

排序。$g(n)$ 是“已经花的钱”，$h(n)$ 是“预计还要花的钱”，$f(n)$ 是“整条路径的预算估计”。因为 h 把目标方向的信息带了进来，A* 会优先朝目标扩展，扩展节点数通常远少于 Dijkstra[17]。

19.3.1 可采纳性：为什么 $h\le h^*$ 就保证最优

这是本章最需要“真的推一遍”的定理。

定理 19.1 (A* 的最优性)

若启发函数可采纳（$0\le h(n)\le h^*(n)$ 且 $h(t)=0$），边权非负，则 A 返回的路径代价等于最优代价 C^*。*

证明（反证法）。设最优路径为 π^*，代价 C^*。假设 A* 在弹出目标 t 时返回的路径代价 $C>C^*$，即 $f(t)=g(t)+h(t)=C>C^*$。

第一步，证明“前沿里始终存在 π^* 上的节点，且其 g 已是最优”。A* 每弹出一个节点就松弛其所有邻居，因此只要 π^* 上某节点的前驱被扩展过，该节点就会被放进 open 表。初始时 $s=n_0$ 在 open 表且 $g(s)=0=g^*(s)$。假设弹出 t 之前，π^* 上已有 $n_0,\dots,n_{k-1}$ 被扩展且 $g(n_i)=g^*(n_i)$，则扩展 n_{k-1} 时松弛边 (n_{k-1},n_k)，得到

$$g(n_k)\le g(n_{k-1})+c(n_{k-1},n_k)=g^*(n_{k-1})+c(n_{k-1},n_k)=g^*(n_k). \tag{19.3}$$

而 $g(n_k)$ 不可能小于最优前缀代价 $g^*(n_k)$（否则 $g^*(n_k)$ 就不是最优），故 $g(n_k)=g^*(n_k)$。归纳得：在弹出 t 的时刻，π^* 上第一个尚未被扩展的节点 n 必在 open 表中，且 $g(n)=g^*(n)$。

第二步，估计这个 n 的 f 值。由 π^* 的最优性，$g^*(n)+h^*(n)=C^*$；又 h 可采纳，$h(n)\le h^*(n)$，于是

$$f(n)=g(n)+h(n)=g^*(n)+h(n)\le g^*(n)+h^*(n)=C^*. \tag{19.4}$$

第三步，利用 A* 的“按 f 最小弹出”性质。既然 t 被弹出，说明它对所有 open 节点都有 $f(t)\le f(n)$，于是

$$C=f(t)\le f(n)\le C^*, \tag{19.5}$$

这与假设 $C>C^*$ 矛盾。因此 $C=C^*$，A* 返回最优解。证毕。

这个证明用了三个前提：边权非负（保证“绕远不会更便宜”）、$h(t)=0$（目标处预算就是已花费）、h 可采纳（保证估计不会高过真实），缺一不可。

19.3.2 一致性：为什么可以不重复扩展

若 h 还满足一致性 $h(n)\le c(n,n')+h(n')$，则沿任意一条边 $n\to n'$，

$$f(n)=g(n)+h(n)\le g(n)+c(n,n')+h(n')=f(n'), \tag{19.6}$$

即 f 沿路径单调不减。注意等式右端正是 $f(n')$，因为 $g(n') = g(n) + c(n, n')$ 是沿这条边到达 n' 的代价（可能还不是最优，但上界成立）。

由此可证“每个节点只扩展一次就得到最优 g”。设节点 n 被弹出时 $g(n) > g^*(n)$。取最优路径上第一个尚未被扩展的节点 n'，由第一步同样的归纳可知 $g(n') = g^*(n')$。沿最优路径从 n' 到 n 反复使用一致性 $h(n_i) \le c(n_i, n_{i+1}) + h(n_{i+1})$ 并求和，得到

$$h(n') \le \big(g^*(n) - g^*(n')\big) + h(n). \tag{19.7}$$

两边同加 $g^*(n')$：

$$f(n') = g^*(n') + h(n') \le g^*(n) + h(n) < g(n) + h(n) = f(n), \tag{19.8}$$

最后一个不等号用到了假设 $g^*(n) < g(n)$。于是 $f(n') < f(n)$，n' 会被先弹出，与“n 先弹出”矛盾。所以弹出时必有 $g(n) = g^*(n)$，closed 集不会误封节点，A* 无需重新打开已关闭节点。

19.3.3 启发函数的设计

设当前节点 (x, y)、目标 (x_g, y_g)，常用三种：

- **曼哈顿距离** $h = |x - x_g| + |y - y_g|$：四邻接栅格的精确下界，可采纳且一致；上例 $h = 14$。
- **欧氏距离** $h = \sqrt{(x - x_g)^2 + (y - y_g)^2}$：任意方向移动都适用；上例 $h = \sqrt{98} = 9.90$，比曼哈顿松，扩展更多。
- **对角距离**（八邻接、直走 1 斜走 $\sqrt{2}$）：设 $d_x = |x - x_g|$、$d_y = |y - y_g|$，则 $h = \max(d_x, d_y) + (\sqrt{2} - 1)\min(d_x, d_y)$；上例 $h = 7 + 0.414 \times 7 = 9.90$，数值与欧氏相同，但在八邻接图上更精确。

启发函数越紧（越接近 h^* 且不超过），扩展越少，但计算 h 本身也越贵。这是工程上的经典取舍：曼哈顿最便宜，欧氏最通用，八邻接地图上用对角距离。

19.3.4 10×10 网格手算例子

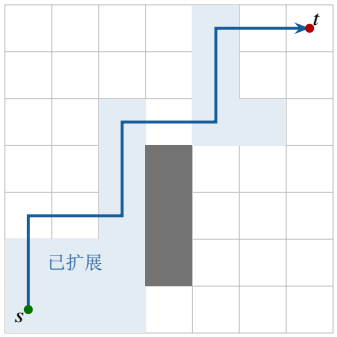


图 19.1: A* 在栅格上的扩展与最终路径示意。浅色格是 f 值相同、被优先扩展的“等值带”；深色格是障碍；折线是最终路径。启发函数的作用就是把扩展限制在朝目标的等值带内，而不是像 Dijkstra 那样铺满整张图。

设栅格 10×10，坐标 (x, y) 取 $0, \ldots, 9$，四邻接直走代价 1。起点 $s = (1, 1)$，目标 $t = (8, 8)$。障碍是一条横墙：第 $y = 2$ 行的 $x = 0, \ldots, 8$ 全部不可通行，只在 $(9, 2)$ 留一个缺口，于是机器人必须先绕到最右侧，才能上到墙的上方。曼哈顿启发 $h = |8 - x| + |8 - y|$，扩展过程见表 19.1。

读表 19.1 要注意两点。第一，墙下方一整排节点的 f 都等于 14，A* 无法区分它们，只能全部扩展，这就是图 19.1 里的“等值带”；启发函数在陷阱地形中会变“钝”。第二，真正的路径代价是 16，而 $h(s) = 14$，说明曼哈顿距离是下界（可采纳），但低估了 2 步——多出来的 2 步正是绕墙的代价。若换用欧氏距离，$h(s) = 9.90$，低估更多，扩展的节点只会更多。这解释了“为什么在长走廊、凹形房间里 A* 会退化成 Dijkstra”。

表 19.1: 手算 A*：关键节点的 g、h、f

节点 (x,y)	到达方式	g	h	$f=g+h$
$(1,1)$	起点	0	14	14
$(2,1)$	向右一步	1	13	14
$(5,1)$	向右四步	4	10	14
$(8,1)$	向右七步	7	7	14
$(9,1)$	再向右一步	8	8	16
$(9,3)$	经缺口上移	10	6	16
$(9,7)$	沿右侧上行	14	2	16
$(8,8)$	向左一步到达	16	0	16

19.4 JPS：跳点搜索

A* 的扩展量还可以再压：栅格上有大量节点处于同一方向、同一代价的“直道”上，逐个扩展毫无意义。JPS（Jump Point Search）[16] 的思路是**只保留必须做决策的“跳点”**（jump point），直道中间的点直接跳过。

核心概念是**强制邻居**（forced neighbor）：假设从父节点 p 沿方向 $\boldsymbol{d}$ 走到当前节点 x。若 x 的某个邻居 n 不在 $\boldsymbol{d}$ 的“自然前进方向”上，但**不经过 x 就无法从 p 以最优代价到达 n**（典型情形是 n 旁边紧贴障碍，x 是该障碍的拐角），则 n 是 x 的强制邻居，x 必须作为跳点保留。JPS 的规则是：沿方向 $\boldsymbol{d}$ 一直“跳”，直到（1）遇到强制邻居，（2）到达目标，（3）撞到障碍；遇到（1）（2）就把该点加入 open 表，遇到（3）则此方向作废。

算法：JPS 的跳点函数 Jump$(x,\boldsymbol{d})$

1. $x' \leftarrow x+\boldsymbol{d}$（沿方向走一步）
2. 若 x' 不可通行，返回空
3. 若 $x'=t$，返回 x'
4. 若 x' 存在强制邻居，返回 x'
5. 若 $\boldsymbol{d}$ 是对角方向：对两个正交分量分别递归 Jump$(x',\boldsymbol{d}_x)$、Jump$(x',\boldsymbol{d}_y)$，任一返回非空则返回 x'
6. 否则返回 Jump$(x',\boldsymbol{d})$

JPS 的最坏复杂度与 A* 同阶，但实际扩展节点数常减少一个数量级（开阔场景中甚至接近直线上的点数）。它的代价是：强制邻居的判定与对角递归让代码复杂，且依赖栅格的 8 邻接规则；在代价值不均匀的代价地图上，JPS 的“对称路径假设”不成立，效果会打折扣。

19.5 Hybrid A*：给车用的 A*

普通 A* 的路径由栅格边组成，会出现 45° 折线，且隐含“可以原地转向、可以任意小半径转弯”的假设。轮式底盘（尤其阿克曼式）做不到：它有最小转弯半径 $R_{\min}$，不能横移，倒车也受限制。Hybrid A*[8] 的做法是：**搜索状态仍是连续的 (x,y,θ)，但用栅格做“已访问”去重**，扩展时不再连相邻格，而是用一串符合车辆运动学的**运动基元**（motion primitive）。

最常用的基元来自自行车模型。设轴距 L、前轮转角 δ、速度 v（可正可负），则

$$\dot{x}=v\cos\theta,\qquad \dot{y}=v\sin\theta,\qquad \dot{\theta}=\frac{v\tan\delta}{L}. \tag{19.9}$$

把 δ 固定为几个离散值（如 $-\delta_{\max},0,+\delta_{\max}$），把时间离散为步长 Δt，就得到一条短弧：

$$x_{k+1}=x_k+v\Delta t\cos\theta_k,\quad y_{k+1}=y_k+v\Delta t\sin\theta_k,\quad \theta_{k+1}=\theta_k+\frac{v\Delta t\tan\delta_k}{L}. \tag{19.10}$$

例如 $L=0.5$ m、$\delta_{\max}=30°$、$v=1$ m/s、$\Delta t=0.2$ s，则每步前进 0.2 m，转角增量 $\Delta\theta=0.2\times\tan 30°/0.5=0.231$ rad $=13.2°$，对应的最小转弯半径 $R_{\min}=L/\tan\delta_{\max}=0.5/0.577=0.866$ m。

“Hybrid”体现在两处：状态连续（x,y,θ 是实数），去重离散（把状态映射到索引 $(\lfloor x/\rho\rfloor,\lfloor y/\rho\rfloor,\lfloor\theta/\Delta\theta\rfloor)$ 记

入 closed 表，ρ 是栅格分辨率）。

启发函数取两个下界的最大值：

- h_{kin}：忽略障碍、只考虑运动学的最短距离（Reeds-Shepp 曲线，允许前进 + 倒车；只许前进时用 Dubins 曲线）；
- h_{grid}：忽略运动学、用 2D 栅格 Dijkstra 算出的带障碍最短距离。

两者都是真实代价的下界（各自忽略了一部分约束），所以 $h = \max(h_{\text{kin}}, h_{\text{grid}})$ 仍可采纳；取最大还能让搜索同时被“运动学可行”和“避开障碍”两个方向引导。

代价函数除了路径长度，还要加惩罚项：倒车惩罚、换挡惩罚、转角变化惩罚、贴近障碍惩罚。工程上这些权重通常设成“1 m 倒车约等于 2–3 m 前进”的量级。

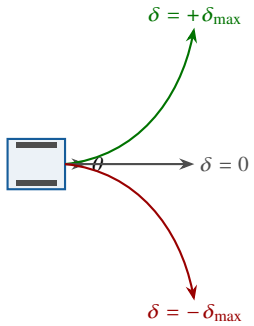


图 19.2: Hybrid A* 从一个状态出发的三条运动基元：左转、直行、右转。每条基元都满足式 (19.9) 的自行车模型，因此生成的路径天然满足最小转弯半径；搜索时把每条基元的终点作为新状态压入 open 表。

Hybrid A* 适合车、差速底盘、以及一切“不能原地 360° 旋转”的平台。代价是状态空间变成三维，节点数明显增加；工程上常配“先粗后细”的两阶段（低分辨率搜索 + 高分辨率精修），以及 anytime 版本（先给一条可行解，再持续优化）。

19.6 采样规划：RRT 与 RRT*

当状态维数很高（机械臂 6–7 维、无人机 3 维位置 + 姿态）或地图没有显式栅格时，图搜索不再适用，改用**采样规划**：在状态空间随机撒点，把点连成一棵树，直到连到目标。

RRT（快速扩展随机树）每次采样 q_{rand}，在树中找最近节点 q_{near}，从 q_{near} 朝 q_{rand} 走固定步长 η 得到 q_{new}，若边不与障碍相交则加入树。RRT 是概率完备的（采样足够多时以概率 1 找到解），但找到的路径往往很绕、不最优。

RRT*[22] 做了两处关键改进。第一，**选择父节点**：不以“最近”为准，而是在半径 r_n 内的所有节点中，选“把它接到 q_{new} 后总代价最小”的那个当父节点。第二，**重连**（rewire）：对半径 r_n 内的每个节点 q，若“经过 q_{new} 到 q”比“q 原来的路径”更短，就把 q 的父节点改成 q_{new}。选择半径随节点数增加而收缩，定义为

$$r_n = \gamma \left(\frac{\log n}{n} \right)^{1/d} \tag{19.11}$$

其中 n 是当前节点数，d 是状态维数，γ 是常数。r_n 随 n 增大而缩小：早期重连范围大（快速优化），后期范围小（局部精修）。在这套机制下，RRT* 是渐近最优的：采样数 $n \to \infty$ 时，返回路径的代价以概率 1 收敛到最优。

工程上 RRT* 的主要问题是收敛慢、结果依赖随机数，所以常配 **Informed RRT***：先用一次搜索找到初始解，再把采样限制在“代价不超过当前最优解”的椭圆内，加速收敛。高维机械臂规划多用 RRT-Connect 或 OMPL 里的各种变体。

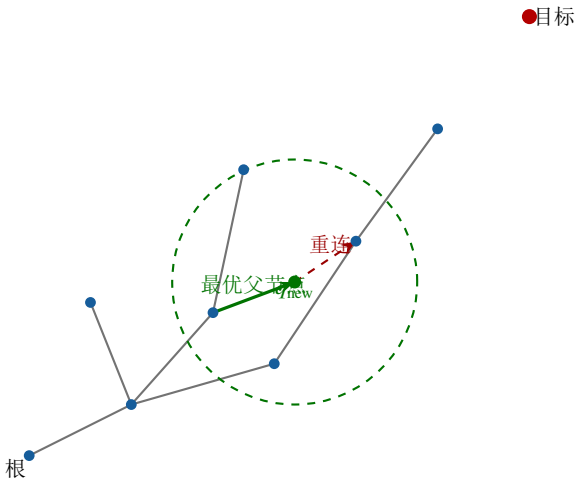


图 19.3: RRT* 的“选父节点 + 重连”。绿色虚线圈是选择半径 r_n；绿色实线是被选中的最优父边；红色虚线是重连后更短的连接。RRT* 用这种“不断改父节点”的方式，把一棵随机的树逐渐拉向最优解。

19.7 其他经典方法与工程实践

可视图（visibility graph）：把障碍物的顶点与起终点两两连线，保留不与障碍相交的边，在图上跑 Dijkstra。它在多边形障碍中给出最短路径，节点数少、结果精确，但“贴边”太危险，且只适合多边形表示的地图。Voronoi 图则相反：它给出“离障碍最远”的路径，安全性好、长度偏长，适合做保守规划。第 21 章会看到 FAR Planner 如何在 3D 中增量维护可视图。

人工势场（potential field）是另一类思路：把目标设为引力源、障碍设为斥力源，机器人沿合力方向走。它极快、极简单，但有局部极小值（机器人卡在凹形障碍前），第 20 章会给出完整公式并讨论改进。

在 Nav2 中，全局规划由 `planner_server` 的插件实现，常用的有：

- `NavfnPlanner`：在代价地图上跑 Dijkstra/A*（可切换），支持 4/8 邻接，最经典、最稳。
- `SmacPlanner2D`：显式 A*，支持未知区域代价、可配置启发权重，适合 2D 差速底盘。
- `SmacPlannerHybrid`：Hybrid A*，支持倒车、最小转弯半径，适合阿克曼与“不能原地转”的平台。
- `SmacPlannerLattice`：状态格（lattice）规划，适合需要精确运动原语的场景。

```yaml
# nav2_params.yaml (excerpt): pick and tune the global planner plugin
planner_server:
  ros__parameters:
    planner_plugins: ["GridBased", "Hybrid"]
    GridBased:                    # 2D A*/Dijkstra for differential drive
      plugin: "nav2_navfn_planner/NavfnPlanner"
      use_astar: true
      tolerance: 0.5
    Hybrid:                       # Hybrid A* for car-like platforms
      plugin: "nav2_smac_planner/SmacPlannerHybrid"
      motion_model: "DUBINS"      # use REEDS_SHEPP if reverse is allowed
      minimum_turning_radius: 0.4
```

代价值如何进入搜索？NavFn 风格的实现把每个栅格的代价值 $c_{\text{cell}} \in [0, 254]$ 映射为附加代价

$$w = c_{\text{neutral}} + \kappa\, c_{\text{cell}}, \tag{19.12}$$

其中中性代价 c_{neutral} 默认取 50（表示“平地上走一格的基准代价”），缩放系数 κ 对应参数 `cost_factor`，默认 3.0；致命障碍（254）直接禁止通行。这样“贴墙走”的边会明显变贵，规划器自然选择离墙远一点。

最后，规划器给出的路径往往仍有小折角，Nav2 用 `smoother_server` 做后处理：以“平滑项 + 贴近原路径

项 + 曲率/障碍惩罚”构成目标函数，梯度下降迭代优化路径点。平滑后的路径更贴合底盘运动学，局部控制器跟踪起来也更轻松。

本章小结

1. 全局规划是带非负边权的图搜索问题；完备性问“找不找得到”，最优性问“是否最短”，启发函数是对剩余代价的估计。
2. Dijkstra 无启发、保证最优，复杂度 $O((V+E)\log V)$；A* 用 $f=g+h$ 引导，h 可采纳时最优，一致时可“不重开节点”。
3. 可采纳性证明的三步：前沿上存在最优路径节点且其 g 已最优，于是该节点 $f\le C^*$；目标以最小 f 弹出，故 $C\le C^*$。
4. 启发函数曼哈顿/欧氏/对角距离都是下界；越紧扩展越少；陷阱地形中 A* 会退化为等值带扩张。
5. JPS 用强制邻居跳过直道；Hybrid A* 用自行车模型基元 + 三维去重 + 双下界启发，解决最小转弯半径问题；RRT* 用“选父 + 重连 + 收缩半径”实现渐近最优。
6. 工程上按底盘选插件：2D 差速用 NavFn 或 Smac2D，阿克曼用 Smac Hybrid；代价值经“中性代价 + 缩放系数”映射为边权，最后用 `smoother_server` 平滑。

第十九章 练习

1. 用反证法复述 A* 最优性证明的三步，并指出每一步用了哪个前提。
2. 在 10×10 栅格上，$s=(1,1)$、$t=(8,8)$，若某格被误标成致命障碍，曼哈顿启发是否仍然可采纳？启发值变不变？
3. 为什么 $h=\max(h_{\text{kin}},h_{\text{grid}})$ 仍然可采纳？如果取两者的最小值会怎样？
4. 给定 $L=0.6$ m、$\delta_{\max}=25°$，求最小转弯半径；若每步前进 $\Delta s=0.15$ m，求每步的转角增量。
5. 设计一个“凹形房间”地图，说明 A* 与 RRT* 各会表现如何；如果你只能改一个参数来加速 A*，改什么？

延伸阅读

- [17]：A* 的原始论文，可采纳性与一致性的定义都出自这里，证明写得比教材更原始也更清楚。
- [16]：JPS 原始论文，读它解决“直道上的节点为什么可以整段跳过”。
- [8]：Hybrid A* 的工程实现细节（启发、基元、两阶段搜索），自动驾驶规划必读。
- [22]：RRT* 的渐近最优性证明与半径公式，想理解采样规划的理论极限就读它。

第二十章 局部规划与控制

本章导读

全局规划给出的是“大方向”，它只有几何、没有时间，也不考虑机器人此刻的速度、加速度和脚下 1 米内突然出现的障碍。局部规划（local planning）要在这条全局路径附近、在毫秒级的时间里，算出一条未来 1–3 秒真正能执行、不撞障碍、满足底盘约束的轨迹，并把它变成速度指令。本章按“采样—优化—预测控制—梯度优化”的演进线讲四种方法：DWA、TEB、MPC 与 EGO-Planner 式的 B 样条梯度法，再补上人工势场与底盘运动学约束，最后给出 Nav2 四个 controller 插件的选型对比。

20.1 局部规划要解决什么

局部规划的输入是：全局路径 $\mathcal{P}$、局部代价地图、当前位姿与速度 $(\boldsymbol{x}, \boldsymbol{u})$、机器人模型（尺寸、$v_{\max}$、$a_{\max}$、$\omega_{\max}$、最小转弯半径）。输出是：未来一段时间的控制序列，或直接是当前时刻的速度指令 (v, ω)。它必须同时满足四件事：

1. **跟踪**：轨迹要贴着全局路径走，不能偏离太远；
2. **避障**：与障碍保持安全距离，包括代价地图里新出现的动态障碍；
3. **可行**：满足速度、加速度、转角限制，不能算出底盘执行不了的动作；
4. **实时**：10–20 Hz 内算完，因为局部规划要在障碍出现后 100 ms 量级内做出反应。

先用一个数字把“实时”的分量说清楚：控制周期 50 ms（20 Hz），留给局部规划的计算时间通常要小于 20 ms（剩下的留给通信、TF 查询与控制平滑）。这意味着任何单次迭代超过 20 ms 的方法都必须降频、降精度，或者只在低维空间里做。

20.2 DWA：把可行速度采样一遍

DWA（Dynamic Window Approach）[13] 的思路非常朴素：**不去设计轨迹，而是枚举“下一时刻可能的速度”，把每个速度对应的短轨迹算出来打分，选最高分的那个执行。**它之所以叫“动态窗口”，是因为待枚举的速度范围受当前速度和加速度限制，是一个随时间“滑动”的窗口。

20.2.1 运动模型与动态窗口

差速底盘（unicycle）的运动学为

$$\dot{x} = v\cos\theta, \qquad \dot{y} = v\sin\theta, \qquad \dot{\theta} = \omega, \tag{20.1}$$

离散化后（步长 Δt）

$$x_{k+1} = x_k + v\Delta t\cos\theta_k, \quad y_{k+1} = y_k + v\Delta t\sin\theta_k, \quad \theta_{k+1} = \theta_k + \omega\Delta t. \tag{20.2}$$

对某一个候选 (v, ω)，从当前位姿递推 N 步就得到一条候选轨迹；若轨迹上任意一点落在致命障碍格上，该候选直接淘汰。

可枚举的速度集合是三个集合的交：

- 硬约束 $V_s = \{(v,\omega) \mid v \in [v_{\min}, v_{\max}],\ \omega \in [-\omega_{\max}, \omega_{\max}]\}$；
- 动态窗口 $V_d = \{(v,\omega) \mid v \in [v_c - a_v\Delta t,\ v_c + a_v\Delta t],\ \omega \in [\omega_c - a_\omega\Delta t,\ \omega_c + a_\omega\Delta t]\}$，其中 v_c, ω_c 是当前速度，a_v, a_ω 是最大线/角加速度；
- 安全约束 $V_a = \{(v,\omega) \mid v \le \sqrt{2\, d_{\text{stop}}(v,\omega)\, a_{\text{brake}}}\}$，表示“以该速度走，还能在撞上之前刹住”。$d_{\text{stop}}$ 是沿该轨迹到最近障碍的距离。

最终 $V_r = V_s \cap V_d \cap V_a$。举个数：$v_c = 0.5$ m/s、$a_v = 1.0$ m/s^2、$\Delta t = 0.1$ s，则 v 只能取 $[0.4, 0.6]$；若 $v_{\max} = 1.2$，真正限制搜索的是动态窗口而不是上限——这正是“动态”的意义：速度不能突变。

20.2.2 评价函数

对每个候选 (v, ω)，用三个（或五个）归一化指标打分：

$$\text{heading}(v,\omega) = \pi - \left|\theta_{\text{goal}} - \theta(t_N)\right|, \tag{20.3}$$

$$\text{clearance}(v,\omega) = \min_{k \in [0,N]} d_{\text{obs}}(x_k, y_k), \tag{20.4}$$

$$\text{velocity}(v,\omega) = |v|, \tag{20.5}$$

其中 $\theta_{\text{goal}} = \text{atan2}(y_g - y_0,\, x_g - x_0)$ 是当前点到目标的方位角，$\theta(t_N)$ 是候选轨迹终点的朝向，d_{obs} 是该点到最近障碍的距离（代价地图的距离变换直接给出）。三项量纲不同，必须先归一化。DWA 原文与 DWB 的常用做法是对所有候选求和后相除：

$$\hat{h} = \frac{\text{heading}}{\sum \text{heading}}, \quad \hat{c} = \frac{\text{clearance}}{\sum \text{clearance}}, \quad \hat{v} = \frac{\text{velocity}}{\sum \text{velocity}}, \tag{20.6}$$

然后

$$G(v,\omega) = \alpha\,\hat{h} + \beta\,\hat{c} + \gamma\,\hat{v}, \qquad \alpha + \beta + \gamma = 1. \tag{20.7}$$

α 大则“更想朝向目标”，β 大则“更想远离障碍”，γ 大则“更想跑得快”。三者的物理含义分别是“方向一致性”“安全裕度”“任务效率”。

表 20.1: 一个 DWA 打分的数值例子（$T = 1$ s，$\alpha = 0.6$、$\beta = 0.3$、$\gamma = 0.1$）

候选 (v,ω)	终点	$\hat{h}$	$\hat{c}$	$\hat{v}$	G
(0.5, 0) 直行	(0.50, 0)	0.852	0.89	0.50	0.828
(0.5, 0.5) 左转	(0.48, 0.12)	0.992	0.56	0.50	0.813
(0.5, −0.5) 右转	(0.48, −0.12)	0.639	1.00	0.50	0.733

表 20.1 里，左转的朝向得分最高（0.992），但离障碍更近导致安全得分低；右转最安全但方向偏得最多；直行两项都不极端，加权后胜出。这个例子说明两点：第一，**权重决定行为风格**，把 β 调大就会变成“贴墙恐惧症”；第二，**分数接近时机器人会犹豫**（0.828 与 0.813 只差 0.015），实际 DWB 会对 G 做时间上的低通平滑，避免相邻周期来回切换。

20.2.3 为什么能实时，局限在哪

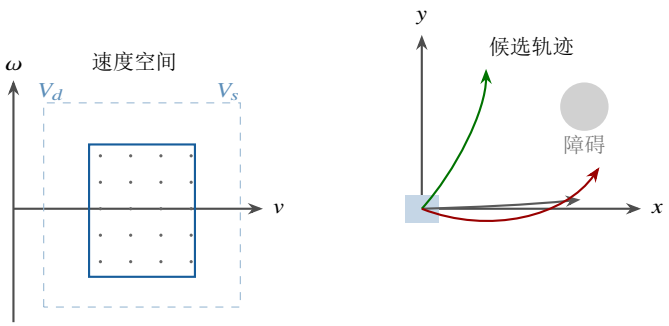


图 20.1: DWA 的两层搜索。左：在速度空间 (v,ω) 中，粗框是动态窗口 V_d，虚框是硬约束 V_s，黑点是采样点；右：每个采样点积分出一条短轨迹，撞障碍的淘汰，其余按式 (20.7) 打分。

计算量很容易估：以 v 方向 10 个、ω 方向 15 个采样计，共 150 条候选；每条积分 $N = 30$ 步、每步 3 个三角函数与一次碰撞查询，总计约 1.5×10^4 次基本运算，在 20 Hz 的周期里绰绰有余。这就是 DWA 能在 2000 年代的嵌入式 CPU 上跑实时避障的原因。

但 DWA 的缺点同样明显：它只在一步速度空间里做贪心，**没有全局视野**，容易陷在局部最优（U 形障碍前反复打转）；对“必须倒车才能脱困”的场景无能为力；T 只有 1–2 s，对高速或大惯量平台不够；参数（α, β, γ、分辨率、T）与场地强相关，换场地要重调。

20.3 TEB：把轨迹变成图优化

TEB（Timed Elastic Band）[33] 换了个思路：不再枚举控制，而是把轨迹表示成一串带时间戳的位姿，**把“轨迹好不好”写成一个目标函数，用图优化求解**。

轨迹变量是位姿序列 $\mathcal{Q} = \{\boldsymbol{s}_0, \boldsymbol{s}_1, \ldots, \boldsymbol{s}_n\}$，其中 $\boldsymbol{s}_k = (x_k, y_k, \beta_k)$（$\beta_k$ 为朝向），相邻位姿之间还有时间间隔 $\Delta T_k > 0$。目标函数由五类惩罚组成：

$$J(\mathcal{Q}, \Delta T) = w_h \sum_k \left[\max(0, r_h - d(\boldsymbol{s}_k, \mathcal{P}))\right]^2 + w_o \sum_k \left[\max(0, r_o - d(\boldsymbol{s}_k, \mathcal{O}))\right]^2 + J_{\text{kin}} + J_{\text{dyn}} + J_{\text{time}}, \tag{20.8}$$

其中

$$J_{\text{dyn}} = \sum_k \left[\max(0, |v_k| - v_{\max})^2 + \max(0, |a_k| - a_{\max})^2\right], \tag{20.9}$$

$$J_{\text{kin}} = \sum_k \left[(y_{k+1} - y_k)\cos\beta_k - (x_{k+1} - x_k)\sin\beta_k\right]^2, \tag{20.10}$$

$$J_{\text{time}} = \left(\sum_k \Delta T_k\right)^2. \tag{20.11}$$

逐项解释：$d(\boldsymbol{s}_k, \mathcal{P})$ 是位姿到全局路径的距离，超过阈值 r_h 才罚，保证“不偏离大方向”；$d(\boldsymbol{s}_k, \mathcal{O})$ 是到障碍的距离，小于安全裕度 r_o 才罚，保证“不贴墙”；J_{kin} 惩罚“斜着走”（速度方向与朝向不一致），这是差速底盘的硬约束；J_{dyn} 用 $v_k = \Delta s_k / \Delta T_k$ 与 $a_k = (v_{k+1} - v_k)/\Delta T_k$ 约束速度、加速度；J_{time} 让总时间尽量短（否则机器人会磨蹭）。

式 (20.8) 的每一项都只涉及 1–2 个位姿节点，天然是**稀疏图**：位姿是节点，惩罚项是超边（hyper-edge），时间间隔也是节点。用 g2o[24] 建图后，做稀疏 Levenberg–Marquardt 迭代，通常 2–5 次迭代收敛；把上一周期的解作为初值（warm start），10–20 Hz 完全能跑。位姿数 n 通常取 10–30，对应 1–3 s 的轨迹。

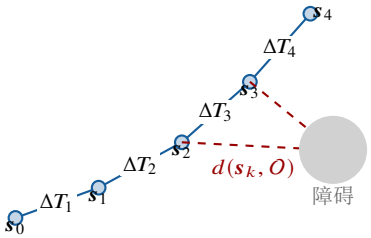


图 20.2: TEB 的图结构：圆形是位姿节点 $\boldsymbol{s}_k$（优化变量），相邻位姿之间的连边带时间间隔 ΔT_k，红色虚线是到障碍的“距离惩罚”超边。优化时全体位姿一起移动、时间间隔一起缩放，所以 TEB 能同时改变“路径形状”和“速度分配”。

TEB 的好处是轨迹平滑、能处理车式底盘（非完整约束写进 J_{kin}）、能把“时间”也当作优化变量，因此停车、让行、窄道通过都比较自然。代价是：目标函数非凸，初值不好会收敛到局部极小；每周期都要建图求解，CPU 占用比 DWB 高一个量级；参数（w_h, w_o、r_h, r_o）比 DWA 更难调。

20.4 MPC：滚动时域的最优控制

MPC（Model Predictive Control）[32] 是控制理论的标准答案：用模型预测未来 N 步，在每条预测轨迹上优化控制序列，只执行第一步，下一周期再重新预测——这就是**滚动时域**（receding horizon）。

设离散状态空间模型

$$\boldsymbol{x}_{k+1} = \boldsymbol{A}\boldsymbol{x}_k + \boldsymbol{B}\boldsymbol{u}_k, \tag{20.12}$$

其中 $\boldsymbol{x} = [x, y, \theta, v, \omega]^\top$，$\boldsymbol{u} = [a, \alpha]^\top$（线加速度、角加速度），$\boldsymbol{A}, \boldsymbol{B}$ 由式 (20.1) 在当前工作点线性化得到。每个控制周期求解

$$\begin{aligned} \min_{\boldsymbol{x}_{1:N}, \boldsymbol{u}_{0:N-1}} \quad & \sum_{k=0}^{N-1} \left(\boldsymbol{x}_k - \boldsymbol{x}_k^{\text{ref}}\right)^\top \boldsymbol{Q} \left(\boldsymbol{x}_k - \boldsymbol{x}_k^{\text{ref}}\right) + \boldsymbol{u}_k^\top \boldsymbol{R} \boldsymbol{u}_k + \left(\boldsymbol{x}_N - \boldsymbol{x}_N^{\text{ref}}\right)^\top \boldsymbol{Q}_f \left(\boldsymbol{x}_N - \boldsymbol{x}_N^{\text{ref}}\right) \\ \text{s.t.} \quad & \boldsymbol{x}_{k+1} = \boldsymbol{A}\boldsymbol{x}_k + \boldsymbol{B}\boldsymbol{u}_k, \quad \boldsymbol{x}_0 = \hat{\boldsymbol{x}}, \\ & \boldsymbol{u}_{\min} \le \boldsymbol{u}_k \le \boldsymbol{u}_{\max}, \quad \boldsymbol{x}_k \in \mathcal{X}_{\text{free}}. \end{aligned} \tag{20.13}$$

$\boldsymbol{Q} \succeq 0$、$\boldsymbol{R} \succ 0$ 是权重矩阵：$\boldsymbol{Q}$ 大则“更想贴合参考轨迹”，$\boldsymbol{R}$ 大则“更不想用大加速度”。约束里的 $\mathcal{X}_{\text{free}}$ 是“不撞障碍”，工程上把每个附近障碍近似成半平面 $\boldsymbol{a}_j^\top \boldsymbol{x} \le b_j$ 加进去，于是整个问题变成**二次规划**（QP），可用 OSQP 之类的稀疏求解器在毫秒级解出。

规模感受一下：$N = 20$、状态 5 维、控制 2 维，决策变量 $20 \times 5 + 20 \times 2 = 140$ 个，约束约几百条，20 Hz 下现代 CPU 完全够用。MPC 与 DWA 的关系可以这样理解：**DWA 是在离散控制集上做一步贪心，MPC 是在连续控制空间里做 N 步带约束的优化**；DWA 不需要模型（只用运动学积分），MPC 需要模型和求解器，但能显式处理加速度、障碍、终端代价，且天然支持预测动态障碍（把障碍位置写成时间的函数即可）。

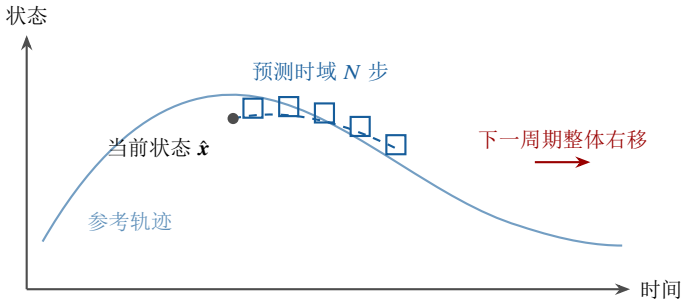


图 20.3: MPC 的滚动时域：每个控制周期都以当前状态为起点，预测未来 N 步并优化；只执行第一步控制，下一周期把时域整体向前平移、用新的测量重新优化。

20.5 梯度类方法：B 样条轨迹优化

DWA 采样、TEB 图优化、MPC 求解 QP，都是在“低维控制/位姿”上做文章。无人机、四足等高速平台更常用**连续时间轨迹优化**：用 B 样条参数化整条轨迹，直接对控制点求梯度 [47, 48]。

p 次 B 样条曲线为

$$\boldsymbol{p}(t) = \sum_{i=0}^{n} B_{i,p}(t)\, \boldsymbol{Q}_i, \tag{20.14}$$

其中 $\boldsymbol{Q}_i \in \mathbb{R}^3$ 是控制点，$B_{i,p}(t)$ 是基函数。它有两个对规划极有用的性质：**局部支撑**（改一个控制点只影响相邻 $p+1$ 段）与**凸包性**（每一段曲线都在相邻 $p+1$ 个控制点的凸包内）。

目标函数通常写成三项之和：

$$J = \lambda_s J_{\text{smooth}} + \lambda_c J_{\text{collision}} + \lambda_d J_{\text{dynamic}}. \tag{20.15}$$

平滑项用二阶差分（近似加速度/曲率）：

$$J_{\text{smooth}} = \sum_{i=1}^{n-1} \left\| \boldsymbol{Q}_{i+1} - 2\boldsymbol{Q}_i + \boldsymbol{Q}_{i-1} \right\|^2. \tag{20.16}$$

碰撞项需要一个“到障碍的距离场”。经典做法是把地图做成 **ESDF**（欧氏符号距离场）[30]：每个体素存“到最近障碍的距离” $d(\boldsymbol{p})$，其梯度 $\nabla d(\boldsymbol{p})$ 指向远离障碍的方向。碰撞代价取二次惩罚

$$J_{\text{collision}} = \sum_{i} \left[\max\left(0,\ d_0 - d(\boldsymbol{Q}_i)\right)\right]^2, \tag{20.17}$$

d_0 是安全距离。对控制点求偏导，用链式法则：

$$\frac{\partial J_{\text{collision}}}{\partial \boldsymbol{Q}_i} = -2\big(d_0 - d(\boldsymbol{Q}_i)\big)\,\nabla d(\boldsymbol{Q}_i), \qquad d(\boldsymbol{Q}_i) < d_0. \tag{20.18}$$

举个数值：$d_0 = 0.5$ m、某控制点处 $d = 0.2$ m，则惩罚为 $(0.3)^2 = 0.09$，梯度大小为 $2 \times 0.3 = 0.6$，方向沿 ∇d（远离障碍），优化后该控制点被推出去。

算法：ESDF 梯度下降优化一段 B 样条轨迹

1. 用前端搜索（A*/Hybrid A*）给出一条初始轨迹，转成 B 样条控制点 $\boldsymbol{Q}_i$
2. 重复直到收敛或达到迭代上限：
3. 　对每个控制点，从 ESDF 查 $d(\boldsymbol{Q}_i)$ 与 $\nabla d(\boldsymbol{Q}_i)$，按式 (20.18) 累加碰撞梯度
4. 　按式 (20.16) 累加平滑梯度，两项加权求和得 $\partial J/\partial \boldsymbol{Q}_i$
5. 　沿负梯度更新 $\boldsymbol{Q}_i$，并检查凸包性质（控制点都不在障碍内则整段安全）
6. 输出轨迹，交给控制器跟踪

EGO-Planner[48] 的关键改进是**不显式建 ESDF**：它用“碰撞对 $\{\boldsymbol{p}, \boldsymbol{v}\}$”（轨迹上的点与对应的障碍表面点）直接构造惩罚，并利用 B 样条凸包性质保证优化后的轨迹不穿障碍。这省掉了建 ESDF 的内存与计算，代价是碰撞对的维护逻辑更复杂。对 RoboMaster 这类 2D 主要、偶尔上坡的场地，完整的 ESDF 通常负担得起，可以先从 Fast-Planner 思路入手。

20.6 人工势场：最直观也最容易卡住

人工势场把目标设成引力源、障碍设成斥力源。引力势与引力（梯度取负）为

$$U_{\text{att}}(\boldsymbol{q}) = \tfrac{1}{2} k_a \left\| \boldsymbol{q} - \boldsymbol{q}_g \right\|^2, \qquad \boldsymbol{F}_{\text{att}} = -\nabla U_{\text{att}} = k_a(\boldsymbol{q}_g - \boldsymbol{q}). \tag{20.19}$$

斥力势（ρ 为到障碍距离，ρ_0 为影响半径）为

$$U_{\text{rep}}(\boldsymbol{q}) = \begin{cases} \tfrac{1}{2} k_r \left(\dfrac{1}{\rho} - \dfrac{1}{\rho_0} \right)^2, & \rho < \rho_0, \\ 0, & \rho \geq \rho_0, \end{cases} \qquad \boldsymbol{F}_{\text{rep}} = k_r \left(\frac{1}{\rho} - \frac{1}{\rho_0} \right) \frac{1}{\rho^2} \nabla \rho. \tag{20.20}$$

数值例子：$k_a = 2$，目标 $(5, 0)$，当前位置 $(2, 0)$，则 $\boldsymbol{F}_{\text{att}} = (6, 0)$；$k_r = 0.5$、$\rho_0 = 1$、$\rho = 0.5$，则 $U_{\text{rep}} = 0.25$，$\left| \boldsymbol{F}_{\text{rep}} \right| = 0.5 \times (2 - 1) \times 4 = 2$，方向远离障碍。

势场法的问题是**局部极小值**：当引力与斥力恰好抵消且不在目标点时，机器人停住。凹形障碍、长墙尽头、目标位于墙后都会出现。改进思路有三类：加随机扰动/模拟退火跳出；用调和函数（满足拉普拉斯方程，理论上无局部极小，但计算贵）；最实用的是**用全局路径做引导**——沿全局路径取“虚拟目标”，让势场只负责局部避障，这就回到了“全局规划 + 局部规划”的分工。

20.7 底盘约束如何进入局部规划

局部规划算出的 (v, ω) 必须能被执行器实现。三种常见底盘：

差速（两轮独立驱动）：设轮距 b、左右轮速 v_l, v_r，则

$$v = \frac{v_r + v_l}{2}, \qquad \omega = \frac{v_r - v_l}{b}. \tag{20.21}$$

例如 $v_r = 1.0$ m/s、$v_l = 0.6$ m/s、$b = 0.4$ m，得 $v = 0.8$ m/s、$\omega = 1.0$ rad/s，转弯半径 $R = v/\omega = 0.8$ m。约束是 $v_l, v_r \in [-v_{\max}, v_{\max}]$，等价于 v 与 ω 的联合约束（不是矩形）。

麦轮（全向）：四轮速度由底盘速度解算，例如前左轮

$$v_{fl} = v_x - v_y - (l_x + l_y)\omega, \tag{20.22}$$

其余三轮按符号变化。麦轮可以横移（$v_y \neq 0$），所以局部规划的状态空间可以加一个横向速度，路径不必先转向再前进。

阿克曼（车式）：前轮转角 δ、轴距 L，有

$$\omega = \frac{v \tan\delta}{L}, \qquad R = \frac{L}{\tan\delta} \geq R_{\min} = \frac{L}{\tan\delta_{\max}}. \tag{20.23}$$

$L = 0.6$ m、$\delta_{\max} = 25°$ 时 $R_{\min} = 0.6/0.466 = 1.29$ m。这条约束必须写进规划器：2D 差速用的 DWA 会算出“原地旋转”，阿克曼执行不了，所以要么用 Hybrid A* 做全局规划，要么在局部规划里用 TEB 的 J_{kin} 或 MPC 的模型约束。

20.8 Nav2 的 controller 插件怎么选

表 20.2: Nav2 常见 controller 插件对比

插件	类型	优点	适用/局限
DWB	速度采样（DWA 的 Nav2 实现）	稳、易调、CPU 低	差速底盘、中低速；窄道与局部最优弱
TEB	图优化	轨迹平滑、支持阿克曼、可倒车	CPU 高、初值敏感、参数多
MPPI	采样 + 预测控制	处理动态障碍、平滑、GPU 友好	需要调采样数与温度参数
RPP	纯跟踪几何法	极快、路径跟踪准	避障弱，靠代价地图限速

新手最常犯的错误是“上来就换 TEB”。正确的顺序是：先用 DWB 把 TF、代价地图、全局规划调通，确认能稳定跟踪；出现“窄道过不去、需要倒车、轨迹抖动”再考虑 TEB 或 MPPI。表 20.2 里没有“最好”，只有“匹配”。

```python
# Minimal DWB-style scoring (illustrative, not production code)
def score(candidates, goal, costmap, w=(0.6, 0.3, 0.1)):
    best, best_g = None, -1.0
    for v, w_ in candidates:
        traj = rollout(v, w_, T=1.0, dt=0.05)   # integrate unicycle
        if collides(traj, costmap):
            continue
        h = heading_score(traj[-1], goal)       # pi - |dtheta|
        c = clearance_score(traj, costmap)      # min obstacle distance
        s = speed_score(v)                      # |v| / v_max
        g = w[0] * h + w[1] * c + w[2] * s
        if g > best_g:
            best, best_g = (v, w_), g
    return best
```

本章小结

1. 局部规划在 1–3 s 的滚动窗口内，同时满足跟踪、避障、运动学可行与 20 Hz 实时四项要求。
2. DWA 在动态窗口 $V_r = V_s \cap V_d \cap V_a$ 内采样 (v, ω)，积分短轨迹，按式 (20.7) 加权打分；快但贪心、易陷局部最优。
3. TEB 把轨迹写成位姿 + 时间间隔的图，用式 (20.8) 的六类惩罚做 g2o 优化；平滑、支持非完整约束，但非凸、CPU 高。

4. MPC 用模型预测 N 步、解带约束的 QP、只执行第一步；比 DWA 多模型与约束，比 TEB 更适合显式预测动态障碍。
5. 梯度法用 B 样条参数化轨迹，平滑项用二阶差分，碰撞项用 ESDF 的二次惩罚与梯度（式 (20.18)），EGO-Planner 用碰撞对避免建 ESDF。
6. 人工势场直观但有局部极小；底盘约束（差速式 (20.21)、阿克曼式 (20.23)）必须进入局部规划，否则算出的动作执行不了。
7. Nav2 选型：DWB 打底，TEB/MPPI 解决窄道与动态障碍，RPP 用于快速纯跟踪。

第二十章 练习

1. 用式 (20.2) 手算：初始位姿 $(0, 0, 0)$，$v = 0.6$ m/s、$\omega = 0.4$ rad/s、$\Delta t = 0.1$ s，求 5 步后的位姿。
2. 在表 20.1 的数据下，若把 β 从 0.3 调到 0.8、α 降到 0.1（保持和为 1），重新排序三个候选，并解释机器人行为会怎样变化。
3. TEB 的 J_{kin} 为什么能表达“不能横移”？如果机器人是麦轮，这一项应该怎么改？
4. MPC 的 $\boldsymbol{Q}$ 与 $\boldsymbol{R}$ 分别调大，对轨迹的跟踪精度与舒适性有什么影响？
5. 给定 $L = 0.5$ m、$\delta_{\max} = 30°$，求阿克曼底盘的最小转弯半径；若要求 $R_{\min} \le 0.8$ m，$\delta_{\max}$ 至少要多大？
6. 有人说“DWA 就是 MPC 的特例”，你同意吗？请从目标函数、约束、求解方式三方面比较。

延伸阅读

- [13]：DWA 原始论文，评价函数与动态窗口的定义都在这里，篇幅不长，适合精读。
- [33]：TEB 论文，重点看它如何把“时间间隔”也作为优化变量，以及 g2o 图的构建。
- [32]：MPC 的标准教材，读第 1–2 章即可建立“滚动时域 + QP”的直觉。
- [47, 48]：B 样条轨迹优化的工程实现，重点看碰撞代价的梯度与“无 ESDF”版本的区别。

第二十一章 3D 导航与 CMU 自主探索栈

本章导读

前两章的规划器都假设世界是一张 2D 平面栅格：障碍是“墙”，可走区域是“空地”。但 RoboMaster 之外的很多场景——室外坡道、台阶、多层建筑、越野地形——机器人会上下起伏，2D 栅格会把“坡”压成“平地”，把“台阶”压成“墙”，规划出的路径要么上不去，要么明明能过却被判死。本章讲 3D 导航，并以用户点名的 **CMU（卡内基梅隆大学 Ji Zhang 组，CMU AirLab）自主探索开发环境**为主线：讲清 terrain_analysis 怎么做 2.5D 地形可通行性分析、local_planner 怎么在地形上避障、FAR Planner 怎么用可视图在未知环境里“试探着规划”、TARE 怎么做分层自主探索。学完本章，你应该能画出这套栈的模块数据流，能推导坡度/粗糙度/台阶高度的公式，并知道如何把 FAST-LIO2 的点云喂给它。

21.1 为什么 2D 导航不够

2D 导航的本质是**降维假设**：把机器人当成一个在平面上运动的圆盘，把世界投影到 $z = 0$ 的平面上。这个假设在平整室内非常好用，代价是丢掉三样信息：

1. **高度**：坡道、台阶、门槛、减速带都有高度变化。2D 栅格一旦把“坡”投影成“可走”，机器人可能因坡度太陡而打滑；把“台阶”投影成“障碍”，则明明能上的小台阶被当成墙。
2. **垂直净空**：桌子底下、桥洞、悬空管道下方，2D 地图只看到“有障碍”，3D 才知道“能从下面过”。
3. **地形质量**：同样平坦的格子，草地、碎石、沙地的可通过性完全不同；2D 只区分“占据/空闲”。

解决思路有两条：**真 3D**（体素地图、ESDF、无人机式轨迹优化）与 **2.5D**（把地图表示成“每个格子一个高度”的高程图）。真 3D 表达力强但内存与计算量大（100 m × 100 m × 10 m、0.1 m 体素就是 10^8 个体素）；2.5D 假设“每个 (x, y) 只有一个高度”，内存是 2D 级别，又能表达坡度和台阶，是地面机器人最实用的折中。CMU 这套栈走的就是 2.5D 路线。

21.2 CMU 自主探索栈的总体架构

CMU AirLab 开源了一套“点云进、速度出”的完整导航与探索系统，名为 Autonomous Exploration Development Environment（下称 CMU 栈），配套论文是 ICRA 2022 的 [4]，其中 FAR Planner 的工作另见 [42]。它的设计哲学是：**把“地形理解”与“规划”解耦**——先由地形分析把 3D 点云压成可通行性地图，再由规划器在可通行性地图上做局部与全局规划。

各模块职责如下（频率为典型量级，具体以仓库配置为准）：

- `terrain_analysis`：订阅 LIO 输出的配准点云（`/registered_scan`），维护一张以机器人为中心的 2.5D 高程栅格，计算每格的坡度、粗糙度、台阶高度与到障碍的距离，发布 `/terrain_map`（每点携带“到最近障碍的距离”，供局部规划做净空判断）。
- `terrain_analysis_ext`：在更大范围内做“扩展地形清理”，输出 `/terrain_map_ext`，供 FAR Planner 构建全局可视图；范围大、分辨率可稍粗。
- `local_planner`：在 `/terrain_map` 上做局部路径规划（本质是带净空约束的图搜索 + 速度平滑），跟踪 `/way_point` 并输出 `/cmd_vel`，同时把碰撞检测做成“机器人尺寸 + 离地间隙”的模板。
- `far_planner`：维护增量可视图（visibility graph），在已知/未知混合环境里规划到目标（或探索目标）的全局路线，输出下一个 waypoint；核心思想是 *attemptable*（可尝试）：允许穿未知区域，走不通再重规划。
- `tare_planner`：自主探索（exploration）规划器，没有人类给的目标，自己决定“去哪里看”，采用局部/全局两层结构：局部层在当前区域找边界（frontier），全局层决定远距离的探索顺序。
- `path_follower`：跟踪一条给定路线（如示教轨迹），不负责避障。

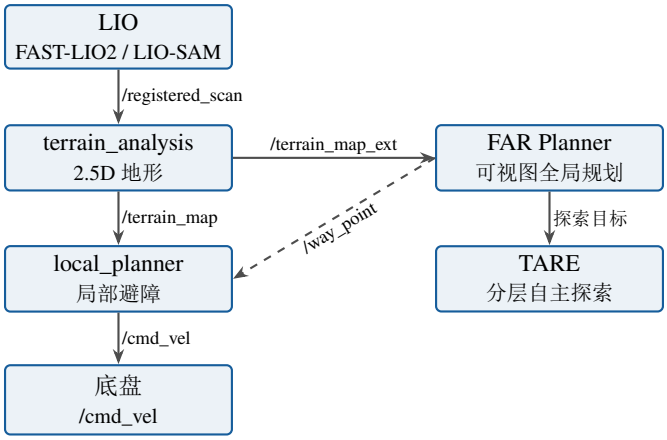


图 21.1: CMU 自主探索栈的模块数据流（简化）。LIO 输出配准后的点云，地形分析把它变成 2.5D 可通行性地图；局部规划器在局部地形图上避障并输出速度；FAR Planner 在全局可视图上选下一个目标点（`/way_point`）；TARE 负责“没有目标时自己找目标”的探索任务。`terrain_analysis_ext`、`path_follower` 与 `visualization_tools` 是辅助模块，图中省略。

- `visualization_tools`：RViz 可视化与调试工具。

表 21.1: CMU 栈主要话题（名字以仓库为准，这里给出最常见的）

话题	方向	内容
`/state_estimation`	输入	LIO 位姿（odom/map）
`/registered_scan`	输入	世界系下的配准点云
`/terrain_map`	中间	局部 2.5D 地形图
`/terrain_map_ext`	中间	扩展地形图（给 FAR）
`/way_point`	中间	局部规划的目标点
`/cmd_vel`	输出	底盘速度指令
`/exploration_paths`	可视化	TARE 的探索路径

21.3 地形分析：把点云压成 2.5D 高程图

定义 21.1 (地形可通行性分析)

地形可通行性分析（terrain traversability analysis）是给每个空间位置打一个“机器人能否安全通过”的分数或标签的过程。常用四个几何量：坡度（slope，局部平面与水平面的夹角）、粗糙度（roughness，局部高度起伏的统计量）、台阶高度（step height，相邻格之间允许的高度突变）、净空（clearance，到最近障碍物的距离）。四者都满足阈值时判为可通行。

21.3.1 2.5D 栅格的组织

地形图是一个栅格 $\{(i,j)\}$，分辨率 r 通常取 0.1–0.2 m，每格记录三样东西：高度 h_{ij}（格内点云 z 的中位数或最低值）、粗糙度 σ_{ij}（格内 z 的标准差）、以及净空 d_{ij}（到最近“障碍格”的平面距离，用距离变换一次算出）。点云到来时，把每个点按 (x,y) 投到格子，更新该格的统计量；机器人移动时用滑动窗口平移整张图（只重算新进入窗口的格子）。

21.3.2 坡度：从高度差到角度

对 4 邻接的两个格子，水平距离就是分辨率 r；高度差 $\Delta z = h_{i'j'} - h_{ij}$。它们连线与水平面的夹角就是坡度：

$$\theta_{\text{slope}} = \arctan\!\left(\frac{|\Delta z|}{r}\right). \tag{21.1}$$

对 8 邻接的对角邻居，水平距离是 $\sqrt{2}\,r$，分母换成 $\sqrt{\Delta x^2+\Delta y^2}$。例：$r=0.1$ m，相邻格高差 0.05 m，则 $\theta=\arctan(0.5)=26.6°$；若高差 0.1 m，则 $\theta=45°$。工程上坡道阈值常取 25°–30°（约 0.44–0.52 rad），因为再陡四轮底盘就开始打滑。

若用格内全部点拟合平面，设平面法向为 $\boldsymbol{n}$，坡度还可以写成

$$\theta_{\text{slope}}=\arccos\left(\frac{\boldsymbol{n}^{\top}\boldsymbol{e}_z}{\|\boldsymbol{n}\|}\right),\tag{21.2}$$

其中 $\boldsymbol{e}_z=(0,0,1)^{\top}$。法向用 PCA 求：对格内点 $\{\boldsymbol{p}_i\}$ 先算均值 $\bar{\boldsymbol{p}}$，再算协方差

$$\boldsymbol{C}=\frac{1}{N}\sum_{i=1}^{N}(\boldsymbol{p}_i-\bar{\boldsymbol{p}})(\boldsymbol{p}_i-\bar{\boldsymbol{p}})^{\top},\tag{21.3}$$

取最小特征值对应的特征向量即法向（这是第 7 章扫描匹配里“平面拟合”的同一套工具）。只有三个点时可以直接用叉积，例：$\boldsymbol{p}_1=(0,0,0)$、$\boldsymbol{p}_2=(1,0,0.1)$、$\boldsymbol{p}_3=(0,1,0.1)$，则

$$\boldsymbol{n}=(\boldsymbol{p}_2-\boldsymbol{p}_1)\times(\boldsymbol{p}_3-\boldsymbol{p}_1)=(1,0,0.1)\times(0,1,0.1)=(-0.1,-0.1,1),\tag{21.4}$$

归一化后 $\hat{\boldsymbol{n}}=(-0.0995,-0.0995,0.995)$，$\theta=\arccos(0.995)=5.7°$，说明这三个点所在的平面几乎是平的。

21.3.3 粗糙度与台阶高度

粗糙度用邻域高度的标准差衡量。设某格及其 3×3 邻域共 9 个高度值，均值 $\bar{h}$，则

$$\sigma=\sqrt{\frac{1}{N}\sum_{i=1}^{N}(h_i-\bar{h})^2}.\tag{21.5}$$

例：5 个高度 $0.02,0.01,0.03,0.00,0.04$（单位 m），均值 0.02，偏差为 $0,-0.01,0.01,-0.02,0.02$，平方和 $=10^{-3}$，方差 $=10^{-3}/5=2\times10^{-4}$，标准差 $\sigma=0.0141$ m，即起伏约 1.4 cm——很平。若 $\sigma>0.03$ m，通常是碎石或草地，应降低速度或判为不可通行。

台阶高度定义为邻域内最大高度突变：

$$h_{\text{step}}=\max_{(i',j')\in\mathcal{N}(i,j)}\left|h_{i'j'}-h_{ij}\right|.\tag{21.6}$$

路缘石 0.15 m、门槛 0.2 m 是典型值；轮式底盘能过的台阶通常不超过 0.1–0.2 m，超过就必须绕行。注意台阶与坡度的区别：0.15 m 的台阶在 0.1 m 分辨率下坡度是 $\arctan(1.5)=56°$，会被坡度阈值直接判死；但在 0.2 m 分辨率下可能被平均成 37°，于是需要台阶项单独把关。

21.3.4 可通行性判定与伪代码

算法：2.5D 地形可通行性判定（单帧）

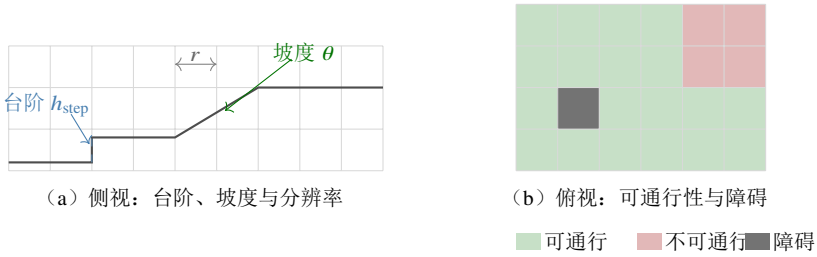
1. 对每个新点，按 (x,y) 投到格子 (i,j)，更新 h_{ij} 与 σ_{ij}
2. 对每个格子，用式 (21.1) 或式 (21.2) 算坡度 θ_{ij}
3. 用式 (21.6) 算台阶高度 h_{step}，用式 (21.5) 算粗糙度 σ
4. 用距离变换算净空 d_{ij}（到最近障碍格的距离）
5. 若 $\theta_{ij}>\theta_{\max}$ 或 $h_{\text{step}}>h_{\max}$ 或 $\sigma>\sigma_{\max}$ 或 $d_{ij}<r_{\text{robot}}$，标记为不可通行
6. 发布 `/terrain_map`：可通行点附带净空值 d_{ij}，供局部规划使用

关键参数的量级（以仓库默认配置为参考，不同平台要重调）：分辨率 $r\approx0.1$–0.2 m；坡度阈值 $\theta_{\max}\approx25°$–30°；台阶阈值 $h_{\max}\approx0.2$–0.3 m（对应代码里的“相对高度阈值”类参数，如 `maxRelZ` 在 0.2–0.3 量级）；粗糙度阈值 $\sigma_{\max}\approx0.03$–0.05 m；机器人半径 r_{robot} 按车体设。车越高、轮越大，这些阈值越宽松。

21.3.5 2.5D 与真 3D 的折中

2.5D 的优点是内存小、查询快、与 2D 规划器兼容（把“可通行性”当地图值，就能复用 A*/Dijkstra）。它的局限也很清楚：

- **一个 (x, y) 只能存一个高度**：桥洞、多层建筑、桌子下方会打架；工程上靠“分层高程图”或“多地图分层”补救。
- **悬空障碍**：树枝、管道只要在点云里，就会把下面的格子标成障碍；需要按高度带过滤（只保留 $[z_{\min}, z_{\max}]$ 的点）。
- **动态物体**：行人、其他机器人会被压成“高地”，导致地形图抖动；需要时间衰减与一致性滤波。



（a）侧视：台阶、坡度与分辨率　　（b）俯视：可通行性与障碍

图 21.2: 2.5D 地形分析的两个视角。左（a）：侧视，格子分辨率 r、台阶高度 h_{step}、坡度 θ 的定义；右（b）：俯视，每个格子按坡度/台阶/粗糙度/净空判为可通行（绿）、不可通行（红）或障碍（黑）。

21.4 局部规划与 FAR Planner

21.4.1 local_planner：在地形图上做带净空的局部规划

CMU 栈的局部规划器与第 20 章的 DWA/TEB 不同：它不是在速度空间采样，而是**在 2.5D 地形图上做带净空约束的图搜索 + 轨迹平滑**。核心步骤是：

1. 以机器人为中心取局部窗口（如 10 m × 10 m），把窗口内的可通行格建成图；
2. 给每个可通行格赋代价：离障碍越近代价越高（用 `/terrain_map` 的净空值），坡度越大代价越高；
3. 从当前位置向 `/way_point` 做图搜索（Dijkstra/A*），得到一条初始路径；
4. 沿路径做“碰撞模板检查”：把机器人的矩形/圆形足迹沿路径扫一遍，任何位置净空不足就剔除或绕行；
5. 对路径做速度规划与平滑（限制线速度、角速度、加速度），输出 `/cmd_vel`。

与 2D 局部规划相比，它多了两件事：**地形代价**（坡度陡的格子即使可通行也贵）与**三维足迹**（车体高度、离地间隙决定“能不能从悬空物下方过”）。这也是为什么它需要 `/terrain_map` 携带净空值，而不是只用 0/1 占据。

21.4.2 FAR Planner：可视图 + attemptable

定义 21.2 (可视图与 FAR Planner)

可视图（visibility graph）是以“关键点”为节点、以“两点之间无碰撞的直线段”为边的图，图上最短路就是多边形障碍环境中的几何最短路。**FAR Planner**（Fast, Attemptable Route Planner）[42] 是 CMU 提出的 3D 增量可视图规划器：节点是自由空间中的采样点，边是可通行的直线段，未知区域允许规划但标记为“可尝试”（attemptable），一旦观测到不通就删边重规划。

它在三件事上与经典 A* 不同：

1. **表示**：A* 在稠密栅格上扩展，FAR 在稀疏可视图上搜索。可视图节点数通常只有几百到几千，而 100 m × 100 m × 0.1 m 的栅格有 10^6 量级格子，所以 FAR 在大范围场景里快得多。

2. **增量维护**：机器人边走边看，FAR 只更新机器人附近的可视图——新观测到的自由区域加点加边，新观测到的障碍删点删边。更新是局部的，不需要每次重建整张图。
3. **attemptable**：这是 FAR 的核心思想。经典导航只走“已确认自由”的区域；但在未知环境里，如果坚持“确认后才走”，机器人只能原地等。FAR 把穿过未知区域的边标成“可尝试”，先规划、先走，走到新观测证明此路不通时，把这条边删掉、重新规划。这样既不会鲁莽撞墙（局部规划仍在实时避障），又不会因为未知而停滞。

算法：FAR Planner 的增量更新与规划

1. 收到新地形图 → 找出机器人附近新变为自由/障碍的格子
2. 对自由格：若离已有节点足够远，采样为新节点，插入 KD 树
3. 对每个新节点：与附近节点连线，做可见性检查；无碰撞则加边，穿过未知区则标为 attemptable
4. 对障碍格：删除落在障碍里的节点，删除与之相交的边
5. 在图上跑 Dijkstra（边权 = 长度 + 未知区域惩罚）
6. 输出路径上下一个 waypoint 给 local_planner；若 waypoint 不可达（走进死路），回到第 1 步重规划

可见性检查（visibility check）是 FAR 里最耗时的操作：对候选边 $\overline{AB}$，沿线段以地形分辨率采样，检查每个采样点所在格的净空是否大于机器人半径。设边长为 L、采样步长 δ，需要检查 $\lceil L/\delta \rceil$ 个点。优化手段是先用“包围盒 + 粗粒度占据检查”快速排除明显相交的边，只对可能可见的边做逐点检查。

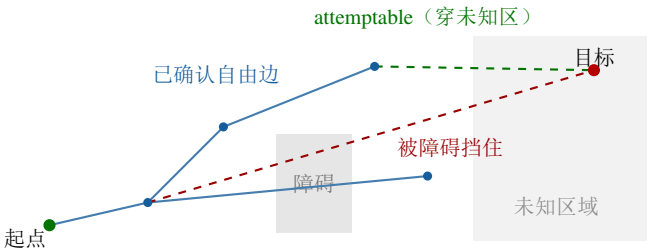


图 21.3: FAR Planner 的可视图。实线是已确认的可通行边，绿色虚线是穿过未知区域的 attemptable 边（先试着走，走不通再删），红色虚线是被障碍挡住、不允许的边。图是稀疏的，节点数远少于栅格数，因此能在大范围未知环境中保持实时。

21.4.3 TARE：从“导航”到“探索”

定义 21.3 (自主探索与 TARE)

自主探索（autonomous exploration）是指没有人工指定目标时，机器人自主选择“下一步去看哪里”，以最快速度把未知区域变成已知的过程。它的目标不是“到达某点”，而是“最大化信息增益 / 最小化剩余未知体积”。**TARE**（Tiered Autonomous Robot Exploration）[4] 是 CMU 栈里的探索规划器，用“局部 + 全局”两层结构解决大范围探索中的短视问题。

探索与导航的区别可以先从“目标从哪来”说起：导航的目标是人给的，探索的目标是算法自己算的。TARE 的目标选择分两层：

- **局部层**：在当前可视图上，从机器人附近的可视节点里找“能看到最多未知区域”的位置，计算探索增益（gain）——通常用“从该点射线投射能看到的未知体素数”近似；再除以路径代价（gain/cost），选比值最大的作为下一个 waypoint。增益/代价的比值让机器人优先去“近且能看很多”的地方。
- **全局层**：当局部区域没有可观的增益（当前房间看完了），再从一张低分辨率的全局地图上找远处的边界（frontier，已知与未知的交界），选一个全局目标，交给 FAR Planner 规划过去。这样避免局部层在“看完的房间里反复打转”，也避免一上来就冲向很远的未知区域。

算法：TARE 式分层探索主循环

1. 更新地图与可视图；若没有剩余未知区域，结束
2. 计算局部候选节点的探索增益 g_i 与路径代价 c_i
3. 若存在 g_i/c_i 超过阈值：选最优节点，令 FAR Planner 规划过去，执行
4. 否则：在全局低分辨率地图上找边界，选增益最大的边界作为全局目标
5. 若全局也没有边界：探索完成；否则规划到全局目标后回到第 1 步

“探索”这个词在 CMU 栈里还包含一层含义：**探索本身就是一个不断试错的过程**。TARE 依赖 FAR 的 attemptable 边去“试探”未知区域，走进去看到更多，再更新增益；这正是 FAR 与 TARE 配套使用的原因——一个负责“敢走”，一个负责“去哪”。

21.5 部署、调参与和 LIO 的配合

21.5.1 与 LIO 的分工

CMU 栈不自己估计位姿，它假设你已经在跑一个 LIO（仓库默认配 FAST-LIO2[41] 或 LIO-SAM），并且 LIO 发布：世界系（或 odom 系）下的配准点云 `/registered_scan` 与位姿 `/state_estimation`。地形分析直接消费配准点云，所以：

- **LIO 的漂移会变成地形图的“重影”**：同一条墙被扫成两层，坡度与净空都会算错，进而导致“幽灵障碍”。室外大范围场景要开回环（LIO-SAM）或用带全局优化的方案。
- **点云去畸变必须做好**：快速转向时点云拖尾会让坡度噪声变大，地形图抖动，局部规划误刹车。
- **外参要准**：雷达相对底盘的高度与俯仰角直接进入地形图；装歪 2°，10 m 外的高度就偏 0.35 m，足以把平地判成坡。

21.5.2 关键参数与常见问题

```
# Typical workflow (ROS 1 / ROS 2 packages share the same idea)
roslaunch vehicle_simulator system_garage.launch    # start simulator + stack
roslaunch terrain_analysis terrain_analysis.launch # 2.5D terrain map
roslaunch local_planner local_planner.launch       # local planner
roslaunch far_planner far_planner.launch           # visibility graph
roslaunch tare_planner tare_planner.launch         # exploration
# Key parameters (names vary slightly across versions):
#   vehicle_height / sensor_height  -> footprint and clearance
#   terrain grid resolution         -> 0.1-0.2 m
#   slope / step / roughness limits -> traversability thresholds
#   maxRelZ                         -> relative height tolerance (m)
```

常见问题与排查方向：**地形图抖动**（点云噪声或外参抖动，先查 LIO 与雷达安装，再加大地形滤波窗口）；**局部规划卡住**（净空阈值太保守或车体尺寸填大，先看 `/terrain_map` 里机器人周围是否被误判为窄）；**FAR 不更新**（`/terrain_map_ext` 没接上，或可视图更新半径小于移动速度 × 更新周期）；**探索原地转**（增益阈值过高或全局层没触发，检查边界提取）。这些问题与第 27 章的排查表是同一套方法：先看数据流有没有，再看数值对不对，最后才怀疑算法。

本章小结

1. 2D 导航把世界压成平面，丢掉了高度、垂直净空与地形质量；2.5D 高程图用“每格一个高度”在内存与表达力之间取得平衡，是地面 3D 导航的主流选择。
2. CMU 栈的分工是“地形理解与规划解耦”：LIO 给配准点云，terrain_analysis 出 2.5D 可通行性地图，local_planner 做带净空的局部规划，FAR Planner 做增量可视图全局规划，TARE 做分层自主探索。
3. 地形可通行性由坡度（式 (21.1)）、粗糙度（式 (21.5)）、台阶高度（式 (21.6)）与净空共同判定；分辨率会“平均掉”小台阶，所以台阶项要单独把关。
4. FAR 用稀疏可视图替代稠密栅格，用增量更新维持实时，用 attemptable 边“先走未知、走不通再删”，适合大范围未知环境。
5. TARE 用局部/全局两层解决探索的短视问题：局部比 gain/cost，全局找 frontier；探索的目标是信息增益而不是到达某点。
6. 这套栈依赖 LIO 的位姿与配准点云；漂移、去畸变、外参都会直接变成地形图误差，进而影响规划。

第二十一章 练习

1. 用式 (21.1) 计算：$r = 0.15$ m、相邻格高差 0.06 m 时的坡度；若阈值是 25°，该格可通行吗？
2. 为什么在 0.2 m 分辨率下台阶会被“平均”掉？请构造一个 0.15 m 台阶的例子，说明坡度项与台阶项判定的差异。
3. 2.5D 地图遇到“桥洞”会怎么错？给出两种工程上的补救方案。
4. FAR 的 attemptable 边与 A* 在未知栅格上“把未知当自由”有什么区别？为什么后者更危险？
5. TARE 的增益/代价比值中，如果只用增益、不看代价，机器人会表现成什么样？
6. 如果 LIO 出现约 1° 的俯仰角误差，10 m 外的高度误差是多少？这会怎样影响地形分析？

延伸阅读

- [4]：CMU 自主探索开发环境的系统论文，读它建立“地形分析—局部规划—FAR—TARE”的整体框架。
- [42]：FAR Planner 论文，重点看可视图的增量更新与 attemptable 策略的定义。
- [30]：ESDF 的增量构建，理解“距离场”之后再看 2.5D 净空会更容易。
- [41]：CMU 栈常用的 LIO 前端，理解它输出的配准点云是地形分析的输入。
- 代码仓库 caochao39/autonomous_exploration_development_environment：有 ROS 1/ROS 2 分支与仿真场景，按 README 跑一遍仿真比读十篇论文更直观。

第二十二章 2D 建图 + 导航完整链路实战

本章导读

前面各章把导航拆成定位、建图、全局规划、局部规划分别讲完了。但真正让机器人跑起来，难点往往不在单个算法，而在“把它们按正确的顺序、正确的坐标系、正确的频率接起来”。本章就是一次完整的工程串联：先用 Cartographer 建一张 2D 栅格地图并存成 pgm/yaml，再用 AMCL 在这张图上定位，最后用 Nav2 做规划与控制。我们会给出可用的配置骨架、仿真验证步骤，以及一张“现象 → 原因 → 解决”的排查表。学完本章，你应该能从零把一台差速底盘在仿真里跑通，并知道每个环节出错时该看哪里。

22.1 全链路总览

整条链路分两个阶段。**建图阶段**：跑 Cartographer[19]，订阅激光 `/scan` 与轮式里程计 `/odom`，边建图边发布 `map→odom` 变换，最终导出 pgm 图片与 yaml 描述文件。**导航阶段**：用 `map_server` 加载刚才的地图作为静态层，用 AMCL[12] 在图上做粒子滤波定位并发布 `map→odom`，Nav2[26] 的行为树调度全局规划与控制，输出 `/cmd_vel` 给底盘。

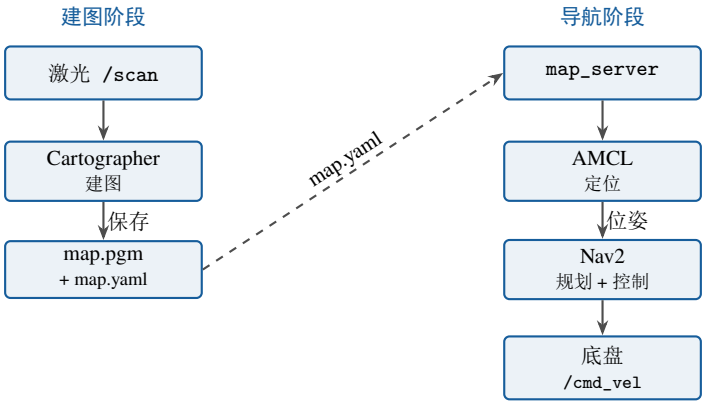


图 22.1: 2D 建图 + 导航全链路。左列是建图阶段：Cartographer 把激光与里程计变成地图文件；右列是导航阶段：map_server 提供先验地图，AMCL 提供定位，Nav2 完成规划与控制。两个阶段共用同一张地图与同一套坐标系约定；导航阶段 AMCL 同样订阅 `/scan`，图中为清晰起见省略。

理解这条链路要抓住一个关键点：**建图与导航是两个“换人不换地图”的进程**。建图时 `map→odom` 由 Cartographer 发布；切到导航后，必须把 Cartographer 关掉，改由 AMCL 发布 `map→odom`。如果两个节点同时发布这一段 TF，tf2 会认为树里有多个父节点，整棵 TF 树失效——这是新手最常踩的坑之一。

22.2 Cartographer 建图实操

22.2.1 配置文件

Cartographer 用 Lua 配置，2D 建图的入口参数通常从一个 `backpack_2d.lua` 派生。下面是一份精简的配置骨架（参数值按 0.05 m 分辨率、室内场景给出，可按场地调整）：

```
-- my_2d.lua : minimal Cartographer 2D configuration
include "backpack_2d.lua"

TRAJECTORY_BUILDER_2D.min_range = 0.3
```

```
TRAJECTORY_BUILDER_2D.max_range = 30.0
TRAJECTORY_BUILDER_2D.missing_data_ray_length = 5.0
TRAJECTORY_BUILDER_2D.use_imu_data = false
TRAJECTORY_BUILDER_2D.num_accumulated_range_data = 1
TRAJECTORY_BUILDER_2D.voxel_filter_size = 0.025
TRAJECTORY_BUILDER_2D.submaps.num_range_data = 90
TRAJECTORY_BUILDER_2D.submaps.grid_options_2d.resolution = 0.05

MAP_BUILDER.use_trajectory_builder_2d = true
POSE_GRAPH.optimize_every_n_nodes = 90
POSE_GRAPH.constraint_builder.min_score = 0.65
POSE_GRAPH.constraint_builder.global_localization_min_score = 0.7
```

几个参数值得解释：`min_range` 与 `max_range` 是激光可用距离，设错会把地面或远处噪点当障碍；`voxel_filter_size` 是体素降采样尺寸，越小越精细但越耗 CPU；`submaps.num_range_data=90` 表示每 90 帧激光构成一个子图，太大则回环约束少、建图容易漂；`resolution=0.05` 是最终栅格分辨率，10 m × 10 m 的场地对应 200 × 200 的栅格，正好与第 18 章的代价地图分辨率一致。若雷达有 IMU 且已标定，把 `use_imu_data` 打开能显著改善快速转向时的建图质量。

22.2.2 建图、结束与导出

启动建图后，遥控机器人把场地走一遍（尽量走小闭环、每个角落都扫到），然后结束轨迹并导出：

```
# ROS 2: start mapping (ROS 1 uses roslaunch cartographer_ros ...)
ros2 launch cartographer_ros backpack_2d.launch.py

# After finishing the tour: close trajectory 0, then write state
ros2 service call /finish_trajectory cartographer_ros_msgs/srv/FinishTrajectory \
  "{trajectory_id:␣0}"
ros2 service call /write_state cartographer_ros_msgs/srv/WriteState \
  "{filename:␣'${HOME}/maps/map.pbstream'}"

# Convert the pbstream into pgm + yaml for map_server
ros2 run cartographer_ros cartographer_pbstream_to_ros_map \
  -map_filestem=${HOME}/maps/map -pbstream_filename=${HOME}/maps/map.pbstream \
  -resolution=0.05
```

导出后得到 `map.pgm`（灰度图：白 = 自由、黑 = 占据、灰 = 未知）与 `map.yaml`（分辨率、原点、阈值）。`map.yaml` 里的 `origin` 是地图左下角在世界系中的位姿，`resolution` 必须与实际一致，否则导航时障碍会整体缩放或偏移——这类错误表现为“地图看起来对，但机器人总是撞同一面墙”。

22.2.3 建图质量自检

在建图现场就应该检查四件事，而不是等导航失败再回头：**墙是否直**（走一圈回到起点，墙若重影说明回环没生效）；**角落是否闭合**（房间四角应是直角，若歪说明里程计外参或时间同步有问题）；**动态物体是否被抹掉**（人走过留下的“鬼影”应被后续观测覆盖）；**未知区域是否太多**（大片灰色意味着没扫到，导航时会被当成障碍或直接判无路）。第 15 章讲的子图与回环机制，正是这些现象背后的原因。

22.3 Nav2 配置

22.3.1 参数文件骨架

Nav2 的参数集中在 `nav2_params.yaml`，按节点分段。下面是一份能跑起来的最小骨架（只保留关键项，实际使用请复制官方完整文件再改）：

```yaml
amcl:                       # localization on the saved map
  ros__parameters:
    use_sim_time: True
    min_particles: 500
    max_particles: 2000
    laser_model_type: likelihood_field
bt_navigator:               # behavior-tree scheduler
  ros__parameters:
    global_frame: map
    robot_base_frame: base_link
planner_server:             # global planning
  ros__parameters:
    expected_planner_frequency: 1.0
    planner_plugins: ["GridBased"]
    GridBased:
      plugin: "nav2_navfn_planner/NavfnPlanner"
      tolerance: 0.5
      use_astar: true
controller_server:          # local planning / tracking
  ros__parameters:
    controller_frequency: 20.0
    controller_plugins: ["FollowPath"]
    FollowPath:
      plugin: "dwb_core::DWBLocalPlanner"
      max_vel_x: 0.26
      max_vel_theta: 1.0
```

要点：`amcl` 的粒子数与激光模型决定定位稳定性，室内小场地 500–2000 个粒子够用；`planner_server` 的 `tolerance` 表示“离目标多近算到达”，设太小会在终点附近反复规划；控制频率取 `controller_frequency=20.0`，对应第 20 章说的实时要求；`max_vel_x=0.26` 是 TurtleBot3 的典型值，实车按自己底盘改。

22.3.2 地图、定位与生命周期

导航启动时，`map_server` 读 `map.yaml` 与 `map.pgm`，把它们发布成 `/map` 话题与静态层；AMCL 订阅 `/scan`、`/map`，发布 `map→odom` 与 `/amcl_pose`。**AMCL 需要一个初始位姿**：在 RViz 里用“2D Pose Estimate”点一下，或者在启动参数里给初始位姿；不给的话粒子会撒满整张地图，表现为“定位一直跳、规划路径乱飞”。

启动顺序由 `lifecycle_manager` 保证：每个 Nav2 节点都是生命周期节点，状态依次是“未配置 → 未激活 → 激活”；管理器按依赖顺序调用 `configure` 与 `activate`，避免“控制器先于代价地图就绪”这类竞争条件。这就是第 18 章说“Nav2 不是一个程序，而是一组被编排的节点”的工程含义。

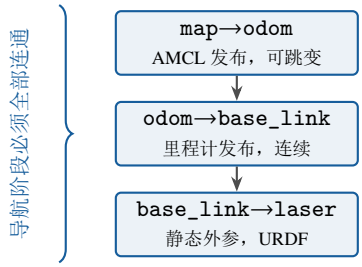


图 22.2: 导航阶段的 TF 链。三段变换分别由 AMCL、里程计、URDF 发布：map→odom 可跳变（重定位时瞬间修正），odom→base_link 必须连续（否则局部控制抖动），传感器外参静态。任何一段断开或重复发布，都会让整条链路不可用。

22.3.3 仿真验证

配置好之后，先在仿真里验证，再上实车。TurtleBot3 是官方示例中最省事的一套：

```
# Terminal 1: simulation world
export TURTLEBOT3_MODEL=burger
ros2 launch turtlebot3_gazebo turtlebot3_world.launch.py

# Terminal 2: navigation (map + AMCL + Nav2 + RViz)
ros2 launch nav2_bringup tb3_simulation_launch.py \
  map:=${HOME}/maps/map.yaml use_sim_time:=true

# Then in RViz: set "2D Pose Estimate", send a "2D Goal Pose"
```

仿真验证的判据是三个“能”：能在地图上定位（/amcl_pose 稳定、粒子云收敛）、能规划（全局路径在 RViz 里出现且不穿墙）、能跟踪（机器人沿路径走、遇到新障碍能绕）。三个都过，再上实车；实车上还要额外验证急停、通信丢包与电池电压低时的行为。

22.4 常见问题排查表

表 22.1: 全链路常见故障：现象 → 原因 → 解决

现象	可能原因	解决
地图漂移、墙重影	回环未触发、里程计标定差、时间不同步	建图走小闭环、查轮径/轮距与时间戳
定位跳变、粒子散开	未给初始位姿、激光与地图不匹配、场景变化大	给初始位姿、检查外参、必要时重定位
规划失败（无路径）	目标在膨胀区/未知区、tolerance 太小	调小 inflation_radius、给 1–2 格容差
机器人原地转	全局路径不可达或控制器无可行轨迹	看 /local_costmap 是否被堵、检查恢复行为
机器人不动	TF 断链、/cmd_vel 未下发、生命周期未激活	tf2_echo、ros2 topic hz、看日志
代价地图不更新	传感器话题没接、update_frequency=0	查 /scan 频率、查 layer 配置

表 22.1 的排查顺序建议固定成一套流程：先确认话题在不在（ros2 topic list 与 ros2 topic hz），再确认 TF 通不通（tf2_echo），然后确认数值对不对（在 RViz 里看地图、粒子与代价地图），最后才怀疑算法。绝大多数“导航不工作”都能在前两步定位，因为算法本身很少突然坏掉，坏掉的通常是数据流。

22.5 性能、安全与恢复行为

性能：20 Hz 控制频率下，主要 CPU 开销在代价地图更新（5 Hz）与局部规划（每周期几十条轨迹）；全局规划 1 Hz，开销小。若 CPU 超载，先降 `costmap` 的更新频率与分辨率，再降局部规划采样数，最后才降控制频率（控制频率一降，避障反应就慢）。

安全：Nav2 的 `behavior_server` 提供三类恢复行为——`spin`（原地旋转重新观测）、`backup`（后退脱困）、`wait`（等待障碍消失）。行为树里通常写成“规划失败 → 清空代价地图 → 旋转 → 再规划 → 后退”，层层兜底。但恢复行为不是万能的：如果定位已经错了，旋转只会让错误更大，所以**恢复行为之前必须确认定位健康**。

急停：工程上必须有独立于 Nav2 的急停链路（物理按钮或 `/cmd_vel` 上的看门狗），不能只依赖行为树。看门狗的逻辑很简单：超过 T 毫秒没收到新指令就下发零速度；T 一般取 200–500 ms，取太小会被正常抖动误触发，取太大则失去保护意义。

速度平滑：`velocity_smoother` 对 `/cmd_vel` 做限幅与一阶低通，保证加速度不超限；调它的时候要注意，平滑太强会让避障“变肉”，反应慢半拍。

本章小结

1. 全链路是“建图阶段”与“导航阶段”两段：Cartographer 出 pgm/yaml，map_server 加载，AMCL 定位，Nav2 规划控制；两段共用一张地图与一套坐标系。
2. 建图阶段结束要 `finish_trajectory` 并 `write_state` 导出 pbstream，再转成 pgm/yaml；`map.yaml` 的 `origin` 与 `resolution` 必须准确。
3. 导航阶段必须把 `map→odom` 的发布权从 Cartographer 交给 AMCL，禁止两个节点同时发布；AMCL 必须给初始位姿。
4. Nav2 参数按节点分段：AMCL 粒子数、planner 的频率与容差、controller 的 20 Hz 与速度上限是最常调的几项；生命周期管理器保证启动顺序。
5. 排查固定顺序：先话题、再 TF、再数值、最后算法；表 22.1 覆盖了最常见的六类故障。
6. 安全靠三层：行为树的恢复行为、独立急停/看门狗、速度平滑；恢复行为之前要先确认定位健康。

第二十二章 练习

1. 如果建图时忘了 `finish_trajectory` 直接 `write_state`，导出的地图可能有什么问题？
2. AMCL 不给初始位姿时，粒子会撒在哪里？为什么表现为“路径乱飞”而不是“机器人不动”？
3. 一个 10 m × 10 m 的场地，分辨率 0.05 m，地图 pgm 至少多大？若分辨率改成 0.1 m，建图与导航各有什么变化？
4. 机器人“原地转”与“不动”分别优先检查什么？请按本章的排查顺序各写三步。
5. 为什么急停不能只靠 Nav2 的恢复行为？给出一个恢复行为反而让情况变糟的具体场景。
6. 把本章的链路换成“用 FAST-LIO2 建 3D 图 + 导出 2D 栅格”的方案，需要额外做哪几步？

延伸阅读

- [19]：Cartographer 原始论文，理解子图、回环与位姿图之后再调 `backpack_2d.lua` 会有的放矢。
- [12]：AMCL 的自适应粒子数（KLD-Sampling），理解“为什么粒子数会自适应”有助于调定位。
- [26]：Nav2 设计论文，配置项与节点关系以它和官方文档为准。
- Nav2 官方文档 docs.nav2.org 与 Cartographer ROS 文档 google-cartographer-ros.readthedocs.io：参数默认值与 launch 示例。

第二十三章 Z-Localization 深度解析

本章导读

这一章解剖一套真实的、开源的、为比赛而生的定位系统：东北大学 T-DT 战队 2026 赛季开源的 Z-Localization。它不是又一篇论文的复现，而是一次工程取舍的完整展示——为什么把 LIO 和先验地图定位拆成前后端？为什么状态机要设四个状态？为什么地图里要存“点矩”而不是点？我们按“架构 → 前端 → 后端 → 使用流程 → 可借鉴之处”的顺序拆解，并把它与 FAST-LIO2、Point-LIO 放在一起对比。读完你应该能判断：什么场景该用它，什么场景该自己改。

23.1 背景与定位

Z-Localization 是东北大学（Northeastern University）T-DT 实验室在 2026 赛季 RoboMaster 超级对抗赛中使用的哨兵定位方案，赛后以 GPL-2.0 协议开源 [29]。它的官方描述是“一套面向移动机器人的 ROS 2 激光惯性定位栈，支持里程计、点云建图、先验地图定位、回环与重定位，也可接入 UWB 提供位置辅助”。

它解决的核心问题，是比赛场景下的三个现实约束：

1. **场地是动态的**：有对手、有弹丸、有观众，地图里不能有动态残影；
2. **先验地图不一定可用**：适应性训练阶段可能来不及建图，需要“无图也能跑”；
3. **定位必须可恢复**：云台转动、撞击、遮挡都可能导致里程计飞掉，系统要能自己找回来。

这些约束决定了它的架构与 FAST-LIO2 那种“一个紧耦合滤波器打天下”的方案不同——它把系统拆成了**前端 ZLIO** 与**后端定位**两部分。这个拆分本身就是本章最值得学习的设计。

23.2 总体架构：前后端分离

23.2.1 为什么拆

作者在设计说明中给了一个很实在的理由：直接把先验地图塞进 LIO，LIO 是紧耦合的，一旦发散很难判断——“有时候位置飞了，或者速度、加速度已经比较离谱了，才能发现里程计爆炸了”。

拆开之后，职责变得清晰：

- **前端（ZLIO）**：只负责在 **odom 系**里估计运动。它维护自己的局部地图，输出高频里程计、去畸变点云、以及每一帧的位姿。
- **后端**：只负责维护 map → odom 的变换。它从前方送来关键帧，把关键帧的全局位姿放进因子图优化，用先验地图配准、回环、UWB 提供全局约束。

这样做有三个好处：

1. **可观测性分析**：先验图配准的海森矩阵可以单独做可观测性分析，判断点云匹配是否退化、定位是否可信；
2. **减少高频大图访问**：前端不必在每次高频更新时访问巨大的先验地图，理论上更省算力；
3. **可恢复**：前端可以独立重启并重建 odom 系，全局位姿的恢复交给后端单独处理，不会“一崩全崩”。

23.2.2 与 SLAM 前后端的区别

注意这里的“前后端”与第 11 章 SLAM 的前后端不完全一样：

- SLAM 的前端 = 里程计 + 数据关联；后端 = 回环 + 全局优化。二者共同维护**一张地图**。
- Z-Localization 的前端 = LIO；后端 = 全局定位（图 23 章 23.1）。前端维护**局部地图**，后端维护**全局一致性**，二者通过关键帧观测连接。

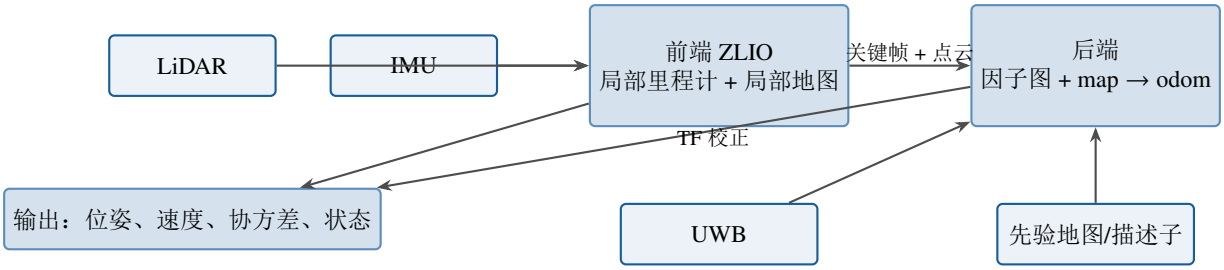


图 23.1: Z-Localization 的前后端架构。前端在 odom 系估计局部运动，后端维护 map 与 odom 之间的变换并输出全局位姿。

这是“定位系统语境下的前端/后端”——正是第 1 章表 1 中的第二种划分。

23.3 前端 ZLIO

前端要解决四件事：误差状态怎么建、怎么启动、怎么去畸变、地图怎么存。

23.3.1 误差状态建模

常见滤波式 LIO 沿用 ESKF 的“直积流形”建模：旋转用乘法更新，位置、速度、零偏用加法更新。ZLIO 采用 $\mathrm{SE}_2(3) \ltimes \mathfrak{se}(3)$ **半直积结构**，把导航状态与 IMU 零偏一起纳入误差变换，目的是让姿态和速度部分的无噪声误差得到精确的线性形式，减少线性化误差。

其误差方程（引自项目设计说明）为

$$\dot{\boldsymbol{\rho}}_p \simeq \boldsymbol{\rho}_v - \hat{\boldsymbol{p}} \times \boldsymbol{\beta}_g + \hat{\boldsymbol{p}} \times (\hat{\boldsymbol{R}} \boldsymbol{n}_\omega), \tag{23.1}$$

$$\dot{\boldsymbol{\theta}} \simeq -\boldsymbol{\beta}_g + \hat{\boldsymbol{R}} \boldsymbol{n}_\omega, \tag{23.2}$$

$$\dot{\boldsymbol{\rho}}_v \simeq \boldsymbol{g} \times \boldsymbol{\theta} - \boldsymbol{\beta}_a + \hat{\boldsymbol{v}} \times (\hat{\boldsymbol{R}} \boldsymbol{n}_\omega) + \hat{\boldsymbol{R}} \boldsymbol{n}_a, \tag{23.3}$$

$$\dot{\boldsymbol{\beta}}_g \simeq \left(\hat{\boldsymbol{R}}(\boldsymbol{\omega}_m - \hat{\boldsymbol{b}}_g)\right) \times \boldsymbol{\beta}_g - \hat{\boldsymbol{R}} \boldsymbol{n}_{bg}, \tag{23.4}$$

$$\begin{aligned} \dot{\boldsymbol{\beta}}_a \simeq & \left(\hat{\boldsymbol{R}}(\boldsymbol{a}_m - \hat{\boldsymbol{b}}_a) + \boldsymbol{g} + \hat{\boldsymbol{v}} \times \hat{\boldsymbol{R}}(\boldsymbol{\omega}_m - \hat{\boldsymbol{b}}_g)\right) \times \boldsymbol{\beta}_g \\ & + \left(\hat{\boldsymbol{R}}(\boldsymbol{\omega}_m - \hat{\boldsymbol{b}}_g)\right) \times \boldsymbol{\beta}_a - \hat{\boldsymbol{v}} \times (\hat{\boldsymbol{R}} \boldsymbol{n}_{bg}) - \hat{\boldsymbol{R}} \boldsymbol{n}_{ba}. \end{aligned} \tag{23.5}$$

这里 $\boldsymbol{\rho}_p, \boldsymbol{\rho}_v, \boldsymbol{\theta}$ 分别对应位置、速度、姿态的误差分量，$\boldsymbol{\beta}_g, \boldsymbol{\beta}_a$ 对应零偏误差。这个模型的推导细节可参考文献 [10]。对读者而言，重要的是理解它的动机：**标准 ESKF 在姿态误差较大时线性化误差明显，而把状态放在更“等变”的群结构上，可以让误差方程在更大范围内保持线性**。

23.3.2 不要求静止的初始化

传统 IMU 初始化要求机器人静止几秒，用加速度均值估计重力方向。但比赛里“定位飞掉重启时，云台可能还在跟随自瞄转动，几乎不能保证静止”。ZLIO 的初始化流程是：

1. 先用一小段 IMU 数据给出重力方向初值；
2. 通过 IMU 传播和点云粗配准得到初步轨迹；
3. 积累一段数据后，把**多帧点云的共面约束**与 **IMU 预积分**放在同一个窗口里联合优化，一起估计位姿、速度、IMU 零偏和重力方向；
4. 每轮优化后，用更新后的轨迹重新去畸变，再继续优化（类似迭代最近点 + 优化的交替）；
5. 收敛后，把坐标系对齐到重力方向，用窗口末帧状态建立滤波初值与初始地图，进入正常里程计更新。

这套流程的价值在于：**把“初始化”从“静止假设”中解放出来**。对任何需要在运动中启动的机器人，这都是可以直接借鉴的思路。

23.3.3 去畸变

ZLIO 的去畸变采用 DLIO[5] 的思路：用 IMU 构造**分段连续轨迹**，按每个点的采样时刻做运动补偿。具体做法是：

1. 把加速度转到 odom 系并加入重力，得到实际运动加速度；
2. 根据相邻 IMU 数据估计角加速度和加速度变化率；
3. 构造这段时间内的姿态与位置轨迹：**位置随时间按三次多项式变化，姿态由区间内变化的角速度积分得到**；
4. 每个点按自己的时间戳找到对应轨迹段，结合外参转换到 odom 系，再统一到本帧结束时刻的 IMU 系。

相比“假设帧内匀速”的简单做法，这能描述扫描期间角速度和加速度的变化，显著减少云台转动、机器人加减速时的点云畸变。计算上，同一 IMU 区间内的点可以复用轨迹参数，开销可控。

23.3.4 地图结构：用统计量代替点

ZLIO 的地图沿用 VoxelMap[43] 的结构：哈希表索引体素，每个体素内部用八叉树组织平面；若体素内点能用一个平面描述就保存平面，否则继续细分，直到满足平面条件或达到最大深度。平整区域用大体素，几何复杂处用小体素。

它真正改动的地方是**历史点的保存方式**：平面建立后，不再保存原始支持点，而是保存**点数、一阶矩、二阶矩**。以体素中心 $\boldsymbol{o}$ 为局部原点，记 $\boldsymbol{x}_i = \boldsymbol{p}_i - \boldsymbol{o}$，保存

$$N = \sum_i 1, \qquad \boldsymbol{s} = \sum_i \boldsymbol{x}_i, \qquad \boldsymbol{Q} = \sum_i \boldsymbol{x}_i \boldsymbol{x}_i^{\top}. \tag{23.6}$$

新点加入时，只需累加 1、$\boldsymbol{x}$、$\boldsymbol{x}\boldsymbol{x}^{\top}$。由这些累计量恢复平面中心与散布矩阵：

$$\boldsymbol{c} = \boldsymbol{o} + \frac{\boldsymbol{s}}{N}, \qquad \boldsymbol{A} = \frac{\boldsymbol{Q}}{N} - \frac{\boldsymbol{s}\boldsymbol{s}^{\top}}{N^2}, \tag{23.7}$$

再取 $\boldsymbol{A}$ 最小特征值对应的单位特征向量作为法向：

$$\boldsymbol{A}\boldsymbol{n} = \lambda_{\min}(\boldsymbol{A})\,\boldsymbol{n}, \qquad \|\boldsymbol{n}\| = 1. \tag{23.8}$$

达到更新上限后，进一步释放统计量，只保留匹配所需的平面信息。

这一改动为什么重要

原始点云地图的内存随点数线性增长，且每来一个新点都要重新遍历支持点拟合平面。改成“点矩 + 均值/协方差”后，每个平面只存固定大小的统计量，更新是 $O(1)$ 的加法。作者给出的实测结果是：Mid360 ramp 数据 CPU 时间减少 72.8%、峰值内存减少 86.6%；Mid70 TN07 数据分别减少 62.6% 和 90.7%。

23.3.5 面建模与残差

建图点的测量误差和位姿误差会影响拟合出的平面，因此平面的中心和法向也带有不确定性。除此之外，实际墙面、地面未必完全平整，即使平均平面估得很准，局部点仍可能偏离它。ZLIO 把这两部分都计入观测方差：从支持点沿法向的散布中扣除已知噪声能解释的部分，剩余方差记为 τ^2。

对变换到 odom 系的扫描点 $\boldsymbol{p}_i$，记测量协方差 $\boldsymbol{\Sigma}_i$、平面参数协方差 $\boldsymbol{\Sigma}_\pi$、残差对平面参数的雅可比 $\boldsymbol{h}_i$，则点面残差及其方差为

$$r_i = \boldsymbol{n}^{\top}(\boldsymbol{p}_i - \boldsymbol{c}), \tag{23.9}$$

$$\sigma_i^2 = \boldsymbol{n}^{\top}\boldsymbol{\Sigma}_i\boldsymbol{n} + \tau^2 + \boldsymbol{h}_i\boldsymbol{\Sigma}_\pi\boldsymbol{h}_i^{\top}. \tag{23.10}$$

多个点匹配到同一个地图平面时，会共享这份平面误差。如果把它们当作相互独立，点数越多，滤波器就越容易“过度相信”这个平面。ZLIO 的处理是**按平面分组**，保留平面参数误差带来的相关性。同组中两个不同点的残差协方差为

$$\mathrm{Cov}(r_i, r_j) \simeq \boldsymbol{h}_i\boldsymbol{\Sigma}_\pi\boldsymbol{h}_j^{\top}, \qquad i \neq j. \tag{23.11}$$

实现时在平面的三个误差自由度内联合处理同组观测（两个法向方向偏差 + 沿法向的位置偏差），消去共享平面误差后，再把整组观测对位姿的约束交给滤波器，从而避免构造随点数增长的大协方差矩阵。记同平面残差向量 $\boldsymbol{r}_\pi$、雅可比 $\boldsymbol{H}_\pi$，最终残差项为

$$\boldsymbol{R}_\pi = \operatorname{diag}\left(\boldsymbol{n}^\top \boldsymbol{\Sigma}_i \boldsymbol{n} + \tau^2\right) + \boldsymbol{H}_\pi \boldsymbol{\Sigma}_\pi \boldsymbol{H}_\pi^\top, \qquad E_\pi = \boldsymbol{r}_\pi^\top \boldsymbol{R}_\pi^{-1} \boldsymbol{r}_\pi. \tag{23.12}$$

这是全书里少见的、把“观测相关性”显式处理的例子。大多数开源 LIO 直接把点面残差当独立，点数一多就会过拟合平面；Z-Localization 的做法值得借鉴。

23.3.6 时空自适应

时间自适应：ZLIO 希望更新频率介于 FAST-LIO 的整帧更新和 Point-LIO 的逐点更新之间，并根据运动与观测信息量自动调整。它把点云按采样时刻切成连续小段，每段去畸变后做一次滤波更新。每次更新后，根据近期线加速度、角加速度和当前点面观测的信息矩阵决定下一次更新间隔。记机体系下加速度与角加速度为 $\boldsymbol{\kappa}$，同一坐标系下按匹配点数平均的信息矩阵为 $\bar{\boldsymbol{\Lambda}}$，允许误差为 ε，则二阶运动项估计的偏差为

$$e^2(\Delta t) \simeq \frac{\Delta t^4}{4} \boldsymbol{\kappa}^\top \bar{\boldsymbol{\Lambda}} \boldsymbol{\kappa}, \tag{23.13}$$

令其等于 ε^2 并解出

$$\Delta t_{\text{next}} = \operatorname{clip}\left(\left(\frac{4\varepsilon^2}{\boldsymbol{\kappa}^\top \bar{\boldsymbol{\Lambda}} \boldsymbol{\kappa}}\right)^{1/4}, \Delta t_{\min}, \Delta t_{\max}\right). \tag{23.14}$$

运动变化大、且这些变化会明显影响当前点面残差时，缩短间隔；反之延长间隔、积累更多点。本轮更新无效或分母为零时取上限。

空间自适应：为减少对固定采样分辨率的依赖，让同一套参数适应不同大小的场景，ZLIO 取保留时间窗口内代表点到雷达距离的中位数并平滑，作为场景尺度；场景越开阔，目标点数越多。再根据目标点数 N_k^* 与实际点数 N_k 的差，用 PD 控制调整采样体素边长：

$$e_k = N_k^* - N_k, \qquad \ell_{k+1} = \operatorname{clip}\left(\ell_k - K_{p,k} e_k - K_{d,k} \frac{e_k - e_{k-1}}{\Delta t_k}, \ell_{\min}, \ell_{\max}\right). \tag{23.15}$$

点数偏少时减小体素（保留更多点），偏多时增大体素。两个增益随场景尺度、点数误差及其变化调整。

23.3.7 不用在线动态点滤除

作者也尝试过在线滤除动态点，但没有找到能同时兼顾计算开销和滤除率的方法。比赛场景中定位特征更多来自墙壁和棚顶，加入动态点滤除前后的定位效果差距很小；考虑额外开销，最终没有在前端采用在线滤除，而是提供**离线**工具，在建图后清理保存的地图。

23.4 后端定位

23.4.1 因子图

后端因子图的每个节点对应一个关键帧，优化的是它在 **map 系**中的位置和姿态。图规模由固定时长的滑动窗口控制，窗口外的旧位姿通过边缘化移出（保留约束信息，见第 13 章）。开启 UWB 在线标定时，UWB 坐标系到 map 系的对齐以及时间偏置也作为共享变量持续保留在图中。

图中的因子有：

- **锚点先验**：初始化/恢复成功后，把可信位姿作为图中的锚点，形成位姿先验。
- **里程计因子**：前端提供相邻关键帧之间的相对运动，把轨迹前后连接起来。
- **先验图配准因子**：有先验地图时，扫描与地图配准提供当前关键帧的全局位姿及约束强度，用来校正前端漂移。
- **回环因子**：没有先验地图时，回环匹配提供当前帧与历史帧之间的相对位姿，修正累积误差。

- **UWB 因子**：约束天线位置，完成时间与坐标对齐后与 UWB 测量比较。
- **重力方向因子**：无先验地图且启用 UWB 时，用前端的重力方向约束横滚和俯仰。

把这套因子图与第 13 章的位姿图对照，你会发现它就是一个“带多种观测的位姿图”：节点是位姿，边是相对位姿约束，只是观测来源更丰富。

23.4.2 状态机

后端不仅做优化，还负责判断“现在信不信得过”。它定义了五个状态：

表 23.1: Z-Localization 后端的定位状态

状态	含义	进入/恢复条件
INITIALIZING	等待首次定位成功	启动时；成功后进入 NORMAL
NORMAL	定位正常	全局观测与预测一致，校正被接受
OBS_DEGRADED	点云质量差、观测弱	可用全局观测与预测冲突，但证据不足以否定当前位姿；拒绝冲突校正，继续用前端相对运动
SUSPECT	前端漂移、初值已偏	冲突持续达阈值，或速度异常、无法局部匹配；保留前端，用地图/描述子/UWB 重新找 TF
LOST	估计器故障或前端发散	SUSPECT 连续恢复失败；需重置前端、重建局部里程计，再用先验图或 UWB 找回全局位姿

判定逻辑分两步：先看点云能提供多少有效约束，再看配准/UWB 观测与当前预测是否一致。进入 SUSPECT 或 LOST 后，若已有可信位姿，输出会**冻结**在该位姿直到恢复成功——这比输出一个错误位姿要安全得多。这个“冻结而不是乱跳”的策略，在比赛里非常实用。

23.4.3 描述子

作者对描述子的理解很到位：“描述子本质上是在设计点云查询的键值对——用描述子作为键，关联地图中的位置和局部点云。新的点云到来时，先按描述子相似度找到候选位置，再通过点云配准确认位姿。”因此他们关心的是**相似度在候选位置上的分布**：正确位置附近能否形成足够尖锐、突出的峰值。

Scan Context（SC）[23]：以雷达为中心，按距离和方位把点云划分为环和扇区，每格记录最高点高度，形成二维描述；比较时尝试不同旋转角度，找到最相似的位置与朝向，为后续配准提供候选和 yaw 初值。SC 对高度分布敏感，生成和查询时都要先做重力矫正、把点云调平。比赛地图只有约 200 个描述子，因此检索直接遍历即可，无需 KD-tree。

G-PROBE[14]：复现了论文中的描述子与候选检索方法，面向 Avia、奥丁之眼这类**有限视场**点云。它把点云表示为带占据概率和不确定性的二维网格，并为地图准备不同方向的视场分支；检索时先筛候选，再在不同朝向下比较共同可见区域的占据与高度分布，同时降低那些在多个朝向上都相似的候选得分。作者指出这部分测试不充分、召回率未达 100%，建议结合自己的雷达视场调参、充分验证后再用。

23.4.4 配准与验收

正常跟踪时，后端积累近期点云，结合当前预测位姿选取附近的先验子图，做点到面配准。初值好时直接细配准；对应点不足或残差大时，先粗配准再回到细层。

弱观测下的求解参考了 DCReg 的思路。典型例子是实验室走廊：沿走廊前后移动一点，点云仍然贴住两侧墙面，配准很难判断这个方向的位置偏差。ZLIO 用**舒尔补**分别分析平移和旋转，再根据弱方向构造预条件器，改善求解的数值条件。

配准后，根据残差尺度和海森矩阵对位姿误差做**白化**：强方向权重大，弱方向权重小，不可观方向不增加约束。再结合图预测的不确定性，通过 **NIS**（归一化新息平方）和卡方检验判断是否接受这次观测。正常跟踪可

以使用部分方向的约束；初始化和重定位则要求**完整位姿**通过验收，再据此确定 TF 并建立锚点。第 2 章讲过，NIS 的二次型服从卡方分布，这里正是它的直接应用。

23.4.5 UWB 初始化

没有先验地图时，UWB 能提供位置，但单个位置无法确定前端 odom 系与全局坐标系之间的 yaw。ZLIO 把同一时刻的前端天线位置与 UWB 位置配对，通过一段运动求出两条轨迹之间的平移和 yaw，横滚俯仰沿用前端的重力方向。

对水平平移和 yaw 的简化对齐模型做可观测性分析：把轨迹去中心、假设位置噪声独立各向同性且方差相同，得到信息矩阵

$$\mathcal{I} = \frac{1}{\sigma_p^2}\begin{bmatrix} N & 0 & 0 \\ 0 & N & 0 \\ 0 & 0 & \sum_i \|\tilde{\boldsymbol{p}}_i\|^2 \end{bmatrix}, \tag{23.16}$$

于是 yaw 的标准差近似为

$$\sigma_\psi \approx \frac{\sigma_p}{\sqrt{\sum_i \|\tilde{\boldsymbol{p}}_i\|^2}}. \tag{23.17}$$

这个式子非常直观：停在原地时（$\tilde{\boldsymbol{p}}_i$ 都接近零）最后一项为零，yaw 无法确定；轨迹在水平面上展开后，航向信息迅速增加。所以“启动后跑开几米，yaw 就可观测了”不是玄学，而是式 (23.17) 的直接结论。

23.4.6 Reset

重置时，后端冻结当前输出并请求前端重置。前端完成重置、重新提供里程计和点云后，后端再接续或重新确定 map → odom 的 TF，恢复定位输出。作者坦言这套 reset 逻辑比较复杂，主要是为了尽量避免极端情况下的问题；如果场景简单，一个普通的重置服务可能就够了。

23.5 使用流程：从 bag 建图到先验定位

官方推荐的完整流程是：**回放建图 → 保存地图 → 自动清理 → CloudCompare 精修 → 网格生成地图包与描述子 → 先验定位**。关键命令如下（节选，完整参数见官方文档）：

Listing 23.1: Z-Localization 建图与定位的关键命令

```
# 1. replay a bag and build the map
ros2 launch bringup bringup.launch.py \
  frontend_config:="$PWD/config/lio.yaml" \
  backend_config:="$PWD/pcd/site.yaml" \
  mode:=mapping use_sim_time:=true rviz:=true

# 2. save the map bundle after the replay finishes
ros2 service call /backend_node/save_map std_srvs/srv/Trigger "{}"

# 3. remove dynamic ghost points offline (FreeDOM-style)
ros2 run backend clean_map_bundle \
  --in "$PWD/pcd/site_raw" --out "$PWD/pcd/site_auto_clean" \
  --lidar-origin -0.011 -0.02329 0.04412

# 4. rebuild map bundle and descriptors from the refined PCD
ros2 run backend build_pcd_map_bundle \
```

```
  --input "$PWD/pcd/site_refined.pcd" --out-bundle "$PWD/pcd/site_prior" \
  --x-min 0 --x-max 28 --y-min 0 --y-max 15 \
  --sensor-z 0.0 --bundle-step 1.5 \
  --map-voxel 0.15 --scan-voxel 0.4 \
  --descriptor-type scan_context --sc-lidar-height 0.35

# 5. run localization against the prior map
ros2 launch bringup bringup.launch.py \
  frontend_config:="$PWD/config/lio.yaml" \
  backend_config:="$PWD/pcd/site.yaml" \
  mode:=localization prior_map:="$PWD/pcd/site_prior" \
  use_sim_time:=true rviz:=true
```

流程里有几个值得注意的工程细节：

- **建图与定位用同一套入口**，只是 mode 不同，这减少了“两套代码不一致”的风险；
- **离线清理动态残影**参考 FreeDOM 的思路，作者实测“非常有效”，但仍不足以直接使用，需要 CloudCompare 手工收尾；
- **描述子按网格重新生成**，而不是建图时直接保存——尤其雷达水平安装时，这样生成的描述子质量更好；
- 保存地图包时保留**关键帧和局部点云**，后续清理需要它们；只存全局点云是不够的。

23.6 与其他 LIO 方案的对比

表 23.2: Z-Localization 与主流 LIO 方案的定位

方案	核心结构	全局定位方式	适合场景
FAST-LIO2	紧耦合 IEKF + iKD-Tree	无（需外接）	单机建图/里程计，算力中等
Point-LIO	逐点更新 + 无帧累积	无（需外接）	剧烈运动、IMU 易饱和
LIO-SAM	因子图 + 预积分 + 回环	回环	大范围建图，有回环
Z-Localization	前端 ZLIO + 后端因子图	先验图配准 + UWB + 回环	有场地先验、要求可恢复的比赛场景

选型时的判断逻辑：

1. 有没有**场地先验地图**？有 → 优先考虑 Z-Localization 这种前后端分离方案；没有 → FAST-LIO2/Point-LIO 先跑起来，或像 Z-Localization 那样用 UWB 过渡。
2. 会不会**剧烈运动/IMU 饱和**？会 → Point-LIO 或空间自适应采样；不会 → FAST-LIO2 足够。
3. 需不需要**定位失效后自动恢复**？需要 → 必须有后端状态机和重定位，这正是 Z-Localization 的核心价值。
4. 算力/内存是否紧张？紧张 → 注意其“用空间换时间”的后端设计（内存占用相对较高），可通过参数降低。

23.7 可借鉴的设计思想

即使你不用这套代码，它的几个设计决策也值得抄进自己的系统：

1. **把“局部估计”和“全局校正”解耦**：不要让一个紧耦合滤波器同时承担两件事，否则发散时你无法定位问题。
2. **状态机比算法更重要**：比赛里“输出冻结”往往比“输出一个略错的位姿”更安全。
3. **地图存统计量而不是点**：精度损失很小，内存和更新开销大幅下降。
4. **显式处理观测相关性**：同一平面的点不独立，忽略相关性会让滤波器过度自信。
5. **先验地图要“精修 + 重新生成描述子”**：地图质量决定定位上限。

6. **用可观测性分析指导调参**：弱方向、yaw 可观测性、NIS 检验，都是把“玄学”变成“可测量”的手段。

本章小结

1. Z-Localization 是面向比赛场景的 ROS 2 激光惯性定位栈，核心是**前端 ZLIO + 后端因子图**的分离架构。
2. 前端负责 odom 系内的局部估计：$\mathrm{SE}_2(3)\times\mathfrak{se}(3)$ 误差建模、不要求静止的初始化、DLIO 式去畸变、基于点矩的 VoxelMap。
3. 后端负责 map→odom 的变换：多源因子图、五状态状态机、Scan Context/G-PROBE 描述子、带舒尔补白化的配准与 NIS 验收、UWB 对齐、Reset。
4. 时空自适应让前端在“整帧更新”与“逐点更新”之间自适应，兼顾精度与资源。
5. 它的工程取舍（冻结输出、离线清图、描述子重建、统计量地图）比具体公式更值得学习。

第二十三章 练习

1. 为什么“把先验地图直接塞进 LIO”会导致发散时难以判断？拆成前后端后，哪些量变得可观测？
2. 由式 (23.17) 解释：为什么 UWB 初始化时“跑几米”就能确定 yaw？如果只走直线，yaw 能确定吗？
3. 点矩地图用 $N, \boldsymbol{s}, \boldsymbol{Q}$ 三个统计量恢复平面。请写出由它们计算平面中心与协方差矩阵的完整步骤。
4. 状态机里 OBS_DEGRADED 和 SUSPECT 的区别是什么？各自应该采取什么动作？
5. 如果你要把 Z-Localization 移植到只有 2D 雷达的机器人上，哪些模块需要改？哪些思想仍然适用？

延伸阅读

- [29]：项目主页与 `docs/design.md`，本章的一手材料。
- [43]：自适应体素地图的原始论文，理解地图结构的必读。
- [5]：连续时间去畸变思路的来源。
- [23]、[14]：两种描述子的原理与适用条件。
- [10]：$\mathrm{SE}_2(3)$ 等变误差建模的理论背景。

第二十四章 前沿方向与展望

本章导读

前面二十三章讲的都是“今天能直接拿来用”的方法。这一章换个视角：看看这个领域正在往哪走。我们不会罗列所有论文，而是挑出四条主线——语义与学习、多模态融合、长期地图维护、学习式导航——说清它们各自解决什么问题、现在到了什么程度、离工程落地还有多远。读完这一章，你应该能判断：哪些“新方法”值得投入时间，哪些还只是实验室里的玩具。

24.1 从几何 SLAM 到语义与学习

24.1.1 语义 SLAM

传统 SLAM 输出的地图是“几何的”：点、面、栅格，不知道哪里是墙、哪里是门、哪里是人。语义 SLAM 在几何之外再估计每个点的类别或每个物体的实例，好处有三：

- **动态物体剔除**：知道“这是人”，就可以把人的点从建图中剔除，而不是靠几何一致性猜；
- **更高层的导航**：可以下“去门口”这种指令，而不只是“去坐标 $(3, 2)$”；
- **回环与重定位**：语义描述子对光照、季节变化更鲁棒。

代价是引入了分割网络，算力开销大、标注依赖强、分割错误会污染地图。目前工程上常见的是“离线语义标注 + 在线几何定位”的折中。

24.1.2 学习式特征与描述子

第 12 章讲的 Scan Context 是手工设计的描述子。学习式方法用神经网络直接学“两帧是否同一地点”：

- 视觉侧：SuperPoint、SuperGlue、LoFTR 等把特征提取与匹配都交给网络，在弱纹理、大视角变化下明显优于 ORB；
- 激光侧：PointNet++、OverlapPredator、diffusion-based 描述子等直接处理无序点云。

问题是：模型需要大量数据、推理耗时、跨场景泛化不确定。比赛场景相对固定，手工描述子往往够用；跨季节、跨场地的长期建图才更能体现学习式方法的优势。

24.1.3 神经隐式表示

NeRF（神经辐射场）和 3D Gaussian Splatting（3DGS）用神经网络或高斯集合表示场景，能渲染出照片级的新视角图像。把它们用于 SLAM 是一个活跃方向：用位姿 + 图像联合优化辐射场，同时得到高保真地图和更精确的位姿。

- 优点：地图紧凑（网络权重或高斯参数）、可渲染、可做光度一致性约束；
- 缺点：训练/优化慢，实时性差，几何精度不一定优于点云，且对大规模场景显存压力大。

目前它更适合“离线高精地图重建”或“小场景实时”，离“比赛场上 10 Hz 定位”还有距离。

24.1.4 端到端里程计

DROID-SLAM 这类方法把特征提取、匹配、位姿优化放进一个可微的迭代网络，端到端训练。它在公开数据集上精度很高，但泛化到新传感器、新场景仍需微调，且算力需求远高于传统方法。对算法组而言，更现实的用法是“用学习式方法做前端的一部分（如特征、描述子、深度补全），后端仍用经典优化”。

24.2 多模态融合

第 9 章讲过 LiDAR + IMU 的紧耦合。前沿方向是把更多模态塞进同一个估计器：

- **LiDAR-视觉-惯性**：FAST-LIVO2[45]、R3LIVE[25] 把点到面残差与光度误差联合优化，在几何退化（长走廊、隧道）或视觉退化（黑暗、强光）时互相补位。
- **事件相机**：输出异步“亮度变化”事件，延迟极低、动态范围极大，适合高速运动与高动态场景；缺点是数据稀疏、传统视觉算法要重写。
- **4D 毫米波雷达**：能测距离、方位、俯仰、速度，雨雾烟尘下仍工作；缺点是点云稀疏、噪声大。
- **UWB 与 GNSS**：作为全局约束，与 LIO 融合（第 23 章 Z-Localization 已有实践）。

多模态融合的核心难点不是“把数据放一起”，而是**时间同步、外参标定、噪声建模、失效检测**。第 26 章讲的标定，在模态越多时越重要。

24.3 长期建图与地图维护

比赛只需要一张场地地图，但真实应用（仓储、园区、家庭）需要机器人在几个月甚至几年里持续使用同一张地图。这就带来“长期建图”问题：

- **多会话 SLAM**：多次建图的结果要能拼接、对齐到同一坐标系；
- **变化检测**：家具挪了、货架变了、墙刷了新漆，地图要能更新而不是被旧地图“带偏”；
- **地图压缩与版本管理**：点云地图动辄几个 GB，需要压缩、分层、增量加载；
- **定位鲁棒性**：场景变化后，描述子检索的召回率会下降，需要“变化感知”的重定位。

Z-Localization 的“先验地图包 + 描述子 + 可选 UWB”已经是一个简化版的多会话方案；工业界更完整的方案通常包含地图服务、版本管理、云端更新。

24.4 学习式导航

第 18–21 章的规划与控制都是“经典方法”：图搜索、采样、优化、MPC。前沿方向是用学习来做决策：

- **学习式局部规划**：用强化学习或模仿学习直接输出速度指令，在复杂动态场景中可能比手工规则更灵活；缺点是可解释性差、安全性难保证、训练成本高。
- **端到端导航**：从传感器输入直接到动作输出（如“视觉导航”），在特定场景（走廊跟随、人群穿行）表现不错，但泛化与安全仍是问题。
- **语义/任务导航**：输入“去厨房拿杯子”，系统自己拆解成探索、定位、抓取子任务，这是具身智能的核心问题，目前仍在实验室阶段。
- **多机器人协同**：多机共享地图、协同探索、任务分配，难点在通信受限下的一致性。

一个务实的判断：**短期内，学习式方法更可能作为经典规划器的“插件”（如学习式代价函数、学习式启发、学习式恢复行为），而不是整体替换**。经典方法提供安全保证与可解释性，学习提供灵活性与泛化。

24.5 对 RoboMaster 的启示

结合比赛场景，给算法组三条建议：

1. **先把经典方法吃透。** FAST-LIO2/Point-LIO、Cartographer、Nav2、Z-Localization 这些开源项目已经覆盖了比赛 90% 的需求。把它们的参数、失效模式、源码吃透，比追新论文的收益高得多。
2. **关注“可恢复性”与“可观测性”。**比赛里最致命的不是精度差 1 cm，而是定位飞了之后起不来。Z-Localization 的状态机与前后端分离思路，比任何新算法都值得抄。

3. **谨慎引入学习式方法**。如果要用，优先用在“离线”或“非安全关键”环节：离线建图清理、描述子训练、仿真环境生成。在线定位与规划仍应以经典方法为主。

本章小结

1. 语义 SLAM 在几何之外增加类别/实例信息，利于动态剔除、高层导航与重定位，但依赖分割网络。
2. 学习式特征、描述子、神经隐式表示在特定场景优于手工方法，但实时性、泛化与算力仍是瓶颈。
3. 多模态融合（LiDAR-视觉-惯性、事件相机、4D 毫米波）是提升鲁棒性的主要方向，核心难点是同步与标定。
4. 长期建图需要多会话对齐、变化检测、地图压缩与版本管理。
5. 学习式导航短期内更可能作为经典规划器的插件，而非整体替代。
6. 对比赛而言，吃透经典开源方案、重视可恢复性，比追新更重要。

延伸阅读

- [45]、[25]：多模态紧耦合的两个代表，读完能理解“视觉如何补几何”。
- [1]：第 11 章之后讨论了学习与估计的结合，适合理性看待“端到端”。
- 3D Gaussian Splatting 原始论文与开源实现：理解神经隐式表示的能力与局限。
- 近三年的 ICRA/IROS/RSS 综述：跟踪语义 SLAM、长期建图、学习式规划的最新进展。

第五部分

工程篇：把算法跑起来

第二十五章 ROS 2 工程实践

本章导读

本章解决一个非常具体的问题：把前面各章的算法，从论文里的公式变成能复现、别人能接手、能装到车上跑起来的工程。ROS 2 是 RM 算法组事实上的标准框架，但新手卡住的地方往往不是算法本身，而是编译不过、话题收不到、TF 查不到、参数改了没生效、时间戳对不上。本章按" 先跑通最小系统、再理解通信机制、最后建立工程习惯" 的顺序，把这些坑逐一填平。读完你应当能独立建一个工作空间、读懂 launch 与参数文件、看懂 TF 树，并用命令行工具定位大部分" 数据没来 / 频率不够 / 时间不对" 类的问题。本章是第 26 章标定与第 27 章调参的脚手架。

25.1 ROS 2 是什么：从节点到计算图

先回答" 为什么需要中间件"。一台 RM 机器人上，雷达驱动、IMU 驱动、LIO、代价地图、规划器、串口通信程序往往是**不同的进程**，甚至由不同人用不同语言写成。如果它们之间用共享内存或自定义 socket 直连，那么每加一个传感器、每换一个算法，就要改一遍所有程序的接口，这是灾难。中间件的作用就是把这些连接抽象成" 数据管道"：每个程序只声明" 我发布什么" 和" 我订阅什么"，连接由框架自动建立。

ROS 2 的通信层是 DDS（Data Distribution Service，数据分发服务）[27]。DDS 有两个关键特性：

- **以数据为中心**：程序发布的是" 名为 `/scan` 的数据流"，而不是" 发给某个地址的消息"。
- **去中心化**：没有 ROS 1 里那个必须最先启动的 `roscore`（master）。每个节点通过组播/单播自动发现对方，任何节点先启动、后启动都能连上。

一次运行中所有节点与它们之间的连接，构成一张**计算图**（computation graph）。图上的元素只有五种，先记住这张表，后面所有工程问题都能归到其中一类：

表 25.1: ROS 2 计算图的五种元素

元素	语义	典型例子
节点 node	一个进程内的计算单元（`rclcpp::Node`）	`fast_lio`、`controller_server`
话题 topic	单向、多对多、连续的数据流	`/lidar/points`、`/cmd_vel`
服务 service	一请求一响应的同步调用	清空代价地图、保存地图
动作 action	长任务，带反馈与取消	" 导航到某点"、`NavigateToPose`
参数 parameter	节点运行时的配置项	`max_vel_x`、`filter_size_surf`

ROS 2 与 ROS 1 的差异不是" 版本升级"，而是通信层换了一整套，这决定了工程写法的不同。表 25.2 列出最影响写代码的几处。

表 25.2: ROS 1 与 ROS 2 的关键差异

维度	ROS 1	ROS 2
通信层	自定义 TCPROS/UDPROS	DDS（可换实现：Fast DDS、Cyclone DDS）
节点发现	中心 master	分布式自发现
服务质量	无统一 QoS	每个话题/订阅可配 QoS
构建系统	catkin + `catkin_make`	ament + `colcon`
进程模型	单 master，节点可合进程	节点即进程，鼓励多进程与组合
实时与安全	基本没有	支持实时执行器、SROS2 加密

一句话理解

ROS 2 把” 程序之间怎么连” 从代码里搬到了配置里（话题名、QoS、参数、launch）。所以调试 ROS 2 问题时，先怀疑**连接配置**，再怀疑算法代码。

25.2 最小工作空间：colcon 编译流程

ROS 2 的代码组织单位是**功能包**（package），包的集合叫**工作空间**（workspace，通常命名为 `*_ws`）。标准目录结构是 `src/`（放源码）、`build/`（中间文件）、`install/`（最终产物）、`log/`（编译日志）。新手最容易犯的错误是把源码直接丢在 `ws` 根目录——`colcon` 只在 `src/` 下递归找包。

一个可用的最小流程如下（在 Ubuntu 22.04 + ROS 2 Humble 下测试）：

Listing 25.1: 创建并编译一个最小工作空间

```
# 1. create workspace
mkdir -p ~/rm_ws/src && cd ~/rm_ws/src
ros2 pkg create --build-type ament_cmake rm_lio \
    --dependencies rclcpp sensor_msgs nav_msgs

# 2. put your source into rm_ws/src/rm_lio/

# 3. build (symlink-install keeps config/launch editable without rebuild)
cd ~/rm_ws
colcon build --symlink-install --cmake-args -DCMAKE_BUILD_TYPE=Release
# build only one package and show full log when debugging
colcon build --packages-select rm_lio --event-handlers console_direct+

# 4. every new terminal must source BOTH layers
source /opt/ros/humble/setup.bash
source ~/rm_ws/install/setup.bash
ros2 pkg list | grep rm_lio
```

三点解释：

1. **source 顺序不能反**。`/opt/ros/humble` 是 underlay（系统安装的 ROS），`install/` 是 overlay（你自己编译的）。先 underlay 后 overlay，overlay 里的同名包才会覆盖系统包。反过来会出现” 改了代码但运行的是旧库” 的灵异现象。
2. `--symlink-install` 让 `install/` 里的 launch、YAML、URDF 变成指向 `src/` 的符号链接，改配置文件不用重新编译。C++ 源码仍然需要重编，但省掉了大量” 改个参数要编译五分钟” 的痛苦。
3. `--cmake-args -DCMAKE_BUILD_TYPE=Release` 非常重要。默认可能是无优化编译，LIO 这类点云算法会慢 2–5 倍，直接表现为” 雷达 10 Hz，里程计出不来”。

`package.xml` 与 `CMakeLists.txt` 里真正需要改的字段不多，见表 25.3。

25.3 话题、服务与动作：什么时候用哪个

三者都是” 节点之间的通信”，但语义完全不同。选错的典型后果是：用服务传 10 Hz 点云（每次请求都要等响应，吞吐崩掉），或者用话题做” 开始导航” 命令（没有反馈、不能取消）。

判断规则可以压缩成三条：

1. 数据是**连续产生、允许丢帧**的（点云、图像、IMU）⇒ 话题。

表 25.3: 功能包两个清单文件的关键字段

文件	字段	作用
`package.xml`	`<depend>`	编译 + 运行都需要的依赖，新手统一写这个即可
`package.xml`	`<exec_depend>`	只有运行期需要（如 Python 依赖、launch 包）
`CMakeLists.txt`	`find_package()`	找依赖；少一个就会在链接期报 undefined reference
`CMakeLists.txt`	`ament_target_dependencies()`	把依赖的头文件路径与库挂到目标上
`CMakeLists.txt`	`install(TARGETS ...)`	不写这行，`ros2 run` 找不到可执行文件
`CMakeLists.txt`	`install(DIRECTORY ...)`	不写这行，装好的包缺少 launch 与 yaml

表 25.4: 三种通信机制的选型表

机制	数据流形态	阻塞性	典型用途
话题	单向、多对多、持续	不阻塞（异步）	传感器流、`/cmd_vel`、`/tf`
服务	一请求一响应	调用方等待	保存地图、清空代价地图、切换参数
动作	长任务 + 周期反馈 + 可取消	可中断	`NavigateToPose`、机械臂抓取

2. 调用是**瞬时完成、需要知道成功与否**的（存图、重置）⇒ 服务。
3. 任务**持续若干秒、过程中要看进度、可能要中途放弃**的（导航、探索）⇒ 动作。

写代码时最容易忽略的是回调线程模型：话题回调运行在执行器（executor）的线程里。如果回调里做了 100 ms 的耗时计算（比如把点云转成另一套坐标系），那么同一执行器里的所有其他回调都会被拖慢，表现为”某个话题频率突然掉了”。第 25.7 节会专门讲这一点。

25.4 QoS：ROS 2 最容易踩的坑

QoS（Quality of Service）是 ROS 2 相对 ROS 1 最大的行为变化。它回答三个问题：

- **可靠性 reliability**：`reliable` 表示传输层保证送达（丢包重传）；`best_effort` 表示尽力而为，丢了就丢了。
- **历史 history + 深度 depth**：保存最近多少条消息。`keep_last` + `depth=1` 表示只留最新一条，`depth=10` 表示留 10 条。
- **持久性 durability**：`volatile` 表示只给”现在在线”的订阅者；`transient_local` 表示晚加入的订阅者也能拿到最后一条（类似”锁存”）。

为什么传感器数据要用 `best_effort`？因为雷达 10 Hz 出点云，单帧几十 MB；如果链路拥塞，`reliable` 会重传旧帧，越积越多，最后延迟爆炸。而定位算法只关心**最新**那帧，旧帧补过来也没用。反过来，控制指令`/ cmd_vel` 必须 `reliable`——丢一条可能就是撞墙。地图则要用 `transient_local`，因为 RViz 或导航节点可能在建图节点之后启动，它仍然需要那张已经发布过的地图。

表 25.5: 常见话题的推荐 QoS 配置

话题	reliability	history/depth	durability
`/lidar/points`	best_effort	keep_last / 5	volatile
`/imu/data`	best_effort	keep_last / 100	volatile
`/odom`、`/tf`	best_effort	keep_last / 10	volatile
`/map`	reliable	keep_last / 1	transient_local
`/cmd_vel`	reliable	keep_last / 1	volatile
`/initialpose`	reliable	keep_last / 1	volatile

QoS 不匹配是新手最常见、也最容易被忽略的故障。DDS 的兼容规则是：发布者提供的质量必须**不弱于**订阅者要求的质量。所以：

- 发布 `reliable`、订阅 `best_effort` ⇒ 兼容，能收到。
- 发布 `best_effort`、订阅 `reliable` ⇒ **不兼容**，一条也收不到。

此时节点不会崩，只是安静地没有数据。排查的命令是：

Listing 25.2: QoS 不匹配的诊断命令

```
# check publisher/subscriber QoS of a topic
ros2 topic info /lidar/points --verbose
# expected output shows "Reliability: BEST_EFFORT" for both sides

# is data actually flowing?
ros2 topic hz /lidar/points
ros2 topic echo /lidar/points --once
```

如果 `ros2 topic info --verbose` 显示”发布者 1 个、订阅者 1 个”但 `ros2 topic hz` 无输出，绝大多数情况就是 QoS 不匹配或话题名不一致（注意前导斜杠与命名空间）。

25.5 参数与 launch：把配置从代码里赶出去

参数是节点的运行时配置。C++ 里必须先声明再使用，声明时给出默认值：

Listing 25.3: 声明与读取参数

```
// in constructor or on_configure()
this->declare_parameter<double>("filter_size_surf", 0.5);
this->declare_parameter<int>("max_iteration", 4);
this->declare_parameter<std::string>("lid_topic", "/lidar/points");

this->get_parameter("filter_size_surf", filter_size_surf_);
this->get_parameter("max_iteration", max_iteration_);
// dynamic update: register a callback instead of polling
param_cb_ = this->add_on_set_parameters_callback(
    [this](const std::vector<rclcpp::Parameter> & params) {
        // validate before applying; reject invalid values
        return rcl_interfaces::msg::SetParametersResult{};
    });
```

参数覆盖的优先级从低到高是：代码默认值 < YAML 参数文件 < launch 文件中的参数 < 命令行 `-p name:=value`。所以”改了 YAML 没生效”的第一反应应该是：检查 launch 里是不是又写了一遍参数，或者命令行的 `-p` 把它压掉了。

launch 文件统一用 Python 写。一个典型的”一个 launch 起整套定位”如下：

Listing 25.4: 定位系统的 launch 文件片段

```
import os
from ament_index_python.packages import get_package_share_directory
from launch import LaunchDescription
from launch.actions import DeclareLaunchArgument
from launch.conditions import IfCondition
from launch.substitutions import LaunchConfiguration
from launch_ros.actions import Node
```

```
def generate_launch_description():
    pkg = get_package_share_directory("rm_lio")
    use_sim_time = LaunchConfiguration("use_sim_time")
    rviz = LaunchConfiguration("rviz")

    lio = Node(
        package="rm_lio", executable="lio_node", name="lio_node",
        output="screen",
        parameters=[os.path.join(pkg, "config", "lio.yaml"),
                    {"use_sim_time": use_sim_time}],
        remappings=[("points", "/lidar/points")],
    )
    rviz_node = Node(
        package="rviz2", executable="rviz2", condition=IfCondition(rviz),
        arguments=["-d", os.path.join(pkg, "rviz", "lio.rviz")],
    )
    return LaunchDescription([
        DeclareLaunchArgument("use_sim_time", default_value="false"),
        DeclareLaunchArgument("rviz", default_value="true"),
        lio, rviz_node,
    ])
```

`output="screen"` 让节点日志打到终端而不只是写进 `~/.ros/log`；`remappings` 是”不改源码就换话题名”的利器，跨团队联调时几乎必用。启动后检查参数是否真的生效：

Listing 25.5: 检查参数实际取值

```
ros2 param list /lio_node
ros2 param get /lio_node filter_size_surf
ros2 param dump /lio_node > /tmp/effective.yaml   # dump what is really used
```

`ros2 param dump` 是排错神器：它输出的是**生效后**的完整参数，可以直接和你的 YAML 对比。

25.6 TF2：坐标系的”宪法”

TF2 维护一棵随时间变化的**坐标系树**。定位系统的所有输出最终都要落到这棵树上，否则 RViz 里的点云会乱飞、Nav2 会报”代价地图收不到机器人位姿”。

TF 的关系是**父子**关系，每个变换有两个时间戳（发布时刻与有效时刻），查询时按时间插值。一台 RM 机器人的典型结构如图 25.1。

几个必须记住的约定：

- `map` 是全局连续但会跳变的坐标系（重定位时整体平移）；`odom` 是局部连续、绝不跳变的坐标系。所以**只有定位模块能发布** `map→odom`，且每次只发一个小修正，不能一次修正几米，否则规划器会看到机器人瞬移。
- `lookupTransform(target, source, time)` 返回的是”把 `source` 系下的向量变到 `target` 系”的变换，即 $T_{target,source}$。方向写反是外参错误最常见的来源。
- 传感器到机体的外参用 `static_transform_publisher` 发布，它们在标定后就是常数，见第 26 章。

Listing 25.6: TF 的发布、查询与诊断

```
# static extrinsics: lidar -> base_link, args: x y z yaw pitch roll
```

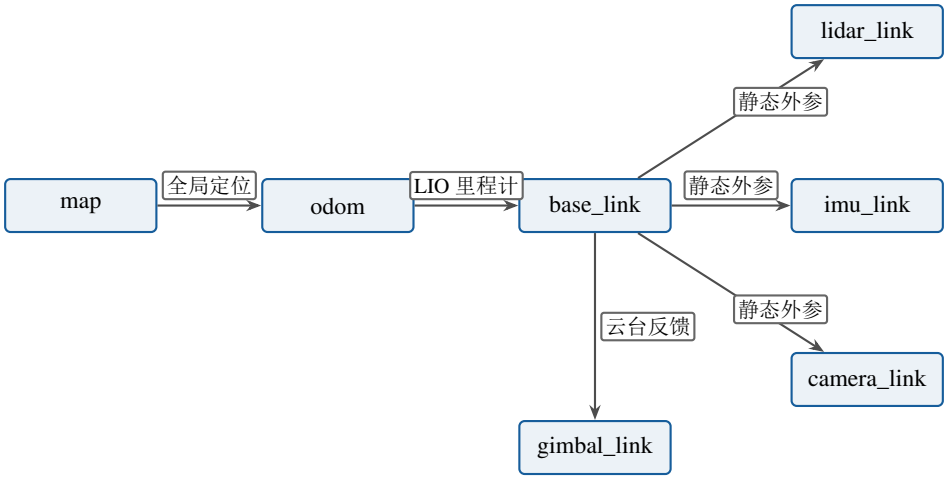


图 25.1: RM 机器人典型 TF 树。map→odom 由全局定位修正，odom→base_link 由里程计给出，传感器到机体的变换是常数外参。

```
ros2 run tf2_ros static_transform_publisher 0.15 0 0.30 0 0 0 \
    base_link lidar_link

# single query at a given time (prints translation and quaternion)
ros2 run tf2_ros tf2_echo map base_link

# draw the whole tree as a pdf and show per-link publish rates
ros2 run tf2_tools view_frames
```

在 C++ 中查询 TF 时，三种异常必须分别处理，否则要么崩溃，要么拿到错误数据：

Listing 25.7: lookupTransform 的三类异常

```
try {
    // TimePointZero means "latest available"; do NOT use it on recorded bags
    auto T = tf_buffer_->lookupTransform("map", "base_link",
                                         tf2::TimePointZero, tf2::durationFromSec(0.1));
} catch (const tf2::LookupException & e) {
    // frame name typo, or the publisher never existed
} catch (const tf2::ConnectivityException & e) {
    // both frames exist but no path connects them (missing static extrinsic)
} catch (const tf2::ExtrapolationException & e) {
    // requested time outside the buffer, or timestamps are out of order
}
```

笔记 时间戳乱序是 LIO 装车后非常常见的故障：点云和 IMU 来自不同驱动，若某一路的时间戳比另一路早了 200 ms，TF 缓冲会不断收到"过去"的数据，表现为 ExtrapolationException 间歇性爆发、RViz 中位姿抖动。处理办法不是调大缓冲区超时，而是回到第 26 章做时间同步。

25.7 调试工具与工程习惯

工具用对了，排错效率差一个数量级。表 25.6 是我自己常用的清单。

工程习惯比工具更重要，以下五条建议按重要性排序：

1. **用命名空间隔离多机/多模块**。launch 里加 namespace="sentry"，所有话题变成/ sentry/odom，避免与另一

表 25.6: ROS 2 常用命令行工具

命令	回答什么问题	备注
`ros2 node list`	有哪些节点活着	先确认进程起来了
`ros2 topic list -t`	有哪些话题、类型是什么	类型不匹配会无法连接
`ros2 topic hz`	频率是多少	判断驱动/算法是否跟得上
`ros2 topic echo --once`	消息内容对不对	看 `header.stamp` 与 `frame_id`
`ros2 topic info -v`	QoS 与连接数	QoS 不匹配的唯一快速判据
`ros2 param get/dump`	参数实际值是多少	确认 YAML 有没有被覆盖
`ros2 bag record -a`	把现场录下来	排错第一原则：先录包
`ros2 bag play --clock`	离线复现 + 仿真时间	配 `use_sim_time:=true`
`tf2_tools view_frames`	TF 树长什么样	有没有断链、哪条链路频率低
`ros2 doctor`	系统层面有没有异常	网络、RMW 实现、环境变量
`rqt_graph`	计算图里谁连谁	一眼看出没连上的节点
`rqt_plot`	数值曲线	调 PID、看速度指令

台车或另一个模块撞名。多机器人系统尤其要注意`/tf`与`/tf_static`必须显式 remap，否则全车 TF 混进一棵树。

2. **仿真时间要一致**。跑 bag 时必须`use_sim_time:=true`且播放时带`--clock`，所有节点的时间源才会统一为 bag 里的`/clock`。混用会导致 TF 全部超时。
3. **日志分级**。`RCLCPP_INFO`用于状态，`RCLCPP_WARN`用于可恢复异常，`RCLCPP_ERROR`用于必须处理的问题。默认级别 INFO，现场排查时用`--ros-args --log-level debug`临时打开，不要长期开 DEBUG（I/O 本身就拖慢实时性）。
4. **执行器与回调组**。默认单线程执行器下所有回调串行执行。对于" 点云处理 20 ms + 控制 1 ms" 这类混合负载，应当用`MultiThreadedExecutor`配合`MutuallyExclusive`回调组把控制与感知分开，否则控制频率会被点云拖垮。
5. **实时性**。对硬实时环节用`taskset -c 2,3`绑定 CPU 核，避免与编译、录包抢核；不要在回调里做内存分配与文件 I/O。

案例：定位节点频率从 20 Hz 掉到 5 Hz

现象：`ros2 topic hz /lio/odom`只有 5 Hz，但雷达 10 Hz 正常。**排查顺序：**(1)`ros2 topic hz /lidar/points`确认输入频率；(2)`top -H`看进程是否 CPU 打满，若单核 100% 则多半是编译没开 Release 或多线程配置错误；(3) 若 CPU 不满，看`ros2 topic info -v`是否有多个订阅者拖慢——`reliable`的慢订阅者会反压发布者；(4) 最后检查是否有人在同一执行器里做`ros2 bag record -a`，录包会把所有大消息序列化一遍。**结论：**先分清" 算不完" 还是" 收不到"，不要一上来就改算法参数。

本章小结

1. ROS 2 用 DDS 取代 ROS 1 的 master，节点自发现；计算图只有节点、话题、服务、动作、参数五种元素，所有工程问题都能归到这五类。
2. 标准编译命令是`colcon build --symlink-install --cmake-args -DCMAKE_BUILD_TYPE=Release`；每个终端必须按 underlay→overlay 顺序 source，否则改的代码不会生效。
3. 选通信机制的三条规则：连续数据用话题，瞬时调用用服务，长任务用动作。
4. QoS 中最关键的是 reliability 与 durability：传感器用`best_effort`，控制用`reliable`，地图用`transient_local`；发布弱、订阅强会一条都收不到。
5. 参数优先级是代码默认 < YAML < launch < 命令行；用`ros2 param dump`看实际生效值。
6. TF 树中`map`→`odom`由全局定位发布且只能小步修正，`odom`→`base_link`由里程计发布，传感器外参是静态变换。
7. 排查问题先录包（`ros2 bag record`），再离线用`topic hz`、`topic info -v`、`tf2_tools`、`rqt_graph`定位。

第二十五章 练习

1. 你的雷达节点发布/ `lidar/points` 用的是默认 QoS（reliable, depth 10），RViz2 订阅时选了 best_effort。能收到数据吗？反过来呢？请说明 DDS 的兼容规则。
2. 一个节点先于雷达驱动启动，它能在启动瞬间收到最后一帧点云吗？需要把哪个 QoS 项改成什么值？地图话题也存在同样问题，请解释为什么地图用 `transient_local`。
3. 写出 `map`→`base\_link` 的完整链式变换，并说明如果 `map`→`odom` 在某一帧突然跳了 1 m，规划器会出现什么现象、应该怎样限制单次修正量。
4. 你在 launch 里给 `lio_node` 传了 YAML，但 `ros2 param get` 显示还是默认值。列出至少三种可能原因。
5. 设计一个实验，证明” 点云回调耗时 20 ms” 是控制频率下降的原因，并给出两种不修改算法的缓解方案。

延伸阅读

- Macenski 等人的 ROS 2 综述 [27]：讲清楚 DDS 选型、实时执行器与 ROS 1 的取舍，读完能理解” 为什么 ROS 2 的 QoS 长这样”。
- ROS 2 官方文档的 About QoS Settings 与 TF2 教程：QoS 兼容表与 TF 广播/监听的权威说明。
- 第 22 章的 Nav2 工程链路：把本章的 launch、参数、TF 用法放到一个真实导航系统里再走一遍。

第二十六章 标定与时间同步

本章导读

标定是定位系统里”最不值得怀疑、却最常出错”的一环。算法调不好时，新手总怀疑滤波器，老手先怀疑标定：外参差 1°，10 m 处就是 17 cm；时间差 10 ms，2 m/s 时就是 2 cm，而且这类误差会随运动放大，表现为”直着走没事，一转弯就飘”。本章把标定拆成四件事：相机内参（张正友法）、IMU 噪声（Allan 方差）、多传感器外参（手眼标定 $AX = XB$）、时间同步。每件事都给到从假设、公式到工程判据的完整链条，读完你应该能自己标定一台车，并判断结果到底”能用”还是”不能用”。

26.1 为什么标定：误差是怎么被放大的

外参误差与时间误差都是与运动耦合的误差：静止时完全看不出来，一动就暴露。先算两个数建立量级感。

26.1.1 旋转外参误差：$d \sin\theta$

设雷达与机体之间存在未被补偿的旋转误差 θ（俯仰、横滚或偏航任一分量），则距离 d 处的点被放错的位置偏差约为

$$\delta = \|\boldsymbol{R}(\theta) - \boldsymbol{I}\|\, d = 2\sin\frac{\theta}{2}\, d \approx \theta d, \tag{26.1}$$

其中 $\boldsymbol{R}(\theta)$ 是绕某轴转 θ 的旋转矩阵；θ 很小时 $2\sin(\theta/2) \approx \theta$（弧度）。

表 26.1: 旋转外参误差造成的点位置偏差（cm，$\delta \approx \theta d$）

$\theta \setminus d$	1 m	3 m	5 m	10 m
0.5°	0.9	2.6	4.4	8.7
1°	1.7	5.2	8.7	17.5
2°	3.5	10.5	17.5	34.9
5°	8.7	26.2	43.6	87.2

工程含义有两条：近处点（1–3 m）对 1° 误差不敏感，**所以”看起来建图还行”不能证明外参对**；定位时匹配的是 8–15 m 外的墙与门，同样的 1° 就是 15–25 cm，表现为缓慢漂移或重定位错误。判据不是”图好看”，而是远处点云是否重影。

26.1.2 时间误差：$v\Delta t$

时间戳错位 Δt 等价于把测量放错位置，偏差近似为

$$\delta = v\,\Delta t, \tag{26.2}$$

其中 v 是该点相对机体的运动速度。

表 26.2: 时间不同步造成的位移偏差（mm，$\delta = v\Delta t$）

$\Delta t \setminus v$	0.5 m/s	1 m/s	2 m/s	4 m/s
1 ms	0.5	1	2	4
5 ms	2.5	5	10	20
10 ms	5	10	20	40
20 ms	10	20	40	80
50 ms	25	50	100	200

最容易被忽略的是**半帧偏移**：有的雷达把一帧的时间戳打在帧**起始**时刻，有的打在**结束**时刻，10 Hz 下二者差 100 ms，比”时钟不同步”严重一个数量级。拿到任何一款雷达，第一件事是查手册确认时间戳定义。

26.2 外参的数学定义与标定链

全书约定 $T_{AB} = (\boldsymbol{R}_{AB}, {}^{A}\boldsymbol{p}_{B})$ 表示”把 B 系向量变到 A 系”：${}^{A}\boldsymbol{p} = \boldsymbol{R}_{AB}{}^{B}\boldsymbol{p} + {}^{A}\boldsymbol{p}_{B}$。于是”雷达在机体系中的外参”记作 T_{BL}，反之 $T_{LB} = T_{BL}^{-1}$。

方向弄反是外参错误的第一大来源，症状很有迷惑性：点云整体旋转/镜像，但建图”自洽”（所有点一致地错），只有和 IMU、视觉对比时才暴露。两个防错手段：一是**物理检验**——雷达水平放置、正前方 5 m 放箱子，外参正确时应满足 $x \approx 5$、$y \approx 0$、$z \approx 0$（机体 x 前、y 左、z 上），任一符号反了立刻看出来；二是**只记一个方向关系**——T_{BL} 的下标”先 B 后 L”读作”L 到 B”，代码里 `extrinsic_T` 填 ${}^{B}\boldsymbol{p}_{L}$，`extrinsic_R` 填 $\boldsymbol{R}_{BL}$，点云先乘 $\boldsymbol{R}_{BL}$ 再加 ${}^{B}\boldsymbol{p}_{L}$。

标定量之间有依赖：手眼标定需要内参先标好，时间偏移又需要外参有初值。顺序与判据见表 26.3。

表 26.3: 标定链：顺序、工具与验收判据

顺序	标定量	常用工具	验收判据
1	相机内参、畸变	OpenCV / Kalibr	重投影 RMS < 0.5 px
2	IMU 噪声与零偏	imu_utils	重力模长 9.81 ± 0.1
3	相机-IMU 外参 + 时间偏移	Kalibr	重投影 < 1 px
4	雷达-IMU 外参 + 时间偏移	手眼标定	点云墙厚 < 2 cm
5	雷达-相机外参	标定板 + PnP	投影边缘偏差 < 2 px

26.3 相机内参：针孔模型与畸变

相机系 C 下的点 ${}^{C}\boldsymbol{P} = [X, Y, Z]^{\top}$（$Z$ 为深度）投到像素 $\boldsymbol{m} = [u, v]^{\top}$ 的过程是”先除以深度做透视，再乘内参矩阵”：

$$s\begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \boldsymbol{K}\begin{bmatrix} X \\ Y \\ Z \end{bmatrix}, \qquad \boldsymbol{K} = \begin{bmatrix} f_x & s_0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}, \tag{26.3}$$

其中 $s = Z$ 是齐次尺度，f_x, f_y 是以像素为单位的焦距（$f_x = f/d_x$，d_x 为像元尺寸），(c_x, c_y) 是**主点**，s_0 是像元倾斜项（多数相机可置零）。

数字例子：$f_x = 600$、$f_y = 540$、$c_x = 320$、$c_y = 240$，${}^{C}\boldsymbol{P} = [1.0, 0.5, 2.0]^{\top}$ m，则

$$u = 600 \times \frac{1.0}{2.0} + 320 = 620, \qquad v = 540 \times \frac{0.5}{2.0} + 240 = 375. \tag{26.4}$$

若深度估计错了 10%（Z 从 2.0 变 2.2），u 变成 592.7，横向误差 27 px——这就是单目深度误差致命的来源。

真实镜头不满足理想针孔。把像素反投影到归一化平面 $x = X/Z$，$y = Y/Z$，记 $r^2 = x^2 + y^2$，Brown–Conrady 模型把理想坐标映射成**含畸变**的坐标：

$$\begin{aligned} x_d &= x\left(1 + k_1 r^2 + k_2 r^4 + k_3 r^6\right) + 2p_1 xy + p_2\left(r^2 + 2x^2\right), \\ y_d &= y\left(1 + k_1 r^2 + k_2 r^4 + k_3 r^6\right) + p_1\left(r^2 + 2y^2\right) + 2p_2 xy, \end{aligned} \tag{26.5}$$

再乘内参得到像素。k_1, k_2, k_3 是**径向**畸变（$k_1 < 0$ 桶形，$k_1 > 0$ 枕形），p_1, p_2 是**切向**畸变（镜头与成像面不平行）。以 $k_1 = -0.28$、$x = y = 0.35$（视场边缘）为例，$r^2 = 0.245$，$x_d = 0.35(1 - 0.28 \times 0.245) = 0.326$，边缘向内拉 0.024 个归一化单位，按 $f_x = 600$ 算是 14.4 px。

26.4 张正友标定法：从单应到内参

张正友 1999 年的平面标定法是今天所有相机内参标定的基础：用同一块平面棋盘在**不同姿态**下拍若干张图，无需知道棋盘位姿即可解出内参。下面完整推导。

26.4.1 第一步：平面到图像的单应

把世界系定在棋盘平面上，$Z=0$，角点 ${}^{W}\boldsymbol{M}=[X,Y,0]^{\top}$，齐次形式 $\tilde{\boldsymbol{M}}=[X,Y,0,1]^{\top}$。投影为 $s\,\tilde{\boldsymbol{m}}=\boldsymbol{K}[\boldsymbol{R}\,|\,\boldsymbol{t}]\tilde{\boldsymbol{M}}$。把 $\boldsymbol{R}$ 按列拆开，利用 $Z=0$ 把第三列乘掉：

$$s\begin{bmatrix}u\\v\\1\end{bmatrix}=\boldsymbol{K}\begin{bmatrix}\boldsymbol{r}_1&\boldsymbol{r}_2&\boldsymbol{r}_3&\boldsymbol{t}\end{bmatrix}\begin{bmatrix}X\\Y\\0\\1\end{bmatrix}=\boldsymbol{K}\begin{bmatrix}\boldsymbol{r}_1&\boldsymbol{r}_2&\boldsymbol{t}\end{bmatrix}\begin{bmatrix}X\\Y\\1\end{bmatrix}. \tag{26.6}$$

于是每张图定义一个 3×3 的**单应矩阵**：

$$\boldsymbol{H}=\begin{bmatrix}\boldsymbol{h}_1&\boldsymbol{h}_2&\boldsymbol{h}_3\end{bmatrix}=\lambda\boldsymbol{K}\begin{bmatrix}\boldsymbol{r}_1&\boldsymbol{r}_2&\boldsymbol{t}\end{bmatrix}, \tag{26.7}$$

其中 λ 是齐次尺度。给 4 组以上角点对应（世界坐标已知、像素可检测），$\boldsymbol{H}$ 能用 DLT 线性解出（每对点 2 个方程，8 个未知量）。

26.4.2 第二步：用旋转矩阵的正交性构造约束

关键观察：$\boldsymbol{r}_1,\boldsymbol{r}_2$ 是旋转矩阵的列，满足**单位长度**与**相互正交**：$\boldsymbol{r}_1^{\top}\boldsymbol{r}_2=0$，$\|\boldsymbol{r}_1\|=\|\boldsymbol{r}_2\|=1$。由 $\boldsymbol{h}_i=\lambda\boldsymbol{K}\boldsymbol{r}_i$ 得 $\boldsymbol{r}_i=\lambda^{-1}\boldsymbol{K}^{-1}\boldsymbol{h}_i$，代入正交条件并约掉 λ^{-2}：

$$\boldsymbol{h}_1^{\top}\boldsymbol{K}^{-\top}\boldsymbol{K}^{-1}\boldsymbol{h}_2=0, \tag{26.8}$$

代入等长条件：

$$\boldsymbol{h}_1^{\top}\boldsymbol{K}^{-\top}\boldsymbol{K}^{-1}\boldsymbol{h}_1=\boldsymbol{h}_2^{\top}\boldsymbol{K}^{-\top}\boldsymbol{K}^{-1}\boldsymbol{h}_2. \tag{26.9}$$

定义对称矩阵 $\boldsymbol{B}=\boldsymbol{K}^{-\top}\boldsymbol{K}^{-1}\in\mathbb{R}^{3\times3}$，上面两式就是关于 $\boldsymbol{B}$ 的两个线性约束。**每张图给出两个约束**，而 $\boldsymbol{B}$ 只有 6 个独立元素，所以 3 张不同姿态的图即可确定 $\boldsymbol{B}$。

26.4.3 第三步：把约束写成线性方程组

把 $\boldsymbol{B}$ 的 6 个独立元素排成 $\boldsymbol{b}=[B_{11},B_{12},B_{22},B_{13},B_{23},B_{33}]^{\top}$。对任意两列 $\boldsymbol{h}_i,\boldsymbol{h}_j$，二次型可写成内积 $\boldsymbol{h}_i^{\top}\boldsymbol{B}\boldsymbol{h}_j=\boldsymbol{v}_{ij}^{\top}\boldsymbol{b}$，其中

$$\boldsymbol{v}_{ij}=\begin{bmatrix}h_{i1}h_{j1}\\h_{i1}h_{j2}+h_{i2}h_{j1}\\h_{i2}h_{j2}\\h_{i3}h_{j1}+h_{i1}h_{j3}\\h_{i3}h_{j2}+h_{i2}h_{j3}\\h_{i3}h_{j3}\end{bmatrix}\in\mathbb{R}^6. \tag{26.10}$$

式 (26.8)、(26.9) 于是成为 $\boldsymbol{v}_{12}^{\top}\boldsymbol{b}=0$ 与 $(\boldsymbol{v}_{11}-\boldsymbol{v}_{22})^{\top}\boldsymbol{b}=0$。$n$ 张图堆成 $\boldsymbol{V}\boldsymbol{b}=\boldsymbol{0}$，$\boldsymbol{V}\in\mathbb{R}^{2n\times6}$。

尺度不确定性是本质的：$\boldsymbol{K}$ 放大 c 倍，$\boldsymbol{B}$ 缩小 c^2 倍，约束完全不变。所以加 $\|\boldsymbol{b}\|=1$，取 $\boldsymbol{V}^{\top}\boldsymbol{V}$ 最小特征值对应的特征向量（即 $\boldsymbol{V}$ 最小奇异值对应的右奇异向量）。实践中 $n\geq5$ 更抗噪，标定软件默认 20–30 张。

26.4.4 第四步：由 B 闭式解出内参

记 $f_x=\alpha$、$f_y=\beta$、$s_0=\gamma$、$c_x=u_0$、$c_y=v_0$，Zhang 给出的闭式解为

$$\begin{aligned} v_0 &= \frac{B_{12}B_{13}-B_{11}B_{23}}{B_{11}B_{22}-B_{12}^2}, \qquad \lambda_0 = B_{33}-\frac{B_{13}^2+v_0\left(B_{12}B_{13}-B_{11}B_{23}\right)}{B_{11}},\\ \alpha &= \sqrt{\frac{\lambda_0}{B_{11}}}, \qquad \beta = \sqrt{\frac{\lambda_0 B_{11}}{B_{11}B_{22}-B_{12}^2}}, \qquad \gamma = -\frac{B_{12}\alpha^2\beta}{\lambda_0}, \qquad u_0 = \frac{\gamma v_0}{\beta}-\frac{B_{13}\alpha^2}{\lambda_0}. \end{aligned} \tag{26.11}$$

λ_0 正是那个被归一化掉的尺度；所有内参只依赖 $\boldsymbol{B}$ 元素之间的**比值**，因此尺度自动约掉。当像元近似正方形（$\gamma\approx 0$，即 $B_{12}\approx 0$）时退化为好记的形式：$u_0=-B_{13}/B_{11}$，$v_0=-B_{23}/B_{22}$，$\alpha=1/\sqrt{B_{11}}$，$\beta=1/\sqrt{B_{22}}$。

26.4.5 第五步：每张图的外参

由 $\boldsymbol{K}^{-1}\boldsymbol{h}_1=\lambda\boldsymbol{r}_1$ 取模长得 $\lambda=1/\left\|\boldsymbol{K}^{-1}\boldsymbol{h}_1\right\|$，于是

$$\boldsymbol{r}_1=\frac{\boldsymbol{K}^{-1}\boldsymbol{h}_1}{\left\|\boldsymbol{K}^{-1}\boldsymbol{h}_1\right\|}, \qquad \boldsymbol{r}_2=\frac{\boldsymbol{K}^{-1}\boldsymbol{h}_2}{\left\|\boldsymbol{K}^{-1}\boldsymbol{h}_2\right\|}, \qquad \boldsymbol{r}_3=\boldsymbol{r}_1\times\boldsymbol{r}_2, \qquad \boldsymbol{t}=\frac{\boldsymbol{K}^{-1}\boldsymbol{h}_3}{\left\|\boldsymbol{K}^{-1}\boldsymbol{h}_1\right\|}. \tag{26.12}$$

两个细节：$\boldsymbol{r}_1,\boldsymbol{r}_2$ 因噪声不一定严格正交，$\boldsymbol{r}_3$ 用叉积保证右手系；λ 的符号由”棋盘在相机前方”确定。最后对 $[\boldsymbol{r}_1\ \boldsymbol{r}_2\ \boldsymbol{r}_3]$ 做 SVD 投影到最近的旋转矩阵：$\boldsymbol{R}=\boldsymbol{U}\boldsymbol{V}^{\top}$，其中 $\boldsymbol{U}\boldsymbol{\Sigma}\boldsymbol{V}^{\top}$ 是那个近似矩阵的 SVD。

26.4.6 第六步：最大似然细化

前五步是最小化**代数误差**，不是真正的重投影误差，且还没考虑畸变。真正的目标是让所有角点的重投影误差平方和最小：

$$J\left(\{\boldsymbol{K},k_1,k_2,p_1,p_2\},\{\boldsymbol{R}_i,\boldsymbol{t}_i\}\right)=\sum_{i=1}^{n}\sum_{j=1}^{m}\left\|\boldsymbol{m}_{ij}-\hat{\boldsymbol{m}}\left(\boldsymbol{K},k_{1:3},p_{1:2},\boldsymbol{R}_i,\boldsymbol{t}_i,{}^{W}\boldsymbol{M}_j\right)\right\|^2, \tag{26.13}$$

其中 $\boldsymbol{m}_{ij}$ 是第 i 张图第 j 个角点的检测像素，$\hat{\boldsymbol{m}}$ 由式 (26.3)、(26.5) 算出。用 Levenberg–Marquardt 求解，初值取前五步结果（畸变初值取 0）。未知量 $4+5+6n$ 个，单目 20 张图约 129 个，OpenCV 利用稀疏结构加速。

判据：$\sqrt{J/(nm)}$ 即”重投影 RMS”，单位像素。<0.3 px 很好，0.3–0.5 px 正常，>1 px 说明角点检测或模型有问题。一组 640×480 相机的正常结果是 $f_x=615.3$、$f_y=616.1$、$c_x=322.7$、$c_y=241.2$、$k_1=-0.2887$、$k_2=0.0721$、$p_1=0.0003$、$p_2=-0.0002$、RMS 0.31 px——注意 $f_x\approx f_y$、主点接近图像中心；若主点偏离中心超过图像宽度的 10%，先查是不是角点顺序错了。

实践要点：(1) 图要覆盖画面四角与中心，畸变系数靠边缘样本；(2) 平面与像平面夹角取 30°–60°，全正对着拍会让单应退化；(3) 15–30 张，包含平移、旋转、远近变化；(4) 板子必须平且够大，纸贴不平会让 RMS 永远卡在 1 px 以上；(5) 标定前后不能动对焦环。

26.5 IMU 标定与 Allan 方差

陀螺与加速度计的测量可写成

$$\tilde{\boldsymbol{\omega}}=\left(\boldsymbol{I}+\boldsymbol{S}_g\right)\boldsymbol{M}_g\boldsymbol{\omega}+\boldsymbol{b}_g+\boldsymbol{n}_g, \qquad \tilde{\boldsymbol{a}}=\left(\boldsymbol{I}+\boldsymbol{S}_a\right)\boldsymbol{M}_a\boldsymbol{a}+\boldsymbol{b}_a+\boldsymbol{n}_a, \tag{26.14}$$

其中 $\boldsymbol{S}$ 是尺度因子误差，$\boldsymbol{M}$ 是轴间非正交，$\boldsymbol{b}$ 是零偏，$\boldsymbol{n}$ 是白噪声。零偏造成随时间线性累积的漂移，可标定也可在线估计；尺度/轴间是常值矩阵误差，绕一圈会表现为”转 90° 实际积分出 91°”；白噪声不可标定，只能用统计量描述。

零偏与尺度的六面法：把 IMU 静置在六个正交朝向各 30–60 s。加速度计每面测得模长应为 9.81，用六面数据可同时解出 $\boldsymbol{b}_a$ 与 $\boldsymbol{S}_a$（$\boldsymbol{M}_a$ 一般取单位阵）。陀螺零偏取长时间静置的均值，但要注意**零偏随温度漂移**，规范做法是上电预热 5–10 分钟再取。

26.5.1 Allan 方差：定义

白噪声与零偏这类随机过程不能用一个数描述，要看它在不同时间尺度上的表现。设采样间隔为 τ_0，得到序列 $y[1],\dots,y[N]$（陀螺为角速度，加速度计为比力）。取簇长度 m，把序列分成 $K=\lfloor N/m\rfloor$ 个不重叠的簇，簇内平均为

$$\bar{y}_k=\frac{1}{m}\sum_{i=1}^{m} y\big[(k-1)m+i\big],\qquad k=1,\dots,K, \tag{26.15}$$

对应平均时间 $\tau=m\tau_0$。**Allan 方差**定义为相邻簇平均之差的方差的一半：

$$\sigma_A^2(\tau)=\frac{1}{2(K-1)}\sum_{k=1}^{K-1}\big(\bar{y}_{k+1}-\bar{y}_k\big)^2,\qquad \sigma_A(\tau)=\sqrt{\sigma_A^2(\tau)}. \tag{26.16}$$

为什么除以 2？若相邻簇平均独立、方差各为 σ^2，差值方差是 $2\sigma^2$，除以 2 正好还原单点方差。为什么用” 相邻差分” 而不用” 减去总均值”？因为后者会把零偏漂移当成正常波动滤掉，而 Allan 方差要保留慢变信息。

它与功率谱密度的关系是

$$\sigma_A^2(\tau)=4\int_0^{\infty} S_y(f)\,\frac{\sin^4(\pi f\tau)}{(\pi f\tau)^2}\,\mathrm{d}f, \tag{26.17}$$

传递函数相当于一个随 τ 变形的带通滤波器：τ 小只看高频，τ 大只看低频。**所以一条 $\sigma_A(\tau)$ 曲线能同时分离出好几类噪声**，这正是它的价值。

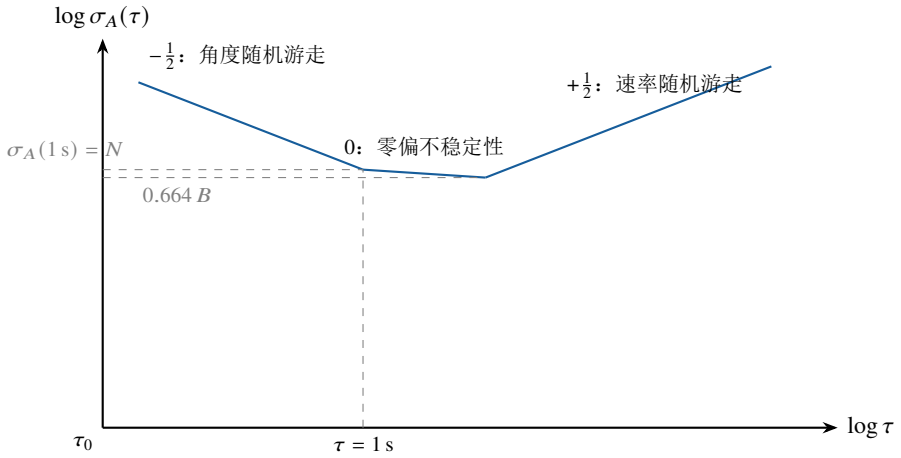


图 26.1: Allan 偏差曲线的双对数示意。左段斜率 $-1/2$ 是白噪声（角度/速度随机游走），中间平台是零偏不稳定性，右段斜率 $+1/2$ 是速率随机游走。

26.5.2 怎么从曲线上读参数

各类噪声的双对数斜率不同，可以逐段拟合（表 26.4）。

表 26.4: Allan 偏差的斜率与噪声类型（y 为角速度或比力）

噪声类型	$\sigma_A(\tau)$ 的形式	斜率	参数读法
量化噪声	$\sqrt{3}\,Q/\tau$	-1	左端最陡段
角度（速度）随机游走	$N/\sqrt{\tau}$	$-1/2$	$\tau=1$ s 处数值 $=N$
零偏不稳定性	$0.664\,B$	0	平台高度 /0.664
速率随机游走	$K\sqrt{\tau/3}$	$+1/2$	右段斜率拟合
漂移率斜坡	$R\tau/\sqrt{2}$	$+1$	最右端上扬段

读数口诀（只记这一条就够用）

$\tau = 1\,\mathrm{s}$ **处读到的** σ_A**，数值上就是这个传感器的噪声密度**（陀螺单位 °/s/$\sqrt{\text{Hz}}$，加速度计单位 m/s^2/$\sqrt{\text{Hz}}$）。陀螺的噪声密度乘 60 即得常用的 °/$\sqrt{\text{h}}$。

数字例子：某 MEMS 陀螺静置 2 h 的 Allan 曲线在 $\tau = 1\,\mathrm{s}$ 处读得 $\sigma_A = 0.014\,°/\mathrm{s}$，换成 SI 单位并换算成角度随机游走：

$$N_g = 0.014 \times \frac{\pi}{180} = 2.44 \times 10^{-4}\ \mathrm{rad/s/\sqrt{Hz}}, \qquad N_g \times 60 = 0.84\ °/\sqrt{\mathrm{h}}, \tag{26.18}$$

与 datasheet 的 0.014 °/s/$\sqrt{\text{Hz}}$ 一致。若 IMU 以 $f_s = 200\,\mathrm{Hz}$ 输出，带宽取 $f_s/2$，则单样本标准差 $\sigma = N_g\sqrt{f_s/2} = 2.4 \times 10^{-3}\,\mathrm{rad/s} \approx 0.14\,°/\mathrm{s}$。曲线平台在 $\tau \approx 50\,\mathrm{s}$ 处约 1.5 °/h，得零偏不稳定性 $B = 1.5/0.664 \approx 2.3\,°/\mathrm{h}$，即静止时零偏在几十秒尺度上有约 $1.1 \times 10^{-5}\,\mathrm{rad/s}$ 的随机浮动。

26.5.3 从 Allan 读数到滤波器参数

LIO/VIO 里要填的是协方差，不是曲线本身，转换只有两步：测量噪声取 N_g^2、N_a^2（FAST-LIO 的 `gyr_cov`、`acc_cov`），零偏随机游走取速率随机游走系数的平方 K^2（`b_gyr_cov`、`b_acc_cov`）。不同开源仓库对" 连续时间噪声密度" 的约定差一个 Δt 的幂次，**同一个 IMU 在不同仓库的最优值可能相差一个常数因子，这是正常的**。工程做法：用 Allan 读数给量级初值，再在同一段 bag 上微调——噪声项调大更信 IMU，调小更信点云（见第 27 章）。

26.6 雷达-IMU 外参：手眼标定 $AX = XB$

待求量是 $X = T_{LI}$（把 IMU 系点变到雷达系），6 个自由度。已知两路轨迹：雷达里程计给出的 $T_{WL}(t_k)$、IMU 积分给出的 $T_{WI}(t_k)$。真实关系为

$$T_{WI}(t) = T_{WL}(t)\,X. \tag{26.19}$$

推导 $AX = XB$。取两个时刻 t_1, t_2。先在 t_1 左乘逆：

$$T_{WI}(t_1)^{-1} = \big(T_{WL}(t_1)X\big)^{-1} = X^{-1}T_{WL}(t_1)^{-1}. \tag{26.20}$$

再右乘 t_2 的式子 $T_{WI}(t_2) = T_{WL}(t_2)X$：

$$T_{WI}(t_1)^{-1}T_{WI}(t_2) = X^{-1}T_{WL}(t_1)^{-1}T_{WL}(t_2)X. \tag{26.21}$$

左边是 IMU 的相对运动 $B = T_{I_1I_2}$，右边括号内是雷达的相对运动 $A = T_{L_1L_2}$，于是 $B = X^{-1}AX$，等价于

$$A\,X = X\,B. \tag{26.22}$$

把 $X = (\boldsymbol{R}_X, \boldsymbol{t}_X)$ 展开，它同时给出旋转与平移两组方程：

$$\boldsymbol{R}_A\boldsymbol{R}_X = \boldsymbol{R}_X\boldsymbol{R}_B, \tag{26.23}$$

$$\boldsymbol{R}_A\boldsymbol{t}_X + \boldsymbol{t}_A = \boldsymbol{R}_X\boldsymbol{t}_B + \boldsymbol{t}_X \implies (\boldsymbol{R}_A - \boldsymbol{I})\boldsymbol{t}_X = \boldsymbol{R}_X\boldsymbol{t}_B - \boldsymbol{t}_A. \tag{26.24}$$

关键结构：式 (26.23) 只含 $\boldsymbol{R}_X$，所以**先解旋转、再解平移**，两步都是线性最小二乘。

第一步：解旋转。把旋转写成四元数，$AX = XB$ 变成

$$\boldsymbol{q}_A \otimes \boldsymbol{q}_X = \boldsymbol{q}_X \otimes \boldsymbol{q}_B. \tag{26.25}$$

定义左乘与右乘矩阵（$\boldsymbol{q} = [q_0, \boldsymbol{q}_v^\top]^\top$）：

$$[\boldsymbol{q}]_L = \begin{bmatrix} q_0 & -\boldsymbol{q}_v^\top \\ \boldsymbol{q}_v & q_0\boldsymbol{I} + [\boldsymbol{q}_v]_\times \end{bmatrix}, \qquad [\boldsymbol{q}]_R = \begin{bmatrix} q_0 & -\boldsymbol{q}_v^\top \\ \boldsymbol{q}_v & q_0\boldsymbol{I} - [\boldsymbol{q}_v]_\times \end{bmatrix}, \tag{26.26}$$

它们满足 $[\boldsymbol{q}]_L\boldsymbol{q}' = \boldsymbol{q} \otimes \boldsymbol{q}'$、$[\boldsymbol{q}]_R\boldsymbol{q}' = \boldsymbol{q}' \otimes \boldsymbol{q}$。于是

$$\big([\boldsymbol{q}_A]_L - [\boldsymbol{q}_B]_R\big)\boldsymbol{q}_X = \boldsymbol{0}. \tag{26.27}$$

每对运动给 4 个方程，堆成 $\boldsymbol{M}\boldsymbol{q}_X = \boldsymbol{0}$（$\boldsymbol{M} \in \mathbb{R}^{4K\times 4}$），取最小奇异值对应的右奇异向量再归一化即为 $\boldsymbol{q}_X$。

可解性条件：所有旋转轴必须”张开”。若机器人只绕竖直轴原地转，A, B 的旋转轴都平行，$\boldsymbol{M}$ 秩不足，X 中绕该轴的分量不可观。所以标定轨迹必须包含绕至少两个不平行轴的旋转：前后倾斜、左右横滚、加上偏航，各转 30° 以上。

第二步：解平移。$\boldsymbol{R}_X$ 已知后，式 (26.24) 是线性方程组，堆 K 对运动：

$$\begin{bmatrix} \boldsymbol{R}_A^{(1)} - \boldsymbol{I} \\ \vdots \\ \boldsymbol{R}_A^{(K)} - \boldsymbol{I} \end{bmatrix} \boldsymbol{t}_X = \begin{bmatrix} \boldsymbol{R}_X \boldsymbol{t}_B^{(1)} - \boldsymbol{t}_A^{(1)} \\ \vdots \\ \boldsymbol{R}_X \boldsymbol{t}_B^{(K)} - \boldsymbol{t}_A^{(K)} \end{bmatrix}, \tag{26.28}$$

即 $\boldsymbol{D}\boldsymbol{t}_X = \boldsymbol{c}$ 的 $\mathbb{R}^{3K\times 3}$ 最小二乘，解 $\hat{\boldsymbol{t}}_X = (\boldsymbol{D}^\top \boldsymbol{D})^{-1}\boldsymbol{D}^\top \boldsymbol{c}$。可解性同样要求旋转轴张开：若所有轴相同，$\boldsymbol{R}_A - \boldsymbol{I}$ 的零空间方向一致，沿该方向平移不可观。

第三步：非线性细化。代数解在噪声下不是最优，最后在 SE(3) 上用残差

$$\boldsymbol{e}_R^{(k)} = \log\!\left((\boldsymbol{R}_A^{(k)} \boldsymbol{R}_X)^\top \boldsymbol{R}_X \boldsymbol{R}_B^{(k)} \right), \qquad \boldsymbol{e}_t^{(k)} = (\boldsymbol{R}_A^{(k)} - \boldsymbol{I})\boldsymbol{t}_X - \boldsymbol{R}_X \boldsymbol{t}_B^{(k)} + \boldsymbol{t}_A^{(k)} \tag{26.29}$$

做最小二乘（Ceres/GTSAM 均可），初值取前两步结果。log 是 $\mathrm{SO}(3) \to \mathfrak{so}(3)$ 的对数映射（第 2 章）。当雷达帧率低而运动剧烈时，两帧之间不能当成瞬时刚体变换，此时应改用连续时间轨迹模型，同时估计外参与时间偏移。

26.7 时间同步与时间偏移估计

一台机器人上至少四类时间源，精度差几个数量级（表 26.5）。最省事又最有效的做法是**硬件触发**：让主时钟同时触发相机曝光与雷达扫描起始，驱动把触发时刻写进 `header.stamp`，采样时刻在物理上就对齐了。

表 26.5: 常见时间同步方案

方案	典型精度	成本	适用
系统时钟（无同步）	秒级漂移	0	不可用于融合
NTP（`chrony`）	0.1–10 ms	0	一般用途、跨机器
PTP（IEEE 1588，硬件时间戳）	$< 1\,\mu$s	需支持 PTP 的网卡	多相机、多雷达
GPS/PPS 秒脉冲	< 100 ns	需 GPS 模块	室外、多机对齐
硬件触发线	曝光级（μs）	需相机/雷达支持	视觉-惯性标配

ROS 2 里的两个开关必须分清：`use_sim_time:=true` 时节点的 `now()` 来自 `/clock` 而非系统时钟，回放 bag 必须打开它并加 `--clock`；消息里的 `header.stamp` 是**传感器采集时刻**，与 `/clock` 无关，回放不会重新打时间戳，所以录制时同步必须已经做好。检查手段是 `ros2 bag info` 看时长与消息数、`ros2 topic echo --field header.stamp` 对比 `date +%s\%N`，以及回放时看 TF 是否频繁报 `ExtrapolationException`。

偏移 δt 的估计。即使时钟同步，传感器内部仍有固定延迟：相机从曝光到驱动拿到图像通常 10–50 ms，雷达时间戳可能打在帧首或帧尾。把这类偏移统一记作 δt，含义是”测量实际对应 $t - \delta t$ 时刻的状态”。它在 δt 内产生的位姿误差是

$$\delta \boldsymbol{p} \approx \boldsymbol{v}\,\delta t, \qquad \delta \boldsymbol{\theta} \approx \boldsymbol{\omega}\,\delta t. \tag{26.30}$$

三种估计方法：(1) **一维搜索**——把点云到地图的平均残差看成 δt 的函数，在 $[-0.1, 0.1]$ s 内粗扫，再用抛物线拟合 $J(\delta t_i) \approx J_{\min} + \tfrac{1}{2}J''(\delta t_i - \widehat{\delta t})^2$ 求极值；(2) **联合优化**——把 $(X, \delta t)$ 一起做非线性优化，按式 (26.30) 补偿后代入式 (26.22) 求残差，交替优化 3–5 轮；(3) **互相关**——对雷达里程计与 IMU 积分的速度曲线做互相关，峰值位置即 δt，快但只在有明显加减速时有效。

验证：δt 标好后，静止时墙厚不变（时间偏移不影响静止），快速旋转时点云不再”扇开”；把 δt 人为置零再跑一遍，若轨迹明显变差，说明这个参数确实重要。

26.8 标定验证与常见错误

标定做完不等于做完，必须验证。四条判据按顺序查，任何一条不过都不要上车：

1. **重力模长**：静止时 $\|\tilde{\boldsymbol{a}}\| = 9.81 \pm 0.1\,\mathrm{m/s^2}$；否则是加速度计尺度或零偏错。
2. **点云墙厚**：静态场景叠加 20 帧，墙与地面厚度应 $< 2\,\mathrm{cm}$。厚度随距离增大是旋转外参误差，厚度与距离无关是平移误差或时间偏移。
3. **回环闭合**：走矩形回到起点，位置差 $< 5\,\mathrm{cm}$、角度差 $< 1°$。
4. **图像叠加**：点云投影到图像，边缘对齐、无系统偏移、无随位置的扭曲。

表 26.6: 标定类故障的现象、原因与处理

现象	可能原因	快速检查	处理
点云整体旋转固定角度	外参旋转符号/方向反	看正前方箱子的 x, y, z	重标或按定义改正号
点云镜像	坐标系手性错	检查是否右手系	改约定或加镜像修正
轨迹一直向一侧漂	陀螺零偏未标/符号错	静止 60 s 看积分角度	重取零偏，查外参方向
快速旋转时点云扇开	时间偏移大	静/动对比墙厚	估 δt 并补偿
静止时点云就厚	外参平移或雷达内参错	近距离叠加 20 帧	检查平移与外参
相机图像边缘扭曲	畸变系数错	直线在图上是否直	重标内参，查 k_1 符号
融合后重力方向偏	加速度计未标	静止模长是否 9.81	六面法重标
漂移快、协方差收缩过快	零偏随机游走参数过小	看协方差曲线	调大 `b_gyr_cov`

本章小结

1. 旋转外参误差按 $d\theta$ 放大、时间误差按 $v\Delta t$ 放大：1° 在 10 m 处 17.5 cm，10 ms 在 2 m/s 时 2 cm；近处建图看不出，远处定位才暴露。
2. 标定链有依赖顺序：内参 → IMU 噪声/零偏 → 相机-IMU → 雷达-IMU → 雷达-相机，每步都有量化判据。
3. 张正友法：由 $\boldsymbol{H} = \lambda \boldsymbol{K}[\boldsymbol{r}_1\ \boldsymbol{r}_2\ \boldsymbol{t}]$ 与旋转列的正交单位性得 $\boldsymbol{h}_1^{\top}\boldsymbol{B}\boldsymbol{h}_2 = 0$、$\boldsymbol{h}_1^{\top}\boldsymbol{B}\boldsymbol{h}_1 = \boldsymbol{h}_2^{\top}\boldsymbol{B}\boldsymbol{h}_2$，解 $\boldsymbol{V}\boldsymbol{b} = \boldsymbol{0}$ 得 $\boldsymbol{B}$（差一尺度），再闭式解内参，最后用重投影误差做 LM 细化。
4. 内参判据：重投影 RMS $< 0.5\,\mathrm{px}$，$f_x \approx f_y$，主点接近图像中心。
5. Allan 方差用相邻簇平均之差的方差的一半定义；$\tau = 1\,\mathrm{s}$ 处读数即噪声密度，斜率 $-1/2$ 是随机游走、0 是零偏不稳定性、$+1/2$ 是速率随机游走。
6. 手眼标定：由 $T_{WI} = T_{WL}X$ 消去世界系得 $AX = XB$，先解四元数方程得到 $\boldsymbol{R}_X$，再解 $(\boldsymbol{R}_A - \boldsymbol{I})\boldsymbol{t}_X = \boldsymbol{R}_X\boldsymbol{t}_B - \boldsymbol{t}_A$ 得 $\boldsymbol{t}_X$；要求旋转轴张开。
7. 时间偏移影响为 $\delta\boldsymbol{p} \approx \boldsymbol{v}\delta t$、$\delta\boldsymbol{\theta} \approx \boldsymbol{\omega}\delta t$；根本解决办法是硬件触发 + PTP 或 GPS 秒脉冲。

第二十六章 练习

1. 某雷达与 IMU 的外参偏航角差 1.5°。估算 10 m 处点的偏差；若只在 2 m 内建图，这个误差会被发现吗？
2. 从 $\boldsymbol{H} = \lambda \boldsymbol{K}[\boldsymbol{r}_1\ \boldsymbol{r}_2\ \boldsymbol{t}]$ 出发，说明 3 张不同姿态的图为何恰好能定出内参；若 3 张姿态相同，方程会怎么退化？
3. 证明 $\boldsymbol{B}$ 只能确定到相差一个尺度，并说明式 (26.11) 为什么不受尺度影响。
4. 某陀螺 Allan 曲线 $\tau = 1\,\mathrm{s}$ 处 $\sigma_A = 0.02\,°/\mathrm{s}$，$\tau = 100\,\mathrm{s}$ 处平台 $\sigma_A = 2\,°/\mathrm{h}$。求 N_g（两种单位）与 B。
5. 手眼标定中若机器人只绕竖直轴旋转，X 的哪几个分量不可观？分别从 $\boldsymbol{M}\boldsymbol{q}_X = \boldsymbol{0}$ 的秩和 $(\boldsymbol{R}_A - \boldsymbol{I})$ 的零空间解释。
6. 雷达时间戳打在帧尾、驱动又加了 30 ms 传输延迟，而代码把它当作帧首采集时刻。等效 δt 是多少？10 Hz、2 m/s 下误差多大？

延伸阅读

- Zhang 的原始论文 A Flexible New Technique for Camera Calibration（PAMI 2000）：本章第 26.4 节的全部推导都来自这里，附录 B 给出式 (26.11) 的完整形式。
- Sola 的 IMU 误差状态文档 [37]：噪声建模与离散化约定讲得最清楚，适合搞不清” 噪声密度” 与” 离散协方差” 换算的人。
- Kalibr wiki 与 OpenCV 标定文档：前者给出相机-IMU 连续时间标定的完整流程与激励要求，后者给出畸变模型与工程细节。
- imu_utils：输入一段静止数据即输出 Allan 曲线与噪声参数，可直接用表 26.4 的读法练手。

第二十七章 调参与排错手册

本章导读

同一套 FAST-LIO，别人的车跑得又稳又准，你抄了同一份配置却在原地炸开——这是新手最挫败的场景。原因很少是”参数玄学”，而是**数据质量问题被误当成参数问题**：时间戳有 100 ms 偏移、外参差 2°、点云频率不稳、CPU 没开优化编译。本章先给一套可复现的调参方法论（一次只改一个、固定数据集、记录指标），再按 FAST-LIO/Point-LIO/Cartographer/Nav2 四套系统列出关键参数”调大/调小”的后果，最后给一份 12 行以上的排错速查表，把”症状 → 原因 → 排查命令 → 处理”直接对应起来。本章是全书最应该放在工位上的一章。

27.1 调参方法论

调参最忌讳两件事：一是同时改三个参数，然后不知道是谁起的作用；二是每换一次参数就换一段数据，结果不可比。正确的做法是一条死规矩和三个习惯。

死规矩：一次只改一个参数，其余全部冻结。

三个习惯：

1. **固定数据集**。选定一段有代表性的 bag（包含直行、转弯、急停、回环），所有对比都在它上面做。换 bag 等于换了考卷，分数没有意义。
2. **先定量指标，再看图**。每次实验至少记录：轨迹与真值的 ATE、终点漂移、CPU 占用、平均耗时、是否丢失跟踪。只看 RViz”感觉还行”是最容易骗自己的。
3. **先数据质量，后算法参数**。按第 26 章的四条判据检查完标定，再动 `filter_size_surf` 这类参数。数据错了，任何参数都调不出来。

表 27.1: 调参记录表模板（强烈建议用表格软件维护）

编号	改动	ATE (m)	平均耗时 (ms)	现象备注
0	基线配置	0.42	18	转弯处有轻微拖影
1	`filter_size_surf` 0.5→0.3	0.31	26	更准但更慢
2	在 1 基础上 `max_iteration` 4→3	0.39	17	精度回落，速度变好
3	回到 0，改外参 z +2 cm	0.44	18	无明显变化，排除平移误差

27.2 FAST-LIO / FAST-LIO2：关键参数

FAST-LIO2 的配置分成 `common`、`preprocess`、`mapping`、`publish` 四段 [41]。下面是能直接跑的骨架，之后逐项解释。

Listing 27.1: FAST-LIO2 配置骨架（YAML）

```yaml
common:
  lid_topic: "/livox/lidar"
  imu_topic: "/livox/imu"
  time_sync_en: false           # true only if driver already syncs stamps
  time_offset_lidar_to_imu: 0.0 # seconds, positive: lidar lags imu

preprocess:
  lidar_type: 1                 # 1=Livox, 2=Velodyne, 3=Ouster
```

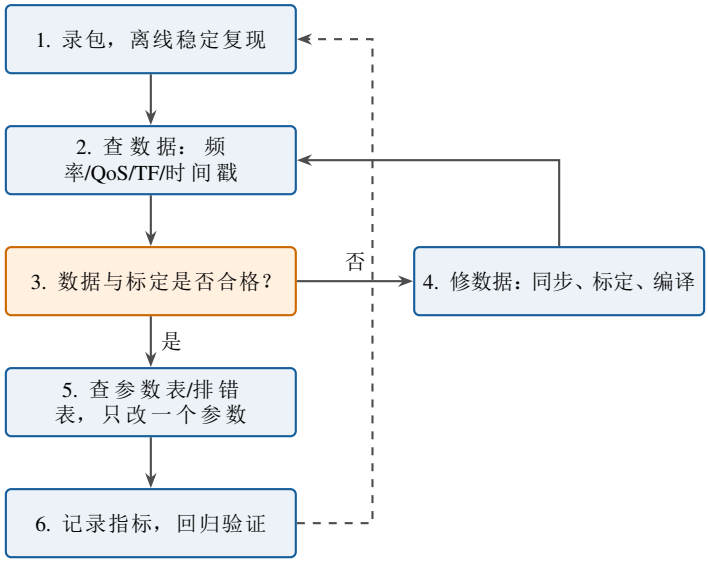


图 27.1: 调参决策流程。核心是第 3 步之前不要碰算法参数；第 6 步之后必须回到第 1 步重新验证，而不是”现场再跑一次看看”。

```
  scan_line: 6
  blind: 0.5                    # ignore points closer than this (m)

mapping:
  acc_cov: 0.1                  # continuous-time accel noise
  gyr_cov: 0.1                  # continuous-time gyro noise
  b_acc_cov: 0.0001             # accel bias random walk
  b_gyr_cov: 0.0001             # gyro bias random walk
  fov_degree: 180
  det_range: 100.0
  extrinsic_est_en: false       # online extrinsic estimation
  extrinsic_T: [0.0, 0.0, 0.0]
  extrinsic_R: [1.0, 0.0, 0.0,
                0.0, 1.0, 0.0,
                0.0, 0.0, 1.0]
publish:
  path_en: true
  scan_publish_en: true
  dense_publish_en: true

filter_size_surf: 0.5           # surface downsample voxel (m)
filter_size_map: 0.5            # map voxel size (m)
cube_side_length: 1000
point_filter_num: 3
max_iteration: 4
```

调 FAST-LIO 的顺序

先外参（错了怎么调都没用）→ 再 `point_filter_num/filter_size_surf`（把实时性调够）→ 再 `max_iteration`（补精度）→ 最后才动 `*_cov`（处理退化场景）。顺序反了会陷入”参数互相打架”。

表 27.2: FAST-LIO2 参数调节方向与后果

参数	建议范围	调大的后果	什么时候调
`extrinsic_T`	实测	整体偏移、点云重影	用卷尺 + 物理检验确定，别猜
`extrinsic_R`	实测	远处点云旋转错位	外参标定，见第 26 章
`filter_size_surf`	0.2–0.5 m	更快但细节丢失	点云太稠导致算不过来时调大
`filter_size_map`	0.2–1.0 m	内存小、精度低	内存吃紧（长走廊）时调大
`point_filter_num`	1–5	更快、更稀疏	CPU 打满时调到 3–4
`max_iteration`	3–8	更准、更慢，易过拟合	转弯重影时加到 5–6
`acc_cov`	10^{-2}–1	更信 IMU	点云退化（长走廊、隧道）时调大
`gyr_cov`	10^{-2}–1	更信 IMU 姿态	快速旋转时姿态滞后，调大
`b_acc_cov`	10^{-5}–10^{-3}	零偏更快跟随	温漂明显时调大
`b_gyr_cov`	10^{-5}–10^{-3}	零偏更快跟随	长时间漂移时调大
`cube_side_length`	500–2000	地图更大、内存更高	场地大（>100 m）时调大
`det_range`	50–200 m	更远处点参与匹配	室内场地调小可提速

27.3 Point-LIO：差异参数

Point-LIO 的卖点是**逐点更新**：每个点都做一次状态更新，而非等一帧点云凑齐，因此在剧烈运动、低帧率雷达上更稳 [18]。它的配置文件结构与 FAST-LIO 相同，差异集中在表 27.3。

表 27.3: Point-LIO 相对 FAST-LIO 的关键差异

参数	典型值	说明
`use_imu_as_input`	0 或 1	两种更新模式的开关，影响鲁棒性与算力；不同版本默认值不同，以仓库示例配置为准
`imu_satu_en`	1	打开 IMU 饱和检测。撞击、弹丸命中导致的瞬时超量程不会把状态带飞
`ivox_grid_resolution`	0.2–0.5 m	iVox 体素边长，越小越精细也越慢
`ivox_nearby_type`	18 或 26	近邻搜索的邻域定义，26 更稳，18 更快
`point_filter_num`	1–2	逐点更新下通常不用激进降采样
`max_iteration`	3–5	单帧内部迭代次数，与逐点更新互补

笔记 不要把两套配置混用。Point-LIO 与 FAST-LIO 虽然参数名大量重叠，但协方差的量纲约定并不完全一致，把 FAST-LIO 的 YAML 直接改名喂给 Point-LIO，常见结果是”能跑但一直缓慢漂移”。正确做法是从各自仓库的示例配置出发，只改外参与话题名。

Listing 27.2: Point-LIO 关键配置（只列与 FAST-LIO 的差异项）

```yaml
common:
  lid_topic: "/livox/lidar"
  imu_topic: "/livox/imu"
  time_sync_en: false

mapping:
  use_imu_as_input: 0        # update mode switch, see repo README
  imu_satu_en: 1             # reject saturated imu samples (hits, impacts)
  ivox_grid_resolution: 0.3  # voxel edge length of the ivox map (m)
  ivox_nearby_type: 18       # 18 or 26 neighbours
  point_filter_num: 1
  max_iteration: 4
  extrinsic_T: [0.0, 0.0, 0.0]
  extrinsic_R: [1.0, 0.0, 0.0,
                0.0, 1.0, 0.0,
                0.0, 0.0, 1.0]
```

27.4 Cartographer：2D 建图参数

Cartographer 的参数分三层：前端（`TRAJECTORY_BUILDER_2D`）、回环（`constraint_builder`）、后端优化（`optimization_problem`）[19]。它最常出的问题是**建图糊、重影、回环失败**，三者对应的参数完全不同（表 27.4）。

表 27.4: Cartographer 2D 关键参数与症状对应

参数	典型值	作用与调节方向
`submaps.resolution`	0.05 m	子图分辨率。调小更精细但内存涨；建图糊可先查这个
`submaps.num_range_data`	60–90	每个子图插入的帧数。调大子图更” 长”，回环更少但更快
`TRAJECTORY_BUILDER_2D.voxel_filter_size`	0.025–0.05 m	前端降采样。调大提速，过大丢细节
`adaptive_voxel_filter.max_length`	0.3–0.5 m	自适应降采样上限。走廊场景调大提速
`min_range/max_range`	0.2/30 m	只使用该距离内的点。室内把 `max_range` 调小可减少动态干扰
`missing_data_ray_length`	5–8 m	无回波方向上的自由空间长度，影响建图” 通透感”
`constraint_builder.min_score`	0.55–0.65	回环最小匹配分数。调高少回环但更准，调低易误回环
`constraint_builder.sampling_ratio`	0.3	回环候选采样比例。调大更准更慢
`optimization_problem.huber_scale`	1e-2–1e1	鲁棒核尺度。误回环导致地图撕裂时调小
`optimize_every_n_nodes`	20–90	每隔多少节点优化一次。调小更实时，调大更稳
`use_imu_data`	true	有 IMU 就打开，否则纯激光在快速旋转时会糊
`num_accumulated_range_data`	1–2	旧版本参数名，等价于每帧累积的雷达数据数

对应的 Lua 配置片段（Cartographer 用 `.lua` 而不是 YAML，参数按表名组织）：

Listing 27.3: Cartographer 2D 关键配置（Lua）

```
TRAJECTORY_BUILDER_2D = {
  use_imu_data = true,
  min_range = 0.2,
  max_range = 30.,
  missing_data_ray_length = 5.,
  voxel_filter_size = 0.025,
  adaptive_voxel_filter = { max_length = 0.5 },
  submaps = {
    num_range_data = 90,
    grid_options_2d = {
      grid_type = "PROBABILITY_GRID",
      resolution = 0.05,
    },
  },
}

POSE_GRAPH.constraint_builder.min_score = 0.6
POSE_GRAPH.constraint_builder.sampling_ratio = 0.3
POSE_GRAPH.optimization_problem.huber_scale = 1e1
POSE_GRAPH.optimize_every_n_nodes = 35
```

三个高频症状的” 参数线索”：

- **建图糊/墙变厚**：先看 `use_imu_data` 是否打开、IMU 是否标定；再看 `num_range_data` 是否过大（子图太长，前端漂移积累）。

- **重影（同一堵墙两条）**：典型是回环没闭上。调低 `min_score` 到 0.5 试，若出现**错误**回环（地图撕裂）说明场景太像，应该调高 `min_score` 并调小 `huber_scale`。
- **回环失败**：确认机器人回到了历史轨迹附近（雷达看不到历史区域的点就无从回环），并检查 `sampling_ratio` 与 `num_range_data`。

27.5 Nav2：代价地图与控制器参数

Nav2 的参数按节点分文件：`controller_server`、`planner_server`、`costmap` 等 [26]。最容易调坏的是**代价地图**与**控制器限速**，多数" 原地打转/贴墙/走不动" 的现象都源于此。

Listing 27.4: Nav2 参数片段（局部代价地图 + 控制器）

```
local_costmap:
  local_costmap:
    ros__parameters:
      update_frequency: 10.0
      rolling_window: true
      width: 6
      height: 6
      resolution: 0.05
      robot_radius: 0.28          # or footprint polygon for non-circular
      plugins: ["voxel_layer", "inflation_layer"]
      voxel_layer:
        plugin: "nav2_costmap_2d::VoxelLayer"
        max_obstacle_height: 1.5  # ignore ceiling; keep 0.3-1.5 for RM
        mark_threshold: 0
        observation_sources: scan3d
        scan3d:
          topic: /lidar/points
          data_type: "PointCloud2"
          clearing: true
          marking: true
          max_obstacle_height: 1.2
      inflation_layer:
        plugin: "nav2_costmap_2d::InflationLayer"
        inflation_radius: 0.55    # >= robot_radius + safety margin
        cost_scaling_factor: 3.0  # smaller -> wider high-cost band

controller_server:
  ros__parameters:
    controller_frequency: 20.0
    FollowPath:
      plugin: "dwb_core::DWBLocalPlanner"
      max_vel_x: 2.5
      max_vel_theta: 3.0
      acc_lim_x: 2.0
      acc_lim_theta: 4.0
      decel_lim_x: -2.0
      xy_goal_tolerance: 0.10
```

```
    yaw_goal_tolerance: 0.10
    transform_tolerance: 0.2
    vx_samples: 20
    vtheta_samples: 40
```

表 27.5: Nav2 关键参数与故障方向

参数	典型值	偏差带来的现象
`robot_radius/footprint`	实测	偏小会贴墙蹭、偏大到不了窄通道
`inflation_radius`	半径 +0.2–0.4 m	偏小常撞、偏大”到处不可通行”
`cost_scaling_factor`	2–5	偏小代价带更宽、更保守；偏大贴着障碍走
`max_vel_x`	1.5–4	偏大来不及转、偏小效率低
`acc_lim_x/acc_lim_theta`	实测	偏大轨迹抖动、偏小走不动
`xy_goal_tolerance`	0.05–0.2 m	偏小原地抖、偏大停不准
`transform_tolerance`	0.1–0.3 s	偏小频繁报 TF 超时、偏大位姿滞后
`controller_frequency`	10–30 Hz	偏低调速慢、偏高吃 CPU
`max_obstacle_height`	0.3–1.5 m	偏大会把天花板/悬挂物当障碍

27.6 排错速查表

下面这张表是本章的核心，遇到问题先按”现象”找到对应行。所有排查命令都在 ROS 2 下验证过（见第 25 章的工具使用）。

表 27.6: 排错速查表：现象 → 原因 → 排查命令 → 处理

现象	可能原因	排查命令	处理
定位起飞（位姿乱跳）	外参错、点云与 IMU 时间错、初始姿态错	`ros2 topic hz /lio/odom`	重标外参；估 δt；确认初值
点云拖影（一帧里点被”拉丝”）	时间偏移大、去畸变失效	静止/旋转对比墙厚	按第 26 章估 δt
地图重影（墙变两条）	回环没闭上、外参差、里程计漂	看历史轨迹是否重叠	调 `min_score`；检查外参
回环错误（地图撕裂）	`min_score` 太低、场景太像	回看约束残差	调高 `min_score`，调小 `huber_scale`
TF 报 LookupException	坐标系名拼错/未发布	`tf2_echo map base_link`	补 static 外参或改 frame_id
TF 报 ExtrapolationException	时间戳乱序、缓冲区不足	`tf2_tools view_frames`	修同步；`use_sim_time` 一致
时间戳乱序/回退	多驱动时钟不同源	`ros2 topic echo --field header.stamp`	硬件触发或统一到一台主机
CPU 打满、频率掉	没开 Release、降采样不足	`top -H`、`ros2 topic hz`	加 `-DCMAKE_BUILD_TYPE=Release`；调大 `point_filter_num`
内存持续增长	地图/容器无限增长	`top` 看 RSS 曲线	限制 cube 数量或体素边长
IMU 数据异常（跳变/饱和）	撞击超量程、驱动错位	画 `/imu/data` 曲线	开饱和检测；换安装位置
静止时也在缓慢漂	零偏未标、温度漂移	静置 60 s 看积分值	重取零偏；调大 `b_gyr_cov`
外参符号错（点云镜像）	坐标系手性/方向反	正前方放箱子看 x, y, z	按定义改正号
UWB 定位跳变	多径、遮挡、基站几何差	看 UWB 与 LIO 差值序列	加抗差（Huber）+ 限幅融合
导航原地转圈	目标不可达、代价地图全占	`ros2 topic echo /plan`	查 `inflation_radius`；确认 `footprint`
机器人贴墙蹭	半径过小、膨胀层不足	看局部代价地图热力图	调大 `inflation_radius`

笔记 所有排查的共同前提是”能复现”。所以第一条纪律永远是先 `ros2 bag record -a` 录一段，然后离线用 `ros2 bag play --clock` 复现。现场改参数再跑一次，往往只是碰巧好了，问题还在。

27.7 日志与可视化

- **RViz2 看四样东西**：点云（是否拖影、墙是否重影）、轨迹（是否闭环）、TF（是否断链）、协方差（`Odometry` 的 covariance 是否收敛）。这四个显示项能覆盖八成问题。
- `rqt_plot` 画 `/cmd_vel` 与 `/odom` 的速度曲线，看限速/加速度是否被异常裁剪。
- **evo** 看轨迹：`evo_traj bag rm.bag /lio/odom --ref /gt/odom`，一眼看出漂移是”慢慢偏”还是”某处突变”。
- **Foxglove** 适合看长时间录包的数据：支持 3D 面板与自定义布局，比 RViz2 更适合离线分析。
- **日志级别**：把”每帧打印”这类调试语句降到 DEBUG，否则 I/O 本身就会让 LIO 掉帧。

一个真实的排错顺序（值得背下来）

现象：建图时墙变厚、转弯处炸开。**顺序**：录包 → 离线复现 → 查 `ros2 topic hz`（输入频率是否正常）→ 查 `tf2_tools view_frames`（有没有断链、频率是否一致）→ 查时间戳差值（点云与 IMU 的时间是否差几十 ms）→ 查外参（正前方箱子）→ 最后才调 `max_iteration` 和 `filter_size_surf`。按这个顺序，九成问题在前三步就找到了。

本章小结

1. 调参铁律：一次只改一个参数、固定数据集、记录定量指标、先修数据质量再动算法参数。
2. FAST-LIO 的调参顺序：外参 → 实时性（`point_filter_num`、`filter_size_surf`）→ 精度（`max_iteration`）→ 退化处理（`*_cov`）。
3. Point-LIO 的差异在逐点更新与 IMU 饱和处理（`use_imu_as_input`、`imu_satu_en`、iVox 参数）；不要混用两套配置。
4. Cartographer 的”糊/重影/回环失败”分别对应前端降采样、回环阈值与子图长度，`min_score` 与 `huber_scale` 是回环的两个关键旋钮。
5. Nav2 的三个高危参数是 `robot_radius`、`inflation_radius`、`cost_scaling_factor`，绝大多数”走不动/贴墙/原地转”都源于此。
6. 排错第一原则是**先录包、离线复现**；第二原则是按”频率 → TF → 时间戳 → 外参 → 参数”的顺序排查。

第二十七章 练习

1. 你的 FAST-LIO 在长走廊里严重漂移，但转弯处正常。应该优先调 `acc_cov/gyr_cov` 还是 `max_iteration`？说明理由。
2. Cartographer 建图出现”墙变成两条平行线”，请给出三种可能原因，并说明每种原因下应该看什么数据来区分。
3. Nav2 机器人在离目标 5 cm 处原地抖动，列出至少三个相关参数并说明调整方向。
4. 为什么”一次只改一个参数”在调 LIO 时尤其重要？请结合参数之间的耦合（如降采样与迭代次数）说明。
5. 设计一个 30 分钟的排错流程，用于判断”定位偶尔跳 30 cm”是时间同步问题还是重定位问题。

延伸阅读

- FAST-LIO2 [41] 与 Point-LIO [18] 的原始论文：前者解释 iKD-Tree 与迭代 EKF，后者解释逐点更新为什么能提高带宽，读完再调 `max_iteration` 会更有方向。
- Cartographer 论文 [19] 与官方 tuning 文档：子图、分支定界回环、后端优化的关系讲得最清楚。
- Nav2 官方文档的 Configuration Guide：每个参数的取值范围与单位，配合第 22 章的实战链路使用。
- 第 17 章：FAST-LIO2 与 Point-LIO 的内部实现细节，是理解本章参数含义的前提。

第二十八章 数据集、评测与复现

本章导读

”我的算法比 FAST-LIO 好”——这句话在组会上说出来之前，必须先回答三个问题：在哪个数据集上、用什么指标、对齐方式是什么。否则它只是一个无法验证的感觉。本章解决的就是”怎么把感觉变成数字”：先列出定位领域最常用的公开数据集与各自适合评什么，再给出 ATE/RPE 的严格定义与 Umeyama 对齐闭式解（这是所有评测工具背后真正在算的东西），然后用 `evo` 把命令和输出对上号，最后讲自采数据的规范与可复现实验的设计。评测能力决定了你能否在一堆参数里做出正确判断，也决定了你的实验别人能不能复现。

28.1 公开数据集

选数据集的原则是：**它必须能暴露你想改进的那种失效模式**。在 KITTI 上刷高分的算法，遇到 RM 场地的动态障碍与对称结构可能一塌糊涂；反之只在自家场地上调参，也无法证明方法本身有效。表 28.1 是常用的几套。

表 28.1: 常用公开数据集对比

数据集	传感器	场景	适合评什么
KITTI Odometry	双目、HDL-64、GPS/IMU	城郊道路、高速	激光/视觉里程计的 ATE，室外长距离
EuRoC MAV	双目、ADIS16448、Vicon/Leica	室内房间与厂房	VIO 的 ATE/RPE，标定与时间同步
TUM VI	双目鱼眼、IMU	室内外混合	视觉惯性，含相机标定基准
NTU VIRAL	Ouster、双目、IMU、UWB	校园、室内外	多传感器融合、UWB 辅助定位
Newer College	Ouster、相机、IMU	校园、含季节变化	长期定位、鲁棒性
M2DGR	鱼眼/双目/事件相机、激光、IMU、GNSS	校园地面机器人	多模态融合、地面平台
RM 自采	Livox、IMU、轮式	比赛场地	你的真实失效模式

几点使用建议：

1. **先用小数据集跑通闭环**。EuRoC 的 MH_01 只有 3 分钟，适合快速验证编译与配置；不要在 KITTI 上调试程序。
2. **真值要看清来源**。Vicon/动捕是 mm 级但只覆盖小场地；RTK 是 cm 级但室内失效；KITTI 的 GPS/IMU 融合真值本身也有误差。**评测精度的下限是真值精度**。
3. **注意传感器差异**。Livox 非重复扫描的点云分布与 Velodyne 机械旋转完全不同，在一套数据上调好的 `filter_size_surf` 换到另一套上必须重调。
4. **自采数据不可替代**。公开数据集用于”证明方法有效”，自采数据用于”证明在 RM 场上能用”，两者都要有。

28.2 评价指标：ATE、RPE 与漂移率

28.2.1 ATE：绝对轨迹误差

ATE（Absolute Trajectory Error）衡量估计轨迹与真值轨迹的整体偏差。设有 N 个对应时刻，估计位姿 $\hat{T}_i$、真值位姿 T_i，最简单的定义是逐点位置误差

$$\boldsymbol{e}_i = \operatorname{trans}\left(\hat{T}_i\right) - \operatorname{trans}\left(T_i\right), \qquad \mathrm{ATE}_{\mathrm{RMSE}} = \sqrt{\frac{1}{N}\sum_{i=1}^{N} \|\boldsymbol{e}_i\|^2}, \tag{28.1}$$

其中 trans(·) 取位姿的平移部分。但这样直接比是**不公平的**：两个轨迹可能只差一个初始位姿（起点朝向不同）或一个全局尺度（单目 SLAM），这不代表算法差。所以必须先做**对齐**，把估计轨迹用一个刚体变换（有时还允许缩放）配到真值上，再算误差。这就是常说的"ATE with SE(3) alignment"。

28.2.2 Umeyama 对齐的闭式解

对齐问题：给定两组三维点 $\{\boldsymbol{x}_i\}$（估计）与 $\{\boldsymbol{y}_i\}$（真值），$i = 1, \dots, N$，求尺度 s、旋转 $\boldsymbol{R}$、平移 $\boldsymbol{t}$，使

$$J(s, \boldsymbol{R}, \boldsymbol{t}) = \frac{1}{N} \sum_{i=1}^{N} \left\| \boldsymbol{y}_i - (s\boldsymbol{R}\boldsymbol{x}_i + \boldsymbol{t}) \right\|^2 \tag{28.2}$$

最小。下面推导闭式解（Umeyama 1991）。

第一步：消去平移。对 $\boldsymbol{t}$ 求偏导并令其为零：

$$\frac{\partial J}{\partial \boldsymbol{t}} = -\frac{2}{N} \sum_{i=1}^{N} (\boldsymbol{y}_i - s\boldsymbol{R}\boldsymbol{x}_i - \boldsymbol{t}) = \boldsymbol{0} \implies \boldsymbol{t} = \boldsymbol{\mu}_y - s\boldsymbol{R}\boldsymbol{\mu}_x, \tag{28.3}$$

其中 $\boldsymbol{\mu}_x = \frac{1}{N}\sum_i \boldsymbol{x}_i$、$\boldsymbol{\mu}_y = \frac{1}{N}\sum_i \boldsymbol{y}_i$。记中心化坐标 $\boldsymbol{x}'_i = \boldsymbol{x}_i - \boldsymbol{\mu}_x$、$\boldsymbol{y}'_i = \boldsymbol{y}_i - \boldsymbol{\mu}_y$，代回 J 得

$$J = \frac{1}{N} \sum_{i=1}^{N} \left\| \boldsymbol{y}'_i - s\boldsymbol{R}\boldsymbol{x}'_i \right\|^2 = \frac{1}{N} \sum_{i=1}^{N} \left\| \boldsymbol{y}'_i \right\|^2 - \frac{2s}{N} \sum_{i=1}^{N} \boldsymbol{y}'^{\top}_i \boldsymbol{R}\boldsymbol{x}'_i + \frac{s^2}{N} \sum_{i=1}^{N} \left\| \boldsymbol{x}'_i \right\|^2, \tag{28.4}$$

中间一步用了 $\|\boldsymbol{a} - \boldsymbol{b}\|^2 = \|\boldsymbol{a}\|^2 - 2\boldsymbol{a}^\top\boldsymbol{b} + \|\boldsymbol{b}\|^2$。记 $\sigma_x^2 = \frac{1}{N}\sum_i \|\boldsymbol{x}'_i\|^2$、$\sigma_y^2 = \frac{1}{N}\sum_i \|\boldsymbol{y}'_i\|^2$，并定义 3×3 的**互协方差矩阵**

$$\boldsymbol{\Sigma} = \frac{1}{N} \sum_{i=1}^{N} \boldsymbol{x}'_i \boldsymbol{y}'^{\top}_i. \tag{28.5}$$

利用迹的性质 $\operatorname{tr}(\boldsymbol{R}\boldsymbol{\Sigma}) = \frac{1}{N}\sum_i \boldsymbol{y}'^{\top}_i \boldsymbol{R}\boldsymbol{x}'_i$（把 $\boldsymbol{\Sigma}$ 展开后交换求和与求迹即可验证），式 (28.4) 变成

$$J = \sigma_y^2 - 2s \operatorname{tr}(\boldsymbol{R}\boldsymbol{\Sigma}) + s^2\sigma_x^2. \tag{28.6}$$

第二步：先定旋转。式 (28.6) 中只有中间一项与 $\boldsymbol{R}$ 有关，所以固定 s 时最小化 J 等价于最大化 $\operatorname{tr}(\boldsymbol{R}\boldsymbol{\Sigma})$。对 $\boldsymbol{\Sigma}$ 做奇异值分解 $\boldsymbol{\Sigma} = \boldsymbol{U}\boldsymbol{D}\boldsymbol{V}^\top$，$\boldsymbol{D} = \operatorname{diag}(d_1, d_2, d_3)$，$d_1 \ge d_2 \ge d_3 \ge 0$。代入：

$$\operatorname{tr}(\boldsymbol{R}\boldsymbol{\Sigma}) = \operatorname{tr}\left(\boldsymbol{R}\boldsymbol{U}\boldsymbol{D}\boldsymbol{V}^\top\right) = \operatorname{tr}\left(\boldsymbol{V}^\top\boldsymbol{R}\boldsymbol{U}\boldsymbol{D}\right) = \sum_{k=1}^{3} d_k \left(\boldsymbol{V}^\top\boldsymbol{R}\boldsymbol{U}\right)_{kk}. \tag{28.7}$$

由于 $\boldsymbol{V}^\top\boldsymbol{R}\boldsymbol{U}$ 是正交矩阵，其对角元不超过 1，故上界是 $\sum_k d_k$，取等当且仅当 $\boldsymbol{V}^\top\boldsymbol{R}\boldsymbol{U} = \boldsymbol{I}$，即 $\boldsymbol{R} = \boldsymbol{V}\boldsymbol{U}^\top$。但正交矩阵还可能是反射（$\det = -1$），而旋转要求 $\det = 1$，于是引入修正矩阵

$$\boldsymbol{S} = \operatorname{diag}\left(1,\, 1,\, \det(\boldsymbol{V}\boldsymbol{U}^\top)\right), \qquad \boldsymbol{R} = \boldsymbol{V}\boldsymbol{S}\boldsymbol{U}^\top. \tag{28.8}$$

这样 $\det(\boldsymbol{R}) = 1$ 且仍取到最大 $\operatorname{tr}(\boldsymbol{R}\boldsymbol{\Sigma}) = \sum_k d_k s_k$（$s_k$ 为 $\boldsymbol{S}$ 的对角元）。

第三步：再定尺度。把 $\boldsymbol{R}$ 代回式 (28.6)，对 s 求导：

$$\frac{\partial J}{\partial s} = -2\operatorname{tr}(\boldsymbol{R}\boldsymbol{\Sigma}) + 2s\sigma_x^2 = 0 \implies s = \frac{\operatorname{tr}(\boldsymbol{R}\boldsymbol{\Sigma})}{\sigma_x^2} = \frac{\sum_{k=1}^{3} s_k d_k}{\sigma_x^2}. \tag{28.9}$$

第四步：平移由式 (28.3) 给出：$\boldsymbol{t} = \boldsymbol{\mu}_y - s\boldsymbol{R}\boldsymbol{\mu}_x$。

评测时的三个约定

1. **是否估计尺度：**单目 SLAM 必须估计 s（否则轨迹差一个倍数）；激光/激光惯性系统通常固定 $s = 1$，只在刚体变换下对齐。
2. **对齐范围：**是" 整条轨迹对齐一次"（全局对齐）还是" 逐段对齐"，结果差很多。`evo` 默认全局对齐。
3. **时间对齐：**估计与真值的时间戳必须先按时间关联（`evo` 会自动关联，但存在偏移时要用`--t_max_diff` 或先手动补偿）。

28.2.3 RPE：相对位姿误差

RPE（Relative Pose Error）衡量” 固定间隔内的相对运动” 的误差，定义在成对的位姿上。取间隔 Δ（按帧数、时间 1 s 或距离 1 m），第 i 对的误差为

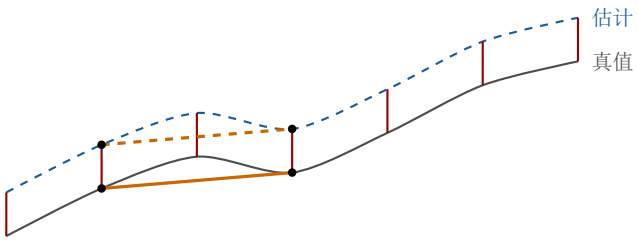
$$\boldsymbol{E}_i = \left(T_i^{-1} T_{i+\Delta}\right)^{-1} \left(\hat{T}_i^{-1} \hat{T}_{i+\Delta}\right), \tag{28.10}$$

即” 真值的相对运动” 与” 估计的相对运动” 之差的变换。分解成平移与旋转两部分：

$$\mathrm{RPE}_{\mathrm{trans},i} = \left\|\mathrm{trans}(\boldsymbol{E}_i)\right\|, \qquad \mathrm{RPE}_{\mathrm{rot},i} = \arccos\left(\frac{\mathrm{tr}\big(\mathrm{rot}(\boldsymbol{E}_i)\big) - 1}{2}\right), \tag{28.11}$$

再对 i 取 RMSE。arccos 那一项是旋转矩阵的**迹公式**：绕单位轴转 θ 的旋转矩阵迹为 $1 + 2\cos\theta$，反解即得旋转角。

为什么里程计看 RPE、SLAM 看 ATE？因为两者关心的东西不同：里程计的核心是” 每走一步的增量准不准”，增量误差会累积成漂移，RPE 正好测增量；SLAM 有回环与后端优化，单步可能不准但整体轨迹闭合得很好，此时 RPE 会很难看而 ATE 很好。所以报指标时通常两个都给：ATE 说整体精度，RPE 说局部一致性。



红：ATE（逐点位置误差）

橙：RPE（Δ 内相对运动的误差）

图 28.1: ATE 与 RPE 的区别。ATE 比较同一时刻的绝对位置（红色短线段），RPE 比较固定间隔 Δ 内的相对运动（橙色线段：真值一条、估计一条，二者之差才是误差）。

28.2.4 漂移率与工程指标

漂移率把 ATE 归一化，便于跨数据集比较：

$$\mathrm{drift} = \frac{\mathrm{ATE}_{\mathrm{RMSE}}}{\text{轨迹长度}} \times 100\%. \tag{28.12}$$

1 km 轨迹 ATE 1 m 是 0.1%，属于很好的水平；室内 50 m 轨迹 0.5 m 就是 1%，需要检查。

除精度外，工程上还必须报三类指标：

- **实时性**：单帧平均耗时、95 分位耗时、是否达到雷达频率。只报平均耗时会掩盖偶发的卡顿。
- **资源**：CPU 单核占用、内存峰值（RSS）、地图内存增长速度。
- **鲁棒性**：连续运行 N 次/小时的失败次数、丢失跟踪后的恢复时间。

28.3 evo：命令与输出解读

`evo` 是评测的事实标准工具，支持 TUM/KITTI/bag 等多种格式。四条最常用的命令：

Listing 28.1: evo 常用命令

```
# 1) inspect two trajectories: align, then plot side by side
evo_traj tum gt.tum --ref est.tum --align --plot

# 2) absolute trajectory error (RMSE), with alignment
evo_ape tum gt.tum est.tum -va --align --plot
```

```
# add --correct_scale when evaluating monocular SLAM

# 3) relative pose error over 1 m of travel
evo_rpe kitti gt.txt est.txt --delta 1 --delta_unit m -va

# 4) compare many runs: each result is a zip saved by evo_ape/evo_rpe
evo_res results/*.zip -p --use_fig --save_table table.csv
```

输出中最该看的是 `RMSE`、`median`、`std` 三个数：RMSE 对大误差敏感（平方后再平均），median 反映典型水平，std 反映抖动。若 median 好而 RMSE 差，说明轨迹中**偶有跳变**；若两者都差，说明是系统性偏移。`--align` 做的是第 28.2 节的 Umeyama 对齐（不含尺度），`--correct_scale` 才估计尺度。

笔记 报数字时的三个必须：必须写对齐方式（有无尺度）、必须写数据段（哪几段序列）、必须写运行次数（取中位数还是最好的一次）。只写"ATE 0.12" 是不可复现的。

28.4 自采数据规范

RM 场地的数据是公开数据集替代不了的，但自采数据最容易烂在文件夹里。规范如下：

1. **时间同步先于录制**。录之前用第 26 章的方法确认两路时间戳一致，并记录同步方式（硬件触发/PTP/NTP）。
2. **标定文件与数据同存**。每次采数据时把当次的相机内参、外参、IMU 噪声参数、激光雷达型号配置与 bag 放在同一目录，文件名带版本号。
3. **真值获取**：动捕精度最高但场地受限；全站仪适合静态标定点；室外用 RTK；也可以先用" 高精度先验地图 + ICP" 得到准真值，但要注明是" 准真值"。
4. **命名与元数据**：建议`日期_场地_平台_传感器_序号`，并写一个 README 记录：谁录的、什么配置、场地描述、已知问题（比如" 这段有人走过"）。
5. **不要只录成功片段**。失败的数据最有价值——炸的那 10 秒决定了你能不能复现并修好问题。

28.5 可复现实验

一个实验能被别人（或三个月后的你自己）复现，必须固定四件事：

1. **代码状态**：记录 git commit hash；有未提交改动时用 `git stash` 或打 tag。
2. **配置快照**：把当次使用的 YAML 与 launch 文件复制到结果目录（不要只记" 改了 `filter_size_surf`"）。
3. **随机性**：ICP/采样类算法、随机种子、线程调度都可能带来波动。固定随机种子，并**至少跑 3 次取中位数**；差异小于波动幅度的改进不能算改进。
4. **环境**：CPU 型号、是否开 Release、是否绑核、ROS 版本、bag 回放速度（是否`-- rate 1.0`）。

做改进对比时用消融表（表 28.2），每一行只允许一个变量不同，并用中位数与波动范围一起报。

表 28.2: 消融实验表模板（每个配置跑 3 次，报中位数 ± 半极差）

配置	改动	ATE (m)	RPE (m)	平均耗时 (ms)
A	基线	0.42 ± 0.03	0.021	18
B	A + iKD-Tree 体素 0.5 m	0.40 ± 0.02	0.020	14
C	B + 时间偏移补偿	0.29 ± 0.01	0.014	14
D	C + `max_iteration` 6	0.27 ± 0.02	0.013	21

笔记 不要用" 最好一次" 的结果参与比较。算法的真实表现是分布，不是单点。如果两个配置的差异落进了多次运行的波动范围，就老老实实说" 无显著差异"，这比编一个漂亮的数字更专业，也更省时间。

本章小结

1. 数据集要选能暴露目标失效模式的：公开数据集证明方法有效，自采数据证明在 RM 场上能用，两者都要。
2. ATE 是逐点绝对误差的 RMSE，通常先做 Umeyama 对齐再算；单目系统要估计尺度，激光系统固定 $s = 1$。
3. Umeyama 闭式解四步：中心化求 $\boldsymbol{\Sigma} = \frac{1}{N}\sum \boldsymbol{x}'\boldsymbol{y}'^{\top}$ → SVD 取 $\boldsymbol{R} = \boldsymbol{V}\boldsymbol{S}\boldsymbol{U}^{\top}$（$\boldsymbol{S}$ 修正反射）→ $s = \mathrm{tr}(\boldsymbol{R}\boldsymbol{\Sigma})/\sigma_x^2$ → $\boldsymbol{t} = \boldsymbol{\mu}_y - s\boldsymbol{R}\boldsymbol{\mu}_x$。
4. RPE 测固定间隔 Δ 内的相对运动误差，是里程计的核心指标；ATE 与 RPE 一起报才完整。
5. 漂移率 = ATE/轨迹长度，便于跨场景比较；工程上还要报耗时分布、内存与失败率。
6. `evo` 四条命令覆盖检查、ATE、RPE、多结果对比；报数字必须带对齐方式、数据段与运行次数。
7. 可复现四要素：代码 commit、配置快照、随机性控制（3 次取中位数）、环境记录。

第二十八章 练习

1. 估计轨迹与真值只差一个初始朝向，不做对齐的 ATE 会很大。请说明这种” 不公平” 在什么情况下不能通过刚体对齐消除（提示：想想单目尺度）。
2. 从式 (28.4) 出发，补全 $\mathrm{tr}(\boldsymbol{R}\boldsymbol{\Sigma})$ 与逐点求和等价的推导步骤（允许用” 展开后交换求和与求迹”）。
3. 为什么 $\boldsymbol{S}$ 的第三个对角元要取 $\det(\boldsymbol{V}\boldsymbol{U}^{\top})$？如果直接取 $\boldsymbol{R} = \boldsymbol{V}\boldsymbol{U}^{\top}$ 会发生什么？
4. 某算法 ATE 的 median 是 0.05 m 而 RMSE 是 0.6 m。请解释这两个数为什么会差这么多，并说明你会去查什么。
5. 你要比较两个参数配置的优劣，两者 ATE 相差 3%，而 5 次运行的波动范围是 ±5%。应该怎么下结论？

延伸阅读

- Umeyama 的原始论文 Least-Squares Estimation of Transformation Parameters Between Two Point Patterns（PAMI 1991）：本章对齐闭式解的全部推导来源。
- `evo` 官方文档 michaelgrupp.github.io/evo：所有命令行参数与输出含义，配合本章第 28.3 节的四条命令即可上手。
- KITTI Odometry 与 EuRoC 官方页面：真实数据、真值格式与评测脚本的权威说明，建议在跑通自己的系统后读它们的评测协议。
- 第 27 章的调参记录表：评测结果最终要落到那张表上才能形成闭环。

第二十九章 RoboMaster 场景实践建议

本章导读

前面各章讲的是” 通用技术”，本章只回答一个问题：**在自己的车上、自己的场地里该怎么选、怎么装、怎么验**。RM 赛场有几个和实验室完全不同的约束：场地对称、动态目标多、弹丸与撞击干扰传感器、算力与功耗被严格限制、比赛期间没有时间调参。因此本章写成一份” 实战清单”——方案选型表、各兵种需求对比、坐标系与标定清单、上车检查表、备赛流程与验收标准、以及几个最容易踩的坑。它不展开成比赛教程，只给你决策时需要的判断依据。

29.1 赛场环境特点

以近年 28 m × 15 m 的标准场地为例，定位系统要面对的环境特征有六条：

1. **高度对称**。场地两侧结构几乎镜像，中央高地、环形通道、基地位置对称。对全局定位（先验地图匹配）来说这是” 歧义源”：局部特征几乎一样，容易匹配到镜像位置。
2. **动态障碍多**。对方机器人（3–4 m/s）、己方队友、飞舞的弹丸都在点云里。弹丸对激光雷达是” 瞬时噪声”，对视觉是” 遮挡 + 光斑”。
3. **结构重复且低矮**。围栏、装甲板、基地支架都是同一种几何，垂直方向的纹理少，容易出现” 走廊式退化”。
4. **光照变化剧烈**。顶灯、装甲板 LED、能量机关的强光、以及比赛期间的明暗切换，对相机是灾难，对激光雷达影响小。
5. **地面材质与平整度**。场地地胶、坡道、上下坡的过渡区域会让轮式里程计打滑。
6. **观众与人员干扰**。场边走动的人会被 VIO 当成运动目标，也曾出现过观众入场导致的定位跳变。

这些特征直接决定了技术选型：**以 3D 激光惯性里程计为主，用先验地图做全局校正，用 UWB 做兜底**，而不是反过来。

29.2 定位方案选型

表 29.1: 定位方案对比（按 RM 场景的实际表现）

方案	优点	主要风险	建议
3D LIO（FAST-LIO2 / Point-LIO）	精度 2–5 cm，不依赖场地改造，输出频率高	退化场景（长走廊、空旷区）漂移；动态目标进地图	主方案，必装
先验地图 + 全局定位	消除累积漂移，冷启动可重定位	地图过期、场地布置变化、对称场景误匹配	主方案，每场前更新地图
UWB	全局绝对、无累积误差、可跨区	多径、金属反射、非视距、安装基站耗时	辅助与兜底
雷达站全局感知	上帝视角，可给全队共享位置	标定复杂、通信延迟、遮挡	有条件就上
轮式 + IMU 航迹推算	便宜、连续、不受视觉影响	打滑、累积漂移快	只作为短时补充

选型的原则是**异构冗余**：激光退化时靠 UWB 兜底，UWB 跳变时靠激光与先验地图压住，先验地图失效时至少还有连续里程计能让车停下来而不是撞墙。表 29.1 中任何单一方案都不足以支撑一场比赛。

29.3 哨兵、步兵与雷达站的差异

三个兵种对定位系统的需求完全不同（表 29.2），用同一套参数配置是常见错误。

具体到实现：

表 29.2: 三个兵种的定位需求对比

维度	哨兵	步兵	雷达站
平台	悬挂/滑轨，云台可 360° 转	全向底盘，可冲坡	静止，可升降
速度	平移 2–3 m/s	冲刺 4 m/s 以上	0
算力预算	高（Jetson 级）	中（受限功耗）	最高（可上台式机）
动态性	云台转动使雷达视角变化	最剧烈：撞击、跳跃、急停	几乎静止
定位重点	底盘稳定 + 云台姿态解耦	高速下短时精度与抗打滑	全局精度与多目标跟踪
关键风险	云台转动导致 TF 抖动	碰撞使 IMU 饱和	与车载机的通信延迟

- **哨兵**：云台与底盘必须解耦。底盘定位用 `map→odom→base\_link`，云台角由编码器反馈单独发布，不要试图用雷达去估计云台角（云台转动时雷达点云在动，容易被误当成底盘在转）。
- **步兵**：撞击与急停会让 IMU 瞬时超量程。Point-LIO 的 `imu_satu_en` 与合理的零偏随机游走参数是必备的；同时要限制单帧修正量，避免定位跳变导致规划器输出暴力动作。
- **雷达站**：几乎静止，重点在全局精度与多目标跟踪，而不是高频里程计。可以把算力全部投给全局匹配与多机器人状态估计，并把结果通过低延迟链路广播给队友。

29.4 坐标系与标定清单

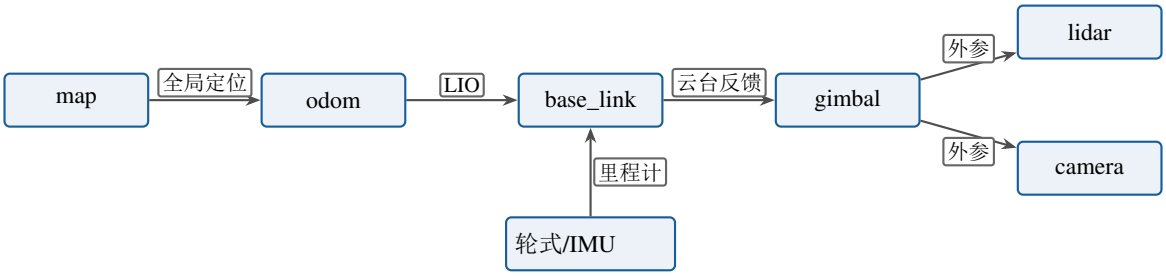


图 29.1: RM 机器人的坐标系与模块关系。注意云台是独立的一节：底盘到云台由编码器反馈，云台到传感器是标定出的常数外参。

标定顺序必须是" 从底盘往外扩"：**底盘 → 雷达 → 云台 → 相机内参 → 云台-相机外参 → UWB 坐标系**。每一环都依赖上一环，跳步会导致误差叠加且难以定位。

上车检查表（10 条，每次上车都过一遍）

1. IMU 静止 60 s，重力模长 $9.81 \pm 0.1\ \mathrm{m/s^2}$，三轴零偏稳定。
2. 正前方 5 m 放箱子，点云坐标 $x \approx 5$、$y \approx 0$、$z \approx 0$，误差 < 5 cm。
3. `tf2_tools view_frames` 无断链，`map→odom` 与 `odom→base\_link` 的发布频率正常。
4. 点云与 IMU 时间戳差值 < 5 ms（回放 bag 时检查）。
5. 静止叠加 20 帧，墙厚 < 2 cm。
6. 云台零位与底盘前向一致，偏差 $< 0.5°$；转一圈云台后底盘位姿不跳。
7. UWB 标定点与实际坐标一致，静态抖动 < 10 cm。
8. 先验地图与当前场地一致（对照场地尺寸与标志物）。
9. 冷启动重定位：10 次至少 9 次成功，耗时 < 10 s。
10. 连续运行 30 min，无内存持续增长、无 TF 超时、无频率下降。

29.5 备赛流程与验收标准

备赛是一条流水线，每一步都有可验收的输出，没通过就不要进下一步。

表 29.3: 备赛流程与验收标准

阶段	要做的事	验收标准
建图	场地空场时多圈扫描，含高地、坡道、边界	回环闭合误差 < 10 cm，无明显重影
地图清理	去掉动态物体、修复空洞、统一分辨率	地图文件可被定位节点加载且帧率达标
先验地图包	建立 map 与场地坐标系的对齐	标定点误差 < 5 cm
定位验证	静置、慢走、快跑、旋转四组测试	位置误差 < 10 cm，姿态 < 1°
导航联调	与规划、控制、行为树联跑	10 次导航任务全部到达，无碰撞
压力测试	连续 1 h、撞击、遮挡、对手干扰	不崩、不持续漂移、可自动恢复

29.6 常见坑

- **动态残影**：对手机器人被写进先验地图，之后每次定位都在” 幻影” 附近拉扯。建图时必须保证场地无人，或事后人工清理动态区域。
- **地图过期**：场地布置（基地、能量机关、围栏位置）每场都可能改。带着上一场的地图下场，重定位会稳定地匹配到错误位置——而且” 看起来很稳定”，最难发现。
- **UWB 多径**：金属结构、装甲板、地面反射会产生非视距测量，表现为偶发的几十厘米跳变。要加抗差（Huber/卡方检验）并做限幅融合，不要直接当观测用。
- **对称导致的误重定位**：场地镜像结构使全局匹配出现双解。解决办法是用不对称特征（基地、飞镖架、观众席）做优先级加权，或把上一帧位置作为先验。
- **算力不足**：哨兵/步兵上同时跑 LIO、导航、视觉识别，CPU 一满，LIO 首先掉帧。要在集成阶段就做资源预算，而不是比赛前才发现。
- **云台转动被误当成底盘运动**：雷达装在云台上时必须用云台角做补偿，否则底盘定位会被云台转动” 带偏”。
- **弹丸与撞击**：弹丸命中雷达或车体会造成瞬时点云爆点、IMU 饱和。要有饱和检测与单帧异常剔除。

本章小结

1. RM 赛场的关键约束：高度对称、动态障碍多、光照剧变、算力受限、比赛期无法调参；这些决定了必须做异构冗余而不是押注单一方案。
2. 推荐方案：3D LIO 为主、先验地图全局定位校正、UWB 兜底、雷达站有条件就上；轮式只作短时补充。
3. 哨兵、步兵、雷达站的算力、动态性与风险差异很大，不能共用一套参数；哨兵的核心是云台与底盘解耦。
4. 标定顺序从底盘往外扩：底盘 → 雷达 → 云台 → 相机内参 → 云台-相机 → UWB；上车前按 10 条检查表逐项确认。
5. 备赛分六步走（建图 → 清理 → 地图包 → 定位验证 → 导航联调 → 压力测试），每步都有量化验收标准。
6. 最容易踩的坑：动态残影、地图过期、UWB 多径、对称误匹配、算力不足、云台转动干扰、弹丸撞击。

延伸阅读

- 第 23 章的 Z-Localization 深度解析与官方仓库 [29]：先验地图定位在 RM 场景的完整工程实现，本章” 方案选型” 里” 先验地图 + 全局定位” 一栏就是它。
- 第 22 章的完整链路实战：建图到导航联调的具体操作，本章的验收标准可以在那里逐项执行。
- 第 17 章：FAST-LIO2 与 Point-LIO 的内部机制，理解它们才能判断” 退化场景该加什么约束”。

第三十章 学习路线与资源清单

本章导读

这本书很厚，没人能一周读完。更现实的问题是：一个刚进组的新队员，应该按什么顺序读、读到哪里算”够用”、什么时候该去碰真代码。本章给出四条时间尺度的路线（0–1 月、1–3 月、3–6 月、6–12 月），每条都配” 读哪几章、跑哪份代码、做成什么事、怎么算过关”；然后给出开源代码的阅读顺序与每份代码” 先看哪个文件”；最后是书单、工具链与实践建议。这一章不是知识，是路线图——它可以被反复翻，每次看都能确认自己走到了哪。

30.1 分阶段路线

30.1.1 第 0–1 月：把环境与概念立起来

目标不是” 懂 SLAM”，而是**能独立编译、跑通、录包、看数据**。

表 30.1: 第 0–1 月：环境与概念

项	内容	验收标准
读书	第 1、2、3 章	能画出定位/建图/导航的整体框图
概念	里程计、SLAM、TF、QoS、外参	能口头解释每个词解决什么问题
环境	Ubuntu + ROS 2 + 编译工具链	独立从零装好并跑通 demo
代码	跑通 FAST-LIO2 的示例 bag	能录自己的 bag 并回放
工具	`ros2 topic/tf2/bag`、RViz2	能查出” 数据没来” 是哪一环

这个月最容易走的弯路是” 一开始就啃公式”。先让数据在屏幕上动起来，再回头看公式，效率高得多。

30.1.2 第 1–3 月：里程计与 IMU

表 30.2: 第 1–3 月：里程计、IMU 与 LIO 入门

项	内容	验收标准
读书	第 4–7 章、第 16 章	能推导位姿递推与 ICP 残差
理论	四元数、ESKF、预积分、扫描匹配	能手推一维卡尔曼更新
代码	KISS-ICP、FAST-LIO2	能改一行代码并解释影响
实验	同一段 bag 上比较轮式/激光/IMU 轨迹	能画出三条轨迹与误差
标定	第 26 章的 IMU 与外参部分	独立完成一次 IMU 零偏与 Allan 分析

表 30.3: 第 3–6 月：SLAM 后端与导航链路

项	内容	验收标准
读书	第 11–17 章	能解释前端/后端/回环各管什么
读书	第 18–23 章	能画出 Nav2 的节点与数据流
代码	Cartographer、Point-LIO	读懂 `submap` 与逐点更新
实验	建图 → 保存 → 定位 → 导航全链路	让车在自己建的地图上走完一条路径
工程	第 25、27 章	能独立排掉一次”定位跳变”

表 30.4: 第 6–12 月：工程化、评测与改进

项	内容	验收标准
读书	第 24、28 章与本书全部	能判断一篇新论文值不值得跟进
评测	evo、ATE/RPE、消融实验	能设计并执行一次可复现对比
改进	在某个模块上做一处真实改进	公开数据集上指标提升且可复现
落地	第 29 章的清单	在实车上稳定运行一整场比赛
输出	写文档、做分享	能把一个模块讲给新队员听懂

30.1.3 第 3–6 月：SLAM 与导航

30.1.4 第 6–12 月：工程化与独立思考

30.2 代码阅读顺序

读代码的顺序应该**由简到繁**，每份代码只读它的”核心那几百行”。表 30.5 给出建议。

表 30.5: 开源代码阅读顺序与入口文件

顺序	仓库	先看什么
1	KISS-ICP [39]	`src/kiss_icp` 的配准与自适应阈值；这是最小可用的激光里程计，一天能读完
2	FAST-LIO2 [41]	`include/ikd-Tree` 与 `src/laserMapping.cpp`：先看”点如何进地图”，再看 IEKF 迭代
3	Point-LIO [18]	逐点更新的主循环与 IMU 饱和处理，理解”为什么它能扛高动态”
4	Cartographer [19]	`cartographer/mapping` 下的 `submap` 与 `constraint_builder`：分支定界回环
5	Nav2 [26]	从 `bt_navigator` 的行为树 XML 入口，依次看 planner 与 controller 插件接口
6	Z-Localization [29]	先验地图定位的完整工程：从地图加载、全局匹配到 `map→odom` 修正

读代码的三个技巧：

1. **先画数据流，再读函数**。每个仓库先写出”输入话题 → 主要处理函数 → 输出话题”，再挑中间最核心的一个函数精读。上来从 `main` 一行行读到底，必然迷路。
2. **带着问题读**。”它的地图怎么存？””重定位失败会怎样？””时间戳在哪里对齐？”带着三个具体问题去读，比通读效率高一个量级。
3. **改一行，跑一遍**。把降采样体素改大一倍，看轨迹怎么变；把迭代次数改成 1，看什么时候炸。动手是最快的理解方式。

30.3 书、课程与文档

- **概率机器人**：Thrun 等的 *Probabilistic Robotics* [38]。定位与 SLAM 的概率视角经典，重点读第 2、3、6、7、8 章，配合第 2 章。
- **状态估计**：Barfoot 的 *State Estimation for Robotics* [1]。李群、SE(3) 上的估计讲得最系统，是读懂 FAST-LIO 的钥匙。
- **ESKF 手册**：Sola 的 *Quaternion Kinematics for the Error-State Kalman Filter* [37]。免费且短，建议打印精读。
- **视觉 SLAM**：高翔等的《视觉 SLAM 十四讲》。若要做视觉/视觉惯性，这是中文最好的入门书。
- **课程**：深蓝学院的多传感器融合定位、激光 SLAM 课程体系完整，作业即代码；Coursera 的 Robotics 专项适合补基础。
- **文档**：ROS 2 官方文档、Nav2 文档、各仓库 README 与 issue 区。**issue 区是被低估的教材**——你能想到的问题，九成已经有人问过并得到了作者解释。

30.4 工具链清单

表 30.6: 定位与建图常用工具链

类别	工具	用在哪
系统	Ubuntu 22.04、ROS 2 Humble	全部工程
语言	C++17、Python 3	算法与脚本
数学	Eigen、Sophus	矩阵、李群
点云	PCL、Open3D	点云处理与可视化
优化	Ceres Solver、GTSAM	后端与标定
视觉	OpenCV、Kalibr	内参与相机-IMU 标定
可视化	RViz2、Foxglove、CloudCompare	调试与地图检查
评测	evo	ATE/RPE 与轨迹对比
录包	`ros2 bag`（mcap）	数据采集与复现

30.5 给新队员的 10 条建议

1. **先跑通，再改代码**。跑不通的东西，读再多论文也没用。
2. **先看数据，再看代码**。点云长什么样、时间戳差多少、TF 对不对，比读实现更快定位问题。
3. **记录每一次实验**。参数、指标、现象三列，坚持三个月你就比同龄人快一倍。
4. **学会看 TF**。定位问题里有一半最终能追溯到某条 TF 断链或方向反了。
5. **重视标定**。外参与时间戳是” 调不回来” 的误差，第 26 章请至少精读一遍。
6. **从失败案例学习**。录下” 炸掉” 的那段数据，比录十段成功数据更有价值。
7. **多读 issue 与提交历史**。作者的修复往往比文档更接近真相。
8. **一次只改一个变量**。无论调参还是做实验，这条纪律都成立。
9. **写文档**。能写清楚一个模块，说明你真的懂了；这也是团队里最稀缺的能力。
10. **不要怕数学，但要按需学**。遇到不懂的公式，先解决眼前问题，再回头补推导。

30.6 结语

写到这里，全书的主线其实只有一句话：**定位是” 用带噪声的观测去估计一个不确定的位姿，并承认它有多不确定”**。里程计篇讲如何递推，SLAM 篇讲如何用历史约束修正，导航篇讲如何用这个位姿去做决策，工程

篇讲如何让它在一台真实的车上稳定运行。四条线索在最后一章汇合——你会发现，前面觉得” 用不上” 的数学，最后都变成了排错时判断” 这到底是参数问题还是数据问题” 的依据。

技术会变：FAST-LIO 之后有 Point-LIO，Cartographer 之后有新的图优化框架，Nav2 也在持续演进。但” 观测—模型—优化—验证” 这套方法论不会变。希望这本书不只是让你会用某几个仓库，而是让你在面对一个从没见过的传感器或算法时，知道该问哪几个问题。

本章小结

1. 学习路线分四段：0–1 月环境与概念、1–3 月里程计与 IMU、3–6 月 SLAM 与导航、6–12 月工程化与改进，每段都有” 读什么、跑什么、做成什么、怎么过关”。
2. 代码阅读由简到繁：KISS-ICP → FAST-LIO2 → Point-LIO → Cartographer → Nav2 → Z-Localization；先画数据流，再精读核心函数。
3. 推荐的” 少而精” 资源：Probabilistic Robotics、State Estimation for Robotics、Sola 的 ESKF 文档、视觉 SLAM 十四讲，以及各仓库的 issue 区。
4. 工具链的核心是” 能复现”：统一环境、固定 bag、用 evo 量化，任何改动都必须能回到同一段数据上验证。
5. 十条建议的核心可以压缩成三条：先跑通再改、一次只改一个变量、写文档。
6. 全书主线：观测—模型—优化—验证；技术会变，方法论不会。

延伸阅读

- 第 1 章与书末附录 A、B：全书的入口与速查工具，建议在按本章路线学习时反复回看。
- 各开源仓库的 README 与 issue 区：KISS-ICP、FAST-LIO2、Point-LIO、Cartographer、Nav2、Z-Localization [29]。遇到具体报错时先搜 issue。
- 视觉 SLAM 十四讲配套代码：若要做视觉惯性方向，这是最好的动手起点。

参考文献

[1] Timothy D. Barfoot. *State Estimation for Robotics*. Cambridge University Press, 2017.

[2] Peter Biber and Wolfgang Strasser. “The Normal Distributions Transform: A New Approach to Laser Scan Matching”. In: *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. 2003, pp. 2743–2748. DOI: 10.1109/IROS.2003.1249285.

[3] Carlos Campos et al. “ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial and Multi-Map SLAM”. In: *IEEE Transactions on Robotics* 37.6 (2021), pp. 1874–1890. DOI: 10.1109/TRO.2021.3075644.

[4] Chao Cao et al. “Autonomous Exploration Development Environment and the Planning Algorithms”. In: *IEEE International Conference on Robotics and Automation (ICRA)*. 2022, pp. 4805–4811. DOI: 10.1109/ICRA46639.2022.9812330.

[5] Kenny Chen, Ryan Nemiroff, and Blake Lopez. “Direct LiDAR-Inertial Odometry: Lightweight LIO with Continuous-Time Motion Correction”. In: *IEEE International Conference on Robotics and Automation (ICRA)*. 2023, pp. 3983–3989. DOI: 10.1109/ICRA48891.2023.10160508.

[6] Brian Curless and Marc Levoy. “A Volumetric Method for Building Complex Models from Range Images”. In: *ACM SIGGRAPH*. 1996, pp. 303–312. DOI: 10.1145/237170.237269.

[7] Frank Dellaert and Michael Kaess. *Factor Graphs for Robot Perception*. Foundations and Trends in Robotics, 2017. DOI: 10.1561/2300000043.

[8] Dmitri Dolgov et al. “Practical Search Techniques in Path Planning for Autonomous Driving”. In: *AAAI Workshop on Intelligent Vehicle Systems*. 2010.

[9] Jakob Engel, Vladlen Koltun, and Daniel Cremers. “Direct Sparse Odometry”. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 40.3 (2018), pp. 611–625. DOI: 10.1109/TPAMI.2017.2658577.

[10] *Equivariant Symmetries for Inertial Navigation Systems*. 2023. arXiv: 2309.03765 [cs.RO].

[11] Christian Forster et al. “On-Manifold Preintegration for Real-Time Visual-Inertial Odometry”. In: *IEEE Transactions on Robotics* 33.1 (2017), pp. 1–21. DOI: 10.1109/TRO.2016.2597321.

[12] Dieter Fox. “Adapting the Sample Size in Particle Filters Through KLD-Sampling”. In: *International Journal of Robotics Research* 22.12 (2003), pp. 985–1003. DOI: 10.1177/0278364903022012001.

[13] Dieter Fox, Wolfram Burgard, and Sebastian Thrun. “The Dynamic Window Approach to Collision Avoidance”. In: *IEEE Robotics and Automation Magazine* 4.1 (1997), pp. 23–33. DOI: 10.1109/100.580977.

[14] *G-PROBE: Cross-FOV Place Recognition and Certainty-Coupled Localization for 3D Point Clouds*. 2026. arXiv: 2607.06782 [cs.RO].

[15] Giorgio Grisetti, Cyrill Stachniss, and Wolfram Burgard. “Improved Techniques for Grid Mapping with Rao-Blackwellized Particle Filters”. In: *IEEE Transactions on Robotics* 23.1 (2007), pp. 34–46. DOI: 10.1109/TRO.2006.889486.

[16] Daniel Harabor and Alban Grastien. “Online Graph Pruning for Pathfinding on Grid Maps”. In: *AAAI Conference on Artificial Intelligence*. 2011, pp. 1114–1119.

[17] Peter E. Hart, Nils J. Nilsson, and Bertram Raphael. “A Formal Basis for the Heuristic Determination of Minimum Cost Paths”. In: *IEEE Transactions on Systems Science and Cybernetics* 4.2 (1968), pp. 100–107. DOI: 10.1109/TSSC.1968.300136.

[18] Dongjiao He et al. “Point-LIO: Robust High-Bandwidth LiDAR-Inertial Odometry”. In: *Advanced Intelligent Systems* 5.6 (2023), p. 2200459. DOI: 10.1002/aisy.202200459.

[19] Wolfgang Hess et al. “Real-Time Loop Closure in 2D LIDAR SLAM”. In: *IEEE International Conference on Robotics and Automation (ICRA)*. 2016, pp. 1271–1278. DOI: 10.1109/ICRA.2016.7487258.

[20] Armin Hornung et al. “OctoMap: An Efficient Probabilistic 3D Mapping Framework Based on Octrees”. In: *Autonomous Robots* 34.3 (2013), pp. 189–206. DOI: 10.1007/s10514-012-9321-0.

[21] Rudolf E. Kalman. “A New Approach to Linear Filtering and Prediction Problems”. In: *Journal of Basic Engineering* 82.1 (1960), pp. 35–45. DOI: 10.1115/1.3662552.

[22] Sertac Karaman and Emilio Frazzoli. “Sampling-based Algorithms for Optimal Motion Planning”. In: *International Journal of Robotics Research* 30.7 (2011), pp. 846–894. DOI: 10.1177/0278364911406761.

[23] Giseop Kim and Ayoung Kim. “Scan Context: Egocentric Spatial Descriptor for Place Recognition within 3D Point Cloud Map”. In: *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. 2018, pp. 4802–4809. DOI: 10.1109/IROS.2018.8593953.

[24] Rainer Kümmerle et al. “g2o: A General Framework for Graph Optimization”. In: *IEEE International Conference on Robotics and Automation (ICRA)*. 2011, pp. 3607–3613. DOI: 10.1109/ICRA.2011.5979949.

[25] Jiarong Lin and Fu Zhang. “R3LIVE: A Robust, Real-time, RGB-colored, LiDAR-Inertial-Visual Tightly-coupled State Estimation and Mapping Package”. In: *IEEE International Conference on Robotics and Automation (ICRA)*. 2022, pp. 10672–10678. DOI: 10.1109/ICRA46639.2022.9811607.

[26] Steve Macenski et al. “The Marathon 2: A Navigation System”. In: *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. 2020, pp. 2718–2725. DOI: 10.1109/IROS45743.2020.9340862.

[27] Steven Macenski et al. “Robot Operating System 2: Design, Architecture, and Uses in the Wild”. In: *Science Robotics* 7.66 (2022), eabm6074. DOI: 10.1126/scirobotics.abm6074.

[28] Anastasios I. Mourikis and Stergios I. Roumeliotis. “A Multi-State Constraint Kalman Filter for Vision-aided Inertial Navigation”. In: *IEEE International Conference on Robotics and Automation (ICRA)*. 2007, pp. 3565–3572. DOI: 10.1109/ROBOT.2007.363958.

[29] mozihe. *Z-Localization: 2026 东北大学 T-DT RoboMaster 哨兵定位开源*. GitHub repository. 2026. URL: https://github.com/mozihe/z-localization.

[30] Helen Oleynikova et al. “Voxblox: Incremental 3D ESDFs for Onboard Planning”. In: *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. 2017, pp. 6024–6030. DOI: 10.1109/IROS.2017.8206504.

[31] Tong Qin, Peiliang Li, and Shaojie Shen. “VINS-Mono: A Robust and Versatile Monocular Visual-Inertial State Estimator”. In: *IEEE Transactions on Robotics* 34.4 (2018), pp. 1143–1155. DOI: 10.1109/TRO.2018.2853729.

[32] James B. Rawlings, David Q. Mayne, and Moritz M. Diehl. *Model Predictive Control: Theory, Computation, and Design*. 2nd. Nob Hill Publishing, 2017.

[33] Christoph Rösmann et al. “Trajectory Modification Considering Dynamic Constraints of Autonomous Robots”. In: *German Conference on Robotics (ROBOTIK)*. 2012, pp. 74–79.

[34] Tixiao Shan and Brendan Englot. “LeGO-LOAM: Lightweight and Ground-Optimized Lidar Odometry and Mapping on Variable Terrain”. In: *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. 2018, pp. 4758–4765. DOI: 10.1109/IROS.2018.8594299.

[35] Tixiao Shan et al. “LIO-SAM: Tightly-coupled Lidar Inertial Odometry via Smoothing and Mapping”. In: *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. 2020, pp. 5135–5142. DOI: 10.1109/IROS45743.2020.9341176.

[36] Tixiao Shan et al. “LVI-SAM: Tightly-coupled Lidar-Visual-Inertial Odometry via Smoothing and Mapping”. In: *IEEE International Conference on Robotics and Automation (ICRA)*. 2021, pp. 5693–5699. DOI: 10.1109/ICRA48506.2021.9561996.

[37] Joan Solà. *Quaternion Kinematics for the Error-State Kalman Filter*. 2017. arXiv: 1711.02508 [cs.CV].

[38] Sebastian Thrun, Wolfram Burgard, and Dieter Fox. *Probabilistic Robotics*. MIT Press, 2005.

[39] Ignacio Vizzo et al. “KISS-ICP: In Defense of Point-to-Point ICP – Simple, Accurate, and Robust Registration If Done the Right Way”. In: *IEEE Robotics and Automation Letters* 8.2 (2023), pp. 1029–1036. DOI: 10.1109/LRA.2023.3236571.

[40] Wei Xu and Fu Zhang. “FAST-LIO: A Fast, Robust LiDAR-Inertial Odometry Package by Tightly-Coupled Iterated Kalman Filter”. In: *IEEE Robotics and Automation Letters* 6.2 (2021), pp. 3317–3324. DOI: 10.1109/LRA.2021.3064227.

[41] Wei Xu et al. “FAST-LIO2: Fast Direct LiDAR-Inertial Odometry”. In: *IEEE Transactions on Robotics* 38.4 (2022), pp. 2053–2071. DOI: 10.1109/TRO.2021.3128032.

[42] Fan Yang et al. “FAR Planner: Fast, Attemptable Route Planner for Navigation in Known and Unknown Environments”. In: *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)* (2022), pp. 5293–5299. DOI: 10.1109/IROS47612.2022.9981164.

[43] Chongjian Yuan et al. “Efficient and Probabilistic Adaptive Voxel Mapping for Accurate Online LiDAR Odometry”. In: *IEEE Robotics and Automation Letters* 7.3 (2022), pp. 8518–8525. DOI: 10.1109/LRA.2022.3187268.

[44] Chongjian Yuan et al. “FAST-LIVO: Fast and Sparse LiDAR-Inertial-Visual Odometry”. In: *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. 2022, pp. 1633–1639. DOI: 10.1109/IROS47612.2022.9982049.

[45] Chongjian Yuan et al. “FAST-LIVO2: Fast, Direct LiDAR-Inertial-Visual Odometry”. In: *IEEE Transactions on Robotics* 40 (2024), pp. 1–18. DOI: 10.1109/TRO.2024.3422052.

[46] Ji Zhang and Sanjiv Singh. “LOAM: Lidar Odometry and Mapping in Real-time”. In: *Robotics: Science and Systems (RSS)*. 2014. DOI: 10.15607/RSS.2014.X.007.

[47] Boyu Zhou et al. “Robust and Efficient Quadrotor Trajectory Generation for Fast Autonomous Flight”. In: *IEEE Robotics and Automation Letters* 4.4 (2019), pp. 3529–3536. DOI: 10.1109/LRA.2019.2927938.

[48] Xin Zhou et al. “EGO-Planner: An ESDF-Free Gradient-Based Local Planner for Quadrotors”. In: *IEEE Robotics and Automation Letters* 6.2 (2021), pp. 478–485. DOI: 10.1109/LRA.2020.3043694.

附录 A 公式与符号速查

A.1 全局符号表

表 A.1: 全书通用符号

符号	名称	含义
W	世界系 / map 系	全局参考坐标系，z 轴向上
O	odom 系	连续、无跳变的局部参考系
B	机体系	默认与 IMU 系重合
L、C	雷达系、相机系	传感器坐标系
$T_{AB} = (R_{AB}, {}^{A}p_{B})$	位姿	从 B 系到 A 系的变换
R、q	旋转矩阵、四元数	$R \in \mathrm{SO}(3)$
p、v、a	位置、速度、加速度	列向量
ω、g	角速度、重力	$g = [0, 0, -9.81]^{\top}$
$\tilde{\omega}$、$\tilde{a}$	IMU 测量值	含零偏与噪声
b_g、b_a	陀螺仪/加速度计零偏	缓慢漂移
x、δx	状态、误差状态	
z、$h(\cdot)$、r	测量、观测函数、残差	
P、Q、R	状态/过程/测量协方差	
J、H	雅可比、海森	
Λ	信息矩阵	$\Lambda = P^{-1}$ 或 $J^{\top}R^{-1}J$
$\pi = (n_{\pi}, d)$	平面参数	$n_{\pi}^{\top}x = d$

A.2 旋转与刚体变换

A.0.0.0.1 旋转矩阵的性质

$$R^{\top}R = I, \quad \det(R) = 1, \quad (R_1R_2)^{-1} = R_2^{\top}R_1^{\top}. \tag{A.1}$$

A.0.0.0.2 Rodrigues 公式

设 $\phi = \theta a$（$\|a\| = 1$），则

$$\exp(\phi^{\wedge}) = I + \sin\theta\, a^{\wedge} + (1 - \cos\theta)(a^{\wedge})^2. \tag{A.2}$$

A.0.0.0.3 对数映射

$$\theta = \arccos\frac{\mathrm{tr}(R) - 1}{2}, \qquad \phi = \frac{\theta}{2\sin\theta}(R - R^{\top})^{\vee}. \tag{A.3}$$

A.0.0.0.4 SE(3) 的逆

$$T^{-1} = \begin{bmatrix} R^{\top} & -R^{\top}t \\ 0^{\top} & 1 \end{bmatrix}. \tag{A.4}$$

A.0.0.0.5 位姿复合与逆

$$T_{AC} = T_{AB}T_{BC}, \qquad T_{AB}^{-1} = T_{BA}. \tag{A.5}$$

A.0.0.0.6 右扰动

$$R \leftarrow R\exp(\delta\phi^{\wedge}), \qquad t \leftarrow t + R\,\delta\rho. \tag{A.6}$$

A.3 概率与滤波

A.0.0.0.7 贝叶斯公式

$$p(x \mid z) = \frac{p(z \mid x)p(x)}{p(z)}. \tag{A.7}$$

A.0.0.0.8 高斯分布

$$p(x) = \frac{1}{(2\pi)^{n/2}\,|\Sigma|^{1/2}} \exp\left(-\frac{1}{2}(x-\mu)^{\top}\Sigma^{-1}(x-\mu)\right). \tag{A.8}$$

A.0.0.0.9 方差与协方差

$$\mathrm{Var}(X) = \mathbb{E}[X^2] - (\mathbb{E}[X])^2, \qquad \mathrm{Cov}(X,Y) = \mathbb{E}[XY] - \mathbb{E}[X]\mathbb{E}[Y]. \tag{A.9}$$

A.0.0.0.10 卡尔曼滤波 预测：

$$\hat{x}_k^- = F_k \hat{x}_{k-1} + B_k u_k, \tag{A.10}$$

$$P_k^- = F_k P_{k-1} F_k^{\top} + Q_k. \tag{A.11}$$

更新：

$$K_k = P_k^- H_k^{\top}\left(H_k P_k^- H_k^{\top} + R_k\right)^{-1}, \tag{A.12}$$

$$\hat{x}_k = \hat{x}_k^- + K_k\left(z_k - h(\hat{x}_k^-)\right), \tag{A.13}$$

$$P_k = (I - K_k H_k) P_k^-. \tag{A.14}$$

A.0.0.0.11 ESKF 误差状态传播与更新：

$$\delta\dot{x} = F\delta x + Gn, \qquad K = P^- H^{\top}(HP^-H^{\top} + R)^{-1}, \qquad \delta x \leftarrow \delta x \oplus Kr. \tag{A.15}$$

A.0.0.0.12 马氏距离与卡方检验

$$d^2 = r^{\top}(HPH^{\top} + R)^{-1} r \sim \chi^2(k). \tag{A.16}$$

A.4 优化

A.0.0.0.13 最小二乘正规方程

$$A^{\top}Ax = A^{\top}b. \tag{A.17}$$

A.0.0.0.14 高斯–牛顿

$$J^{\top}J\,\delta x = -J^{\top}r, \qquad x \leftarrow x + \delta x. \tag{A.18}$$

A.0.0.0.15 Levenberg–Marquardt

$$\left(J^{\top}J + \lambda I\right)\delta x = -J^{\top}r. \tag{A.19}$$

A.0.0.0.16 Huber 鲁棒核

$$\rho(s) = \begin{cases} s, & s \le \delta^2, \\ 2\delta\sqrt{s} - \delta^2, & s > \delta^2, \end{cases} \qquad w(s) = \frac{\rho'(s)}{2\sqrt{s}} = \begin{cases} 1, & s \le \delta^2, \\ \delta/\sqrt{s}, & s > \delta^2. \end{cases} \tag{A.20}$$

A.5 点云配准

A.0.0.0.17　ICP 闭式旋转解　给定去质心点对 (p_i, q_i)，构造

$$W = \sum_i p_i q_i^\top = U\Sigma V^\top, \qquad R^* = VU^\top, \tag{A.21}$$

若 $\det(R^*) < 0$，取 $V' = [v_1, v_2, -v_3]$ 后重算。平移为 $t^* = \bar{q} - R^*\bar{p}$。

A.0.0.0.18　点到面残差

$$r_i = n_i^\top \left(Rp_i + t - q_i\right). \tag{A.22}$$

A.6 SLAM 后端

A.0.0.0.19　位姿图残差

$$r_{ij} = \log\left(T_{ij}^{-1} T_i^{-1} T_j\right)^\vee \in \mathbb{R}^6. \tag{A.23}$$

A.0.0.0.20　舒尔补　分块系统

$$\begin{bmatrix} H_{cc} & H_{cp} \\ H_{pc} & H_{pp} \end{bmatrix} \begin{bmatrix} \delta x_c \\ \delta x_p \end{bmatrix} = \begin{bmatrix} b_c \\ b_p \end{bmatrix} \tag{A.24}$$

消去相机变量后：

$$\left(H_{pp} - H_{pc} H_{cc}^{-1} H_{cp}\right) \delta x_p = b_p - H_{pc} H_{cc}^{-1} b_c. \tag{A.25}$$

A.0.0.0.21　边缘化　用舒尔补把旧状态消去，得到新的先验因子；注意这会引入线性化误差。

A.7 导航

A.0.0.0.22　A* 评价函数

$$f(n) = g(n) + h(n), \tag{A.26}$$

其中 g 是起点到 n 的实际代价，h 是到目标的启发式下界（可采纳）。

A.0.0.0.23　DWA 评价函数

$$G(v, \omega) = \alpha \cdot \text{heading}(v, \omega) + \beta \cdot \text{clearance}(v, \omega) + \gamma \cdot \text{velocity}(v, \omega). \tag{A.27}$$

A.0.0.0.24　Cartographer 扫描匹配得分

$$S(T) = \sum_k M_{\text{smooth}}\left(T\, h_k\right), \tag{A.28}$$

其中 M_{smooth} 是双线性插值后的概率栅格。

A.0.0.0.25　控制代价（MPC）

$$\min_{u_{0:N-1}} \sum_{k=0}^{N-1} \left(x_k - x_k^{\text{ref}}\right)^\top Q \left(x_k - x_k^{\text{ref}}\right) + u_k^\top R u_k \quad \text{s.t. } x_{k+1} = f(x_k, u_k). \tag{A.29}$$

附录 B 术语中英对照表

本表按主题分组，给出本书出现的主要术语中英文对照及一句话解释，便于阅读英文论文与源码时查阅。

B.1 数学与估计

表 B.1: 数学与估计基础术语

中文	英文	说明
位姿	pose	位置 + 姿态，SE(3) 元素
姿态	orientation	旋转部分，SO(3) 元素
刚体变换	rigid transformation	旋转 + 平移
齐次坐标	homogeneous coordinates	用 4 维向量表示 3 维点，便于矩阵复合
旋转矩阵	rotation matrix	满足 $\boldsymbol{R}^{\top}\boldsymbol{R}=\boldsymbol{I}$、$\det \boldsymbol{R}=1$
四元数	quaternion	用 4 个数表示旋转，无奇异
欧拉角	Euler angle	roll/pitch/yaw，直观但有万向锁
万向锁	gimbal lock	pitch 为 ±90° 时自由度退化
李群	Lie group	既是群又是流形，如 SO(3)、SE(3)
李代数	Lie algebra	李群单位元处的切空间，可做加法
指数映射	exponential map	李代数 $\to$ 李群
对数映射	logarithm map	李群 $\to$ 李代数
雅可比矩阵	Jacobian	一阶偏导矩阵
海森矩阵	Hessian	二阶偏导矩阵
泰勒展开	Taylor expansion	用多项式局部逼近函数
最小二乘	least squares	最小化残差平方和
正规方程	normal equation	$\boldsymbol{A}^{\top}\boldsymbol{A}\boldsymbol{x}=\boldsymbol{A}^{\top}\boldsymbol{b}$
高斯–牛顿	Gauss–Newton	非线性最小二乘的迭代解法
LM	Levenberg–Marquardt	带阻尼的高斯–牛顿
期望	expectation	概率加权平均
方差	variance	偏离期望的平方的期望
协方差	covariance	两个变量共同变化的程度
协方差矩阵	covariance matrix	所有两两协方差组成的矩阵
高斯分布	Gaussian distribution	正态分布
贝叶斯公式	Bayes' formula	先验 + 似然 $\to$ 后验
后验	posterior	观测后的状态分布
似然	likelihood	给定状态下观测的概率
卡方分布	chi-square distribution	标准正态平方和；用于一致性检验
马氏距离	Mahalanobis distance	按协方差加权的距离
信息矩阵	information matrix	协方差矩阵的逆，衡量约束强度
特征值	eigenvalue	$\boldsymbol{A}\boldsymbol{v}=\lambda\boldsymbol{v}$ 中的 λ
奇异值分解	SVD	$\boldsymbol{A}=\boldsymbol{U}\boldsymbol{\Sigma}\boldsymbol{V}^{\top}$

B.2 定位、里程计与 SLAM

B.3 导航、工程与其他

表 B.2: 定位与 SLAM 术语

中文	英文	说明
定位	localization	估计机器人在参考系中的位姿
绝对定位	absolute localization	直接在地图系中定位
相对定位	relative localization	相对起点递推，会累积误差
全局定位	global localization	初始位姿未知时的定位
重定位	relocalization	跟丢后重新找回位姿
绑架问题	kidnapped robot problem	机器人被瞬间移动后的定位问题
里程计	odometry	用传感器增量递推位姿
航迹推算	dead reckoning	基于速度/转向积分推位姿
漂移	drift	累积误差导致的缓慢偏离
前端	front-end	数据关联 + 局部位姿估计
后端	back-end	全局优化、回环、地图修正
数据关联	data association	确定观测之间的对应关系
扫描匹配	scan matching	用两帧点云估计相对位姿
点云配准	point cloud registration	同上，更通用的说法
迭代最近点	ICP	交替找最近点与求最优变换
正态分布变换	NDT	用体素内高斯分布建模点云
关键帧	keyframe	被选入地图的代表帧
局部地图	local map	当前帧附近的地图子集
先验地图	prior map	离线建好、定位时加载的地图
回环检测	loop closure detection	判断是否回到历史地点
位姿图	pose graph	节点 = 位姿、边 = 相对约束的图
因子图	factor graph	用因子表示观测的图模型
光束法平差	bundle adjustment	同时优化位姿与路标的非线性最小二乘
舒尔补	Schur complement	分块消元，BA 的核心技巧
边缘化	marginalization	消去旧状态但保留其信息
鲁棒核	robust kernel	降低大残差影响的损失函数
后端优化	back-end optimization	全局最小二乘求解
建图	mapping	把观测融合成环境表示
占据栅格	occupancy grid	每格记录占据概率的 2D 地图
八叉树	octree	空间递归划分的 3D 地图
点云	point cloud	3D 点的集合
体素	voxel	三维栅格单元
降采样	downsampling	减少点数，常用体素质心法
去畸变	deskewing	补偿单帧点云内的运动畸变
运动畸变	motion distortion	扫描期间运动导致的点云变形
可观测性	observability	某方向能否由观测唯一确定
退化	degeneracy	观测对某方向没有约束
紧耦合	tightly coupled	多传感器放进同一估计器
松耦合	loosely coupled	各传感器独立估计后融合
预积分	preintegration	IMU 在两帧间的积分量与初值解耦
零偏	bias	传感器零输入时的非零输出
随机游走	random walk	零偏等量随时间的随机漂移
惯性测量单元	IMU	陀螺仪 + 加速度计
激光惯性里程计	LIO	LiDAR + IMU 里程计
视觉惯性里程计	VIO	相机 + IMU 里程计

表 B.3: 导航与工程术语

中文	英文	说明
导航	navigation	定位 + 地图 + 规划 + 控制
全局规划	global planning	在全局地图上找可行路径
局部规划	local planning	跟踪全局路径并实时避障
代价地图	costmap	栅格化的代价表示，供规划查询
膨胀层	inflation layer	按距离给障碍物周围加代价
行为树	behavior tree	Nav2 的任务编排机制
生命周期	lifecycle	ROS 2 节点的状态管理
动态窗口法	DWA	在速度空间采样避障
时间弹性带	TEB	把轨迹表示为带时间戳的图优化
模型预测控制	MPC	滚动时域优化控制
人工势场	artificial potential field	引力 + 斥力避障
可视图	visibility graph	连接可见节点的图，用于全局规划
自主探索	autonomous exploration	在未知环境中边建图边找目标
地形分析	terrain analysis	从点云提取坡度、粗糙度、台阶
可通行性	traversability	某区域能否安全通过
探索增益	exploration gain	探索某区域能获得的新信息量
外参	extrinsic	传感器之间的位姿变换
内参	intrinsic	相机内部成像参数
标定	calibration	估计内参/外参/时间偏移
时间同步	time synchronization	统一多传感器时间基准
时间戳	timestamp	数据采集时刻
点频	point rate	激光雷达每秒输出点数
线数	number of channels	激光雷达垂直方向束数
角分辨率	angular resolution	相邻光束的角度间隔
盲区	blind zone	雷达无法测距的最近距离
视场角	field of view (FOV)	传感器可观测的角度范围
飞行时间	time of flight (ToF)	通过光/声飞行时间测距
超宽带	UWB	室内无线定位技术
全球导航卫星系统	GNSS	GPS/北斗等的统称
实时动态差分	RTK	GNSS 厘米级定位技术
惯性测量单元	IMU	陀螺仪 + 加速度计
机器人操作系统	ROS	机器人软件框架
数据分发服务	DDS	ROS 2 的通信中间件
服务质量	QoS	ROS 2 通信的可靠性等配置
坐标变换	TF	ROS 中的坐标系变换树
话题	topic	发布/订阅通信
服务	service	请求/响应通信
动作	action	带反馈的长任务通信
数据包	rosbag	传感器数据的录制文件
绝对轨迹误差	ATE	估计轨迹与真值的整体偏差
相对位姿误差	RPE	固定间隔内的相对误差